

Programación Orientada a Objetos

Libro Guía

Eric Ross Cortés

Versión 0.3b

2023

Copyright © 2023 Eric Ross C.

This book and its on-line version are distributed under the terms of the Creative Commons Attribution-NoDerivs 1.0 license. A reference copy of this license may be found at <http://creativecommons.org/licenses/by-nd/1.0/legalcode>.

About this, ChatGPT (GPT-3.5) says:

If you publish your book under the Creative Commons Attribution-NoDerivs 1.0 license, it means that others are allowed to download and share your work, but they must do so with proper attribution to you, the original author, and they cannot make any derivative works or modifications to your book. However, they are not allowed to sell it for profit.

The "NoDerivs" aspect of the license explicitly prohibits others from creating derivative works, and selling your work without permission would typically constitute a derivative work. So, anyone who downloads your book under this license can share it, but they cannot sell it without violating the terms of the license.

It's important to note that while the license allows for free distribution, it doesn't mean someone can profit from your work without your consent. They can only redistribute it for non-commercial purposes and must give you proper credit as the author. If someone wants to use your work for commercial purposes, they would typically need to seek your permission or a separate license from you.

ISBN: xxx-x-xxxxxx-x

Primera impresión, xxxx.

Portada: Desierto, Región de Antofagasta, Año 2015

Usar una imagen de un desierto como portada para un libro acerca de Programación Orientada a Objetos puede ser una elección interesante y simbólica. Un desierto puede representar la simplicidad y la claridad. La POO busca simplificar la programación al dividir el código en objetos y abstracciones, lo que a menudo conduce a un código más limpio y claro. Un desierto también suele ser un paisaje amplio y abierto, lo que puede reflejar algunos principios que se buscan en la POO. Además, algunos conceptos en POO, como clases y objetos, son conceptos que en la práctica son fundamentales y básicos, y un desierto, como una imagen simple y básica de la naturaleza, puede simbolizar estos conceptos esenciales.

<https://www.tinkerlabs.cl/eross/>

Libro Guía
Programación Orientada al Objeto
Versión 0.3b

ERIC ROSS CORTÉS

23 de octubre de 2023



If I press these keys in the right order,
I can do anything!

Índice general

0.1.	¿Qué necesitas para enfrentar este libro?	4
0.1.1.	El modelo Dreyfus	4
0.1.2.	El efecto Dunning-Kruger	5
I	Libro Guía	7
1.	Nivelación	9
1.1.	¿Qué es Java?	9
1.2.	Ejecutando Java	10
1.3.	Compilando programas	10
1.4.	El proceso de desarrollo	11
1.5.	Documentando los programas	11
1.6.	Datos y sus tipos	13
1.7.	Estructura de los programas en Java	17
1.8.	Inicio del programa	18
1.9.	Convenciones de escritura	19
1.10.	Operadores	20
1.11.	Lectura desde el teclado	21
1.12.	Diferencias en el if	23
1.13.	Incrementando variables	25
1.14.	Ciclos	27
1.14.1.	for	27
1.14.2.	while	29
1.14.3.	do	31
1.15.	Arreglos	32
1.15.1.	Arreglos unidimensionales	32
1.15.2.	Arreglos bidimensionales	34
1.15.3.	Arreglos tridimensionales	35
1.15.4.	Mostrando los valores de un arreglo	35
1.15.5.	Conociendo el tamaño de un arreglo	37
1.15.6.	Errores al manipular arreglos	38
1.15.7.	Algoritmos de ordenamiento en arreglos	39
1.15.8.	Eliminación de un elemento en un arreglo	41
1.15.9.	Inserción ordenada en un arreglo	43
1.16.	Funciones	44
1.16.1.	Estructura de los programas	47
1.16.2.	Traspaso de parámetros	48

1.16.3. Escribiendo funciones	53
1.17. Documentando los programas II	54
1.18. Archivos	56
1.18.1. Formato de las líneas	59
1.19. Ejercicios	60
2. Introducción a la Ingeniería de Software	67
2.1. Introducción	67
2.2. Ingeniería de Software	68
2.3. Aseguramiento de la calidad	71
2.4. Depuración	72
2.5. Validación de datos	73
2.6. Excepciones	74
2.6.1. Uso de <code>IllegalArgumentException</code>	79
2.7. Ejercicios	81
3. Introducción a la POO	83
3.1. Clases	85
3.2. El modelo del dominio	85
3.2.1. Atributos del modelo	86
3.2.2. Multiplicidad	87
3.2.3. Relacionamiento entre conceptos	89
3.2.4. Recomendaciones	94
3.3. Clases II	95
3.3.1. Mensajes a las instancias	97
3.3.2. Mensajes a las clases	98
3.3.3. Métodos	99
3.3.4. Implementación en Java	99
3.3.5. El misterio de la ejecución de los programas en Java	103
3.3.6. Cómo se construyen las instancias	105
3.3.7. El constructor	107
3.3.8. <code>setters</code> y <code>getters</code>	108
3.3.9. Probando la clase <code>Persona</code>	109
3.3.10. <code>this</code>	110
3.3.11. Probando la clase <code>Persona</code> (II)	111
3.3.12. El valor <code>null</code>	112
3.3.13. Probando la clase <code>Persona</code> (III)	113
3.3.14. Alcance de las variables	117
3.4. Ejercicios	122
4. Interludio	129
4.1. El misterio de las referencias en Java	129
4.2. Comparando objetos	134
4.3. Inicialización de atributos de instancia	138
4.4. Ejercicios	140
5. Colecciones	141
5.1. Implementación	144
5.2. Genéricos	148

5.3.	Generalización de Colecciones	151
5.3.1.	Iteradores	151
5.4.	Ejercicios	155
6.	Un ejemplo completo	157
6.1.	Problema 1	157
6.1.1.	Descripción del problema	157
6.1.2.	Análisis del problema	158
6.1.3.	Clases	162
6.1.4.	Implementando los requerimientos	165
6.1.5.	Diagrama de clases	174
7.	POO 2	175
7.1.	Otra técnica para diseñar programas	175
7.2.	Sobre-escritura de métodos	178
7.3.	La visibilidad de lo heredado	180
7.4.	Lo privado NO se hereda	182
7.5.	Clases que no se pueden instanciar	184
7.6.	El principio de sustitución de Liskov	187
7.7.	Métodos Virtuales	189
7.8.	Polimorfismo	191
7.9.	Clases abstractas	192
7.10.	No todo es abstracto	194
7.11.	Herencia	201
7.12.	Interfaces	202
7.12.1.	Un primer ejemplo	202
7.13.	Sobrecarga de métodos	206
7.14.	Casting con objetos	210
7.15.	toString	213
7.16.	final	216
7.16.1.	Variables, parámetros y atributos	216
7.16.2.	Métodos	217
7.16.3.	Clases	217
7.17.	Ejercicios	218
8.	SOLID	219
8.1.	Single Responsibility	220
8.2.	Open/Closed	221
8.3.	Liskov Substitution	224
8.4.	Interface Segregation	226
8.5.	Dependency Inversion	230
8.6.	Ejercicios	232
9.	Arquitecturando las aplicaciones	233
9.1.	El Sistema	234
9.2.	Contratos	234
9.3.	Documentando Contratos	235
9.4.	Contratos del sistema	238
9.5.	Cuando las precondiciones no se cumplen	238

9.6.	Todas las operaciones tienen contrato	241
9.7.	Resumen de reglas	243
10.	Patrones de diseño	245
10.1.	Clasificación	245
10.2.	Patrones Creacionales	245
10.2.1.	Singleton	246
10.2.2.	Factory	247
10.2.3.	Abstract Factory	249
10.2.4.	Builder	253
10.2.5.	Prototype	258
10.3.	Patrones Estructurales	260
10.3.1.	Adapter	260
10.3.2.	Composite	263
10.3.3.	Proxy	267
10.3.4.	Flyweight	271
10.3.5.	Facade	274
10.3.6.	Bridge	276
10.3.7.	Decorator	277
10.4.	Patrones de Comportamiento	280
10.4.1.	Template	280
10.4.2.	Mediator	280
10.4.3.	Chain of Responsibility	280
10.4.4.	Observer	285
10.4.5.	Strategy	290
10.4.6.	Command	293
10.4.7.	State	299
10.4.8.	Visitor	304
10.4.9.	Interpreter	307
10.4.10.	Iterator	307
10.4.11.	Memento	308
11.	Otros ejemplos completos	313
11.1.	Problema 1	313
11.1.1.	Iniciando el análisis	314
11.1.2.	Diseñando la solución	316
11.1.3.	Implementando la solución	319
11.1.4.	Usando el Sistema	332
11.1.5.	Diagrama de clases actualizado	336
11.2.	Problema 2	337
11.2.1.	Trabajo a realizar	339
11.2.2.	Iniciando el análisis	339
11.2.3.	Diseñando la solución	340
11.2.4.	Implementando la solución	344
11.2.5.	Usando el Sistema	351
11.2.6.	Clases del dominio	352
11.2.7.	Diagrama de clases actualizado	361
11.3.	Mascotas que cambian de dueño	362
11.3.1.	Trabajo a realizar	362

11.3.2.	Iniciando el análisis	363
11.3.3.	Diseñando la solución	363
11.3.4.	Implementando la solución	364
11.3.5.	Diagrama de clases actualizado	374
11.4.	Mini evaluador	375
11.4.1.	Iniciando el análisis	377
11.4.2.	Diseñando la solución	377
11.4.3.	Implementando la solución	379
11.4.4.	Diagrama de clases actualizado	385
11.5.	Vehículos de diferentes tipos	386
11.5.1.	Analizando el problema	386
11.5.2.	Diseñando la solución	387
11.5.3.	Implementando la solución	388
11.5.4.	Implementando el Sistema	392
11.5.5.	Diagrama de clases final	399
12.	El Final	401
12.1.	Lo aprendido	401
12.2.	Invitación a seguir aprendiendo	402
12.3.	Finalmente...	402
II	Ejercicios Propuestos	405
13.	Ejercicios Propuestos	407
13.1.	Mantenciones UCN	407
13.2.	La repisa	410
13.3.	Doña Filomena	413
13.4.	Aerolíneas PR	415
III	Apéndices	419
A.	Instalación del JDK	421
A.1.	Cómo instalar el JDK de Java en Windows	421
A.1.1.	Requisitos previos	421
A.1.2.	Descargando el instalador del JDK	421
A.1.3.	Instalando el JDK	422
A.1.4.	Configuración de la variable de entorno <code>JAVA_HOME</code>	422
A.1.5.	Verificando la instalación de Java	422
A.1.6.	Configurando la variable de entorno <code>PATH</code>	423
A.2.	Cómo instalar el JDK de Java en Ubuntu Linux	423
A.2.1.	Actualizar el sistema	424
A.2.2.	Instalar el JDK de Java	424
A.2.3.	Verificar la instalación	424
B.	El recolector	425
C.	Protecciones	429

D. javadoc	431
D.1. Formato General	431
D.2. Recomendaciones	433
D.3. Tags	434
D.4. Algunos ejemplos	434
E. Uso de interfaces en Swing	437
F. Utilidades	445
F.1. ¿Qué es Integer?	445
F.2. Expresiones Lambda	447
F.3. forEach() en la interfaz Iterable	448
F.4. Interfaces funcionales	451
F.5. Tipos de excepciones en Java	453
F.5.1. Checked Exceptions	453
F.5.2. Unchecked Exceptions	454
F.5.3. Cuándo usar una o la otra	455
G. Refactorización	457
G.1. ¿Qué es el refactoring?	457
G.2. ¿Por qué es importante el refactoring?	457
G.3. Ejemplos de refactoring en Java	458
H. Clases Avanzadas	461
H.1. StringBuilder	461
H.2. Hashtable	462
H.2.1. Ejemplos	463
H.2.2. ¿Por qué usar una Hashtable?	464
H.3. Ordenando	465
H.3.1. Interfaz Comparable	465
H.3.2. Usando un Comparator y Collections	465
H.3.3. Usando un Comparator y .sort()	468
H.3.4. Ordenando otros tipos de objetos	469

Prefacio

El uso adecuado de un prefacio en cualquier obra literaria es como un ensayo introductorio. El ensayo ha sido escrito por el autor y por lo general habla de la inspiración y la creación de la obra y la forma en que el autor desarrolla la idea central o la historia de la obra. En los libros de no-ficción el prefacio también tiende a establecer el propósito y el alcance del trabajo. Un prefacio normalmente contiene una lista de agradecimientos del autor a las personas que ayudaron, en diversos grados, en la realización de la obra.

Corresponde entonces hablar acerca del por qué existe este pequeño libro que estás leyendo. Se podría decir que se inició hace muchos años, desde que se tiene la noción de lo necesario que es enseñar y aprender programación orientada al objeto.

El problema con el enunciado anterior, es que hay muchísimos libros que tratan de introducir el mundo de la orientación al objeto a los lectores, pero como cada persona es diferente, cada autor lo hace a su modo. Algunos lo toman como un ejercicio académico y estructuran sus escritos de acuerdo a algún programa de algún curso o asignatura, y otros simplemente hacen su relato de acuerdo a algún orden que ellos y ellas consideren apropiado.

Este libro, la verdad, sigue este último enfoque: a partir de la experiencia tanto al enseñar como al usar objetos para diseñar y crear software, se ha ido destilando cierto orden “deseado” en que los tópicos se van introduciendo, y la verdad es una tarea titánica encontrar un orden que sea más o menos correcto, y sobretodo, que sea lo más lógico posible y que permita seguir introduciendo temas progresivamente, sin tener que realizar “saltos de fe”.¹

Lo que se quiere lograr con este libro es introducir al lector en todas las temáticas necesarias para tener un entendimiento claro de lo que es la orientación al objeto, cómo se usa, por qué se usa, y dejarlo listo para continuar su aprendizaje con temas más complejos (y estructuras de datos más complicadas).

Ojalá este libro se pueda leer como una buena novela, en que cada capítulo introduce nuevos personajes, más acción, y nos deje con ganas de una secuela.

¹Con salto de fe nos referimos a aquellas cosas que se aprenden, pero que solo se usan, hasta que más adelante se entiende realmente de qué tratan.

Acknowledgements

- A special word of thanks goes to Professor Don Knuth² (for T_EX) and Leslie Lamport³ (for L^AT_EX).
- I'll also like to thank TeXstudio's⁴ developers for their awesome L^AT_EX editor.
- Well... I also used Visual Studio Code⁵.
- And LuaT_EX⁶.
- A la profesora Loreto Telgie que durante muchos años creó material imprescindible para la asignatura de **Programación Avanzada**.
- Al profesor Cristian Chiang que creó material de estudio y ejercicios durante años.
- A Boris Bugueño y Felipe Peña, por cooperar a este documento con sus revisiones y sugerencias.

Eric Ross Cortés

<https://www.tinkerlabs.cl/eross/>

²<https://www-cs-faculty.stanford.edu/~knuth/>

³<http://www.lamport.org/>

⁴<https://texstudio.org/>

⁵<https://code.visualstudio.com/>

⁶<https://www.luatex.org/>

Introducción

Considering the current sad state of our computer programs, software development is clearly still a black art, and cannot yet be called an engineering discipline.

– Bill Clinton

Programming is like kicking yourself in the face, sooner or later your nose will bleed.

– Kyle Woodbury

Talk is cheap. Show me the code.

– Linus Torvalds

El objetivo primordial de una introducción es enganchar al lector, despertar su interés. La introducción debe transmitir lo buen escritor que eres, recuerda que tu misión será intrigar al lector para que compre tu libro.

¿Por qué este libro se llama como se llama? Por qué no solamente se llama **Programación**?

Todos los títulos son arbitrarios, y lo único que queremos hacer es marcar una diferencia entre lo que se llama programación estructurada y programación orientada al objeto.

Como aquí y ahora no sabes efectivamente qué es la programación orientada al objeto, podemos decir lo siguiente, incluso antes de que empieces a leer este libro: ¡te va a cambiar la vida! Bueno, tal vez no cambiará nada en tu vida, pero sí tendrá un efecto tan grande como el cambio que sucedió en ti cuando aprendiste a programar (por ejemplo en **Python**).

Cuando continúes avanzando en este libro, y vayas aprendiendo los tópicos que hay en estas N páginas, podrás resolver problemas de una forma que actualmente (asumiendo que tu aventura está recién comenzando) desconoces. Ésto no quiere decir que literalmente cualquier cosa se puede resolver programándola de forma estructurada, pero el diseñar software usando objetos, herencia, enviando mensajes, es otra cosa.

0.1. ¿Qué necesitas para enfrentar este libro?

Sería ideal poder decir que no necesitas nada para comenzar a leer este libro, pero lamentablemente, sería publicidad engañosa. La verdad, necesitas tener en tu mente muchas cosas antes de poder recién pensar las páginas que siguen. De hecho, este libro está estructurado pensando en que esas cosas ya las sabes. Tal vez algún día este libro se transforme en un libro de esos que se pueden leer solos, y el título se transforme en algo como “Programación Orientada al Objeto desde CERO”, pero por ahora, se espera que ya tengas el tópico “Programación” en tu mente.

Aparte de lo anterior, hay un par de conceptos que sería ideal que conocieras. La verdad, es muy posible que ya los conozcas, por que asumimos que ya tienes algo de experiencia al programar, pero es totalmente necesario que en la medida que tu aprendizaje avance, puedas sentir que efectivamente vas avanzando, y pasas de estar entre los novatos a estar entre los gurús. Además, también conviene que sepas y aprendas a reconocer cuando hay una discordancia entre lo que crees que sabes y lo que sabes realmente.

0.1.1. El modelo Dreyfus

Hay muchos modelos de aprendizaje que tratan acerca de la adquisición de habilidades. Uno de ellos se llama el modelo Dreyfus. Éste comenzó con dos hermanos, allá por los años 70, ya que un día se preguntaron qué tan fácil o difícil sería hacer que un computador aprendiera cosas, por ejemplo, que aprendiera a jugar ajedrez.

Para hacer ento comenzaron investigando cómo funciona el proceso de aprendizaje, de manera que después pudieran programar una computadora para que lo hiciera. Y uno de los temas que investigaron fue cómo algunas profesiones que requieren un alto nivel de habilidades (por ejemplo, pilotos de aviones) adquieren conocimiento, y lo que descubrieron fue que a medida que se desarrollan habilidades, se pasa por cinco niveles a medida que se aprende la habilidad:

Novato Solamente necesita las reglas. El novato dirá “no me digas nada, solo dime las reglas para poder realizar la tarea. Si estoy aprendiendo a jugar tenis, solo necesito que me digas la forma en que le puedo pegar a la cosa redonda con la cosa que parece un bastón. No importa cuántas veces falle, pero lo seguiré intentando, aplicando ciegamente las reglas que conozco, y cuando logre pegarle a la pelota sentiré que ya domino el tenis completamente.”

En este punto no tiene ninguna relevancia para el novato la posición del cuerpo, o cómo posicionar los pies, o las velocidades, el giro de la pelota, el balance del cuerpo, o cómo apuntar la pelota. Toda esa información solamente serviría para agobiarlo. El novato solamente quiere pegarle a la cosa redonda amarilla con la raqueta y nada más.

Pero después de un tiempo, al empezar a mejorar en golpear la pelota, el novato pasará al siguiente nivel.

Novato avanzado En este punto el novato entiende las reglas, y trata de entender por qué las reglas son como son.

Un novato en un arte marcial puede comenzar aprendiendo todos los movimientos básicos, y en algún momento cuestionar el por qué cierto movimiento se hace de la forma en que se hace. El novato puede empezar a poner en práctica su nueva versión de un movimiento (por ejemplo,

en vez de realizar un bloqueo de la forma en que se le enseñó, comenzará a practicarla a “su” manera), y darse cuenta que efectivamente “su” manera de realizarlo no es para nada efectiva respecto a la forma inicial que aprendió. Esto reforzará lo aprendido inicialmente.

En este nivel el novato está probando las reglas, viendo qué pasa en los casos extremos, y de esta forma empieza a internalizar las reglas, y eventualmente pasará al siguiente nivel.

Competente En este nivel, la persona ya puede aplicar las reglas para lograr un objetivo. Por ejemplo, si es competente haciendo café, ya se le puede pedir uno, y se puede confiar que sabrá hacerlo, ya que tiene internalizadas las reglas, sabe hervir agua, y poner café dentro de una taza, usar competentemente una cuchara, e incluso puede preguntar si es necesario ponerle azúcar o leche.

Este es el nivel al que la mayoría de las personas llegan cuando aprenden una habilidad. Si la persona decide que quiere realmente invertir tiempo y esfuerzo en aprender a fondo una habilidad, debe pasar al siguiente nivel.

Capaz Acá realmente se internalizan las reglas. Las soluciones a los problemas comienzan a aparecer completamente formadas en la cabeza de la persona. Se empieza a tener visión, intuición e instinto. Pero frecuentemente la persona no tiene la suficiente confianza en sí misma y vuelve a usar las reglas como línea base.

En este nivel la persona aun está aprendiendo a confiar en su instinto, pero eventualmente aprende más y más, comienza a investigar, a ver cómo otras personas resuelven los problemas, comienza a probar cosas que están fuera de las reglas, y eventualmente pasa al siguiente nivel.

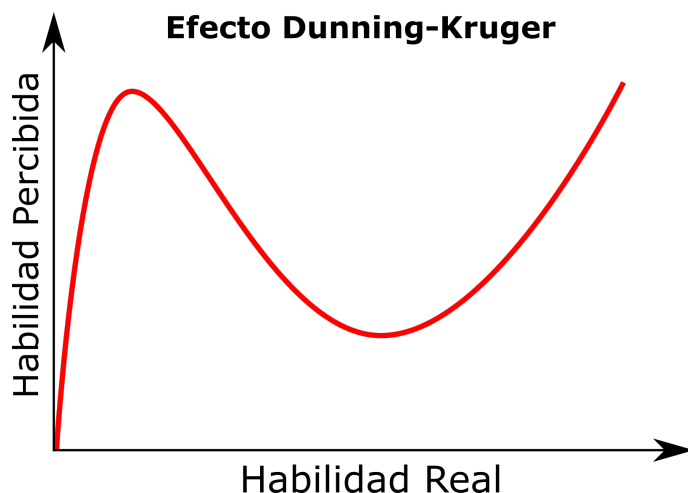
Experto La persona en este nivel trasciende las reglas. Ahora entiende por qué existen las reglas, ya que alguien como ella las creó. Y en este punto puede pensar sin reglas, cumple sus objetivos y es capaz de usar su experiencia creativamente para resolver problemas.

0.1.2. El efecto Dunning-Kruger

El efecto Dunning-Kruger es un sesgo cognitivo, en el cual las personas con poco conocimiento o habilidad tienden a sobrevalorar su propia habilidad o conocimiento, sintiéndose algunas veces superiores al compararse con otras personas con mayor preparación.

Las personas que tienen este sesgo fallan al reconocer su propia limitación, o saben tan poco del tema en cuestión que consideran que eso corresponde a todo lo que se pueden aprender y ya han dominado todo lo que necesitan saber.

Generalmente esto se muestra en forma gráfica tal como se ve en la siguiente figura.



A finales de los años 90, David Dunning y Justin Kruger de la Universidad de Cornell concluyeron que: “La sobrevaloración del incompetente nace de la mala interpretación de la capacidad de uno mismo. La infravaloración del competente nace de la mala interpretación de la capacidad de los demás”. La hipótesis de su trabajo apuntaba a probar si:

1. Los individuos incompetentes tienden a sobrestimar su propia habilidad.
2. Los individuos incompetentes son incapaces de reconocer la habilidad de otros.
3. Los individuos incompetentes son incapaces de reconocer su extrema insuficiencia.
4. Si pueden ser entrenados para mejorar sustancialmente su propio nivel de habilidad, estos individuos pueden reconocer y aceptar su falta de habilidades previa.

Es importante que todas las personas que buscan adquirir una habilidad (por ejemplo, programar) reconozcan en dónde se encuentran, de forma que el proceso de aprendizaje se lleve a cabo de manera normal y completa.

Parte I

Libro Guía

1

Nivelación

En este capítulo aprenderás lo necesario para poder hacer la transición desde el lenguaje Python, hacia el lenguaje `Java`, que se utilizará en el resto de este libro. Aun cuando es cierto que se puede escribir lo que necesitamos en Python, se decidió utilizar `Java` ya que es algo que toda persona de la profesión debería aprender.

1.1. ¿Qué es Java?

Java es un lenguaje de programación de alto nivel, orientado al objeto y basado en clases. Hace años uno de sus lemas era “escríbelo una vez, ejecútalo en todas partes”, significando que el código `Java`, una vez compilado, puede ejecutarse en todas las plataformas que soporten `Java`, sin necesidad de recompilar el código.

Los programas escritos en `Java` se compilan a “bytecode” que puede ejecutarse en cualquier máquina virtual `Java`¹, independiente de la arquitectura de la máquina real. Mientras exista una JVM para un sistema operativo y arquitectura, entonces el bytecode se podrá ejecutar sin problemas.

La sintaxis de `Java` es similar a la de C y C++, pero sin todas las capacidades de bajo nivel. Esto significa que no hay acceso directo a la arquitectura de la máquina local, sino que se utiliza una capa de abstracción entre el hardware y los programas `Java`, y las instrucciones de bajo nivel de la máquina no necesariamente son las mismas que las instrucciones del bytecode.

Hoy, 14 de abril de 2022, `Java` está en el lugar 3 de popularidad en el índice TIOBE².

¹JVM, Java Virtual Machine

²<https://www.tiobe.com/tiobe-index/>

1.2. Ejecutando Java

Si ya tenemos una aplicación escrita en `Java`, y solo necesitamos ejecutarla, entonces lo único que se requiere es instalar el JRE³. Al instalarlo, se instala la JVM y permitirá ejecutar programas ya compilados.

1.3. Compilando programas

Por otro lado, si queremos compilar código escrito en `Java`, se requiere instalar un JDK⁴. Éste instalará una serie de aplicaciones que permitirán compilar el código y transformarlo en bytecode. Este bytecode se almacena como archivos con extensión `.class`, y son estos archivos los que se “ejecutan” en la JVM (que el JDK también incluye).

Si solo instalamos el JDK, los programas se deben compilar usando la herramienta `javac`, que es un programa de línea de comandos. En este caso, normalmente los archivos `.java` se editan en algún editor de texto, posteriormente se compilan y al final se ejecutan. Por ejemplo, se podría crear este archivo `main.java` en un editor:

```
1 public class Main
2 {
3     public static void main(String[] a)
4     {
5         System.out.println("Hola mundo");
6     }
7 }
```

y compilarlo al ejecutar⁵ lo siguiente:

```
javac Main.java
```

y finalmente ejecutarlo:

```
java Main
```

con lo que se obtendría por pantalla lo siguiente:

```
Hola mundo
```

Nótese que al compilar el archivo, se especifica el nombre completo de éste, pero al ejecutarlo, solo se debe indicar el nombre, sin la extensión (ni `.java` ni `.class`).

Generalmente, para mejorar la experiencia al programar, se instalan entornos integrados de desarrollo⁶, que permiten realizar todas las tareas (escribir código, compilar, ejecutar, depurar) en un solo lugar.

³Java Runtime Environment

⁴Java Development Kit. Ver capítulo A

⁵En Windows se requerirá abrir una ventana de “command prompt” o “símbolo del sistema”, y ejecutar los comandos. En Linux solo hay que abrir un terminal.

⁶IDE: integrated development environment, entorno integrado de desarrollo

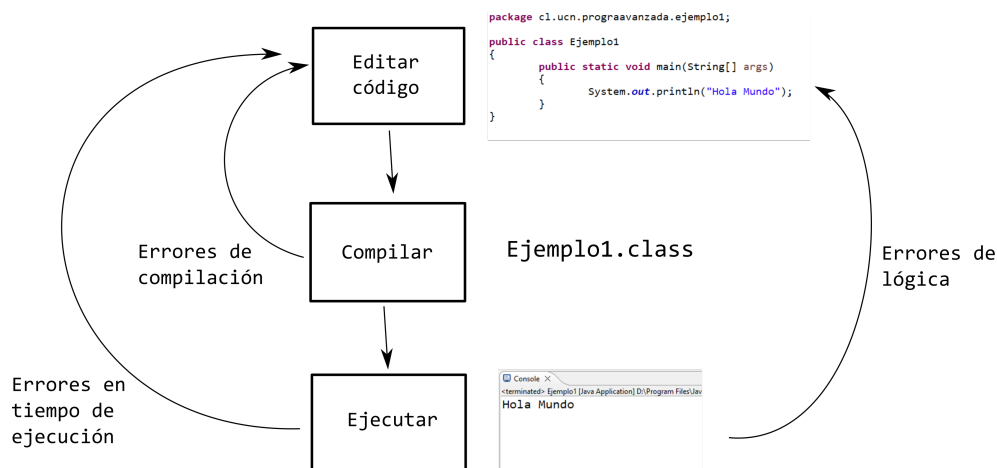


Figura 1.1: Archivo creado y listo para continuar la implementación

1.4. El proceso de desarrollo

Como se ve en la figura 1.1, cuando escribamos programas se seguirá un proceso cíclico que avanzará dependiendo de los problemas que se encuentren.

Si todo va bien, el proceso seguirá la secuencia *editar* → *compilar* → *ejecutar*, pero al encontrar algún problema (por ejemplo, errores en la sintaxis) al realizar el proceso de compilación el compilador nos entregará un listado de errores que debemos corregir antes de poder avanzar a la siguiente etapa.

Por otro lado, aun cuando el proceso de compilación se realice correctamente, incluso así se pueden encontrar errores, en la forma de problemas que causan que el programa termine su ejecución de forma anticipada y catastrófica, y errores en los que todo parece estar bien, pero se entregan resultados incorrectos.

Son estos últimos tipos de errores los más complicados de resolver, ya que requiere analizar la lógica utilizada al programar, y corregir lo implementado.

1.5. Documentando los programas

Cuando escribimos nuestros programas, ya debes saber que es una buena práctica documentar lo que hacemos. Y una de las forma de hacer ésto es agregando “comentarios” al código. Por ejemplo:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         // Vamos a escribir un saludo a la pantalla
6         //
7         System.out.println("Hola Mundo");
8     }
9 }

```

En el caso anterior, hemos agregado dos líneas de comentarios. Como se puede ver, en Java los comentarios de este tipo se inician usando `//`, y todo lo que esté a la derecha del símbolo será ignorado por el compilador:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         System.out.println("Hola Mundo"); // Vamos a escribir un saludo a la pantalla
6     }
7 }
```

Muchas veces vamos a querer escribir comentarios como estos:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         // Vamos a iniciar el programa, y antes de
6         // realizar cualquier acción, saludaremos
7         // al usuario para que sepa que es bienvenido
8         // y bienvenida a usar nuestro programa
9         //
10        System.out.println("Hola Mundo");
11    }
12 }
```

Aun cuando no hay problemas al hacer lo anterior, en Java (y otros lenguajes) se puede escribir así:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         /*
6         Vamos a iniciar el programa, y antes de
7         realizar cualquier acción, saludaremos
8         al usuario para que sepa que es bienvenido
9         y bienvenida a usar nuestro programa
10        */
11        System.out.println("Hola Mundo");
12    }
13 }
```

O sea, todo lo que esté entre `/*` y `*/` será tratado como comentario e ignorado.

Muchas veces verás código que está escrito de esta forma:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         /*
6         * Vamos a iniciar el programa, y antes de
7         * realizar cualquier acción, saludaremos
8         * al usuario para que sepa que es bienvenido
9         * y bienvenida a usar nuestro programa
10        */
11        System.out.println("Hola Mundo");
12    }
13 }
```


En este caso, se escribe de esa forma solo por motivos cosméticos.

Hay un tema interesando al hablar de documentación del código en Java, llamado `javadoc`, pero lo dejaremos para más adelante. Para más detalles, puedes ver el anexo D en la página 431.

1.6. Datos y sus tipos

Al hablar de Java y compararlo con el lenguaje Python, una de las principales diferencias que vamos a encontrar es que en su forma más natural, en Python cuando se necesita usar una variable, simplemente se escribe su nombre y “automáticamente” ésta se crea, con el valor que le estemos asignando en ese momento. Por ejemplo:

```
1 a = 10
2 b = 20
3 c = a + b
4 print(c)
```

En este caso, al llegar a la línea 1 se crea la variable `a` y se le asigna el valor entero 10. En la línea 2 sucede algo similar. Y en la línea 3 se crea la variable `c`. En ningún caso fue necesario realizar ninguna acción más allá de escribir el nombre de la variable y asignarle un valor para que ésta fuera creada.

Lo anterior no significa que las variables no tengan “tipo”: el intérprete Python mantiene un registro del tipo de todas las variables. En este sentido, Python es un lenguaje:

Fuertemente tipado Existen restricciones acerca de cómo se pueden mezclar los tipos de datos. No se permiten realizar acciones que sean incompatibles con el tipo de dato que se está manipulando. El ejemplo más típico es al querer concatenar una variable que contiene un entero con otra que contiene un string: lo que siempre terminamos haciendo es convertir el entero a string usando `str(...)` para poder realizar la concatenación.

Dinámico El tipo de las variables se determina en tiempo de ejecución. Se tiene toda la libertad para asociar nombres (variables) a diferentes objetos de tipos diferentes. Siempre que no trates de realizar acciones inválidas para el tipo actual, a Python no le importará el tipo de las variables ni que reutilices una variable primero para un tipo y después para otro.

Al contrario, Java es un lenguaje de “tipado estático”, en el que el tipo de las variables se debe conocer al momento en que éstas se usan. Si esto no se realiza se producirá un “error en tiempo de compilación”.

A este proceso le llamaremos “declarar la variable”, y nos permitirá especificar dos cosas: el nombre de la variable y su tipo. En Java existe la diferencia entre tipos de datos *primitivos* y *no primitivos*. Los no primitivos los veremos más adelante. Por ahora es necesario especificar que Java tiene los siguientes tipos de datos primitivos:

Tipos enteros Permiten almacenar solamente valores enteros. Dependiendo del “tamaño” del número que se quiera almacenar, se puede elegir entre los siguientes tipos:

Nombre	Tamaño	Valor Mínimo	Valor Máximo
byte	1 byte	-128	127
short	2 bytes	-32,768	32,767
int	32 bits	-2,147,483,648	2,147,483,647
long	8 bytes	-9,223,372,036,854,775,808	9,223,372,036,854,775,807

Tipos decimales Permiten almacenar valores decimales. Dependiendo del tamaño del valor y la precisión buscada, se puede elegir entre dos tipos:

Nombre	Tamaño	Descripción
float	4 bytes	Este tipo de datos se llama de “precisión simple”. Almacenará valores decimales, pero más allá del sexto decimal, se vuelve impreciso. Por ello, no se debe usar para almacenar valores precisos, como valores monetarios. El rango de valores válido va desde el $1,40239846 \times 10^{-45}$ al $3,40282347 \times 10^{38}$
double	8 bytes	Este tipo es de “doble precisión”, así que almacenará hasta 15 dígitos decimales. Por ello, no se debe usar para almacenar valores precisos, como valores monetarios. Su rango de valores válido va desde el $4,9406564584124654 \times 10^{-324}$ al $1,7976931348623157 \times 10^{308}$

Otros Permiten almacenar valores que no pertenecen a ninguna de las dos categorías anteriores:

Nombre	Tamaño	Descripción
boolean	1 byte	Almacena solamente dos valores: verdadero o falso
char	2 bytes	Almacena solamente una letra

Algunos ejemplos incluyendo errores. Nótese que este código *no* compila:

```

1 public class Main
2 {
3     public static void main(String[] argumentos)
4     {
5         int a = 10;
6         int b = -20;
7         int c = 571_123; // Los _ se ignoran, pero permite que los humanos
8                          // no se pierdan al leer números grandes
9         byte d = 200; // Error: el valor es más grande que el tipo
10        byte d2 = 100;
11        short e = 10_571;
12        short e2 = -32999; // Error: el valor es más grande que el tipo
13
14        int f = 100_000_000;
15        long g = 1_234_567_890;
16        long h = 1234567890;
17        float i = 3.14; // Error: javac cree que es un double!
18        float j = 3.14f; // Nos aseguramos de que entienda de que, aun
19                          // cuando parece un double, lo queremos como float
20        double k = 3.13457599923384753929348;
21        double m = 3.13457599923384753929348d;
22
23        boolean n = true;
24        boolean p = false;
25
26        char q = 'a';
27        char r = 65;
28    }
29 }
```

Algo importante de recordar, es que en Java, al tener los tipos de variables rangos de valores estrictos, suceden cosas interesantes al trabajar cerca de esos extremos:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         int a = Integer.MAX_VALUE; // Este es el valor máximo del tipo integer
6         System.out.println(a);
7
8         int b = a + 1; // Overflow!
9         System.out.println(b);
10
11        int c = Integer.MIN_VALUE; // Este es el valor mínimo del tipo integer
12        System.out.println(c);
13
14        int d = c - 1; // Underflow
15        System.out.println(d);
16    }
17 }
```

Lo que se obtendrá por la pantalla es:

```
2147483647
-2147483648
-2147483648
2147483647
```

En otras palabras, cuando un valor entero se “pasa” por el extremo positivo (a esto se le llama “overflow”), el valor se devuelve por el otro extremo (el valor más negativo). De la misma manera, se genera una condición similar cuando tratamos de disminuir un valor más allá del valor mínimo: a esto se le llama “underflow”. Revisa los problemas 1 y 2.

Además de todo lo anterior, hay veces en que nos encontraremos en esta situación:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         double d = 3.14;
6
7         int i = d; // Error! Type mismatch: cannot convert from double to int
8     }
9 }
```

Esto significa que no es posible convertir el valor que está en la variable `d` y traspasarlo a la variable entera `i`. Pero por otro lado, esto funciona sin problemas:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int i = 43143117;
6         double d = i;
7     }
8 }
```

La lógica detrás de este comportamiento, es que el tipo `double` puede almacenar valores más grandes que el tipo `int`. Si tratamos de realizar la asignación, ¿qué pasa con los casos en que el valor que

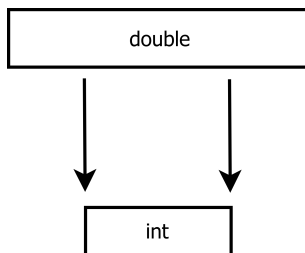


Figura 1.2: Tratando de copiar un valor almacenado en un tipo *double* a un tipo *int*.

queremos transferir no es representable como entero? En el caso de querer asignar un valor decimal, se podría pensar que la solución más sencilla es simplemente solo pasar su parte entera y truncar (¿o redondear?) el valor, pero en el caso general, los valores guardados en el tipo `double` son más grandes que los `int`, como se ve en la figura 1.2.

Debido a esto, es que de forma automática no es posible hacer la transformación cuando pasamos de un tipo “grande” a uno “pequeño”. Pero, sin embargo, es totalmente válido hacer la transformación a la inversa, pasando de un tipo “pequeño” a uno “grande”. Mira de nuevo la figura 1.2, pero imagina que las flechas apuntan en la dirección inversa.

En los casos en que realmente se quiera convertir un valor y almacenarlo en un tipo más pequeño que el original, se tiene que aplicar “casting”. Por ejemplo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         double d = 3.14;
6
7         System.out.println(d);
8
9         int i = (int) d; // casting a entero!
10
11        System.out.println(i);
12    }
13 }
```

En este caso, aplicamos casting explícito, para decirle al compilador que nos hacemos responsables de la conversión, y que realmente queremos pasar el valor a entero. Si ejecutamos lo anterior, se obtiene:

```
3.14
3
```

Como el casting es algo explícito, nos tenemos que hacer responsables de su funcionamiento, y saber qué hacer en todos los casos, por ejemplo, qué pasa cuando el valor inicial no cabe dentro de la variable destino:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         double d = 7859178931057.3143151d;
6
7         System.out.println(d);
8
9         int i = (int) d;
10
11        System.out.println(i);
12    }
13 }
```

La salida de lo anterior es:

```
7.859178931057314E12
2147483647
```

En otras palabras, no se generó ningún error, pero la variable de destino quedó totalmente llena. Revisa el problema 3.

1.7. Estructura de los programas en Java

Ya hemos visto varios ejemplos, pero es hora de definir claramente cómo es un programa en Java.

Recordando cómo se escriben programas en Python, si uno se pregunta cuál es el programa Python más corto, la respuesta es que es un archivo en blanco.

Por otro lado, el programa más corto que podríamos escribir en Java es exactamente este:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5     }
6 }
```

Como se puede ver, se requiere escribir más para poder tener el mismo efecto (en otras palabras, un programa que no hace nada).

Por otro lado, por si no ha sido evidente, los programas en Java están llenos de { y }, ya que estos símbolos toman el lugar de la indentación que se usaba en Python. Por ejemplo:

```

1 a = 20
2 if a > 10:
3     a = a + 1
4     print(a)
5     b = a * 100
6     print(b)
7 print(a)

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int a = 20;
6         if (a > 10) {
7             a = a + 1;
8             System.out.println(a);
9             int b = a * 100;
10            System.out.println(b);
11        }
12        System.out.println(a);
13    }
14 }

```

Ambos códigos hacen exactamente lo mismo. Como se puede ver, en la versión en Java cada variable **debe** ser declarada antes de ser utilizada (y solo se puede declarar una vez), y la forma de delimitar dónde empieza y dónde termina un **bloque** es usando {}. En el código Python, las líneas de la 3 a la 6 son líneas que solamente se ejecutarán si la condición del if es verdadera. Las líneas 7 a la 10 del código Java hacen lo mismo, pero están delimitadas por los símbolos {}. En este sentido, se dice que el **bloque** del if se “abre” en la línea 6, y se cierra en la línea 11.

Debido a lo anterior, en Java existe más libertad al escribir el código:

```

1 public class Ejemplo1 { public static void main(String[] args){int a = 20;
2 if (a > 10) {a = a + 1; System.out.println(a);int b = a * 100;
3 System.out.println(b);} System.out.println(a); }}

```

Este código es efectivamente igual al anterior. Es compilable y el resultado que entregará es el mismo. La única diferencia es que a las personas que tengan que leerlo y modificarlo se les hará muy complicado. Al compilador no le importa cómo está escrito el código: le basta con que sea un código válido y que respete las reglas del lenguaje.

Como te habrás dado cuenta, otra cosa que se utiliza en Java pero no en Python son los ;. Dicho símbolo se utiliza para indicar cuándo termina una línea. ¡Es por eso también que el código Java se puede escribir de cualquier forma!

1.8. Inicio del programa

Otra diferencia entre Python y Java es el lugar donde se inicia la ejecución.

En Python, si “ejecutamos” un archivo, la ejecución se iniciará con la primera línea ejecutable que encuentre:

```

1 # Este es un programa
2
3 def sumador(a, b):
4     c = a + b
5     return c
6
7 q = 10
8 print(sumador(q, q + 1))

```

En este caso, la ejecución se iniciará en la línea 7. Por otro lado, en Java:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         // Este es un programa
6
7         int a = 10;
8
9         a = a + 200;
10
11        System.out.println(a);
12    }
13 }
```

La ejecución se inicia en la línea 3. Más precisamente, el programa se inicia en `main`. Todos nuestros programas comenzarán su ejecución en el `main`, independientemente de dónde se ubique.⁷

1.9. Convenciones de escritura

Aun cuando no es obligatorio, se considera una “buena práctica” el escribir código siguiendo ciertas reglas que hagan que lo escrito sea más fácil de leer, de entender y de modificar.

Técnicamente al compilador no le interesa el cómo está escrito el código (ver el ejemplo en la página 18), pero a las personas que trabajan con dicho código sí les va a interesar.

En particular, en Java existe una serie de convenciones que se utilizan para que código sea lo más legible posible. Algunas son:

Nombres de variables En Java se utiliza lo que se llama **camelCase** para las variables. Esto significa que los nombres de las variables deben empezar con minúscula, y después cada palabra debe comenzar con mayúscula. Por ejemplo, `numMascotas` es un buen nombre de variable, mientras que `NumMascota` no lo es. No se debe usar guión (`_`) para separar los componentes del nombre.

Nombres de otros identificadores Generalmente se utiliza **PascalCase** para otros identificadores. Por ejemplo, `VendedorLibros` o `Vehículo`.

Indentación Aun cuando el uso de las llaves nos permite delimitar dónde empiezan y terminan los bloques, es buena práctica usar indentación de 4 espacios para demarcar visualmente el código que pertenece a un bloque.

Ubicación de llaves En general, las llaves se abren y se cierran en su propia línea, excepto en ciertos casos y en diferentes estilos: muchas veces la llave de apertura (`{`) se deja en la misma línea en el caso del `if`, `while`, `for`, etc.

Uso de espacios en blanco Es buena práctica dejar espacios en blanco al hacer asignaciones, por ejemplo, `int a = 10 * b;` en vez de `int a=10*b;`.

Para más detalles, las convenciones oficiales del lenguaje Java se pueden encontrar en <https://www.oracle.com/technetwork/java/codeconventions-150003.pdf>.

⁷Más adelante escribiremos programas más complejos, donde la frase anterior tendrá más sentido.

1.10. Operadores

Es más que seguro que los programas que se van a escribir necesitan comparar cosas, y tal vez preguntar más de una cosa a la vez. Es por eso que existen los operadores relacionales y lógicos. En general, todos los lenguajes de programación los incorporan, pero se escriben de forma diferente:

Nombre	Python	Java	Retorna verdadero si...
Mayor que	<code>x > y</code>	<code>x > y</code>	x es estrictamente mayor que y
Menor que	<code>x < y</code>	<code>x < y</code>	x es estrictamente menor que y
Mayor o igual que	<code>x >= y</code>	<code>x >= y</code>	x es mayor o igual que y
Menor o igual que	<code>x <= y</code>	<code>x <= y</code>	x es menor o igual que y
Igual que	<code>x == y</code>	<code>x == y</code>	x es igual que y
Diferentes que	<code>x != y</code>	<code>x != y</code>	x es diferente que y
Y (and)	<code>x and y</code>	<code>x && y</code>	tanto x e y son verdaderos
O (or)	<code>x or y</code>	<code>x y</code>	uno de los dos es verdadero
No (not)	<code>not x</code>	<code>!x</code>	x es falso

Nótese que hay algunos “pythonismos” que hay que reestructurar para poder expresarlos en Java:

```

1 x = 15
2
3 if 1 < x < 30:
4     print(x)

```

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int x = 15;
6         if (1 < x && x < 30) {
7             System.out.println(x);
8         }
9     }
10 }

```


1.11. Lectura desde el teclado

En Python es relativamente fácil leer valores desde el teclado: basta usar la función `input(...)` y una conversión al tipo final, si es que fuera necesario (`int(input())`):

```
1 n1 = float(input("Ingrese nota 1: "))
2 n2 = float(input("Ingrese nota 2: "))
3 n3 = float(input("Ingrese nota 3: "))
4 prom = (n1 + n2 + n3) / 3
5 print("Promedio es", prom)
6 if prom >= 4:
7     print("Aprobó")
8 else:
9     print("Reprobó")
```

En cambio, en Java la lectura desde el teclado es ligeramente más complicada:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         Scanner scanner = new Scanner(System.in);
6
7         double n1 = Double.valueOf(scanner.nextLine());
8         double n2 = Double.valueOf(scanner.nextLine());
9         double n3 = Double.valueOf(scanner.nextLine());
10
11         double prom = (n1 + n2 + n3) / 3;
12
13         System.out.println("El promedio es " + prom);
14
15         if (prom >= 4) {
16             System.out.println("Aprobó");
17         } else {
18             System.out.println("Reprobó");
19         }
20
21         scanner.close();
22     }
23 }
```

Aun cuando es posible usar otros mecanismos para leer, diferentes a `nextLine()`, es recomendable leer desde el teclado línea a línea, manualmente separar las partes de lo leído, cuando sea necesario:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         Scanner scanner = new Scanner(System.in);
6
7         System.out.println("Ingresa 3 números separados por espacios:");
8
9         String linea = scanner.nextLine();
10
11         String[] partes = linea.split(" ");
12
13         System.out.println("Primer número: " + Integer.valueOf(partes[0]));
14         System.out.println("Segundo número: " + Integer.valueOf(partes[1]));
15         System.out.println("Tercer número: " + Integer.valueOf(partes[2]));
16
17         scanner.close();
18     }
19 }
```

Como ya lo debes saber (y/o adivinar), `.split(...)` es lo que nos permitirá separar un `String` de acuerdo a algún delimitador. En la sección 1.18.1 en la página 59 veremos eso en más detalle.

1.12. Diferencias en el if

En Python es normal usar `elif` en ciertos casos, cuando se quiere probar diferentes alternativas y/o casos. En Java se puede realizar lo mismo, pero escrito de forma diferente:

<pre> 1 x = 15 2 3 if x > 30: 4 print("uno") 5 elif x > 20: 6 print("dos") 7 elif x > 10: 8 print("tres") 9 else: 10 print("nada") </pre>	<pre> 1 public class Ejemplo1 2 { 3 public static void main(String[] args) 4 { 5 int x = 15; 6 7 if (x > 30) { 8 System.out.println("uno"); 9 } else if (x > 20) { 10 System.out.println("dos"); 11 } else if (x > 10) { 12 System.out.println("tres"); 13 } else { 14 System.out.println("nada"); 15 } 16 } 17 } </pre>
---	--

Aunque no es complicado, hay que tener cuidado al tener un `if` dentro de otro `if`:

<pre> 1 x = 10 2 y = 20 3 4 if x < 10: 5 if y > 10: 6 print("caso uno") 7 else: 8 print("caso dos") 9 else: 10 if y < 10: 11 print("caso tres") 12 else: 13 print("case cuatro") </pre>	<pre> 1 public class Ejemplo1 2 { 3 public static void main(String[] args) 4 { 5 int x = 10, y = 20; 6 7 if (x < 10) { 8 if (y > 10) { 9 System.out.println("caso uno"); 10 } else { 11 System.out.println("caso dos"); 12 } 13 } else { 14 if (y < 10) { 15 System.out.println("caso tres"); 16 } else { 17 System.out.println("caso cuatro"); 18 } 19 } 20 } 21 } </pre>
--	--

Muchas veces, se tiene diferentes casos, y solo queremos probar el valor concreto de una variable y realizar diferentes acciones dependiendo de qué valor tenemos:

```

1 x = 3
2
3 if x == 1:
4     print("uno")
5 elif x == 2:
6     print("dos")
7 elif x == 3:
8     print("tres")
9 elif x == 4:
10    print("cuatro")
11 else:
12    print("otro")

```

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int dato = 3;
6
7         if (dato == 1) {
8             System.out.println("uno");
9         } else if (dato == 2) {
10            System.out.println("dos");
11        } else if (dato == 3) {
12            System.out.println("tres");
13        } else if (dato == 4) {
14            System.out.println("cuatro");
15        } else {
16            System.out.println("otro");
17        }
18    }
19 }

```

Este último caso es tan frecuente, que muchos lenguajes (entre ellos Java) proveen palabras especiales para manejarlos:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int dato = 3;
6
7         switch(dato) {
8             case 1:
9                 System.out.println("uno");
10                break;
11             case 2:
12                System.out.println("dos");
13                break;
14             case 3:
15                System.out.println("tres");
16                break;
17             case 4:
18                System.out.println("cuatro");
19                break;
20             default:
21                System.out.println("otro");
22            }
23        }
24    }

```

La instrucción `switch` permite “saltar” a un caso, dependiendo del valor que tiene la variable. Si es que hay un caso para el valor dado, la ejecución saltará al código dentro del caso, y al llegar a la instrucción `break`, continuará después del `switch`. En el caso de que ningún “caso” sea una coincidencia, la ejecución saltará al caso por defecto (`default`). Revisa los ejercicios 6 y 7 para más detalles.

1.13. Incrementando variables

Es muy común que se requiera incrementar el valor de una variable en 1. En Python se puede hacer eso de varias maneras. En Java también:

```

1 a = 1
2 a = a + 1
3 a += 1
4
5 print(a) # imprime 3

```

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int a = 1;
6
7         a = a + 1;
8         a += 1;
9         a++;
10
11        System.out.println(a); // imprime 4
12    }
13 }

```

Nótese que en Java (y otros lenguajes) podemos usar la expresión `variable++` para indicar que queremos incrementar en 1 el valor de la variable.

Incluso también existe la expresión `++variable`. La pregunta es ¿existe alguna diferencia entre las dos?

La verdad es que por el uso que se le dará en este libro, podemos considerar que ambas expresiones son iguales, pero la verdad es que técnicamente son diferentes.

Para entender el por qué, primero es necesario decir lo siguiente: *el valor de una expresión es el valor resultado de esa expresión.*

¿Qué significa lo anterior? Mira este ejemplo:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int a = 1;
6         int b = (a = a + 1);
7
8         System.out.println(a); // imprime 2
9         System.out.println(b); // imprime 2
10    }
11 }

```

¿Qué pasó en la línea 6? Fíjate que tenemos la expresión `a = a + 1`. Esta expresión tiene el efecto concreto de incrementar el valor de `a` en 1. Pero también fíjate que a la variable `b` se le está asignando un valor: `int b = .....` ¿Qué valor se le asigna?

Para contestar la pregunta, tenemos que fijarnos qué valor está siendo asignado con la expresión `a = a + 1`. En este caso particular, la variable `a` adquirirá el valor 2, por lo que el “resultado” de la expresión será 2, y es ese valor el que será asignado a `b`.

Lo mismo puede ocurrir de otras formas:

```

1 public class Ejemplo1

```

```
2 {  
3     public static void main(String[] args)  
4     {  
5         int a = 2;  
6  
7         if ((a = a * 3) > 5) {  
8             System.out.println("Hola!");  
9         }  
10  
11         System.out.println(a); // imprime 6  
12     }  
13 }
```

La pregunta es: ¿se imprime "Hola!"? En este caso, el if básicamente está preguntando si $2 \times 3 > 5$, y como $6 > 5$ entonces el println se ejecuta, y la variable a adquiere el valor 6.

Y ¿qué pasa con a++ y ++a? Ejecutemos este código:

```
1 public class Ejemplo1  
2 {  
3     public static void main(String[] args)  
4     {  
5         int a = 2;  
6  
7         int q = a++;  
8  
9         System.out.println(a); // imprime 3  
10  
11         System.out.println(q); // ???  
12     }  
13 }
```

Si aplicamos lo anterior, podríamos decir que q adquirirá el valor final de a (después de incrementada, o sea, 2) .

Pero no es así. La verdad, es que la salida por pantalla es la siguiente:

```
3  
2
```

Podría parecer que es un error, pero la verdad es que a la expresión a++ se la conoce como “usar y después incrementar”. En otras palabras, primero se “usa” el valor de la variable y posteriormente se incrementa. Es por eso que primero se asigna a q (se “usa”) y posteriormente el valor se incrementa, pero en ese punto del tiempo el valor ya fue asignado.

Por otro lado, también existe un “incrementar y después usar”:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int a = 2;
6
7         int q = ++a;
8
9         System.out.println(a); // imprime 3
10
11        System.out.println(q); // ???
12    }
13 }

```

En este caso, la salida por pantalla es:

```

3
3

```

Esto se debe a que el valor primero se incrementa y después se “usa” (asigna a q). Esto se puede usar de diferentes maneras. Por ejemplo:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int i = 0;
6
7         while (i++ < 5) {
8             System.out.println(i);
9         }
10    }
11 }

```

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int i = 0;
6
7         while (++i < 5) {
8             System.out.println(i);
9         }
10    }
11 }

```

No continúes avanzando hasta que sepas la diferencia en ejecución de estos códigos.

1.14. Ciclos

Al igual que en Python, en Java tenemos a nuestra disposición una serie de formas de ejecutar ciclos, de acuerdo a las necesidades de nuestro programa particular.

1.14.1. for

Por ahora, veremos la forma clásica de escribir el ciclo for en Java. Más adelante revisaremos otras formas más específicas.

En su forma más natural, el ciclo for se construye de la siguiente forma:

```

for (inicializador ; condicion ; incremento) {
    // bloque
}

```

Por ejemplo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         for(int i=0;i<5;i++) {
6             System.out.println(i);
7         }
8     }
9 }
```

Este ciclo repetirá 5 veces el print de la línea 6. Fíjate que la variable *i* controla el ciclo, y ésta va a adquirir valores entre el 0 y el 4, ya que la condición es *< 5*. Además, en cada iteración, la variable se incrementará en 1, debido al *i++*.

Otro ejemplo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         for(int i=5;i<20;i+=3) {
6             System.out.println(i);
7         }
8     }
9 }
```

En este caso, el ciclo igual itera 5 veces, pero los valores que adquiere la variable *i* son diferentes: en vez de ir del 0 al 4, va del 5 al 17.

Dependiendo del problema, hay veces en que la variable que controla el *for* no se usará, por lo que tendremos libertad de elegir sus valores. Pero en otras circunstancias dicho valor se puede utilizar para otras acciones, además de controlar el ciclo. En esos casos, es necesario que la variable adquiriera los valores correctos para el problema que estemos solucionando.

Hay que hacer notar que en este ciclo, si se declara una variable en el inicializador del *for*, ésta no será visible fuera del bloque:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         for(int i=5;i<20;i+=3) {
6             System.out.println(i);
7         }
8         System.out.println(i); // Error de compilación
9     }
10 }
```


Si por alguna razón se quiere utilizar el valor final de la variable, entonces es necesaria declararla **fuera** del ciclo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int i;
6         for(i=5;i<20;i+=3) {
7             System.out.println(i);
8         }
9         System.out.println(i);
10    }
11 }
```

1.14.2. while

Tal como existe en Python, en Java podemos usar ciclos `while` cuando no sabemos cuántas veces el ciclo se va a repetir, sino que necesitamos depender de una condición que nos dirá cuándo es necesario detener el ciclo. Por ejemplo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int j = 0;
6
7         while (j < 5) {
8             System.out.println(j++);
9         }
10    }
11 }
```

Esto producirá la siguiente salida por la pantalla:

```
0
1
2
3
4
```

Fíjate que `j` inicia en cero, y que el `print` “usa” la variable antes de incrementarla.

Obviamente podemos hacer cosas más complicadas:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int i = 0, j = 0;
6
7         while (i < 5) {
8             j = i + 1;
9
10            while (j++ < 5) {
11                System.out.print("*");
12            }
13            System.out.println();
14            i++;
15        }
16    }
17 }
```

Fíjate bien, que esto producirá lo siguiente:

```
****
***
**
*
```

Nótese, que dependiendo de las circunstancias, en todo ciclo `while` existe la posibilidad de que el programa nunca “entre” al ciclo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int i = 10, j = 0;
6
7         while (i < 5) {
8             j = i + 1;
9
10            while (j++ < 5) {
11                System.out.print("*");
12            }
13            System.out.println();
14            i++;
15        }
16        System.out.println("FIN");
17    }
18 }
```

1.14.3. do

En Java también tenemos la posibilidad de usar otro tipo de ciclo, que se llama `do`. Este ciclo es parecido al `while` pero tiene la diferencia de que la condición se verifica al final del ciclo:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int i = 1;
6
7         do {
8             i++;
9             System.out.println("Hola " + i);
10        } while (i < 3);
11    }
12 }

```

La salida por pantalla de este ejemplo es:

```

Hola 2
Hola 3

```

Nótese que en este ciclo, al contrario que en el `while`, la ejecución “entra” al ciclo por lo menos una vez, hasta que se evalúa la condición, y si se resuelve como verdadera, el ciclo continúa.

Ésto se puede usar a nuestro favor, para hacer cosas como lo siguiente:⁸

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         Scanner s = new Scanner(System.in);
6         String respuesta;
7
8         do {
9             System.out.print("Ingrese respuesta (S/N): ");
10            respuesta = s.nextLine();
11        } while (!respuesta.equalsIgnoreCase("s") && !respuesta.equalsIgnoreCase("n"));
12
13        System.out.println("Muchas gracias por participar");
14        s.close();
15    }
16 }

```

A esta estructura, usando un ciclo `do`, muchas veces la usaremos cuando se requiera validar los datos que nos llegan desde el teclado, para asegurarnos que corresponden a valores que están dentro del rango esperado por el programa. En el caso anterior, el programa solo esperaba `S` o `N`.

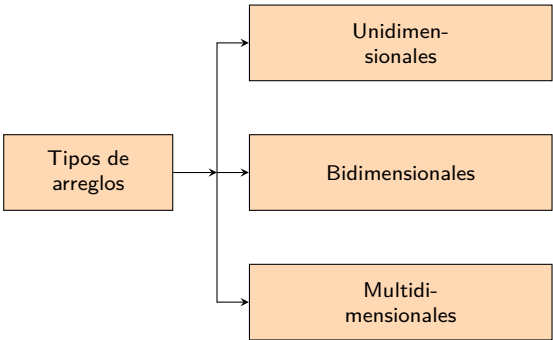
⁸`equalsIgnoreCase(...)` es la forma en que podemos comparar dos Strings, pero haciendo que dicha comparación considere como iguales dos String que solo difieren en sus minúsculas y mayúsculas. Bajo este enfoque, `"hola"` y `"HoLa"` son iguales

1.15. Arreglos

Como sucede en muchos lenguajes, Java nos provee la posibilidad de usar arreglos en los programas.

Como ya debes saber, un arreglo es una colección de elementos, que se almacenan todos bajo un mismo nombre, y que se pueden acceder a través de algo llamado un *índice*.

La cantidad de índices que se requieren para ubicar un elemento dentro de dicho arreglo depende de la cantidad de dimensiones del arreglo. Por ejemplo, para un arreglo de una dimensión (también llamado vector), se requiere solamente un índice. Para un arreglo de tres dimensiones (o también llamado cubo), se requieren tres índices.



En Java, las dimensiones se especifican usando [], y en general necesitaremos un par de [] por cada dimensión.

1.15.1. Arreglos unidimensionales

A estos arreglos los llamaremos comúnmente *vectores*, y los podemos imaginar de esta forma:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
32	12	87	93	45	75	81	65	17

En Java, el primer elemento en cada dimensión se encuentra en el índice 0, y el último en $n-1$, siendo n la cantidad de elementos en dicha dimensión.

Al usar vector en Java, es necesario realizar un par de acciones extras respecto a lo que se hace en Python. Por ejemplo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] arreglo;
6
7         arreglo = new int[10];
8
9         arreglo[0] = 10;
10        arreglo[1] = 14;
11        arreglo[2] = 15;
12
13        System.out.println(arreglo[2]);
14        System.out.println(arreglo[3]);
15    }
16 }
```

En otras palabras, tal como se hace con cualquier otra variable en Java, primero se debe “declarar” la variable que nos permitirá acceder a los elementos del arreglo. En este caso, en la línea 5, se está declarando un vector que guardará números enteros. A diferencia que Python, en Java los arreglos en general solo pueden almacenar elementos de un mismo tipo.

Después, en la línea 7, se realiza el paso de crear realmente el espacio en memoria para almacenar el contenido del arreglo. En el ejemplo, se está reservando espacio para almacenar 10 números enteros.

Entre las líneas 9 a la 11 se muestra cómo se puede cambiar el valor de una “celda” del vector. Hay que hacer notar, que a diferencia de Python, en Java los arreglos se crean de un cierto tamaño (10 en este ejemplo), y siempre ocuparán ese espacio en la memoria.

Ésto nos lleva a la pregunta: si el arreglo se creó con 10 espacios, donde cada espacio puede almacenar un número entero, y el arreglo ya tiene 10 posiciones, antes de ejecutarse la línea 9, ¿qué valores hay en el vector? La verdad es que en Java, los arreglos se inician con el *valor por defecto* del tipo del vector. En este caso, como el tipo es `int`, su valor por defecto es 0.

En las líneas 13 y 14 se muestra cómo se puede consultar el valor que está almacenado en alguna posición del vector. En este caso, el índice [2] fue asignado previamente en la línea 11, pero el índice [3] no fue asignado. Si ejecutamos este programa, podremos ver que entrega esta salida por pantalla:

```
15
0
```

Un detalle interesante es que en Java (y en otros lenguajes), es posible declarar y asignar al mismo tiempo. Usando esta técnica, el programa anterior se transforma en lo siguiente:

```
1 public class Ejemplo1 {
2     public static void main(String[] args)
3     {
4         int[] arreglo = new int[10];
5
6         arreglo[0] = 10;
7         arreglo[1] = 14;
8         arreglo[2] = 15;
9
10        System.out.println(arreglo[2]);
11        System.out.println(arreglo[3]);
12    }
13 }
```

1.15.2. Arreglos bidimensionales

Si agregamos un par de [] más, podemos tener un arreglo de una dimensión superior. En este caso, a esos arreglos les llamaremos *matrices*. Gráficamente, lo normal es imaginarse que las matrices tienen esta estructura:

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[.]	[n-1]
[0]	32	12	87	93	45	75	81	65	17
[1]	87	46	34	61	19	88	76	98	32
[2]	98	51	66	53	81	15	51	52	67
[.]	81	64	26	38	12	11	10	87	46
[m-1]	56	34	25	27	38	32	33	16	76

Para usarlas se deben seguir los mismos pasos anteriores:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[][] matriz = new int[4][5];
6
7         matriz[2][1] = 30;
8         matriz[2][2] = 32;
9         matriz[0][0] = 36;
10        matriz[3][4] = 38;
11
12        System.out.println(matriz[1][1]);
13        System.out.println(matriz[3][4]);
14    }
15 }
```

En este libro, el primer índice en la matriz corresponderá a la fila, y el segundo índice a la columna. Por lo tanto, al ejecutar el código anterior, la matriz se vería así:

	[0]	[1]	[2]	[3]	[4]
[0]	36	0	0	0	0
[1]	0	0	0	0	0
[2]	0	30	32	0	0
[3]	0	0	0	0	38

1.15.3. Arreglos tridimensionales

Solo como ejercicio, podemos crear arreglos de orden superior al 2, pero en el libro no se usarán:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[][][] cubo = new int[4][5][2];
6
7         cubo[0][0][0] = 10;
8         cubo[3][1][0] = 11;
9         cubo[2][4][1] = 12;
10        cubo[3][0][1] = 13;
11
12        System.out.println(cubo[2][4][1]);
13    }
14 }
```

1.15.4. Mostrando los valores de un arreglo

Si en algún momento queremos mostrar los valores presentes en un arreglo, podríamos intentar hacer lo siguiente:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] vector = new int[4];
6
7         for(int i=0;i<4;i++) {
8             vector[i] = i + 2;
9         }
10
11        System.out.println(vector);
12    }
13 }
```

Ésto no producirá ningún error, pero la salida por pantalla no será la que esperamos:

```
[I@7852e922
```

Al contrario que en Python, al imprimir por pantalla un arreglo, éste no se escribe de una forma ordenada, sino que por defecto nos aparecerán unos símbolos que representan a la memoria reservada para dicho arreglo.

Para poder escribir los valores contenidos en un arreglo, será necesario recorrer el arreglo, escribiendo cada valor de forma independiente:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] vector = new int[4];
6
7         for(int i=0;i<4;i++) {
8             vector[i] = i + 2;
9         }
10
11         // Vamos a imprimir los valores en el vector
12
13         for(int i=0;i<4;i++) {
14             System.out.print(vector[i] + ", ");
15         }
16         System.out.println();
17     }
18 }
```

Con este programa obtendremos esta salida:

```
2, 3, 4, 5,
```

Nótese que al final de la salida, hay un , de más. Revisa el problema 12 para solucionarlo.

Si en vez de un arreglo de una dimensión, queremos escribir un arreglo de dos dimensiones, se debe realizar algo equivalente:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[][] matriz = new int[4][5];
6
7         matriz[2][1] = 30;
8         matriz[2][2] = 32;
9         matriz[0][0] = 36;
10        matriz[3][4] = 38;
11
12
13        for(int f=0;f<4;f++) {
14            for(int c=0;c<5;c++) {
15                System.out.print(matriz[f][c] + " ");
16            }
17            System.out.println();
18        }
19    }
20 }
```

Lo que producirá esta salida:

```
36 0 0 0 0
0 0 0 0 0
0 30 32 0 0
0 0 0 0 38
```

Nótese que básicamente estamos escribiendo los números uno a uno, separados por un espacio en blanco. Sería ideal que las columnas de datos aparecieran alineadas. Revisa el problema 13.

1.15.5. Conociendo el tamaño de un arreglo

Dependiendo de lo que hagamos, muchas veces vamos a querer saber el tamaño que tiene un arreglo dado.

En Java ésto es posible simplemente preguntándole al arreglo por su tamaño:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] vector = new int[5];
6
7         System.out.println("Tengo " + vector.length + " celdas");
8     }
9 }
```

Lo que producirá esta salida:

```
Tengo 5 celdas
```

Nótese que `.length` nos indicará el *tamaño* del vector, independiente de cuántas celdas hayamos usado con datos. De hecho, el vector parte lleno de ceros, así que técnicamente igual estamos usando todas las celdas.

Por otro lado, en el caso de arreglos de mayores dimensiones, podemos hacer lo mismo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[][] matriz = new int[3][9];
6
7         System.out.println("Tengo " + matriz.length + " filas");
8         System.out.println("Tengo " + matriz[0].length + " columnas");
9     }
10 }
```

Lo que producirá esta salida:

```
Tengo 3 filas
Tengo 9 columnas
```

En este caso, al preguntarle a la matriz por su tamaño, nos indicará su tamaño en filas. Para saber cuántas columnas tiene, le preguntamos a una de las filas (`matriz[0].length`) por su tamaño, lo que equivale a la cantidad de columnas de la matriz completa.

Por otro lado, en muchos problemas no vamos a saber con antelación la cantidad de celdas que se necesitarán, y como en Java el tamaño de los arreglos no se puede modificar una vez creados, estaremos obligados a crear arreglos de tamaño grande o sobredimensionados.

En ese caso, además del arreglo en sí, también necesitaremos tener una variable que nos indique hasta dónde hemos ocupado el arreglo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] vector = new int[50];
6         int último = 0;
7
8         vector[último++] = 5;
9         vector[último++] = 3;
10        vector[último++] = 9;
11        vector[último++] = 4;
12
13        System.out.println("Tengo " + vector.length + " celdas");
14        System.out.println("y hay " + último + " ocupadas");
15    }
16 }
```

Lo que producirá esta salida:

```
Tengo 50 celdas
y hay 4 ocupadas
```

1.15.6. Errores al manipular arreglos

De vez en cuando sucederá que al manipular un arreglo, por alguna razón tratamos de acceder a un elemento más allá del límite de dicho arreglo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] vector = new int[5];
6         int último = 0;
7
8         vector[último++] = 5;
9         vector[último++] = 3;
10        vector[último++] = 9;
11        vector[último++] = 4;
12        vector[último++] = 1;
13        vector[último++] = 8;
14
15        System.out.println("Tengo " + vector.length + " celdas");
16        System.out.println("y hay " + último + " ocupadas");
17    }
18 }
```

En este caso, lo que obtendremos como salida al ejecutar del programa es:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5
at cl.ucn.prograavanzada.ejemplo1.Ejemplo1.main(Ejemplo1.java:13)
```

Este es un error (y más adelante veremos en detalle qué significa), pero por ahora podemos decir que durante la ejecución del programa (nótese que el código compiló y el programa se ejecutó) se detectó un problema en la línea 13, y el error detectado es del tipo `ArrayIndexOutOfBoundsException`, que básicamente indica que hemos tratado de acceder a un índice fuera del rango válido del vector. Y el

error incluso nos dice que el índice incorrecto fue el 5. La consecuencia del error es que el programa terminó su ejecución de forma inesperada.

Cuando se obtienen estos errores, debería ser fácil detectar el problema, ya que el texto nos indica de qué problema se trata, en qué línea sucedió, y el valor que causó el término de la ejecución del programa.

1.15.7. Algoritmos de ordenamiento en arreglos

Una de las acciones más útiles y requeridas al trabajar con arreglos es ordenar su contenido de acuerdo a algún criterio. En este caso, se parte con un arreglo que está desordenado, y se termina con un arreglo ordenado.

```

1 public static void main(String[] args)
2 {
3     // Con esto estamos creando un arreglo de 8 posiciones, inicializado con
4     // los valores mostrados
5     int[] arreglo = { 5, 3, 7, 6, 8, 9, 1, 2 };
6
7     for(int a=0;a<arreglo.length;a++) {
8         System.out.print(arreglo[a] + " ");
9     }
10    System.out.println();
11
12    /*
13     * La idea de este algoritmo es avanzar una posición a la
14     * vez (variable A), y comparar el valor en A con todos los
15     * demás valores (usando B). Si en algún momento se detecta que
16     * los elementos están en el orden incorrecto (A versus B),
17     * entonces se intercambian.
18     *
19     * Nótese que una vez que el ciclo de B termina, el valor que
20     * queda en la posición A es el correcto, y no hay que volver
21     * a modificarlo.
22     */
23    for(int a=0;a<arreglo.length-1;a++) {
24        for(int b=a+1;b<arreglo.length;b++) {
25            if (arreglo[a] > arreglo[b]) {
26                int temp = arreglo[a];
27                arreglo[a] = arreglo[b];
28                arreglo[b] = temp;
29            }
30        }
31    }
32
33    for(int a=0;a<arreglo.length;a++) {
34        System.out.print(arreglo[a] + " ");
35    }
36    System.out.println();
37 }

```

Al ejecutar el código anterior se obtiene:

```

5 3 7 6 8 9 1 2
1 2 3 5 6 7 8 9

```

Si fuera necesario invertir el orden, basta modificar la condición dentro del ciclo `for` interno.

El caso anterior es el caso clásico donde se tiene un arreglo que contiene la información. Existen situaciones más complicadas, en que se está almacenando más que un simple número, y por ejemplo, se tiene un número y un `String`, y se requiere tomar ésto en consideración al realizar el ordenamiento. Por ejemplo, se podría necesitar ordenar los `String` de manera alfabética, pero en el caso de que existan elementos que compartan el mismo `String` (por ejemplo, podrían ser nombres), los datos se tienen que ordenar por el número (que podría ser la edad relacionada):

```

1 public static void main(String[] args)
2 {
3     int[] edades = { 7, 3, 1, 6, 8, 7 };
4     String[] nombres = { "Ximena", "Pedro", "Ximena", "José", "Diego", "Diego" };
5
6     for(int a=0;a<edades.length;a++) {
7         System.out.print(nombres[a] + "/" + edades[a] + " ");
8     }
9     System.out.println();
10
11    for(int a=0;a<edades.length-1;a++) {
12        for(int b=a+1;b<edades.length;b++) {
13            if (nombres[a].compareTo(nombres[b]) > 0) {
14                int temp = edades[a];
15                edades[a] = edades[b];
16                edades[b] = temp;
17                String aux = nombres[a];
18                nombres[a] = nombres[b];
19                nombres[b] = aux;
20            } else if (nombres[a].compareTo(nombres[b]) == 0) {
21                if (edades[a] > edades[b]) {
22                    int temp = edades[a];
23                    edades[a] = edades[b];
24                    edades[b] = temp;
25                    String aux = nombres[a];
26                    nombres[a] = nombres[b];
27                    nombres[b] = aux;
28                }
29            }
30        }
31    }
32
33    for(int a=0;a<edades.length;a++) {
34        System.out.print(nombres[a] + "/" + edades[a] + " ");
35    }
36    System.out.println();
37 }

```

A lo anterior se le puede llamar ordenamiento multicriterio, y no necesariamente siempre será de dos niveles. Además, el código se puede refactorizar⁹ para que no tenga tantos defectos, pero se verá más adelante.

Al ejecutar el código se obtiene:

```

Ximena/7 Pedro/3 Ximena/1 José/6 Diego/8 Diego/7
Diego/7 Diego/8 José/6 Pedro/3 Ximena/1 Ximena/7

```

Se puede observar que el resultado final es justamente el buscado: para el caso en que más de una persona tenga el mismo nombre, se ordenan por edad.

⁹¿Qué es eso? Para más detalles mira el anexo G en la página 457

Nótese que en el ejemplo anterior lo que se buscaba era ordenar el arreglo completo. En algunos casos vamos a querer ordenar solo una parte del arreglo, por ejemplo, cuando estamos usando un arreglo sobredimensionado. En ese caso, basta que los ciclos solo consideren el rango de valores válidos, y no `arreglo.length`.

Si observaste con atención el ejemplo anterior, seguramente detectaste algo que no se ha explicado aun, y es el uso de algo llamado `compareTo(...)`. Sin entrar en muchos detalles, dicha sentencia nos permitirá comparar dos `Strings` para saber cuál es el orden relativo entre ellos. Por ejemplo:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         String a = "hola";
6         String b = "chao";
7         String c = "alo";
8
9         System.out.println(a.compareTo(b));
10        System.out.println(c.compareTo(a));
11    }
12 }
```

Al ejecutar el código se obtiene la siguiente salida por pantalla:

```

5
-7
```

El significado de los números es que el valor que se obtiene indica cuál es el `String` que debería ir `antes` al ordenarlos de manera alfabética. En el primer caso, el valor 5 indica que la variable `b` contiene el `String` menor (`"chao"` en este caso) al compararla con `"hola"`. En el segundo ejemplo, el valor -7 indica que es la variable `c` la que contiene el `String` menor (`"alo"` en este caso) al compararla con `"hola"`.

1.15.8. Eliminación de un elemento en un arreglo

Otra operación interesante sobre un arreglo es la “eliminación” de un elemento. En realidad, este proceso corresponde a mover los elementos a la derecha del elemento eliminado, para así sobrescribirlo. En este caso, conviene tener una variable extra que nos permita saber cuánto del arreglo está realmente ocupado: es posible que después de eliminaciones sucesivas, un arreglo de tamaño N termine con 0 espacios usados realmente:

```

1 public static void main(String[] args)
2 {
3     int ocupado = 8;
4     int[] arreglo = { 5, 3, 7, 6, 8, 9, 1, 2 };
5
6     for(int a=0;a<ocupado;a++) {
7         System.out.print(arreglo[a] + " ");
8     }
9     System.out.println();
10
11     /*
12     * Esto es complicado, ya que técnicamente no podemos "eliminar"
13     * realmente un elemento desde un arreglo. Lo único que podemos hacer
14     * es lo siguiente:
```

```
15  *
16  * Si tenemos un arreglo así:
17  * [ a b c d e f . . . . . ] ocupado = 6
18  *
19  * y queremos eliminar la "c", entonces hay que ubicar la posición:
20  *
21  * [ a b c d e f . . . . . ] ocupado = 6
22  *      ^
23  *
24  * y "mover" los elementos que están a la derecha de esa posición:
25  *
26  * [ a b d e f . . . . . ] ocupado = 5
27  *
28  * y reduciendo los "ocupados"
29  */
30  int elemento = 8;
31
32  for(int i=0;i<ocupado;i++) {
33      if (arreglo[i] == elemento) {
34          for(int j=i+1;j<ocupado;j++,i++) {
35              arreglo[i] = arreglo[j];
36          }
37          ocupado--;
38          break;
39      }
40  }
41
42  for(int a=0;a<ocupado;a++) {
43      System.out.print(arreglo[a] + " ");
44  }
45  System.out.println();
46  }
```

La salida de este programa es:

```
5 3 7 6 8 9 1 2
5 3 7 6 9 1 2
```

1.15.9. Inserción ordenada en un arreglo

Si al eliminar un elemento se tienen que mover elementos hacia la derecha, al insertar un nuevo elemento hay que hacer la operación inversa: mover elementos hacia la derecha para liberar una celda del arreglo.

En este caso, se asume que el arreglo siempre se mantendrá ordenado, ya que si no es así, entonces el algoritmo no tiene sentido.

```

1 public static void main(String[] args)
2 {
3     // Este arreglo siempre estará ordenado de menor a mayor
4     int[] arreglo = { 2, 3, 5, 6, 8, 9, 0, 0 };
5     int ocupado = 6;
6
7     for(int a=0;a<ocupado;a++) {
8         System.out.print(arreglo[a] + " ");
9     }
10    System.out.println();
11
12    int nuevo = 7; // este es el nuevo valor a insertar
13    int i;
14
15    for(i=0;i<ocupado;i++) {
16        if (nuevo < arreglo[i]) {
17            break;
18        }
19    }
20
21    for(int j=ocupado;j>i;j--) {
22        arreglo[j] = arreglo[j-1];
23    }
24    arreglo[i] = nuevo;
25    ocupado++;
26
27    for(int a=0;a<ocupado;a++) {
28        System.out.print(arreglo[a] + " ");
29    }
30    System.out.println();
31 }

```

Al ejecutar este código, se obtiene:

```

2 3 5 6 8 9
2 3 5 6 7 8 9

```

El código anterior es interesante, ya que funciona en todos los casos: tanto si el nuevo elemento es menor a todos los actualmente presentes en el arreglo, si es mayor que todos, o si quedará al medio.

La ejecución llegará a la línea 21 tanto en el caso de encontrar la posición donde hay que insertar el nuevo elemento, o si se revisa el arreglo completamente y no se encuentra una posición apropiada, lo que significa que el elemento debe quedar al final. En ambos casos, `i` apunta a la posición donde el nuevo elemento se tiene que posicionar. En la línea 21 se hace espacio para el elemento copiando los valores hacia la derecha, y finalmente se posiciona el elemento nuevo en la posición `i` y se incrementa `ocupado` para indicar que hay un nuevo elemento en el arreglo.

1.16. Funciones

Al igual que en Python, es una buena costumbre dividir nuestros programas en trozos que sean más fáciles de entender, de escribir y de corregir. Cuando usamos Python, usamos `def` para definir una función¹⁰. En Java se escriben de forma diferente, pero son equivalentes:

```

1 import numpy as np
2
3 def imprimirMatriz(m):
4     for x in range(len(m)):
5         for y in range(len(m[0])):
6             if m[y][x] > 50:
7                 print("+", end="")
8             elif m[y][x] < 50:
9                 print("-", end="")
10            else:
11                print("=", end="")
12            print()
13        print()
14
15 m = np.zeros([4, 4])
16
17 for x in range(len(m)):
18     for y in range(len(m[0])):
19         m[y][x] = np.random.uniform(0, 100)
20
21 imprimirMatriz(m)
22
23 for x in range(len(m)):
24     for y in range(len(m[0])):
25         m[y][x] = 2 * m[y][x]
26
27 imprimirMatriz(m)

```

```

1 public class Ejemplo1
2 {
3     public static void imprimir(int[][] m)
4     {
5         for (int i=0;i<m.length;i++) {
6             for (int j=0;j<m[0].length;j++) {
7                 if (m[i][j] > 50) {
8                     System.out.print("+");
9                 } else {
10                    if (m[i][j] < 50) {
11                        System.out.print("-");
12                    } else {
13                        System.out.print("=");
14                    }
15                }
16            }
17            System.out.println();
18        }
19        System.out.println();
20    }
21
22    public static void main(String[] args)
23    {
24        int[][] mat = new int[4][4];
25        double t;
26
27        for (int i=0;i<mat.length;i++) {
28            for (int j=0;j<mat[0].length;j++) {
29                t = Math.random() * 100.0;
30                mat[i][j] = (int) t;
31            }
32        }
33        imprimir(mat);
34
35        for (int i=0;i<mat.length;i++) {
36            for (int j=0;j<mat[0].length;j++) {
37                mat[i][j] = 2 * mat[i][j];
38            }
39        }
40        imprimir(mat);
41    }
42 }

```

Al momento de usar funciones, es importante diferenciar entre dos elementos relacionados, similares, pero que no son lo mismo:

Parámetro formal Es lo que está en la línea 3 del código en Java. Corresponde a la definición del parámetro, y en términos concretos lo podemos pensar como una variable que recibirá un valor

¹⁰En este caso, estamos hablando de funciones y procedimientos

que viene desde “afuera” de la función.

Parámetro real Es lo que realmente se pasa a la función, como en las líneas 33 y 40. En este caso, el valor de la variable en esos puntos del programa se pasan al momento de invocar la función. Nótese que en ambos casos, el valor que se pasa es diferente (mejor dicho, el contenido de la matriz es diferente).

Tal como podríamos esperar, el cuerpo de la función (entre las líneas 5 a la 19) están dentro de un par de llaves que definen dónde empieza y dónde termina la función. En este caso, técnicamente se trata de un procedimiento, ya que está definido usando la palabra `void`, así que no retornará nada al momento de invocarla.

Para definir una función, es necesario cambiar la palabra `void` por el tipo específico que se retornará:

```

1 public class Ejemplo1
2 {
3     public static void imprimir(int[][] m)
4     {
5         ...
6     }
7
8     public static int[][] inicializar(int n)
9     {
10        int[][] mat = new int[n][n];
11        double t;
12
13        for (int i=0;i<mat.length;i++) {
14            for (int j=0;j<mat[0].length;j++) {
15                t = Math.random() * 100.0;
16                mat[i][j] = (int) t;
17            }
18        }
19        return mat;
20    }
21    public static void main(String[] args)
22    {
23        int[][] mat = inicializar(4);
24        imprimir(mat);
25
26        for (int i=0;i<mat.length;i++) {
27            for (int j=0;j<mat[0].length;j++) {
28                mat[i][j] = 2 * mat[i][j];
29            }
30        }
31        imprimir(mat);
32    }
33 }

```

En la línea 8 se está definiendo una función, que retornará una matriz, y que recibirá como parámetro el tamaño de la matriz que la función tiene que crear y retornar. En la línea 23 se llama a la función, y el resultado que retorne se asigna al nombre `mat`. Obviamente podemos crear otras funciones:

```
1 public class Ejemplo1
2 {
3     public static void imprimir(int[][] m)
4     {
5         ...
6     }
7
8     public static int sumar(int[][] m)
9     {
10        int suma = 0;
11        for (int i=0;i<m.length;i++) {
12            for (int j=0;j<m[0].length;j++) {
13                suma += m[i][j];
14            }
15        }
16        return suma;
17    }
18    public static void amplificar(int[][] m, int k)
19    {
20        for (int i=0;i<m.length;i++) {
21            for (int j=0;j<m[0].length;j++) {
22                m[i][j] *= k;
23            }
24        }
25    }
26    public static int[][] inicializar(int n)
27    {
28        ...
29    }
30    public static void main(String[] args)
31    {
32        int[][] mat = inicializar(4);
33
34        imprimir(mat);
35        System.out.println(sumar(mat));
36
37        amplificar(mat, 2);
38        imprimir(mat);
39        System.out.println(sumar(mat));
40    }
41 }
```

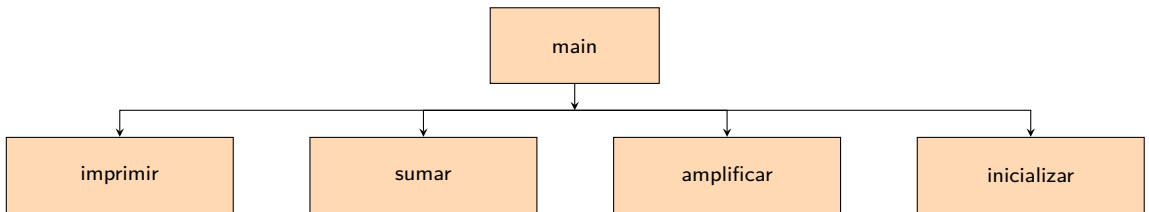
Es de esperar que si vamos a crear una función, es para usarla muchas veces. Por ejemplo, la función `sumar`, que retorna la suma de los valores presentes en la matriz, se usa dos veces (líneas 35 y 39), y la función `imprimir` también se usa dos veces.

La alternativa a no usar funciones es simplemente duplicar el código cada vez que se requiera, pero a estas alturas, eso no es opción, y se considera muy mala práctica.

1.16.1. Estructura de los programas

Desde ahora en adelante, en la mayoría de los programas que se construyan, será necesario tener clara la estructura de éste, en términos de qué funciones están definidas y cómo se usan.

Por ejemplo, para el caso anterior, podríamos decir lo siguiente:

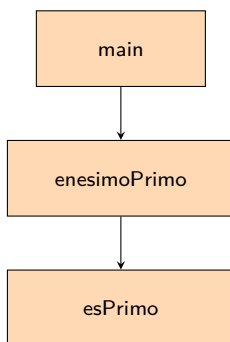


Si tuviéramos el caso de querer calcular el *enésimo* número primo, podríamos considerar el siguiente programa:

```

1 public class Ejemplo1
2 {
3     public static boolean esPrimo(int n)
4     {
5         if (n == 1) return false;
6
7         for (int x = 2; x < n; x++)
8             if (n % x == 0)
9                 return false;
10        return true;
11    }
12
13    public static int enesimoPrimo(int n)
14    {
15        int faltan = n;
16        int candidato = 0;
17        while (faltan > 0) {
18            candidato = candidato + 1;
19            if (esPrimo(candidato)) {
20                faltan = faltan - 1;
21            }
22        }
23        return candidato;
24    }
25
26    public static void main(String[] args)
27    {
28        for (int i = 1; i < 20; i++)
29            System.out.println("El primo " + i + " es " + enesimoPrimo(i));
30    }
31 }
  
```

Y si dibujamos su estructura, tendríamos lo siguiente:



1.16.2. Traspaso de parámetros

¿Qué pasaría si ejecutamos este código?

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int a = 10, b = 20;
6
7         incrementador(a);
8         incrementador(b);
9
10        System.out.println(a);
11        System.out.println(b);
12    }
13
14    public static void incrementador(int variable)
15    {
16        variable += 1;
17    }
18 }
```

En este caso, al ejecutar el código, se obtiene esta salida:

```
10
20
```

Es importante analizar el por qué se obtiene ese resultado. La respuesta es que en Java los parámetros se traspasan “por valor”.

Lo que ésto significa es que al momento de realizar el llamado a la función (línea 7 por ejemplo), el valor que le llega a la función a través del parámetro *variable* es el *valor* de la variable *a* en la línea 7, y el valor de *b* en la línea 8.

Al momento en que la ejecución se transfiere a la línea 14, el parámetro *variable* adquiere el valor copiado desde donde sea que fue llamada la función, y cualquier modificación que se realice a la variable dentro de la función tendrá solamente un *alcance local*.

Si queremos que este código haga lo que tiene que hacer, podríamos reescribirlo de la siguiente forma:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int a = 10, b = 20;
6
7         a = incrementador(a);
8         b = incrementador(b);
9
10        System.out.println(a);
11        System.out.println(b);
12    }
13
14    public static int incrementador(int variable)
15    {
16        variable += 1;
17        return variable;
18    }
19 }
```

En este caso, al ejecutar este código, se obtiene esta salida:

```
11
21
```

Volviendo al primer ejemplo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int a = 10;
6
7         incrementador(a);
8
9         System.out.println(a);
10    }
11
12    public static void incrementador(int variable)
13    {
14        variable += 1;
15    }
16 }
```

Dentro del contexto del main existe la variable a, y en ella se pueden guardar valores, y a la vez se puede leer su contenido.

Al momento en que se hace la llamada a la función `sumador`, en la línea 7, el valor que tiene la variable se *copia* desde el espacio de memoria de la variable a al espacio en memoria donde existirá el parámetro. Fíjate en la figura 1.3.

Al empezar a ejecutarse la función `sumador`, el parámetro `variable` se comporta como una variable normal, y la función puede leer su contenido, e incluso modificar su valor, pero lo que está modificando es un espacio de memoria *diferente* al de la variable a. Fíjate de nuevo en la figura 1.3.

Una vez que la función retorna, el main continúa, y cuando quiere leer el valor de a (ya que en la línea

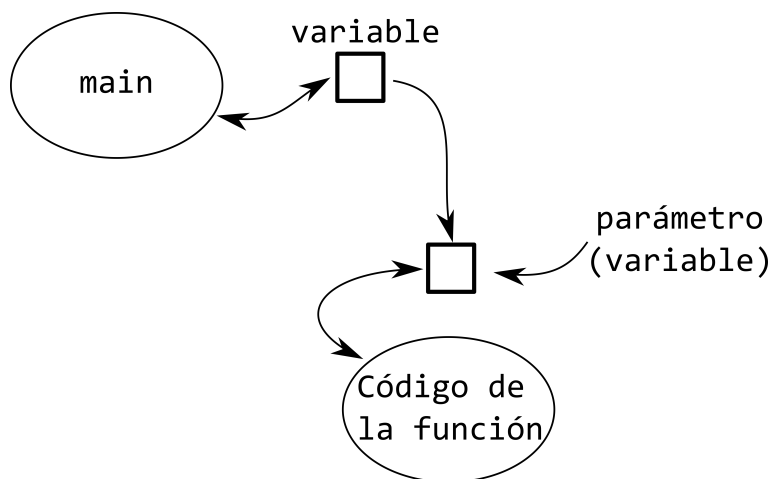


Figura 1.3: Paso de valores a los parámetros de una función

9 necesita escribirlo por la pantalla), el valor lo lee desde una espacio de memoria diferente a donde estaba el parámetro, que fue modificado por la función.

Así es como funciona el paso de parámetros por valor. Pero mira este ejemplo y trata de determinar qué pasará:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] unVector = new int[4];
6
7         for(int i=0;i<unVector.length;i++)
8             unVector[i] = i * 10;
9
10        incrementador(unVector);
11
12        for(int i=0;i<unVector.length;i++)
13            System.out.println(unVector[i]);
14    }
15
16    public static void incrementador(int[] variable)
17    {
18        for(int i=0;i<variable.length;i++)
19            variable[i] += 1;
20    }
21 }
  
```

Se podría pensar que debido a que al hacer la llamada a la función en la línea 10 solo se copia el valor de la variable, al modificar los valores en la línea 19 el cambio se realizaría en el contexto de la función incrementador. Si ejecutamos el código, obtendremos lo siguiente:

```

1
11
21
31
  
```

¿Cómo se puede explicar esto?

La explicación anterior sigue siendo válida, pero hay que entender la forma en que Java interpreta y accesa a los datos cuando se trata de tipos de datos no escalares, o sea, variables simples de tipo primitivo (int, long, etc.).

En ese caso, al pasar un vector como parámetro, lo que la variable pone en la casilla que corresponde al parámetro no es el valor completo del vector, sino que es la ubicación en la memoria donde reside dicho vector. Mira la figura 1.4.

Cuando desde Java accedemos a los contenidos de un vector, lo que se hace es determinar la ubicación de la celda buscada sumando la dirección de memoria donde *empieza* el vector mas el valor del índice. Realizando dicha suma se puede saber la ubicación de memoria de la celda que se quiere referenciar.

Como al llamar a la función se está copiando la ubicación de la memoria, al llegar la ejecución a la función y ésta lee o modifica una posición del vector, estará modificando el mismo lugar de la memoria que se ve desde el main.

Pero mira qué pasa en este caso:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] unVector = new int[4];
6
7         for(int i=0;i<unVector.length;i++)
8             unVector[i] = i * 10;
9
10        incrementador(unVector);
11
12        for(int i=0;i<unVector.length;i++)
13            System.out.println(unVector[i]);
14    }
15
16    public static void incrementador(int[] variable)
17    {
18        variable = new int[10];
19
20        for(int i=0;i<variable.length;i++)
21            variable[i] += 1;
22    }
23 }
```

Fíjate en la línea 18, y mira la figura 1.5. Al ejecutar la línea 18, lo que se está realizando es cambiar el valor que se guarda en el parámetro, y se le deja *apuntando* a otra ubicación de la memoria. Es por esto, que las líneas siguientes, aun cuando realizan el incremento en 1 a los contenidos del vector, en este punto están haciendo referencia a un nuevo vector, y al retornar la función, el vector original no ha sido modificado.

Esta forma de operar no se limita solo a los vectores. También se aplica a todos los arreglos, y a otros tipos que se verán más adelante.

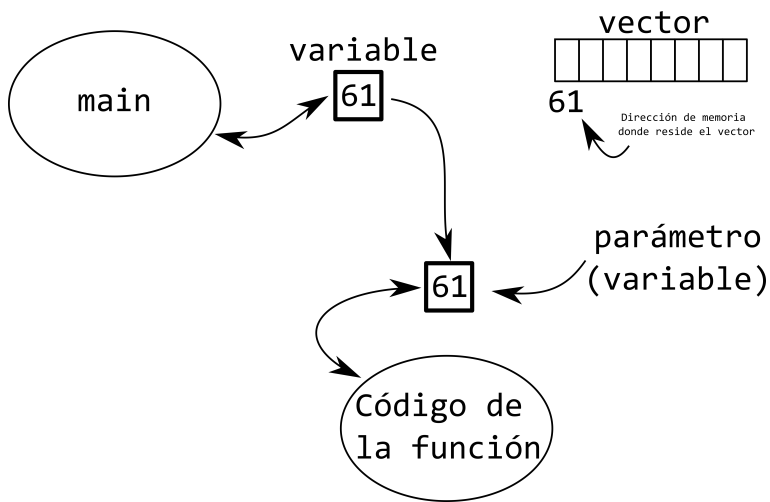


Figura 1.4: Paso de un vector como parámetro de una función

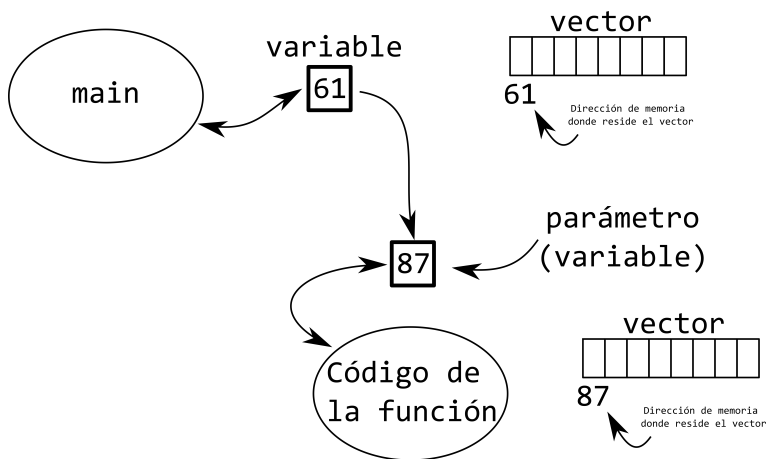


Figura 1.5: Paso de un vector como parámetro de una función y asignación de un nuevo vector.

1.16.3. Escribiendo funciones

Hasta ahora, cuando se han escrito funciones en los ejemplos, no se ha dado ninguna explicación acerca de su estructura o cómo se escriben. Por ahora, se dará una receta a seguir, y más adelante se explicarán los detalles y el por qué ciertas cosas se escriben como se escriben.

En general, escribiremos las funciones “dentro” de la *clase* donde estamos trabajando¹¹. No importa el orden en que se escriban. Solo tienen que estar bien escritas, con sus llaves correctamente balanceadas.

```
1 public class Ejemplo1
2 {
3     public static void amplifica(double[] vector)
4     {
5         // acá amplificamos los elementos de un vector
6     }
7     public static void main(String[] args)
8     {
9         // El "main" puede estar en cualquier parte
10        // dentro de la clase
11    }
12    public static String modificador(String texto)
13    {
14        // Acá modificamos un String
15        return texto + "_final";
16    }
17    public static void revisar(long valor)
18    {
19        // ...
20    }
21 }
```

Por ahora, la receta también indica que las funciones se deben definir así:

```
1 public class Ejemplo1
2 {
3     public static [RETORNO] [NOMBRE] ([PARÁMETROS])
4     {
5         // ...
6     }
7 }
```

El retorno puede ser cualquier tipo, o la palabra especial `void` para indicar que no se retornará nada. El nombre debe ser un nombre apropiado para la función. Debe ser una acción, y estar bien escrita usando camel case. Los parámetros se especificando como una serie de `tipo nombre` separados por coma. En los casos en que la función no necesite ningún parámetro, simplemente se escribe `()`.

¹¹Más adelante veremos qué es una clase

Por ejemplo:

```
1 public class Ejemplo1
2 {
3     public static void amplifica(double[] vector)
4     {
5         // Acá se recibe un parámetro,
6         // y no se retorna nada
7     }
8     public static void main(String[] args)
9     {
10    }
11    public static String modificador(String texto)
12    {
13        // Acá recibimos un parámetro y retornamos
14        // un String
15        return texto + "_final";
16    }
17    public static void revisar(long valor, double otroValor)
18    {
19        // Acá no retornamos nada, y recibimos dos
20        // parámetros
21    }
22    public static int generarNúmero()
23    {
24        // Esta función no recibe parámetros, y retorna un
25        // número cada vez que es invocada
26        return 10;
27    }
28 }
```

1.17. Documentando los programas II

Ahora que ya podemos escribir programas más complejos, es necesario documentarlos, de forma que dejemos registro de lo que hace cada función y cada variable, para hacernos la vida más fácil en el futuro.

Para ésto, usaremos el estándar que trae Java, y que se llama javadoc. La forma de documentar se basa en agregar comentarios con cierto formato, que permitirá que las personas que lean el código lo entiendan, además de permitir que la IDE lea la documentación y genere ayudas que nos apoyarán a la hora de escribir código.

Revisemos el código anterior, pero ahora comentado:

```
1 public class Ejemplo1
2 {
3     /**
4      * Esta clase amplifica cada elemento del vector y
5      * lo multiplica por 2.
6      *
7      * @param vector El vector que debe amplificar
8      */
9     public static void amplifica(double[] vector)
10    {
11        // Acá se recibe un parámetro,
12        // y no se retorna nada
```

```

13     }
14
15     public static void main(String[] args)
16     {
17     }
18
19     /**
20      * Modifica un String que recibe como parámetro y retorna
21      * un String modificado.
22      *
23      * @param texto El String que hay que modificar
24      * @return Un String con el contenido del String original,
25      *         pero modificado de acuerdo a lo especificado.
26      */
27     public static String modificador(String texto)
28     {
29         // Acá recibimos un parámetro y retornamos
30         // un String
31         return texto + "_final";
32     }
33
34     /**
35      * Revisa los valores pasados como parámetros para asegurarse
36      * de que están en el rango correcto.
37      *
38      * @param valor El primer valor a verificar
39      * @param otroValor El segundo valor a verificar
40      */
41     public static void revisar(long valor, double otroValor)
42     {
43         // Acá no retornamos nada, y recibimos dos
44         // parámetros
45     }
46
47     /**
48      * Genera y retorna un nuevo número
49      *
50      * @return El nuevo número generado de acuerdo al diseño
51      *         del proceso.
52      */
53     public static int generarNúmero()
54     {
55         // Esta función no recibe parámetros, y retorna un
56         // número cada vez que es invocada
57         return 10;
58     }
59 }

```

Estos comentarios, además de servir para leer directamente el código, nos ayudarán a la hora de escribir, ya que en el caso de las IDEs, permite que se muestre ayuda en línea mientras programamos, como se ve en la figura 1.6. Más adelante aplicaremos este estilo de comentarios a otras estructuras en el código, pero por ahora basta documentar claramente cada función que compone nuestro programa. Más detalles acerca del estándar `javadoc` se puede encontrar en el anexo D.

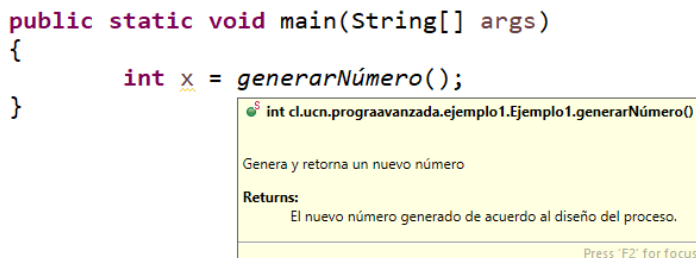


Figura 1.6: Eclipse mostrando la documentación generada a partir de los comentarios incluidos en el código.

1.18. Archivos

En Java es relativamente fácil leer el contenido de archivos que necesitemos procesar. En general se siguen los siguientes pasos:

1. Abrir el archivo
2. Leer línea a línea el archivo, usando los datos leídos
3. Cerrar el archivo

Por ejemplo, si tenemos el siguiente archivo con datos:

```

Este es
un archivo de
ejemplo para revisar
la lectura
de archivos
desde
Java

```

Y el siguiente programa:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args) throws IOException
4     {
5         File file = new File("d:/temp/ejemplo.txt");
6         Scanner lector = new Scanner(file);
7
8         while (lector.hasNextLine()) {
9             String linea = lector.nextLine();
10            System.out.println("Línea: '" + linea + "'");
11        }
12
13        lector.close();
14    }
15 }

```

La salida por pantalla será:

```
Línea: 'Este es'  
Línea: 'un archivo de'  
Línea: 'ejemplo para revisar'  
Línea: 'la lectura'  
Línea: 'de archivos'  
Línea: 'desde'  
Línea: 'Java'
```

Nótese que este programa tiene algunas características extras que es necesario explicar.

En primer lugar, para que el programa funcione requiere unas líneas extras al inicio:

```
1 import java.io.File;  
2 import java.io.IOException;  
3 import java.util.Scanner;  
4  
5 public class Ejemplo1  
6 {  
7     public static void main(String[] args) throws IOException  
8     {  
9         File file = new File("d:/temp/ejemplo.txt");  
10        Scanner lector = new Scanner(file);  
11  
12        while (lector.hasNextLine()) {  
13            String linea = lector.nextLine();  
14            System.out.println("Línea: '" + linea + "'");  
15        }  
16  
17        lector.close();  
18    }  
19 }
```

Dichas líneas que comienzan con `import` le indican al compilador de `Java` que se hará uso de funciones que no estarán definidas en este archivo, sino que vienen incluidas en el entorno `Java`. En particular, se utilizará un `Scanner` para leer desde un archivo, se usará `File` para poder referenciar el archivo que queremos leer, y se usará `IOException` en caso de que la apertura del archivo genere algún error (por ejemplo, que el archivo no exista).

Si estas líneas no se incluyen, el programa no compilará. Pero generalmente las IDEs modernas detectan el uso de estas características y pueden incluir los `import` de manera automática.

La forma en que este programa funciona es que en las líneas 9 y 10 se abre el archivo, conectado a un `Scanner` que nos permitirá leer desde el archivo. Posteriormente, lo que se hará es leer una a una las líneas del archivo, hasta que se acaben. Ésto se logra a través del ciclo `while` de la línea 12, que básicamente repetirá dicho ciclo mientras aun existan líneas que leer desde el archivo. En cada iteración del ciclo, se leerá la siguiente línea, y en este caso, se hace un `print` a la pantalla de la línea recién leída. Finalmente, en la línea 17 se cierra el `Scanner` para liberar recursos de memoria.

Nótese que en este caso, la intención era leer todas las líneas del archivo. Puede que existan otras situaciones en que la lectura se debe controlar de otra forma. Por ejemplo, si tenemos este archivo:

```
3  
2  
Hola
```

```
Mundo
3
Perro
Gato
Conejo
2
Pera
Manzana
```

Su formato es evidentemente diferente al anterior. Analiza el texto hasta que descubras cuál es el formato.

Como el formato es distinto, la estrategia para leerlo será también diferente:

```
1 import java.io.File;
2 import java.io.IOException;
3 import java.util.Scanner;
4
5 public class Ejemplo1
6 {
7     public static void main(String[] args) throws IOException
8     {
9         File file = new File("d:/temp/ejemplo.txt");
10        Scanner lector = new Scanner(file);
11
12        int num = Integer.valueOf(lector.nextLine());
13
14        for(int i=0;i<num;i++) {
15            int cant = Integer.valueOf(lector.nextLine());
16
17            for(int j=0;j<cant;j++) {
18                String linea = lector.nextLine();
19                System.out.println("Línea: " + linea + "'");
20            }
21        }
22
23        lector.close();
24    }
25 }
```

En las líneas 12 y 15 se usa `Integer.valueOf(...)` para convertir un `String` a `int`.

Las posibilidades de diferentes formatos de archivos es prácticamente infinita. Solamente se requiere diseñar correctamente el algoritmo de lectura, procesamiento y almacenamiento de los datos.

Algo que es necesario destacar, es que la ruta al archivo que se quiere abrir (en este caso, en la línea 9), debe incluir la extensión del archivo, y a menos que el archivo se encuentre presente en el mismo directorio donde se está ejecutando el programa, se debe especificar su ruta. En los sistemas operativos Windows, la forma más fácil de hacer esto es usando `/` tal como aparece en el ejemplo (aun cuando nativamente en Windows el separador de ruta es un `\`). En los sistemas operativos basados en Unix, se usa `/` de forma normal.

1.18.1. Formato de las líneas

Algo que es necesario tomar en cuenta al procesar las líneas de los archivos es el formato de cada una de ellas, ya que es posible que cada una contenga información relacionada que es necesario separar para que sea de utilidad. Por ejemplo, si tenemos este archivo:

```
ximena,43
mario,33
juan,41
cristina,39
```

Podríamos inmediatamente interpretarlo como que en cada línea hay información de una persona, y que en la misma línea está escrito el nombre de la persona y su edad (o talla de zapato, o cualquier otro dato).

Para poder usar el contenido del archivo, es necesario leer cada línea, y aplicarle un proceso extra a cada una:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args) throws IOException
4     {
5         File file = new File("d:/temp/ejemplo.txt");
6         Scanner lector = new Scanner(file);
7
8         while (lector.hasNextLine()) {
9             String linea = lector.nextLine();
10            String[] partes = linea.split(",");
11
12            String nombre = partes[0];
13            int edad = Integer.valueOf(partes[1]);
14
15            System.out.println("La persona " + nombre + " tiene " + edad);
16        }
17
18        lector.close();
19    }
20 }
```

En este caso, lo que se quiere lograr es *dividir* cada línea leída, separando usando la coma, para después poder usar los diferentes componentes para lo que sea necesario.

1.19. Ejercicios

1. Construye un programa para explicitar lo que sucede en otros casos de overflowing y underflowing. Por ejemplo, con tipos como `byte`, `short`, etc.
2. Construye un programa para ver qué pasa con el overflowing y underflowing de los tipos `float` y `double`.
3. Construye un programa para analizar otros casos de casting explícito. Por ejemplo, `int`→`short`.
4. Ejecute este código mentalmente, y después ejecútelo realmente para verificar que entiende cómo funcionan los `if` en `Java`.

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int dato = 1;
6
7         if (dato == 1) {
8             System.out.println("1");
9         }
10        if (dato == 2) {
11            System.out.println("2");
12        }
13        if (dato == 3) {
14            System.out.println("1");
15        } else {
16            System.out.println("Ni 1,ni 2,ni 3");
17        }
18    }
19 }
```

5. Ejecute este código mentalmente, y después ejecútelo realmente para verificar que entiende cómo funcionan los `if` en `Java`.

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int dato = 1;
6
7         if (dato == 1) {
8             System.out.println("1");
9         } else if (dato == 2) {
10            System.out.println("2");
11        } else if (dato == 3) {
12            System.out.println("3");
13        } else {
14            System.out.println("Ni 1,ni 2,ni 3");
15        }
16    }
17 }
```


6. Ejecute este código para determinar el comportamiento del código en este caso:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int dato = 33;
6
7         switch(dato) {
8             case 1:
9                 System.out.println("uno");
10                break;
11             case 2:
12                 System.out.println("dos");
13                 break;
14             case 3:
15                 System.out.println("tres");
16                 break;
17             case 4:
18                 System.out.println("cuatro");
19                 break;
20         }
21     }
22 }
```

7. Ejecute este código para determinar su comportamiento en este caso:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int dato = 2;
6
7         switch(dato) {
8             case 1:
9                 System.out.println("uno");
10            case 2:
11                System.out.println("dos");
12            case 3:
13                System.out.println("tres");
14            case 4:
15                System.out.println("cuatro");
16            default:
17                System.out.println("nada");
18        }
19    }
20 }
```

8. Realiza pruebas para ver si existe alguna diferencia al escribir el ciclo for de estas dos maneras:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         for(int i=0;i<10;i++) {
6
7         }
8         for(int i=0;i<10;++i) {
9
10        }
11    }
12 }
```

9. De la misma forma que existe el operador ++, también existe el --. Investiga qué hace este código:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int q = 4;
6
7         while(--q > 0) {
8             System.out.println(q);
9         }
10
11        q = 4;
12
13        while(q-- > 0) {
14            System.out.println(q);
15        }
16    }
17 }
```

10. Construya un programa que lea desde el teclado una serie de datos que vienen organizados de la siguiente forma:

```
3
2
hola
amigos
4
letras
muchas
letras
muchas
2
me
despido
```

11. Construya un programa que lea una serie de calificaciones desde el teclado, e indique el promedio y la cantidad de dichas calificaciones que fue superior a dicho promedio:

```
Ingrese la cantidad de notas: 4
Ingrese la nota 1: 3.7
Ingrese la nota 2: 5.2
Ingrese la nota 3: 1.8
Ingrese la nota 4: 6.7
El promedio es 4.3500000000000005
Se ingresaron 2 notas superiores al promedio
```

12. Corrige este código para que la salida por pantalla correctamente no contenga un , extra al final.

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[] vector = new int[4];
6
7         for(int i=0;i<4;i++) {
8             vector[i] = i + 2;
9         }
10
11         // Vamos a imprimir los valores en el vector
12
13         for(int i=0;i<4;i++) {
14             System.out.print(vector[i] + ", ");
15         }
16         System.out.println();
17     }
18 }

```

13. Modifica este código, de forma que al escribir el contenido de la matriz, las columnas queden alineadas. Fíjate que debes revisar cada columna, determinar su ancho adecuado, y al escribir cada valor agregar la cantidad de espacios suficientes para que todo quede alineado, considerando el tamaño del valor que quieres escribir.

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int[][] matriz = new int[4][5];
6
7         matriz[2][1] = 30;
8         matriz[2][2] = 32;
9         matriz[0][0] = 36;
10        matriz[3][4] = 38;
11
12
13        for(int f=0;f<4;f++) {
14            for(int c=0;c<5;c++) {
15                System.out.print(matriz[f][c] + " ");
16            }
17            System.out.println();
18        }
19    }
20 }

```

14. Construye un programa que pregunte un número N, y después permita ingresar N números enteros. Posteriormente, haz que el programa pregunte un número Q, y que finalmente informe si el número Q estaba o no dentro de los N números ingresados.

En este caso, guarda los números en un vector, en el mismo orden en que se van ingresando. Y después para saber si Q está presente, busca en forma secuencial en el vector.

15. Construye un programa que tenga el mismo comportamiento que el programa anterior, pero que ahora, en vez de ir guardando los números en el mismo orden en que se van ingresando, haz que los guardes de forma ordenada, siempre manteniendo el vector ordenado de menor a mayor.

Después, cuando sea el momento de buscar Q en el vector, ten en cuenta que si al analizar el elemento en la posición $[i]$, si éste es menor que Q , ya puedes dejar de buscar, porque al estar los datos ordenados, no hay posibilidad de que se encuentre en lo que resta del vector.

16. Construye un programa que haga lo mismo que el programa anterior, incluyendo el proceso de guardar los elementos en orden.

Pero después, cuando sea el momento de buscar Q en el vector, haz lo siguiente:

- a) Asigna una variable i con cero (representa la primera posición del vector)
- b) Asigna una variable j con $N-1$ (representa la última posición del vector)
- c) Asigna una variable k con $\frac{i+j}{2}$ (representa la posición central del vector)
- d) Compara el valor de Q con el valor de vector $[k]$
- e) Si Q es mayor que vector $[k]$, entonces asigna $i = k + 1$
- f) Si Q es menor que vector $[k]$, entonces asigna $j = k - 1$
- g) Si Q es igual que vector $[k]$, termina el proceso porque lo encontraste
- h) Si i es mayor que j , termina el proceso porque no lo encontraste
- i) Vuelve al punto (c)

Este proceso de búsqueda se llama *búsqueda binaria*, ya que en cada iteración elimina a la mitad de los candidatos. Es un proceso mucho más rápido de búsqueda que la búsqueda secuencial del problema 15, pero requiere que el vector esté ordenado.

17. Se necesita un programa que controle los proyectos de una empresa dedicada al rubro de la minería. La empresa almacena los proyectos en ejecución y los nombres de los integrantes del equipo de trabajo en un archivo llamado "proyectos.txt". Este archivo tiene en cada línea: el código del proyecto (número correlativo que empieza en 0), el nombre de un integrante del proyecto y porcentaje de dedicación al proyecto (número entero). Debe considerar que los códigos de proyectos se pueden repetir dado que un proyecto puede tener varios integrantes, o sea, que una persona puede estar trabajando en más de un proyecto. Los proyectos en el archivo vienen en cualquier orden.

Un ejemplo del archivo "proyectos.txt":

```
0,daniel contreras,25
2,jorge rivera,30
1,daniel contreras,20
0,jorge rivera,40
2,daniel contreras,25
```

En el proyecto código=0, trabajan daniel contreras con un 25 % de dedicación de su tiempo y jorge rivera con un 40 %.

Una vez terminada la lectura de los datos, se debe desplegar:

- a) El nombre del integrante que se encuentra más ocupado, esto es el que tiene el mayor porcentaje de su tiempo ocupado. Considere que hay un solo mayor.

- b) La cantidad de proyectos.
- c) La cantidad de integrantes.
- d) El promedio de integrantes por proyecto, por ejemplo, un valor de 4 significa que en promedio en cada proyecto trabajan 4 personas.
- e) El porcentaje de proyectos que tienen entre 10 y 15 integrantes.
- f) El porcentaje de dedicación a cada uno de los proyectos de una persona cuyo nombre se lee desde pantalla. Debe continuar preguntando nombres hasta que se ingrese el nombre FIN.

Considere:

- El archivo no tiene datos erróneos. O sea, no debe preocuparse de controlar que las líneas tengan la cantidad de campos esperados, ni que los porcentajes estén correctos.
- Un proyecto tiene a lo más 15 integrantes.
- Hay a lo más 50 proyectos y 200 empleados

Se pide:

- a) Dibuje la(s) estructura(s) de datos a utilizar, junto con una breve descripción.
- b) Dibuje las estructura del programa
- c) Código `Java` de su aplicación.

18. Completa este código, de forma que lo que `imprimir(...)` escribe a la pantalla tenga el mismo formato que `Arrays.toString(...)`.¹²

```
1 import java.util.Arrays;
2
3 public class Print
4 {
5     public static void main(String[] args)
6     {
7         int[] vector = {5,8,5,3,78,44,11,8,5,67};
8
9         System.out.println(Arrays.toString(vector));
10
11         imprimir(vector);
12     }
13
14     private static void imprimir(int[] vector)
15     {
16         // COMPLETAR
17     }
18 }
```

¹²Desde ahora en adelante si necesitas imprimir por pantalla un vector, puedes usar `Arrays.toString(...)`.

2

Introducción a la Ingeniería de Software

2.1. Introducción

Hasta ahora, todos los programas que hemos creado no son más que eso: programas que realizan una función, y cuyo tiempo de vida no sobrepasa un par de días (o tal vez semanas, pero no más allá de eso).

En el mundo real, la construcción de software sigue siendo un trabajo artesanal, pero muchos de los productos desarrollados tienen una expectativa de vida que se mide en años. Bajo esa visión, la pregunta es si podemos seguir haciendo software de la misma forma en que lo hemos estado haciendo, o podemos hacerlo mejor para que nuestro software tenga una buena vida.

Lo primero que tenemos que considerar, es que si hablamos de software que durará mucho tiempo siendo utilizado, seguramente requerirá de modificaciones para adaptarlo a los cambios del ambiente, introducir nuevas características o simplemente corregir problemas producto del desarrollo del software en sí. Pero para hacer esas modificaciones, las personas encargadas necesitan que el software esté bien escrito, de forma que las modificaciones se puedan realizar bien y rápido.

Es por eso que cuando hacemos programas en un alcance mayor, se dice que necesitamos realizar una buena “ingeniería de software” del producto. Esto quiere decir que no solo nos preocuparemos de escribir el programa, de cualquier forma, sino que nos preocuparemos de que el software tenga una alta calidad, que cumpla lo que se le pide, que tenga las características necesarias para poder ser “mantenido” (o sea, modificado y corregido), y que lo acompañe toda la documentación necesaria para que se mantenga funcionando todo el tiempo debido.

Es muy normal que a la vez que creamos nuestro software, tengamos que crear documentos, tales como:

- Manuales de usuario
- Manuales de administrador

- Manuales de desarrollo, sistema, etc.
- Diagramas de diseño
- etc.

Además de lo anterior, se puede considerar que el código bien comentado y entendible es documentación también.

Por todo lo anterior, muchas veces se dice que

$$\text{Software} = \text{Programas} + \text{Documentación}$$

En resumen, la creación de software es una actividad muy costosa, artesanal, muchas veces entregado con retraso y sobre el presupuesto.

2.2. Ingeniería de Software

Si pudiéramos definir lo que es la IdS¹, es simplemente *el estudio de los principios y las metodologías para el desarrollo de sistemas de software*.

En este sentido, se trata de buscar la forma de aplicar enfoques sistemáticos, que sean disciplinados y medibles, tanto al desarrollo como al mantenimiento de los sistemas de software. Se trata de administrar de forma ordenada todo el ciclo de vida de los productos de software.

De forma muy general, la forma clásica de enfrentar la administración de este ciclo de vida es dividirlo en los siguientes grandes elementos:

Diagnóstico En este punto simplemente diagnosticamos la situación, para tener una idea de cuál es el problema que requiere un sistema de software para ser solucionado.

Factibilidad Es necesario evaluar si el sistema de software es realmente la solución al problema, e incluso, si es que se decide que el problema se soluciona con un sistema de software, revisar si es factible de implementar, dadas las restricciones del entorno (tiempo, recursos, calidad, etc.).

Análisis En este punto se revisa lo más detalladamente posible el problema para determinar con la mayor cantidad de certeza posible qué es lo que se quiere realizar.

Diseño Se debe tener una idea clara de cómo se realizará el software. Para cada requerimiento encontrado, se debería saber cómo se va a implementar.

Implementación En este punto el sistema realmente se implementa. Se procede a codificar el sistema de software, y aplicar las actividades necesarias para asegurarse de que se está realizando de la forma correcta, de que entrega los resultados correctos y en general, cumple los requerimientos y las expectativas de los clientes. En este punto también se realizan pruebas al software (pruebas unitarias, de integración, de aceptación, etc.) para asegurar la calidad de lo creado.

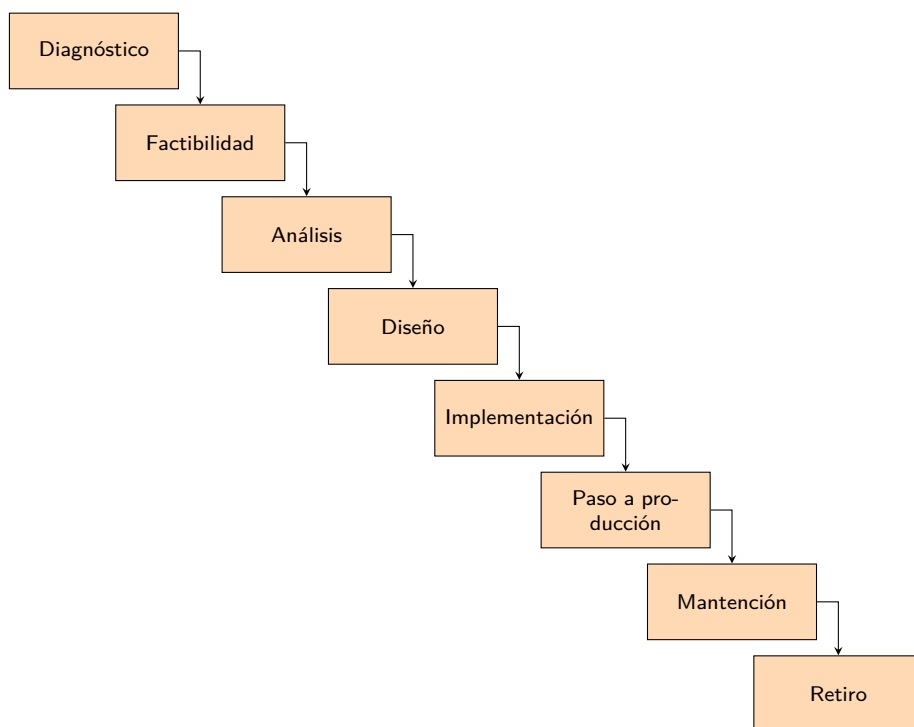
¹Ingeniería de Software

Paso a producción Una vez creado el software, éste se “instala”, para que sea utilizado por el cliente.

Mantenición Esta etapa es quizás la etapa más larga, y corresponde a mantener el software funcionando en el largo plazo. Seguramente en este punto se incluirán tareas de soporte, tanto correctivo, como perfectivo y adaptativo. Cada tipo de soporte permite que el software se mantenga funcionando, hasta que se tome la decisión de retirarlo.

Retiro Este es el punto final de un sistema de software. Por diversas razones (costos, obsolescencia, etc.) se puede tomar la decisión de “apagar” el sistema. En este punto se pueden incluir actividades como la de guardar toda la información almacenada para referencia futura, o realizar tareas de movimiento de los datos a un nuevo sistema que podría estar reemplazando al actual.

Todo lo anterior corresponde a la visión *clásica* de la ingeniería de software, en que el proceso tiene *etapas*, y dichas etapas se realizan de forma secuencial, una después de la otra. A esto generalmente se le llama el “modelo cascada” (sobre todo cuando se ve desde el punto de la implementación del software), ya que cada etapa va “cayendo” hacia la siguiente:



Actualmente, el enfoque utilizado para el desarrollo de productos de software ha cambiado, y aunque las fases inicial y final son similares, los procesos de desarrollo actuales se orientan más a tener *iteraciones* en vez de tener *fases*.

Esto significa que los procesos que están en medio se repiten muchas veces, y el sistema de software se va construyendo de forma incremental y iterativa. En otras palabras:

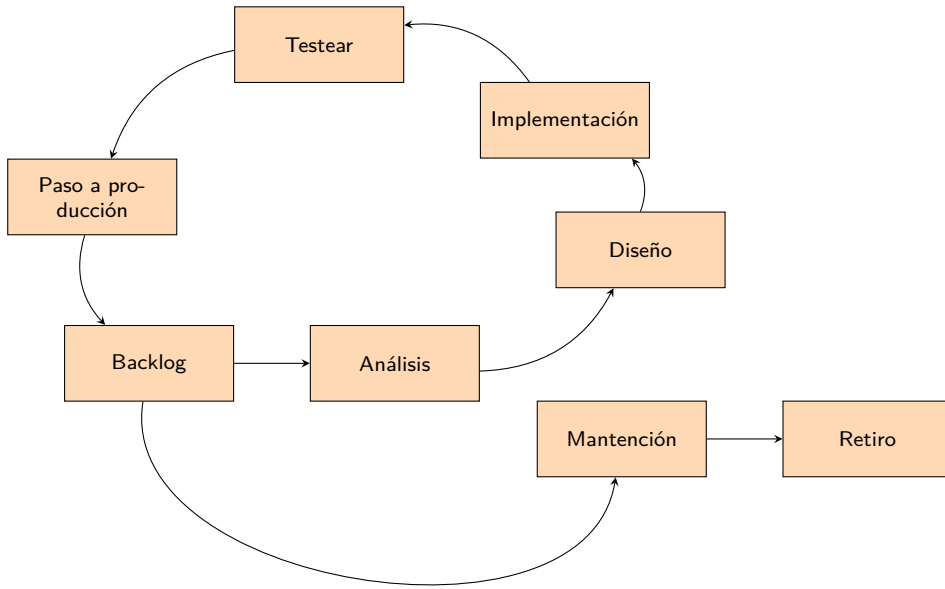


Figura 2.1: Un ciclo iterativo e incremental

Iterativa Se realiza el mismo proceso muchas veces a lo largo del desarrollo del software.

Incremental Poco a poco, en la medida de que se van realizando las actividades iterativas, el software va ganando funcionalidades, hasta llegar a contener todo lo que debe tener en término de las funcionalidades esperadas y así satisfacer a los clientes.

Otra cosa que ha cambiado, es que los equipos de trabajo son diferentes. Normalmente, en el proceso de cascada cada etapa la podía realizar un conjunto diferentes de personas. Es normal que las tareas de “análisis” la desarrollen los “analistas”, mientras que las tareas de desarrollo las realicen ingenieras e ingenieros. Pero actualmente, los equipos de trabajo generalmente son integrales, o sea, es el equipo completo quien se encarga de realizar todas las tareas, y en cada iteración se aseguran de que todo el proceso se cumpla para alcanzar los objetivos.

A esta forma de hacer software se la denomina “ágil”, y se puede resumir así como se ve en la figura 2.1.

Por otro lado, independiente del mecanismo usado para construir el software, posterior a la etapa de paso a producción², la etapa de la mantención y soporte de éste generalmente se clasifica en tres tipos diferentes:

Soporte Correctivo Es simplemente corregir errores. Puede que el software se pasó a producción conteniendo errores conocidos, o que durante su uso se detectaron nuevos. En cualquier caso, este

²Con “producción” se entiende que el software está instalado en el entorno final, donde es usado por los clientes reales.

tipo de soporte tiene la misión de corregir estos errores para que el software funcione correctamente de acuerdo a los requerimientos de los clientes.

Soporte Adaptativo Puede que cuando el software se pasó a producción cumplía todos los requisitos del cliente, pero es normal que las condiciones cambien: cambia el entorno, cambian los procesos, cambian los clientes, y por lo tanto es necesario que el software evolucione y sea modificado para adaptarse a este nuevo entorno diferente al entorno original.

Soporte Perfectivo Parecido al anterior, pero se relaciona con los cambios que se realizan para “perfeccionar” el software debido a necesidades y solicitudes del cliente. También puede incluir cambios para que el software sea, por ejemplo, más rápido, bonito, etc.

2.3. Aseguramiento de la calidad

Independiente de cómo se construya el producto de software, la ingeniería de software debería preocuparse de asegurar que el software se construye con la calidad debida. Y muchas veces, ésto se va a traducir en la construcción y ejecución de *pruebas* que verifiquen si el software entrega los resultados correctos.

La forma en que esto se realiza generalmente es mediante la creación de “casos de prueba”, que se componen de una “entrada” y una “salida esperada”. Normalmente la salida se construye manualmente, a partir del conocimiento de cómo *debería* funcionar el software. Posteriormente el software se ejecuta, se alimenta con la “entrada” y se compara si produce la misma salida que la salida esperada. Si es así, entonces el software superó la prueba. Si no es así, entonces es necesario realizar las correcciones necesarias hasta que la logre superar.

Esto quiere decir que las pruebas que se realicen sobre el software deben construirse con cuidado, ser fieles a la realidad y al uso que se le dará al software, y deben existir en una cantidad tal que permita asegurar que todas las funcionalidades del sistema son cubiertas, tanto en casos de éxito como de fallo.

Por ejemplo, si estuviéramos testeando una pantalla donde un usuario debe escribir su RUT para poder ingresar al sistema, se podrían crear una serie de casos de la siguiente forma para asegurarnos de que el sistema solamente acepta RUTs correctamente escritos:³

³Por si te preguntas qué significa “RUT correctamente escrito”, ésto significa verificar si el RUT fue escrito de forma correcta, utilizando el mismo RUT para averiguarlo. Cada RUT contiene información que permite “autovalidarlo” y poder detectar cuándo la secuencia de dígitos está mal escrita. Esta información está almacenada en la última parte del RUT, o sea, lo que viene después del guión. A esto se le llama el “dígito verificador” y es un valor que se calcula a partir de los elementos restantes del RUT. Un programa puede leer un RUT completo, incluyendo su dígito verificador, re-calcular el dígito verificador y rechazar el ingreso cuando el dígito calculado es diferente al dígito ingresado. Para más detalles del algoritmo con el que se puede calcular el dígito verificador, puedes revisar https://es.wikipedia.org/wiki/Rol_Único_Tributario

Código	Datos de entrada	Salida Esperada	Salida Real
Caso1	19791794-6	RUT válido	
Caso2	24996616-9	RUT válido	
Caso3	19363927-5	RUT válido	
Caso4	24834645-0	RUT válido	
Caso5	25621929-8	RUT válido	
Caso6	12795794-0	RUT válido	
Caso7	22244372-5	RUT válido	
Caso8	15449792-7	RUT válido	
Caso9	13791394-6	RUT no válido	
Caso10	24993636-9	RUT no válido	
Caso11	19363927-5	RUT no válido	
Caso12	24854655-0	RUT no válido	
Caso13	25691919-8	RUT no válido	
Caso14	12791714-0	RUT no válido	
Caso15	22211372-5	RUT no válido	
Caso16	15449192-7	RUT no válido	

A medida que los casos de prueba se van ejecutando, en la columna “Salida Real” se escribe la salida que produjo el software, y se deben tomar las medidas que correspondan cuando no coincida con lo que se esperaba.

2.4. Depuración

Una actividad que todo el mundo realiza al escribir software, es el proceso de depuración, o debugging en inglés. Esta actividad corresponde simplemente a encontrar y resolver los defectos que tenga el software.

Aun cuando muchos de los defectos se pueden encontrar usando algún tipo de test, muchas veces el proceso de encontrar la fuente final de los problemas requiere realizar una serie de pasos que conducirán a descubrir la causa raíz del problema, y su solución.

Dentro de las técnicas más usadas para realizar lo anterior están el análisis de logs, el monitoreo del sistema, los vaciados de memoria, el uso de puntos de quiebre⁴ y el profiling. En Java, si se usa una IDE apropiada, se puede examinar el estado del software funcionando, y analizar el contenido de las variables, aplicar breakpoints de forma condicional, e incluso cambiar el estado del programa de forma dinámica.

En el peor de los casos, la técnica más antigua y usada es simplemente agregar al código, en los puntos claves de sospecha de problemas, instrucciones del tipo `print` para verificar el flujo del programa e incluso el contenido de las variables.

⁴Breakpoints

2.5. Validación de datos

Dependiendo del software que se esté construyendo, muchas veces será necesario verificar y asegurarse que los datos ingresados desde fuentes externas (teclado, archivos, otros) tengan el formato esperado. El hacer esto garantizará que el programa funcionará, y no tendrá un comportamiento incorrecto por estar manipulando un dato que tiene cierto formato, cuando en realidad el dato tiene un formato o contenido totalmente diferente.

Por ejemplo, si el software le pide al usuario que ingrese una fecha, es necesario validar que ésta se ingresó en el formato indicado (por ejemplo, año-mes-día) y que además es una fecha válida (por ejemplo, 2022-01-01). En caso de que el usuario ingrese una fecha inválida (ya sea por formato o por el valor, por ejemplo, 2022-04-35), el software debe tomar las medidas necesarias para recuperarse de ésta situación.

Los datos más comunes que generalmente deben ser verificados para asegurar un formato correcto serán:

- RUTs
- Montos (de acuerdo a los requisitos, tal vez se requieren solo enteros positivos, o valores dentro de un rango, etc.)
- Fechas (en términos de sus componentes, y también en término del rango de valores de cada componente)
- Calificaciones
- Edades
- etc.

En general, es fácil crear un ciclo en Java que permita verificar que el dato ingresado cumpla los requisitos impuestos. En la práctica, podemos tomar uno de dos caminos:

1. Después de leer el dato y verificar que no cumple lo requerido, informar la situación y terminar el programa:

```
1 public class Ejemplo1 {
2     public static void main(String[] args) throws IOException {
3         Scanner lector = new Scanner(System.in);
4
5         System.out.println("Ingrese una edad: ");
6         int edad = Integer.valueOf(lector.nextLine());
7
8         lector.close();
9
10        if (edad < 0 || edad > 200) {
11            System.out.println("Edad inválida. Terminando el programa");
12            return;
13        }
14
15        // Procesamos la edad
16    }
17 }
```

2. Después de leer el dato y verificar que no cumple lo requerido, continuar preguntando hasta que se ingrese un valor válido. En este caso, podemos escribirlo de dos maneras:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args) throws IOException
4     {
5         Scanner lector = new Scanner(System.in);
6
7         System.out.println("Ingrese una edad: ");
8         int edad = Integer.valueOf(lector.nextLine());
9
10        while (edad < 0 || edad > 200) {
11            System.out.println("Edad inválida.");
12            System.out.println("Ingrese una edad: ");
13            edad = Integer.valueOf(lector.nextLine());
14        }
15
16        lector.close();
17        // Procesamos la edad
18    }
19 }
```

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args) throws IOException
4     {
5         Scanner lector = new Scanner(System.in);
6         int edad;
7
8         do {
9             System.out.println("Ingrese una edad: ");
10            edad = Integer.valueOf(lector.nextLine());
11        } while (edad < 0 || edad > 200);
12
13        lector.close();
14
15        // Procesamos la edad
16    }
17 }
```

En este caso, la versión con `do..while` es la más compacta y con menos código repetido.

2.6. Excepciones

Otra forma a través de la cual podemos aumentar la calidad del software, es tomar precauciones contra la aparición de errores en tiempo de ejecución.

Este tipo de errores, al contrario que los errores de compilación, no impiden que el programa se ejecute, pero cuando se generan, a menos que sean manejadas de forma correcta, causarán el término anticipado de éste.

Las excepciones⁵ tienen la característica de que se generan en un punto del código, y a menos que sean controladas, subirán por la secuencia de llamadas de funciones hasta que, o es atajada, o llega al

⁵En inglés, *exceptions*

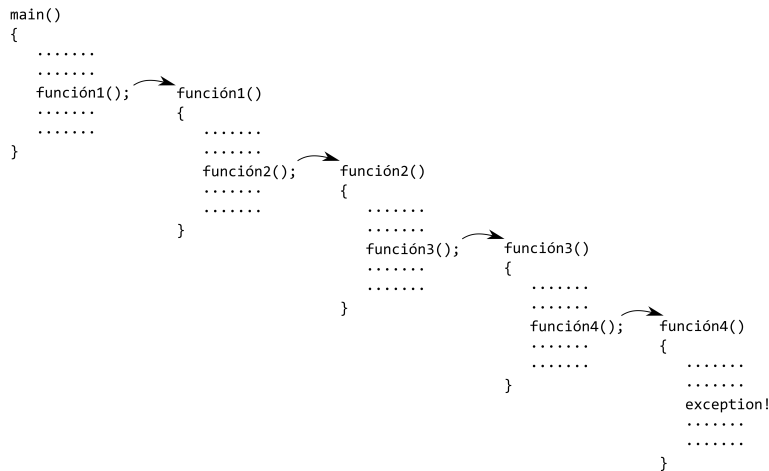


Figura 2.2: Lanzamiento de una excepción

main, sale y termina el programa. Por ejemplo, en la figura 2.2 dentro de la función4 se lanza una excepción. Si la excepción no es atajada causará que el programa termine anticipadamente.

La forma en que las excepciones se pueden atajar es simplemente escribiendo unas nuevas palabras claves del lenguaje para indicar que queremos ejecutar ciertas líneas de código, pero si se detecta una condición de error (una excepción), queremos atajarla.

Este “atajar” una excepción no tiene que suceder necesariamente en el mismo punto donde se lanza la excepción, por ejemplo, en la función4 en la figura 2.2, sino que puede suceder en cualquier parte de la cadena de llamadas que hizo que la ejecución llegara a la función4.

Las palabras claves para atajar excepciones son dos: try y catch:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         try {
6             // Ejecutar código peligroso!
7             // que puede tener
8             // muchas líneas
9
10        } catch (Exception e) {
11            // La ejecución continua acá si es que alguna operación
12            // dentro del try lanzó una excepción.
13        }
14    }
15 }

```

Tal como se ve en la figura 2.3, se puede detallar que la excepción fue atajada al nivel de la función2, aun cuando ésta realmente se lanzó dentro de función4. Así funcionan las excepciones.

En los casos en que una excepción no se ataja, se recibirá un mensaje de error detallando qué excepción no fue controlada. Por ejemplo:

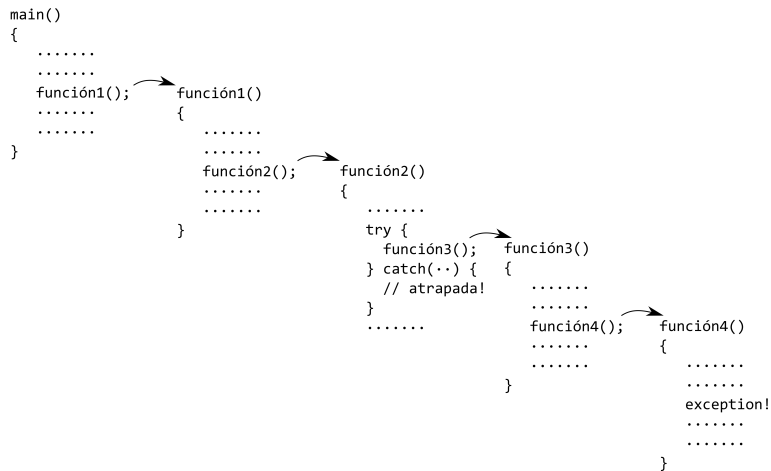


Figura 2.3: Atajando una excepción

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         int a = 2 / 0;
6     }
7 }

```

Al ejecutar el código, se obtendrá la siguiente salida por la consola:

```

Exception in thread "main" java.lang.ArithmeticException: / by zero
at cl.ucn.prograavanzada.ejemplo1.Ejemplo1.main(Ejemplo1.java:5)

```

Siempre que ésto suceda, se mostrará el tipo de excepción (`ArithmeticException`), alguna información extra (`/ by zero`) y la secuencia de llamadas que causó el problema, incluyendo el nombre del archivo y el número de línea donde sucedió.

Por ejemplo:

```

1 public class Ejemplo1 {
2     public static void main(String[] args) {
3         función1();
4     }
5
6     private static void función1() {
7         función2();
8     }
9
10    private static void función2() {
11        función3();
12    }
13
14    private static void función3() {
15        int a = 1 / 0;
16    }
17 }

```


Al ejecutar el código, se obtendrá la siguiente salida por la consola:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
at cl.ucn.prograavanzada.ejemplo1.Ejemplo1.función3(Ejemplo1.java:22)
at cl.ucn.prograavanzada.ejemplo1.Ejemplo1.función2(Ejemplo1.java:17)
at cl.ucn.prograavanzada.ejemplo1.Ejemplo1.función1(Ejemplo1.java:12)
at cl.ucn.prograavanzada.ejemplo1.Ejemplo1.main(Ejemplo1.java:7)
```

Nótese el detalle de la secuencia de llamada. A estos datos se les llama el *stack trace*⁶.

En Java, las instrucciones completas para controlar excepciones son try-catch-finally. Por ejemplo:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         función1();
6     }
7
8     private static void función1()
9     {
10        try {
11            función2();
12            System.out.println("Esta línea no se ejecuta!");
13        } catch (Exception e) {
14            System.out.println("Ha ocurrido un error");
15        } finally {
16            System.out.println("Esto siempre se ejecuta");
17        }
18    }
19
20    private static void función2()
21    {
22        función3();
23    }
24
25    private static void función3()
26    {
27        int a = 1 / 0;
28    }
29 }
```

Al ejecutar este programa, se obtiene lo siguiente:

```
Ha ocurrido un error
Esto siempre se ejecuta
```

En otras palabras, si es que se lanza una excepción, la ejecución se interrumpe, y si hay un catch apropiado, la ejecución continuará en dicho catch. Independiente de si ocurrió o no ocurrió el error, el código dentro del bloque finally siempre se ejecutará.

Un elemento importante para tener en cuenta es que las instrucciones pueden lanzar diferentes tipos de excepciones. En Java se han predefinido una serie de *tipos* de excepciones, para indicar diferentes condiciones de error. Las más comunes son:

ArithmeticException producto de una condición aritmética (ejemplo: división entre cero).

⁶Podrías mirar en https://en.wikipedia.org/wiki/Stack_trace

NumberFormatException al tratar de convertir una cadena de texto con formato incorrecto a número.

NullPointerException al trabajar con objetos nulos. Por ejemplo, modificar los atributos de un objeto nulo.

IndexOutOfBoundsException cuando se trata de acceder a una posición inexistente dentro de un arreglo (por ejemplo, acceder a la posición -1 de un vector, o a la posición N o superior).

IllegalArgumentException se utiliza para indicar que una función ha recibido parámetros incorrectos o “argumentos” inapropiados (por ejemplo, un nombre nulo)

A veces puede suceder que se puede tener un bloque de código, donde básicamente cada línea puede lanzar un tipo de excepción diferente. En este caso podemos escribir algo como lo siguiente:

```
1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         try {
6             función1();
7         } catch (ArithmeticException e) {
8             // Error
9         }
10
11        try {
12            función2();
13        } catch (NullPointerException e) {
14            // Error
15        }
16
17        try {
18            función3();
19        } catch (IndexOutOfBoundsException e) {
20            // Error
21        }
22
23        try {
24            función4();
25        } catch (IllegalArgumentException e) {
26            // Error
27        }
28    }
29 }
```

Pero también, Java nos entrega la posibilidad de escribir lo siguiente:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         try {
6             función1();
7             función2();
8             función3();
9             función4();
10        } catch (ArithmeticException e) {
11            // Error
12        } catch (NullPointerException e) {
13            // Error
14        } catch (IndexOutOfBoundsException e) {
15            // Error
16        } catch (IllegalArgumentException e) {
17            // Error
18        }
19    }
20 }

```

Esto significa que se intentarán ejecutar todas las líneas, y si se lanza una excepción, la ejecución saltará hacia el `catch` correspondiente.

Nótese, que si la excepción lanzada no corresponde a ninguno de los tipos especificados en las sentencias `catch`, la excepción seguirá subiendo, ya que no fue atajada, y podrá potencialmente terminar anticipadamente la ejecución del programa. Además, si al llamar a cualquiera de las funciones anteriores se lanza una excepción, el flujo del programa saltará directamente al `catch` correspondiente, y el resto de las llamadas a `funciónX()` no se realizarán. Por ejemplo, si dentro de `función2()` se lanza una excepción, las llamadas a `función3()` y `función4()` no se realizarán.

2.6.1. Uso de `IllegalArgumentException`

Hasta que se diga lo contrario en este libro, se usará la excepción `IllegalArgumentException` para indicar cuando una función recibe un parámetro con un valor inválido. Por ejemplo, si una función debe recibir una edad, la función verificará que el valor esté dentro del rango válido, y si no es así, debe lanzar una excepción de tipo `IllegalArgumentException`.

Por ejemplo:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         Scanner scanner = new Scanner(System.in);
6
7         System.out.println("Ingrese distancia 1: ");
8         int d1 = Integer.valueOf(scanner.nextLine());
9
10        System.out.println("Ingrese distancia 2: ");
11        int d2 = Integer.valueOf(scanner.nextLine());
12
13        int dt = sumarDistancias(d1, d2);
14    }

```

```

15     System.out.println("Distancia total: " + dt);
16     scanner.close();
17 }
18
19 private static int sumarDistancias(int d1, int d2)
20 {
21     if (d1 < 0)
22         throw new IllegalArgumentException("Distancia 1 no puede ser negativa");
23     if (d2 < 0)
24         throw new IllegalArgumentException("Distancia 2 no puede ser negativa");
25
26     return d1 + d2;
27 }
28 }

```

En este ejemplo, se podría considerar que el ingresar una distancia negativa es un error, por lo tanto, en la función `sumarDistancias` se verifica que los valores pasados en los parámetros efectivamente son valores válidos. Si no lo son, entonces se lanza una excepción. En este caso, el código principal necesita modificarse para tomar en cuenta esta situación:

```

1 public class Ejemplo1
2 {
3     public static void main(String[] args)
4     {
5         Scanner scanner = new Scanner(System.in);
6
7         System.out.println("Ingrese distancia 1: ");
8         int d1 = Integer.valueOf(scanner.nextLine());
9
10        System.out.println("Ingrese distancia 2: ");
11        int d2 = Integer.valueOf(scanner.nextLine());
12
13        try {
14            int dt = sumarDistancias(d1, d2);
15            System.out.println("Distancia total: " + dt);
16        } catch (IllegalArgumentException e) {
17            System.out.println("Error sumando distancias: " + e.getMessage());
18        }
19        scanner.close();
20    }
21
22    private static int sumarDistancias(int d1, int d2)
23    {
24        if (d1 < 0)
25            throw new IllegalArgumentException("Distancia 1 no puede ser negativa");
26        if (d2 < 0)
27            throw new IllegalArgumentException("Distancia 2 no puede ser negativa");
28
29        return d1 + d2;
30    }
31 }

```

Se agregó un `try-catch` para controlar el caso de que la función lance una excepción. En ese caso, se escribe por pantalla un mensaje apropiado. Más adelante se explicará en detalle cómo funciona este proceso, pero por ahora basta con saber que el mensaje que se especifica en la línea 25 o 27, se recupera en la línea 17 para mostrarse por la pantalla en caso de error.

Hay un par de detalles respecto a las excepciones que te pueden interesar en la página 453.

2.7. Ejercicios

1. Si el software es *programas + documentacin*, ¿qué documentación es típica en el desarrollo de software?
2. Piensa en todas las fases que tiene el ciclo de vida tradicional del desarrollo de software ¿Se asemejan a la forma en que hiciste tu último software? (ya sea un taller, laboratorio, etc) ¿En qué se asemejan? ¿En qué difiere?
3. ¿Por qué se le llama “cascada” al ciclo de vida tradicional?
4. ¿Cuál es la diferencia entre el método “cascada” y el método “ágil”?
5. ¿Qué significa iterativo? ¿Qué significa incremental? ¿Cuál es la diferencia?
6. Piensa en los 3 tipos de soporte. ¿Has realizado alguno de ellos?
7. Imagina que estás escribiendo una función a la que le pasas como único parámetro un número entero, y la función tiene que retornar un `boolean` indicando si el número es primo o no. Prepara una tabla con casos de prueba para verificar que tu función opera de manera correcta. ¿Cuántos casos usarías para verificar que realmente funciona?
8. Imagina que estás escribiendo una función a la que le pasas como parámetros una fecha y un entero d , y la función retorna una nueva fecha, que corresponde a la fecha original más la cantidad d de días (o sea, retorna $fecha + d$). ¿Cuántos casos de prueba ejecutarías para verificar que tu función funciona? ¿Consideraste febreros con 29 días? ¿Consideraste que los meses a veces tienen 30 días, y otras veces 31?
9. Crea un programa que pida que se ingrese un RUT, y que lo continúe preguntando hasta que el usuario ingrese un RUT correcto.
10. Modifica el programa anterior para que si el usuario ingresa un RUT incorrecto 3 veces, el programa termine.
11. Crea un programa que de manera deliberada trate de leer “afuera” de un arreglo, pero que atrape la excepción.
12. Crea un programa que sume un arreglo con edades, pero que lance una `IllegalArgumentException` si alguna de las edades es inválida.

3

Introducción a la POO

La programación orientada al objeto (POO) es una forma diferente de enfrentar los problemas, donde en vez de concentrarse en funciones y variables, en encontrar la forma de representar el problema usando listas, matrices, con muchos ciclos y funciones, ahora trataremos de encontrar los *objetos* que mejor representen el problema que se quiere solucionar, usarlos e hacer que se comuniquen entre ellos para resolver la situación.

Pero, ¿qué es un objeto? Un objeto es básicamente algo que tiene sentido en el contexto del problema que estamos resolviendo. Un objeto tiene una conducta bien definida, además de un estado, y una identidad que lo diferencia de otros objetos del mismo tipo.

Al resolver problemas usando objetos, modelaremos una solución y se determinarán los objetos que participarán, además de los mensajes que recibirán y acciones que ejecutarán. Es a través de la interacción entre los objetos que el problema será solucionado.

Es importante en este punto indicar que al trabajar de esta forma, generalmente el problema se comienza a dividir en diversas funcionalidades que trabajan de forma más o menos independiente. Aun más, esas funcionalidades se comunican entre ellas pasándose mensajes, usando lo que se llama su “interfaz pública”. Esto significa que las funcionalidades (los objetos) pondrán a disposición de los otros objetos una serie de mensajes, y si un objeto quiere comunicarse con otro solamente podrá hacerlo enviando alguno de esos mensajes predefinidos. Desde este punto de vista, un objeto no debería nunca ver la funcionalidad *interna* de otro objeto, sino que cuando quiere pedirle algo, le envía un mensaje a través de la interfaz pública que éste publica. La forma en que un objeto lleva a cabo y responde al mensaje que recibe es un “secreto”, y no debería ser conocido por nadie externo al objeto.

El punto anterior es una de las bases de la POO, y se conoce normalmente bajo el nombre de *encapsulación*. Esto quiere decir que los detalles de la implementación de un objeto deben quedar ocultos, y lo único visible debe ser su interfaz pública.

Pero, ¿cómo se define lo que debe representar cada objeto? ¿qué cosas debe contener?

La respuesta a lo anterior puede responderse pensando primero en los conceptos que componen el contexto del problema. Siempre será posible determinar una serie de “cosas” que son las que pertenecen al ámbito de lo que se quiere solucionar. Y esas “cosas”, en el ámbito del problema, se comportarán de cierta manera y tendrán diferentes características. Nótese, que una misma “cosa” puede estar presente

en muchos problemas diferentes, pero dependiendo del problema particular, dicha “cosa” puede tener diferentes características y/o comportamiento. Por ejemplo, cuando hablamos de la cosa “Mascota”, si estamos resolviendo el problema de administrar una Clínica Veterinaria las mascotas tendrán ciertas características (especie, raza, fecha de nacimiento, peso, etc.), pero el mismo objeto mascota tendrá otras características en el caso de estar resolviendo el problema de administrar un Hotel de Mascotas (en este caso, quizás interese más el tamaño de la mascota, si es que es sociable, los detalles de su alimentación, etc).

A esta diferencia la llamaremos *abstracción*, en el sentido de que dependiendo del problema que estemos resolviendo, es la abstracción que usaremos en nuestro sistema. Esto literalmente significa que dependiendo del problema, al incorporar una “cosa” (o sea, un objeto) al software, nos abstraeremos de aquellas características y comportamientos que no son necesarios conocer ni modelar y que no tengan que ver con el problema actual. Si una característica de un objeto no es relevante para el problema, no será necesario incluirla en el modelo.

Esto también significa que dependiendo de la perspectiva desde donde se mire un objeto, éste puede significar cosas diferentes para distintos tipos de actores.

Debido a todo lo dicho anteriormente, es una buena práctica seguir algunos lineamientos al momento de decidir el modelo a aplicar para resolver un problema:

- La abstracción de un objeto debe preceder a las decisiones acerca de su implementación. Esto significa que es más importante que los objetos tengan un diseño “limpio” más que sean fáciles de implementar.
- La implementación de un objeto debe ser un secreto para todos los “clientes” de dicho objeto. Como se dijo antes, desde afuera de un objeto lo único que es visible es la interfaz pública. Pero el cómo un objeto implementa una acción determinada no debe ser conocido por nadie que no sea el objeto en sí mismo.
- Complementando lo anterior, el diseño de la solución debe ser de tal forma que se deben limitar las dependencias directas entre los objetos, y nunca un objeto debe depender de su funcionamiento de los detalles internos de otros objetos.

En resumen, siempre hay que buscar que nuestros diseños tengan dos características clave: **alta cohesión** (hacen lo que saben hacer, y lo hacen bien), y **bajo acoplamiento** (dependen de una cantidad mínima de otros objetos para funcionar).

En Java, la base de toda la abstracción será la *clase*.

3.1. Clases

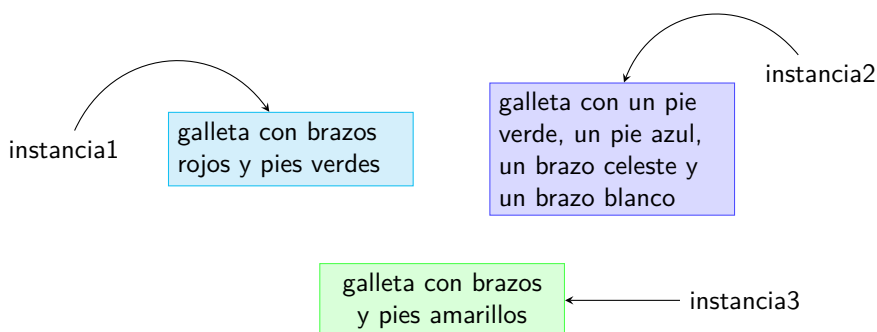
Una clase será para nosotros la fuente de los objetos que usaremos para resolver el problema. Será el equivalente a una “plantilla” a partir de la cual definiremos de forma genérica cómo serán todos los objetos creados a su imagen.

El ejemplo más claro de cómo funciona este mecanismo es el proceso de crear galletas. A partir de un molde, se puede crear un sin número de galletas. El molde define un conjunto de características que todas las galletas creadas van a tener, pero con diferentes “valores”. Por ejemplo, el molde puede definir que las galletas serán de forma humana, con dos piernas y dos brazos, y cada galleta creada tendrá dos piernas y dos brazos, pero cada galleta podrá tener piernas y brazos de diferentes colores. El molde podrá definir la forma general del cuerpo, pero cada galleta podría ser pintada de diferente manera.



Entonces, desde ahora, vamos a decir que una *clase* es una forma de poder definir una estructura y un comportamiento común. A partir de una clase podremos generar *instancias*, que corresponden a los objetos “vivos” creados a partir de la clase. Siguiendo con nuestro ejemplo, el molde de la galleta es la *clase*, y las galletas creadas cada una es una *instancia*.

Entonces, podremos decir que una clase define una serie de atributos y cada instancia guarda valores específicos para dichos atributos. Estos atributos describen de forma cuantitativa las características que tendrá cada instancia. Por ejemplo, el color de los zapatos de cada galleta de la figura. La clase definirá que existe dicho atributo, y cada instancia definirá el color concreto que tiene sus zapatos. Estos valores se guardan dentro de cada instancia, y aunque cada instancia tiene el mismo campo (o sea, la misma variable para guardar el valor), cada instancia se almacena de forma separada, lo que conforma su *estado interno*.

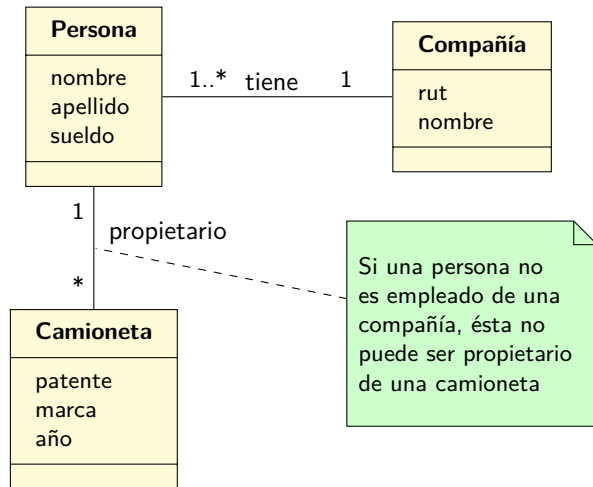


3.2. El modelo del dominio

Durante la fase inicial del desarrollo de software es normal que se empiecen a descubrir los objetos que pertenecerán a la solución. Además, todo ese conocimiento normalmente se documenta de forma de

poder tener una descripción más técnica de la solución, y para poder comunicar lo que se ha descubierto.

La forma más común de documentar el software en esta etapa es usando un **modelo del dominio**. Este modelo presenta una vista simplificada de cómo los objetos se van a comportar e interactuar entre ellos, y permite representar las “cosas” o eventos que transcurren en el entorno del negocio. Por ejemplo:



En lo que resta del libro, usaremos una forma especial de diagramar los conceptos que componen nuestro software. Esta forma se llama **UML**¹, y corresponde a un lenguaje en el área de ingeniería de software que provee las herramientas para visualizar el diseño de los sistemas de forma estándar. En este caso particular, el modelo del dominio se considera una forma simplificada del **diagrama de clases**² que veremos más adelante.

Ahora veremos los elementos que componen el modelo.

3.2.1. Atributos del modelo

Algo que casi siempre se descubre rápidamente son los “atributos” que deben tener los conceptos capturados hasta ahora. Cada atributo permite almacenar el valor que adquiere alguna característica de dicho concepto. Por ejemplo, en el caso anterior, cada **Persona** tendrá `nombre`, `apellido` y `sueldo`, ya que esas son las características que tienen todas las personas en el contexto del problema que se está resolviendo.

Para visualizar esta situación, muchas veces se puede usar un **diagrama de objetos**³, que representa a los objetos “vivos” con los valores que tienen cada uno de sus atributos:

¹Unified Modeling Language

²Este es un diagrama definido en UML

³Este es otro diagrama definido en UML

p1: Persona	p2: Persona	p3: Persona	p4: Persona
nombre="Ximena" apellido="Flores" sueldo=1200000	nombre="Pedro" apellido="Gómez" sueldo=1100000	nombre="Marcia" apellido="Rojo" sueldo=1300000	nombre="Javier" apellido="Meza" sueldo=900000

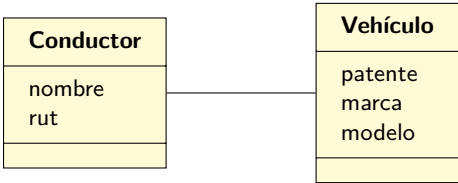
En este punto, interesa descubrir y documentar la existencia de los atributos. No interesa aun determinar de forma exacta el “tipo” de cada atributo (en el sentido del “tipo” de variable).

3.2.2. Multiplicidad

La multiplicidad es algo que nos permite especificar el cómo se relacionan los diferentes conceptos de nuestra solución, de forma de poder capturar información que será relevante más adelante cuando continuemos avanzando en el diseño de nuestro software.

En general, los conceptos que se detectan al analizar el problema no sirven de mucho cuando se consideran por separado, sino que muchas veces su valor dentro de la solución proviene de cómo se relacionan con otros conceptos, ya que representan la realidad del problema que se está solucionando.

Para documentar estos relacionamientos entre conceptos, primero dibujaremos líneas entre ellos. Así estaremos indicando que de alguna u otra forma, los conceptos trabajan juntos:



Como parte del análisis del problema, deberíamos poder detectar la forma concreta en que se produce el relacionamiento, en los siguientes términos:

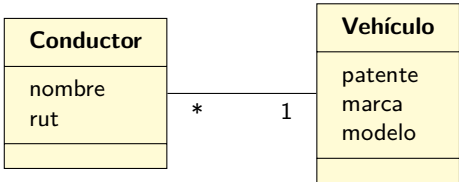
- Si tengo **un** vehículo, ¿con cuántos conductores se relacionará? Dependiendo del problema, puede que un vehículo solo es operado por una sola persona, o puede que lo conduzcan muchos conductores, etc. Esta es una información que es importante determinar correctamente y documentarla.
- Si tengo **un** conductor, ¿con cuántos vehículos se relaciona? De nuevo, dependiendo del problema, puede que durante toda su vida un conductor solo conduzca un vehículo, como también puede ser que tenga muchos asignados, etc.

Habiendo respondido las preguntas anteriores, se deben traspasar las respuestas al diagrama. Para esto, se escriben números (y otros símbolos) en los extremos de la línea. Las opciones disponibles son:

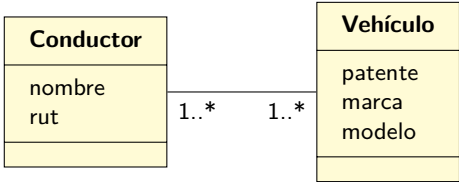
1 (u otro número) Indica un número específico y mandatorio.

- * Indica cualquier número, incluido el cero. A veces también se escribe una N.
- 1..* (o algún otro intervalo) Indica que el relacionamiento puede realizarse con esa cantidad de elementos.
- 1,2,3 Indica que el relacionamiento solo se realizará con la cantidad indicada

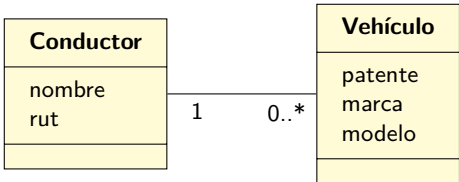
Los símbolos se ubican en la línea, y siempre usando como referencia el concepto que estamos considerando y desde donde estamos haciendo la pregunta. Por ejemplo, si la pregunta “un conductor, ¿cuántos vehículos maneja?” se responde con “1”, dicho “1” se escribirá al extremo de la línea, cerca del concepto Vehículo. De forma equivalente, si nos preguntamos “un vehículo, ¿cuántos conductores tiene asignado?” y respondemos “muchos”, entonces eso lo escribimos en la línea, pero al lado del concepto Conductor:



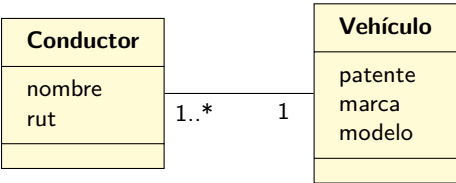
No está de más recordar que aun cuando tengamos los mismos conceptos, la forma en que éstos se relacionan puede cambiar el significado de lo que queremos implementar. Fíjate en lo diferente que es decir: “a un vehículo se le pueden asignar varios conductores, y un conductor está asignado a muchos vehículos”:



en vez de decir “un vehículo siempre tiene asignado a un solo conductor, pero un conductor puede manejar varios vehículos, o también ninguno”:



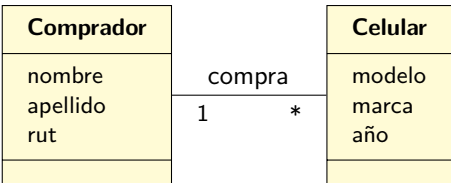
o tal vez decir “a un conductor se le asigna de forma permanente un vehículo, pero el vehículo puede ser compartido por varios conductores, pero siempre va a estar asignado por lo menos a uno”:



Detectar incorrectamente las multiplicidades en el modelo, aunque no es un problema irreparable, finalmente puede causar complicaciones si no se corrigen oportunamente, ya que cada tipo de multiplicidad se implementa de manera diferente.

3.2.3. Relacionamiento entre conceptos

Como se dijo arriba, una de las cosas más importantes que se deben detectar al analizar un problema son los relacionamientos que se generan entre los conceptos que participan. Básicamente, es la asociación que se construye de forma natural en el contexto del problema. Por ejemplo:



En este ejemplo, al leer el diagrama uno puede saber:

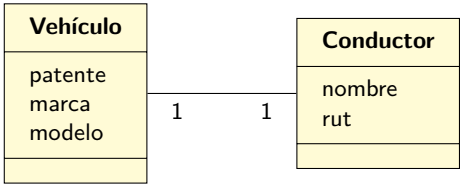
- Un comprador compra cero o más celulares.
- Un celular solamente es comprado por un comprador.

Cada relacionamiento detectado se transformará en una línea que une los conceptos. Optativamente se puede colocar un verbo para facilitar la lectura. En este diagrama, los relacionamientos son bidireccionales, de forma que pueden leerse en ambas direcciones.

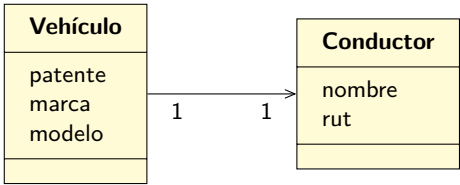
Dependiendo de lo que se detecte respecto al problema que estamos resolviendo, los relacionamientos entre los conceptos pueden ser de diferentes formas (y significar cosas diferentes).

Asociación

Ésta es la forma básica de relacionamiento entre dos conceptos. Éstos conceptos pueden existir independientemente uno del otro y solo se asocian para representar alguna situación en particular del software. Se dibuja con una línea simple.

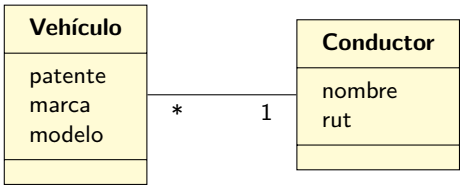


Por defecto, la asociación es bidireccional. Esto significa que el vehículo sabe quién su conductor, y el conductor sabe cuál es su vehículo. Si de acuerdo al problema, se requiere que ésto no sea así, se debe cambiar la línea simple por una flecha:

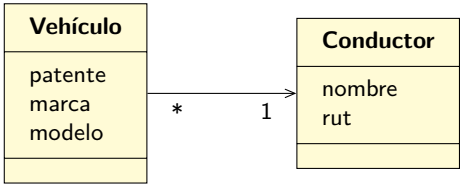


En este caso, el vehículo sabe quién es su conductor, pero el conductor no sabe qué vehículo maneja.

Si lo que se quiere representar es que un conductor puede manejar varios vehículos, el diagrama sería así:



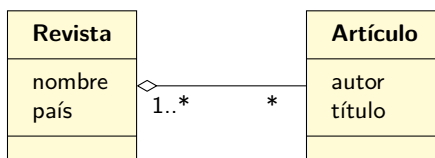
En este caso el conductor sabe cuáles son todos los vehículos que maneja, y cada vehículo sabe quién es su conductor (ya es que la versión bidireccional). Si el problema requiere que no sea así, se puede usar la versión unidireccional:



El vehículo sabe quién es su conductor, pero el conductor no sabe qué vehículos conduce.

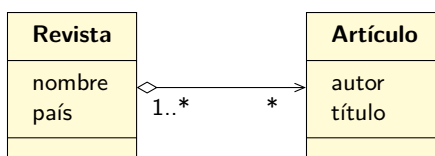
Agregación

A veces sucede que lo que se quiere representar es el relacionamiento entre el “todo” y sus partes:



De acuerdo a este modelo, una revista contiene una colección de artículos, pero además un artículo dado pertenece a lo menos una revista. Este tipo de relacionamiento es más fuerte que la asociación simple, ya que indica algún tipo de pertenencia, aunque no significa que la existencia de un concepto dependa del otro. En el caso anterior, si se borra una revista, no se borrarán los artículos.

Tal como está dibujada, la relación es bidireccional, pero también se puede modelar una versión unidireccional:

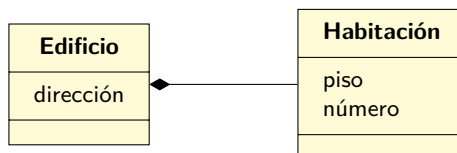


En este caso, una revista contiene una colección de artículos, pero cada artículo no sabe en qué revistas ha sido agregado.

Generalmente la agregación se puede leer como una relación en que un concepto “tiene-un”. Ésto no significa que un elemento sea dueño de otro, o sea, los conceptos pueden existir de forma independiente uno de otros.

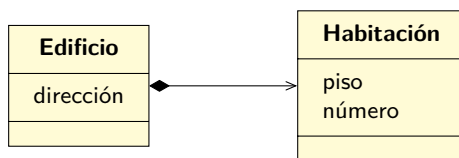
Composición

En UML también se puede modelar un concepto similar al anterior llamado *composición*, pero debido a que no lo usaremos en este libro, solo se mostrará un ejemplo:



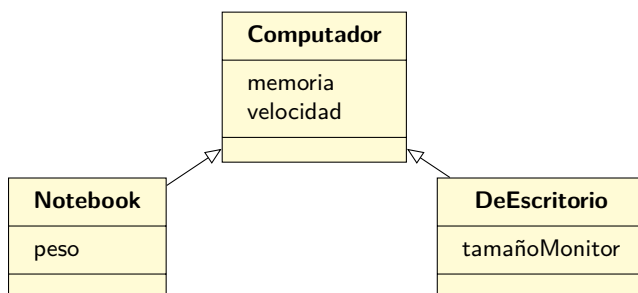
Este relacionamiento se puede leer como que la habitación es “parte-de” un edificio. En este caso, el concepto mayor es dueño del otro, por lo que sus ciclos de vida están unidos (si el concepto mayor desaparece, entonces los conceptos menores también). En el ejemplo anterior, si un edificio desaparece, también deberían desaparecer todas las habitaciones que eran parte de él.

Si se requiere direccionalidad, también se puede especificar de la siguiente forma:

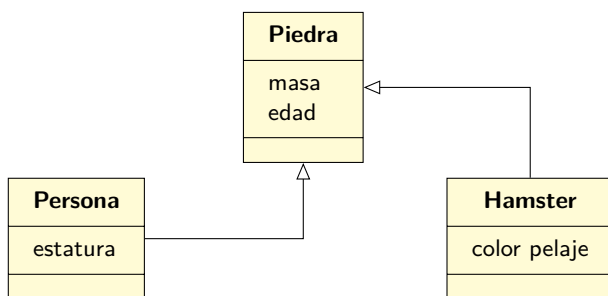


Generalización

A veces los problemas se pueden modelar mejor al considerar que un concepto es un tipo especial de otro concepto. Puede que el concepto derivado tome comportamientos a partir de un comportamiento base, pero que los modifique o haga más complejos o diferentes. En UML se puede diagramar como en este ejemplo:



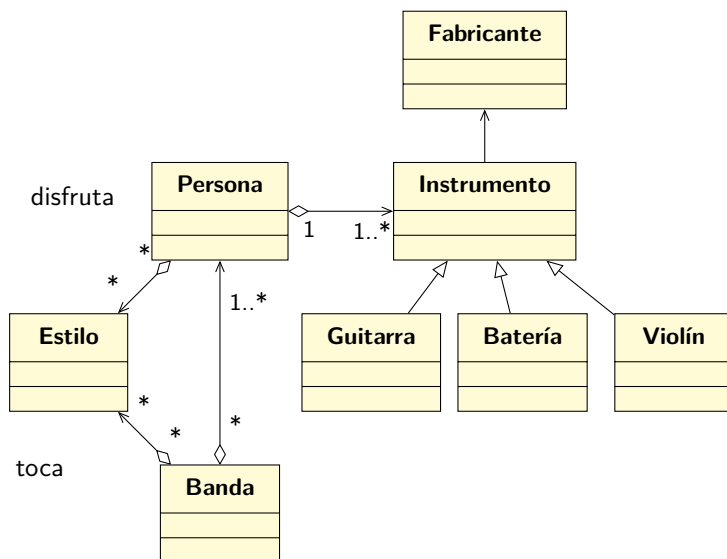
En este caso, podemos considerar que “Notebook” y “DeEscritorio” son tipos particulares de “Computador”. Como tal, obtienen su comportamiento, pero también lo especializan, y en este caso agregan características exclusivas para cada tipo de concepto. Nótese, que el hecho de que varios conceptos tengan características similares, no es razón suficientes para organizarlos en términos de una generalización. Por ejemplo:



En este caso, si nos guiamos solo por las características, podríamos llegar a pensar que este es un modelo válido. Aun cuando puede que existan situaciones en que un hamster es un tipo especial de piedra (y lo mismo para las personas), lo más seguro es que el modelo no está representando el dominio del problema en forma correcta, aun cuando un hamster tenga masa y edad, tal como una piedra.

Esta forma de relación también se puede leer como “es-una”. Más adelante a este tipo de relacionamiento le llamaremos *herencia*.

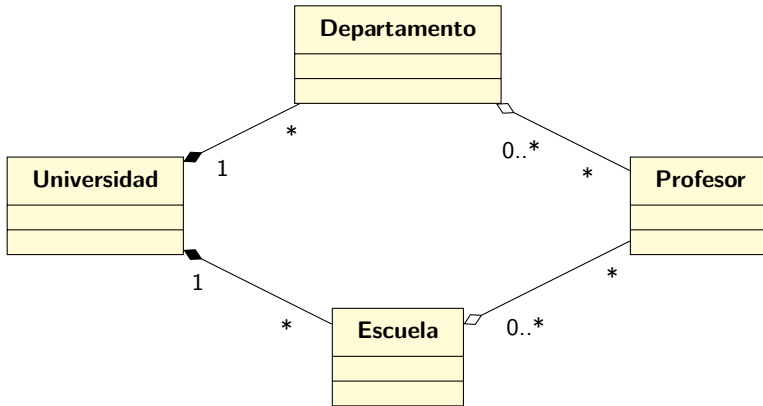
Por ejemplo:



En este caso, lo que se puede leer a partir del diagrama es:

- Existen 3 tipos diferentes de instrumentos.
- Cada instrumento sabe quién es su fabricante, pero un fabricante no sabe qué instrumentos ha fabricado.
- Una persona sabe tocar varios instrumentos musicales (por lo menos 1), pero cada instrumento no sabe quién lo toca.
- A una persona le gustan muchos estilos de música, pero puede que no le guste ninguno.
- Una banda se compone de varias personas
- Una banda toca música de varios estilos, pero puede que no tenga ningún estilo determinado.

Otro ejemplo:



Lo que se puede leer a partir del diagrama es:

- Una universidad es una composición de muchos departamentos y muchas escuelas (cero o más).
- Al ser una composición, si se elimina una universidad, también se deberían eliminar los departamentos y escuelas relacionados.
- Hay profesores que trabajan en departamentos y escuelas.
- En un departamento/escuela trabajan cero o más profesores.
- Cada profesor puede trabajar en cero o más departamentos, y en cero o más escuelas.
- Entre departamento/escuela y profesor hay agregaciones, así que si un departamento o escuela se elimina, los profesores relacionados no se eliminan.
- Todos los relacionamientos son bidireccionales

3.2.4. Recomendaciones

Al crear un modelo del dominio, es bueno seguir algunos lineamientos básicos:

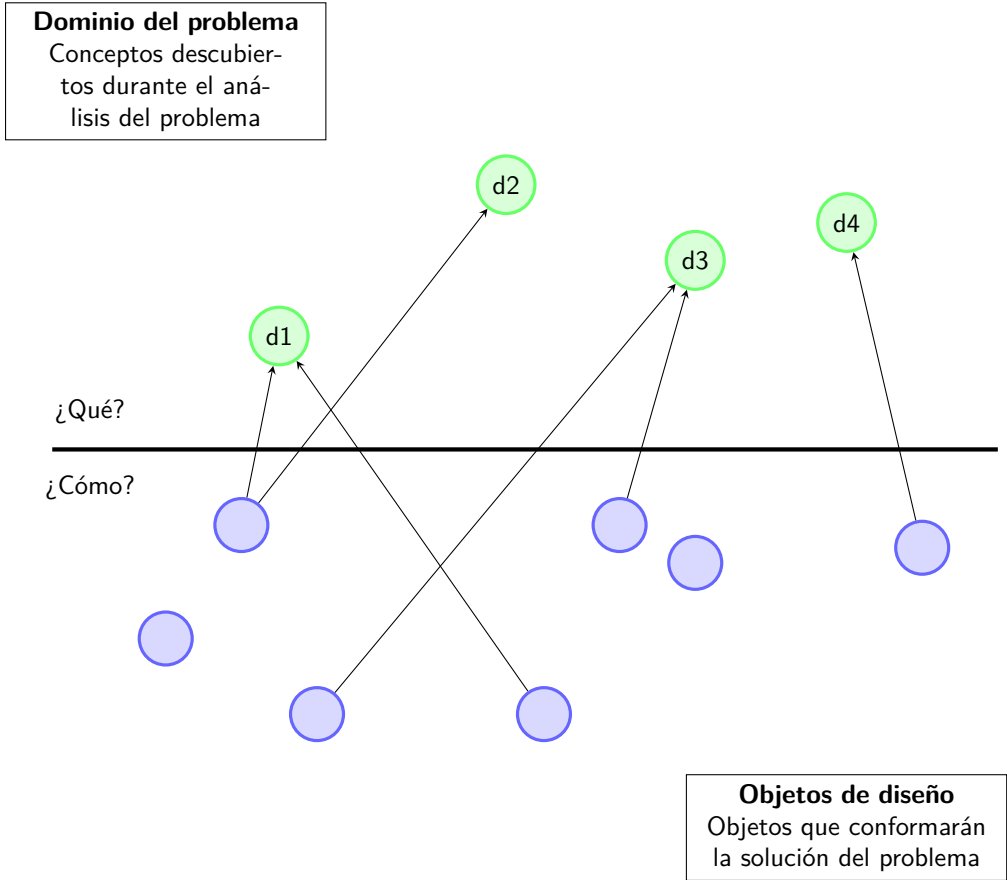
1. A partir del problema, hacer una lista con los conceptos que aparecen mencionados. Estos conceptos no necesariamente corresponden a “cosas”, sino que a veces pueden incluso incorporarse elementos que representen situaciones o eventos que suceden en dicho problema.
2. Es una buena idea dibujar los conceptos ya que facilita la realización del siguiente punto.
3. Se deben identificar las asociaciones entre los conceptos, detallando el tipo de cada una. El hecho de que algo sea una “agregación” o una “asociación direccional” implica consecuencias diferentes, por lo que es importante determinar de forma correcta el tipo de estas asociaciones, y agregarlas al diagrama del modelo.
4. Incorporar los atributos necesarios para que el modelo funcione. Se debe tener cuidado al identificar aquellas características que pueden ser representadas por números o textos, y que tienen sentido en el problema que se está resolviendo.

- 5. A pesar de lo dicho en el punto 3, no hay que estresarse respecto al tipo específico de “diamante” que se debe escribir. Como dice Craig Larman [3]: “En caso de duda, déjalo afuera”. En la realidad, al momento de implementar los relacionamientos en código, hay poca diferencia entre asociación, agregación y composición.

3.3. Clases II

Durante nuestro análisis, que tuvo como resultado la creación de un **modelo del dominio** de la solución del problema, seguramente se detectaron y diagramaron una serie de “conceptos”, que mediante su interacción lograrán formar la solución al problema.

Pero para poder seguir avanzando rumbo a crear un software, se requiere continuar el proceso, y descubrir cómo implementar los conceptos anteriormente mencionados. Es totalmente normal que al diseñar nuestra solución, aparezcan nuevos elementos, o también que elementos desaparezcan o se combinen con otros para formar nuevos constructos.



Una vez descubiertos los objetos que pertenecen al diseño de la solución, al igual en que el caso del modelo del dominio, necesitamos documentar lo encontrado. Para ésto, se usa el **diagrama de clases**,

que utiliza los mismos elementos ya vistos con anterioridad, pero llegando a un nivel de profundidad mayor y describiendo técnicamente las clases, para que después se puedan implementar en el lenguaje de programación final.

En este caso, comenzaremos a dibujar cada “concepto” detectado que se transformará en clase, con todos los detalles que sean razonables en esta etapa. Es más que seguro que durante el desarrollo del software se construyan varias versiones de este diagrama, así que no es necesario que se construya de forma 100 % correcta desde el principio. A medida que el entendimiento del problema y la estructura de la solución se vayan conociendo, el diagrama de clases evolucionará de forma acorde.

En particular, en este momento, además de capturar todos los atributos, agregaremos las operaciones⁴ que contendrá cada clase. Un ejemplo de una clase con mayor contenido sería:

Persona
-rut: String -nombre: String -dirección: String
+Persona(rut:String, nombre:String) +getRut():String +getNombre():String +getDirección():String

En UML, las clases se diagraman usando un rectángulo que tiene 3 secciones: la superior contiene el nombre de la clase (además de información extra que se verá más adelante), la parte central contiene los atributos de la clase, y la parte inferior contiene sus operaciones.

Tanto los atributos como las operaciones deben tener una indicación que determine su visibilidad. Ésto se especifica escribiendo un símbolo antes del atributo u operación:

Símbolo	Nombre	Descripción
+	Pública	El elemento es visible a todos los objetos del sistema
-	Privada	El elemento solamente es visible a las instancias de la misma clase donde está definido el atributo u operación
#	Protegido	El elemento solo es visible para la clase y sus subclases ^a
~	Package	El elemento solo es visible para las clases del mismo paquete ^a

^a Más adelante se revisará este concepto en detalle.

Para el caso anterior, la clase Persona tiene tres atributos, ambos privados, y sus cuatro operaciones son públicas (no te preocupes por ahora de la primera operación; la veremos en detalle más adelante).

Además de especificar la visibilidad, tanto los atributos como las operaciones deben indicar el tipo de dato sobre los que operan. En el caso de los atributos, éstos se especifican escribiendo:

⁴Un poco más adelante se describirá lo que es una “operación”.

visibilidad nombreAtributo: tipo

En general, trataremos de usar los tipos de datos con que se implementarán finalmente las clases, o sea, usaremos los tipos que provee Java⁵. En el caso de Persona, todos los atributos son de tipo String.

Por otro lado, las operaciones (excepto la primera que aparece en el diagrama, ya que es especial) se escriben de la forma

visibilidad nombreOperación(parámetros): tipoRetorno

Los parámetros, si es que la operación recibe alguno, se escriben separados por coma, y cada uno se especifica con el nombre del parámetro y su tipo, similar a cómo se escriben los atributos: nombreParámetro: tipo. El tipo de retorno indica el tipo de valores que retornará la operación. Si es que la operación no retorna valores, entonces no es necesario definir esta parte:

Persona
-rut: String -nombre: String -dirección: String
+Persona(rut:String, nombre:String) +getRut():String +getNombre():String +getDirección():String +agregarDescendiente(nombre:String)

En este caso, hemos agregado una nueva operación: agregarDescendiente(..) es un mensaje que se le envía a una instancia cuando se necesita agregar a esa instancia un descendiente. La operación no retorna ningún valor ya que se asume que siempre funcionará correctamente.

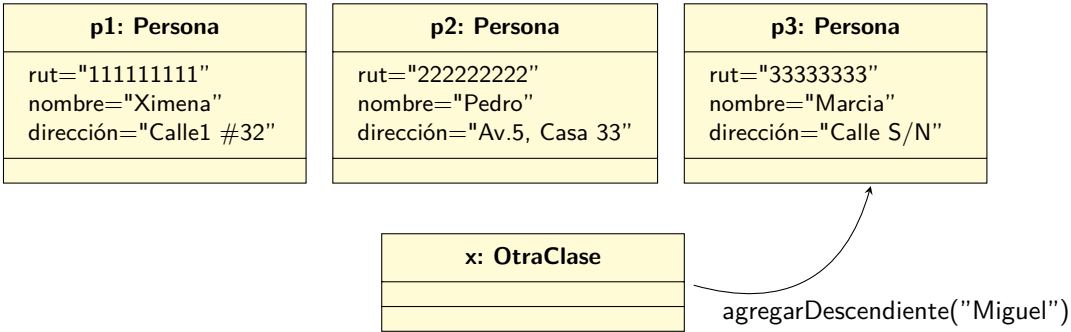
3.3.1. Mensajes a las instancias

Todas las operaciones definidas en una clase podrán ser invocadas (o sea, se podrá enviar un mensaje a la instancia), pero es importante recordar que a partir de la clase, seguramente se crearán múltiples instancias de ésta, cada una con su propio estado interno:

p1: Persona	p2: Persona	p3: Persona
rut="11111111" nombre="Ximena" dirección="Calle1 #32"	rut="22222222" nombre="Pedro" dirección="Av.5, Casa 33"	rut="33333333" nombre="Marcia" dirección="Calle S/N"

y cada instancia tiene la posibilidad de recibir el mismo tipo de mensaje, lo que causará que la instancia en particular que recibe el mensaje cambie su estado interno:

⁵Eventualmente aprenderemos que no solo podemos usar los tipos predefinidos en Java, sino que eventualmente toda clase es un tipo.



En este caso, estamos hablando de un mensaje a la instancia, o en otras palabras, la instancia está recibiendo un mensaje.

3.3.2. Mensajes a las clases

Por otro lado, también es válido y es posible enviar mensajes a las clases en sí (no a las instancias construidas a partir de dicha clase). En este caso, al enviar el mensaje se cambia el estado de dicho objeto (la clase en sí se puede considerar como un objeto en ese sentido), pero se tiene la particularidad de que dicho estado es visible a todas las instancias creadas a partir de la clase.

Y, ¿cómo se guarda ese estado? Usando atributos de la clase. Éstos serán atributos que tendrán la característica de ser visibles a todas las instancias creadas. Además, si el valor del atributo se modifica, las instancias (y la misma clase) siempre verán el nuevo valor.

Si como parte de la solución se detecta la necesidad de que una clase tenga atributos y/o operaciones pertenecientes a la clase, se deben incorporar en el diagrama de clases. En este caso, se ha detectado que es buena idea que todas las instancias de **Persona** sepan cuál es el valor del sueldo mínimo que actualmente está vigente:

Persona
-rut: String -nombre: String -dirección: String <u>-sueldoMínimo: int</u>
+Persona(rut:String, nombre:String) +getRut():String +getNombre():String +getDirección():String +agregarDescendiente(nombre:String) <u>+configurarSueldoMínimo(valor: int)</u>

3.3.3. Métodos

Es conveniente en este punto agregar una nueva palabra a nuestro vocabulario. Desde ahora, hablaremos de un **método** cuando nos estemos refiriendo al cuerpo de código que implementa una operación. Hasta ahora podríamos habernos referido a este mismo concepto como “función” o “procedimiento”, pero en el mundo de la orientación al objeto, se acostumbra decir *método*, para hacer la diferencia entre operaciones que pertenecen a una clase, versus operaciones que pueden estar desconectadas de cualquier objeto. En la práctica en Java ésto no es factible, pero existen otros lenguajes donde es perfectamente posible tener funciones que no están relacionadas con ninguna clase, pero que igual pueden ser invocadas desde cualquier contexto.

De esta forma, diremos que las clases tienen métodos, en vez de decir operaciones. Diremos que hay métodos de instancia, cuando se refieren a mensajes que invocan operaciones a nivel de las instancias, y diremos que hay métodos de clase cuando se refiere a lo mismo, pero a nivel de la clase.

3.3.4. Implementación en Java

Ahora que ya hemos diseñado parte de la solución, se han determinado todos los objetos que van a participar, cada uno con sus atributos y métodos, tanto de instancia como de clase, el siguiente paso es implementar el diseño en Java.

Para ésto, cada clase se debería implementar de acuerdo a las buenas prácticas. En particular:

- Cada clase debería residir en su propio archivo (hay excepciones a esta regla, pero se verán más adelante).
- Se debería respetar la nomenclatura Java para escribir los nombres de clases, atributos y métodos. Para más detalles, ver 1.9.
- Se deberían respetar los lineamientos acerca de cómo escribir el cuerpo de los métodos (espacios, líneas en blanco, posición de las llaves, etc).
- Se debe documentar el código, de acuerdo a la especificación javadoc. Para más detalles, ver 1.17.

Por ejemplo, para la clase anterior, su código Java sería el siguiente (para efectos de ahorrar espacio se omitieron los comentarios javadoc):

```
1 public class Persona
2 {
3     private String rut;
4     private String nombre;
5     private String dirección;
6     private static int sueldoMinimo;
7
8     public Persona(String elRut, String elNombre)
9     {
10         rut = elRut;
11         nombre = elNombre;
12     }
13     public String getNombre()
14     {
15         return nombre;
16     }
17     public String getRut()
18     {
19         return rut;
20     }
21     public String getDirección()
22     {
23         return dirección;
24     }
25     public static void configurarSueldoMinimo(int valor)
26     {
27         sueldoMinimo = valor;
28     }
29     public void agregarDescendiente(String nombre)
30     {
31         // Pendiente!
32     }
33 }
```

Nótese cómo tanto el atributo como la operación de la clase se escriben con el modificador `static`. Generalmente en vez de decir operaciones/atributos “de la clase” se les conoce directamente como métodos y atributos **estáticos**.

Es conveniente destacar que al momento de crear una clase en Java, el orden en que se escriben sus componentes no es relevante. Generalmente los atributos se escriben primero (en la parte de “arriba” de la clase), y después vienen las operaciones, pero esto no es para nada obligatorio. Técnicamente, los atributos y las operaciones se pueden escribir en cualquier orden, e incluso más, se puede mezclar atributos con operaciones de cualquier forma. Lo único que se requiere es que la clase tenga la sintaxis correcta. Por ejemplo:


```

1 public class Vehículo
2 {
3     public String getModelo()
4     {
5         return modelo;
6     }
7     private String marca; private String fechaEntrega;
8     private String patente;
9     public Vehículo(String laMarca, String elModelo, String laFechaInicioFabricación)
10    {
11        marca = laMarca;
12        modelo = elModelo;
13        fechaInicioFabricación = laFechaInicioFabricación;
14        fechaEntrega = null;
15        patente = null;
16    }
17    private String modelo;
18    public String getMarca()
19    {
20        return marca;
21    }
22    private String fechaInicioFabricación;
23 }

```

Aun cuando el código anterior es 100 % válido, se recomienda seguir el lineamiento de primero escribir (y documentar) los atributos, y posteriormente las operaciones de la clase.

Supongamos que debido al análisis del problema, necesitamos modificar la clase Persona para agregarle un nuevo método llamado `getDirecciónEnMayúsculas()`. Este método no espera ningún parámetro, y retornará la dirección de la persona (o sea, el valor almacenado en el atributo dirección de la instancia que reciba el mensaje), pero convertido completamente a mayúsculas:

Persona
-rut: String -nombre: String -dirección: String <u>-sueldoMínimo: int</u>
+Persona(rut:String, nombre:String) +getRut():String +getNombre():String +getDirección():String +getDirecciónEnMayúsculas():String +agregarDescendiente(nombre:String) <u>+configurarSueldoMínimo(valor: int)</u>

y de la misma forma, completamos el código:

```
1 public class Persona
2 {
3     private String rut;
4     private String nombre;
5     private String dirección;
6     private static int sueldoMínimo;
7
8     public Persona(String elRut, String elNombre)
9     {
10         rut = elRut;
11         nombre = elNombre;
12     }
13     public String getNombre()
14     {
15         return nombre;
16     }
17     public String getRut()
18     {
19         return rut;
20     }
21     public String getDirección()
22     {
23         return dirección;
24     }
25     public String getDirecciónEnMayúsculas()
26     {
27         // Java nos provee con una forma de, a partir
28         // de un String, obtener un nuevo String con
29         // el mismo contenido del original, pero
30         // convertido a mayúsculas.
31         //
32         return dirección.toUpperCase();
33     }
34     public static void configurarSueldoMínimo(int valor)
35     {
36         sueldoMínimo = valor;
37     }
38     public void agregarDescendiente(String nombre)
39     {
40         // Pendiente!
41     }
42 }
```

Ahora podríamos preguntarnos: ¿cómo probamos lo que hemos hecho? La respuesta es que crearemos un programa para poner a prueba esta clase.

3.3.5. El misterio de la ejecución de los programas en Java

Como ya se ha dicho antes (ver la página 19), en Java los programas se inician desde un lugar especial. Dicho lugar no es nada más que un método estático llamado `main`.

Por costumbre, es normal que dicho método se cree dentro de una clase llamada `App` o `Main`, o algo similar, para indicar que por ahí se iniciará el programa, pero esto no es estrictamente necesario.

Una posible clase `App` podría ser:

```
1 public class App
2 {
3     public static void main(String[] args)
4     {
5     }
6 }
```

Esta clase es 100 % legal, y compila correctamente y se ejecuta sin problemas, pero no realiza ninguna acción. Si queremos poner a prueba nuestra clase `Persona`, entonces debemos instanciar alguna:

```
1 public class App
2 {
3     public static void main(String[] args)
4     {
5         Persona p;
6
7         p = new Persona("11111111", "Ximena");
8
9         System.out.println("Hola! He creado una persona!");
10    }
11 }
```

Este ejemplo sirve para demostrar varias situaciones:

- Las instancias de una clase se crean usando la palabra `new`. Dicha acción retorna una nueva instancia, que es posible asignar a una variable. Ver 3.3.6 para más detalles.
- La variable que usamos para “guardar” la nueva instancia es una variable como cualquier otra, pero tiene la diferencia de que su “tipo” no es un tipo de los llamados primitivos (ver 1.6), sino que su tipo es una clase. Desde ahora tenemos que saber que *toda clase es un tipo*.

Volviendo al tema de la ejecución del programa, ya se dijo que la ejecución se inicia en el método `main`. Pero, más específicamente, la ejecución se iniciará en el método `main` de “cierta” clase. Ésto se puede ver mejor al ejecutar los programas usando la línea de comandos.

En este caso, al ejecutar el comando `java` para iniciar un programa, se le debe indicar el *nombre* de la clase que contiene el método estático `main`:

```
$ java App
Hola! He creado una persona!
```

Ahora, modifiquemos nuestra clase Persona de esta forma:

```
1 public class Persona
2 {
3     public static void main(String[] args) // esto es nuevo!
4     {
5         System.out.println("No pasa nada por acá");
6     }
7     private String rut;
8     private String nombre;
9     private String dirección;
10    private static int sueldoMínimo;
11
12    public Persona(String elRut, String elNombre)
13    {
14        rut = elRut;
15        nombre = elNombre;
16    }
17    public String getNombre()
18    {
19        return nombre;
20    }
21    public String getRut()
22    {
23        return rut;
24    }
25    public String getDirección()
26    {
27        return dirección;
28    }
29    public String getDirecciónEnMayúsculas()
30    {
31        return dirección.toUpperCase();
32    }
33    public static void configurarSueldoMínimo(int valor)
34    {
35        sueldoMínimo = valor;
36    }
37    public void agregarDescendiente(String nombre)
38    {
39        // Pendiente!
40    }
41 }
```

Fíjate que lo único que se ha hecho es agregarle un método estático a la clase Persona.

Después de volver a compilar los archivos, veamos lo que pasa cuando ejecutamos nuevamente nuestro programa:

```
$ java App
Hola! He creado una persona!
```

Todo normal hasta ahora. Pero veamos qué pasa con esto:

```
$ java Persona
No pasa nada por acá
```

Para poder entender lo que ha sucedido, solo hay que recordar ésto: por convención los programas en

Java se inician en el método estático `main`⁶. El método estático `main` es simplemente un método más en la clase, y de la misma forma que no hay restricciones en que diferentes clases tengan atributos que tengan el mismo nombre, muchas clases pueden tener un método estático llamado `main`. Es por eso que para poder ejecutar correctamente un programa en Java, lo que realmente se requiere (es lo único que se requiere) es el nombre de la clase. A partir de ahí, el intérprete Java buscará el método estático `main` dentro de la clase y comenzará a ejecutar el código que ahí está.

Si es que tratamos de invocar el intérprete Java en una clase que no tiene un método estático `main` pasaría lo siguiente:

```
$ java NoTengoMain
Error: Main method not found in class NoTengoMain, please define the main method as:
    public static void main(String[] args)
```

Literalmente el intérprete nos está diciendo que no ha encontrado el método `main`, e incluso nos dice cómo tenemos que escribirlo.

Lo bueno de usar una IDE es que normalmente todo el proceso anterior es algo que nunca veremos, ya que la IDE se ocupa de realizar todo y simplemente solo tenemos que preocuparnos de presionar el botón correcto.

3.3.6. Cómo se construyen las instancias

En este punto, y antes de continuar probando si es que nuestra clase `Persona` funciona, nos conviene describir una operación “extra” que ha aparecido en los diagramas anteriores, pero que hemos ignorado hasta ahora.

Como se vió más arriba, la palabra `new` se usa para crear nuevas instancias de una clase. Ésto causa que básicamente se reserve un espacio de memoria donde se guardará dicha instancia, y todos los valores de sus atributos. Por ejemplo, en el caso anterior, es como si cada vez que se crea una instancia nueva, se crea una nueva variable `nombre` en algún lugar de la memoria.

Lo anterior nos lleva a preguntarnos: cuando una instancia se crea, ¿qué valores tienen sus atributos? Hagamos una prueba:

```
1 public class TestValores {
2     public static void main(String[] args) {
3         TestValores t;
4         t = new TestValores();
5         t.mostrar();
6     }
7     private String nombre;
8     private int valor;
9     private double nota;
10
11     public void mostrar() {
12         System.out.println("Mostrando mis valores:");
13         System.out.println("nombre = " + nombre + "");
14         System.out.println("valor = " + valor + "");
15         System.out.println("nota = " + nota + "");
16     }
17 }
```

⁶Hay un par de detalles extras, pero por ahora no vale la pena complicarnos.

Como se ve, esta clase contiene el método `main`, así que se puede ejecutar directamente. En la línea 7 se crea una instancia de la clase, y en la línea 8 se le envía a esa instancia un mensaje, que causa que la ejecución del código salte a la línea 15. En ese punto, se imprimirá por pantalla el contenido de las variables de esa instancia.

Si ejecutamos este programa, obtendremos lo siguiente:

```
$ java TestValores
Mostrando mis valores:
nombre = null
valor = 0
nota = 0.0
```

Como se puede ver, cuando se creó una nueva instancia, los atributos dentro de dicha instancia adquirieron ciertos valores especiales. Estos valores se llaman *valores por defecto*, y se asignan de la siguiente forma:

- Los tipos enteros (`int`, `long`, `short`, `byte`) se inicializan con valor 0 (cero).
- Los tipos de punto flotante (`double`, `float`) se inicializan con sus versiones respectivas de 0.0.
- Los tipos boolean se inicializan a `false`.
- Los tipos que corresponden a clases, se inicializan en `null`. Más adelante revisaremos qué significa. Nótese que el tipo `String` es una clase, así que su valor por defecto es `null`.

Algo muy importante que debe determinarse a partir del análisis del problema, es decidir qué valores se necesitan para crear instancias de cada clase. Por ejemplo, si estamos solucionando un problema que tiene que ver con vehículos en una línea de armado, es posible que al momento en que vayamos a crear una instancia de `Vehículo` para representar a un vehículo que actualmente se está armando, no tengamos todos los valores de sus atributos. Por ejemplo, si tenemos definida la clase `Vehículo` así:

Vehículo
-fechaInicioFabricación: String
-modelo: String
-marca: String
-fechaEntrega: String
-patente: String

Al momento de crear la instancia no tendremos valores para asignar a todos atributos (seguramente `fechaEntrega` y `patente` no estarán definidos al momento de crear la instancia). Es por ésto, que el proceso de análisis nos debe indicar qué conjunto de atributos son los necesarios para poder crear una nueva instancia de la clase. Hay veces en que se necesitará que todos los atributos tengan valor, y en otros casos (como en el caso de `Vehículo`) solo se requerirán algunos.

3.3.7. El constructor

De acuerdo a lo anterior, y para poder crear instancias de las clases, se debe definir el **constructor** de la clase, el cuál es una operación que se invoca automáticamente después de que la memoria para la instancia ha sido creada, y que permitirá inicializar los valores de los atributos de la instancia, ya sea a partir de valores que recibe como parámetros, o de acuerdo a alguna lógica interna. En este caso, la clase Vehículo se completaría así:

Vehículo
-fechaInicioFabricación: String -modelo: String -marca: String -fechaEntrega: String -patente: String
+Vehículo(marca: String, modelo: String, fechaInicioFabricación: String)

Y el código que implementa a la clase sería así:

```

1 public class Vehículo
2 {
3     private String fechaInicioFabricación;
4     private String modelo;
5     private String marca;
6     private String fechaEntrega;
7     private String patente;
8
9     public Vehículo(String laMarca, String elModelo, String laFechaInicioFabricación)
10    {
11        marca = laMarca;
12        modelo = elModelo;
13        fechaInicioFabricación = laFechaInicioFabricación;
14        fechaEntrega = null;
15        patente = null;
16    }
17 }
```

El hecho de definir una operación que tenga el mismo nombre que la clase nos permite diferenciar al constructor de la clase. Nótese también que en el código Java, el constructor no retorna nada: en su definición no indica qué cosa retorna, y tampoco tiene un `return`. Ésto es totalmente innecesario, ya que se sabe que el constructor se usa para inicializar una nueva instancia, así que no requiere retornar nada, ya que está totalmente definido sobre qué tipo de dato opera.

En este caso, el análisis del problema dictó que para poder instanciar un nuevo Vehículo, solo se requiere que tenga valores para la marca, modelo y fecha de inicio de fabricación. Además, se eligió explícitamente asignar valores `null` en las líneas 14 y 15, aunque no es estrictamente necesario.

Es necesario hacer notar que las clases *siempre* tienen un constructor, incluso cuando no se ha definido uno de forma explícita. Si una clase la escribimos sin constructor, el compilador generará uno automáticamente, llamado el constructor por defecto. Éste no recibirá ningún parámetro, y no realizará ninguna acción más allá de inicializar todos los atributos a sus valores por defecto. Algo equivalente a:

```
1 public class Vehículo
2 {
3     private String fechaInicioFabricación;
4     private String modelo;
5     private String marca;
6     private String fechaEntrega;
7     private String patente;
8
9     public Vehículo()
10    {
11    }
12 }
```

¿Qué pasaría si estamos poniendo al día el sistema, y necesitamos crear vehículos que ya salieron de la línea de producción, pero ahora recién se van a crear como instancias? En ese caso, una opción sería construir las instancias, y después a cada una invocarle los “setters”⁷ de cada atributo para configurarlo. Pero para ésto dependemos de que dichos “setters” existan, pero es posible que la clase no los contenga ya que son atributos que por diseño no se deberían cambiar.

Una solución a ésto sería tratar de definir múltiples constructores, uno para cada combinación de atributos que sean necesaria para crear las instancias de la forma en que se requieran. Ésto se verá más adelante cuando revisemos el tema de *sobrecarga*.

3.3.8. setters y getters

Una pregunta importante que deberíamos responder es la siguiente: si se crea una instancia de alguna clase, ¿cómo se cambia su estado?

Para esto, normalmente cada clase definirá una serie de métodos para tanto “setear” el valor de sus atributos, como para consultarlos. A dichos métodos generalmente se les llama “setters” y “getters”, y son métodos normales, pero que permitirán interrogar y configurar las instancias.

Aun cuando pudiera parecer buena idea que las clases deben tener setters y getter para todos sus atributos, en la práctica ésto depende del diseño del software. Hay veces en que será necesario tener muchos “setters”, pero muchas veces también existirán atributos que no podrán ser “seteados” directamente.

Una buena regla a seguir es que las clases deben definir setters solo para los atributos que es necesario poder cambiar. Por ejemplo, una **Persona** puede cambiar de edad, así que tal vez sea lógico incluir un método `setEdad(...)`, pero por otro lado, es posible que esas instancias de **Persona** nunca cambien de RUT, así que en ese caso, no habría razón para incluir el setter correspondiente.

Respecto a los getters, aun cuando puede parecer que agregarle getters a todos los atributos es algo inocuo, en la práctica ésto le puede dar acceso a nuestros clientes para que cambien algo de forma indebida. Por ejemplo, una clase **Camión** podría tener un getter para obtener uno de sus neumáticos (representado por una instancia de la clase **Neumático**). Al recibir la instancia a través del getter, nada impide que empecemos a llamar sus métodos, y tal vez modificar su estado, sin que la clase dueña del neumático se entere.

⁷O en otras palabras, operaciones de la instancia que permiten cambiar los valores de los atributos de dicha instancia.

3.3.9. Probando la clase Persona

Volvamos a la clase Persona:

```

1 public class Persona
2 {
3     private String rut;
4     private String nombre;
5     private String dirección;
6     private static int sueldoMínimo;
7
8     public Persona(String elRut, String elNombre)
9     {
10         rut = elRut;
11         nombre = elNombre;
12     }
13     public String getNombre()
14     {
15         return nombre;
16     }
17     public String getRut()
18     {
19         return rut;
20     }
21     public String getDirección()
22     {
23         return dirección;
24     }
25     public String getDirecciónEnMayúsculas()
26     {
27         return dirección.toUpperCase();
28     }
29     public static void configurarSueldoMínimo(int valor)
30     {
31         sueldoMínimo = valor;
32     }
33     public void agregarDescendiente(String nombre)
34     {
35         // Pendiente!
36     }
37 }

```

Y a la clase App que ejecutaremos para poner a prueba el funcionamiento de Persona:

```

1 public class App
2 {
3     public static void main(String[] args)
4     {
5         Persona p1 = new Persona("11111111", "Ximena");
6         Persona p2 = new Persona("22222222", "María");
7         Persona p3 = new Persona("33333333", "Fernando");
8
9         System.out.println("Hola! He creado 3 personas!");
10
11         System.out.println("El nombre de p1 es " + p1.getNombre());
12         System.out.println("El nombre de p2 es " + p2.getNombre());
13         System.out.println("El nombre de p3 es " + p3.getNombre());
14     }
15 }

```

Al ejecutar este programa, se obtiene el siguiente resultado:

```
$ java App
Hola! He creado 3 personas!
El nombre de p1 es Ximena
El nombre de p2 es María
El nombre de p3 es Fernando
```

Lo que significa que la clase funciona, las instancias están siendo creadas de forma apropiada (ya que guardan los parámetros que les llegan a través del constructor), y están respondiendo correctamente al mensaje `getNombre()`. Al invocar a dicho método, se retorna el nombre que está almacenado en la instancia correspondiente, el que finalmente se escribe por pantalla.

Es muy interesante notar en este punto, que cada vez que se hace una llamada al método `getNombre()` en las líneas 11, 12 y 13, la ejecución salta al mismo lugar: al método definido en la línea 13 de la clase `Persona`. Es más interesante preguntarse ¿cómo es que se pueden obtener 3 valores diferentes de nombre, si al final lo que se está ejecutando es el mismo código? Y la respuesta es que *todo depende del contexto*.

3.3.10. `this`

Cuando se llama al método `getNombre()` efectivamente se está invocando literalmente el mismo código, pero cada vez el contexto de ejecución es diferente: la primera vez es la instancia `p1`, después es la instancia `p2` y finalmente la instancia `p3`. Pero, ¿cómo afecta eso a la ejecución del método? ¿cómo es que realmente funciona? Considera esta nueva y mejorada versión de `Persona`:

```
1 public class Persona
2 {
3     private String rut;
4     private String nombre;
5     private String dirección;
6     private static int sueldoMinimo;
7
8     public Persona(String elRut, String elNombre)
9     {
10         this.rut = elRut;
11         this.nombre = elNombre;
12     }
13     public String getNombre() { return this.nombre; }
14     public String getRut() { return this.rut; }
15     public String getDirección() { return this.dirección; }
16
17     public String getDirecciónEnMayúsculas()
18     {
19         return this.dirección.toUpperCase();
20     }
21     public static void configurarSueldoMinimo(int valor)
22     {
23         sueldoMinimo = valor;
24     }
25     public void agregarDescendiente(String nombre)
26     {
27         // Pendiente!
28     }
29 }
```

Fíjate que en varias líneas (10, 11, 15, etc), se ha agregado una nueva palabra a nuestro vocabulario. Esta nueva palabra es `this`, y su funcionamiento es bastante simple: *this siempre hace referencia a la instancia actual*.

Con este conocimiento, ya deberíamos poder dilucidar cómo funciona el método `getNombre()` y su capacidad para retornar diferentes valores. Como se dijo recién, todo depende del contexto, y cuando la ejecución llega a la línea 15, `this` apuntará a la instancia de `Persona` a donde se envió el mensaje. De esta forma, se puede saber desde qué lugar de la memoria leer el valor del nombre (o a qué lugar de la memoria escribirlo).

3.3.11. Probando la clase `Persona` (II)

Sigamos completando nuestro programa de prueba:

```

1 public class App
2 {
3     public static void main(String[] args)
4     {
5         Persona p1 = new Persona("11111111", "Ximena");
6         Persona p2 = new Persona("22222222", "María");
7         Persona p3 = new Persona("33333333", "Fernando");
8
9         System.out.println("Hola! He creado 3 personas!");
10
11        System.out.println("El nombre de p1 es " + p1.getNombre());
12        System.out.println("El nombre de p2 es " + p2.getNombre());
13        System.out.println("El nombre de p3 es " + p3.getNombre());
14
15        System.out.println("Dirección de p3 es " + p3.getDirecciónEnMayúsculas());
16    }
17 }

```

Al ejecutar este programa, se obtiene el siguiente resultado:

```

$ java App
Hola! He creado 3 personas!
El nombre de p1 es Ximena
El nombre de p2 es María
El nombre de p3 es Fernando

Exception in thread "main" java.lang.NullPointerException
    at cl.ucn.prograavanzada.ejemplo1.Persona.getDirecciónEnMayúsculas(Persona.java:27)
    at cl.ucn.prograavanzada.ejemplo1.App.main(App.java:17)

```

Lo que apareció por pantalla es el resultado de la clásica excepción que no fue capturada de forma apropiada. Para saber cómo funcionan, puedes revisar en la página 74 el punto 2.6 para más detalles.

Como se ve en el stack trace, desde la línea 17 en `App.java` se invoca el método `getDirecciónEnMayúsculas` y en la línea 27 de `Persona.java` se generó la excepción `NullPointerException`, la cual no fue atajada y provocó que el programa terminara de forma abrupta.

Para resolver este problema, necesitamos mirar en la línea 27 del archivo Persona.java:

```
27 public String getDirecciónEnMayúsculas()
28 {
29     return this.dirección.toUpperCase();
30 }
```

Para entender lo que está pasando, debemos preguntarnos: ¿cuando la instancia recibe el mensaje, qué valor tiene el atributo dirección? Fíjate que al invocarse el método, a la vez se invoca el método toUpperCase() de la instancia referenciada por dirección. En ese momento del tiempo, ¿qué valor tiene dirección?

3.3.12. El valor null

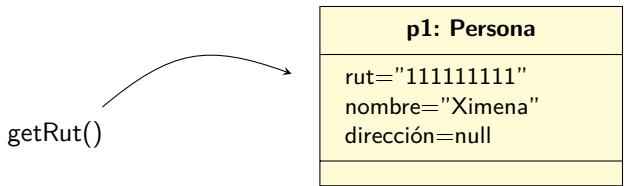
De acuerdo al experimento que se hizo antes en la página 106, pudimos determinar que las variables de tipo String se inicializan con su valor por defecto, que es null, y como nunca le hemos cambiado el valor a la variable dirección (fíjate que en ningún momento se le ha asignado nada), estamos tratando de enviarle el mensaje toUpperCase() a una instancia que no existe.

Cada vez que enviamos un mensaje, debemos asegurarnos de que realmente existirá una instancia que lo reciba, ya que si no se hace así, la máquina virtual lanzará la excepción NullPointerException.

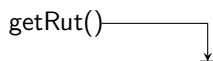
Veamos un ejemplo más sencillo:

```
1 public class TestNull
2 {
3     public static void main(String[] args)
4     {
5         Persona p1 = new Persona("11111111", "Ximena");
6         Persona p2 = null;
7
8         System.out.println(p1.getRut());
9         System.out.println(p2.getRut());
10    }
11 }
```

En la línea 8 se está realizando la siguiente acción:



¿Y qué acción se realiza en la línea 9?



En este caso, ¿a quién se le está enviando el mensaje `getRut()`? Tal como está dibujado, la flecha en vez de apuntar a una instancia, está apuntando “a tierra”. De esta forma vamos a dibujar cuando una variable apunta a `null`. Y la verdad, no se le puede enviar un mensaje a `null`.

Cada vez que enviemos un mensaje usando una variable que esté apuntando a `null` se lanzará la excepción `NullPointerException`. Veamos qué pasa si ejecutamos el código anterior:

```
$ java TestNull
11111111
Exception in thread "main" java.lang.NullPointerException
    at cl.ucn.prograavanzada.ejemplo1.TestNull.main(TestNull.java:9)
```

En nuestro caso inicial, la excepción se lanza ya que cuando llamamos al método `getDirecciónEnMayúsculas` la instancia no ha inicializado el atributo `dirección`. Si te fijas, en el constructor de `Persona` solo se le asignan valores a los atributos `rut` y `nombre`. El resto de los atributos (o sea, `dirección`) quedan inicializados con sus valores por defecto, y como se trata de un `String`, su valor por defecto es `null`. Y al tratar de enviar el mensaje `toUpperCase()`, esa instancia de `String` es `null`, así que inmediatamente se lanza la excepción.

3.3.13. Probando la clase `Persona` (III)

Ya que descubrimos que nuestra clase `Persona` tiene un bug⁸, realicemos una modificación para que funcione como es debido:⁹

```
1 public class Persona
2 {
3     private String rut;
4     private String nombre;
5     private String dirección;
6     private static int sueldoMínimo;
7
8     public Persona(String elRut, String elNombre)
9     {
10         this.rut = elRut;
11         this.nombre = elNombre;
12         this.dirección = "sin dirección";
13     }
14     public String getNombre() { return this.nombre; }
15     public String getRut() { return this.rut; }
16     public String getDirección() { return this.dirección; }
17     public String getDirecciónEnMayúsculas() { return this.dirección.toUpperCase(); }
18     public static void configurarSueldoMínimo(int valor) { sueldoMínimo = valor; }
19     public void agregarDescendiente(String nombre) { /* Pendiente! */ }
20 }
```

En la línea 12 le hemos asignado explícitamente un valor al atributo `dirección` de todas las instancias que se creen. De esta forma, al ejecutar nuestro programa de prueba, ahora entrega el siguiente resultado:

⁸https://es.wikipedia.org/wiki/Error_de_software

⁹La vamos a escribir de forma compacta, solo para ahorrar espacio

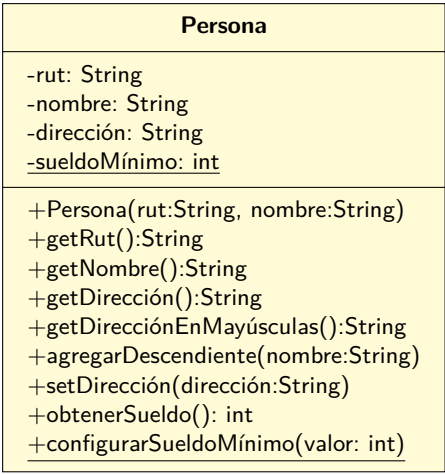
```
$java App
Hola! He creado 3 personas!
El nombre de p1 es Ximena
El nombre de p2 es María
El nombre de p3 es Fernando
Dirección mayúsculas de p3 es SIN DIRECCIÓN
```

Para continuar probando la clase, imaginemos que como parte del análisis las personas deben poder calcular su sueldo, y que éste se calcula así:

- Si la persona vive en Santiago, corresponde a un 120 % del sueldo mínimo.
- Si vive en el resto del país, corresponde al 180 % del sueldo mínimo.

El problema con esta versión, es que no tenemos ninguna forma de calcular un sueldo real, ya que el sueldo depende de la dirección, pero no tenemos ninguna forma de modificar la dirección donde vive cada persona. Como a partir del análisis se determinó que lo único necesario para que una clase exista es su rut y su nombre, tendremos que agregar un nuevo método para poder modificar la dirección de una instancia de Persona que ya existe. O sea, tenemos que agregar un “setter” para dicho atributo.

Modifiquemos el diagrama de clases:



y actualicemos la clase Persona:

```
1 public class Persona
2 {
3     private String rut;
4     private String nombre;
5     private String dirección;
6     private static int sueldoMínimo;
7
8     public Persona(String elRut, String elNombre)
9     {
10         this.rut = elRut;
11         this.nombre = elNombre;
```

```

12     this.dirección = "sin dirección";
13 }
14 public String getNombre() { return this.nombre; }
15 public String getRut() { return this.rut; }
16 public String getDirección() { return this.dirección; }
17 public void setDirección(String dir) { this.dirección = dir; }
18 public String getDirecciónEnMayúsculas() { return this.dirección.toUpperCase(); }
19 public static void configurarSueldoMínimo(int valor) { sueldoMínimo = valor; }
20 public void agregarDescendiente(String nombre) { /* Pendiente! */ }
21
22 public int obtenerSueldo()
23 {
24     double f = 1.8;
25     if (this.dirección.equals("Santiago")) {
26         f = 1.2;
27     }
28     return (int) (sueldoMínimo * f);
29 }
30 }

```

Actualicemos nuestro código de prueba, para ver si la clase calcula bien su sueldo:

```

1 public class App
2 {
3     public static void main(String[] args)
4     {
5         Persona p1 = new Persona("11111111", "Ximena");
6         Persona p2 = new Persona("22222222", "María");
7         Persona p3 = new Persona("33333333", "Fernando");
8
9         System.out.println("Hola! He creado 3 personas!");
10
11         System.out.println("El nombre de p1 es " + p1.getNombre());
12         System.out.println("El nombre de p2 es " + p2.getNombre());
13         System.out.println("El nombre de p3 es " + p3.getNombre());
14
15         System.out.println("Dirección de p3 es " + p3.getDirecciónEnMayúsculas());
16
17         Persona.configurarSueldoMínimo(1000);
18
19         p2.setDirección("Punta Arenas");
20         System.out.println("Sueldo de p2 es " + p2.obtenerSueldo());
21         p2.setDirección("Santiago");
22         System.out.println("Sueldo de p2 es " + p2.obtenerSueldo());
23     }
24 }

```

Al ejecutar este programa, se obtiene:

```

$ java App
Hola! He creado 3 personas!
El nombre de p1 es Ximena
El nombre de p2 es María
El nombre de p3 es Fernando
Dirección mayúsculas de p3 es SIN DIRECCIÓN
Sueldo de p2 es 1800
Sueldo de p2 es 1200

```

Revisemos el código ya que hay un par de cosas nuevas.

De partida, fíjate en la línea 17. En dicha línea se está invocando el método `configurarSueldoMínimo(...)`, pero el mensaje no se le está enviando a una instancia en particular, sino que se está enviando a la clase. Ésto es así ya que el método está definido de esa forma. Para enviar el mensaje no se necesita tener una instancia, sino que solamente la clase.

A partir de lo anterior, al invocar `obtenerSueldo()`, el método hace uso del atributo estático `sueldoMínimo` para realizar el cálculo y retornar el valor que le corresponde a la instancia. Pero tal como se ha dicho varias veces antes, el valor va a ser el mismo para todas las instancias. Por ejemplo:

```
1 public class App
2 {
3     public static void main(String[] args)
4     {
5         Persona p1 = new Persona("11111111", "Ximena");
6         Persona p2 = new Persona("22222222", "María");
7         Persona p3 = new Persona("33333333", "Fernando");
8
9         p2.setDirección("Punta Arenas");
10        p3.setDirección("Santiago");
11
12        Persona.configurarSueldoMínimo(1000);
13
14        System.out.println("Sueldo de p2 es " + p2.obtenerSueldo());
15        System.out.println("Sueldo de p3 es " + p3.obtenerSueldo());
16
17        Persona.configurarSueldoMínimo(2000);
18
19        System.out.println("Sueldo de p2 es " + p2.obtenerSueldo());
20        System.out.println("Sueldo de p3 es " + p3.obtenerSueldo());
21    }
22 }
```

Al ejecutar ésto obtenemos:

```
$ java App
Sueldo de p2 es 1800
Sueldo de p3 es 1200
Sueldo de p2 es 3600
Sueldo de p3 es 2400
```

En la línea 12 se configuró el valor general del sueldo mínimo, y después en las líneas 14 y 15 cada instancia retornó "su" valor para el sueldo. Posteriormente se actualizó el sueldo mínimo y se volvió a consultar por cada sueldo. Todo funciona correctamente y tal como se esperaba.

3.3.14. Alcance de las variables

Muchas veces vas a encontrar código que está escrito de esta forma:

```

1 public class Producto
2 {
3     private String código;
4     private int lote;
5     private String descripción;
6
7     public Producto(String elCódigo, int elLote, String laDescripción)
8     {
9         código = elCódigo;
10        lote = elLote;
11        descripción = laDescripción;
12    }
13 }
```

En otras palabras, cuando se recibe un parámetro que tiene el mismo nombre que un atributo, muchas veces se le cambia el nombre al parámetro para evitar que pase lo siguiente:

```

1 public class Producto
2 {
3     private String código;
4     private int lote;
5     private String descripción;
6
7     public Producto(String código, int lote, String descripción)
8     {
9         código = código; // The assignment to variable código has no effect
10        lote = lote;
11        descripción = descripción;
12    }
13 }
```

Para poder explicar el por qué de la advertencia, es necesario hablar del alcance de las variables, y también del orden de prioridad para resolver ambigüedades.

Alcance de la clase

Todos los atributos de una clase tienen alcance de clase. Por lo tanto, estas variables se pueden usar en cualquier lugar dentro de la clase, pero no afuera.

```

1 public class Producto
2 {
3     private int cantidad = 0;
4
5     public void unMétodo()
6     {
7         cantidad++;
8     }
9
10    public void otroMétodo()
11    {
12        int q = cantidad + 4;
13    }
14 }
```

Si el atributo `cantidad` fuera declarado como `public` entonces podría ser accedido desde fuera de la clase de forma directa.

Alcance de método

Al declarar una variable dentro de un método, ésta tendrá alcance dentro de método, por lo que solamente será visible dentro de éste.

```
1 public class Producto
2 {
3     public void unMétodo()
4     {
5         int cantidad = 10;
6     }
7
8     public void otroMétodo()
9     {
10        int q = cantidad + 4; // cantidad cannot be resolved to a variable
11    }
12 }
```

Alcance de ciclo

Las variables declaradas como parte del control de un ciclo solamente serán visibles dentro del ciclo.

```
1 public class Producto
2 {
3     public void unMétodo()
4     {
5         for(int i=0;i<10;i++) {
6             System.out.println(i);
7         }
8         System.out.println("Último valor " + i); // i cannot be resolved to a variable
9     }
10 }
```

Alcance de bloque

Las variables declaradas dentro de un *bloque* solamente serán visibles dentro del mismo bloque.

```
1 public class Producto
2 {
3     public void unMétodo()
4     {
5         int valor = 0;
6
7         if (valor > 10) {
8             int factor = 100;
9             valor += factor;
10        }
11        System.out.println("Último valor " + valor);
12        System.out.println("Factor=" + factor); // cannot be resolved to a variable
13    }
14 }
```

Y no necesariamente nos tenemos que limitar a un nivel de anidamiento:

```

1 public class Producto
2 {
3     public void unMétodo()
4     {
5         int valor = 0;
6
7         if (valor > 10) {
8             int factor = 100;
9             valor += factor;
10
11             if (factor < 50) {
12                 int q = valor + factor;
13                 System.out.println(q);
14             }
15             System.out.println("q=" + q); // cannot be resolved to a variable
16         }
17         System.out.println("Último valor " + valor);
18         System.out.println("Factor=" + factor); // cannot be resolved to a variable
19         System.out.println("q=" + q); // cannot be resolved to a variable
20     }
21 }

```

Shadowing

Vamos a decir que una variable le hace “sombra” a otra variable cuando tenemos el mismo nombre declarado en alcances que se solapan. En este caso, el alcance de más bajo nivel tomará prioridad sobre el alcance de más alto nivel.

Por ejemplo:

```

1 public class Ejemplo2
2 {
3     private int valor = 0;
4
5     public void unMétodo()
6     {
7         int valor = 2;
8         System.out.println(valor);
9     }
10 }

```

En este caso, al consultar el valor de la variable `valor` en la línea 8, toma precedencia la declaración dentro de `unMétodo` de la línea 7, así que lo que se escribirá en la pantalla es el número 2.

Pero miremos este otro ejemplo:

```

1 public class Ejemplo2 {
2     private int valor = 0;
3
4     public void unMétodo() {
5         System.out.println(valor);
6         int valor = 2;
7         System.out.println(valor);
8     }
9 }
10 }

```

En este caso, en la línea 7 se consulta por el valor de la variable `valor`. En ese punto del tiempo, la única variable que existe con ese nombre es el atributo de la instancia, por lo que se imprime el valor 0. Al avanzar a la línea 9, en ese momento el símbolo más cercano con el nombre `valor` es la variable local declarada en la línea 8, así que se imprime el valor 2:

```
$ java Ejemplo2
0
2
```

Por otro lado, si tenemos un método que tiene parámetros, éstos se comportan a fin de cuentas como una variable más, así que no podemos hacer esto:

```
1 public class Ejemplo2
2 {
3     private int valor = 0;
4
5     public void unMétodo(int valor)
6     {
7         System.out.println(valor);
8
9         int valor = 2; // Duplicate local variable valor
10        System.out.println(valor);
11    }
12 }
13 }
```

Ya que tendríamos una variable local duplicada. Lo que sí puede suceder es ésto:

```
1 public class Ejemplo2
2 {
3     private int valor = 0;
4
5     public void unMétodo(int valor)
6     {
7         System.out.println(valor);
8     }
9
10    public static void main(String[] args){
11        Ejemplo2 e = new Ejemplo2();
12        e.unMétodo(100);
13    }
14 }
```

En este caso, uno debe preguntarse qué pasa en la línea 7: al tratar de acceder a la variable `valor`, ¿cuál es el lugar más cercado donde está declarada? Y la respuesta es que es el parámetro del método. Si en ese punto quisiéramos hacer referencia al atributo de la instancia en vez de al parámetro, estamos obligados a escribirlo de esta forma:

```
1 public class Ejemplo2
2 {
3     private int valor = 0;
4
5     public void unMétodo(int valor)
6     {
7         System.out.println(this.valor);
8     }
9
10    public static void main(String[] args){
11        Ejemplo2 e = new Ejemplo2();
12        e.unMétodo(100);
13    }
14 }
```

En este caso estamos diciendo explícitamente que queremos el valor del atributo. Es por ésto que cuando vemos código así:

```
1 public class Producto
2 {
3     private String código;
4     private int lote;
5     private String descripción;
6
7     public Producto(String código, int lote, String descripción)
8     {
9         código = código; // The assignment to variable código has no effect
10        lote = lote;
11        descripción = descripción;
12    }
13 }
```

Obtenemos la advertencia de que hay líneas que no tienen efecto, ya que se le está asignando a una variable el valor de la misma variable. Es una mejor práctica escribir:

```
1 public class Producto
2 {
3     private String código;
4     private int lote;
5     private String descripción;
6
7     public Producto(String código, int lote, String descripción)
8     {
9         this.código = código;
10        this.lote = lote;
11        this.descripcion = descripción;
12    }
13 }
```

3.4. Ejercicios

1. ¿Qué es POO?
2. ¿Qué es un objeto?
3. ¿Qué es el estado de un objeto?
4. ¿Qué es la “interfaz pública” de un objeto? ¿Existe una “interfaz privada”?
5. Define el concepto de “encapsulación”
6. Define el concepto de “abstracción”
7. Indica por qué es tan importante seguir los lineamientos de que los diseños (y el código) tengan una “alta cohesión” y “bajo acoplamiento”.
8. ¿Qué es una clase en Java?
9. ¿Qué herramienta se usa para dibujar el modelo del dominio?
10. ¿Cuál es la diferencia entre una “clase” y una “instancia”?
11. ¿Cuál es la diferencia entre un “modelo del dominio” y un “diagrama de objetos”?
12. ¿Qué es un “atributo”?
13. ¿Cómo se indica la multiplicidad en un diagrama del dominio?
14. ¿Cuál es la diferencia entre “asociación” y “agregación”?
15. ¿Cuál es la diferencia entre “agregación” y “composición”?
16. ¿Qué es la generalización?
17. Si hay varios conceptos que comparten muchos de sus atributos ¿es razón suficiente para crear una relación de generalización entre ellos? ¿Por qué sí? ¿Por qué no?
18. Si queremos agregar un atributo privado, llamado `colorPelo`, ¿en qué parte del diagrama de clases se dibuja?
19. Si queremos agregar un método público a la clase anterior, ¿en qué parte del diagrama de clases se dibuja?
20. ¿Cuál es la diferencia entre un mensaje a una instancia y un mensaje a una clase?
21. Completa la clase `Persona` de la página 100, agregando los comentarios en `javadoc`.
22. Crea dos clases en el mismo directorio, ambas con su correspondiente método `main`, realizando diferentes acciones (por ejemplo, que escriban diferentes textos en la pantalla). Compila y ejecuta ambos programas hasta que te familiarices con el hecho de que pueden existir muchos métodos `main` en un proyecto.
23. Construye una clase `Televisor`, que tenga varios atributos privados, además de métodos públicos para sus getters, y un constructor que requiera especificar la marca, modelo y número de serie de cada instancia al ser creada.

24. ¿Qué es `this` en Java?
25. ¿Qué es `null` en Java?
26. ¿Cuál es la diferencia entre `this` y el valor “vacío”?
27. ¿En dónde se usa `null`? ¿Qué cosas no son posibles al usar `null`?
28. ¿Por qué es importante saber cómo funciona el alcance de las variables?
29. ¿Cuál es la diferencia entre “alcance de método” y “alcance de clase”?
30. ¿Cuál es la diferencia entre “alcance de ciclo” y “alcance de bloque”?
31. ¿Es bueno o malo el “shadowing”?

32. Construye el modelo del dominio para este problema:

Todo el país sabe que vienen muchas elecciones en el país, gobernador, constituyente, plebiscito de aprobación de la nueva constitución, etc. Los partidos políticos y las alianzas entre ellos están haciendo sus campañas y negociaciones para determinar a los candidatos. Nosotros tenemos la responsabilidad de ir a votar.

El Serval ha llamado a licitación para la construcción de un programa en Java que maneje la información relacionada con las mesas de votación por comuna, por local, inscritos por mesa, etc., Se requiere hacer de nuevo la aplicación que está actualmente en operación, ya que está a punto de quedar obsoleta. De la misma forma que opera el sistema actual, una persona ingresa con su rut, y le sale dónde le toca votar (local y su dirección), en qué mesa y si es vocal de mesa o no. De una mesa interesa su número, el cual es un correlativo que empieza en 1 por cada comuna. De una comuna interesa su nombre y de un votante nombre, rut y comuna donde vive.

Se sabe que existen locales de votación de distinta magnitud, es decir hay locales en que funcionan más de 40 mesas (grandes) y locales donde funcionan hasta 40 mesas (pequeños)

Se sabe que para los lugares de votación pequeños existe una persona a cargo, para los grandes son varias las personas a cargo (mínimo 2 y máximo 5). Respecto a los locales de votación también es importante saber que cobran un valor que debe ser pagado por el gobierno. Este valor se calcula según:

- Para un local pequeño se cobra \$500 por cada persona que debe votar en el local.
- Para un local grande se cobra un valor fijo (que viene en el archivo Locales.txt y es el mismo para todos los locales grandes) más \$100.000 por cada persona a cargo más \$10.000 por cada mesa del local

De un local de votación interesa, entre otros, su código, nombre, dirección y saber la comuna a la que pertenece. Se sabe que en una local de votación pueden votar tanto hombres como mujeres, lo mismo que en una mesa.

El Serval se debe preocupar de tener la información actualizada de comunas, locales, votantes, vocales de mesa, mesas, etc.. Esto lo hace 5 meses antes de la votación y un mes antes de cada votación se cierra el sistema para actualizar la información.

33. Construye el modelo del dominio para este problema:

Taylor es un reparador de instrumentos musicales que desde hace tiempo tiene un exitoso taller, y ha experimentado un crecimiento sostenido en los últimos años debido a la calidad del trabajo que

realiza. Producto de su propio éxito ha empezado a sufrir por la cantidad de trabajos y clientes que tiene que atender, hasta llegar al punto de dejar de cumplir con las fechas de entregas, o reparando equivocadamente algunos instrumentos, provocando enojo entre sus clientes.

Para evitar todos los problemas anteriormente mencionados, es necesario construir un software que le permita a Taylor llevar un mejor control de los trabajos que realiza, y poder detectar cualquier situación anómala.

Durante las últimas semanas se realizaron una serie de entrevistas con Taylor, las que han permitido capturar una serie de requisitos que él considera importantes para este software que quiere mandar a construir. En particular, Taylor espera que el software procese una serie de archivos donde él normalmente guarda la información de sus clientes, los instrumentos que repara y sus reparaciones.

En primer lugar, Taylor tiene un archivo llamado `instrumentos.txt` donde almacena los datos de sus clientes y los instrumentos que dichos clientes le han mandado para reparar. El archivo tiene la siguiente estructura:

`RUT_CLIENTE, NOMBRE_CLIENTE, CÓDIGO_INSTRUMENTO, TIPO_INSTRUMENTO, ...`

El `RUT_CLIENTE` y `NOMBRE_CLIENTE` identifican al cliente y dueño del instrumento.

El `CÓDIGO_INSTRUMENTO` es un texto que Taylor le asigna a los instrumentos cuando los recibe por primera vez. Este texto es único para cada instrumento (como si fuera un `RUT`). El `TIPO_INSTRUMENTO` especifica qué tipo de instrumento musical es. Actualmente Taylor solamente repara guitarras, pianos y tambores. Dependiendo del tipo de instrumento, la línea contiene los otros datos del instrumento:

Tipo	Atributos específicos
Guitarra	CantidadCuerdas: entero Tipo: normal, barítona, soprano
Piano	Tamaño: normal, vertical, mini
Tambor	Diámetro: entero

Por ejemplo:

```
1111,Juan Perez,G13441,Guitarra,6,normal
1111,Juan Perez,G13442,Guitarra,6,barítona
2222,Maria Gonzalez,FKJ08,Piano,vertical
1111,Juan Perez,G13444,Tambor,55
3333,Ximena C,X331,Piano,normal
```

Taylor tiene un segundo archivo, llamado `reparaciones.txt`, donde almacena las reparaciones que le encargan sus clientes. Este archivo tiene la siguiente estructura:

`RUT_CLIENTE, CÓDIGO_INSTRUMENTO, FECHA_INICIO, CANT_COMENTARIOS`

`RUT_CLIENTE` identifica al cliente que entrega el instrumento a reparar, y `CÓDIGO_INSTRUMENTO` identifica el instrumento específico que debe ser reparado. `FECHA_INICIO` indica la fecha cuando Taylor comenzó la reparación, y `CANT_COMENTARIOS` es la cantidad de anotaciones que Taylor hizo en la libreta donde registra cada acción que hace en una reparación.

Por ejemplo:

2222,FKJ08,2021-01-15,4
3333,X331,2020-12-01,3
1111,G13442,2021-01-01,11
2222,FKJ08,1998-04-20,6

Nótese que los instrumentos pueden haber sido enviados a reparaciones muchas veces en su vida. La FECHA_INICIO permite diferenciar, para un instrumento dado, una reparación de otra, ya que un mismo instrumento no puede empezar a repararse dos veces en la misma fecha.

Taylor espera que después de procesados los archivos, el software le permita solucionar dos problemas:

- a) Calcular el precio que debe cobrar por cada reparación. Actualmente él lo hace “al ojo”, pero le gustaría aplicar una fórmula para hacer el cálculo más justo para todos:

$CostoTotal = PrecioBase + FactorInstrumento \times CantidadComentarios$

- 1) *PrecioBase* es un valor que Taylor configura, y que es fijo para todos los instrumentos
- 2) *FactorInstrumento* es un valor que depende del tipo de instrumento reparado:

Tipo	FactorInstrumento
Guitarra	$0,5 \times cantidadCuerdas$
Piano	3,2
Tambor	$0,01 \times diametro$

- 3) *CantidadComentarios* es la cantidad de anotaciones que Taylor realizó en la reparación a la que se le está calculando el precio final. Se supone que entre más comentarios hace Taylor en una reparación, es porque la reparación tuvo más pasos y por lo tanto fue más complicada.

- b) Calcular la cantidad total de horas utilizadas en cada reparación. Esto Taylor lo calcula de esta forma:

$TotalHoras = 2 \times FactorInstrumento \times CantidadComentarios$

34. Construye el modelo del dominio para este problema:

Don Andrés es el alcalde de la pequeña ciudad de Las Pulgas. En esta ciudad todos los fines de semanas se constituye una feria al aire libre, donde los productores de la zona ofrecen sus productos a los habitantes de la ciudad y turistas, principalmente frutas y verduras.

Como una forma de mejorar el comercio, atraer más turistas y, en general, crear un espacio más competitivo para que los clientes puedan comprar a los mejores precios, don Andrés ha ideado una forma centralizada de compra de productos, de forma de garantizar que siempre que una persona quiere comprar frutas y verduras, obtendrá el mejor precio.

Para realizar esto, don Andrés le ha pedido a los comerciantes que el día anterior a la feria, cada uno anote en un archivo (llamado oferta.txt) la lista de productos que va a vender, especificando el stock disponible (en kilogramos) y el precio de venta de cada kilogramo de dicho producto. Cada vendedor se identifica por un código único. Por ejemplo:

13,naranja,20,100
13,manzana,10,200
13,zanahoria,10,300

```
27,zanahoria,5,350
30,manzana,20,170
30,naranja,30,90
30,pepino,10,300
37,zanahoria,50,350
```

En este caso, la primera línea indica que el productor 13 venderá naranja, que además tiene 20 kilogramos disponibles a un precio de 100 el kilogramo. Nótese que en el archivo no hay combinaciones vendedor, producto repetidas (en otras palabras, por ejemplo, 13,naranja solo aparece una vez, pero el vendedor 13 puede aparecer muchas veces, lo mismo que naranja).

Don Andrés necesita desarrollar un programa que permita que las personas que compran en la feria indiquen el producto que quieren comprar y la cantidad de kilogramos que necesitan, de forma que siempre obtengan el menor precio.

Después de leer el archivo con los vendedores y sus productos, el programa debe presentar al usuario un pequeño menú de opciones, donde se pueda seleccionar la acción a realizar:

Ingrese una opción:

Q: Salir

C: Comprar un producto

R: Reporte

>

Si el usuario selecciona la opción Q entonces el programa debe terminar su ejecución.

Si el usuario selecciona la opción C entonces el programa debe preguntar el nombre del producto que se desea comprar y la cantidad de kilogramos que se está buscando:

Ingrese nombre del producto: naranja

Ingrese cantidad de kilos a comprar: 9

Una vez ingresados los valores que representan lo que el usuario quiere comprar, el programa debe buscar al vendedor que tenga stock del producto (o sea, que aun disponga de esa cantidad de kilogramos de producto) y que ofrezca el menor precio. Si hay más de un vendedor que cumpla la condición, se puede elegir cualquiera. Una vez ubicado ese vendedor, se debe realizar la compra e indicar por pantalla que la compra se realizó y con qué vendedor:

9 kilos de naranja comprados a vendedor 30

Obviamente, al realizar la transacción se le debe descontar al vendedor los kilogramos comprados por el usuario, de forma que las siguientes compras tomen esto en consideración.

En caso de que el usuario quiera comprar una cantidad de producto, pero no exista ningún vendedor con suficiente stock, se debe indicar la situación por pantalla:

Ingrese nombre del producto: naranja

Ingrese cantidad de kilos a comprar: 100

No se puede comprar 100 de naranja: ningún vendedor tiene stock

Si el usuario especifica un producto que no existe, entonces se cancela la compra y se debe volver al menú principal:

Ingrese una opción:

Q: Salir

C: Comprar un producto

R: Reporte

>C

Ingrese nombre del producto: sandía

Producto 'sandía' no existe!

Ingrese una opción:

Q: Salir

C: Comprar un producto

R: Reporte

>C

Ingrese nombre del producto: celular

Producto 'celular' no existe!

Ingrese una opción:

Q: Salir

C: Comprar un producto

R: Reporte

>C

Ingrese nombre del producto: pepino

Ingrese cantidad de kilos a comprar: 1

1 kilos de pepino comprados a vendedor 30

Si el usuario selecciona la opción R entonces el programa debe escribir por pantalla un resumen con las ventas realizadas en la feria:

Reporte de ventas:

Vendedor 30 vendió 9kg de naranja a 90

Vendedor 30 vendió 1kg de pepino a 300

O sea, por cada vendedor y por cada producto que ofrece, la cantidad de kilogramos vendidos, tal como se ven en el ejemplo. No es necesario ordenar el listado.

35. Construye el modelo del dominio para este problema:

En la cocinería “Doña Filomena” se preocupan de la salud de sus comensales, por lo que desde el año pasado han iniciado acciones para informar de mejor forma a sus clientes acerca de la calidad de las comidas que ofrecen.

Para tal efecto, doña Filomena ha empezado a planear diariamente los platos que ofrece, y en un archivo (oferta.txt) escribe el detalle de toda la oferta de un día, de la siguiente forma:

ALMUERZO1,entrada,ensalada normal,200

ALMUERZO1,fondo,arroz,150,guatitas,100

ALMUERZO1,postre,helado de vainilla,150

```
CENA1,fondo,arroz,200,asado,150
ALMUERZO2,entrada,ensalada chilena,300
ALMUERZO2,fondo,pure,200,asado,200
```

En este archivo primero aparece el nombre del servicio, después el plato (cada servicio se compone de varios “platos”), y finalmente la lista de componentes de dicho plato. Para cada componente, aparece su nombre y la cantidad de gramos que se usa.

Además, doña Filomena tiene un archivo (`componentes.txt`) donde están definidos los ingredientes que usa cada componente:

```
ensalada normal,lechuga,0.7,aceitunas,0.15,tomate,0.15
arroz,arroz,1
guatitas,guatitas,0.3,tomate,0.2,agua,0.3,arvejas,0.2
helado de vainilla,helado de vainilla,1
arroz,arroz,1
asado,carne,1
ensalada chilena,tomate,0.75,cebolla,0.25
pure,papa,0.75,leche,0.15,mantequilla,0.10
```

Cada línea de este archivo indica el componente, y los ingredientes necesarios para preparar el componente, especificando la proporción de cada ingrediente.

Acompañando a este archivo, doña Filomena también llena un archivo (`ingredientes.txt`) con los ingredientes que tiene disponibles:

```
pure,300
helado de vainilla,800
carne,250
cebolla,100
arroz,90
guatitas,170
lechuga,10
aceitunas,30
tomate,50
agua,0
arvejas,70
leche,80
mantequilla,230
```

Cada ingrediente indica además la cantidad de calorías por cada 100 gramos de alimento.

4

Interludio

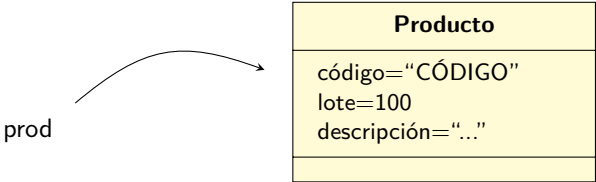
4.1. El misterio de las referencias en Java

Antes de continuar analizando otras técnicas de POO que son necesarias de aprender, para poder entenderlas apropiadamente, es necesario saber cómo funcionan las referencias en Java. Algo de este tema ya se vió anteriormente (por ejemplo, en la página la sección 1.4 en la página 52), pero acá lo revisaremos con más detalle.

Básicamente, lo que se quiere mostrar es que en Java, cuando se trabaja con objetos, *todo es una referencia*. Veamos este ejemplo:

```
1 public class Ejemplo3
2 {
3     public static void main(String[] args)
4     {
5         Producto prod = new Producto("CÓDIGO", 100, "DESCRIPCIÓN");
6     }
7 }
```

Este código lo único que hace es crear una instancia de Producto, y dejarla “apuntada” por una variable llamada prod. Gráficamente corresponde a:

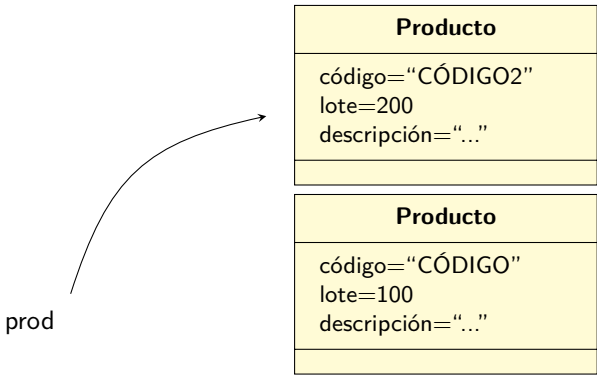


No hay que cometer el error de pensar que la variable `prod` de alguna forma “contiene” la instancia de `Producto` que se acaba de crear. La forma correcta de pensar en esta situación es que la instancia fue creada, por lo que existe en algún lugar de la memoria, y la variable lo único que hace es permitirnos llegar a la instancia para enviarle mensajes (recuerda al `this` en la página 110).

Algo que es totalmente válido, es cambiar la variable `prod` y hacer que referencie a “otra” instancia de la clase `Producto` (como la variable está definida con ese tipo, solamente puede referenciar instancias de esa clase):

```
1 public class Ejemplo3
2 {
3     public static void main(String[] args)
4     {
5         Producto prod = new Producto("CÓDIGO", 100, "DESCRIPCIÓN");
6
7         prod = new Producto("CÓDIGO2", 200, "DESCRIPCIÓN2");
8     }
9 }
```

En este caso, hasta la línea 5 lo que se tiene es exactamente igual que antes, pero después, en la línea 7, se crea otra instancia de `Producto` y se hace que la variable `prod` la referencie (o sea, estamos haciendo que apunte a esta nueva instancia):



A partir de ese momento, todos los mensajes enviados usando la variable `prod` llegarán a esa nueva instancia.

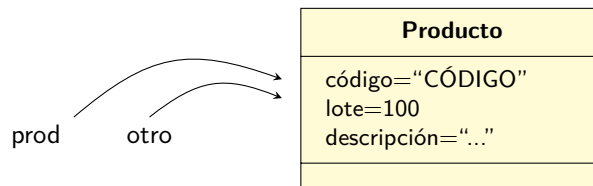
¿Y qué pasa con la instancia que dejó de estar apuntada por la variable? En este caso, debido a que esa instancia dejó de estar referenciada por alguna variable, entonces es una instancia inalcanzable, que ya no podrá ser utilizada, y en poco tiempo el *recolector de basura* reclamará el espacio que ocupa la instancia para uso futuro. Al contrario que en otros lenguajes, en **Java** cuando se termina de utilizar una instancia, simplemente podemos olvidarnos de ella y el recolector de basura hará su trabajo. En otros lenguajes la tarea de liberar la memoria utilizada es un paso que se debe realizar de forma explícita, muchas veces invocando un *destructor* de la clase.¹

¿Qué hacemos si no queremos que la otra instancia sea “olvidada”? Entonces tenemos que mantenerla apuntada, de alguna forma:

¹Si quieres conocer algunos detalles extras de cómo funciona el recolector de basura, puedes ver el anexo B

```
1 public class Ejemplo3
2 {
3     public static void main(String[] args)
4     {
5         Producto prod = new Producto("CÓDIGO", 100, "DESCRIPCIÓN");
6
7         Producto otro = prod;
8
9         prod = new Producto("CÓDIGO", 100, "DESCRIPCIÓN");
10    }
11 }
```

¿Qué significa la línea 7? Tal como se dice más arriba, si tu esquema mental es que las variable `prod` de alguna forma “contiene” la instancia, entonces la línea no tendrá ningún sentido para ti (o no tendrá el sentido correcto). Lo que está sucediendo es que hasta la línea 7, tendremos esta situación:



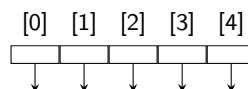
Ambas variables apuntarán a la misma instancia. Ésto no significa que ambas variables “tengan” cada una una copia de la instancia, sino que *literalmente* ambas variables referencian a la misma cosa.

Ésto significa que tanto si usamos la variable `prod` o la variable `otro` para enviar un mensaje, la que recibirá dicho mensaje es la misma instancia. Estaremos afectando y cambiando el estado de la misma cosa.

Generalmente cuando se crean muchas instancias de las clases, lo común es almacenarlas todas juntas en alguna estructura, y la estructura más usada es un arreglo. Por ejemplo:

```
1 public class Ejemplo3
2 {
3     public static void main(String[] args)
4     {
5         Producto[] productos = new Producto[5];
6     }
7 }
```

En este caso, es importante recalcar que este código **lo único que está haciendo** es crear un arreglo que en el futuro podrá guardar *referencias* a instancias de la clase `Producto`. Es equivalente a tener lo siguiente:



Cada celda del arreglo podría apuntar a una instancia de `Producto`, pero actualmente todas están apuntando a null (hay que recordar el concepto de *valor por defecto* que aparece en la página 112). Comprobémoslo:

```
1 public class Ejemplo3
2 {
3     public static void main(String[] args)
4     {
5         Producto[] productos = new Producto[5];
6
7         System.out.println(productos[2]);
8     }
9 }
```

y se obtiene:

```
$ java Ejemplo3
null
```

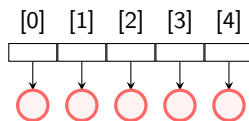
Si hacemos algo diferente:

```
1 public class Ejemplo3
2 {
3     public static void main(String[] args)
4     {
5         Producto[] productos = new Producto[5];
6
7         for(int i=0;i<5;i++) {
8             productos[i] = new Producto("C" + i, 10 * i, "D" + i);
9         }
10
11         System.out.println(productos[2].getCódigo());
12     }
13 }
```

Se obtiene algo diferente:

```
$ java Ejemplo3
C2
```

Lo cuál gráficamente es equivalente a:



O sea, cada posición del arreglo apunta a una instancia diferente de Producto. Pero no necesariamente tiene que ser siempre así:

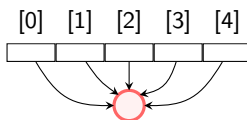

```
1 public class Ejemplo3
2 {
3     public static void main(String[] args)
4     {
5         Producto[] productos = new Producto[5];
6
7         Producto uno = new Producto("C", 10, "D");
8
9         for(int i=0;i<5;i++) {
10             productos[i] = uno;
11         }
12
13         System.out.println(productos[2].getCódigo());
14         System.out.println(productos[4].getCódigo());
15     }
16 }
```

Se obtiene:

```
$ java Ejemplo3
```

```
C
C
```

Y gráficamente:



En general, una variable puede apuntar a cualquier instancia (o a `null`). Por otro lado, no hay límite en la cantidad de variables que pueden estar apuntando a una instancia en un momento dado.

Lo que sí nunca es válido, es enviarle mensajes a `null`, sino vamos a obtener una `NullPointerException` que vimos en la página 112.

4.2. Comparando objetos

Algo que vamos a realizar muchas veces al crear software es comparar objetos para determinar si se tratan de lo mismo o no.

En Java, hay que tener un cuidado especial al comparar elementos, ya que dependiendo de si se trata de valores primitivos o clases, la forma en que se hace la comparación es diferente, o mejor dicho, lo que se quiere comparar no es lo que uno cree que está comparando.

Cuando se trata de valores primitivos, podemos hacer lo que siempre se ha hecho:

```
1 public class Ejemplo4
2 {
3     public static void main(String[] args)
4     {
5         int uno = 1;
6         int dos = 2;
7         int otroUno = 1;
8
9         if (uno == dos) {
10             System.out.println("Uno es igual a dos??");
11         }
12         if (uno == otroUno) {
13             System.out.println("Uno es igual a uno");
14         }
15     }
16 }
```

Al ejecutar este programa, obtendremos lo que esperamos de forma normal:

```
$ java Ejemplo4
Uno es igual a uno
```

En el caso de objetos, veamos qué pasa:

```
1 public class Ejemplo4
2 {
3     public static void main(String[] args)
4     {
5         Producto a = new Producto("C", 10, "D");
6         Producto b = new Producto("C", 10, "D");
7
8         if (a == b) {
9             System.out.println("Son iguales!");
10        } else {
11            System.out.println("No son iguales!");
12        }
13    }
14 }
```

Lo que se obtiene es:

```
$ java Ejemplo4
No son iguales!
```

¿Cómo podemos explicar ésto?

La respuesta es que como se analizó antes (página 52), las referencias a fin de cuentas no son nada más que números, que se interpretan como un objeto que está en la dirección de memoria indicada por la referencia. Es por eso que podemos pasar una referencia como un parámetro y podemos modificar el estado de la instancia apuntada.²

En el caso de la línea 8, el `if` está preguntando si ambas variables están apuntando a la misma instancia (o lo que es lo mismo, si ambas referencias tienen el mismo valor). Como eso es falso (ya que cada variable apunta a su propia instancia de `Producto`), entonces se entra al `else`.

Considera el siguiente código:

```
1 public class Ejemplo4
2 {
3     public static void main(String[] args)
4     {
5         Producto a = new Producto("C", 10, "D");
6         Producto b = new Producto("C", 10, "D");
7         Producto c = b;
8
9         if (c == b) {
10             System.out.println("Son iguales!");
11         } else {
12             System.out.println("No son iguales!");
13         }
14     }
15 }
```

En la línea 7 se está haciendo que ambas referencias apunten a la misma instancia. Y es por eso que cuando se llega al `if` en la línea 9, la expresión es verdadera:

```
$ java Ejemplo4
Son iguales!
```

Este comportamiento será útil algunas veces, por lo que es importante recordarlo. Pero también sería útil tener alguna forma de comparar los objetos para ver si son iguales desde el punto de vista “lógico”, o mejor dicho, ver si es que diferentes objetos representan la misma situación o condición.

Por ejemplo, en el último código, dependiendo del problema, uno podría considerar que ambas instancias de `Producto` son iguales, ya que se construyeron usando los mismos parámetros, ambas tienen el mismo código, el mismo número de lote y la misma descripción. Podría ser que aunque son instancias diferentes, al final representan la misma cosa en la realidad.

En Java ésto se puede realizar, y para eso usaremos un método que todos los objetos pueden tener, y que se llama `equals`. Este método debe ser creado en cada clase que consideremos importante, y nos permitirá escribir código como este (en este ejemplo, se usarán instancias de `String` para ejemplificar mejor el caso):

²Parece un trabalenguas, pero es exactamente como funciona.

```
1 public class Ejemplo5
2 {
3     public static void main(String[] args)
4     {
5         String s1 = new String("hola");
6         String s2 = new String("hola");
7
8         if (s1 == s2) {
9             System.out.println("Son físicamente iguales");
10        }
11        if (s1.equals(s2)) {
12            System.out.println("Son lógicamente iguales");
13        }
14    }
15 }
```

Al ejecutar este código, se obtiene:

```
$ java Ejemplo5
Son lógicamente iguales
```

Nótese cómo, a pesar de ser instancias diferentes, debido a que contienen el mismo texto, se considera que ambos strings son iguales.

En nuestro caso, a partir del análisis del problema debemos obtener los criterios que determinarán las condiciones de igualdad de nuestros objetos. Por ejemplo, si en el caso de `Producto`, basta con que el código y la descripción sean iguales para considerar que dos instancias representan el mismo elemento en la realidad, podemos escribir nuestro propio método `equals`:

```
1 public class Producto
2 {
3     private String código;
4     private int lote;
5     private String descripción;
6
7     public Producto(String elCódigo, int elLote, String laDescripción)
8     {
9         código = elCódigo;
10        lote = elLote;
11        descripción = laDescripción;
12    }
13
14    public String getCódigo() { return código; }
15    public int getLote() { return lote; }
16    public String getDescripción() { return descripción; }
17
18    public boolean equals(Object elOtro)
19    {
20        if (this == elOtro)
21            return true;
22        if (elOtro == null)
23            return false;
24        if (getClass() != elOtro.getClass())
25            return false;
26        Producto other = (Producto) elOtro;
27
28        if (código == null) {
29            if (other.código != null)
30                return false;
```

```

31     } else if (!código.equals(other.código))
32         return false;
33
34     if (descripción == null) {
35         if (other.descripción != null)
36             return false;
37     } else if (!descripción.equals(other.descripción))
38         return false;
39
40     return true;
41 }
42 }

```

Se ve amenazador, ¿cierto? Este es el método `equals` que generó la IDE *Eclipse*, pero si te das cuenta, la mayoría del código existe para tratar de asegurarse de que las condiciones son las correctas para poder comparar los atributos de las dos instancias siendo comparadas. A partir de la línea 28 se comienzan a hacer las comparación, e incluso se toma en cuenta cuando los atributos pueden ser `null`. Fíjate que en la línea 26 se hace casting para convertir un objeto en `Producto`, de forma muy similar al casting de tipos primitivos que se vió en la página 16. Una explicación más completa del casting al usar objetos se verá en la página 210.

Si ponemos a prueba el código:

```

1 public class Ejemplo4
2 {
3     public static void main(String[] args)
4     {
5         Producto a = new Producto("C", 10, "D");
6         Producto b = new Producto("C", 10, "D");
7
8         if (a == b) {
9             System.out.println("Son físicamente iguales!");
10        } else {
11            System.out.println("No físicamente son iguales!");
12        }
13
14        if (a.equals(b)) {
15            System.out.println("Son lógicamente iguales");
16        } else {
17            System.out.println("No son lógicamente iguales");
18        }
19    }
20 }

```

Obtenemos lo siguiente:

```

$ java Ejemplo4
No físicamente son iguales!
Son lógicamente iguales

```

Finalmente, tal vez el análisis del problema nos permita saber cosas que la IDE *Eclipse* no sabe, y podremos optimizar el cuerpo del método `equals`:

```
1 public class Producto
2 {
3     ...
4     public boolean equals(Object elOtro)
5     {
6         Producto other = (Producto) elOtro;
7
8         if (!código.equals(other.código))
9             return false;
10
11        if (!descripción.equals(other.descripción))
12            return false;
13
14        return true;
15    }
16 }
```

Obtenemos lo siguiente:

```
$ java Ejemplo4
No físicamente son iguales!
Son lógicamente iguales
```

4.3. Inicialización de atributos de instancia

Aprovechemos que existe la clase `Producto` para revisar una técnica que puede ser útil en el futuro. Creemos la clase imaginaria `Venta`, que siempre tiene 3 instancias de `Producto`:

```
1 public class Venta
2 {
3     private Producto p1;
4     private Producto p2;
5     private Producto p3;
6
7     public Venta(int codProd1, int codProd2, int codProd3)
8     {
9         p1 = new Producto(codProd1);
10        p2 = new Producto(codProd2);
11        p3 = new Producto(codProd3);
12    }
13 }
```

En este caso estamos describiendo a una clase que para poder ser instanciada requiere obligatoriamente que se le pasen 3 parámetros, y con los valores que le llegan, instanciará los productos que seguramente después usará en algún proceso posterior.

Por otro lado, a veces nos encontraremos en esta otra situación:

```
1 public class UnComputador
2 {
3     private CPU cpu;
4     private Pantalla pantalla;
5
6     public UnComputador()
7     {
8         this.cpu = new CPU();
9         this.pantalla = new Pantalla();
10    }
11 }
```

Este caso es diferente al anterior, ya que la clase solo posee un constructor simple, muy parecido al constructor por defecto, pero de igual manera se usa para inicializar sus atributos con los valores requeridos. A veces, para evitarnos crear el constructor, será más fácil escribir esta clase de la siguiente manera:

```
1 public class UnComputador
2 {
3     private CPU cpu = new CPU();
4     private Pantalla pantalla = new Pantalla();
5 }
```

Funcionalmente, las dos últimas versiones son equivalentes: al momento de instanciar un nuevo objeto UnComputador, sus atributos se inicializarán correctamente, pero con la última versión escribiremos menos código.

4.4. Ejercicios

1. ¿Por qué se dice que “todo es una referencia”?
2. ¿Es verdad que “todo es una referencia”? ¿Hay casos en que no es verdad?
3. Crea un arreglo, donde cada elemento del arreglo referencie a una instancia diferente de `Producto`. Ahora, crea una función que “invierta” el arreglo.
4. Crea un programa que te permita internalizar cómo funciona el operador `==`: primero para variables “primitivas”, y después para instancias de clases.
5. Continuando con la pregunta anterior, ¿te parece correcto el resultado de ejecutar este código?:

```

1 public class TS
2 {
3     public static void main(String[] args)
4     {
5         String a = "hola";
6         String b = "hola";
7
8         System.out.println(a == b);
9
10        a = new String("hola");
11        b = new String("hola");
12
13        System.out.println(a == b);
14    }
15 }

```

```

$ java TS
true
false

```

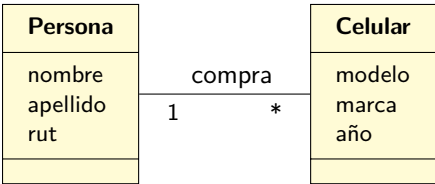
Investiga el por qué de este comportamiento.

6. Crea varias versiones de una clase `Persona`, con diferentes implementaciones del método `equals(...)`, y construye un programa que ponga a prueba las implementaciones, con diferentes casos de acuerdo a cómo esperas que se comporte la clase.
7. Crea una clase `Mascota` que referencie a una instancia de la clase `Persona` para identificar a la persona que es dueña de la mascota. Al instanciar una mascota, ¿es necesario inicializar inmediatamente su atributo `dueño`? Piensa en el tipo de problemas donde las mascotas pueden existir sin dueño. Piensa en los problemas donde es obligatorio que al instanciar una mascota se tenga que especificar su dueño. Agrega un constructor a `Mascota` para impedir que se pueda instanciar sin especificar su dueño.
8. Modifica el ejemplo en la página 133 para que después de los últimos dos `println(...)`, llames a `setCódigo(...)` para cambiarle el código a la instancia de `Producto` ubicado en el índice 1. Después, agregar otro `println` para escribir el código de la instancia ubicada en el índice 2. ¿Es lógico lo que sucede?

5

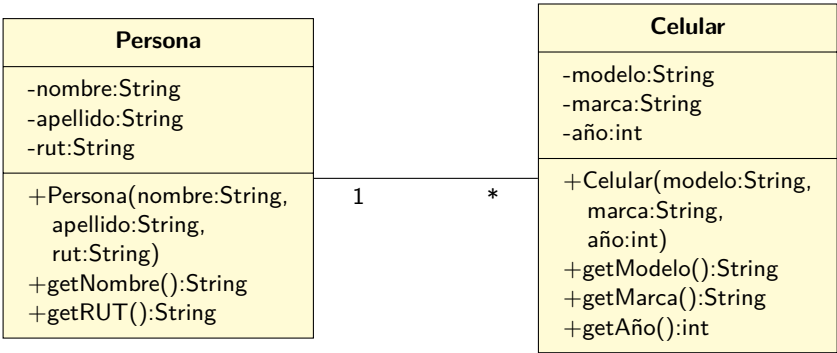
Colecciones

Al analizar los problemas, es muy posible que se hayan detectado que algunos elementos del modelo del dominio se relacionan de la siguiente forma:

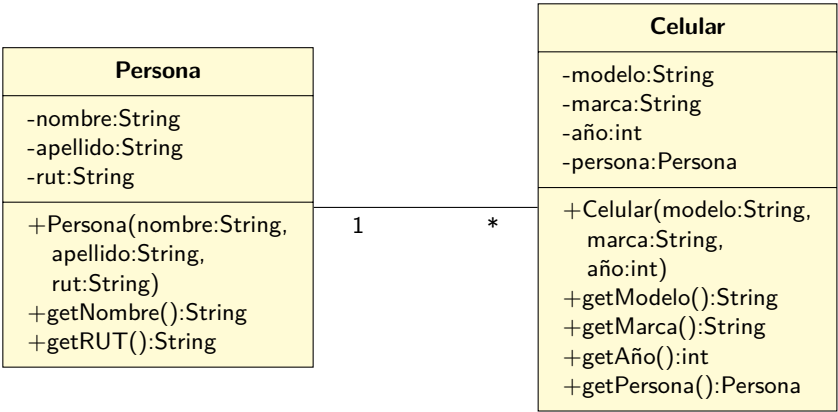


En otras palabras, nos referiremos a relacionamientos en que la multiplicidad es mayor a uno. En este caso, un elemento del lado izquierdo se relaciona con varios (*) elementos del lado derecho.

Como se puede esperar, en algún momento este modelo se transformará en un diagrama de clases, y es probable que su primera versión será similar a esto:

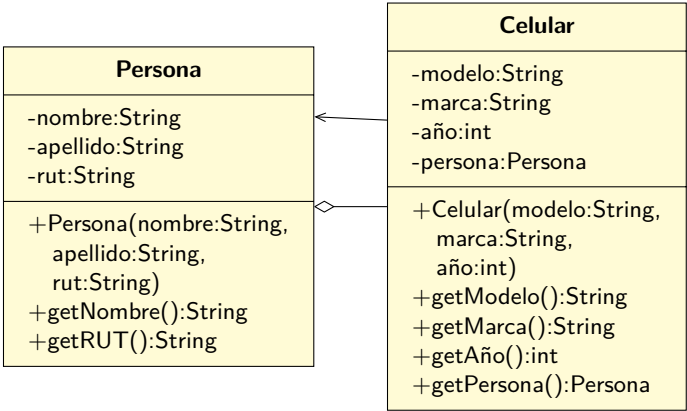


Lo que le falta a este diagrama, y que usaremos explícitamente en este libro, es indicar los atributos que permitirán mantener el relacionamiento. Por un lado tenemos **1**, entonces el **Celular** necesita algo para referenciar a su **Persona**:



Nótese que de acuerdo al diagrama, un Celular siempre está relacionado con una Persona (la multiplicidad es 1, no es * ni 0..1), así que debemos tomar los resguardos necesarios para mantener esa regla.

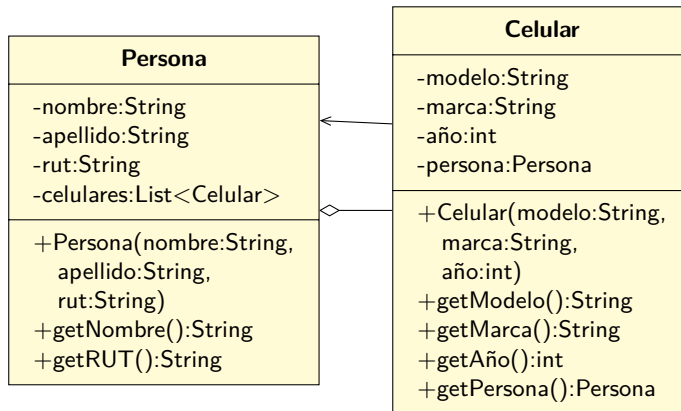
Por el otro lado tenemos *, y desde ahora en adelante vamos a diagramar la situación de la siguiente forma:



Desde ahora en adelante, cada vez que exista una asociación con multiplicidad mayor a 1, a la clase que es “dueña” de los elementos le agregaremos una “colección” para poder mantener referenciadas a las instancias de la clase dependiente (en este caso, **Persona** tendrá un nuevo atributo). Antes de ver la nueva versión del diagrama de clases, es necesario decidir el tipo de dato que se usará para representar la “colección”.

En general existen varias opciones, pero en este libro usaremos una de las más simples: el tipo **List**.

Dado lo anterior, el diagrama de clases final quedará de la siguiente forma:



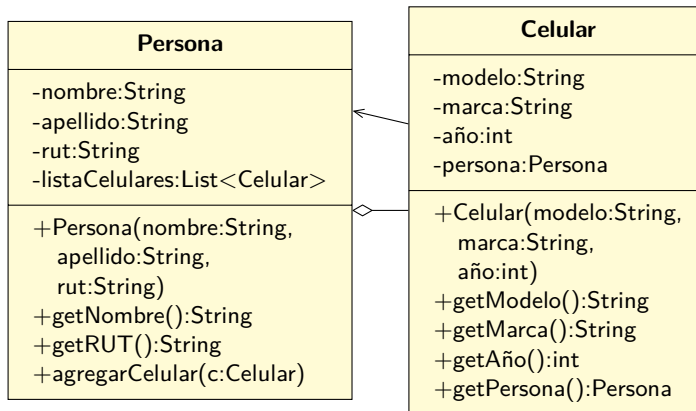
Si te preguntas qué significa `List<Celular>`, es algo que se revisará un poco más adelante (en la página 148), pero por ahora vamos a decir que la variable `celulares` es una variable que nos permitirá referenciar a muchas instancias de la clase `Celular`.

En otras palabras, `celulares` se comportará como una “colección” de `Celulares`, y su tipo concreto es `List`.

Una `List` es una clase interna de Java que nos provee con las operaciones necesarias para almacenar una serie de elementos de un mismo tipo, similar a cómo lo hace un arreglo, pero con operaciones de más alto nivel, que permitirán que la clase “dueña” del contenedor no tenga que preocuparse de la administración de los datos. Por ejemplo, algunas operaciones que las `List` tienen son:

- Construir una nueva lista, ya sea vacía, o copiando las referencias desde otra lista.
- Agregar una nueva instancia del elemento a la lista.
- Buscar una instancia, de acuerdo a algún criterio.
- Eliminar una instancia, buscándola de acuerdo a algún criterio.

Además de lo anterior, será necesario agregar explícitamente los métodos necesarios a la clase “dueña” de la relación para poder administrar los elementos de la lista. En particular, si el problema así lo requiere, la clase `Persona` puede tener un método `agregarCelular(...)` para permitir que se le agregue un nuevo `Celular` a una persona:



5.1. Implementación

Comencemos con la clase **Celular**:

```

1 public class Celular
2 {
3     private String modelo, marca;
4     private int año;
5     private Persona persona;
6
7     public Celular(String modelo, String marca, int año)
8     {
9         this.modelo = modelo;
10        this.marca = marca;
11        this.año = año;
12    }
13
14    public String getModelo() { return modelo; }
15    public String getMarca() { return marca; }
16    public int getAño() { return año; }
17    public Persona getPersona() { return persona; }
18    public void setPersona(Persona p) { this.persona = p; }
19    public String toString() { return this.marca + "/" + this.modelo + "/" + this.año; }
20 }

```

Ahora veamos la clase **Persona**:

```

1 public class Persona
2 {
3     private String nombre;
4     private List<Celular> listaCelulares;
5
6     public Persona(String nombre)
7     {
8         this.nombre = nombre;
9     }
10
11    public void agregarCelular(Celular celular)
12    {
13        // Este es la forma normal de agregar un nuevo elemento al "final" de la lista
14        this.listaCelulares.add(celular);
15    }
16 }

```

```

15         celular.setPersona(this); // Esto es importante!
16     }
17
18     public String getNombre()
19     {
20         return nombre;
21     }
22
23     public String toString()
24     {
25         return nombre + ": " + listaCelulares.toString();
26     }
27 }
28

```

Y poniendo todo a prueba:

```

1 public class TestCelular
2 {
3     public static void main(String[] args)
4     {
5         Persona p1 = new Persona("Ximena");
6
7         Celular c1 = new Celular("A123", "XinC", 2022);
8         Celular c2 = new Celular("C9", "XinC", 2021);
9         Celular c3 = new Celular("P77", "Sanqo", 2022);
10
11         p1.agregarCelular(c1);
12         p1.agregarCelular(c2);
13         p1.agregarCelular(c3);
14
15         System.out.println(p1);
16     }
17 }

```

Al ejecutar el código se obtiene:

Exception in thread "main" java.lang.NullPointerException:

```

    Cannot invoke "java.util.List.add(Object)" because "this.listaCelulares" is null
    at Persona.agregarCelular(Persona.java:14)
    at TestCelular.main(TestCelular.java:11)

```

¿Qué sucedió? Lo que sucedió es que olvidamos un paso muy importante: al crear la instancia de **Persona** en la línea 5, y después llamar a **agregarCelular(...)** en la línea 11, la **listaCelulares** de esa persona tiene valor **null**, ya que nunca fue inicializada en el constructor. Lo correcto es que dicho atributo debe ser inicializado, por lo que modificaremos el constructor de esta manera:

```

1 public class Persona
2 {
3     ...
4
5     public Persona(String nombre)
6     {
7         this.nombre = nombre;
8         this.listaCelulares = new ArrayList<>();
9     }
10
11     ...
12 }

```

Ahora sí, al ejecutar el programa se obtiene:

```
Ximena: XnC/A123/2022,XnC/C9/2021,Sanqo/P77/2022
```

Todo está correcto hasta ahora, pero seguramente la línea `this.listaCelulares = ...` te parece extraña. Por ahora, y hasta que lleguemos al capítulo 7.12, usaremos las siguientes reglas al implementar los relacionamientos que tengan “N”:

1. En la clase que es “dueña” del relacionamiento, el tipo de la variable será `List<Algo>`, siendo `Algo` el tipo concreto de la clase dependiente.
2. Se inicializará el atributo usando `atributo = new ArrayList<>()`; en el constructor de la clase “dueña”

Compliquemos un poco el ejemplo, para ponerlo a prueba:

```
1 public class TestContenedorCelular
2 {
3     public static void main(String[] args)
4     {
5         Persona p1 = new Persona("Ximena");
6
7         Celular c1 = new Celular("A123", "XnC", 2022);
8         Celular c2 = new Celular("C9", "XnC", 2021);
9         Celular c3 = new Celular("P77", "Sanqo", 2022);
10
11         p1.agregarCelular(c1);
12         p1.agregarCelular(c2);
13         p1.agregarCelular(c3);
14
15         System.out.println(p1);
16
17         Persona p2 = new Persona("Fran");
18
19         p2.agregarCelular(c2); // El celular que era de Ximena ahora es de Fran
20
21         System.out.println();
22         System.out.println(p1);
23         System.out.println(p2);
24     }
25 }
```

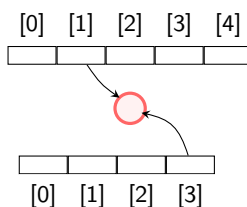
Al ejecutar el nuevo ejemplo se obtiene:

```
Ximena: XnC/A123/2022,XnC/C9/2021,Sanqo/P77/2022
```

```
Ximena: XnC/A123/2022,XnC/C9/2021,Sanqo/P77/2022
```

```
Fran: XnC/C9/2021
```

Hay un problema: aun cuando efectivamente `Fran` tiene el celular asignado, también aparece en la lista de `Ximena`. Esto pasó ya que dicho celular aun está en la lista de celulares de `Ximena`. Básicamente, el mismo celular está siendo referenciado por ambas colecciones.



Hay varias formas de solucionar ésto. Por un lado, podríamos agregar una llamada a un método nuevo `removeCelular` cada vez que invoquemos a `agregarCelular`:

```

1 public class TestContenedorCelular
2 {
3     public static void main(String[] args)
4     {
5         Persona p1 = new Persona("Ximena");
6
7         Celular c1 = new Celular("A123", "XinC", 2022);
8         Celular c2 = new Celular("C9", "XinC", 2021);
9         Celular c3 = new Celular("P77", "Sanqo", 2022);
10
11         p1.agregarCelular(c1);
12         p1.agregarCelular(c2);
13         p1.agregarCelular(c3);
14
15         System.out.println(p1);
16
17         Persona p2 = new Persona("Fran");
18
19         p1.removeCelular(c2); // <----
20         p2.agregarCelular(c2); // El celular que era de Ximena ahora es de Fran
21
22         System.out.println();
23         System.out.println(p1);
24         System.out.println(p2);
25     }
26 }

```

Pero ésto significa que para no cometer ningún error, se tendría que hacer dicho llamado cada vez que se invoca a `agregarCelular`, en todos los casos (en las líneas 11, 12 y 13). Además, siempre se tendría que saber quién es la instancia dueña del celular, por lo que la llamada tendría que ser algo como:

```

1 c2.getPersona().removeCelular(c2);
2 p2.agregarCelular(c2);

```

Pero además, en nuestro ejemplo los `Celulares` inicialmente están sin dueño, así que técnicamente tendríamos que escribir:

```

1 if (c2.getPersona() != null) {
2     c2.getPersona().removeCelular(c2);
3 }
4 p2.agregarCelular(c2);

```

Y habría que escribir lo mismo cada vez que se agrega un `Celular` a una `Persona`.

Probemos haciendo algo diferente. Modificando la clase `Persona`:

```

1 public class Persona
2 {
3     ...
4
5     public void agregarCelular(Celular celular)
6     {
7         this.listaCelulares.add(celular);
8
9         // Si el celular era de alguien, se lo quitamos :-D
10        if (celular.getPersona() != null)
11            celular.getPersona().removerCelular(celular);
12
13        celular.setPersona(this);
14    }
15
16    private void removerCelular(Celular celular)
17    {
18        this.listaCelulares.remove(celular);
19    }
20
21    ...
22 }

```

Nótese cómo el método `removerCelular` es privado. Por ahora es lo que se necesita para que el proceso funcione.

Nótese también que aun cuando `Persona` tiene una lista para mantener a todos sus `Celulares` referenciados, dicha lista nunca es expuesta como público. Todas las operaciones para mantener el relacionamiento (`agregarCelular(...)`) son parte de la interfaz pública de `Persona`, por lo que tiene todo el control de lo que sucede cuando se quiere asociar un nuevo `Celular`. La verdad es que la lista es un detalle de implementación que no debería ser público, ya que si se deja público cualquier cliente podría agregar nuevos `Celulares` a la lista y la `Persona` no se enteraría:

```

1 public class TestContenedorCelular
2 {
3     public static void main(String[] args)
4     {
5         Persona p2 = ...;
6         List<Celular> celulares = p2.getCelulares();
7         celulares.add(new Celular("X", "X", 0)); // Malo malo!
8     }
9 }

```

Tal vez la clase `Persona` realiza algún otro proceso cada vez que recibe un celular. Tal vez los contabiliza, o selecciona el que más le gusta, o como en el ejemplo anterior, le “roba” el celular a su dueña anterior. Todo esto no se podría realizar si se accede directamente al relacionamiento sin pasar por el dueño de dicho relacionamiento.

5.2. Genéricos

En Java, desde hace muchos años que existe la posibilidad de escribir clases que puedan operar sobre diferentes elementos, pero sin saber el tipo concreto hasta que la clase es instanciada. Por ejemplo, una lista de números, una lista de personas, una lista de mascotas, etc. En términos concretos, seguramente todas esas listas tendrían las mismas operaciones, y los algoritmos implementados seguramente serían

iguales, y la única diferencia es que en un caso el tipo de dato almacenado será un `String`, o `Persona`, etc.

En `Java` a este concepto se le llama *genéricos*¹.

La forma en que opera esta técnica es que al momento de declarar una clase, ésta se declara indicando que el tipo concreto que va a manipular (por ejemplo, la clase `Persona`) es un tipo que solo se va a saber cuando se cree la instancia.

Veamos un ejemplo. Imaginemos que necesitamos una clase para guardar pares de instancias. Pueden ser pares de `Persona`, pares de `Celular`, pares de `Mascota`, etc. En condiciones normales, se tendría que crear algo así:

```
1 public class ParMascota
2 {
3     private Mascota primero;
4     private Mascota segundo;
5
6     public ParMascota(Mascota a, Mascota b)
7     {
8         this.primero = a;
9         this.segundo = b;
10    }
11
12    public Mascota getPrimero() { return primero; }
13    public Mascota getSegundo() { return segundo; }
14 }
```

Y usarlo de esta forma:

```
1 public class Test
2 {
3     public static void main(String[] args)
4     {
5         ParMascota par = new ParMascota(new Mascota("A"), new Mascota("B"));
6
7         System.out.println(par.getSegundo().getNombre());
8     }
9 }
```

Y si se quisiera tener un `ParPersona`, habría que crear una clase equivalente, cuya única diferencia sería que el tipo ya no sería `Mascota` sino que `Persona`. Pero si usamos generics, podemos escribir lo siguiente:

```
1 public class Par<T>
2 {
3     private T primero;
4     private T segundo;
5
6     public Par(T a, T b)
7     {
8         this.primero = a;
9         this.segundo = b;
10    }
11
12    public T getPrimero() { return primero; }
13    public T getSegundo() { return segundo; }
14 }
```

¹generics en inglés.

Nótese que en ninguna parte de la clase aparece ni `Mascota` ni `Persona`, y que en todos los lugares donde se utilizaba el tipo, se ha reemplazado por un nombre especial llamado `T`.

Al escribir `<T>` al lado del nombre de la clase se está definiendo que ésta requiere de un dato extra al instanciarse. Y que al instanciarse, el tipo `T` será reemplazado por el tipo de la clase concreta. En este caso, para poder instanciar un `Par<T>` se tiene que escribir lo siguiente:

```
1 public class Test
2 {
3     public static void main(String[] args)
4     {
5         Par<Mascota> par = new Par<Mascota>(new Mascota("A"), new Mascota("B"));
6
7         System.out.println(par.getSegundo().getNombre());
8     }
9 }
```

Como se puede ver, al momento de instanciar `Par`, se está indicando que el tipo `T` corresponderá en la realidad al tipo `Mascota`, y todo el resto de la clase `Par` continuará funcionando normalmente (solo que en todo lugar donde decía `T`, ahora será `Mascota`).

Obviamente también se puede hacer lo siguiente:

```
1 public class Test
2 {
3     public static void main(String[] args)
4     {
5         Par<Mascota> par1 = new Par<Mascota>(new Mascota("A"), new Mascota("B"));
6         Par<Persona> par2 = new Par<Persona>(new Persona(22), new Persona(31));
7
8         System.out.println(par1.getSegundo().getNombre());
9         System.out.println(par2.getPrimero().getEdad());
10    }
11 }
```

En otras palabras, sin realizar ninguna modificación, ahora tenemos un `Par` que en vez de tener `Mascotas`, ahora contiene `Personas`. Nótese que como los métodos `getPrimero()` y `getSegundo()` están definidos como retornando `T`, entonces al invocarlos en las líneas 8 y 9 lo que retornan es el tipo concreto (`Mascota` y `Persona`), por lo que directamente se puede invocar `getNombre()` y `getEdad()`, que son métodos de `Mascota` y `Persona` respectivamente.

Esta técnica efectivamente nos permitirá escribir clases genéricas, que pueden operar sobre una amplia variedad de tipos, los que solo serán especificados al momento de instanciar la clase.

Pero existen algunas restricciones que es necesario conocer al convertir las clases para que usen genéricos.

La principal es que una clase que use genéricos no podrá crear nuevas instancias del tipo genérico, o arreglos del tipo. Por ejemplo:

```
1 public class BadList<E>
2 {
3     public void algo()
4     {
5         E[] arreglo = new E[100]; // Cannot create a generic array of E
6
7         E inst = new E(); // Cannot instantiate the type E
8     }
9 }
```

5.3. Generalización de Colecciones

Ahora podemos aclarar el por qué estábamos escribiendo las cosas de la manera en que las estábamos escribiendo.

En el caso del relacionamiento N-a-1, en el lado de la clase “dueña”, declararemos una colección de tipo `List`, indicando el tipo específico que almacenaremos en dicha lista. En el caso anterior, eran instancias de `Celular`.

Además, en el constructor de la clase “dueña” tendremos que instanciar la lista, pero aunque usaremos `ArrayList`, es interesante también saber que en la práctica en `Java` tenemos dos opciones para hacer eso:

`LinkedList` Corresponde a una lista doblemente enlazada, que contiene todas las operaciones definidas en la interfaz `List`.

`ArrayList` Corresponde a una clase que también implementa la interfaz `List`, pero implementada internamente con arreglos.

Dentro de las operaciones más interesantes, que ambas tienen, se encuentran:

<code>int size()</code>	Retorna la cantidad de elementos almacenados en la lista
<code>boolean isEmpty()</code>	Indica si la lista está vacía o no
<code>boolean contains(...)</code>	Indica si un elemento está en la lista, haciendo comparaciones lógicas usando <code>equals</code>
<code>boolean add(E e)</code>	Agrega un nuevo objeto a la lista
<code>void clear()</code>	Remueve todos los elementos de la lista
<code>E get(int index)</code>	Retorna el elemento que está en la posición <code>index</code> en la lista

Para más detalles respecto a la interfaz `List`, se puede ver <https://docs.oracle.com/javase/8/docs/api/java/util/List.html>.

Una ventaja de usar las colecciones de esta forma es que el atributo que mantendrá la lista de celulares de la persona se ha declarado de forma genérica (al usar `List`). Y recién en el constructor se instancia el tipo concreto de lista que se usará. En otras palabras, se guarda como un detalle de implementación el tipo de lista, y todo se programa contra la interfaz, que es uno de los principios **SOLID** que se ven en capítulo 8.

5.3.1. Iteradores

Una de las principales acciones que se realizan al usar colecciones, es la iteración sobre todos los elementos guardados en dicha colección.

Muchas veces, las colecciones proveen métodos que permiten realizar esa operación:

```

1 public class Test
2 {
3     public static void main(String[] args)
4     {
5         int n = lista.size();
6         for(int i=0;i<n;i++) {
7             String elemento = lista.get(i);
8             System.out.println(elemento);
9         }
10    }
11 }

```

El principal problema de utilizar este enfoque es que asume que el acceso al dato que está en la posición `i` es instantáneo: en cada iteración del ciclo, se pide un elemento, y no hay registro de lo que se realizó en la iteración anterior. Ésto puede resultar bien en el caso de colecciones implementador con arreglos, pero en el caso de otro tipo de colecciones (por ejemplo, implementados con listas enlazadas, o de alguna otra forma que podríamos considerar “lenta”), el hecho de invocar `lista.get(i)` puede traducirse en que siempre se comience desde el inicio de la lista, hasta llegar al elemento buscado. A fin de cuentas, el código va a pasar mucho más tiempo moviéndose en las listas que realizando otras tareas que son las que se quieren implementar dentro del ciclo `for`.

Una solución a lo anterior es crear algún tipo de estructura que nos permita mantener una “memoria” de dónde vamos en la iteración, de forma que cuando se quiera realizar el movimiento a la siguiente posición, dicho movimiento sea “barato”, y no se tengan que realizar grandes esfuerzos en operaciones que se pueden ahorrar.

Por ejemplo, consideremos este código:

```

1 public class Test
2 {
3     public static void main(String[] args)
4     {
5         testLinkedList();
6         testArrayList();
7     }
8
9     private static void testArrayList()
10    {
11        ArrayList<Integer> lista = new ArrayList<Integer>();
12
13        for(int i=0;i<40000;i++) { lista.add(i); }
14
15        long start = System.currentTimeMillis();
16
17        int sum = 0;
18        for(int i=0;i<40000;i++) { sum += lista.get(i); }
19
20        long elapsed = System.currentTimeMillis() - start;
21        System.out.println("ArrayList: La suma es " + sum + " en " + elapsed + " ms");
22    }
23    private static void testLinkedList()
24    {
25        LinkedList<Integer> lista = new LinkedList<Integer>();
26
27        for(int i=0;i<40000;i++) { lista.add(i); }
28
29        long start = System.currentTimeMillis();
30    }

```

```

31     int sum = 0;
32     for(int i=0;i<40000;i++) { sum += lista.get(i); }
33
34     long elapsed = System.currentTimeMillis() - start;
35     System.out.println("LinkedList: La suma es " + sum + " en " + elapsed + " ms");
36 }
37 }

```

Al ejecutar este programa en un i7 de 2.1GHz, se obtiene la siguiente salida:

```

$ java Test
LinkedList: La suma es 799980000 en 1043 ms
ArrayList: La suma es 799980000 en 3 ms

```

Nótese cómo ambos códigos (`testLinkedList()` y `testArrayList()`) son iguales, y solamente se diferencian en la clase concreta que instancian para realizar la prueba de velocidad. El usar `ArrayList` es casi 300 veces más rápido que usar `LinkedList` en este caso, ya que cada vez que se necesita el elemento en la posición `i`, el recorrido *siempre* comienza desde el inicio de la lista.

Ahora, considera esta nueva versión:

```

1 public class Test
2 {
3     public static void main(String[] args)
4     {
5         testLinkedList();
6         testArrayList();
7     }
8
9     private static void testArrayList()
10    {
11        ArrayList<Integer> lista = new ArrayList<Integer>();
12
13        for(int i=0;i<40000;i++) { lista.add(i); }
14
15        long start = System.currentTimeMillis();
16
17        int sum = 0;
18        Iterator<Integer> it = lista.iterator();
19        while (it.hasNext()) { sum += it.next(); }
20
21        long elapsed = System.currentTimeMillis() - start;
22        System.out.println("ArrayList: La suma es " + sum + " en " + elapsed + " ms");
23    }
24    private static void testLinkedList()
25    {
26        LinkedList<Integer> lista = new LinkedList<Integer>();
27
28        for(int i=0;i<40000;i++) { lista.add(i); }
29
30        long start = System.currentTimeMillis();
31
32        int sum = 0;
33        Iterator<Integer> it = lista.iterator();
34        while (it.hasNext()) { sum += it.next(); }
35
36        long elapsed = System.currentTimeMillis() - start;
37        System.out.println("LinkedList: La suma es " + sum + " en " + elapsed + " ms");
38    }
39 }

```

```
$ java Test
LinkedList: La suma es 799980000 en 6 ms
ArrayList: La suma es 799980000 en 6 ms
```

El uso de un iterador permitió aprovechar el hecho de que una lista enlazada permite un rápido recorrido por sus elementos, siempre y cuando se “recuerde” en qué elemento estamos.

Hay otros casos donde se usa el concepto de iterador (y de cursor), pero están fuera del alcance de este libro. Por ahora, nos interesa saber que existen mecanismos para poder recorrer los contenidos de una colección de forma ordenada, y sin tener que preocuparnos de su tamaño y de controlar el índice.

De hecho, una forma muy fácil y eficiente de recorrer colecciones es usar la segunda versión del ciclo **for**:

```
1 public class Test2
2 {
3     public static void main(String[] args)
4     {
5         ArrayList<String> lista = new ArrayList<String>();
6
7         // agregar elementos a lista
8
9         // ahora recorremos la lista
10        for(String x : lista) {
11            System.out.println(x);
12        }
13    }
14 }
```

5.4. Ejercicios

1. Crea una clase genérica que contenga un método `agregar(...)` que permita agregarle un nuevo elemento a la clase. Por ejemplo, una persona recibiendo un premio. En este caso, el tipo de premio debe ser genérico, y la persona solamente debe guardar los últimos 3 premios que se le entregaron:

```
1 Persona<Medalla> p = new Persona<>();
2
3 p.agregar(new Medalla("ORO"));
4 p.agregar(new Medalla("ORO2"));
5 p.agregar(new Medalla("BRONCE"));
6 p.agregar(new Medalla("PLATA"));
7 p.agregar(new Medalla("ORO3"));
8
9 System.out.println(p.getPremios())
```

```
Mis 3 últimos premios son: ORO3 PLATA BRONCE
```


6

Un ejemplo completo

Para poder a prueba lo aprendido hasta ahora, se desarrollarán algunos problemas típicos, que tienen que ver con crear un modelo del dominio, modelar las clases correspondientes e implementar código Java que responda a los requerimientos.

6.1. Problema 1

6.1.1. Descripción del problema

Se desea construir un programa que maneje la información de los pasajeros que utilizan los servicios de la aerolínea “Vuela Alto”.

Un pasajero es identificado por medio de su RUT e interesa conocer su nombre, correo electrónico y si es o no pasajero frecuente.

Un ticket de viaje posee un código único, el nombre del pasajero, la fecha del viaje y un valor. Este ticket está asociado a un vuelo específico de los que ofrece la aerolínea.

La línea aérea tiene un listado oficial fijo de vuelos que proporciona, los cuales tienen un horario de salida desde un aeropuerto y un horario de llegada al aeropuerto de destino, además del código con el cual se identifican.

A partir de esta información es necesario construir una aplicación que permita realizar las siguientes operaciones:

1. Registrar un nuevo pasajero en la línea aérea.
2. Ingresar un nuevo vuelo a la línea aérea.
3. Ingresar un nuevo aeropuerto a la red de vuelos.
4. Realizar la venta de un ticket de viaje.

- 5. Listado de pasajeros de un viaje dado.
- 6. Lista el total de ventas por concepto de pasajes.
- 7. Listado de vuelos disponibles.
- 8. Top 10 de los pasajeros frecuentes con más viajes.
- 9. Listado de los viajes realizados, con sus respectivos pasajeros.

6.1.2. Análisis del problema

A partir de la lectura directa de la situación, se pueden descubrir rápidamente algunos conceptos que seguramente formarán parte de la solución.

Por un lado, el concepto pasajero:

Pasajero
rut nombre email esPasajeroFrecuente

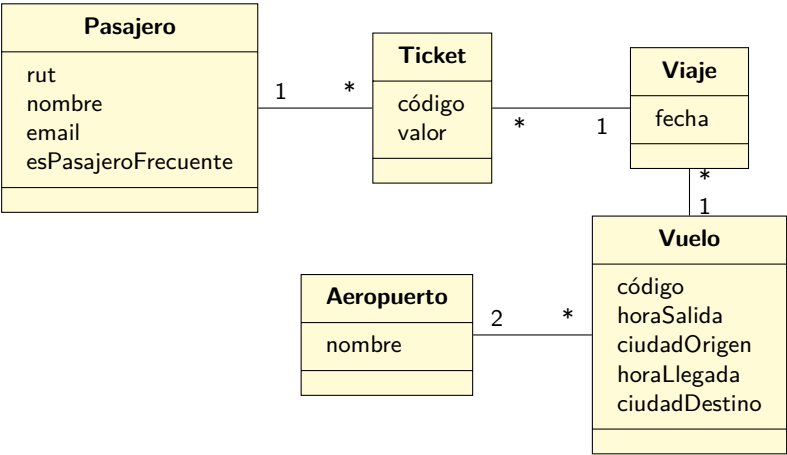
El texto también habla del concepto de ticket y de vuelo, además de aeropuerto:

Pasajero	Ticket	Vuelo
rut nombre email esPasajeroFrecuente	código nombrePasajero fecha valor	código horaSalida ciudadOrigen horaLlegada ciudadDestino

Aeropuerto
nombre

Tomando en cuenta lo que dice el texto, los vuelos identifican el hecho de que la aerolínea tiene definidas las combinaciones de origen→destino que administra, y los tickets a fin de cuentas de alguna forma están asociados a un viaje, que sería algo equivalente a un vuelo realizado en cierta fecha.

Vamos a modificar el diagrama para agregar el concepto de “viaje” y comenzar a dibujar algunos relacionamientos:



Bajo este esquema, un pasajero puede tener varios tickets a su nombre, y por otro lado, en un viaje también hay muchos tickets. Cada ticket está asociado a un viaje en particular, y un viaje es la realización de un vuelo, el que sale y llega a aeropuertos en una fecha determinada.

Nótese que movimos el atributo fecha desde Ticket a Viaje, ya que para nosotros un Viaje representará la realización de un Vuelo, pero en una fecha específica. A partir de ahí, es lógico que el Ticket se relacione con el Viaje. Nótese además que un Vuelo está relacionado con 2 aeropuertos, uno siendo la salida y el otro la llegada. Además, el Ticket ya no tiene el nombre del pasajero, ya que ahora está relacionado con Pasajero directamente.

Pareciera que este modelo representa el problema. Veamos si podemos contestar las preguntas:

Registrar un nuevo pasajero en la línea aérea En este caso, se requiere algo “extra” para poder almacenar a los pasajeros que se van registrando, en el sentido de que aun cuando podemos crear pasajeros, no tenemos donde guardarlos y mantenerlos referenciados¹. Tal como está el modelo hasta ahora, los únicos pasajeros que están presentes (y referenciados) son aquellas personas que tienen un ticket (ya que el ticket hace referencia tanto al viaje como al pasajero). Si vamos a agregar pasajeros “nuevos”, que aun no tienen un ticket, actualmente no tenemos forma de hacerlo. Para lograrlo, necesitamos agregar un objeto que permita tener referenciados a todos los pasajeros. A este objeto le llamaremos `listaPasajeros`.

Ingresar un nuevo vuelo a la línea aérea En este punto ni siquiera hemos completado lo que inicialmente decía el problema. El texto dice “la línea aérea tiene un listado oficial fijo de vuelos”, pero el modelo actualmente no lo soporta. Para poder tener este listado, es necesario agregar un nuevo objeto que nos permitirá mantener referenciados todos los vuelos. A este objeto le llamaremos `listaVuelos`.

Ingresar un nuevo aeropuerto a la red de vuelos Hasta ahora, el modelo solo registra aquellos aeropuertos que son referenciados desde `Vuelo`. Para agregar una nueva instancia de `Aeropuerto` necesitamos algo dónde guardarla. Para ésto, incluiremos un nuevo objeto llamado `listaAeropuertos`.

Realizar la venta de un ticket de viaje Este requerimiento está cubierto, ya que solo se requiere identificar el pasajero, el vuelo, e instanciar un nueva objeto `Ticket`, que será relacionado tanto con el pasajero como con el viaje.

¹Recuerda que a menos que una instancia sea referenciada ésta será eventualmente tomada por el recolector de basura

Listado de pasajeros de un viaje dado Teniendo identificada una instancia de `Viaje`, es fácil iterar sobre todos los tickets de dicho viaje, y para cada ticket navegar hacia el pasajero correspondiente.

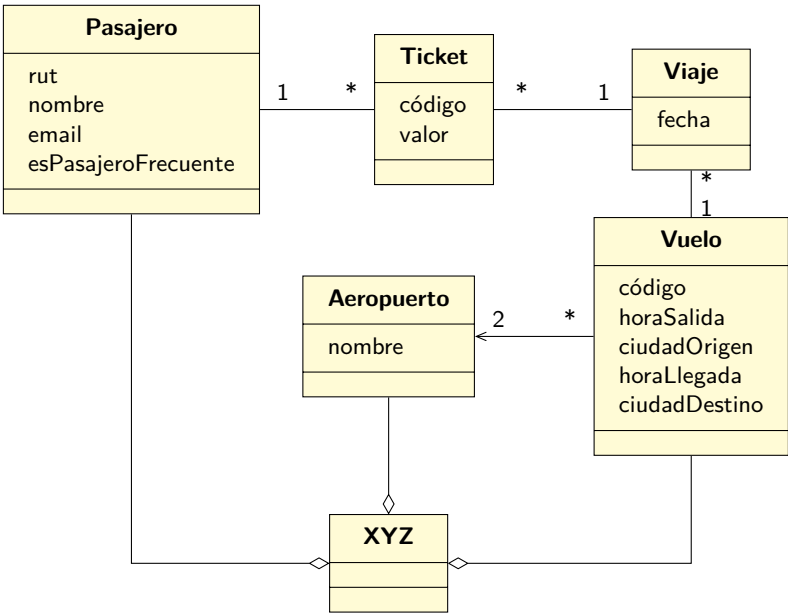
Lista el total de ventas por concepto de pasajes Para hacer ésto, se puede utilizar `listaVuelos` para iterar sobre todos los vuelos, y a cada uno preguntarle por el total en ventas. Ésta operación, por su lado, iterará sobre todos los viajes de dicho vuelo, y le preguntará por el total de ventas. Esta operación simplemente iterará sobre su lista de tickets para ir acumulando el valor.

Listado de vuelos disponibles Partiendo de la `listaVuelos`, se puede iterar para mostrar todos los vuelos.

Top 10 de los pasajeros frecuentes con más viajes A partir de `listaPasajeros`, se puede determinar qué personas, de las que tienen la marca `esPasajeroFrecuente`, tienen más tickets.

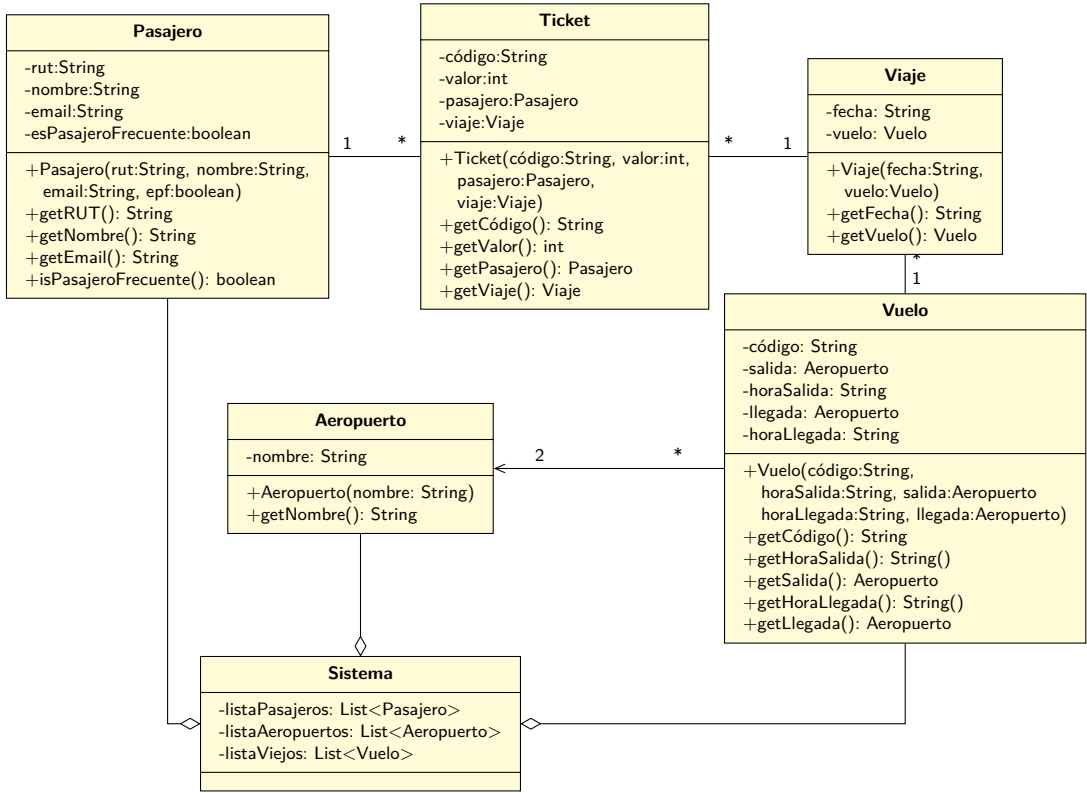
Listado de los viajes realizados, con sus respectivos pasajeros Partiendo de la `listaVuelos`, se puede navegar hacia cada viaje, hacia cada ticket para mostrar cada pasajero.

Incluyendo todo lo mencionado anteriormente, el modelo sería:



Pero, ¿qué pasa con esa clase **XYZ**? Aun cuando no es un objeto que haya estado mencionado en el problema mismo, el algo que tendremos que agregar a la solución, ya que se requiere un objeto que permita mantener en todo momento referenciadas las instancias tanto de **Pasajero**, **Aeropuerto** y **Vuelo**. A este objeto lo vamos a llamar **Sistema**.

Ya que hemos determinado que el modelo ya representa la solución al problema, comencemos a crear el diagrama de clases. Nótese que se agregan los constructores apropiados, y los setters y getters que correspondan.



En Java, cuando se tiene un atributo de tipo boolean, el “getter” se acostumbra escribirlo de como `isXYZXYZ`.

Aun cuando este diagrama de clases pareciera completo, la verdad es que falta tomar una decisión respecto a un par de elementos faltantes.

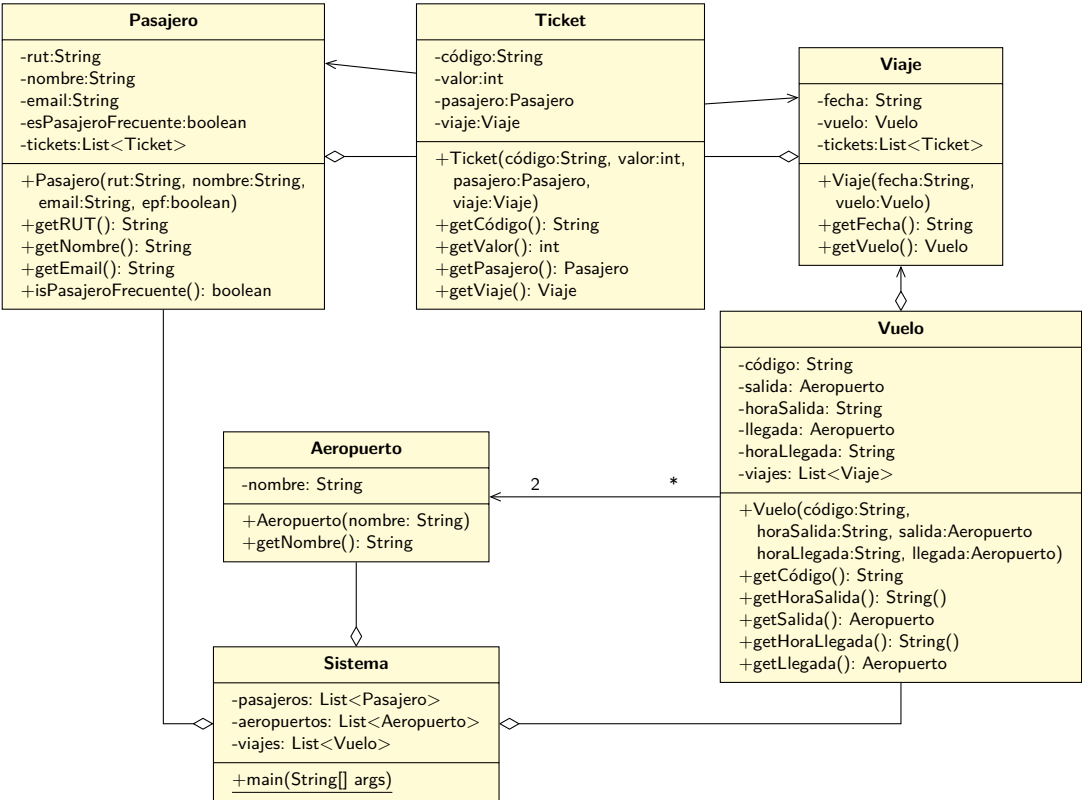
Por un lado, falta decidir dónde se incluirá el el `main`. Por ahora, agregaremos el método en la clase `Sistema`. Más adelante revisaremos esta decisión.

Además, falta agregar al diagrama el cómo tanto el `Pasajero` como el `Viaje` mantendrán referenciadas las instancias de `Ticket` que le corresponden a cada uno: por un lado, cada `Pasajero` debe saber qué instancias de `Ticket` ha comprado, y por otro lado, cada instancia de `Viaje` debe saber qué instancias de `Ticket` se han generado (para ese viaje).

Entonces, todas las instancias de `Pasajero` y `Viaje` tendrán una lista que les permitirá saber de los tickets que están relacionados con ellos.

En el caso de `Vuelo` se genera el mismo argumento, ya que cada instancia de `Vuelo` requiere saber de todas las instancias de `Viaje` que se han generado, y que tienen relación con ella. Así que se le agregará una lista de `Viaje` para ese efecto.

Por último, se simplificará el nombre de algunas variables para evitar usar `listaX`, siendo que no es estrictamente necesario.



6.1.3. Clases

Ya que se ha construido el diagrama de clases, podemos construir el código que corresponde a cada clase de la solución del problema.

Aeropuerto

```
1 public class Aeropuerto
2 {
3     private String nombre;
4
5     public Aeropuerto(String nombre)
6     {
7         this.nombre = nombre;
8     }
9
10    public String getNombre()
11    {
12        return nombre;
13    }
14 }
```

Pasajero

```
1 public class Pasajero
2 {
3     private String rut;
4     private String nombre;
5     private String email;
6     private boolean esPasajeroFrecuente;
7     private List<Ticket> tickets;
8
9     public Pasajero(String rut, String nombre, String email, boolean esPasajeroFrecuente
10    )
11    {
12        this.rut = rut;
13        this.nombre = nombre;
14        this.email = email;
15        this.esPasajeroFrecuente = esPasajeroFrecuente;
16        this.tickets = new ArrayList<>();
17    }
18
19    public String getRut() { return rut; }
20    public String getNombre() { return nombre; }
21    public String getEmail() { return email; }
22    public boolean isPasajeroFrecuente() { return esPasajeroFrecuente; }
```

Ticket

```
1 public class Ticket
2 {
3     private String código;
4     private int valor;
5     private Pasajero pasajero;
6     private Viaje viaje;
7
8     public Ticket(String código, int valor, Pasajero pasajero, Viaje viaje)
9     {
10        this.código = código;
11        this.valor = valor;
12        this.pasajero = pasajero;
13        this.viaje = viaje;
14    }
15
16    public String getCódigo() { return código; }
17    public int getValor() { return valor; }
18    public Pasajero getPasajero() { return pasajero; }
19    public Viaje getViaje() { return viaje; }
20 }
```

Viaje

```
1 public class Viaje
2 {
3     private String fecha;
4     private Vuelo vuelo;
5     private List<Ticket> tickets;
```

```

6
7 public Viaje(String fecha, Vuelo vuelo)
8 {
9     this.fecha = fecha;
10    this.vuelo = vuelo;
11    this.tickets = new ArrayList<>();
12 }
13
14 public String getFecha() { return fecha; }
15 public Vuelo getVuelo() { return vuelo; }
16 }

```

Vuelo

```

1 public class Vuelo
2 {
3     private String código;
4     private Aeropuerto salida;
5     private String horaSalida;
6     private Aeropuerto llegada;
7     private String horaLlegada;
8     private List<Viaje> viajes;
9
10    public Vuelo(String código,
11                String horaSalida, Aeropuerto salida,
12                String horaLlegada, Aeropuerto llegada)
13    {
14        this.código = código;
15        this.horaSalida = horaSalida;
16        this.salida = salida;
17        this.horaLlegada = horaLlegada;
18        this.llegada = llegada;
19        this.viajes = new ArrayList<>();
20    }
21
22    public String getCódigo() { return código; }
23    public Aeropuerto getSalida() { return salida; }
24    public String getHoraSalida() { return horaSalida; }
25    public Aeropuerto getLlegada() { return llegada; }
26    public String getHoraLlegada() { return horaLlegada; }
27 }

```

Sistema

```

1 public class Sistema
2 {
3     public static void main(String[] args)
4     {
5         Sistema app = new Sistema();
6         app.procesar();
7     }
8
9     private List<Pasajero> pasajeros;
10    private List<Aeropuerto> aeropuertos;
11    private List<Vuelo> vuelos;
12 }

```



```

13
14 public Sistema()
15 {
16     pasajeros = new ArrayList<>();
17     aeropuertos = new ArrayList<>();
18     vuelos = new ArrayList<>();
19 }
20
21 private void procesar()
22 {
23     // Pendiente
24 }
25 }

```

Revisemos los requerimientos del problema, para ver cómo se implementarían.

6.1.4. Implementando los requerimientos

Como forma base de operar, definiremos la clase Sistema de esta manera:

```

1 public class Sistema
2 {
3     public static void main(String[] args)
4     {
5         Sistema app = new Sistema();
6         app.procesar();
7     }
8
9     private List<Pasajero> pasajeros;
10    private List<Aeropuerto> aeropuertos;
11    private List<Vuelo> vuelos;
12
13
14    public Sistema()
15    {
16        pasajeros = new ArrayList<>();
17        aeropuertos = new ArrayList<>();
18        vuelos = new ArrayList<>();
19    }
20
21    private void procesar()
22    {
23        Scanner s = new Scanner(System.in);
24
25        invocarOperaciónXYZ(s);
26
27        s.close();
28    }
29 }

```

Desde el método `procesar()` se invocarán los métodos específicos para realizar los requerimientos. En caso de que la operación requiera leer datos desde el teclado, se le pasará la instancia de `Scanner` definida en la línea 23.

Registrar un nuevo pasajero en la línea aérea

```

1 private void registrarNuevoPasajero(Scanner s)
2 {
3     System.out.println("Registrando un nuevo pasajero:");
4
5     System.out.print("RUT: ");
6     String rut = s.nextLine();
7
8     System.out.print("Nombre: ");
9     String nombre = s.nextLine();
10
11    System.out.print("Email: ");
12    String email = s.nextLine();
13
14    System.out.print("Es pasajero frecuente? (S/N): ");
15    boolean epf = s.nextLine().equalsIgnoreCase("S");
16
17    Pasajero nuevo = new Pasajero(rut, nombre, email, epf);
18    pasajeros.add(nuevo);
19 }

```

Ingresar un nuevo vuelo a la línea aérea

```

1 private void registrarNuevoVuelo(Scanner s)
2 {
3     System.out.println("Registrando un nuevo vuelo:");
4
5     System.out.print("Código: ");
6     String código = s.nextLine();
7
8     System.out.print("Origen: ");
9     Aeropuerto origen = buscarAeropuerto(s.nextLine());
10
11    System.out.print("Hora de salida: ");
12    String horaSalida = s.nextLine();
13
14    System.out.print("Destino: ");
15    Aeropuerto destino = buscarAeropuerto(s.nextLine());
16
17    System.out.print("Hora de llegada: ");
18    String horaLlegada = s.nextLine();
19
20    Vuelo nuevo = new Vuelo(código, horaSalida, origen, horaLlegada, destino);
21    vuelos.add(nuevo);
22 }

```

En este punto, nos hemos dado cuenta que en las líneas 9 y 15 se requiere poder buscar un aeropuerto a partir del nombre ingresado desde el teclado. **Ésto debería haber sido detectado antes, en la etapa de diseño.**

Se agregará el método `buscarAeropuerto(...)` a la clase `Sistema` para lograr que este requerimiento se complete exitosamente.

```

1 private Aeropuerto buscarAeropuerto(String nombre)
2 {
3     for(Aeropuerto a : aeropuertos) {
4         if (nombre.equals(a.getNombre())) return a;
5     }
6     return null;

```

```
}
```

Ingresar un nuevo aeropuerto a la red de vuelos

```
1 private void registrarNuevoAeropuerto(Scanner s)
2 {
3     System.out.println("Registrando un nuevo aeropuerto:");
4
5     System.out.print("Nombre: ");
6     String nombre = s.nextLine();
7
8     Aeropuerto nuevo = new Aeropuerto(nombre);
9     aeropuertos.add(nuevo);
10 }
```

Realizar la venta de un ticket de viaje

Este requerimiento necesita que se pregunten los datos necesarios, y que se genere una instancia de Ticket que asociará a un Pasajero con un Vuelo. Como lo que se requiere son las *instancias* de cada uno, seguramente se requerirán métodos extras en ciertas clases:

```
1 private void venderTicket(Scanner s)
2 {
3     System.out.println("Venta de ticket:");
4
5     System.out.print("Código: ");
6     String código = s.nextLine();
7
8     System.out.print("Valor: ");
9     int valor = Integer.valueOf(s.nextLine());
10
11     System.out.print("Nombre del pasajero: ");
12     Pasajero pasajero = buscarPasajero(s.nextLine());
13
14     System.out.print("Código de vuelo: ");
15     String códigoVuelo = s.nextLine();
16
17     Vuelo vuelo = buscarVuelo(códigoVuelo);
18
19     System.out.print("Fecha: ");
20     String fecha = s.nextLine();
21
22     Viaje viaje = vuelo.buscarViaje(fecha);
23
24     Ticket ticket = new Ticket(código, valor, pasajero, viaje);
25
26     pasajero.agregarTicket(ticket);
27     viaje.agregarTicket(ticket);
28 }
```

En este caso, al igual que el caso anterior, nos hemos dado cuenta de que faltan elementos para poder completar el requerimiento. Y al igual que en el caso anterior, **ésto debería haberse descubierto en la etapa de diseño**. Una fase de diseño cuidadoso puede detectar todos estos requerimientos.

En particular, se requiere poder buscar a un pasajero a partir de su nombre. Para eso, se agregará un `buscarPasajero(...)` en `Sistema`:

```
1 private Pasajero buscarPasajero(String nombre)
2 {
3     for(Pasajero p : pasajeros) {
4         if (nombre.equals(p.getNombre())) return p;
5     }
6     return null;
7 }
```

Se realizará lo mismo con los vuelos, ya que se requiere poder localizar un vuelo en particular a partir de su código:

```
1 private Vuelo buscarVuelo(String códigoVuelo)
2 {
3     for(Vuelo v : vuelos) {
4         if (v.getCódigo().equals(códigoVuelo)) return v;
5     }
6     return null;
7 }
```

Además de lo anterior, es necesario agregarle un método a `Vuelo`, para que podamos buscar un `Viaje` conociendo la fecha:

```
1 public class Vuelo
2 {
3     ....
4     public Viaje buscarViaje(String fecha)
5     {
6         for(Viaje el : viajes) {
7             if (el.getFecha().equalsIgnoreCase(fecha)) {
8                 return el;
9             }
10        }
11        return null;
12    }
13 }
```

Y por último, una vez que se creó la instancia de `Ticket`, hay que agregar dicha instancia al `Pasajero` y al `Viaje`:

```
1 public class Pasajero
2 {
3     ...
4     public void agregarTicket(Ticket ticket)
5     {
6         tickets.add(ticket);
7     }
8 }
```

```
1 public class Viaje
2 {
3     ....
4     public void agregarTicket(Ticket ticket)
5     {
6         tickets.add(ticket);
7     }
8 }
```

Con todas las modificaciones anteriores, entonces el método `venderTicket(...)` queda completo.

Listado de pasajeros de un viaje dado

En este caso, vamos a partir desde una instancia de `Viaje`. Como sabemos que cada viaje mantiene un registro de todos sus `Tickets`, y cada `Ticket` sabe quién es su `Pasajero`, basta hacer un ciclo de la siguiente forma, agregando este método en `Sistema`:

```

1  private void listarPasajeros(Viaje viaje)
2  {
3      for(Ticket t : viaje.getTickets()) {
4          Pasajero pasajero = t.getPasajero();
5          System.out.println(pasajero.getRut() + "," +
6                          pasajero.getNombre() + "," +
7                          pasajero.getEmail());
8      }
9  }
```

Pero en la línea 3 hay un problema. Tal como está escrito, se tendría que agregar a `Viaje` un “getter” de su lista de tickets, pero eso dejaría abierta la posibilidad de que un cliente de la clase pudiera agregarle cosas a dicha lista, y sin que la instancia de `Viaje` se entere:

```

1  private void listarPasajeros(Viaje viaje)
2  {
3      for(Ticket t : viaje.getTickets()) {
4          Pasajero pasajero = t.getPasajero();
5          System.out.println(pasajero.getRut() + "," +
6                          pasajero.getNombre() + "," +
7                          pasajero.getEmail());
8      }
9      // Abusamos de la inocencia del programador y usamos la puerta que dejó abierta
10     viaje.getTickets().add(new Ticket("123", 100, null, viaje));
11 }
```

Para que esto no suceda, podríamos cambiar la forma en que obtenemos la lista de `Tickets`. En vez de pedirle a `Viaje` que nos retorne una referencia a su lista interna donde guarda sus `Tickets`, podríamos hacer que nos agregue las instancias de `Ticket` a una lista que desde acá mismo le pasamos por parámetro:

```

1  private void listarPasajeros(Viaje viaje)
2  {
3      List<Ticket> tickets = new ArrayList<>();
4      viaje.getTickets(tickets);
5
6      for(Ticket t : tickets) {
7          Pasajero pasajero = t.getPasajero();
8          System.out.println(pasajero.getRut() + "," +
9                          pasajero.getNombre() + "," +
10                         pasajero.getEmail());
11     }
12     // Esto no va a tener ningún efecto negativo
13     tickets.add(new Ticket("123", 100, null, viaje));
14 }
```

Así que modificaremos `Viaje` para agregarle el método `getListaTickets(...)`, y lo documentamos para que quede claro qué es lo que hace el método:

```

1 public class Viaje
2 {
3     ...
4     /**
5      * Agrega a la lista indicada todas las instancias de Ticket
6      * de este Viaje.
7      *
8      * @param lista La lista donde se agregarán los Tickets de este Viaje
9      */
10    public void getTickets(List<Ticket> lista)
11    {
12        lista.addAll(this.tickets);
13    }
14 }

```

Hay otras soluciones a este problema, por ejemplo, retornar una lista que sea inmutable, pero quedarán pendientes hasta próximo aviso.

Lista el total de ventas por concepto de pasajes

En este requerimiento, hay que recorrer todos los Viajes realizados (usando la lista de Vuelos), y para cada Viaje, recorrer todos los Tickets, que es donde está almacenado el valor que nos interesa:

```

1 private void mostrarTotalVentas()
2 {
3     int total = 0;
4
5     for(Vuelo vuelo : vuelos) {
6
7         List<Viaje> viajes = new ArrayList<>();
8
9         vuelo.getViajes(viajes);
10
11        for(Viaje viaje : viajes) {
12
13            List<Ticket> tickets = new ArrayList<>();
14
15            viaje.getTickets(tickets);
16
17            for(Ticket ticket : tickets) {
18                total += ticket.getValor();
19            }
20        }
21    }
22    System.out.println("El total de ventas es " + total);
23 }

```

Para que esto funcione, solo se requiere agregarle el método `getViajes(...)` a `Vuelo`, usando el mismo esquema usado en el requerimiento anterior:

```

1 public class Vuelo
2 {
3     ....
4     public void getViajes(List<Viaje> lista)
5     {
6         lista.addAll(this.viajes);
7     }
8 }

```

Pero, tal vez podamos hacer algo mejor. Considera esta nueva versión del método en **Sistema**:

```

1  private void mostrarTotalVentas()
2  {
3      int total = 0;
4
5      for(Vuelo vuelo : vuelos) {
6          total += vuelo.getTotalVentas();
7      }
8      System.out.println("El total de ventas es " + total);
9  }

```

Básicamente le estamos pidiendo a cada Vuelo que haga lo que tenga que hacer para calcular el total de ventas. Y como cada Vuelo tiene toda la información necesaria, realizará la operación de forma correcta y nos retornará el resultado. Esta versión es mucho mejor ya que no nos preocupamos de cómo se obtiene la información, ni de la organización interna de los datos, sino que simplemente le preguntamos al objeto responsable de la información que nos entregue el valor ya calculado.

Para que esta versión funcione, se debe agregar el método correspondiente a Vuelo:

```

1  public class Vuelo
2  {
3      ...
4      public int getTotalVentas()
5      {
6          int total = 0;
7
8          for(int i=0;i<listaViajes.getTamaño();i++) {
9              Viaje viaje = listaViajes.getElemento(i);
10
11              total += viaje.getTotalVentas();
12          }
13          return total;
14      }
15  }

```

Y también el método que corresponde en Viaje:

```

1  public class Viaje
2  {
3      ....
4      public int getTotalVentas()
5      {
6          int total = 0;
7
8          for(Ticket ticket : tickets) {
9              total += ticket.getValor();
10          }
11          return total;
12      }
13  }

```

Nótese que a fin de cuentas el código es similar entre la primera y la segunda versión; la diferencia fundamental es que en la segunda versión la respuesta a la consulta del total de ventas la responde el objeto adecuado.

Listado de vuelos disponibles

En este caso, solo iteramos por la lista de Vuelos:

```

1 private void listarVuelosDisponibles()
2 {
3     System.out.println("Vuelos disponibles:");
4
5     for(Vuelo vuelo : vuelos) {
6         System.out.println(vuelo.getCódigo() +
7             " Desde " + vuelo.getSalida() + "/" + vuelo.getHoraSalida() +
8             " Hasta " + vuelo.getLlegada() + "/" + vuelo.getHoraLlegada()
9         );
10    }
11 }

```

Top 10 de los pasajeros frecuentes con más viajes

La estrategia en este caso será primero filtrar los pasajeros frecuentes, y después ordenarlos por la cantidad de viajes de cada uno:

```

1 private void mostrarTop10PasajerosFrecuentesMásViajes()
2 {
3     List<Pasajero> frecuentes = new ArrayList<>();
4
5     for (Pasajero pasajero : pasajeros) {
6         if (!pasajero.isPasajeroFrecuente())
7             continue;
8
9         frecuentes.add(pasajero);
10    }
11
12    int n = frecuentes.size();
13
14    for (int a = 0; a < n - 1; a++) {
15        for (int b = a + 1; b < n; b++) {
16            // Obtiene el elemento en el índice indicado
17            Pasajero pa = frecuentes.get(a);
18            Pasajero pb = frecuentes.get(b);
19
20            if (pa.getCantViajes() < pb.getCantViajes()) {
21                intercambiar(frecuentes, a, b);
22            }
23        }
24    }
25
26    if (n > 10) n = 10;
27
28    System.out.println("Top 10 pasajeros frecuentes:");
29    for (int i = 0; i < n; i++) {
30        Pasajero p = frecuentes.get(i);
31        System.out.println(p.getRut() + " " + p.getNombre());
32    }
33 }

```

Pero para que ésto funcione, se requiere agregar el método `getCantViajes()` a `Pasajero`, y también el método `intercambiar(...)` en `Sistema`:


```

1 public class Pasajero
2 {
3     ...
4     public int getCantViajes()
5     {
6         // Asumiremos que un ticket corresponde a un viaje
7         return this.tickets.size();
8     }
9 }

1 public class Sistema
2 {
3     ...
4     private void intercambiar(List<Pasajero> lista, int a, int b)
5     {
6         Pasajero temp = lista.get(a);
7         lista.set(a, lista.get(b));
8         lista.set(b, temp);
9     }
10 }

```

Listado de los viajes realizados, con sus respectivos pasajeros

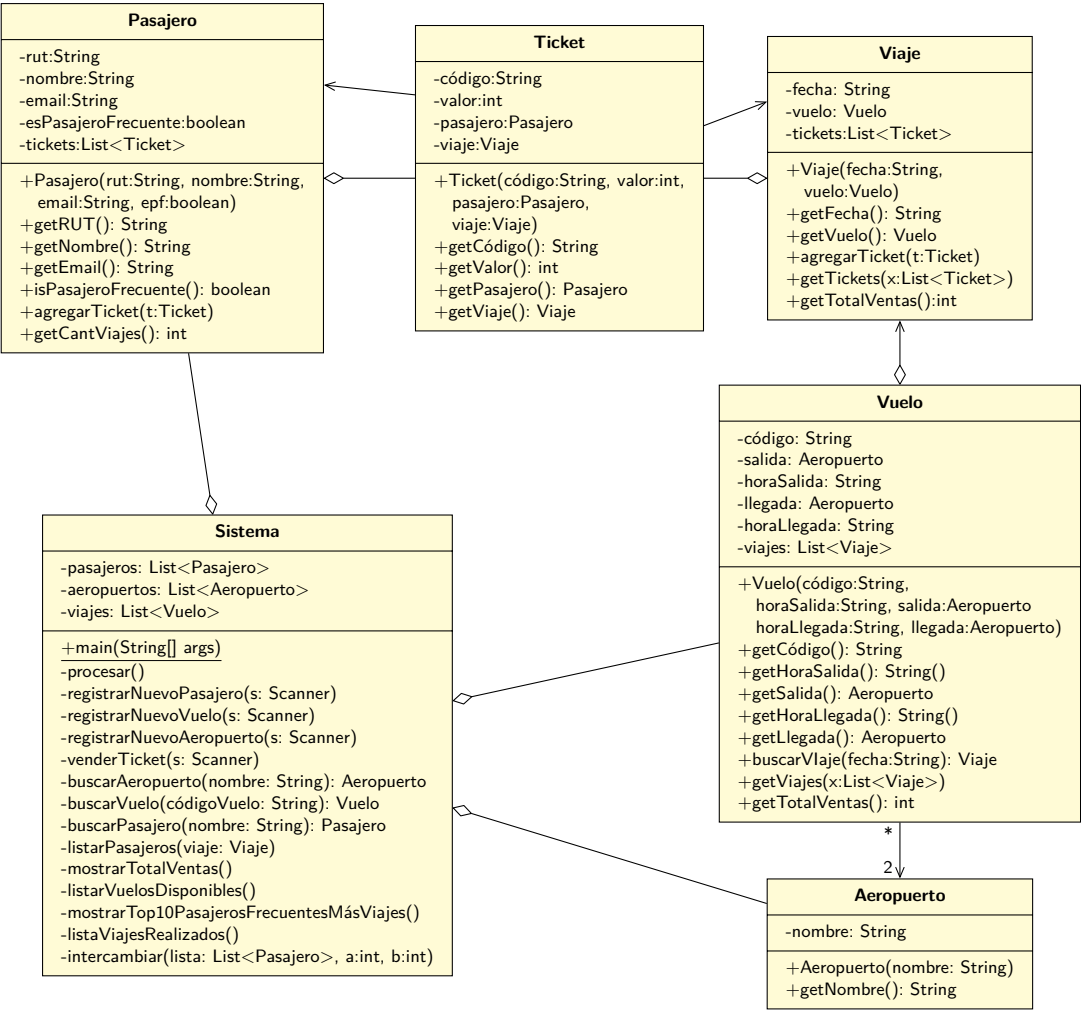
En este caso, basta iterar y navegar por los objetos hasta encontrar la información que se está buscando:

```

1 private void listaViajesRealizados()
2 {
3     for(Vuelo vuelo : vuelos) {
4
5         List<Viaje> listaViajes = new ArrayList<>();
6
7         vuelo.getViajes(listaViajes);
8
9         for (Viaje viaje : listaViajes) {
10
11             List<Ticket> listaTickets = new ArrayList<>();
12
13             viaje.getTickets(listaTickets);
14
15             System.out.println(viaje.getVuelo().getCódigo() + " " +
16                             viaje.getFecha());
17
18             for(Ticket ticket : listaTickets) {
19                 Pasajero pasajero = ticket.getPasajero();
20
21                 System.out.println(pasajero.getRut() + " " +
22                                 pasajero.getNombre());
23             }
24         }
25     }
26 }
27

```

6.1.5. Diagrama de clases



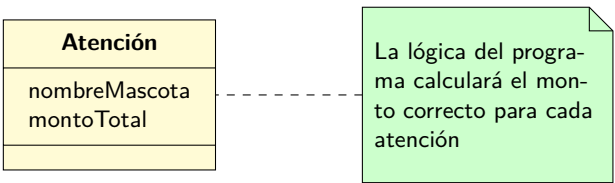
7

POO 2

7.1. Otra técnica para diseñar programas

Imaginemos que tenemos que resolver un problema donde queremos modelar el funcionamiento de una clínica veterinaria donde se atienden perros, gatos y cobayas. Por cada tipo de animal se cobra un monto diferente por las atenciones. La idea es poder registrar estas atenciones, y tener algún historial de las veces en que cada mascota ha sido atendida, para después poder calcular un total del gasto de cada una.

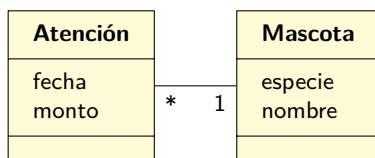
Una forma de enfrentar el problema es creando un solo concepto llamado atención, que adentro contenga el tipo de animal, su nombre (o algún otro dato que lo identifique), y que mediante la lógica del programa calcule el monto de la atención¹:



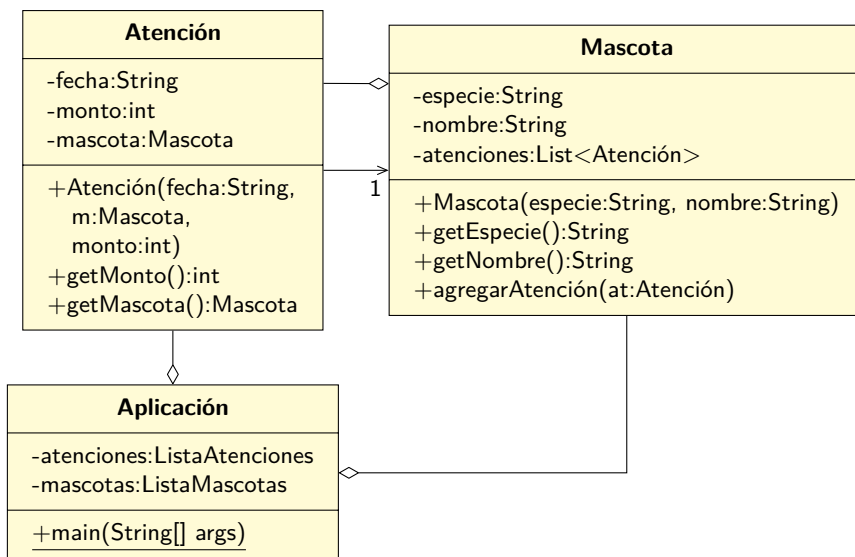
El problema con este modelo es que no es una buena práctica almacenar los datos de esta forma. Si se quisiera obtener el total de atenciones de una mascota dada, se tendrían que revisar *todas* las atenciones para buscar las que son de interés, y recién en ese momento sumar sus montos. La verdad, es que el software podría funcionar, pero es mejor pensar el problema en otros términos.

Una alternativa es tener un registro de mascotas, donde cada mascota atendida esté individualizada, y que cada una sepa qué atenciones ha tenido:

¹O sea, usando muchos `if`



Si obtenemos el diagrama de clases a partir del modelo podemos obtener algo como lo siguiente:



Basándonos en lo anterior, implementemos cómo se crearía una nueva atención:

```

1 class Aplicación
2 {
3     ....
4     private void registrarAtención(String fecha, String nombreMascota)
5     {
6         Mascota m = buscarPorNombre(mascotas, nombreMascota);
7
8         int monto = 0;
9         String especie = m.getEspecie();
10        if (especie.equals("Perro")) {
11            monto = 1000;
12        } else if (especie.equals("Gato")) {
13            monto = 1200;
14        } else if (especie.equals("Cobaya")) {
15            monto = 900;
16        }
17        Atención at = new Atención(fecha, m, monto);
18
19        atenciones.add(at);
20        m.agregarAtención(at);
21    }
22 }

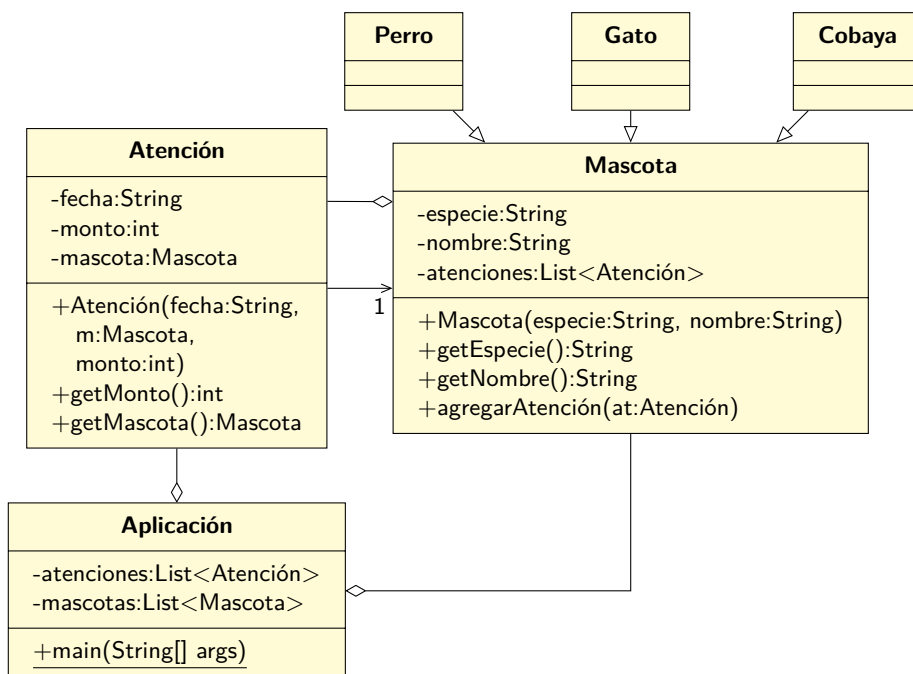
```

En este código no estamos considerando el caso de que la mascota no exista, solo nos enfocaremos en

buscar la mascota, generar la atención, y registrar dicha atención tanto en la lista general de atenciones, como en la lista particular de atenciones de la misma mascota.

El principal problema del código anterior es que toda la lógica del cálculo del monto de la atención está escrito en la clase `it`, y debido a cómo es el relato del problema, no es un buen lugar para tener ese tipo de lógica. En primer lugar, la clase `Aplicación` no debería saber ese tipo de cosas: no es responsabilidad de dicha clase tener ese conocimiento. Tal como dice el problema, el `monto` de la atención depende del tipo de mascota atendida, por lo que la responsabilidad de conocer el costo debería estar por allá. Además, podríamos hacernos la pregunta: y si agregamos un nuevo tipo de mascota, ¿qué hay que hacer? En este caso lo que habría que hacer es complejizar el `if` que determina el monto. Y más aun, siempre queda en el aire el tema de qué pasa cuando el `String` que guarda la especie no coincide con ningún tipo cubierto por el `if/else`.

En este caso, sería recomendable aplicar una nueva técnica, que es una de las bases de la POO. Veamos este diagrama de clases:



Fíjate en los símbolos que unen a `Mascota` y las clases que están arriba. Todas esas clases están relacionadas entre sí, pero no en términos de la multiplicidad (como es el caso de asociaciones y agregaciones), sino que mediante la **especialización**. Ésto ya se mencionó antes cuando se habló del modelo del dominio (página 92), pero a partir de ahora le llamaremos *herencia*.

Tomando eso en cuenta, diremos que existe una herencia entre `Mascota`, `Perro`, `Gato` y `Cobaya`. O mejor dicho que tanto `Perro`, `Gato` y `Cobaya` son *subclases* de `Mascota`, o también se dice que son sus clases hijas.

¿Por qué nos conviene estructurar las cosas de esta forma? Nos conviene ya que hay una operación que aunque todas las `Mascotas` tienen, *cada especie la ejecuta de forma diferente*. Estamos hablando (en

el caso de este problema) del cálculo del precio de atención.

Todas las especies tienen un precio de atención, pero cada especie concreta lo implementa de forma diferente (en este caso, el valor es diferente por cada especie). Pero ¿cómo es que eso nos sirve para resolver y hacer más simple la solución al problema?

Para explicar esto, primero es necesario conocer un par de cosas extras.

7.2. Sobre-escritura de métodos

Cuando hablamos de que cada subclase puede implementar un comportamiento diferente al de su clase base, ¿cómo se hace en Java? Veamos este ejemplo:

```
1 public class Seleccionadora
2 {
3     public static void main(String[] args)
4     {
5         Seleccionadora x = new Seleccionadora();
6
7         int r = x.seleccionarMayor(10, 25);
8         System.out.println(r);
9     }
10
11     public int seleccionarMayor(int i, int j)
12     {
13         if (i > j) return i;
14         return j;
15     }
16 }
```

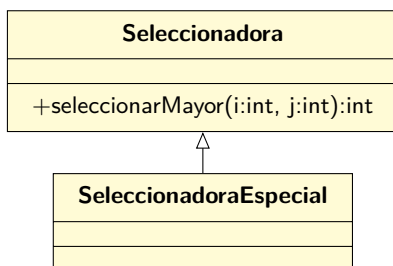
Si ejecutamos esto, obtenemos lo esperado, sin sorpresas:

```
$ java Seleccionadora
25
```

Ahora, creemos esta clase:

```
1 public class SeleccionadoraEspecial extends Seleccionadora
2 {
3 }
```

Nótese que hay una palabra nueva para agregar al vocabulario. En este caso, `extends` es la forma en que le decimos al compilador de Java que la clase `SeleccionadoraEspecial` es una subclase de `Seleccionadora`:



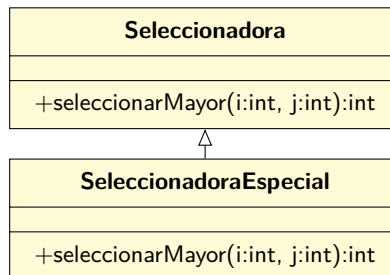
Imaginemos que `SeleccionadoraEspecial` quiere mejorar la forma en que el método `seleccionarMayor` está implementado. Entonces, vamos a actualizar `SeleccionadoraEspecial` para reflejar este conocimiento:

```

1 public class SeleccionadoraEspecial extends Seleccionadora
2 {
3     public int seleccionarMayor(int i, int j)
4     {
5         return i + j;
6     }
7 }

```

El diagrama de clases cambia a:



y modificamos el `main` para probar este cambio:

```

1 public class Seleccionadora
2 {
3     public static void main(String[] args)
4     {
5         SeleccionadoraEspecial x = new SeleccionadoraEspecial();
6
7         int r = x.seleccionarMayor(10, 25);
8         System.out.println(r);
9     }
10
11     public int seleccionarMayor(int i, int j)
12     {
13         if (i > j) return i;
14         return j;
15     }
16 }

```

Si ejecutamos esta nueva versión:

```

$ java Seleccionadora
35

```

No hay mayores sorpresas en este resultado. En la línea 5, estamos instanciando `SeleccionadoraEspecial`, y después invocamos su método `seleccionarMayor()`, que simplemente suma ambos números y retorna el resultado. En este caso, la subclase está modificando el comportamiento de la clase base. Ambas tienen declarado el mismo método (con el mismo tipo de retorno y mismos parámetros), pero difieren en el comportamiento de la operación.

Pero para hacer aún más interesante este proceso, es necesario aprender otros conceptos extras.

7.3. La visibilidad de lo heredado

Es interesante destacar algunas cosas que suceden cuando se crea una subclase. Por ejemplo:

```
1 public class Moneda
2 {
3     public int getValor()
4     {
5         return 0;
6     }
7 }

```

```
1 public class MonedaDe100 extends Moneda
2 {
3 }

```

```
1 public class TestMonedas
2 {
3     public static void main(String[] args)
4     {
5         MonedaDe100 m100 = new MonedaDe100();
6         System.out.println(m100.getValor());
7     }
8 }

```

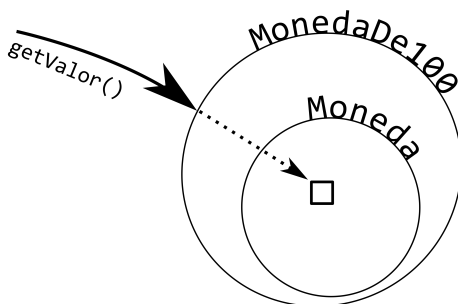
En este caso, podría parecer que este código no debería compilar, ya que se está invocando a `getValor()` sobre una instancia de `MonedaDe100`, y esa clase no tiene definido el método, pero la verdad es que compila y se ejecuta correctamente:

```
$ java TestMonedas
0
```

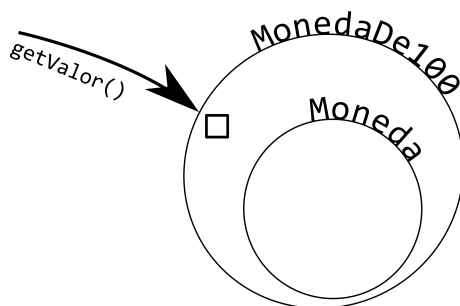
¿Cómo se puede explicar este comportamiento?

La forma más directa de explicarlo, es que al momento de declarar `MonedaDe100` como una subclase de `Moneda`, ésta tendrá acceso a todas las cosas declaradas en `Moneda` (excepto lo declarado como `private`).

Otra forma de verlo es pensar en que cada instancia de `MonedaDe100` tiene “adentro” una instancia de `Moneda`, y que cuando la instancia de `MonedaDe100` recibe un mensaje, es la instancia interna la que lo contesta:



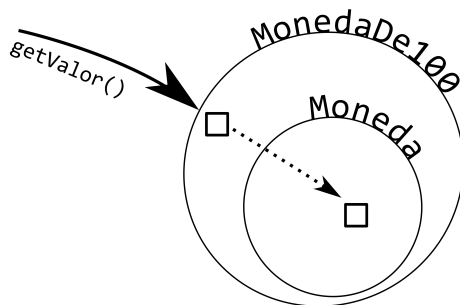
Lo anterior funcionaría de esa manera ya que `MonedaDe100` no tiene definido el método `getValor()`, o sea, no lo ha sobre-escrito (ver 7.2). En el caso de que `MonedaDe100` definiera ese método (o sea, sobre-escribiera el método de la clase base), entonces el mensaje se vería así:



y el código tiene que reflejar la sobre-escritura del método:

```
1 public class MonedaDe100 extends Moneda
2 {
3     @Override
4     public int getValor()
5     {
6         return 100;
7     }
8 }
```

Pero esto no es todo lo que se puede hacer. A veces, desde un método que ha sido sobre-escrito se requiere invocar el método “original”:



En ese caso, desde `Java` se debe usar la palabra `super`, que es parecida al `this`, pero en vez de apuntar a la instancia actual, siempre apunta a la instancia de la super-clase actual:

```

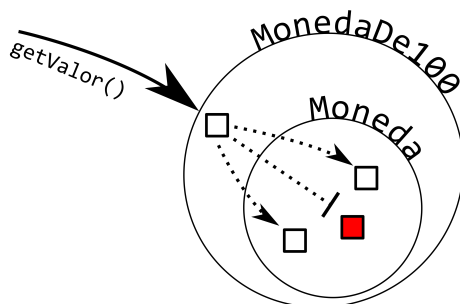
1 public class MonedaDe100 extends Moneda
2 {
3     @Override
4     public int getValor()
5     {
6         int vs = super.getValor();
7
8         if (vs > 100) return vs;
9
10        return 100;
11    }
12 }

```

Nótese cómo en la línea 6 se llama a la versión “original” del método (el equivalente a la línea punteada de la figura).

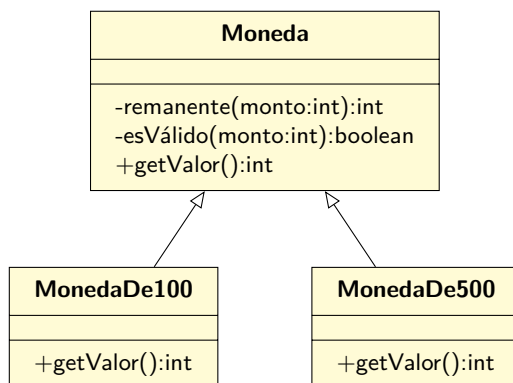
7.4. Lo privado NO se hereda

Por el hecho de que básicamente las subclases tienen “adentro” una instancia de su super clase, pudiera parecer lógico que se debería tener acceso a todo lo que fue declarado en la super clase:



La verdad es que esto no es así. En la figura la subclase está tratando de llamar a una operación que la clase base declaró como **private**, lo que causa un error de compilación.

Viéndolo con otro ejemplo más completo:



Es perfectamente válido que la clase `Moneda` tenga métodos privados:

```

1 public class Moneda
2 {
3     public int getValor()
4     {
5         return 0;
6     }
7
8     private boolean esVálido(int monto)
9     {
10        return monto > 0;
11    }
12
13    private int remanente(int monto)
14    {
15        while (true) {
16            int newMonto = monto - getValor();
17            if (!esVálido(newMonto)) return monto;
18            monto = newMonto;
19        }
20    }
21 }

```

Tal como está declarada la clase, es ilegal llamar a los métodos `esVálido(...)` y `remanente(...)` desde fuera de la clase. La única clase que puede llamar a dichos métodos es la misma clase `Moneda`:

```

1 public class TestMonedas
2 {
3     public static void main(String[] args)
4     {
5         MonedaDe100 m100 = new MonedaDe100();
6
7         int r = m100.remanente(10);
8         // The method remanente(int) from the type Moneda is not visible
9     }
10 }

```

Y como se decía recién, ésto también se aplica a las clases heredadas:

```

1 public class MonedaDe100 extends Moneda
2 {
3     @Override
4     public int getValor()
5     {
6         // The method remanente(int) from the type Moneda is not visible
7         int r = remanente(100);
8
9         int vs = super.getValor();
10
11        if (vs > 100) return vs;
12
13        return 100;
14    }
15 }

```

Los métodos privados son tan privados, que incluso las subclases pueden volver a declararlos, incluso como si fueran públicos (en este caso, haciendo público el método `esVálido(...)` que en la clase base es privado):

```
1 public class MonedaDe100 extends Moneda
2 {
3     @Override
4     public int getValor()
5     {
6         int vs = super.getValor();
7
8         if (vs > 100) return vs;
9
10        return 100;
11    }
12
13    public boolean esVálido(int monto)
14    {
15        return false;
16    }
17 }
```

Pero lo que no se puede realizar es reducir la visibilidad de un elemento heredado. Por ejemplo, si la clase `Moneda` definiera un método `public void test()`, entonces en `MonedaDe100`:

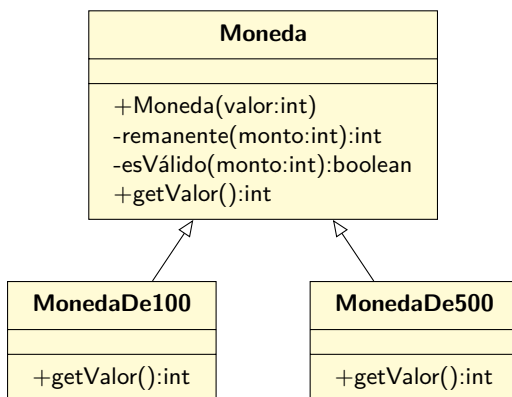
```
1 public class MonedaDe100 extends Moneda
2 {
3     ...
4     // Cannot reduce the visibility of the inherited method from Moneda
5     private void test()
6     {
7     }
8 }
```

7.5. Clases que no se pueden instanciar

Otra característica interesante al diseñar un programa usando herencia, es que se puede tomar la decisión de que ciertas clases no deben ser instanciables, ya que las que realmente interactuarán con el resto del sistema son las clases derivadas.

Aun cuando ésto se puede realizar usando otra técnica que será vista más adelante en la página 192, es importante saber cómo funciona esta versión de este comportamiento.

En este caso, para las clases `Moneda`, se podría tomar la decisión de realizar la siguiente modificación (nótese el nuevo constructor en `Moneda`):



Acá se ha tomado la decisión de que al instanciar una **Moneda**, ésta reciba el valor que presentará. Por lo tanto, se debe modificar el código de la clase:

```

1 public class Moneda
2 {
3     private int valor;
4
5     public Moneda(int valor)
6     {
7         this.valor = valor;
8     }
9     ...
10 }

```

Pero, al momento de hacer la modificación, las clases **MonedaDe100** y **MonedaDe500** dejan de funcionar:

```

1 // Implicit super constructor Moneda() is undefined for default constructor.
2 // Must define an explicit constructor
3 public class MonedaDe100 extends Moneda
4 {
5
6 }

```

Si nos imaginamos que al instanciar una **MonedaDe100**, también se debe instanciar la **Moneda** que lleva adentro, entonces tenemos que preguntarnos ¿cómo la vamos a instanciar, si para construir esa **Moneda** necesitamos invocar a su constructor pasándole un parámetro?

La respuesta es que también es necesario modificar **MonedaDe100** para realizar explícitamente la acción de inicializar la **Moneda**:

```

1 public class MonedaDe100 extends Moneda
2 {
3     public MonedaDe100()
4     {
5         super(100);
6     }
7 }

```

Al usar la palabra **super** de esa forma, estamos invocando al constructor de la superclase, pasándole como parámetro el valor que dicha clase espera para inicializar la nueva instancia de **Moneda**.

Hasta ahora todo esto no había sido necesario, ya que la clase superior solo tenía el constructor por defecto, y las clases derivadas lo llamaban de forma automática. El hecho de incorporar al modelo clases que dejan de tener el constructor por defecto nos obliga a empezar a llamarlos de forma explícita, pasándoles los parámetros correspondientes y requeridos.

Hay que hacer notar que esto funciona ya que el constructor de `Moneda` es `público`. Por ejemplo, si fuera `privado`:

```
1 public class Moneda
2 {
3     private int valor;
4
5     private Moneda(int valor)
6     {
7         this.valor = valor;
8     }
9     ...
10 }
```

Entonces `MonedaDe100` no podría instanciarse ya que sería imposible acceder al constructor privado:

```
1 public class MonedaDe100 extends Moneda
2 {
3     public MonedaDe100()
4     {
5         // The constructor Moneda(int) is not visible
6         super(100);
7     }
8 }
```

Evidentemente ésto significa que tampoco es factible instanciar una instancia de `Moneda` en ningún otro caso.

Pero, ¿qué pasa si en vez de `public` o `private`, cambiamos su visibilidad a `protected`?

```
1 public class Moneda
2 {
3     private int valor;
4
5     protected Moneda(int valor)
6     {
7         this.valor = valor;
8     }
9     ...
10 }
```

En este caso, el constructor de `MonedaDe100` vuelve a tener acceso al constructor, así que compila sin errores:

```
1 public class MonedaDe100 extends Moneda
2 {
3     public MonedaDe100()
4     {
5         super(100);
6     }
7 }
```

Pero, la clase `Moneda` no será instanciable desde fuera del `package`² donde está declarada:

²Pronto veremos qué es un `package`

```
1 public class TestMonedas
2 {
3     public static void main(String[] args)
4     {
5         // The constructor Moneda(int) is not visible
6         Moneda m = new Moneda(1);
7     }
8 }
```

Es importante destacar que tal como está declarado, el constructor es visible desde las subclases y desde todas las clases del mismo package. Revisa la tabla en el capítulo C en la página 429 para más detalles respecto a cómo encontrar la protección correcta para tus constructores.

7.6. El principio de sustitución de Liskov

Este principio es algo que usaremos en el resto del libro, y es la base de muchos de los diseños que se revisarán, y dice:

Los subtipos deben ser sustituibles por su tipo base

A una variable de un tipo dado se le puede asignar un valor de cualquiera de sus subtipos

En lenguajes como Java, ésto se puede realizar rápidamente, al declarar una clase como subclase de otra. A partir de ese momento, en cualquier parte de nuestro código que usemos una instancia de **X** podemos usar instancias de **Y**, **Z**, **W**, siempre que **Y**, **Z**, **W** sean subclases de **X**.

Por ejemplo:

```
1 public class SubclassTest
2 {
3
4     public static void main(String[] args)
5     {
6         ClaseBase x;
7
8         x = new ClaseDerivada1();
9         x = new ClaseDerivada2();
10        x = new ClaseDerivada3();
11    }
12 }
```

En este sentido, las instancia de `ClaseDerivada1` se pueden considerar que de todas formas son `ClaseBase`, de la misma forma en que las manzanas, naranjas y peras son frutas. De hecho, esto también es válido:

```
1 public class SubclassTest
2 {
3
4     public static void main(String[] args)
5     {
6         ClaseBase x;
7
8         x = new ClaseDerivada1();
9         x = new ClaseDerivada2();
10        x = new ClaseDerivada3();
11
12        unMétodo(x);
13        unMétodo(new ClaseDerivada2());
14    }
15
16    private static void unMétodo(ClaseBase x)
17    {
18    }
19
20 }
```

Es importante destacar que para que el principio de sustitución se cumpla a cabalidad, es necesario que las subclasses se comporten como su superclase. Por ejemplo, volviendo al ejemplo anterior:

```
1 public class Seleccionadora
2 {
3     public static void main(String[] args)
4     {
5         Seleccionadora x = new SeleccionadoraEspecial();
6
7         int r = x.seleccionarMayor(10, 25);
8         System.out.println(r);
9     }
10
11    public int seleccionarMayor(int i, int j)
12    {
13        if (i > j) return i;
14        return j;
15    }
16 }
```

En la línea 5 estamos aplicando el principio de sustitución, pero claramente la **SeleccionadoraEspecial** no se comporta como debe, ya que no está retornando el mayor, que es lo que claramente el método dice que va a hacer.

En este sentido, es importante que cada vez que se especializa el comportamiento (al sobrescribir un método), se cumpla el contrato³ definido, incluyendo todas las restricciones en sus parámetros de entrada, los cambios de estado del objeto y los efectos colaterales producto de la ejecución de la operación.

Para encontrarle mayor sentido al último código, es necesario conocer el siguiente concepto.

³Ver 9.2 en la página 234

7.7. Métodos Virtuales

Al momento de sobrescribir un método en una clase derivada (por lo tanto, el método está presente tanto en la clase base como en la subclase), es importante tener en consideración algo que sucede en Java:

En Java, todos los métodos son virtuales

¿Qué significa ésto? Considera este código:

```
1 public class Seleccionadora
2 {
3     public static void main(String[] args)
4     {
5         Seleccionadora x = new Seleccionadora();
6
7         System.out.println(x.seleccionarMayor(10, 25));
8     }
9
10    public int seleccionarMayor(int i, int j)
11    {
12        if (i > j) return i;
13        return j;
14    }
15 }
```

Es el mismo código de antes, por lo que no hay sorpresas al ejecutarlo:

```
$ java Seleccionadora
25
```

Pero si modificamos la línea 5:

```
1 public class Seleccionadora
2 {
3     public static void main(String[] args)
4     {
5         Seleccionadora x = new SeleccionadoraEspecial();
6
7         System.out.println(x.seleccionarMayor(10, 25));
8     }
9
10    public int seleccionarMayor(int i, int j)
11    {
12        if (i > j) return i;
13        return j;
14    }
15 }
```

Ya vimos que el resultado es 35, pero ¿cómo? Hagamos el ejemplo más complicado:

```
1 public class Seleccionadora
2 {
3     public static void main(String[] args)
4     {
5         Seleccionadora x = new Seleccionadora();
6     }
```

```

7      int r = x.invocarSeleccionador(x, 10, 25);
8
9      System.out.println(r);
10   }
11
12   public int invocarSeleccionador(Seleccionadora x, int a, int b)
13   {
14       return x.seleccionarMayor(a, b);
15   }
16
17   public int seleccionarMayor(int i, int j)
18   {
19       if (i > j) return i;
20       return j;
21   }
22 }

```

Mira cómo en la línea 7 se invoca el método `invocarSeleccionador(...)`, y se le pasa como primer parámetro la instancia creada en la línea 5. Dentro del método, solo se recibe la instancia y se invoca finalmente a `seleccionarMayor(...)`. Pero qué pasa si cambiamos el `main` para que haga esto:

```

1 public class Seleccionadora
2 {
3     public static void main(String[] args)
4     {
5         Seleccionadora x = new SeleccionadoraEspecial(); // Cambio!
6
7         int r = x.invocarSeleccionador(x, 10, 25);
8
9         System.out.println(r);
10    }
11
12    public int invocarSeleccionador(Seleccionadora x, int a, int b)
13    {
14        return x.seleccionarMayor(a, b);
15    }
16
17    public int seleccionarMayor(int i, int j)
18    {
19        if (i > j) return i;
20        return j;
21    }
22 }

```

En la línea 5 se crea una instancia de `SeleccionadoraEspecial`, y se la referencia usando un tipo de variable del tipo base. Y después se pasa la instancia a `invocarSeleccionador(...)`. Dentro de ese método, solo llega la instancia, pero la pregunta es: en la línea 14, ¿cómo se sabe a qué versión de `seleccionarMayor(...)` invocar? En ese punto del código no hay nada que nos diga que la instancia que llega como parámetro es de un tipo o de otro: solo se sabe que es una instancia de `Seleccionadora` (o alguna de sus derivados) y podemos pensar en que la clase derivada tiene ambos métodos (el propio, y el que hereda).

La respuesta es que aun cuando en la línea 14 pareciera que solo tenemos una instancia de `Seleccionadora`, la clase en sí sabe lo que es. Y al saberlo, puede “seleccionar” la versión correcta del método, para el tipo de instancia que realmente es. En este caso, es una instancia de `SeleccionadoraEspecial`, así que se invoca el método de dicha clase.

A esto se le llama un “método virtual”: la decisión de qué versión del método invocar se toma en tiempo de ejecución, dependiendo del tipo concreto de la instancia. También se le llama *despacho dinámico*. Una consecuencia de esto es que solo mirando el código, no podemos saber qué operación se ejecutará. Por ejemplo:

```
1 public int invocarSeleccionador(Seleccionadora x, int a, int b)
2 {
3     return x.seleccionarMayor(a, b);
4 }
```

Si solo tenemos eso, no hay forma de hacer el ruteo de este código, a menos que sepamos el tipo concreto al que está apuntando la instancia `x`.

Existen otros lenguajes en que el hecho de que un método sea o no virtual es una decisión de diseño, por ejemplo, C++.

7.8. Polimorfismo

Todo lo anterior nos permitirá construir soluciones de software que tengan un comportamiento bastante especial. Definiremos jerarquías de clases, donde en las clases base tendremos operaciones que después serán especializadas en las subclases, y aprovechando el despacho dinámico, podremos solo usar variables del tipo base, y las clases sabrán qué métodos invocar de forma automática.

De esta forma, usando un solo símbolo podremos representar diferentes tipos y comportamientos, lo que nos permitirá solucionar problemas complejos que se pueden modelar de forma natural en esta forma.

Un ejemplo clásico para demostrar este efecto es parecido a lo que se vió recién:

```
1 public class Animal
2 {
3     public String hablar()
4     {
5         return null;
6     }
7 }
```

```
1 public class Perro extends Animal
2 {
3     public String hablar()
4     {
5         return "Guau guau";
6     }
7 }
```

```
1 public class Conejo extends Animal
2 {
3     public String hablar()
4     {
5         return "[Los conejos no hablan]";
6     }
7 }
```

```
1 public class MainAnimales
2 {
3     public static void main(String[] args)
4     {
5         habla(new Perro());
6         habla(new Conejo());
7     }
8
9     private static void habla(Animal animal)
10    {
11        System.out.println(animal.hablar());
12    }
13 }
```

La clase base define un comportamiento, y las clases derivadas lo especializan. Desde el `main`, simplemente se llama a la operación, y en cada caso la ejecución llega al método apropiado para el tipo de dato concreto que se tiene:

```
$ java MainAnimales
Guau guau
[Los conejos no hablan]
```

Este ejemplo es correcto en su funcionamiento, pero podría ser mejor, por que tal como está escrito, cualquiera podría instanciar un nuevo `Animal`, pero eso no es algo que se debería permitir. En este sentido, la clase `Animal` solo existe para que su método `hablar()` lo puedan sobrescribir sus clases derivadas.

7.9. Clases abstractas

Continuando con el ejemplo, debemos agregar una nueva palabra a nuestro vocabulario:

```
1 public abstract class Animal
2 {
3     public String hablar() {
4         return null;
5     }
6 }
```

El efecto de la palabra `abstract` al especificar la clase, es que hace que el tipo sea imposible de instanciar:

```
1 public class MainAnimales
2 {
3     public static void main(String[] args)
4     {
5         new Animal(); // Cannot instantiate the type Animal
6     }
7 }
```

La razón de ser de las clases abstractas es proveer una base que las clases derivadas puedan sobrescribir y especializar. Pero aun hay un problema. Por ejemplo, es totalmente válido definir un nuevo tipo de `Animal` de esta forma:

```

1 public class Gato extends Animal
2 {
3 }

```

Aun cuando todo el código está orientado a que cada subclase de `Animal` tenga su propia forma de hablar, aun podemos crear la subclase sin necesidad de definir la operación. Para que ésto no suceda, debemos hacer que la operación **también** sea abstracta:⁴

```

1 public abstract class Animal
2 {
3     public abstract String hablar();
4 }

```

Con esta modificación, la clase `Gato` ya no es válida tal como está escrita:

```

1 // Error: The type Gato must implement the inherited abstract method Animal.hablar()
2 public class Gato extends Animal
3 {
4
5 }

```

Lo que se soluciona agregando el método a la clase, ya que se supone que cada `Animal` habla de forma diferente:

```

1 public class Gato extends Animal
2 {
3     public String hablar()
4     {
5         return "Miau";
6     }
7 }

```

Si agregamos al Gato en el main:

```

1 public class MainAnimales
2 {
3     public static void main(String[] args)
4     {
5         habla(new Perro());
6         habla(new Conejo());
7         habla(new Gato());
8     }
9
10    private static void habla(Animal animal)
11    {
12        System.out.println(animal.hablar());
13    }
14 }

```

Obtenemos:

```

$ java MainAnimales
Guau guau
[Los conejos no hablan]
Miau

```

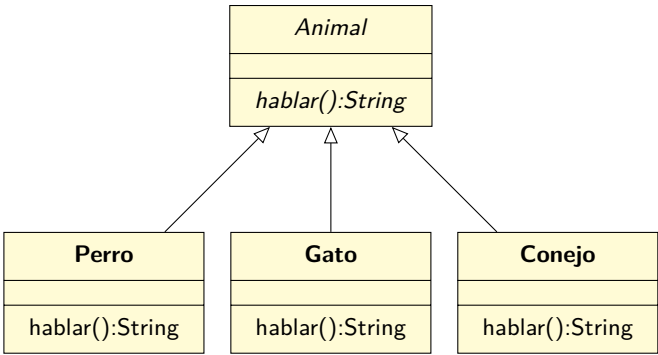
⁴Nótese que los métodos abstractos no tienen "cuerpo".

Una observación: en los ejemplos anteriores se ha omitido una palabra que las IDE (por ejemplo, Eclipse) agregan cuando la sobre-escritura de métodos se realiza usando la interfaz gráfica. Por ejemplo, en el `Gato`, al usar la automatización de la IDE, lo que se genera es esto:

```
1 public class Gato extends Animal
2 {
3     @Override
4     public String hablar()
5     {
6         return "Miau";
7     }
8 }
```

La IDE generó automáticamente la anotación⁵ `@Override`. Técnicamente ésta no es necesaria, o sea, el código va a compilar exitosamente de cualquier modo, pero es una ayuda para el compilador, de forma de informarle que el método indicado está pensado para sobre-escribir algo en la clase base. Si un método marcado de esa forma no sobre-escribe correctamente algo, entonces el compilador lanzará un error.

Para indicar que una clase es abstracta, en el diagrama se dibuja usando *itálicas*. A las operaciones también se les aplica lo mismo:



Cuando se muestran las operaciones en las clases concretas, entonces se escriben de forma normal (no en *itálica*).

7.10. No todo es abstracto

Aun cuando en el punto anterior puede haberse entendido que cuando se crea una clase abstracta, ésta debe ser completamente abstracta y no contener nada más, la verdad es que en la práctica se puede aprovechar el comportamiento de las clases para realizar cosas interesantes.

⁵annotation en inglés. Para ver más detalles acerca de las anotaciones, se puede consulta <https://docs.oracle.com/javase/tutorial/java/annotations/>

Por ejemplo, considera lo siguiente:

```

1 public class MainMaquinaria
2 {
3     public static void main(String[] args)
4     {
5         Piscina piscinaA = new Piscina(7400, 7400); // capacidad, litros actuales
6         Piscina piscinaB = new Piscina(4400, 0);
7
8         Bomba bomba = new Bomba(piscinaA, piscinaB);
9
10        System.out.println("Origen: " + piscinaA.getLitrosActuales() +
11                            ", destino: " + piscinaB.getLitrosActuales());
12
13        while (piscinaA.getLitrosActuales() > 0) {
14            int movidos = bomba.moverAgua();
15
16            System.out.println("Movidos: " + movidos +
17                              " origen: " + piscinaA.getLitrosActuales() +
18                              ", destino: " + piscinaB.getLitrosActuales());
19
20            if (movidos == 0) {
21                System.out.println("No se pudo mover más agua");
22                break;
23            }
24        }
25    }
26 }

```

Queremos simular el proceso de mover una cantidad de agua usando una **Bomba** para extraer líquido desde la **piscinaA** y dejarlo en la **piscinaB**. En el **main** se ve el proceso, que se repite mientras a la **piscinaA** le quede agua. El método **moverAgua** de la **Bomba** es la que realiza el proceso, y retorna la cantidad de agua movida en esa iteración.

Una primera versión de **Bomba** podría ser:

```

1 public class Bomba {
2     private Piscina origen;
3     private Piscina destino;
4
5     public Bomba(Piscina origen, Piscina destino)
6     {
7         this.origen = origen;
8         this.destino = destino;
9     }
10
11    public int moverAgua()
12    {
13        int extraidos = this.origen.extraer(147);
14        int movidos = this.destino.agregar(extraidos);
15        if (movidos < extraidos) {
16            int dif = extraidos - movidos;
17            int devueltos = this.origen.agregar(dif);
18            if (devueltos != dif) {
19                // Esto no debería ocurrir!
20                throw new IllegalStateException();
21            }
22        }
23        return movidos;
24    }
25 }

```

En este caso, tenemos una **Bomba** que siempre trata de mover de a 147 litros en cada iteración. Pero ¿qué pasa si queremos incluir en el modelo diferentes tipos de **Bomba**? Tal vez algunas más grandes o más pequeñas. Una alternativa es agregar la capacidad de la Bomba en el constructor, para así definir su capacidad al momento de crear la instancia:

```
1 public class Bomba
2 {
3     private Piscina origen;
4     private Piscina destino;
5     private int capacidad;
6
7     public Bomba(int capacidad, Piscina origen, Piscina destino)
8     {
9         this.capacidad = capacidad;
10        this.origen = origen;
11        this.destino = destino;
12    }
13
14    public int moverAgua()
15    {
16        int extraidos = this.origen.extraer(this.capacidad);
17        int movidos = this.destino.agregar(extraidos);
18        if (movidos < extraidos) {
19            int dif = extraidos - movidos;
20            int devueltos = this.origen.agregar(dif);
21            if (devueltos != dif) {
22                // Esto no debería ocurrir!
23                throw new IllegalStateException();
24            }
25        }
26        return movidos;
27    }
28 }
```

Obviamente, cuando instanciamos la **Bomba** tenemos que especificar su capacidad también.

Pero ¿qué pasa si queremos modelar otros comportamientos? Por ejemplo, a veces pasa que las bombas disminuyen su capacidad en la medida que pasa el tiempo. ¿Cómo hacemos eso con lo que tenemos hasta ahora? La verdad, es que la clase **Bomba** se complicaría ya que tendríamos que considerar todo lo anterior (llenando el código con **if** para considerar cada caso), y ver cómo calcular la capacidad en cada iteración.

Pero hay una alternativa, que en el fondo incluso es más fácil que todo lo anterior. Vamos a cambiar la línea 16 por una llamada a un nuevo método:


```

1 public class Bomba {
2     private Piscina origen;
3     private Piscina destino;
4
5     public Bomba(Piscina origen, Piscina destino)
6     {
7         this.origen = origen;
8         this.destino = destino;
9     }
10
11    public int moverAgua()
12    {
13        int extraidos = extraerDesdeOrigen();
14        int movidos = this.destino.agregar(extraidos);
15        if (movidos < extraidos) {
16            int dif = extraidos - movidos;
17            int devueltos = this.origen.agregar(dif);
18            if (devueltos != dif) {
19                // Esto no debería ocurrir!
20                throw new IllegalStateException();
21            }
22        }
23        return movidos;
24    }
25
26    protected int extraerDesdeOrigen()
27    {
28        int extraidos = this.origen.extraer(147);
29        return extraidos;
30    }
31 }

```

Aun dice 147, pero pronto cambiaremos eso:

```

1 public abstract class Bomba {
2     private Piscina origen;
3     private Piscina destino;
4
5     public Bomba(Piscina origen, Piscina destino)
6     {
7         this.origen = origen;
8         this.destino = destino;
9     }
10
11    public int moverAgua()
12    {
13        int extraidos = extraerDesdeOrigen();
14        int movidos = this.destino.agregar(extraidos);
15        if (movidos < extraidos) {
16            int dif = extraidos - movidos;
17            int devueltos = this.origen.agregar(dif);
18            if (devueltos != dif) {
19                // Esto no debería ocurrir!
20                throw new IllegalStateException();
21            }
22        }
23        return movidos;
24    }
25
26    protected abstract int extraerDesdeOrigen();
27 }

```

Desde ahora, la clase `Bomba` pasó a ser abstracta, así que no es instanciable, y si se quiere crear una subclase a partir de ella, se debe implementar el método `extraerDesdeOrigen()`. Aprovecharemos esto para crear una “bomba pequeña”:

```

1 public class BombaPequeña extends Bomba
2 {
3     public BombaPequeña(Piscina origen, Piscina destino)
4     {
5         super(origen, destino);
6     }
7
8     @Override
9     protected int extraerDesdeOrigen()
10    {
11        int extraidos = this.origen.extraer(13); // The field Bomba.origen is not
12        visible
13        return extraidos;
14    }
15 }
```

Pero apareció un error de compilación, ya que no hemos tomado la precaución de preparar la clase `Bomba` para que sea extendible. Si queremos que una clase pueda ser derivada, tenemos que dejar disponibles los atributos y métodos necesarios para que las subclases puedan implementar las operaciones que necesiten implementar. En este caso, el problema es el atributo `origen`, ya que está declarado como `private`, así que nadie (incluso las clases derivadas) pueden tener acceso a él.

La solución a esto es, o declarar un `getter` para obtener el `origen`, o declarar el atributo como `protected`, para que las subclases lo puedan acceder (y sería **muy mala idea** declararlo como `public`). En este caso, haremos esto último:

```

1 public abstract class Bomba
2 {
3     protected Piscina origen;
4     ...
5 }
```

Con esta modificación, solo falta corregir el `main` para que instancie una `BombaPequeña`:

```

1 public class MainMaquinaria
2 {
3     public static void main(String[] args)
4     {
5         Piscina piscinaA = new Piscina(7400, 7400);
6         Piscina piscinaB = new Piscina(4400, 0);
7
8         Bomba bomba = new BombaPequeña(piscinaA, piscinaB);
9         ...
10    }
11 }
```

Si ejecutamos el programa, funcionará bien, moviendo agua lentamente de una piscina a otra. Creemos una bomba que sea más rápida:

```

1 public class BombaGrande extends Bomba
2 {
3     public BombaGrande(Piscina origen, Piscina destino)
4     {
5         super(origen, destino);
6     }
7
8     @Override
9     protected int extraerDesdeOrigen()
10    {
11        int extraidos = this.origen.extraer(999);
12        return extraidos;
13    }
14 }

```

Y actualicemos el main:

```

8 Bomba bomba = new BombaGrande(piscinaA, piscinaB);

```

Si ejecutamos esta versión, rápidamente se vaciará la `piscinaA`:

```

$ java MainMaquinaria
Origen: 7400, destino: 0
Movidos: 999 origen: 6401, destino: 999
Movidos: 999 origen: 5402, destino: 1998
Movidos: 999 origen: 4403, destino: 2997
Movidos: 999 origen: 3404, destino: 3996
Movidos: 404 origen: 3000, destino: 4400
Movidos: 0 origen: 3000, destino: 4400
No se pudo mover más agua

```

Y, ¿qué pasa si la bomba es antigua y pierde rendimiento a medida que funciona? Ésto lo podemos modelar fácilmente creando una nueva subclase de Bomba:

```

1 public class BombaVieja extends Bomba
2 {
3     private int extracciones = 0;
4
5     public BombaVieja(Piscina origen, Piscina destino)
6     {
7         super(origen, destino);
8     }
9
10    @Override
11    protected int extraerDesdeOrigen()
12    {
13        extracciones++;
14        int rendimiento = 900 - extracciones * 91;
15        if (rendimiento < 0) rendimiento = 0;
16
17        int extraidos = this.origen.extraer(rendimiento);
18        return extraidos;
19    }
20 }

```

Este tipo de Bomba, entre más agua extrae, menos es su capacidad de extracción. Y si se lleva al límite, simplemente no extrae más agua:

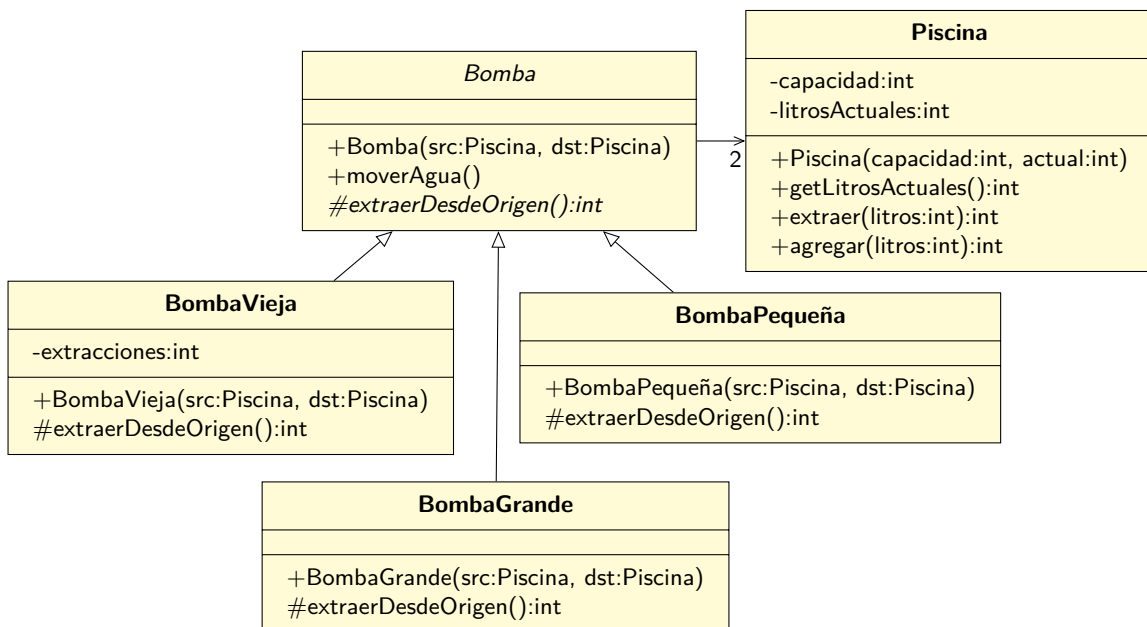
```

$ java MainMaquinaria
Origen: 7400, destino: 0
Movidos: 809 origen: 6591, destino: 809
Movidos: 718 origen: 5873, destino: 1527
Movidos: 627 origen: 5246, destino: 2154
Movidos: 536 origen: 4710, destino: 2690
Movidos: 445 origen: 4265, destino: 3135
Movidos: 354 origen: 3911, destino: 3489
Movidos: 263 origen: 3648, destino: 3752
Movidos: 172 origen: 3476, destino: 3924
Movidos: 81 origen: 3395, destino: 4005
Movidos: 0 origen: 3395, destino: 4005
No se pudo mover más agua

```

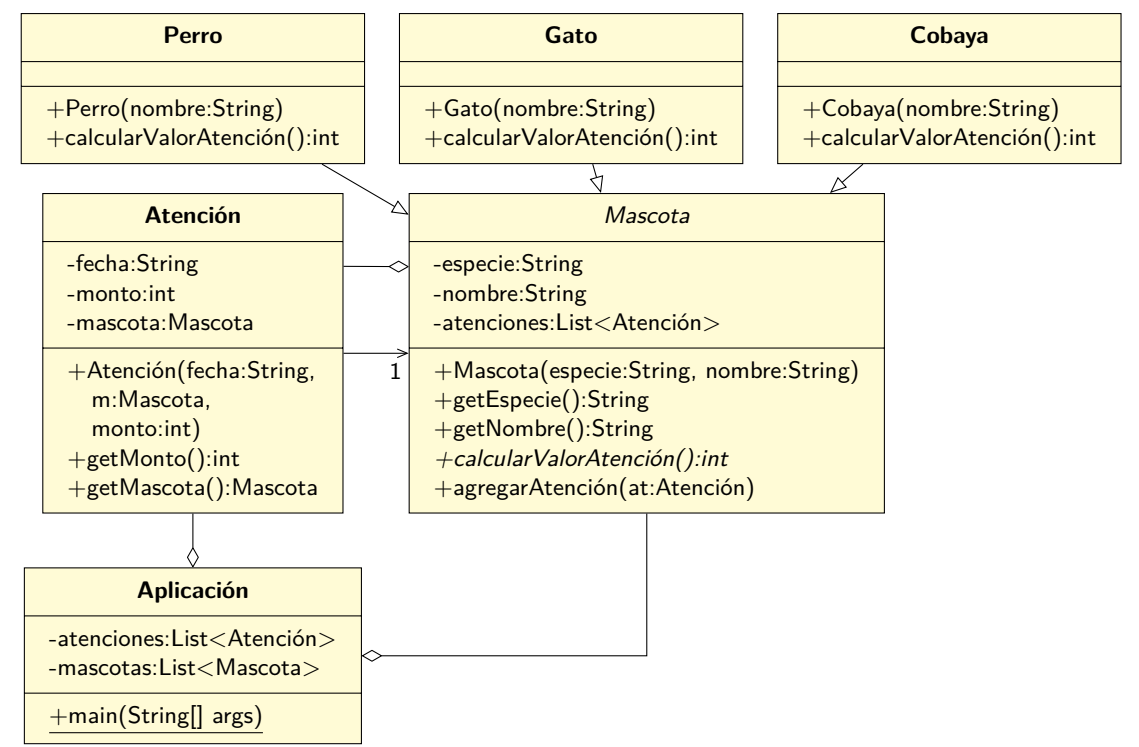
Entonces, en resumen, nótese cómo una clase puede tener un comportamiento base, pero igual ser declarada como abstracta, de forma de forzar a que se tengan que declarar clases derivadas y que completen su comportamiento. Y una ventaja de hacer las cosas de esta manera, es que esas clases derivadas pueden tener cualquier comportamiento: lo único que tienen que hacer es respetar el contrato de los métodos que están implementando.

Para este problema, el diagrama de clases se vería así:



7.11. Herencia

Volvamos al problema de la clínica veterinaria. Ahora que ya sabemos muchas cosas, podemos completarlo para implementarlo de una mejor manera:



Lo que se cambió fue que **Mascota** se transformó en una clase abstracta, que contiene un método abstracto `calcularValorAtención()`, que retornará el valor de una atención para la especie que corresponda. Nótese cómo cada especie está forzada a implementar la operación, y retornará el valor correcto para cada una.

Entonces, modificamos el método que registra una **Atención**:

```
1 public class Aplicación
2 {
3     ...
4     private void registrarAtención(String fecha, String nombreMascota)
5     {
6         Mascota m = buscarPorNombre(mascotas, nombreMascota);
7         int monto = m.calcularValorAtención();
8         Atención at = new Atención(fecha, m, monto);
9
10        atenciones.add(at);
11        m.agregarAtención(at);
12    }
13 }
```

Nótese cómo a este nivel no se toma ninguna decisión lógica, sino que solo se recopila la información y se registra la atención en los lugares que corresponde. Cada tipo de `Mascota` sabe cuánto vale su atención:

```
1 public class Perro extends Mascota
2 {
3     @Override
4     public int calcularValorAtención()
5     {
6         return 1000;
7     }
8 }
```

```
1 public class Gato extends Mascota
2 {
3     @Override
4     public int calcularValorAtención()
5     {
6         return 1500;
7     }
8 }
```

```
1 public class Cobaya extends Mascota
2 {
3     @Override
4     public int calcularValorAtención()
5     {
6         return 900;
7     }
8 }
```

7.12. Interfaces

Existe un concepto que es ligeramente similar a tener clases abstractas, y que nos servirá para definir las operaciones que nuestras clases deben tener.

De hecho, este concepto es casi equivalente a tener una clase totalmente abstracta, sin operaciones propias, y que solo sirve para forzar cierta “interfaz pública” en las clases que la derivan.

7.12.1. Un primer ejemplo

Imaginemos que vamos a crear un programa para conducir unos vehículos con ruedas. En particular, la persona dueña de estos vehículos usa uno diferente para ir a trabajar dependiendo del día de la semana.

Una de las primeras cosas que podemos concluir, a partir de nuestro conocimiento de cómo operan diferentes vehículos, es que en todos los casos, podemos avanzar y frenar, además de girar a la izquierda y derecha. Independiente de cómo realizan la acción, esto corresponde a la forma en que los humanos interactúan con la máquina. El detalle interno de cómo un vehículo particular realiza la acción no debería preocuparnos (por lo menos, hasta más adelante): tal vez un vehículo acelera usando un carburador, otro puede ser 100 % electrónico, o incluso podría ser a vapor (lo que implicaría echarle carbón a la caldera y abrir válvulas para dejar pasar el vapor).

Con esto definido, creemos la interfaz pública de todos los vehículos usando una nueva palabra que debemos agregar a nuestro vocabulario. La llamaremos **Vehículo**:

```
1 public interface Vehículo
2 {
3     void acelerar();
4     void frenar();
5     void girarDerecha();
6     void girarIzquierda();
7 }
```

Además, creemos un programa de prueba para ejercitar nuestro código:

```
1 public class TestVehiculos
2 {
3     public static void main(String[] args)
4     {
5         Vehículo ve = crearVehículo();
6
7         for(int i=0;i<100;i++) {
8             if (i % 2 == 0) {
9                 ve.acelerar();
10            } else if (i % 3 == 0) {
11                ve.frenar();
12            } else if (i % 5 == 0) {
13                ve.girarIzquierda();
14            } else if (i % 7 == 0) {
15                ve.girarDerecha();
16            }
17        }
18    }
19
20    private static Vehículo crearVehículo()
21    {
22        return null;
23    }
24 }
```

En este caso, desde el `main` le pediremos a la función `crearVehículo()` que nos cree un vehículo, pero por ahora, solo para probar, retornaremos `null`. En este caso, el código solamente compila: si lo tratamos de ejecutar fallará con un `NullPointerException`. Además, tenemos un ciclo para llamar a los diferentes métodos, de acuerdo a si el número es divisible por 2, 3, 5, o 7. Nótese que todo el código se ha escrito sin saber el tipo “real” de objeto que recibiremos (o sea, no sabemos la clase concreta, si es que es un auto, o carro tirado a caballos, o a vapor, etc). Simplemente estamos usando las operaciones que nos provee la interfaz. Más adelante podemos construir clases que implementen el comportamiento concreto de cada tipo, pero mientras implementen la interfaz especificada, el código seguirá funcionando sin problemas.

Ahora, veamos la creación de los vehículos. Modificaremos el método `crearVehículo()` de esta forma:

```

1 private static Vehículo crearVehículo()
2 {
3     // Vamos a obtener el día de la semana de "hoy"
4     Calendar date = Calendar.getInstance();
5     int dayOfTheWeek = date.get(Calendar.DAY_OF_WEEK);
6
7     // Dependiendo del día, es el vehículo que usamos:
8     switch(dayOfTheWeek) {
9         case Calendar.MONDAY:
10        case Calendar.THURSDAY:
11        case Calendar.FRIDAY:
12            // return new TeslaE3();
13            break;
14        default:
15            // return new Carruaje();
16    }
17    return null;
18 }

```

Por ahora, los return de las clases concretas están comentados, ya que necesitamos implementar dichas clases primero. Veamos el caso del `Carruaje`:

```

1 public class Carruaje implements Vehículo
2 {
3     private Caballo caballo = new Caballo();
4
5     @Override
6     public void acelerar() { caballo.darOrdenAcelerar(); }
7
8     @Override
9     public void frenar() { caballo.darOrdenFrenar(); }
10
11    @Override
12    public void girarDerecha() { caballo.darOrdenGirarDerecha(); }
13
14    @Override
15    public void girarIzquierda() { caballo.darOrdenGirarIzquierda(); }
16 }

```

Acá debemos agregar una nueva palabra a nuestro vocabulario. `implements` le indica al compilador que la clase `Carruaje` se compromete a implementar todas las operaciones definidas en la interfaz `Vehículo`. Si ésto no sucede, se generará un error de compilación.⁶

El carruaje es tirado por un caballo, por lo que las operaciones del vehículo se traducen finalmente en órdenes al caballo. Por ahora, lo implementaremos así:

```

1 public class Caballo
2 {
3     public void darOrdenAcelerar() { }
4     public void darOrdenFrenar() { }
5     public void darOrdenGirarDerecha() { }
6     public void darOrdenGirarIzquierda() { }
7 }

```

En el caso del otro tipo de vehículo se requiere tener motores:

⁶La verdad, es que ésto no es 100% cierto. Hay formas en que se puede hacer que la clase no implemente todos los métodos, pero ésto implica declarar la clase como abstracta. Pero al final, las clases concretas (las que se van a poder instanciar) no se van a salvar de tener que implementar todos los métodos para poder ser instanciadas.


```

1 public class Motor
2 {
3     public void acelerar() { }
4     public void frenar() { }
5 }

```

Entonces, tendremos lo siguiente:

```

1 public class TeslaE3 implements Vehículo
2 {
3     private Motor motorIzquierda = new Motor();
4     private Motor motorDerecha = new Motor();
5
6     @Override
7     public void acelerar()
8     {
9         motorIzquierda.acelerar();
10        motorDerecha.acelerar();
11    }
12
13    @Override
14    public void frenar()
15    {
16        motorIzquierda.frenar();
17        motorDerecha.frenar();
18    }
19
20    @Override
21    public void girarDerecha()
22    {
23        motorIzquierda.acelerar();
24        motorDerecha.frenar();
25    }
26
27    @Override
28    public void girarIzquierda()
29    {
30        motorIzquierda.frenar();
31        motorDerecha.acelerar();
32    }
33 }

```

Supondremos que este vehículo tiene tracción delantera, y que tiene un motor conectado a cada rueda, por lo tanto para cumplir las operaciones de la interfaz, la secuencia de pasos que tiene que realizar es diferente (respecto al vehículo tirado por un caballo), pero igual cumple cada operación.

Desde el punto de vista del “usuario” de la interfaz (o sea, los que van a instanciar estas clases), no hay diferencia entre ambos tipos de vehículos: ambos operan de la misma forma, ambos son **Vehículo**.

Ahora podemos descomentar las líneas para crear las clases específicas de vehículos:

```

1 private static Vehículo crearVehículo()
2 {
3     // Vamos a obtener el día de la semana de "hoy"
4     Calendar date = Calendar.getInstance();
5     int dayOfTheWeek = date.get(Calendar.DAY_OF_WEEK);
6
7     // Dependiendo del día, es el vehículo que usamos:
8     switch(dayOfTheWeek) {
9         case Calendar.MONDAY:
10        case Calendar.THURSDAY:
11        case Calendar.FRIDAY:
12            return new TeslaE3();
13        default:
14            return new Carruaje();
15    }
16 }

```

7.13. Sobrecarga de métodos

En el punto 7.2 (página 178) se analizó lo que significa la sobre-escritura de métodos, y es la base para la especialización de las subclases. Pero hay algo similar que podemos hacer para hacerle la vida más fácil a las personas que escriben código y son “clientes” de nuestras clases.

Veamos un ejemplo:

```

1 public class Printer
2 {
3     public void print(String texto)
4     {
5         System.out.println("Escribiendo: " + texto);
6     }
7 }

```

Esta clase sirve para escribir cosas a la pantalla. Por ejemplo:

```

1 public class Test6
2 {
3     public static void main(String[] args)
4     {
5         Printer p = new Printer();
6
7         p.print("Hola Mundo!");
8     }
9 }

```

En este caso, es muy fácil escribir un `String`, pero si quisiéramos escribir un `int`, tendríamos que primero convertirlo a `String`:

```

1 public class Test6 {
2     public static void main(String[] args) {
3         Printer p = new Printer();
4
5         p.print("Hola Mundo!");
6         p.print("" + 2022); // No es la mejor forma de convertir un int a String
7     }
8 }

```

Con lo que se obtiene:

```
$ java Test6
Escribiendo: Hola Mundo!
Escribiendo: 2022
```

Pero si queremos escribir el valor de otros tipos de variables, se tendría que convertir todo a String:

```
1 public class Test6
2 {
3     public static void main(String[] args)
4     {
5         Printer p = new Printer();
6
7         p.print("Hola Mundo!");
8         p.print(" " + 2022);
9         p.print(" " + true);
10        p.print(" " + 3.14);
11    }
12 }
```

```
$ java Test6
Escribiendo: Hola Mundo!
Escribiendo: 2022
Escribiendo: true
Escribiendo: 3.14
```

En este caso, tal vez sería mejor agregar métodos adecuados a la clase `Printer` para que sea más fácil escribir a la pantalla (más fácil desde el punto de vista del cliente que usa la clase):

```
1 public class Test6
2 {
3     public static void main(String[] args)
4     {
5         Printer p = new Printer();
6
7         p.printString("Hola Mundo!");
8         p.printInt(2022);
9         p.printBoolean(true);
10        p.printDouble(3.14d);
11    }
12 }
```

Si diseñamos la clase `Printer` para que funcione de esa forma, entonces los clientes podrán usarla más fácilmente:

```
1 public class Printer
2 {
3     public void printString(String txt) { System.out.println("Escribiendo: " + txt); }
4
5     public void printInt(int i) { System.out.println("Escribiendo: " + i); }
6
7     public void printBoolean(boolean b) { System.out.println("Escribiendo: " + b); }
8
9     public void printDouble(double d) { System.out.println("Escribiendo: " + d); }
10 }
```

Pero hemos incluido una debilidad a la clase `Printer`, ya que cada uno de los métodos repite el mismo String ("Escribiendo"). Es mejor refactorizar la clase para mejorarla

```

1 public class Printer
2 {
3     public void printString(String txt) { System.out.println("Escribiendo: " + txt); }
4
5     public void printInt(int i)          { this.printString("" + i); }
6
7     public void printBoolean(boolean b) { this.printString("" + b); }
8
9     public void printDouble(double d)   { this.printString("" + d); }
10 }

```

Esta versión es mucho mejor, pero podemos realizar una última mejora para hacerle la vida más fácil a nuestros clientes:

```

1 public class Printer
2 {
3     public void print(String txt) { System.out.println("Escribiendo: " + txt); }
4
5     public void print(int i)
6     {
7         // En vez de hacer la fea concatenación, aprovecharemos que la clase
8         // String provee el método valueOf(...) para convertir algo a String
9         //
10        this.print(String.valueOf(i));
11    }
12
13    public void print(boolean b) { this.print(String.valueOf(b)); }
14
15    public void print(double d) { this.print(String.valueOf(d)); }
16 }

```

No es ningún error tener una serie de métodos con el mismo nombre, pero con diferentes parámetros. A ésto se le llama “sobrecarga de métodos”. Por lo tanto, el código que usa a esta clase queda mucho más simple:

```

1 public class Test6
2 {
3     public static void main(String[] args)
4     {
5         Printer p = new Printer();
6
7         p.print("Hola Mundo!");
8         p.print(2022);
9         p.print(true);
10        p.print(3.14d);
11    }
12 }

```

De esta forma, el cliente de la clase no tiene que preocuparse de encontrar el método “correcto”, sino que solo debe invocar el método `print`, y el compilador invocará de forma correcta la versión del método que tiene los parámetros que coincidan con los elementos pasados al hacer la llamada.

Hay otro ejemplo clásico que podemos usar para ejemplificar la sobrecarga de métodos. Imaginemos que tenemos una clase que nos permite representar puntos en un plano cartesiano:

```

1 public class Punto2D
2 {
3     private int x, y;
4
5     public Punto2D(int x, int y)
6     {
7         this.x = x;
8         this.y = y;
9     }
10
11     public double distanciaCon(Punto2D otro)
12     {
13         int dx = this.x - otro.x;
14         int dy = this.y - otro.y;
15         return Math.sqrt(dx * dx + dy * dy);
16     }
17 }

```

Esta clase además nos provee la funcionalidad de poder calcular la distancia entre “este” punto y “otro” punto. Para realizar la operación se recibe el “otro” punto como parámetro, y se hace la operación matemática apropiada.

Si en las líneas 13 y 14 parece extraño que podamos acceder directamente a los atributos `x` e `y` de la “otra” clase, hay que recordar que esa “otra” clase es igualmente una instancia de la misma clase, así que hay total acceso a todo lo privado de ella.

Pero también podría ser útil para los usuarios de la clase que se pudiera calcular la distancia entre “this” punto y un par de coordenadas x, y cualquiera. Sobrecarguemos el método:

```

1 public class Punto2D
2 {
3     private int x, y;
4
5     public Punto2D(int x, int y)
6     {
7         this.x = x;
8         this.y = y;
9     }
10
11     public double distanciaCon(Punto2D otro)
12     {
13         int dx = this.x - otro.x;
14         int dy = this.y - otro.y;
15         return Math.sqrt(dx * dx + dy * dy);
16     }
17
18     public double distanciaCon(int x, int y)
19     {
20         int dx = this.x - x;
21         int dy = this.y - y;
22         return Math.sqrt(dx * dx + dy * dy);
23     }
24 }

```

Nótese cómo en las líneas 20 y 21 se aplica lo aprendido respecto al orden en que se buscan las variables para evitar ambigüedades. Pero aun esta clase tiene código duplicado, lo que siempre es un problema. Refactoricemos:

```
1 public class Punto2D
2 {
3     private int x, y;
4
5     public Punto2D(int x, int y)
6     {
7         this.x = x;
8         this.y = y;
9     }
10
11     public double distanciaCon(Punto2D otro)
12     {
13         return this.distanciaCon(otro.x, otro.y);
14     }
15
16     public double distanciaCon(int x, int y)
17     {
18         int dx = this.x - x;
19         int dy = this.y - y;
20         return Math.sqrt(dx * dx + dy * dy);
21     }
22 }
```

Como se puede adivinar, usar sobrecarga de métodos no es una obligación, pero es una buena idea para hacerle la vida más fácil a nuestros clientes.

7.14. Casting con objetos

En la página 16 vimos el concepto de casting, pero ahora revisaremos cómo se aplica cuando las variables son referencias a objetos, en vez de datos primitivos.

Para ejemplificar, creemos algunas clases que tienen una relación de herencia entre ellas. Primero una clase base:

```
1 public class Motor
2 {
3 }
```

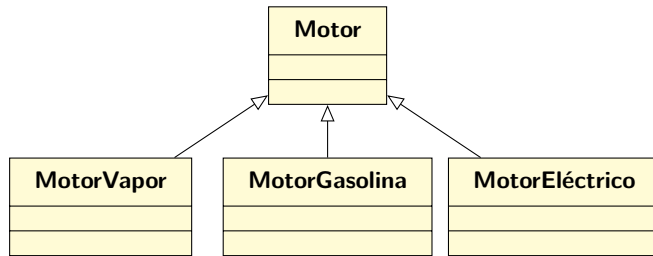
Y después unas clases derivadas:

```
1 public class MotorEléctrico extends Motor
2 {
3 }
```

```
1 public class MotorGasolina extends Motor
2 {
3 }
```

```
1 public class MotorVapor extends Motor
2 {
3 }
```

En otras palabras, hemos creado esta pequeña jerarquía de clases:



Hay cosas que no necesitan casting, como el asignar una instancia a su **super** clase (ver 7.6):

```
1 public class TestMotor
2 {
3     public static void main(String[] args)
4     {
5         Motor m1 = new MotorVapor();
6     }
7 }
```

Pero hay acciones en las que inmediatamente se generan errores:

```
1 public class TestMotor
2 {
3     public static void main(String[] args)
4     {
5         Motor m1 = new MotorVapor();
6
7         MotorVapor mv = m1; // Type mismatch: cannot convert from Motor to MotorVapor
8     }
9 }
```

Es importante entender el por qué se genera el mensaje de error. Uno podría pensar que por el hecho de que en la línea 5 se está construyendo una instancia de **MotorVapor** y la estamos dejando referenciada con una variable de tipo **Motor**, después en la línea 7 no habrá problema en referenciarla con una variable de su tipo concreto, ya que en el fondo la instancia sigue siendo un **MotorVapor**.

Pero la verdad es que hay que tener en cuenta que por el hecho de estar usando una variable de tipo **Motor**, técnicamente se podría estar apuntando a cualquier instancia de las subclases. O sea, con un **Motor** se puede referenciar a un **MotorVapor**, **MotorGasolina** y **MotorEléctrico**.

Por ejemplo, en la línea 5 se podría estar instanciando la clase **MotorGasolina**, y en la línea 7 se trataría de convertir dicho motor a **MotorVapor**, lo que es inválido ya que no son tipos compatibles, así que no se puede automáticamente realizar la conversión. De hecho, en este sentido podemos cometer muchos errores:

```

1 public class TestMotor
2 {
3     public static void main(String[] args)
4     {
5         Motor m1 = new MotorGasolina();
6         Motor m2 = new MotorEléctrico();
7         Motor m3 = null;
8         Motor m4 = new Motor();
9
10        MotorVapor mv1 = m1; // Type mismatch: cannot convert from Motor to MotorVapor
11        MotorVapor mv2 = m2; // Type mismatch: cannot convert from Motor to MotorVapor
12        MotorVapor mv3 = m3; // Type mismatch: cannot convert from Motor to MotorVapor
13        MotorVapor mv4 = m4; // Type mismatch: cannot convert from Motor to MotorVapor
14    }
15 }

```

En la práctica, las únicas asignaciones que se pueden realizar de forma válida son aquellas que van “subiendo” por la jerarquía de clases: por ejemplo, una instancia de `MotorGasolina` siendo referenciada por un tipo `Motor`. Cualquier otra conversión, en alguna otra dirección, por ejemplo, bajando desde `Motor` a `MotorVapor`, o entre clases hermanas como `MotorGasolina` a `MotorVapor`, no se puede realizar automáticamente.

Algo importante acá es la palabra *automáticamente*. Siempre es factible tratar de forzar la conversión aplicando casting:

```

1 public class TestMotor
2 {
3     public static void main(String[] args)
4     {
5         Motor m1 = new MotorGasolina();
6         Motor m2 = new MotorEléctrico();
7         Motor m3 = null;
8         Motor m4 = new Motor();
9         Motor m5 = new MotorVapor();
10
11        MotorVapor mv1 = (MotorVapor) m5;
12    }
13 }

```

Este código compilará exitosamente, y también se ejecutará sin problemas, ya que literalmente estamos convirtiendo una instancia de `MotorVapor` para que vuelva a ser apuntada por una variable del tipo concreto.

Aplicando casting también se puede compilar exitosamente código que contenga otros castings:

```

1 public class TestMotor
2 {
3     public static void main(String[] args)
4     {
5         Motor m1 = new MotorGasolina();
6         Motor m2 = new MotorEléctrico();
7         Motor m3 = null;
8         Motor m4 = new Motor();
9         Motor m5 = new MotorVapor();
10
11        MotorVapor mv1 = (MotorVapor) m5;
12        MotorVapor mv2 = (MotorVapor) m1;
13    }
14 }

```


Fíjate cómo en la línea 12 se está convirtiendo una referencia de tipo `Motor`, que apunta a una instancia de `MotorGasolina`, a una referencia de tipo `MotorVapor`. Pareciera que todo está bien, pero al ejecutar este código se obtiene:

```
$ java TestMotor
Exception in thread "main" java.lang.ClassCastException:
ejemplo7.MotorGasolina cannot be cast to ejemplo7.MotorVapor
at ejemplo7.TestMotor.main(TestMotor.java:12)
```

La `ClassCastException` es la forma en que la máquina virtual nos dice que los tipos no son compatibles. No es como en el caso de la línea 11, donde el casting es válido, ya que hay compatibilidad (de hecho, `m5` es de tipo `MotorVapor`), por lo que no hay error. Pero definitivamente no se puede convertir una variable de un tipo a un tipo de una clase “hermana”, aun cuando le pidamos al compilador que haga casting.

En resumen, cuando hacemos casting con objetos, hay que asegurarse de que los tipos son compatibles entre sí.

7.15. toString

Hasta ahora, en muchas clases que hemos revisado ha aparecido el método `toString()` y solo se ha dicho que es un método que debería retornar un `String` que represente el estado actual de una instancia determinada.

Lo anterior es correcto, pero conviene aclarar un par de puntos. Usemos esta clase como ejemplo:

```
1 public class Figura
2 {
3     private String tipo;
4
5     public Figura(String tipo)
6     {
7         this.tipo = tipo;
8     }
9 }
```

Podemos instanciar esta clase, y extrañamente, aun cuando no tiene declarado el método `toString()`, de igual manera lo podemos invocar:

```
1 public class TestToString
2 {
3     public static void main(String[] args)
4     {
5         Figura f = new Figura("cuadrado");
6
7         System.out.println(f.toString());
8     }
9 }
```

Aunque el resultado que se obtiene no tiene mucho sentido:

```
ejemplo15.Figura@7852e922
```

Si queremos obtener algo que nos sea útil, debemos crear explícitamente el método:

```

1 public class Figura
2 {
3     private String tipo;
4
5     public Figura(String tipo)
6     {
7         this.tipo = tipo;
8     }
9
10    @Override
11    public String toString()
12    {
13        return "Figura [tipo=" + tipo + "]";
14    }
15 }

```

Y en este caso, al repetir el test, se obtiene lo que estamos esperando:

Figura [tipo=cuadrado]

Un detalle que puede ser importante, es que si vamos a simplemente hacer “print” del resultado de ejecutar `toString()`, podemos escribir directamente:

```

1 public class TestToString
2 {
3     public static void main(String[] args)
4     {
5         Figura f = new Figura("cuadrado");
6
7         System.out.println(f); // hey! sin toString()!
8     }
9 }

```

Lo anterior es posible ya que el método `println` de la clase involucrada (`PrintStream`), provee este método:

```

1 public class PrintStream {
2     ...
3     public void println(Object x) {
4         String s = String.valueOf(x);
5         ...
6     }
7     ...
8 }

```

Que a la vez invoca al método estático `valueOf()` de la clase `String`:

```

1 public class String {
2     ...
3     public static String valueOf(Object obj) {
4         return (obj == null) ? "null" : obj.toString();
5     }
6     ...
7 }

```

Y si el objeto pasado por parámetro no es `null` (y recuerda que inicialmente es una instancia de `Figura`), se termina llamando a `toString()` del tipo `Object`, el cuál está declarado en dicha clase:

```
1 public class Object {  
2     ...  
3     public String toString() {  
4         ...  
5     }  
6     ...  
7 }
```

Y volviendo a **Figura**, ésta sobrescribe el método, así que como en **Java** todos los métodos son virtuales, la ejecución termina llegando a ese método:

```
1 public class Figura  
2 {  
3     ...  
4     @Override  
5     public String toString()  
6     {  
7         return "Figura [tipo=" + tipo + "];"  
8     }  
9 }
```

y se obtiene la salida correcta por la pantalla:

```
Figura [tipo=cuadrado]
```

7.16. final

Algo que seguramente no se verá mucho en este libro, pero que igual es importante conocer, es la palabra `final` en Java.

Dependiendo del contexto, su significado es diferente.

7.16.1. Variables, parámetros y atributos

En este caso, el uso de `final` al declarar una variable indica que se espera que dicha variable (o parámetro o atributo) no cambie de valor. En caso de realizarse alguna operación que trate de cambiar su valor, el código no va a compilar:

```
1 public class TestFinal
2 {
3     public static void main(String[] args)
4     {
5         int q = 10;
6
7         TestFinal test = new TestFinal();
8         test.proceso(q);
9     }
10
11     private final int pos = 10;
12
13     private void proceso(final int q)
14     {
15         final int x = q;
16
17         // The final local variable x cannot be assigned.
18         // It must be blank and not using a compound assignment
19         x += 10;
20
21         // The final local variable q cannot be assigned.
22         // It must be blank and not using a compound assignment
23         q += 20;
24
25         // The final field TestFinal.pos cannot be assigned
26         this.pos++;
27     }
28 }
```

Es muy recomendable que a menos que sepamos que los valores de los parámetros van a cambiar, los declaremos usando `final`.

7.16.2. Métodos

Si como parte del diseño necesitamos que un método no se pueda sobre-escribir en una de sus clases derivadas, entonces se le debe aplicar la palabra `final`:

```

1 public class TestFinal
2 {
3     public static void main(String[] args)
4     {
5         int q = 10;
6
7         TestFinal test = new TestFinal();
8         test.proceso(q);
9     }
10
11     final protected void proceso(final int q)
12     {
13     }
14 }
```

Si se trata de sobre-escribir el método, se producirá un error de compilación:

```

1 public class TestFinalHija extends TestFinal
2 {
3     @Override
4     protected void proceso(int q) // Cannot override the final method from TestFinal
5     {
6         System.out.println("Estoy acá!");
7     }
8 }
```

7.16.3. Clases

Por otro lado, si como parte del diseño se requiere que una clase no sea heredable, entonces ésta se tiene que marcar como `final`, tal como sucede con la clase `String`:

```

1 public final class String
2 {
3     ...
4 }
```

Si se intenta realizar la herencia, se generará un error de compilación:

```

1 // The type TestHerenciaString cannot subclass the final class String
2 public class TestHerenciaString extends String
3 {
4 }
```

7.17. Ejercicios

1. Crea una clase que permita calcular el tamaño de una línea, especificando la línea usando dos puntos en el plano. Para hacernos la vida más fácil, crea una versión donde los puntos son coordenadas enteras, y crea otra versión donde los puntos son números decimales (float y double). Usa sobrescritura de métodos para que no tengamos que preocuparnos de llamar al método correcto.
2. Complementa el ejemplo anterior, y crea versiones del método para cuando la línea se define usando combinaciones de puntos definidos como enteros y decimales. Por ejemplo, calcula el tamaño de la línea entre los puntos $(-3,21; 8)$ y $(4; 4,33)$.

3. Construye una clase `InstrumentoMusical`, que tenga un atributo llamado `frecuencia` y un método llamado `tocar()`, que “toca” la frecuencia indicada en el atributo (por ahora, solo pon un `println` en el método, para indicar que el instrumento está “tocando” la frecuencia de sonido indicada).

Crea una clase `Violín` que hereda de `InstrumentoMusical`, y agrégale un método llamado `articular(...)` que recibe como parámetro el nombre de una nota musical. Sobre-escribe el método `tocar()` en `Violín`, de forma que cuando sea llamado, primero configure la `frecuencia` del instrumento a partir de la nota musical que está siendo tocada en el violín, y después llame a `tocar()` en `InstrumentoMusical` para finalmente tocar el sonido con la frecuencia correcta (a partir de la nota musical del violín).

4. En el problema anterior, ¿qué técnica usaste para que la `frecuencia` pudiera ser visible y modificable desde la clase derivada `Violín`? Modifica tu código para crear varias versiones:

- a) `frecuencia` es `private`, y tiene getter y setter público.
- b) `frecuencia` es `protected`, y no tiene getter ni setter.
- c) `frecuencia` es `public`, y no tiene getter ni setter.

¿Cuál crees que es la mejor versión?

5. En el problema anterior, haz que `InstrumentoMusical` no sea instanciable.
6. ¿Qué pasaría si haces que `InstrumentoMusical` sea *abstracta*? ¿Qué pasaría si el método `tocar()` en `InstrumentoMusical` fuera abstracto?
7. ¿Es posible que `InstrumentoMusical` fuera una interface?
8. ¿En qué sentido el casting con objetos funciona? ¿Horizontal? ¿Vertical? ¿Ninguno de los anteriores?
9. Revisa el código fuente de la clase `Object` para que veas cómo está implementado el método `toString()`.
10. Revisa el código fuente de `System` para ver cómo funciona `out` y cómo funciona su `println(...)`.
11. En la página 205 completa las operaciones de las clases `Motor` y `Caballo` para que escriban por pantalla mensajes apropiados cada vez que sus métodos son llamados.

8

SOLID

Es interesante cómo a pesar de todo lo revisado hasta ahora respecto a técnicas de modelado y buenas prácticas al crear software orientado al objeto, de igual manera si no se tiene cuidado y se es riguroso al construir los programas, podemos obtener software difícil de escribir, imposible de mantener y que incluso podría no satisfacer los requerimientos iniciales.

Para aumentar la solidez de lo que se escribe, allá por el año 2000 Robert C. Martin¹ escribió un artículo titulado “Design Principles and Design Patterns”[4], donde introdujo una serie de principios generales, de los cuales cinco se conocieron más adelante como los *Principios SOLID*².

Estos principios nos pueden guiar para escribir software que sea más mantenible, entendible y flexible. Aplicándolos de forma apropiada nuestro software será más resistente a los problemas que se generan cuando éste envejece y crece en tamaño.

Los principios SOLID se llaman de esa forma por los conceptos que forman la palabra (en inglés):

- **Single responsibility** (Responsabilidad única)
- **Open/Closed** (Abierto/Cerrado)
- **Liskov substitution** (Sustitución de Liskov)
- **Interface Segregation** (Segregación por interfaces)
- **Dependency Inversion** (Inversión de dependencias)

La mayoría de estos conceptos ya se han revisado de alguna u otra forma, pero acá se mostrarán algunos ejemplos específicos para demostrar su utilidad.

¹Uncle Bob

²En el 2004 por Michael Feathers[1]

8.1. Single Responsibility

Una clase solo debe tener una responsabilidad. Más aun, solamente debe tener una razón para cambiar.

¿Para qué sirve diseñar las clases de esta forma?

Testing Una clase que solo tiene una responsabilidad será más fácil de testear.

Bajo acoplamiento Una clase más “pequeña” tendrá menos dependencias.

Organización Las clases pequeñas son más fáciles de entender, organizar y buscar que las clases con mas funcionalidad.

Veamos un ejemplo:

```
1 public class Camión
2 {
3     private String patente;
4     private int cargaMáxima;
5     private int cargaActual;
6
7     public Camión(String patente, int máxima, int actual)
8     {
9         this.patente = patente;
10        this.cargaMáxima = máxima;
11        this.cargaActual = actual;
12    }
13
14    public boolean cargar(int carga)
15    {
16        int nuevaActual = cargaActual + carga;
17        if (nuevaActual > cargaMáxima) {
18            return false;
19        }
20        cargaActual = nuevaActual;
21        return true;
22    }
23 }
```

Estamos modelando un camión que tiene una capacidad de carga definida, y al cuál podemos agregar más carga. El camión recibirá la carga mientras tenga capacidad, sino nos lo indicará retornando false en la operación `cargar(...)`.

Esta clase cumple muy bien el principio de tener una sola responsabilidad: es un modelo del camión que realiza las operaciones de acuerdo al problema que se quiere solucionar, y las realiza bien, sin involucrar a otros elementos ajenos a su preocupación principal.

Ahora, supongamos que apareció un requerimiento que dice que se debe escribir por pantalla el estado de cada camión, indicando su patente y su carga. Implementemos lo primero que se nos ocurra:


```
1 public class Camión
2 {
3     ...
4     public void escribirAConsola()
5     {
6         // Acá va el código para escribir este camión a
7         // la pantalla
8     }
9 }
```

Como muchas veces las primeras ideas son pésimas, este código, que a primera vista pareciera que no tiene ningún problema, la verdad es que viola el principio de la responsabilidad única. Por un lado tenemos la responsabilidad de implementar las operaciones que hacen que un camión sea un camión, pero uno también debería preguntarse de quién es la responsabilidad de “escribir” al camión a la pantalla.

La mejor forma de resolver ésto es implementar una clase separada que se encargue de escribir cosas a la pantalla, por ejemplo, camiones:

```
1 public class CamiónPrinter
2 {
3     public void escribirAConsola(Camión camión)
4     {
5     }
6 }
7 }
```

Con esta clase nueva, estamos liberando a la clase `Camión` para que haga lo que le corresponde, y además, tenemos la posibilidad de aumentar el diseño de la clase `Printer`. Si se hace de buena manera, se podría implementar de forma de tener una serie de clases `Printer`, cada una imprimiendo camiones a diferentes elementos (por ejemplo, a la pantalla, a un archivo, a un email, etc.), y todo sin tener que modificar ni aumentarle las responsabilidades a la clase `Camión`.

8.2. Open/Closed

El principio de abierto/cerrado significa que las clases deben estar abiertas para ser extendidas, pero cerradas para modificaciones. Si se logra hacer ésto, podremos dejar de modificar clases que potencialmente funcionan correctamente, y por lo tanto bajamos la posibilidad de agregar nuevos “bugs” a software existente.

Volviendo a la clase `Camión`, imaginemos que tiempo después decidimos que los camiones ahora se pueden pintar de diferentes colores. Evidentemente lo más fácil de hacer es simplemente abrir el archivo donde está la clase, y agregarle los elementos nuevos:

```

1 public class Camión
2 {
3     private String patente;
4     private int cargaMáxima;
5     private int cargaActual;
6     private String color; // Nuevo atributo
7
8     public Camión(String patente, int máxima, int actual, String color)
9     {
10         this.patente = patente;
11         this.cargaMáxima = máxima;
12         this.cargaActual = actual;
13         this.color = color;
14     }
15
16     // el resto del contenido de la clase
17 }

```

Con este pequeño cambio, hemos introducido una serie de modificaciones que pueden tener consecuencias insospechadas. Incluso, se ha modificado el constructor, por lo que es posible que el proyecto actual (o algún cliente que no sospecha de ningún cambio) tenga problemas debido a la incompatibilidad de la modificación.

Para aplicar correctamente este principio, lo mejor es mantener cerrada la clase, pero extenderla para incorporar la nueva funcionalidad:

```

1 public class CamiónConColor extends Camión
2 {
3     private String color;
4
5     public CamiónConColor(String patente, int máxima, int actual, String color)
6     {
7         super(patente, máxima, actual);
8         this.color = color;
9     }
10 }

```

De esta manera, no hay cambios incompatibles, no se modificó una clase que potencialmente funciona bien, y si algún cliente quiere usar la nueva versión de Camión, puede hacerlo sin temer a que la versión original de Camión deje de funcionar.

Veamos un ejemplo un poco más complejo:

```

1 public class ServicioPagos
2 {
3     public void pagar(int monto, String tipo)
4     {
5         if ("efectivo".equalsIgnoreCase(tipo)) {
6             // Implementación de este tipo de pago
7
8         } else if ("tarjeta de crédito".equalsIgnoreCase(tipo)) {
9             // Implementación de este tipo de pago
10
11         } else if ("tarjeta de débito".equalsIgnoreCase(tipo)) {
12             // Implementación de este tipo de pago
13         }
14         throw new IllegalArgumentException();
15     }
16 }

```

Este código podría ser mejor. De hecho, este código es muy mala idea, ya que si fuera necesario agregar un nuevo tipo de pago, se tendría que modificar la clase (abrir el archivo, modificar, recompilar, agregar potenciales errores, etc).

Es mejor diseñar el sistema de pagos de esta forma:

```
1 public interface MétodoDePago
2 {
3     void pagar(int monto);
4 }
```

```
1 public class PagoEfectivo implements MétodoDePago
2 {
3     public void pagar(int monto)
4     {
5     }
6 }
```

```
1 public class PagoTarjetaCrédito implements MétodoDePago
2 {
3     public void pagar(int monto)
4     {
5     }
6 }
```

```
1 public class PagoTarjetaDébito implements MétodoDePago
2 {
3     public void pagar(int monto)
4     {
5     }
6 }
```

```
1 public class ServicioPagos2
2 {
3     private MétodoDePago métodoPago;
4
5     public ServicioPagos2(MétodoDePago métodoPago)
6     {
7         this.métodoPago = métodoPago;
8     }
9
10    public void pagar(int monto)
11    {
12        /// Implementation
13
14        métodoPago.pagar(monto);
15
16        /// Implementation
17    }
18 }
```

Al dividir el código de esta forma ya no es necesario modificar las clases si es que se agrega un nuevo método de pago.

Para ver una forma de mejorar esta idea, revisa el capítulo 10.

8.3. Liskov Substitution

Este principio ya se revisó antes, pero en ese momento se describió como las reglas que se deben seguir para poder asignar diferentes tipos de variables al trabajar en una jerarquía de clases.

Ahora en cambio, vamos a hablar de las decisiones de diseño y cómo el principio afecta y guía la forma en que el sistema se construye.

Imaginemos que estamos modelando un problema donde participan computadores, y todos son de tipo laptop, así que tienen pantallas que se cierran y son generalmente de tamaño relativamente pequeño. Como vamos a incorporar diferentes modelos de laptops, crearemos una interfaz con las operaciones que todos deben cumplir:

```
1 public interface Computador
2 {
3     /**
4      * Limpia el computador
5      */
6     void limpiar();
7
8     /**
9      * Abre la pantalla del computador para encenderlo
10     */
11     void abrirPantalla();
12 }
```

Ahora implementaremos un tipo concreto de laptop:

```
1 public class XinYon331LA implements Computador
2 {
3     private Pantalla pantalla;
4     private Teclado teclado;
5     private CPU cpu;
6
7     public void limpiar()
8     {
9         pantalla.limpiar();
10        teclado.limpiar()
11    }
12
13    public void abrirPantalla()
14    {
15        pantalla.abrir();
16        cpu.encender();
17    }
18 }
```

Todo bien hasta ahora, ya que nuestro notebook implementa todas las operaciones de la interfaz, orquestando el funcionamiento de sus diferentes componentes.

Pero ahora, nos hemos dado cuenta de que dejamos sin implementar a todos los computadores de escritorio que no son laptops. Así que tratemos de hacer que funcione bajo el esquema que hemos desarrollado:

```

1 public class ComputadorEscritorio implements Computador
2 {
3     private Pantalla pantalla;
4     private Teclado teclado;
5     private CPU cpu;
6
7     public void limpiar()
8     {
9         pantalla.limpiar();
10        teclado.limpiar()
11    }
12
13    public void abrirPantalla()
14    {
15        // Esto no tiene sentido para un computador
16        // de escritorio
17    }
18 }

```

Al tratar de incorporar este nuevo tipo de computador, nos hemos dado cuenta de que el diseño propuesto (la interfaz `Computador`) no es apropiada para lo que queremos lograr. Al contrario, está en directa oposición a lo que dice el principio, ya que una instancia de la clase `ComputadorEscritorio` no se comporta como la interfaz que está implementando.

Este problema es más difícil de corregir, ya que requiere rediseñar la solución, y por lo tanto volver a diseñar interfaces e implementar clases. Aun así, es algo que es necesario hacer y que pagará dividendos en la futura vida del software.

Revisemos un ejemplo clásico, que aparece en el libro *Working Effectively with Legacy Code* de Michael C. Feathers.

En este caso, imaginemos que al construir cierto sistema, se creó la clase `Rectángulo`:

```

1 public class Rectángulo
2 {
3     public Rectángulo(int x, int y, int ancho, int alto) { ... }
4     public void setAncho(int ancho) { ... }
5     public void setAlto(int alto) { ... }
6     public int getArea() { ... }
7 }

```

Si se necesita crear una clase `Cuadrado`, podemos hacer que sea una subclase de `Rectángulo`:

Pareciera que sí, ya que se puede pensar que un `Cuadrado` es un tipo especial de `Rectángulo`:

```

1 public class Cuadrado
2 {
3     public Cuadrado(int x, int y, int ancho) { ... }
4 }

```

`Cuadrado` hereda los métodos `setAncho(...)` y `setAlto(...)` de `Rectángulo`. ¿Cuál será el área si ejecutamos el siguiente código?

```

1 Rectángulo r = new Cuadrado(0,0,0);
2 r.setAncho(3);
3 r.setAlto(4);

```

Si el área es 12, ¡entonces el Cuadrado no es un Cuadrado! Lo que podríamos hacer es sobre-escribir `setAncho` y `setAlto` en el `Cuadrado` para que lo mantenga con su geometría correcta (o sea, que ambos métodos modifiquen la variable `ancho`), pero ésto podría producir resultados interesantes.

Este es un ejemplo clásico de una violación de este principio. Los objetos de las subclases deben ser sustituibles por objetos de la superclase. Si no lo son, entonces podríamos tener errores silenciosos en nuestro código.

8.4. Interface Segregation

Este principio significa simplemente que las interfaces con muchas operaciones se deben dividir en interfaces más pequeñas, de forma de asegurarse que las clases solamente implementen lo que necesitan, y no tienen que implementar operaciones innecesarias solo por el hecho de implementarlas.

Por ejemplo, analicemos el trabajo de un profesor o profesora. Por un lado, tiene que hacerse cargo de la enseñanza del material que tiene a cargo. Pero también puede estar a cargo de crear las evaluaciones que le hará a sus estudiantes, además de llevar a cabo la evaluación y después realizar la larga tarea de calificar los trabajos. Finalmente, debe entregar los resultados y hacerse cargo de la retroalimentación para cada estudiante.

Basándonos en lo anterior, podríamos definir una interfaz con todas las tareas de profesoras y profesores:

```
1 public interface Profesor
2 {
3     Evaluación crearEvaluación();
4     void realizarEvaluación(Evaluación ev);
5     void calificarEntregas(Evaluación ev);
6     void publicarResultados(Evaluación ev);
7     void entregarRetroalimentación(Evaluación ev);
8 }
```

Generalmente no tendríamos problema con esta definición, pero hay que reconocer que no todos los profesores y profesoras son buenos haciendo todas estas actividades. Tal como está definida la interfaz, estamos obligados a que todas las personas tengan que implementar todas las operaciones.

Pero podemos mejorar esto, al dividir esta gran interfaz en partes. Sabemos que hay personas que son buenas calificando los trabajos:

```
1 public interface ProfesorCalificador
2 {
3     void calificarEntregar(Evaluación ev);
4     void publicarResultados(Evaluación ev);
5 }
```

También hay personas que son excelentes e imaginativas al crear las evaluaciones:

```
1 public interface ProfesorCreador
2 {
3     Evaluación crearEvaluación();
4 }
```

Hay personas a las que les encanta el proceso de realizar la evaluación:

```

1 public interface ProfesorEvaluador
2 {
3     void realizarEvaluación(Evaluación ev);
4 }

```

Y finalmente hay personas expertas en entregar la retroalimentación de cada evaluación:

```

1 public interface ProfesorRetroalimentador
2 {
3     void entregarRetroalimentación(Evaluación ev);
4 }

```

De esta forma estamos siendo realistas y reconociendo que no todas las personas son buenas haciendo todo, y podremos asignar las tareas de acuerdo a las capacidades y el interés de las personas. Por ejemplo, tal vez podríamos modelar el problema de forma que los encargados de llevar a cabo las evaluaciones y realizar la retroalimentación son un equipo especial de profesores y profesoras:

```

1 public class ProfesorDeSala implements ProfesorEvaluador, ProfesorRetroalimentador
2 {
3     public void realizarEvaluación(Evaluación ev)
4     {
5     }
6
7     public void entregarRetroalimentación(Evaluación ev)
8     {
9     }
10 }

```

Pero los encargados de la creación, calificación, publicación de resultados son otros profesores:

```

1 public class UnidadDeCalidad implements ProfesorCalificador, ProfesorCreador
2 {
3     public Evaluación crearEvaluación()
4     {
5         return null;
6     }
7
8     public void calificarEntrega(Evaluación ev)
9     {
10    }
11
12    public void publicarResultados(Evaluación ev)
13    {
14    }
15 }

```

Veamos otro ejemplo, a partir de algo que ya hicimos: escribir Camiones a la pantalla. En este caso, podríamos comenzar definiendo una interfaz que tenga todas las formas posibles en que un camión se puede “escribir”:

```

1 public interface CamiónPrinter
2 {
3     public void escribirAConsola(Camión camión);
4     public void enviarPorEmail(Camión camión);
5 }

```

Como se puede ver, la clase que se tenga que encargar de escribir camiones va a tener que hacerse cargo de todo, incluso si su única preocupación es escribir a la pantalla. Esto causa que incluso es posible que se obtengan implementaciones parciales que rompen el contrato de la interfaz:

```
1 public class CamiónPrinterPantalla implements CamiónPrinter
2 {
3     public void escribirAConsola(Camión camión)
4     {
5         System.out.println(camión);
6     }
7
8     public void enviarPorEmail(Camión camión)
9     {
10         // Este método no lo vamos a utilizar
11     }
12 }
```

En este caso, tal como el principio lo indica, sería mejor dividir la interfaz en operaciones separadas, de forma que las clases implementen solo lo que necesitan.

Pero por otro lado, también podríamos segregar la interfaz de otra forma, al crear solamente una de ellas, con la operación que se requiere:

```
1 public interface CamiónPrinter
2 {
3     public void print(Camión camión);
4 }
```

Y hacer que existan muchas y diferentes implementaciones concretas que se especialicen en “imprimir” camiones en diferentes formatos:

```
1 public class CamiónPrinterConsola implements CamiónPrinter
2 {
3     public void print(Camión camión)
4     {
5         // Acá va el código para imprimir el camión
6         // a la pantalla
7     }
8 }
```

```
1 public class CamiónPrinterEmail implements CamiónPrinter
2 {
3     public void print(Camión camión)
4     {
5         // Acá va el código para enviar el camión
6         // por email
7     }
8 }
```

Así, en el momento en que se necesite mostrar un camión por la pantalla, solo tenemos que instanciar el tipo correcto:

```
1 public class CamiónTester
2 {
3     public static void main(String[] args)
4     {
5         Camión c1 = new Camión("AA1234", 1200, 400);
6
7         CamiónPrinter printer = new CamiónPrinterEmail();
8         printer.print(c1);
9     }
10 }
```


Si se quisiera imprimir el camión en otro medio, bastaría cambiar la línea 7 para instanciar el Printer que corresponda. De hecho, usando algo que se revisará más adelante (en el capítulo 10), podríamos plantear el siguiente esquema:

```
1 public class CamiónPrinterFactory
2 {
3     public static CamiónPrinter obtener(String tipo)
4     {
5         if (tipo.equalsIgnoreCase("email")) {
6             return new CamiónPrinterEmail();
7         }
8         if (tipo.equalsIgnoreCase("consola")) {
9             return new CamiónPrinterConsola();
10        }
11        throw new IllegalArgumentException(tipo + " no existe");
12    }
13 }
```

Con lo que la App se transforma en:

```
1 public class CamiónTester
2 {
3     public static void main(String[] args)
4     {
5         Camión c1 = new Camión("AA1234", 1200, 400);
6
7         CamiónPrinterFactory.obtener("email").print(c1);
8     }
9 }
```

El código anterior aun tiene algunos inconvenientes que podríamos corregir, pero por ahora sirve para mostrar lo que podemos lograr con un buen diseño y una buena distribución de las tareas al crear las interfaces y mantenerlas segregadas.

8.5. Dependency Inversion

Al aplicar este principio, podremos crear software compuesto de módulos con bajo acoplamiento entre sí. En vez de hacer que los módulos (sobre todo los de “alto” nivel) dependan directamente de otros módulos (generalmente de “bajo” nivel), ahora las dependencias que se crearán en el código serán sobre las abstracciones y no sobre las implementaciones concretas.

Por ejemplo, volvamos a un ejemplo anterior, donde estábamos modelando un computador. En este caso, queremos modelar un computador que sea operado por una persona y al que podamos hacer seguimiento respecto al nivel de uso de su procesador. Así que creamos una clase para representar dicho computador, y hacemos que en el constructor se instancien las clases concretas que lo componen:

```
1 public class ComputadorEscritorio
2 {
3     private Pantalla pantalla;
4     private Teclado teclado;
5     private CPU cpu;
6
7     public ComputadorEscritorio()
8     {
9         pantalla = new Pantalla();
10        teclado = new Teclado();
11        cpu = new CPU();
12    }
13 }
```

Y, ¿cuál es el problema? **¡Estamos acoplando todas las clases!** No existe forma de hacer que el computador use una pantalla diferente, o un teclado diferente o un nuevo modelo de CPU. Claro, podríamos crear una nueva clase llamada `ComputadorEscritorio2`, pero ese es un camino que nunca terminará, y al final, cuando estemos en `ComputadorEscritorio42` nos cuestionaremos las decisiones que tomamos al principio.

¿Qué otra cosa podemos hacer? ¡Invertir la dependencia! Primero, en vez de que `ComputadorEscritorio` instancie sus componentes, éstos le llegarán como parámetros al constructor:

```
1 public class ComputadorEscritorio
2 {
3     private CPU cpu;
4     private Pantalla pantalla;
5     private Teclado teclado;
6
7     public ComputadorEscritorio(CPU cpu, Pantalla pantalla, Teclado teclado)
8     {
9         this.cpu = cpu;
10        this.pantalla = pantalla;
11        this.teclado = teclado;
12    }
13 }
```

Y de hecho, tanto CPU, Pantalla y Teclado serán interfaces. Así, podremos crear versiones concretas de las clases, e instanciar un `ComputadorEscritorio` de acuerdo a lo que necesitamos:

```
1 public class PantallaDe32Pulgadas implements Pantalla
2 {
3 }

1 public class CPUÚltimaGeneraciónMuyRápido implements CPU
2 {
3 }

1 public class TecladoEspañol implements Teclado
2 {
3 }

1 public class ComputadorTester
2 {
3     public static void main(String[] args)
4     {
5         CPU cpu = new CPUÚltimaGeneraciónMuyRápido();
6         Pantalla p = new PantallaDe32Pulgadas();
7         Teclado t = new TecladoEspañol();
8
9         ComputadorEscritorio ce = new ComputadorEscritorio(cpu, p, t);
10    }
11 }
```

¿Cuál es la ventaja? Se ha desacoplado el computador de las implementaciones concretas de cada uno de sus componentes. Ésto nos ha permitido crear un computador genérico, que funcionará mientras los componentes respeten los contratos impuestos por las interfaces (y se supone que por eso mismo las implementan).

Con ésto, si queremos cambiar algún componente, basta instanciar el nuevo y pasarlo al constructor, y nada va a cambiar.

En resumen, cuando nuestro código depende de una interface, seguramente esa dependencia es pequeña y no molestará. Nuestro código no debe cambiar a menos que la interface cambie, y las interfaces cambian con menor frecuencia que el código que las implementa. Cuando tenemos interfaces, podemos editar las clases que las implementan, o agregar nuevas clases, sin impactar el código que ya ha sido escrito.

8.6. Ejercicios

1. Imagina que tienes que crear un programa que procesa datos, y que por ahora esos datos vienen de un archivo, con cierto formato. Tu programa funciona y entrega los resultados correctos.

Ahora resulta que hay que modificar tu programa para que lea otros dos tipos de archivo, con formatos diferentes, pero que traen la misma información (por ejemplo, piensa en que si en el formato original los datos venían todos en la misma línea, ahora vienen separados de forma que están “dispersos” en muchas líneas).

Cómo modificarías tu programa para poder leer este nuevo formato, pero sin tener que reescribir todo lo que has hecho hasta ahora.

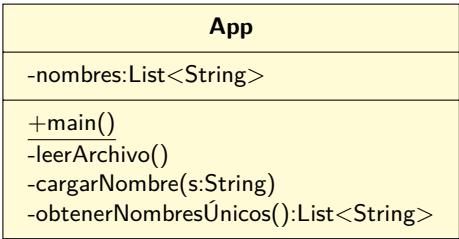
9

Arquitecturando las aplicaciones

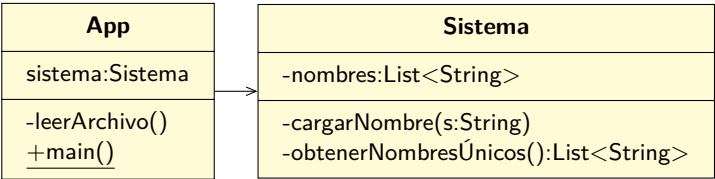
Ya hemos avanzado bastante, conociendo los elementos básicos (y algunos no tan básicos) de la programación orientada al objeto. Pero la pregunta que nos deberíamos hacer ahora es ¿qué falta?

Y lo que nos falta ahora es poner un poco más de rigurosidad en el cómo construimos las aplicaciones. Cuando se revisó el ejemplo 6.1 en la página 157 éste se construyó exitosamente, y cumplieron los objetivos que se plantearon desde el comienzo. Pero por otro lado, la aplicación era básicamente una clase `App` con un sinnúmero de cosas adentro, incluyendo operaciones y atributos, que seguramente será difícil de mantener y modificar en el futuro.

Desde ahora en adelante vamos a acostumbrarnos a trabajar de una forma “especial”. Se tendrá una clase `App` que representará a la aplicación en sí, pero ella no hará el “trabajo sucio”. Cada vez que se tenga que realizar algún proceso que sea parte integral del problema que se está solucionando, se le pedirá a otra clase que lo haga. Por ejemplo, si tenemos que cargar datos desde un archivo con nombres y contar la cantidad de nombres únicos encontrados, en vez de tener esta aplicación de una clase:



Vamos a usar esta arquitectura:



Desde ahora en adelante, la **App** le delegará la mayoría del trabajo al **Sistema**, de manera de separar las preocupaciones y las responsabilidades (ver 8.1), y que cada clase se preocupe de una sola tarea. La **App** se encargará de la interacción con los usuarios, y el **Sistema** de realizar los procesos que tienen que ver con solucionar el problema.

9.1. El Sistema

Usando el modelo del dominio (que se analizó en 3.2 en la página 85) como punto de partida, se podrá empezar a dar forma a la solución final del software que se quiere implementar¹. Y para arquitecturar de forma apropiada la solución, se deben diseñar las operaciones que el **Sistema** va a contener.

En general, el **Sistema** va tener las operaciones que tengan sentido respecto al problema que se está solucionando:

- Cargar un dato
- Asociar un dato a otro dato
- Obtener un resultado
- etc.

Cada operación debe estar bien definida, incluyendo los parámetros que requiere para funcionar. Todo esto se debe documentar creando una **interfaz** (a la que llamaremos **Sistema**), y usando el estándar **javadoc** (página 431) se debe crear lo que se llama el “contrato” de cada operación.

9.2. Contratos

Un enfoque (entre muchos) al diseñar software es el llamado **Diseño por Contrato**. En este enfoque se espera que las personas que diseñan software definan interfaces de comunicación formales, precisas y verificables entre los componentes que componen el software. Ésto se traduce en que los componentes se especifican usando conjuntos de precondiciones, postcondiciones, invariantes y efectos colaterales. Estas especificaciones se denominan el “contrato”, usurpando la metáfora de las condiciones y obligaciones que tiene un contrato comercial.

Bajo este enfoque se asume que todos los clientes que quieran invocar alguna operación tomarán precauciones y se asegurarán de satisfacer todas las precondiciones especificadas en el contrato. Pero por otro lado, también es buena idea (usando programación defensiva) que el servicio revise si todas las precondiciones se han cumplido, y si no es así, generar un error apropiado para que el cliente se entere de la situación.

En términos concretos, cada operación que se diseñe debe:²

¹“final” en este caso ya que el software nunca termina de evolucionar. “final” en el sentido de la primera versión.

²Hay otras cosas que se pueden agregar a los contratos, pero por ahora usaremos estas dos.

- Indicar qué condiciones el cliente debe cumplir para poder invocar la operación: esto es, *las obligaciones del cliente*.
- Indicar qué cosas se garantizan al salir de la operación: esto es, *las obligaciones del proveedor del servicio*.

En términos más técnicos, el contrato de cada operación debería contener lo siguiente:

- El significado de cada parámetro.
- Qué valores son aceptables, y qué valores son inaceptables en los parámetros.
- El significado del valor de retorno.
- Qué valores se van a retornar.
- Condiciones bajo las cuales se generarán errores, sus tipos y su significado.
- Efectos colaterales de invocar la operación.
- Cualquier otra condición que sea necesaria cumplir para poder invocar la operación, en términos de estado del sistema (precondiciones)
- El nuevo estado del sistema una vez ejecutada la operación (postcondiciones)

De esta forma, las clases que son clientes podrán invocar las operaciones de las clases proveedoras siempre que cumpla lo especificado en la operación, y podrá esperar que el proveedor le retorne información también dentro de lo especificado.

9.3. Documentando Contratos

La forma en que describiremos los contratos de las operaciones será usando el estándar **javadoc** (página 431), de la siguiente forma:

```

1 public interface Comparador
2 {
3     /**
4      * Compara dos números enteros e indica si ambos son iguales
5      *
6      * PRE:
7      * - Ninguna
8      *
9      * POST:
10     * - Ninguna
11     *
12     * @param a El primer valor entero que queremos comparar
13     * @param b El segundo valor entero que queremos comparar
14     * @return true o false dependiendo de si a y b son iguales o no
15     */
16     boolean comparar(int a, int b);
17
18     /**
19     * Compara tres números enteros e indica si los tres son iguales

```

```

20  *
21  * PRE:
22  * - Ninguna
23  *
24  * POST:
25  * - Ninguna
26  *
27  * @param a El primer valor entero que queremos comparar
28  * @param b El segundo valor entero que queremos comparar
29  * @param c El tercer valor entero que queremos comparar
30  * @return true o false dependiendo de si a, b y c tiene el
31  * mismo valor, o no
32  */
33  boolean comparar(int a, int b, int c);
34
35  /**
36  * Compara dos instancias de String, e indica si el valor de los
37  * strings son iguales. La comparación se realiza usando
38  * el valor de los strings.
39  *
40  * PRE
41  * - Ninguna
42  *
43  * POST
44  * - Ninguna
45  *
46  * @param a La referencia al primer String a comparar
47  * @param b La referencia al segundo String a comparar
48  * @return true si ambos strings tienen el mismo valor. false
49  * en caso contrario.
50  */
51  boolean comparar(String a, String b);
52
53  /**
54  * Compara tres instancias de String, e indica si el valor de los
55  * strings son iguales. La comparación se realiza usando
56  * el valor de los strings.
57  *
58  * PRE
59  * - Ninguna
60  *
61  * POST
62  * - Ninguna
63  *
64  * @param a La referencia al primer String a comparar
65  * @param b La referencia al segundo String a comparar
66  * @param c La referencia al tercer String a comparar
67  * @return true si los tres strings tienen el mismo valor. false
68  * en caso contrario.
69  */
70  boolean comparar(String a, String b, String c);
71  }

```

En el caso anterior, todas las operaciones indican que no tienen precondiciones que se deban cumplir al invocarlas. Por otro lado, aun cuando indican que no cumplen ninguna postcondición, la verdad es que la cláusula `@return` indica una: en cada caso se describe lo que se retornará y el significado de lo retornado. A partir de ahora, se usará la `postcondición` para indicar cambios en el estado del “sistema”, y la cláusula `@return` para especificar lo que retorna, incluyendo cualquier significado o condición que el cliente tenga que tomar en consideración.

Por ejemplo:

```

1  /**
2  * Esta interface define las operaciones de todos los Filtradores.
3  *
4  * El objetivo de este servicio es proveer una forma de filtrar
5  * palabras "malas". A las instancias de esta interfaz se le proveerán
6  * palabras una a una, y se le podrá preguntar por la cantidad
7  * de palabras "filtradas" y las "no-filtradas".
8  *
9  * @author Eric Ross
10 */
11 public interface Filtrador
12 {
13     /**
14     * Agrega una nueva palabra al filtrador
15     *
16     * PRE:
17     * - El String con la palabra contiene una sola palabra
18     *   (no contiene espacios, ni frases, ni puntuaciones, ni símbolos)
19     *
20     * POST:
21     * - La nueva palabra fue ingresada al filtrador, para que
22     *   los otros métodos puedan tomarla en cuenta
23     *
24     * @param palabra Un String conteniendo una sola palabra que
25     *                 será ingresada al filtrador. No se hace diferencia
26     *                 entre minúsculas y mayúsculas.
27     */
28     void agregarPalabra(String palabra);
29
30     /**
31     * Retorna la cantidad total de palabras que han sido ingresadas al filtrador
32     * hasta ahora.
33     *
34     * @return La cantidad de palabras que han sido ingresadas al filtrador
35     * hasta ahora.
36     */
37     int getCantidadPalabrasTotal();
38
39     /**
40     * Retorna la cantidad de palabras "malas" que han sido ingresadas al filtrador
41     * hasta ahora.
42     *
43     * @return La cantidad de palabras "malas" que hasta ahora han sido ingresadas
44     * al filtrador
45     */
46     int getCantidadPalabrasMalas();
47 }

```

Nótese que en el código anterior, incluso se especificó el contrato general de la interface, indicando para qué sirve, y además se especificó el contrato de cada operación. En estos casos, generalmente las **post-condiciones** se usarán para declarar cómo cambia el sistema una vez que el sistema retorna de la llamada a la operación, y la cláusula **@return** también complementará el contrato, de forma que los clientes sepan qué esperar de la llamada, en términos de las condiciones bajo las que se retorna cada tipo de valor.

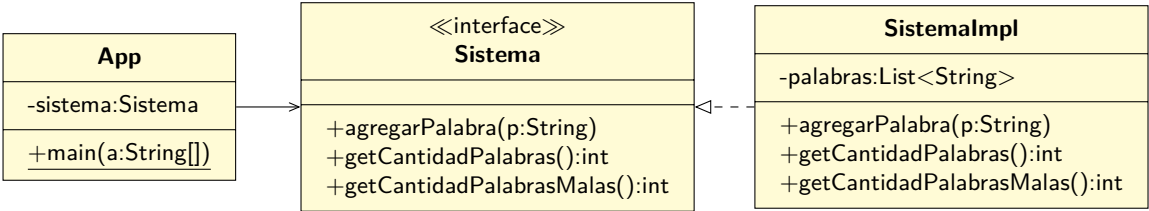
En los casos anteriores no se especificó pre y post-condiciones en las operaciones que no lo requieren.

9.4. Contratos del sistema

Reuniendo todo lo anterior, a partir de ahora nuestros sistemas tendrán las siguientes características:

- Existirá una interfaz llamada `Sistema`, que incluirá todas las operaciones que serán llamadas desde la `App`.
- Cada operación del `Sistema` (y el `Sistema` mismo) será documentado, especificando el contrato de cada elemento, de acuerdo a lo revisado anteriormente.
- Existirá una clase concreta que implementará la interfaz `Sistema`. El nombre de esa clase será `SistemaImpl`.

Por ejemplo, para el caso del `Filtrador`, el diagrama de clases será:



9.5. Cuando las precondiciones no se cumplen

Existen diferentes visiones respecto al cumplimiento de las pre-condiciones y su relación con los contratos.

Por un lado, se dice que las pre-condiciones, al estar definidas “fuera” de las operaciones, no son algo de lo que las operaciones mismas deban preocuparse: los clientes deben asegurarse de cumplir las precondiciones, y la operación no debe hacerse responsable de lo que suceda si no se cumplen. En términos concretos, es equivalente a decir:

```
1 public int operación(String nombre)
2 {
3     nombre = nombre.toUpperCase();
4     return 0;
5 }
```

En este caso, si el contrato de la operación especifica que el `nombre` no puede ser `null`, la operación confiará en que siempre será así, por lo que operará sobre el parámetro sin ningún tipo de revisión , y en este caso invocará un método sin preocuparse de verificar si la referencia es `null`. Si el parámetro fuera `null`, la operación `toUpperCase()` fallará con `NullPointerException`, pero ésto no debería suceder ya que no debería nunca invocarse a `operación(...)` con un parámetro `null`.

La otra forma de ver la misma situación es que la operación no puede simplemente confiar en que los clientes respetarán el contrato, por lo que debe tomar medidas para asegurar que las condiciones de

funcionamiento de la operación se cumplen, antes de comenzar a procesar la operación en sí.³ En este ejemplo, sería equivalente a lo siguiente:

```

1 public int operación(String nombre)
2 {
3     if (nombre == null) throw new IllegalArgumentException();
4
5     nombre = nombre.toLowerCase();
6
7     return 0;
8 }

```

La versión que usaremos será la de *programar defensivamente*. No nos bastará simplemente confiar en que los clientes harán lo que tienen que hacer. Desde ahora en adelante cuando nuestros contratos especifiquen pre-condiciones, pero éstas no se cumplan, el **Sistema** tendrá la obligación de señalar la condición de error.

Pero el ejemplo recién mostrado tiene un problema, ya que lanza un tipo de excepción que de acuerdo a su clasificación cae en la categoría de “unchecked exception”. (ver F.5.2 en la página 454 para más detalles), pero lo que queremos lograr es que los clientes realmente se preocupen en el caso de que no estén respetando el contrato.⁴

Entonces, se usarán “checked exceptions” para poner de manifiesto la posibilidad de que las pre-condiciones no se cumplan. Para ello, se tienen que definir tipos específicos de excepciones que describan cada condición de error posible de cada operación.

Por ejemplo, en el caso del sistema de filtrado de palabras, lo anterior se traduce en que las operaciones que lo requieran, deben definir que lanzarán una excepción en caso de no cumplirse las condiciones apropiadas:

```

1 public interface Sistema
2 {
3     /**
4      * Agrega una nueva palabra al filtrador
5      *
6      * PRE:
7      * - El String con la palabra contiene una sola palabra
8      *   (no contiene espacios, ni frases, ni puntuaciones, ni símbolos)
9      *
10     * POST:
11     * - La nueva palabra fue ingresada al filtrador, para que
12     *   los otros métodos puedan tomarla en cuenta
13     *
14     * @param palabra Un String conteniendo una sola palabra que
15     *                será ingresada al filtrador. No se hace diferencia
16     *                entre minúsculas y mayúsculas.
17     * @throws FiltradorException Cuando la palabra pasada como parámetro
18     *                             no cumple las condiciones.
19     */
20     void agregarPalabra(String palabra) throws FiltradorException;
21
22     ...
23 }

```

³Algo parecido a la ley de Postel

⁴En el ejemplo mostrado obviamente se preocuparán, ya que si se lanza la excepción, el programa terminará su ejecución. Pero lo que se quiere lograr es que los clientes que invoquen la operación sepan por adelantado que la operación va a tomar totalmente en cuenta la definición del contrato, y cualquier desviación será rechazada.

Nótese que ahora la definición de la operación dice explícitamente que puede lanzar (**throws**) una excepción de tipo **FiltradorException**, y también el contrato escrito en **javadoc** contiene la cláusula **@throws** para indicar lo mismo.

La excepción de tipo **FiltradorException** no es algo que venga incluida en el **JDK**, sino que la tenemos que crear, definiendo una subclase de **Exception** (para que la nueva excepción sea del tipo “checked”):

```
1 public class FiltradorException extends Exception
2 {
3     public FiltradorException(String message)
4     {
5         super(message);
6     }
7 }
```

Obviamente la implementación del sistema debe incluir las verificaciones pertinentes:

```
1 public class SistemaImpl implements Sistema
2 {
3     private List<String> palabras = new LinkedList<>();
4
5     @Override
6     public void agregarPalabra(String palabra) throws FiltradorException
7     {
8         if (palabra == null)
9             throw new FiltradorException("La palabra no puede ser null");
10        if (palabra.length() == 0)
11            throw new FiltradorException("La palabra no puede ser vacía");
12        if (palabra.indexOf(" ") != -1)
13            throw new FiltradorException("La palabra no es una palabra");
14
15        palabras.add(palabra);
16    }
17
18    ...
19 }
```

Todo lo anterior causará que al momento de usar el **Sistema**, se tenga que tomar en cuenta la posibilidad de incumplimiento del contrato:

```
1 public class App
2 {
3     public static void main(String[] args)
4     {
5         Sistema s = new SistemaImpl();
6
7         s.agregarPalabra("hola 123"); // Unhandled exception type FiltradorException
8     }
9 }
```

Por lo que hay que manejar explícitamente el caso de recibir una excepción:

```

1 public class App {
2     public static void main(String[] args) {
3         Sistema s = new SistemaImpl();
4
5         try {
6             s.agregarPalabra("hola 123");
7         } catch (FiltradorException e) {
8             //
9         }
10    }
11 }

```

9.6. Todas las operaciones tienen contrato

Es muy importante destacar que, aunque acá se ha puesto énfasis (y hasta cierto punto, obligación) en que se deben documentar claramente los contratos del `Sistema`, no hay que perder de vista que:

¡Todas las operaciones tienen contrato!

Cada vez que vamos a diseñar cómo los diversos objetos y sus métodos se combinan para lograr solucionar el problema que tenemos en frente, estamos adquiriendo compromisos respecto a cómo esperamos que cada método se comporte. Y esos compromisos dependen del significado que nosotros mismos le asignamos a las cosas (clases, métodos, etc).

Cada persona tendrá un estilo diferente para estructurar las soluciones, y seguramente lo que para una persona significa algo, para otra persona significará otra cosa, pero es importante que dentro de lo que se diseña (y se implementa) las operaciones (y los objetos, y en general todo) respeten y cumplan el contrato que se ideó para ellas, incluso si no está documentado explícitamente como en el caso del `Sistema`.

Por ejemplo, si al diseñar la solución nos encontramos en la necesidad de dividir un método en varias partes (por ejemplo, para reutilizar código, o simplemente por que el método estaba quedando muy extenso), lo más seguro es que cada persona realizará el proceso de forma diferente, pero lo más importante, es que le asignarán significado diferente a las acciones. Veamos un ejemplo muy simple:

```

1 public static void main(String[] args) {
2     for(int i=0;i<10;i++) {
3         Libro libro = null;
4         int n = contarLibros(), j;
5         for(j=0;j<n;j++) {
6             if (libros[j].getNúmero() == i) {
7                 libro = libros[j];
8             }
9         }
10        if (j == n) {
11            System.out.println("No existe el libro " + i);
12        } else {
13            System.out.println("Título: " + libro.getAuthor());
14        }
15    }
16 }

```

En el código anterior, claramente la intención es buscar una instancia determinada de **Libro** y hacer algo con esa instancia. Lo malo de este código es que mezcla dos actividades diferentes en un solo lugar: el primer **for** que determina las instancias de **Libro** que se quieren buscar, y el proceso de búsqueda en sí (que también acá se implementa usando un **for**). Podemos refactorizar este programa de muchas formas, pero un ejemplo puede ser:

```

1 public static void main(String[] args)
2 {
3     for(int i=0;i<10;i++) {
4         Libro libro = buscarLibro(i);
5
6         if (libro == null) {
7             System.out.println("No existe el libro " + i);
8         } else {
9             System.out.println("Título: " + libro.getAuthor());
10        }
11    }
12 }

```

Nótese cómo se han cambiado varios elementos acá: lo más evidente es que se factorizó todo el código que era relevante solo para buscar un **Libro** y se movió a su propio espacio. Pero además, se cambió la forma en que se detecta cuando un **Libro** no existe. En la versión original se comprobaba el valor de la variable de iteración de la búsqueda para determinar la razón de haber terminado el ciclo (ya sea que se revisó todo el arreglo, o se encontró la instancia buscada antes de que se acabara dicho arreglo). En la nueva versión, parece que si el **Libro** no existe se retorna **null**.

Es justamente ese tipo de decisiones las que hay que recordar, ya que ellas son el contrato de la operación: en este caso, se decidió que si un **Libro** no existe se retornará **null**. Nótese que esa no es la única forma de realizar esto, pero de todas las opciones disponibles, se eligió ese comportamiento, y por lo tanto, desde ahora en adelante, cada vez que se vaya a usar dicha operación hay que respetar su contrato y tomar en cuenta que nos puede retornar **null** en ciertas condiciones. ¡Y la mejor forma de no olvidar es escribirlo!

```

1 /**
2  * Busca el libro que tenga el número buscado, de la lista de libros.
3  *
4  * @param num El número de libro que andamos buscando
5  * @return El libro de la lista que tenga el número buscado, o null si no existe un
6  *         libro
7  *         con ese número.
8  */
9 private static Libro buscarLibro(int num)
10 {
11     int n = contarLibros();
12     Libro libro = null;
13
14     for (int j = 0; j < n; j++) {
15         if (libros[j].getNúmero() == num) {
16             libro = libros[j];
17         }
18     }
19     return libro;
20 }

```

9.7. Resumen de reglas

Aun cuando a estas alturas está claro que el diseño interno de las aplicaciones, y en particular las operaciones y acciones que realizan las clases se pueden construir de cualquier forma, nos aseguraremos de que nuestras aplicaciones (en el contexto de este libro) se diseñen e implementen siguiendo ciertos lineamientos:⁵

1. No hay instancias de `Scanner` en el `Sistema`.
2. No se leen archivos desde el `Sistema`.
3. No se retornan objetos del dominio desde el `Sistema`.
4. No se reciben objetos del dominio en el `Sistema`.
5. No se deben usar `Excepcion` para controlar el flujo. Las Excepciones se lanzan para señalar situación de excepción (por ejemplo, incumplimiento de los contratos), no para indicar, por ejemplo, que un usuario no existe.

⁵Es cierto que algunos de estos lineamientos pueden parecer arbitrarios, y la verdad, es que en cierto sentido lo son. Pero en el fondo nos servirán para lograr que las aplicaciones cumplan las dos reglas más importantes en el desarrollo de software: el bajo acoplamiento y la alta cohesión.

10

Patrones de diseño

Muchas veces, al diseñar nuestros programas, nos encontraremos con problemas que ya se han resuelto antes. En esos casos, en vez de tratar de re-inventar algo desde cero, nos conviene conocer algunas de estas soluciones para ver si son factibles de aplicarse a nuestro caso particular.

En la ingeniería de software, los patrones de diseño son soluciones generales y reusables a problemas recurrentes dentro de un contexto en el diseño de software. No se trata de algo que generalmente se pueda transformar directamente en código, sino que son descripciones o ideas acerca de cómo resolver un problema.

Se considera que los patrones de diseño es la formalización de buenas prácticas a la hora de diseñar un sistema.

La clasificación y los patrones que se revisarán a continuación se encuentran en el libro [2].

10.1. Clasificación

Generalmente los patrones de diseño se clasifican en 3 grupos dependiendo del tipo de problema que resuelven:

Creacionales Se encargan de la creación de objetos.

Estructurales Se encargan de la estructura de las clases.

Comportamiento Se encargan de encontrar mejores interacciones entre los objetos, de forma de fomentar el bajo acoplamiento, y mejorar la flexibilidad de los diseños.

10.2. Patrones Creacionales

Hay 5 patrones en esta categoría:

10.2.1. Singleton

Este patrón restringe la creación de una clase de forma que solo existan una instancia durante la ejecución del programa.

Para realizar lo anterior se pueden utilizar varias técnicas, cada una con diferentes ventajas y desventajas, pero en general hacen lo siguiente:

- La clase tiene un constructor privado para evitar que se pueda instanciar desde “afuera”.
- Tiene una variable estática privada para mantener referenciada la única instancia de la clase.
- Existe un método estático público para retornar la única instancia de la clase.

Una versión simple y útil puede ser:

```
1 public class SimpleLogger
2 {
3     private static final SimpleLogger instance = new SimpleLogger();
4
5     // Constructor privado!
6     private SimpleLogger(){}
7
8     public static SimpleLogger getInstance()
9     {
10         return instance;
11     }
12 }
```

Una versión ligeramente más optimizada puede ser:

```
1 public class LazyLogger
2 {
3     private static LazyLogger instance;
4
5     private LazyLogger(){}
6
7     public static LazyLogger getInstance(){
8         if(instance == null){
9             instance = new LazyLogger();
10        }
11        return instance;
12    }
13 }
```

Obviamente, cualquiera que quiera usar la clase, no puede simplemente implementarla, sino que tiene que hacer la llamada correcta para recibir la instancia única:

```
1 public class SingletonTest
2 {
3     public static void main(String[] args)
4     {
5         // The constructor LazyLogger() is not visible
6         //LazyLogger logger = new LazyLogger();
7
8         LazyLogger logger = LazyLogger.getInstance();
9     }
10 }
```

Este es uno de los patrones de diseño más ampliamente utilizados en el desarrollo de software. Algunas de las razones para usarlo son:

Acceso Global : El patrón proporciona un punto de acceso global a la instancia única de la clase, lo que facilita la comunicación y el uso de esa instancia desde cualquier parte del programa. Esto evita la necesidad de pasar instancias entre clases o módulos, lo que puede simplificar el diseño y reducir la complejidad.

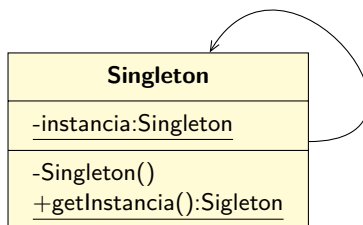
Ahorro de Recursos : Al garantizar una sola instancia, el patrón ayuda a ahorrar recursos del sistema. Esto es especialmente importante en casos en los que la creación de múltiples instancias sería costosa en términos de memoria o recursos de CPU.

Inicialización Tardía : El patrón permite la inicialización tardía de la instancia única. Esto significa que la instancia se crea solo cuando se solicita por primera vez, lo que puede mejorar el rendimiento y reducir la carga inicial del programa.

Evitar Inconsistencias : En aplicaciones que involucran datos compartidos o configuraciones globales, un singleton evita problemas de consistencia y estado incoherente, ya que garantiza que todas las partes del programa trabajen con la misma instancia de la clase.

Facilita el Testing : El patrón puede facilitar las pruebas unitarias y de integración, ya que se puede controlar y predecir el estado de una única instancia. Esto simplifica la creación de pruebas y la depuración.

El diagrama UML de este patrón es:



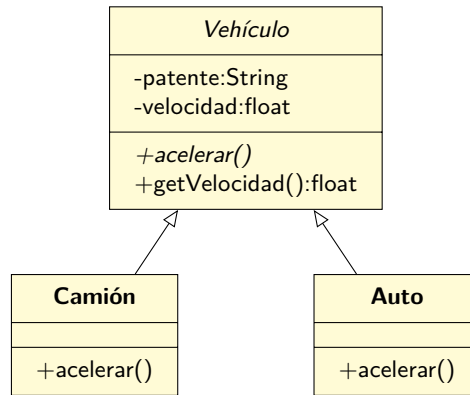
10.2.2. Factory

Este patrón asume la responsabilidad de la creación de instancias de una clase.

Se usa cuando se tiene una clase con muchas subclases, y basándonos en algún dato, se requiere retornar una instancia de las subclases. Con este patrón la responsabilidad de la instanciación se saca desde el programa y se deja en una clase factory.

Pero, ¿por qué debemos preocuparnos de quién crea las instancias? Este patrón nos permite concentrarnos y programar contra la interfaz en vez de contra la implementación. Además, desacopla el código y lo hace más fácil de modificar.

Por ejemplo:



En este caso, en vez que el código cliente tome la decisión de qué versión de `Vehículo` instanciar:

```

1 public class TestFactory
2 {
3     public static void main(String[] args)
4     {
5         Vehículo ve = null;
6
7         if (args[0].equalsIgnoreCase("auto")) {
8             ve = new Auto(args[1]);
9         } else if (args[0].equalsIgnoreCase("camión")) {
10            ve = new Camión(args[1]);
11        }
12
13        ve.acelerar();
14        System.out.println("Velocidad actual: " + ve.getVelocidad());
15    }
16 }
  
```

Le delegamos la tarea a una clase especializada en instanciar `Vehículos`:

```

1 public class VehiculoFactory
2 {
3     public static Vehículo getVehículo(String tipo, String patente)
4     {
5         if ("auto".equalsIgnoreCase(tipo)) return Auto PC(patente);
6         else if ("camión".equalsIgnoreCase(tipo)) return new Camión(patente);
7         return null;
8     }
9 }
  
```

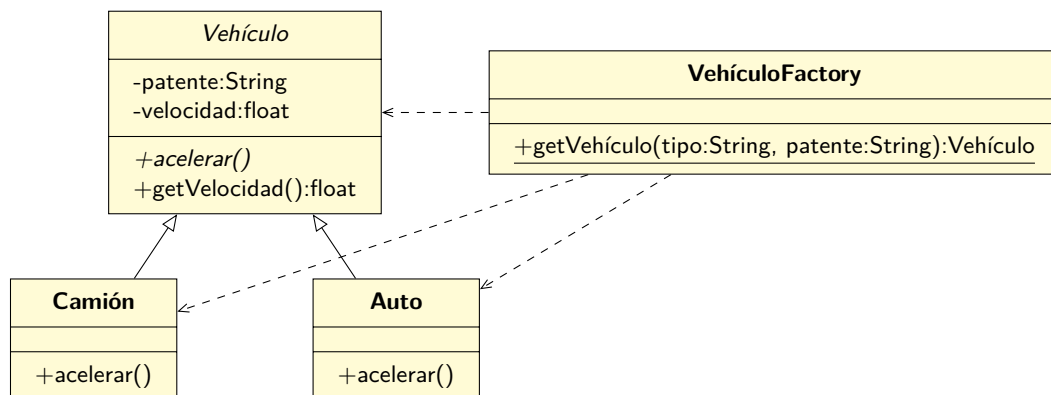
Y el código cliente se transforma en:

```

1 public class TestFactory
2 {
3     public static void main(String[] args)
4     {
5         Vehículo ve = VehiculoFactory.getVehículo(args[0], args[1]);
6         ve.acelerar();
7         System.out.println("Velocidad actual: " + ve.getVelocidad());
8     }
9 }
  
```

Muchas veces, se puede utilizar un **Singleton** para que la **Factory** no se pueda instanciar, o la podemos estructurar como en el ejemplo, con un método estático.

Nótese que el código cliente deja de estar programado contra las instancias concretas, y se vuelve mucho más genérico y mantenible. Por ejemplo, si se agregan nuevos tipos de **Vehículo** el código no debería cambiar.



La línea punteada significa que existen una dependencia: la **Factory** depende, usa los tipos, pero no mantiene referencias a ellos ni tampoco tiene una relación de herencia.

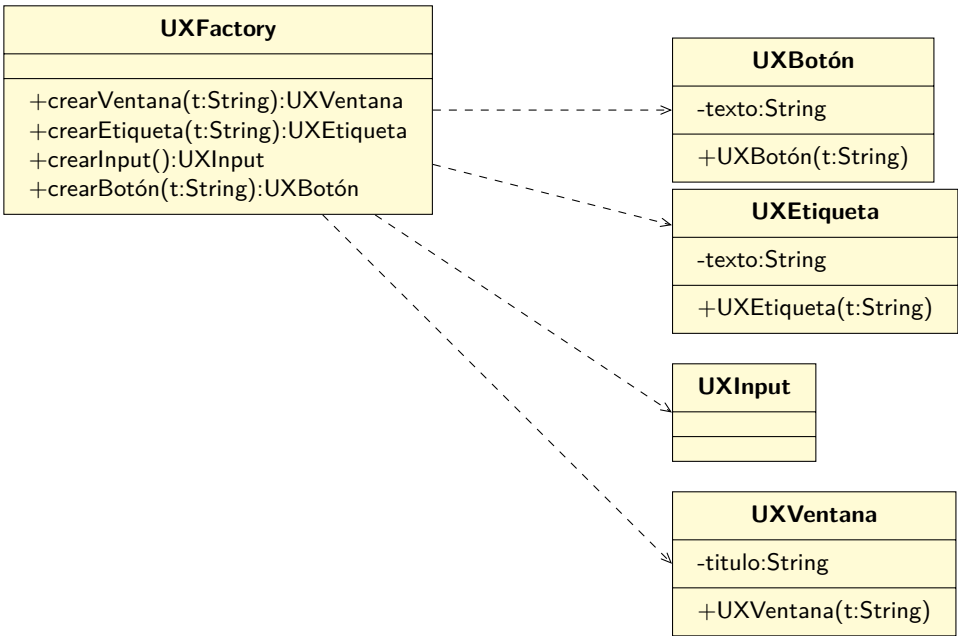
10.2.3. Abstract Factory

En el patrón **Factory** existe una sola clase que retorna diferentes instancias de las subclases dependiendo de algún parámetro, y seguramente se usa un **if-else** o **switch** para determinar el tipo correcto a retornar. Dicha clase **Factory** es concreta, y nosotros como clientes sabemos explícitamente qué clase es.

Pero hay ocasiones en que para mejorar el diseño del programa, nos convendrá no saber el tipo específico de **Factory** que usaremos. Solo sabremos que cumple cierta interfaz, pero no el tipo concreto.¹

Veamos un ejemplo: imaginemos que estamos creando una aplicación que puede funcionar en diferentes sistemas operativos, pero que independiente de dónde se ejecute, tiene que crear una ventana gráfica y agregar algunos elementos (como etiquetas y botones). Lo que haremos es definir una **Factory** que se encargue de crear los elementos que se requieren:

¹Y muchas veces tampoco nos importará su tipo específico.



Pero, ¿cómo implementamos las diferencias entre sistemas operativos? Una forma es tomar en consideración esa diferencia usando `if/switch` dentro de las clases que representan los elementos gráficos, pero si se hace de esa forma, estaremos acoplando nuestro código, de forma que agregar un nuevo tipo de sistema operativo será muy complicado, y si hay que realizar alguna modificación se tendrían que manipular muchos archivos. De hecho, queremos lograr que nuestro código sea tan simple como esto:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         UXFactory factory = AbsOSFactory.getS0Factory();
6         App app = new App(factory);
7         app.ejecutar();
8     }
9 }
```

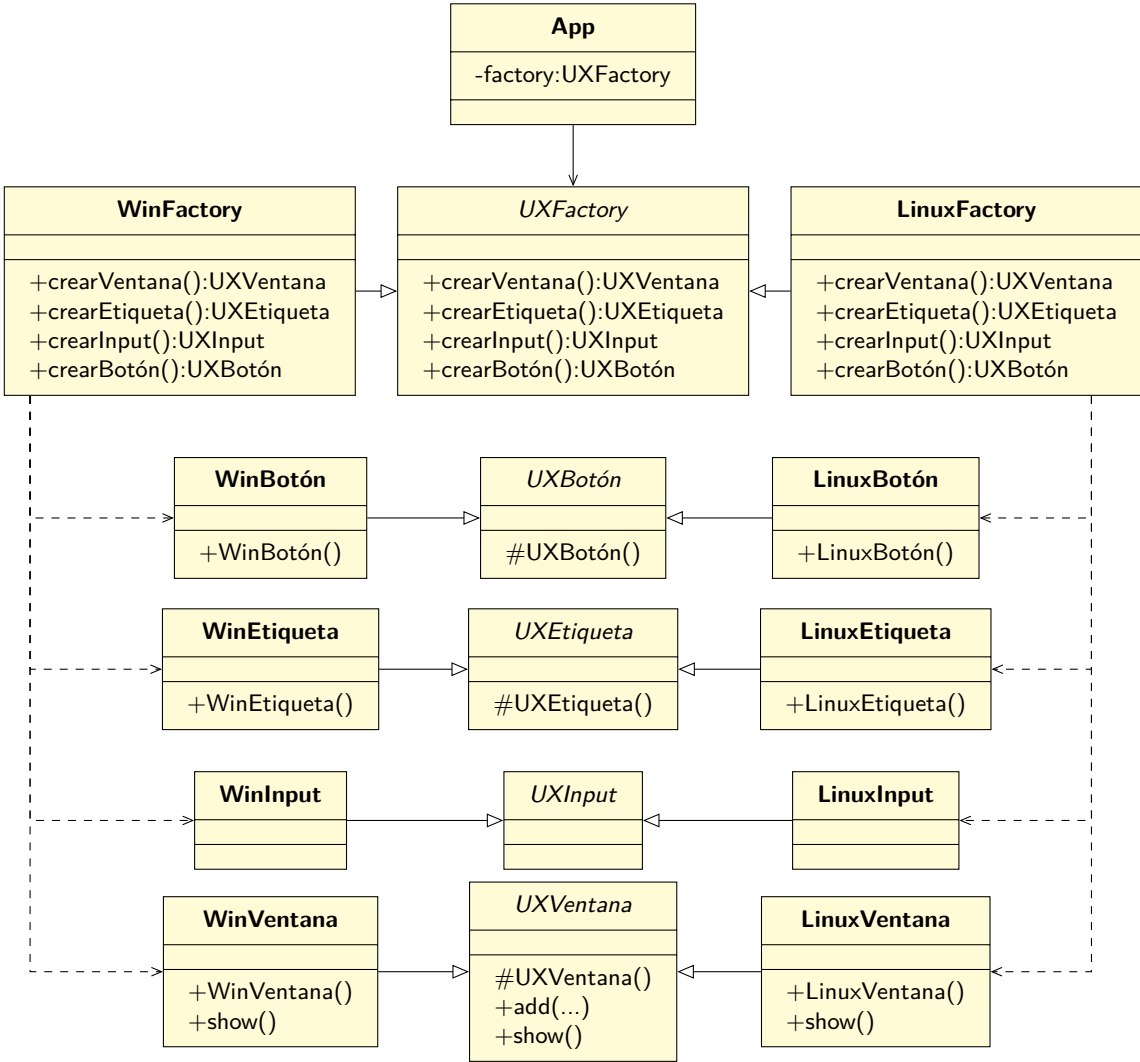
Y que la clase App sea así:

```
1 public class App
2 {
3     private UXFactory factory;
4
5     public App(UXFactory factory)
6     {
7         this.factory = factory;
8     }
9
10    public void ejecutar()
11    {
12        UXVentana ventana = factory.crearVentana();
13
14        ventana.add(factory.crearEtiqueta());
15        ventana.add(factory.crearInput());
16    }
17 }
```

```
16     ventana.add(factory.crearBotón());
17     ventana.show();
18 }
19 }
```

¿Cómo se logra?

Usando el patrón **Abstract Factory** transformaremos la clase **UXFactory** para que sea abstracta, y se definirán clases concretas que servirán como **Factory** para cada tipo de sistema operativo que requiera elementos gráficos:



Puede parecer que hacer algo así le agrega complejidad a la solución, pero la verdad, estamos totalmente desacoplando las clases concretas de sus interfaces, la clase cliente (**App**) solo manipulará las instancias sin saber lo que realmente son, y agregar un nuevo sistema operativo será muy sencillo.

Bajo este esquema, `UXFactory` es:

```

1 public interface UXFactory
2 {
3     UXVentana crearVentana();
4     UXEtiqueta crearEtiqueta();
5     UXInput crearInput();
6     UXBotón crearBotón();
7 }

```

Y `UXVentana` es:

```

1 public abstract class UXVentana
2 {
3     protected UXVentana() { }
4
5     public abstract void add(UXElement crearEtiqueta);
6
7     public abstract void show();
8 }

```

Por ejemplo, para el caso del subsistema `Linux`, se tiene la siguiente `Factory`:

```

1 public class LinuxFactory implements UXFactory
2 {
3     @Override
4     public UXVentana crearVentana() { return new LinuxVentana(); }
5
6     @Override
7     public UXEtiqueta crearEtiqueta() { return new LinuxEtiqueta(); }
8
9     @Override
10    public UXInput crearInput() { return new LinuxInput(); }
11
12    @Override
13    public UXBotón crearBotón() { return new LinuxBotón(); }
14 }

```

Y el botón con sabor `Linux`:

```

1 public class LinuxBotón extends UXBotón
2 {
3     protected LinuxBotón()
4     {
5         super();
6     }
7 }

```

Y la `Ventana` correspondiente:

```

1 public class LinuxVentana extends UXVentana
2 {
3     protected LinuxVentana() { super(); }
4
5     @Override
6     public void add(UXElement crearEtiqueta) { }
7
8     @Override
9     public void show() { }
10 }

```


Obviamente por ahora el cuerpo de los métodos está vacío, pero las versiones de `Windows` podrían usar llamadas directas a `Win32`, mientras que las de `Linux` podrían usar `GTK` o `Qt`.

En resumen, usar este patrón es una buena idea por muchas razones:

Abstracción de la Creación de Objetos Una de las ventajas clave del patrón es que abstrae el proceso de creación de objetos. En lugar de crear objetos directamente utilizando el operador `new`, se utiliza una interfaz o clase abstracta para definir métodos de creación. Esto oculta los detalles de implementación y permite cambiar las clases concretas que se instancian sin afectar al resto del código.

Encapsulación de Familias de Objetos El patrón agrupa la creación de objetos relacionados en una única fábrica. Esto facilita la encapsulación de la lógica de creación y garantiza que los objetos sean coherentes y compatibles entre sí. Por ejemplo, si estamos construyendo una aplicación de creación de muebles, podríamos tener una fábrica de muebles modernos y otra de muebles clásicos, ambas con sus propias implementaciones coherentes de sillas, mesas y sofás.

Flexibilidad en la Configuración El patrón proporciona flexibilidad en la configuración del sistema, ya que permite cambiar la familia de objetos que se utiliza en tiempo de ejecución. Esto es útil en situaciones en las que se necesita cambiar dinámicamente la implementación de ciertos componentes de la aplicación, como cambiar el motor de una aplicación de videojuegos en función del hardware del usuario.

Cumplimiento de Principios de Diseño SOLID El patrón es coherente con varios principios de diseño importantes, como el Principio de Responsabilidad Única y el Principio de Sustitución de Liskov. Promueve la separación de preocupaciones al evitar la lógica de creación dispersa en todo el código y garantiza que las clases derivadas (productos concretos) sean intercambiables sin afectar la funcionalidad del programa.

Facilita las Pruebas Unitarias La abstracción de la creación de objetos facilita las pruebas unitarias, ya que permite la inyección de dependencias de fábricas en lugar de objetos concretos. Esto simplifica la simulación y la prueba de diferentes configuraciones y comportamientos.

Escalabilidad y Mantenibilidad El patrón es útil en proyectos a gran escala, ya que facilita la adición de nuevas familias de objetos sin afectar el código existente. Cuando se necesita extender la funcionalidad de la aplicación para incluir nuevos tipos de productos, simplemente se agrega una nueva fábrica con sus implementaciones correspondientes, sin alterar el código existente.

10.2.4. Builder

Generalmente se espera que la cantidad de atributos de una clase se mantenga bajo control, pero a veces no se puede evitar que ciertas clases crezcan en funcionalidad y en tamaño. Lo anterior trae como consecuencia que muchas veces el constructor de la clase crece y tiene que recibir muchos parámetros para poder construir instancias que contengan la información necesaria para poder operar.

Se podría pensar entonces en crear constructores que solo tengan una cantidad mínima de parámetros, y después confiar en una serie de `setters` para terminar de configurar la instancia, pero ésto tiene la consecuencia de que es posible dejar objetos en estado inconsistente, a menos que se llamen a todos los `setters` que se requieran, lo cuál no necesariamente sucederá.

El patrón **Builder** viene a solucionar este problema, al proveer una forma de construcción de instancias paso-a-paso que permitirá siempre obtener objetos totalmente consistentes.

Para crear un **Builder** para una clase en particular, es necesario realizar los siguientes pasos:

- Definir una clase que contendrá todos los atributos de la clase original. Por ejemplo, si se quiere generar un **Builder** para la clase **Casa**, entonces la nueva clase se llamará **CasaBuilder** y tendrá todos los atributos de **Casa**.
- **CasaBuilder** debe tener un constructor público con todos los atributos mínimos necesarios para poder construir una **Casa**.
- **CasaBuilder** debe tener métodos para configurar todos los atributos optativos, y cada uno de esos métodos debe retornar la misma instancia de **CasaBuilder**.
- **CasaBuilder** debe tener un método **build()** que retornará una nueva instancia de **Casa** construida a partir de los valores configurados previamente.
- Idealmente se debe limitar el acceso al constructor de **Casa**, para evitar que se puedan instanciar sin pasar por **CasaBuilder**.

Por ejemplo, en vez de hacer esto:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         Casa casa = new Casa("Prat", 132, "Coquimbo", 1, 122, "rojo", 9941, true);
6     }
7 }
8 }
```

Se espera hacer lo siguiente:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         CasaBuilder builder = new CasaBuilder("Prat", 132, "Coquimbo", 1);
6
7         Casa casa = builder
8             .setColor("Blanco")
9             .setConsumoAgua(122)
10            .setConsumoElectricidad(9941)
11            .setTieneEstufa(true)
12            .build();
13     }
14 }
```

Nótese que esta última versión es incluso más simple de entender: el constructor solo recibe los parámetros mínimos, y el significado del resto de los elementos es evidente debido al nombre de los métodos que se invocan en el **Builder** (lo que no pasaba en la primera versión pasándole todos los parámetros al constructor).

La clase **Casa** estará definida así:

```

1 public class Casa
2 {
3     private String calle;
4     private int número;
5     private String ciudad;
6     private int cantPisos;
7     private int consumoAgua = 0;
8     private int consumoElectricidad = 0;
9     private boolean tieneEstufa = false;
10    private String color = "sin color";
11
12    public Casa(String calle, int número, String ciudad, int cantPisos,
13               int consumoAgua, String color, int consumoElectricidad, boolean tieneEstufa)
14    {
15        this.calle = calle;
16        this.número = número;
17        this.ciudad = ciudad;
18        this.cantPisos = cantPisos;
19        this.consumoAgua = consumoAgua;
20        this.color = color;
21        this.consumoElectricidad = consumoElectricidad;
22        this.tieneEstufa = tieneEstufa;
23    }
24 }

```

Y `CasaBuilder` así:

```

1 public class CasaBuilder
2 {
3     private String calle;
4     private int número;
5     private String ciudad;
6     private int numPisos;
7     private String color;
8     private int consumoAgua;
9     private int consumoElectricidad;
10    private boolean tieneEstufa;
11
12    public CasaBuilder(String calle, int número, String ciudad, int numPisos)
13    {
14        this.calle = calle;
15        this.número = número;
16        this.ciudad = ciudad;
17        this.numPisos = numPisos;
18    }
19
20    public CasaBuilder setColor(String color)
21    {
22        this.color = color;
23        return this;
24    }
25
26    public CasaBuilder setConsumoAgua(int consumoAgua)
27    {
28        this.consumoAgua = consumoAgua;
29        return this;
30    }
31
32    public CasaBuilder setConsumoElectricidad(int consumoElectricidad)
33    {
34        this.consumoElectricidad = consumoElectricidad;

```

```

35     return this;
36 }
37
38 public CasaBuilder setTieneEstufa(boolean tieneEstufa)
39 {
40     this.tieneEstufa = tieneEstufa;
41     return this;
42 }
43
44 public Casa build()
45 {
46     return new Casa(this.calle, this.número, this.ciudad, this.numPisos,
47                     this.consumoAgua, this.color, this.consumoElectricidad,
48                     this.tieneEstufa);
49 }
50 }

```

Aun cuando puede parecer que no hay mucha ganancia, cuando se tienen clases con muchos parámetros en el constructor, y de los cuales solo unos pocos son mandatorios para la creación de las instancias, usar un **Builder** puede hacer que el código quede mucho más legible, y menos propenso a errores.

Una optimización que se puede realizar con el código anterior es buscar la manera de evitar que la clase **Casa** pueda ser instanciada directamente. Ésto se puede lograr haciendo que su constructor sea **private**, pero en ese caso **CasaBuilder** no podría accederlo tampoco, excepto si hacemos lo siguiente:

```

1 public class Casa
2 {
3     private String calle;
4     private int número;
5     private String ciudad;
6     private int cantPisos;
7     private int consumoAgua;
8     private int consumoElectricidad;
9     private boolean tieneEstufa;
10    private String color;
11
12    private Casa(String calle, int número, String ciudad, int cantPisos,
13                int consumoAgua, String color, int consumoElectricidad,
14                boolean tieneEstufa)
15    {
16        this.calle = calle;
17        this.número = número;
18        this.ciudad = ciudad;
19        this.cantPisos = cantPisos;
20        this.consumoAgua = consumoAgua;
21        this.color = color;
22        this.consumoElectricidad = consumoElectricidad;
23        this.tieneEstufa = tieneEstufa;
24    }
25
26    public static class Builder
27    {
28        private String calle;
29        private int número;
30        private String ciudad;
31        private int numPisos;
32        private String color;
33        private int consumoAgua;
34        private int consumoElectricidad;
35        private boolean tieneEstufa;

```

```

36
37     public Builder(String calle, int número, String ciudad, int numPisos)
38     {
39         this.calle = calle;
40         this.número = número;
41         this.ciudad = ciudad;
42         this.numPisos = numPisos;
43     }
44
45     public Builder setColor(String color) { this.color = color; return this; }
46
47     public Builder setConsumoAgua(int consumoAgua)
48     {
49         this.consumoAgua = consumoAgua;
50         return this;
51     }
52
53     public Builder setConsumoElectricidad(int consumoElectricidad)
54     {
55         this.consumoElectricidad = consumoElectricidad;
56         return this;
57     }
58
59     public Builder setTieneEstufa(boolean tieneEstufa)
60     {
61         this.tieneEstufa = tieneEstufa;
62         return this;
63     }
64
65     public Casa build()
66     {
67         return new Casa(this.calle, this.número, this.ciudad,
68             this.numPisos, this.consumoAgua,
69             this.color, this.consumoElectricidad,
70             this.tieneEstufa);
71     }
72 }
73 }

```

En **Java** se pueden declarar clases que se crean dentro de otras clases. En este caso, si se declara como **static**, ésta solo podrá acceder a los atributos estáticos de la clase externa. Para accederla se debe especificar el nombre de dicha clase externa. Estas clases pueden acceder a los datos estáticos de la clase externa, incluyendo los elementos privados, y esa característica se usó para hacer que el constructor de **Casa** se convirtiera a privado para evitar que se pueda instanciar directamente, y forzar a que siempre se tenga que usar **CasaBuilder**.

Una última optimización al momento de usar **CasaBuilder**:

```

1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         Casa casa = new Casa.Builder("Prat", 132, "Coquimbo", 1)
6             .setColor("Blanco")
7             .setConsumoAgua(122)
8             .setConsumoElectricidad(9941)
9             .setTieneEstufa(true)
10            .build();
11     }
12 }

```

10.2.5. Prototype

Existen situaciones en que la creación de una instancia desde cero es complicada o costosa. Por ejemplo, puede ser que la inicialización depende de datos externos (leídos desde un archivo u otras fuentes), y si lo que se busca es crear muchas copias del mismo objeto, pasar repetidamente por el mismo proceso sería una pérdida de recursos.

Una solución para este problema es simplemente crear la primera instancia “costosa”, y después copiar todos los atributos desde el objeto original a un nuevo objeto recién creado, algo como:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         Libro libro = new Libro(1322); // Cargamos el libro 1322 desde un archivo
6
7         // Ahora necesitamos copiar el libro
8
9         Libro copia = new Libro(); // Este es un libro vacío
10
11         copia.setNúmero(libro.getNúmero());
12         copia.setAuthor(libro.getAuthor());
13         // etc
14         // etc
15     }
16 }
```

El problema con este enfoque es que no necesariamente la clase va a exponer su estado interno al mundo exterior. No necesariamente tendrá getters ni setters para todos sus atributos, por lo que la copia del objeto no podrá ser un duplicado exacto del original.

Por otro lado, este enfoque acopla el código de la copia (en este caso, el `main`) al `Libro`, por lo que cualquier modificación que se haga impactará partes del sistema que no deberían impactar.

El patrón `Prototype` puede ser una ayuda para mejorar la situación, en el sentido de que nos dice que el objeto que se copia debe ser el responsable de proveer el mecanismo de copiado, y que no lo debe realizar ninguna otra clase.²

Desde el punto de vista operativo, el patrón implica la creación de un nuevo método llamado `clone()` en la clase copiada. Todo lo que hay que hacer en dicho método es crear una nueva instancia de la clase, y copiar todos los valores desde el objeto original a la nueva copia. Como el método es parte de la clase, en `Java` tendrá completo acceso al estado interno y privado de la misma clase.

En `Java` ya existe la interfaz `Cloneable` que sirve para marcar las clases que soportarán la operación de clonar la instancia:

²En este punto podríamos discutir respecto a si la copia va a ser una copia superficial (`shallow copy`) o profunda (`deep copy`), pero lo dejaremos para después

```

1 public class Libro implements Cloneable
2 {
3     private int número;
4     private String autor;
5
6     public Libro(int número)
7     {
8         this.número = número;
9
10        // Esta operación es muy costosa!
11        this.autor = LibroLoader.cargar(número).getAutor();
12    }
13
14
15    private Libro() {}
16
17    public String getAutor() { return autor; }
18    public int getNúmero() { return número; }
19
20    @Override
21    protected Object clone() throws CloneNotSupportedException
22    {
23        Libro nuevo = new Libro();
24        nuevo.número = this.número;
25        nuevo.autor = this.autor;
26        return nuevo;
27    }
28
29    @Override
30    public String toString()
31    {
32        return "Libro [número=" + número + ", autor=" + autor + "];
33    }
34 }

```

Y ponemos todo a prueba de esta forma:

```

1 public class Main
2 {
3     public static void main(String[] args) throws CloneNotSupportedException
4     {
5         Libro libro = new Libro(1322); // Cargamos el libro 1322 desde un archivo
6
7         // Ahora necesitamos copiar el libro
8
9         Libro nuevo = (Libro) libro.clone();
10
11        System.out.println(libro);
12        System.out.println(nuevo);
13    }
14 }

```

Nótese que en este caso (y en general, es parte del contrato de la interfaz), la operación `clone()` está marcada como que podría lanzar `CloneNotSupportedException`, así que es necesario, o atajar la exception, o marcar el main como lanzando esa exception.

Al obtener la copia del objeto original, podemos usarlo tal como está, o podemos empezar a modificarlo de acuerdo a lo que se necesite.

10.3. Patrones Estructurales

Hay 7 patrones en esta categoría:

10.3.1. Adapter

Imaginemos que se ha creado un programa que se encarga de procesar datos que están almacenados en un archivo, y para desacoplar la aplicación del proceso de lectura, se ha hecho lo siguiente:

```
1 public class Main {  
2     public static void main(String[] args) throws IOException  
3     {  
4         Lector lector = new LectorArchivo("d:/temp/archivo.txt");  
5  
6         Registro registro;  
7  
8         while ((registro = lector.sgteRegistro()) != null) {  
9             procesarRegistro(registro);  
10        }  
11    }  
12  
13    private static void procesarRegistro(Registro registro)  
14    {  
15        System.out.println("Un registro:");  
16        System.out.println(registro.getString(0));  
17        System.out.println(registro.getString(1));  
18        System.out.println(registro.getString(2));  
19        System.out.println();  
20    }  
21 }
```

El archivo que se está leyendo tiene este formato:

```
hola,mundo,hola  
este,es,un  
mensaje,muy,corto
```

Al ejecutar el programa se obtiene:

```
Un registro:  
hola  
mundo  
hola
```

```
Un registro:  
este  
es  
un
```

```
Un registro:  
mensaje  
muy  
corto
```


En otras palabras, se ha delegado el proceso de abrir el archivo y realizar la lectura de bajo nivel a la clase `LectorArchivo`, la que nos entregará los datos usando instancias de `Registro`, el cuál contiene un método `getString(n)` para obtener los “campos” de cada registro (se puede pensar que cada registro es una línea del archivo, y que cada campo es una columna dentro de cada línea).

Para facilitar lo anterior, se creó una interface llamada `Lector`:

```

1 public interface Lector
2 {
3     /**
4      * Retorna el siguiente registro desde la fuente de datos.
5      *
6      * @return El siguiente registro, o null si es que no hay
7      * más registros.
8      */
9     Registro sgteRegistro();
10 }
```

Y la clase `Registro` es muy sencilla:

```

1 public class Registro
2 {
3     private String[] partes;
4
5     public Registro(String linea)
6     {
7         partes = linea.split(",");
8     }
9
10    public String getString(int index)
11    {
12        return partes[index];
13    }
14 }
```

Todo lo anterior funciona muy bien, pero después de tener el programa funcionando, se nos informa que hay otro formato de archivo que es necesario leer:

```

hola2
mundo2
hola2
este2
es2
un2
mensaje2
muy2
corto2
```

Y que además alguien ya ha escrito la clase que lee dicho formato y tenemos que usarla:

```

1 public class LectorLineaALinea
2 {
3     private Scanner scanner;
4
5     public LectorLineaALinea(String fileName) throws IOException
6     {
7         scanner = new Scanner(new File(fileName));
8     }
9
10    public String siguienteLinea()
11    {
12        if (!scanner.hasNextLine()) return null;
13        return scanner.nextLine();
14    }
15 }

```

El problema es que tenemos código ya escrito que usaba la forma de operar de la interfaz `Lector`, y sería complicado tener que adaptar dicho código para que opere usando la clase nueva (tomando en consideración que hay que soportar ambos formatos de archivo). Además, también sería un arduo trabajo cambiar la clase `LectorLineaALinea` para que opere como si fuera una instancia de `Lector`.

En este caso, el patrón `Adapter` nos dice lo que podemos hacer: debemos crear una clase que nos provea la nueva funcionalidad, pero sin tener que modificar el código que ya está escrito, o sea, algo que por un lado le parezca a `App` una instancia de `Lector`, pero que por el otro lado se comuniquen correctamente con una instancia de `LectorLineaALinea`.

```

1 public class LectorLineaALineaAdapter implements Lector
2 {
3     private LectorLineaALinea lector;
4
5     public LectorLineaALineaAdapter(LectorLineaALinea lector)
6     {
7         this.lector = lector;
8     }
9
10    @Override
11    public Registro sgteRegistro()
12    {
13        String a = lector.siguienteLinea();
14        if (a == null) return null;
15
16        String b = lector.siguienteLinea();
17        if (b == null) return null;
18
19        String c = lector.siguienteLinea();
20        if (c == null) return null;
21
22        String tmp = a + "," + b + "," + c;
23        return new Registro(tmp);
24    }
25 }

```

Esta clase nos permite usar una instancia que tiene una interfaz incompatible, y es el `Adapter` el que se encargará de traducir los mensajes, de forma que `App` siga operando sin cambios, tal como si estuviera usando una instancia de `Lector`:

```

1 public class Main
2 {
3     public static void main(String[] args) throws IOException
4     {
5         LectorLineaALinea objetivo = new LectorLineaALinea("d:/temp/archivo2.txt");
6         Lector lector = new LectorLineaALineaAdapter(objetivo);
7
8         Registro registro;
9
10        while ((registro = lector.sgteRegistro()) != null) {
11            procesarRegistro(registro);
12        }
13
14    }
15
16    private static void procesarRegistro(Registro registro)
17    {
18        System.out.println("Un registro:");
19        System.out.println(registro.getString(0));
20        System.out.println(registro.getString(1));
21        System.out.println(registro.getString(2));
22        System.out.println();
23    }
24 }

```

Nótese que aparte de las líneas donde se instancia al **Adapter**, el resto del código no sufrió modificaciones.

Generalmente este patrón es útil cuando se necesita integrar código antiguo y/o que no se puede modificar, y en general, cuando se necesita incorporar clases cuyas interfaces no son compatibles con el código actual.

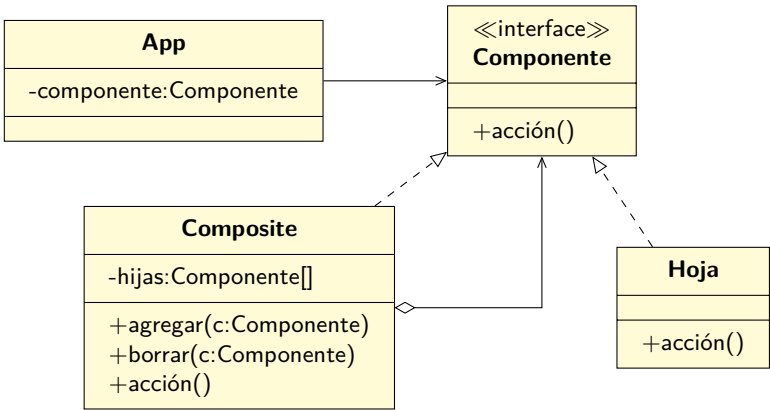
10.3.2. Composite

Hay veces en que nos enfrentaremos a problemas en que la suma de las partes es igualmente una parte. Por ejemplo, si modelamos una unidad de una organización, que a la vez puede estar compuesta de otras unidades, si enviamos un mensaje a la unidad superior, dicho mensaje debería va a llegar a todas las unidades componentes, como un proceso recursivo que se repite hasta que toda la jerarquía de unidades se recorra completa.

O también podemos modelar un sistema donde se dibujan “ventanas” (por ejemplo, usando la **API WIN32**), y las ventanas pueden alojar a otras ventanas, y así recursivamente. Si queremos mover la ventana de nivel superior, se terminará enviando el mensaje de movimiento a toda la jerarquía involucrada.

Al aplicar el patrón **Composite** podemos simplificar este problema, al distinguir claramente entre 3 tipos de objetos que participarán en la solución:

- El componente base, que es el que tiene la interfaz pública que todos los objetos deben implementar.
- La hoja, que implementa el comportamiento de aquellos objetos que no contienen otros objetos.
- El composite, que implementa el comportamiento del componente base, pero contiene otros objetos.



Para solucionar el problema actual, primero se creará una interfaz para definir la interfaz pública de las "ventanas":

```
1 public interface Ventana
2 {
3     void mover(int x, int y);
4     void dibujar();
5 }
```

En este problema, todas las **Ventanas** tendrán una posición, así que se podrán mover. Además se podrán dibujar a la pantalla. Por ejemplo, podemos tener un **Botón**:

```
1 public class Botón implements Ventana
2 {
3     private int x = 0;
4     private int y = 0;
5
6     public Botón(int x, int y)
7     {
8         this.x = x;
9         this.y = y;
10    }
11
12    @Override
13    public void mover(int dx, int dy)
14    {
15        this.x += dx;
16        this.y += dy;
17    }
18
19    @Override
20    public void dibujar()
21    {
22        System.out.println("Dibujando BOTÓN en (" + x + "," + y + ")");
23    }
24 }
```

Y una **Etiqueta**:

```

1 public class Etiqueta implements Ventana {
2     private int x = 0;
3     private int y = 0;
4
5     public Etiqueta(int x, int y) {
6         this.x = x;
7         this.y = y;
8     }
9
10    @Override
11    public void mover(int dx, int dy)
12    {
13        this.x += dx;
14        this.y += dy;
15    }
16
17    @Override
18    public void dibujar()
19    {
20        System.out.println("Dibujanda ETIQUETA en (" + x + "," + y + ")");
21    }
22 }

```

Ambas clases anteriores serían equivalente a las hojas mencionadas previamente. Por también podemos tener algo que cumpla el rol de **Composite**:

```

1 public class Frame implements Ventana {
2     private int x = 0;
3     private int y = 0;
4     private LinkedList<Ventana> ventanas = new LinkedList<>();
5
6     public void agregar(Ventana v) { ventanas.add(v); }
7
8     public Frame(int x, int y) {
9         this.x = x;
10        this.y = y;
11    }
12
13    @Override
14    public void mover(int dx, int dy)
15    {
16        this.x += dx;
17        this.y += dy;
18
19        for(Ventana v : ventanas) {
20            v.mover(dx, dy);
21        }
22    }
23
24    @Override
25    public void dibujar()
26    {
27        System.out.println("Dibujando FRAME en (" + x + "," + y + ")");
28        for(Ventana v : ventanas) {
29            v.dibujar();
30        }
31        System.out.println();
32    }
33 }

```

Nótese cómo `Frame` implementa la interfaz `Ventana`, pero a la vez contiene una serie de otras `Ventanas`, y al momento de ella dibujarse, dibuja también a todas sus `Ventanas` hijas.

Para poner a prueba todo esto podemos escribir algo como lo siguiente:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         Frame f = new Frame(100, 200);
6         f.dibujar();
7         f.mover(10, 20);
8         f.dibujar();
9
10        f.agregar(new Botón(5, 7));
11        f.agregar(new Etiqueta(10, 20));
12        f.dibujar();
13        f.mover(5, 3);
14        f.dibujar();
15
16        Frame f0 = new Frame(3, 3);
17        f0.agregar(new Etiqueta(100, 200));
18        f0.agregar(f);
19        f0.dibujar();
20    }
21 }
```

En este caso, se puede destacar cómo se construye primero el `Frame f`, agregándole varios elementos, y después esa misma `Frame f` se agrega al `Frame f0`. Al ejecutar este código se obtiene la siguiente salida por pantalla:

Dibujando FRAME en (100,200)

Dibujando FRAME en (110,220)

Dibujando FRAME en (110,220)

Dibujando BOTÓN en (5,7)

Dibujanda ETIQUETA en (10,20)

Dibujando FRAME en (115,223)

Dibujando BOTÓN en (10,10)

Dibujanda ETIQUETA en (15,23)

Dibujando FRAME en (3,3)

Dibujanda ETIQUETA en (100,200)

Dibujando FRAME en (115,223)

Dibujando BOTÓN en (10,10)

Dibujanda ETIQUETA en (15,23)

Este patrón es útil cuando se encuentran situaciones en que hay que manipular objetos simples y complejos de la misma manera.

10.3.3. Proxy

De vez en cuando nos encontraremos con situaciones en que usar un objeto impone una carga en el sistema, debido al tiempo necesario para obtener los datos, uso de recursos escasos, etc. Además, si existe la posibilidad de que dichos objetos no se usen de forma completa, la situación es más delicada aun. Por ejemplo, crear una instancia de `Persona`, pero solo utilizar uno o dos de sus atributos, o incluso no utilizarla para nada, siendo que se cargó la instancia completa, incluyendo todas sus dependencias.

En estos casos es útil aplicar el patrón `Proxy`, de forma de crear un tipo de objeto que pueda tomar el puesto del objeto real, y que pueda administrar los mensajes que se le envían a dicho objeto. En este caso, solo si el objeto se trata de utilizar realmente se llamarían a los métodos reales, e incluso se podrían reutilizar llamadas previas para tratar de evitar hacer llamadas costosas.

Imaginemos que tenemos una clase que se encarga de leer un archivo desde una ubicación remota, y que nos provee la siguiente interfaz pública:

```
1 public class LectorLineasArchivo
2 {
3     private String fileName;
4     private Scanner scanner;
5     private int indexActual;
6
7     public LectorLineasArchivo(String fileName)
8     {
9         this.fileName = fileName;
10        open();
11    }
12
13    private void open()
14    {
15        try {
16            scanner = new Scanner(new File(fileName));
17        } catch (FileNotFoundException e) {
18            scanner = null;
19        }
20        indexActual = -1;
21    }
22
23    public String getSgteLinea()
24    {
25        indexActual++;
26        return scanner.nextLine();
27    }
28
29    public String getLinea(int index)
30    {
31        if (index <= indexActual) {
32            open();
33        }
34        String linea = null;
35        while (index > indexActual) {
36            linea = getSgteLinea();
37        }
38        return linea;
39    }
40 }
```

Imaginemos también que dentro del código que usa a esta clase, se ha detectado que hay veces en la

que clase se instancia y no se usa, y además el hecho de llamar repetidamente a `getLínea(index)` es una operación muy costosa.

Si ponemos a prueba la clase:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         LectorLíneasArchivo lector = new LectorLíneasArchivo("palabras.txt");
6
7         System.out.println(lector.getSgteLínea());
8         System.out.println(lector.getSgteLínea());
9         System.out.println(lector.getSgteLínea());
10
11        System.out.println(lector.getLínea(7));
12
13        System.out.println(lector.getLínea(0));
14        System.out.println(lector.getLínea(7));
15        System.out.println(lector.getLínea(7));
16        System.out.println(lector.getLínea(7));
17    }
18 }
```

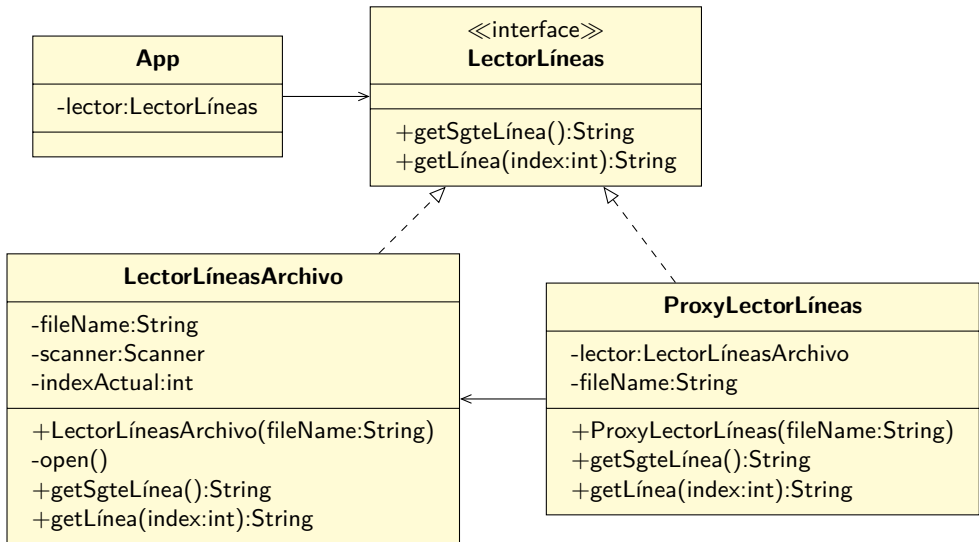
Al usar este archivo:

Este
es
un
texto
que
tiene
muchas
palabras

Se obtendrá lo siguiente por la pantalla:

Este
es
un
palabras
Este
palabras
palabras
palabras

El patrón **Proxy** nos dice que debemos crear una nueva clase que tenga la misma interfaz que la clase anterior, y en vez de instanciar `LectorLíneasArchivo` usar el **Proxy**:



Para lograr lo anterior, vamos a crear la interface **LectorLíneas**:

```

1 public interface LectorLíneas
2 {
3     String getSgteLínea();
4
5     String getLínea(int index);
6 }
  
```

Además de modificar **LectorLíneasArchivo** para que implemente la interfaz:

```

1 public class LectorLíneasArchivo implements LectorLíneas {
2     ...
  
```

Crear el **Proxy**:

```

1 public class ProxyLectorLíneas implements LectorLíneas {
2     private String fileName;
3     private LectorLíneasArchivo lector;
4
5     public ProxyLectorLíneas(String fileName) {
6         this.fileName = fileName;
7     }
8
9     @Override
10    public String getSgteLínea()
11    {
12        if (lector == null) lector = new LectorLíneasArchivo(fileName);
13        return lector.getSgteLínea();
14    }
15
16    @Override
17    public String getLínea(int index)
18    {
19        if (lector == null) lector = new LectorLíneasArchivo(fileName);
20        return lector.getLínea(index);
21    }
22 }
  
```

Y modificar el `Main` para que use el `Proxy` en vez de la clase original. Nótese que aparte de modificar la creación del objeto, no se tuvo que cambiar nada más en el código:

```

1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         LectorLíneas lector = new ProxyLectorLíneas("palabras.txt");
6
7         System.out.println(lector.getSgteLínea());
8         System.out.println(lector.getSgteLínea());
9         System.out.println(lector.getSgteLínea());
10
11        System.out.println(lector.getLínea(7));
12
13        System.out.println(lector.getLínea(0));
14        System.out.println(lector.getLínea(7));
15        System.out.println(lector.getLínea(7));
16        System.out.println(lector.getLínea(7));
17    }
18 }

```

Aprovechando que a través del `Proxy` tenemos control del lector, podemos aplicar una pequeña optimización:

```

1 public class ProxyLectorLíneas implements LectorLíneas
2 {
3     private String fileName;
4     private LectorLíneasArchivo lector;
5
6     public ProxyLectorLíneas(String fileName)
7     {
8         this.fileName = fileName;
9     }
10
11    @Override
12    public String getSgteLínea()
13    {
14        if (lector == null) lector = new LectorLíneasArchivo(fileName);
15        return lector.getSgteLínea();
16    }
17
18    @Override
19    public String getLínea(int index)
20    {
21        if (index == cachedIndex) {
22            return cachedLine;
23        }
24        if (lector == null) lector = new LectorLíneasArchivo(fileName);
25        String línea = lector.getLínea(index);
26        cachedIndex = index;
27        cachedLine = línea;
28        return línea;
29    }
30
31    private String cachedLine = null;
32    private int cachedIndex = -1;
33 }

```

Ya que sabemos que llamar a `getLínea(index)` en el `Lector` original es muy costoso, desde el `Proxy`

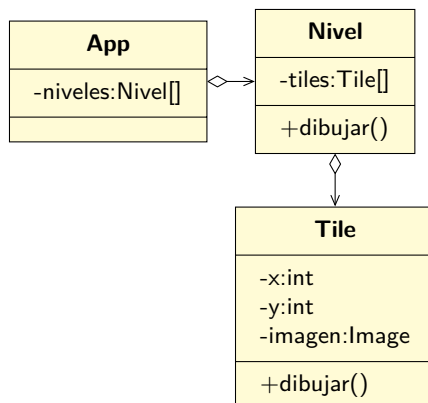
guardamos la última llamada y su respuesta, de forma que si la siguiente llamada a ese método tiene el mismo valor para `indice`, se retorna el valor guardado en vez de invocar al `Lector` original.

10.3.4. Flyweight

Supongamos que estamos creando un juego de plataformas estilo 8bit, en donde un jugador tiene que desplazarse lateralmente por niveles muy grandes, y que debe superar varias dificultades para lograr terminar cada etapa.

Supongamos también que para dibujar las imágenes de fondo que el jugador verá en la pantalla, se ha creado una clase llamada `Tile`.³ Esta clase contendrá lo necesario para dibujar cada `Tile` de cada nivel.

Para realizar lo anterior, cada `Tile` contendrá varios atributos: por un lado debe tener algo para indicar la posición del `Tile` dentro del nivel, y también contener los datos gráficos de la imagen a la que representa, por ejemplo, suelo, agua, cielo, etc.



Si hacemos un cálculo rápido respecto a la cantidad de recursos que se requiere para dibujar un nivel, consideremos lo siguiente:

- Un nivel tendrá 7680 pixels de ancho y 1088 de alto
- Cada `Tile` representará un trozo de 16x16 de pantalla
- Por lo tanto, se requieren 480 `Tiles` de ancho y 68 de alto
- La cantidad total de `Tiles` es 32640
- Cada `Tile` contendrá por lo menos 2 atributos (su posición x, y), y cada atributo es un `int`, por lo que se requieren 8 bytes de memoria
- Además, la información gráfica requiere 3 bytes por píxel, así que cada `Tile` requiere 768 bytes
- El total de memoria utilizada por `Tile` es 776

³La traducción a español sería teja, baldosa, azulejo; nos quedaremos con `tile`

- El total de memoria utilizada en concepto de **Tiles** por nivel es 25328640 bytes, o alrededor de 24MB.

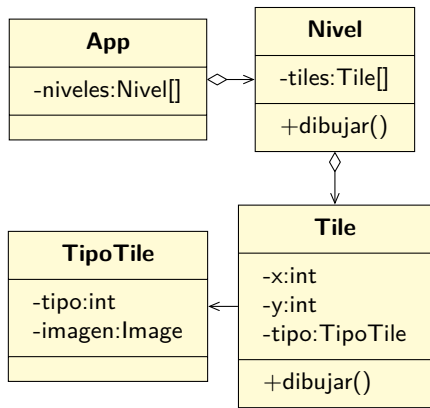
Puede que 24MB no parezca mucho hoy en día, pero si lo multiplicamos por la cantidad de niveles que puede tener un juego, los MB se empiezan a acumular. Pero podemos hacerlo mejor, aplicando el patrón **Flyweight**.

Este patrón nos dice que lo podemos aplicar cuando tenemos muchos objetos similares (por ejemplo, las 32640 instancias de **Tile**), cuyas características se pueden dividir en dos tipos:

Extrínsecas que corresponden al estado del objeto, y que puede ser alterado por el mundo externo. En nuestro caso, cada **Tile** tiene coordenadas, que pueden ser cambiadas.

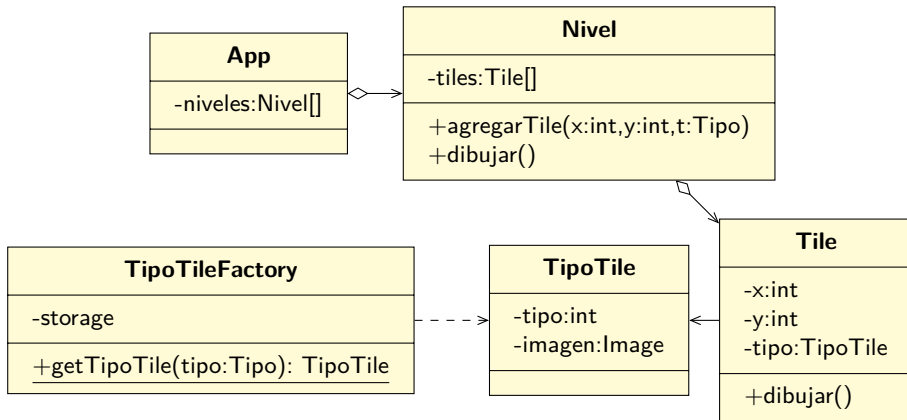
Intrínsecas que corresponden a datos que se mantienen constantes, y que están dentro de los objetos. Estos datos se leen pero no se cambian. En el caso de los **Tiles**, la información gráfica de cada tipo de **Tile** es constante. Por ejemplo, todos los **Tiles** de tipo suelo comparten la misma imagen gráfica.

Bajo este nuevo esquema, cada **Tile** dejaría de contener la información intrínseca, y ésta pasaría a un nuevo objeto para poder compartir los datos entre las instancias de **Tile** que son del mismo tipo.



Puede parecer que es lo mismo de antes, pero el objetivo de este esquema es que las instancias de **TipoTile** se comparten entre las miles de instancias de **Tile**. De hecho, en este caso, solo deberían existir tantas instancias de **TipoTile** como tipos de **Tiles** hay en el juego (suelo, agua, cielo, etc.).

Generalmente, para administrar el ciclo de vida de estas clases es común crear un **Factory**, que vaya guardando referencias a las instancias con sus tipos, de forma que cuando se le pide crear una instancia de un tipo ya visto, retorne el objeto previamente guardado.



Una posible implementación de `TipoTileFactory` puede ser:⁴

```

1 public class TipoTileFactory
2 {
3     private static Hashtable<TipoTile.Tipo, TipoTile> storage = new Hashtable<>();
4
5     public static TipoTile getTipoTile(TipoTile.Tipo tipo)
6     {
7         TipoTile t;
8
9         if (!storage.containsKey(tipo)) {
10             t = new TipoTile(tipo);
11             storage.put(tipo, t);
12         } else {
13             t = storage.get(tipo);
14         }
15         return t;
16     }
17 }
  
```

Y la parte relevante de la clase `Nivel` sería:

```

1 public class Nivel
2 {
3     private List<Tile> tiles = new LinkedList<>();
4
5     public void agregarTile(int x, int y, TipoTile.Tipo tipo)
6     {
7         TipoTile tipoTile = TipoTileFactory.getTipoTile(tipo);
8         Tile tile = new Tile(x, y, tipoTile);
9         tiles.add(tile);
10    }
11 }
  
```

Bajo este nuevo esquema, la cantidad de memoria usada por nivel es de 524556 bytes, o alrededor de 512KB (522240 bytes para los 32640 instancias de `Tile`, y 2316 bytes para las 4 instancias de `TipoTile`, una para suelo, agua, cielo). Esto es un 2% del valor original. Eso es lo bueno usar de el patrón `Flyweight`.

⁴En el código se usa algo nuevo: `Hashtable`. Si quieres saber qué es y para qué se usa, anda a la página 462

10.3.5. Facade

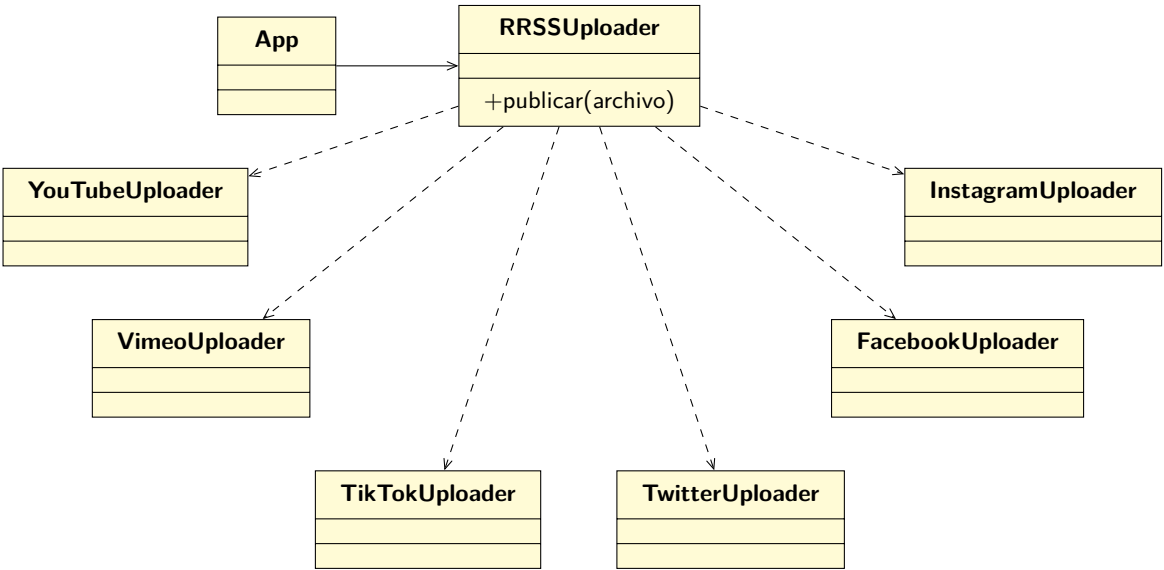
Hay veces en que nuestros sistemas necesitan usar otros sistemas, y dependiendo de la complejidad de dichos sistemas, nos tendremos que preocupar de inicializar los objetos, mantener las dependencias, cuidar que los métodos se ejecuten en el orden correcto, recordar cerrar recursos cuando sea necesario, etc.

En estos casos, puede ser apropiado aplicar el patrón **Fachada** para hacernos la vida más fácil. Este patrón nos dice que en estos casos podemos crear una “fachada” entre nuestro sistema y el resto de los sistemas, visibilizando solo lo necesario para que nuestra aplicación funcione, y dejando en manos de la fachada el ciclo de vida y los cuidados necesarios de los otros subsistemas.

Imaginemos que nuestro sistema trata de solucionar el problema de publicar videos e imágenes en distintas plataformas. Asumiendo que existen bibliotecas de funciones y clases que permiten manejar cada plataforma de manera particular, el problema es cómo incorporar todas esas diferentes plataformas y diferencias de uso en nuestro software. Además, se puede pensar en qué habría que hacer si se agrega alguna plataforma nueva en el futuro.

La forma inocente de hacer ésto es simplemente administrar todo de forma directa, con todos los problemas y requerimientos que ésto conlleva.

Pero si aplicamos el patrón **Fachada**, deberíamos lograr tener lo siguiente:



En vez de que el código principal de nuestro sistema se tenga que preocupar de las particularidades de cada subsistema, se delega ese proceso a una clase particular, que ocultará los detalles al resto del código.

Lo anterior trae ventajas desde el punto de vista de disminuir la complejidad de nuestro código, además que minimiza el esfuerzo de actualizar los subsistemas o agregar nuevos. Lo único que se tendría que modificar es la clase que implementa la **Fachada**.

Revisemos otro ejemplo para clarificar dudas. En este caso, crearemos un sistema de audio con múltiples componentes y utilizaremos una fachada para simplificar su uso.

Primero definiremos los componentes del sistema de audio:

```
1 // Subsistema para amplificador de sonido
2 class Amplifier {
3     void on() {
4         System.out.println("Amplificador encendido");
5     }
6
7     void setVolume(int volume) {
8         System.out.println("Configurando el volumen del amplificador a " + volume);
9     }
10
11    void off() {
12        System.out.println("Amplificador apagado");
13    }
14 }
```

```
1 // Subsistema para reproductor de DVD
2 class DvdPlayer {
3     void on() {
4         System.out.println("Reproductor de DVD encendido");
5     }
6
7     void play(String movie) {
8         System.out.println("Reproduciendo la película: " + movie);
9     }
10
11    void stop() {
12        System.out.println("Reproductor de DVD detenido");
13    }
14
15    void off() {
16        System.out.println("Reproductor de DVD apagado");
17    }
18 }
```

```
1 // Subsistema para proyector
2 class Projector {
3     void on() {
4         System.out.println("Proyector encendido");
5     }
6
7     void wideScreenMode() {
8         System.out.println("Modo de pantalla panorámica activado en el proyector");
9     }
10
11    void off() {
12        System.out.println("Proyector apagado");
13    }
14 }
```

Ahora crearemos la fachada que permitirá interactuar con el sistema de audio completo:

```

1 // Fachada que simplifica la interacción con el sistema de audio
2 class HomeTheaterFacade {
3     private Amplifier amplifier;
4     private DvdPlayer dvdPlayer;
5     private Projector projector;
6
7     public HomeTheaterFacade() {
8         amplifier = new Amplifier();
9         dvdPlayer = new DvdPlayer();
10        projector = new Projector();
11    }
12
13    public void watchMovie(String movie) {
14        System.out.println(";Preparándose para ver una película!");
15        amplifier.on();
16        amplifier.setVolume(5);
17        dvdPlayer.on();
18        dvdPlayer.play(movie);
19        projector.on();
20        projector.wideScreenMode();
21    }
22
23    public void endMovie() {
24        System.out.println(";Terminando la película!");
25        amplifier.off();
26        dvdPlayer.stop();
27        dvdPlayer.off();
28        projector.off();
29    }
30 }

```

Ahora la clase principal para poner todo a prueba:

```

1 public class Main {
2     public static void main(String[] args) {
3         HomeTheaterFacade homeTheater = new HomeTheaterFacade();
4
5         String movie = "La película de ejemplo";
6
7         homeTheater.watchMovie(movie);
8
9         // Simulamos ver la película durante un tiempo
10        try {
11            Thread.sleep(5000); // Espera de 5 segundos
12        } catch (InterruptedException e) {
13            e.printStackTrace();
14        }
15
16        homeTheater.endMovie();
17    }
18 }

```

10.3.6. Bridge

Este patrón se usa para desacoplar la implementación de las interfaces, de manera de ocultarle a los clientes los detalles de implementación.

10.3.7. Decorator

Existen muchas situaciones en que se necesita alterar el comportamiento de un objeto, y lo primero en que se piensa es en crear una subclase y especializar dicho comportamiento. Pero realizar herencias no es algo que sirva en todos los casos. Considera que:

- La herencia es estática, se define al momento de la compilación. No se puede alterar el comportamiento de los objetos en tiempo de ejecución. Lo único que se puede hacer es quitar un objeto y reemplazarlo con otro.
- En Java las clases solo puede tener una superclase, así que no se puede heredar el comportamiento de varias clases simultáneamente.

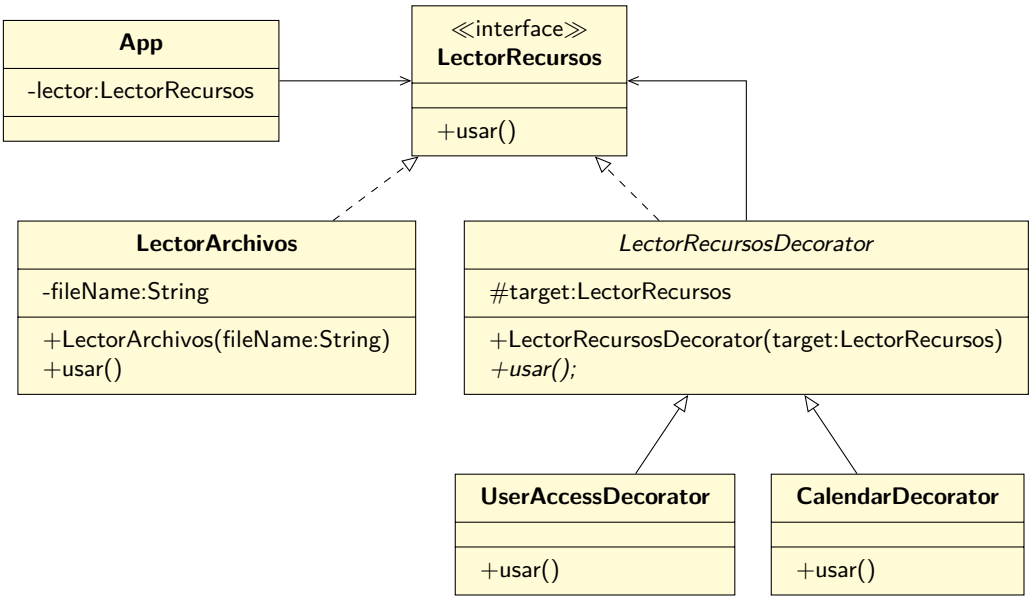
Una muy buena alternativa para superar estos problemas es usar el patrón **Decorator**, que se puede resumir como un **wrapper** (o envoltorio en español). Al usar este patrón se tiene que crear un wrapper alrededor de otro objeto (el objeto objetivo). El wrapper tendrá los mismos métodos que el objetivo, y le delegará todos los mensajes, pero podrá realizar modificaciones ya sea haciendo algo antes o después de pasarle los mensajes al objetivo.

La idea de fondo es que los clientes no sepan que están operando sobre un wrapper en vez del objeto original. De hecho, al instanciar un wrapper se le pasará un objeto que respete la misma interfaz. De esta forma, un objeto se puede envolver en muchos wrappers, sin que el cliente se dé cuenta.

Imaginemos que vamos a construir un software que pone a disposición de un cliente una serie de “recursos” (por ejemplo, archivos), pero para que el cliente los pueda acceder, tiene que cumplir una serie de revisiones (por ejemplo, que el cliente tenga acceso al recurso, que el día y hora sea la correcta, etc.). Además, todas las revisiones dependen de cómo se configura el sistema, por lo que no todas las revisiones estarán activas al mismo tiempo.

Una forma de realizar lo anterior es tratar de encontrar un esquema de interfaces y herencia que permita revisar todo lo que se tiene que revisar, pero como el esquema es variable, eventualmente se llegará al punto de tener que usar una cantidad sustancial de `if` para tomar en cuenta todas las posibles variaciones.

Otra forma de solucionar este problema es seguir los lineamientos del patrón **Decorator**. En este caso, cada revisión será un decorador, y de hecho, todas las clases involucradas implementarán la misma interface, de forma que el cliente final no se entere de qué clase concreta está manipulando:



Para facilitar la creación de la instancia de `LectorArchivo`, usaremos una `Factory`, de la siguiente forma:

```
1 public class LectorFactory
2 {
3     public static boolean checkUserAccess = true;
4     public static boolean checkCalendarAccess = true;
5
6     private static void leerConfiguración()
7     {
8         // Desde un archivo de configuración leeremos el valor que
9         // hay que asignarle a checkUserAccess y checkCalendarAccess
10    }
11
12    public static LectorRecursos crear(String fileName)
13    {
14        leerConfiguración();
15        LectorRecursos lector = new LectorArchivos(fileName);
16
17        if (checkUserAccess) {
18            lector = new UserAccessDecorator(lector);
19        }
20        if (checkCalendarAccess) {
21            lector = new CalendarDecorator(lector);
22        }
23
24        return lector;
25    }
26 }
```

Y ahí se puede ver cómo funciona el wrapper. A medida que se requiere agregarle funcionalidad a la clase que hace el trabajo, ésta se va envolviendo en diferentes decoradores, cada uno agregando algo más, hasta que finalmente se le entrega al cliente una instancia de algo que implementa las operaciones, pero no sabe el tipo concreto que recibirá.

```

1 public interface LectorRecursos
2 {
3     void usar();
4 }

```

Y la clase concreta que hace el trabajo es la siguiente:

```

1 public class LectorArchivos implements LectorRecursos
2 {
3     private String fileName;
4
5     public LectorArchivos(String fileName)
6     {
7         this.fileName = fileName;
8     }
9
10    @Override
11    public void usar()
12    {
13        System.out.println("LectorArchivos.usar(): " + fileName);
14    }
15 }

```

La clase que sirve de base para los decoradores requiere recibir la instancia de `LectorRecursos` que será el objetivo:

```

1 public abstract class LectorRecursosDecorator implements LectorRecursos
2 {
3     protected LectorRecursos target;
4
5     public LectorRecursosDecorator(LectorRecursos target)
6     {
7         this.target = target;
8     }
9 }

```

Y los decoradores simplemente tomarán la decisión de llamar a su `LectorRecursos` objetivo, o lanzar una excepción en caso de que las condiciones no se cumplan.

```

1 public class UserAccessDecorator extends LectorRecursosDecorator {
2     public UserAccessDecorator(LectorRecursos target)
3     {
4         super(target);
5     }
6
7     @Override
8     public void usar()
9     {
10        System.out.println("UserAccessDecorator.usar()");
11        if (!tieneAccesoAlRecurso())
12            throw new IllegalAccessError("El usuario no tiene acceso");
13
14        this.target.usar();
15    }
16
17    private boolean tieneAccesoAlRecurso()
18    {
19        return false;
20    }
21 }

```

```
1 public class CalendarDecorator extends LectorRecursosDecorator
2 {
3     public CalendarDecorator(LectorRecursos target)
4     {
5         super(target);
6     }
7
8     @Override
9     public void usar()
10    {
11        System.out.println("CalendarDecorator.usar()");
12        if (!esFechaDeAcceso())
13            throw new IllegalArgumentException("No se puede acceder por fecha");
14
15        this.target.usar();
16    }
17
18    private boolean esFechaDeAcceso()
19    {
20        return false;
21    }
22 }
```

De esta forma es fácil agregar comportamiento extra a una clase de forma dinámica, y sin que los clientes de la clase se enteren, ya que la lógica se estructura en capas que son configurables en tiempo de ejecución.

10.4. Patrones de Comportamiento

Hay 11 patrones en esta categoría:

10.4.1. Template

Este patrón muestra cómo crear métodos vacíos, y dejar pendiente la implementación de éstos a las subclases.

10.4.2. Mediator

Este patrón muestra cómo proveer un medio de comunicación centralizado entre diferentes objetos del sistema.

10.4.3. Chain of Responsibility

Existen situaciones en que nuestro software contendrá algún tipo de “solicitudes” que deben ser revisadas para determinar si se pueden llevar a cabo o no. Estas solicitudes tienen muchas formas y usos, pero generalmente pasarán por una secuencia de revisión, y podrán ser aceptadas o rechazadas en cualquier punto de dicha revisión.

Un ejemplo típico de este tipo de secuencias o cadenas de revisión es cuando se tiene un sistema que genera eventos, y dichos eventos pueden ser registrados de diferentes maneras (a la pantalla, a archivos, enviados por email, etc.). Cuando se crea una de esos eventos, éste podrían pasar por cada eslabón de la cadena, y cada receptor podría examinar el evento y decidir si tiene que tomar acción, o pasar la solicitud al siguiente elemento de la cadena (o también puede decidir terminar la solicitud directamente).

Otro ejemplo es la implementación de un sistema de aprobaciones de compras, donde las personas crean una solicitud para realizar una compra y se la pasan a su jefatura directa para ser aprobada. Si la solicitud es aprobable a ese nivel, (y cumple los requisitos necesarios) se aprueba, pero si no cae dentro de la competencia de la jefatura, ésta la envía al siguiente nivel de aprobación, y el proceso se va repitiendo hasta que finalmente la solicitud es aprobada o rechazada.

Como siempre, todo lo anterior se puede implementar a base de condicionales, pero lo más seguro es que el código resultante estará totalmente acoplado a las condiciones actuales, y sobretodo, no será fácilmente modificable.

En estos casos, es útil usar el patrón **Chain of Responsibility**, que nos dice que tenemos que convertir los comportamientos particulares de cada eslabón de la cadena en clases aisladas (llamadas “**handlers**”⁵), y que la solicitud debe pasarse por la cadena. La cadena estará formada por instancias de una interfaz (o clase base) que contendrá una referencia al siguiente eslabón. Cuando una solicitud se mueve por la cadena, cada uno de dichos eslabones tendrá la oportunidad de analizar la solicitud para ver si le corresponde resolverla (retornando una respuesta, y terminando todo el proceso), o pasarla al siguiente eslabón.

En nuestro caso de aprobaciones de compras, el programa lo vamos a escribir así:

```

1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         Aprobador jefatura = configurarCadenaAprobación();
6
7         testSolicitud("Compra papel fotocopia", jefatura, new Solicitud.Builder()
8             .setMonto(5000)
9             .setCantidadCotizaciones(3)
10            .setTieneBeneficioCompañía(true)
11            .build());
12
13         testSolicitud("Compra camioneta", jefatura, new Solicitud.Builder()
14             .setMonto(27000000)
15             .setCantidadCotizaciones(3)
16             .setTieneBeneficioCompañía(true)
17             .build());
18
19         testSolicitud("Construcción edificio", jefatura, new Solicitud.Builder()
20             .setMonto(150000000)
21             .setCantidadCotizaciones(3)
22             .setTieneBeneficioCompañía(true)
23             .setTieneAnálisisDeRiesgo(true)
24             .setTieneEstudioMercado(true)
25             .build());
26     }
27     ...
28 }
```

⁵manejadores, en inglés

Notese que estamos usando el patrón **Builder** para hacernos más fácil la vida al crear la instancias de **Solicitud** (ver página 253).

El método para crear la cadena de aprobación no es necesario que sea siempre el mismo: por ejemplo, se puede leer desde un archivo. En este caso, como ejemplo, se ha definido así:

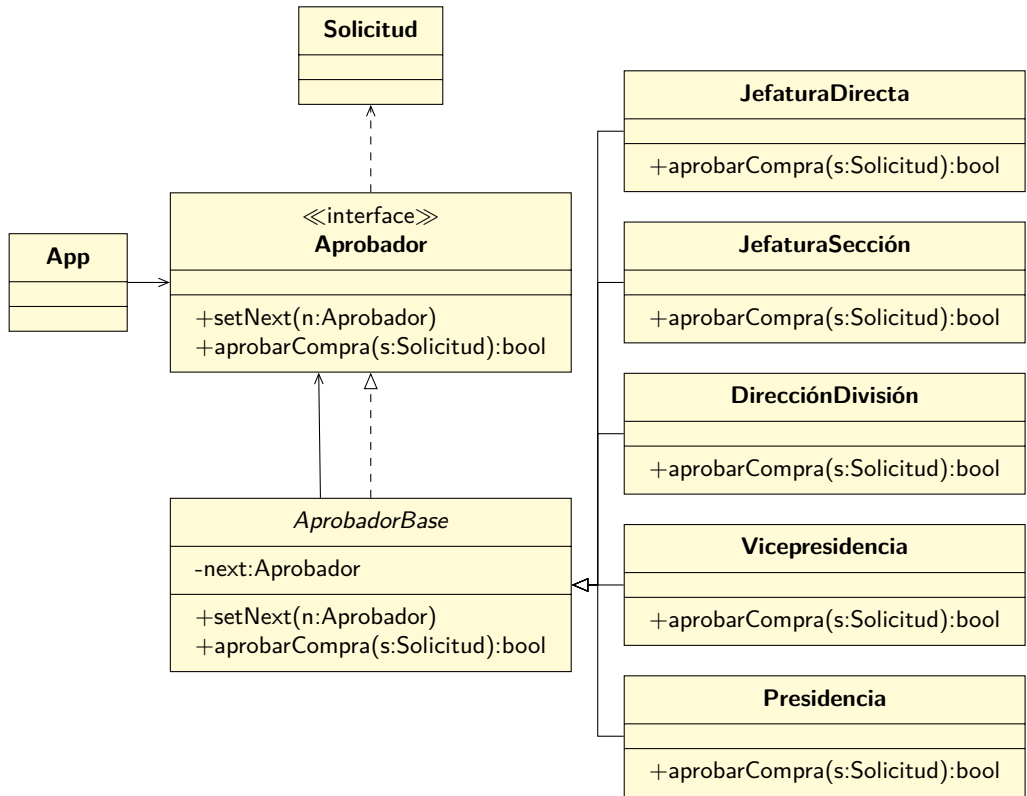
```
1 private static Aprobador configurarCadenaAprobación()
2 {
3     Aprobador jefatura = new JefaturaDirecta();
4     Aprobador jsección = new JefaturaSección();
5     Aprobador jdivisión = new DirecciónDivisión();
6     Aprobador jvice = new Vicepresidencia();
7     Aprobador jpresidencia = new Presidencia();
8
9     jefatura.setNext(jsección);
10    jsección.setNext(jdivisión);
11    jdivisión.setNext(jvice);
12    jvice.setNext(jpresidencia);
13
14    return jefatura;
15 }
```

Y el método que revisa si la solicitud se puede aprobar o no es el siguiente:

```
1 private static void testSolicitud(String titulo, Aprobador jefatura, Solicitud solicitud
2 )
3 {
4     boolean resultado = jefatura.aprobarCompra(solicitud);
5
6     if (resultado) {
7         System.out.println(titulo + ": aprobada");
8     } else {
9         System.out.println(titulo + ": NO aprobada");
10    }
11 }
```

Obviamente, si quisiéramos se podría implementar toda esta lógica sin usar el patrón, pero estaríamos llenando el código de condicionales que a la larga serían difíciles de cambiar, e incluso de modificar en tiempo de ejecución.

Al usar el patrón, podemos descomponer el problema, y quedarnos con objetos que al trabajar juntos logran crear una solución más desacoplada, y con más libertad de ser utilizada de diferentes formas para tener flexibilidad en el modelo:



Veamos cómo se implementa la interfaz **Aprobador**:

```

1 public interface Aprobador
2 {
3     void setNext(Aprobador js);
4
5     boolean aprobarCompra(Solicitud solicitud);
6 }

```

A partir de ella definimos **AprobadorBase**, que será la base de todos los aprobadores:

```

1 public abstract class AprobadorBase implements Aprobador {
2     private Aprobador next;
3
4     @Override
5     public void setNext(Aprobador next)
6     {
7         this.next = next;
8     }
9
10    @Override
11    public boolean aprobarCompra(Solicitud solicitud)
12    {
13        if (next != null)
14            return next.aprobarCompra(solicitud);
15
16        return false;
17    }
18 }

```

Veamos el aprobador `JefaturaDirecta`:

```
1 public class JefaturaDirecta extends AprobadorBase
2 {
3     @Override
4     public boolean aprobarCompra(Solicitud solicitud)
5     {
6         if (solicitud.getMonto() <= 10000) {
7             return true;
8         }
9         return super.aprobarCompra(solicitud);
10    }
11 }
```

Y el aprobador `Presidencia`:

```
1 public class Presidencia extends AprobadorBase
2 {
3     @Override
4     public boolean aprobarCompra(Solicitud solicitud)
5     {
6         if (solicitud.getMonto() > 1000000) {
7             if (!solicitud.isTieneBeneficioCompañía()) {
8                 return false;
9             }
10            if (solicitud.getCantCotizaciones() < 3) {
11                return false;
12            }
13            if (!solicitud.isTieneAnálisisDeRiesgo()) {
14                return false;
15            }
16            if (!solicitud.isTieneEstudioMercado()) {
17                return false;
18            }
19            return true;
20        }
21        return super.aprobarCompra(solicitud);
22    }
23 }
```

Al usar este patrón se tendrá completa libertad de configurar la cadena y de cómo se procesan las solicitudes. Además, se extraerá la lógica de cada proceso, cada eslabón tendrá solo una responsabilidad y se podrán incluir nuevos handlers sin que la aplicación se entere ni tenga que ser modificada. Con esto estamos cumpliendo tanto el principio de responsabilidad única como el principio de abierto/cerrado. Para más detalles se puede ver la página 221.

10.4.4. Observer

De vez en cuando nos vamos a encontrar con la necesidad de saber cuándo pasan algunos eventos que pueden ser de interés para otras partes del sistema. Considera el siguiente código:

```
1 public class Main {
2     public static void main(String[] args) {
3         AudioPlayer player = new AudioPlayer();
4
5         player.abrir("c:/audios/audio1.txt");
6
7         player.play();
8         player.cerrar();
9     }
10 }
```

En este caso, estamos creando una clase para reproducir audio, que se lee desde un archivo. Al invocar `play()` se reproduce todo el audio contenido en el archivo.

Tal vez, dentro del diseño de la aplicación, podrían existir otros objetos que estén interesados en algunas situaciones que suceden cuando un audio se reproduce. Por ejemplo, el objeto que controla la pantalla (la interfaz de usuario) podría querer saber cuando un archivo se comienza a reproducir (para por ejemplo, indicar en la pantalla el nombre del archivo siendo reproducido), o a medida que el audio avanza, ir indicándole al usuario lo que sucede, o tal vez registrar en otro archivo en qué posición estamos, ya que si el sistema se cae, después queremos volver al mismo punto donde estábamos.

Hay muchas formas de hacer lo anterior, pero en la mayoría de los casos involucra acoplar el código que realiza las acciones (en este caso, `abrir(...)` o `play()`) con otros objetos para poder “avisarles” que “algo” ha sucedido.

Al contrario, el patrón **Observer** nos dice que podemos realizar lo anterior, pero evitando el acoplar clases, utilizando un esquema de subscripción, donde los objetos que están interesados en ciertos “eventos” (los subscriptores) se registran para poder recibir avisos de cosas interesantes que sucedan desde un “publicador”.

El esquema es bastante simple, y solo requiere que el “publicador” tenga una lista con sus subscriptores, y métodos que permitan agregar y removerlos. Cada vez que algo interesante sucede, el “publicador” revisa su lista de subscriptores e invoca algún método específico para avisarle a sus clientes que ha sucedido algo. La mejor forma de hacer lo anterior es forzando a que todos los subscriptores implementen la misma interfaz, y que ésta incluya métodos apropiados para comunicar los eventos, e información del contexto que pueda ser necesaria para que los clientes le encuentren sentido a dicho evento.

Vamos a modificar el código anterior para agregarle un subscriptor al `AudioPlayer`:

```
1 public class Main {
2     public static void main(String[] args) throws IOException {
3         AudioPlayer player = new AudioPlayer();
4         InterfazUsuario gui = new InterfazUsuario();
5
6         player.subscribe(gui);
7
8         player.abrir("c:/audios/audio1.txt");
9         player.play();
10        player.cerrar();
11    }
12 }
```

La idea de fondo es que siempre que dentro de la ejecución de las tareas del `AudioPlayer` sucede algún evento interesante, todos los subscriptores se enterarán, y harán lo que tengan que hacer, de acuerdo a la misión de cada uno.

Lo único que cada subscriptor debe cumplir es implementar la interfaz apropiada. En este caso, el `AudioPlayer` administra las subscripciones de esta forma:

```
1 public class AudioPlayer
2 {
3     private AudioEventManager aem = new AudioEventManager();
4
5     public void subscribe(AudioEventSubscriber s) { aem.subscribe(s); }
6     public void unsubscribe(AudioEventSubscriber s) { aem.unsubscribe(s); }
7
8     ...
9 }
```

Como se puede ver, le delega el mantenimiento de la lista de subscriptores a una clase aparte:

```
1 public class AudioEventManager
2 {
3     private List<AudioEventSubscriber> subscribers;
4
5     public AudioEventManager() { subscribers = new LinkedList<>(); }
6
7     public void subscribe(AudioEventSubscriber s) { subscribers.add(s); }
8
9     public void unsubscribe(AudioEventSubscriber s) { subscribers.remove(s); }
10
11     public void notifyOpen(String fileName) {
12         subscribers.forEach((x) -> { x.open(fileName); });
13     }
14
15     public void notifyClose() { subscribers.forEach((x) -> { x.close(); }); }
16
17     public void notifyPlay() { subscribers.forEach((x) -> { x.play(); }); }
18
19     public void notifyPlaying(double d){ subscribers.forEach((x) -> { x.playing(d); }); }
20 }
```

Esta clase tiene los métodos `notify*` que permitirán que el `AudioPlayer` notifique a sus subscriptores que algún evento interesante ha pasado.⁶

Es muy importante que cada subscriptor implemente la interfaz adecuada, que hace que todo funcione:

```
1 public interface AudioEventSubscriber
2 {
3     void open(String fileName);
4     void close();
5     void play();
6     void playing(double p);
7 }
```

En el caso de la interfaz de usuario, ésta responde a los eventos de esta forma:

⁶Por si tienes curiosidad de la sintaxis de los `forEach` que aparecen en esos métodos, esa forma de escribir se llama `Java Lambda Expressions`, y en este caso nos permite ahorrar líneas de código para ejecutar algo con cada elemento de la lista.

```

1 public class InterfazUsuario implements AudioEventSubscriber
2 {
3     @Override
4     public void open(String fileName){ System.out.println("GUI: abriendo " + fileName);}
5
6     @Override
7     public void close() { System.out.println("GUI: cerrando"); }
8
9     @Override
10    public void play() { System.out.println("GUI: iniciando play"); }
11
12    @Override
13    public void playing(double p) { System.out.println("GUI: reproduciendo " + p); }
14 }

```

Obviamente esta clase podría encargarse de escribir el estado de la reproducción a una pantalla especial, pero por ahora solo escribirá a la consola normal.

Veamos cómo el `AudioPlayer` hace las notificaciones:

```

1 public class AudioPlayer
2 {
3     private Scanner scanner;
4     private int length;
5     private AudioManager aem = new AudioManager();
6
7     public void abrir(String fileName) throws IOException
8     {
9         aem.notifyOpen(fileName);
10
11         // Acá va la lógica de abrir el archivo de audio
12         length = calcularLongitud(fileName); // calcula la longitud del archivo
13         scanner = new Scanner(new File(fileName));
14     }
15
16     public void cerrar()
17     {
18         aem.notifyClose();
19
20         // Acá va la lógica para cerrar el archivo
21         scanner.close();
22     }
23
24     public void play()
25     {
26         aem.notifyPlay();
27
28         // Acá va la lógica para reproducir el audio.
29         // A medida que el audio se va reproduciendo
30         // se debe notificar el avance del archivo
31         int linea = 0;
32         while (scanner.hasNextLine()) {
33             linea++;
34             String line = scanner.nextLine();
35             SoundDevice.play(line);
36             aem.notifyPlaying(linea / (double) length);
37         }
38     }
39     ...
40 }

```

Si ejecutamos este programa, vamos a obtener algo como esto:

```
GUI: abriendo c:/audios/audio1.txt
GUI: iniciando play
SoundDevice: playing!
GUI: reproduciendo 0.04
SoundDevice: playing!
GUI: reproduciendo 0.08
SoundDevice: playing!
GUI: reproduciendo 0.12
SoundDevice: playing!
GUI: reproduciendo 0.16
SoundDevice: playing!
GUI: reproduciendo 0.2
...
SoundDevice: playing!
GUI: reproduciendo 0.92
SoundDevice: playing!
GUI: reproduciendo 0.96
SoundDevice: playing!
GUI: reproduciendo 1.0
GUI: cerrando
```

Nótese cómo el `AudioPlayer` solo se encarga de reproducir el audio, y al emitir la notificación, la interfaz de usuario recibe el mensaje y hace lo que tiene que hacer.

La belleza del patrón es que sin tener que modificar nada de la clase original, podemos agregar nuevos subscriptores. Por ejemplo, podemos crear esta nueva clase:

```
1 public class PositionRecorder implements AudioEventSubscriber
2 {
3     @Override
4     public void open(String fileName)
5     {
6
7     }
8
9     @Override
10    public void close()
11    {
12
13    }
14
15    @Override
16    public void play()
17    {
18
19    }
20
21    @Override
22    public void playing(double p)
23    {
24        System.out.println("PositionRecorder: Guardando posición " + p);
25    }
26 }
```

Y nuestro código solo requiere registrar el nuevo subscriptor:

```

1 public class Main
2 {
3     public static void main(String[] args) throws IOException
4     {
5         AudioPlayer player = new AudioPlayer();
6
7         InterfazUsuario gui = new InterfazUsuario();
8         PositionRecorder pr = new PositionRecorder();
9
10        player.subscribe(gui);
11        player.subscribe(pr);
12
13        player.abrir("c:/audios/audio1.txt");
14
15        player.play();
16
17        player.cerrar();
18    }
19 }

```

Y seguirá funcionando tal como esperamos:

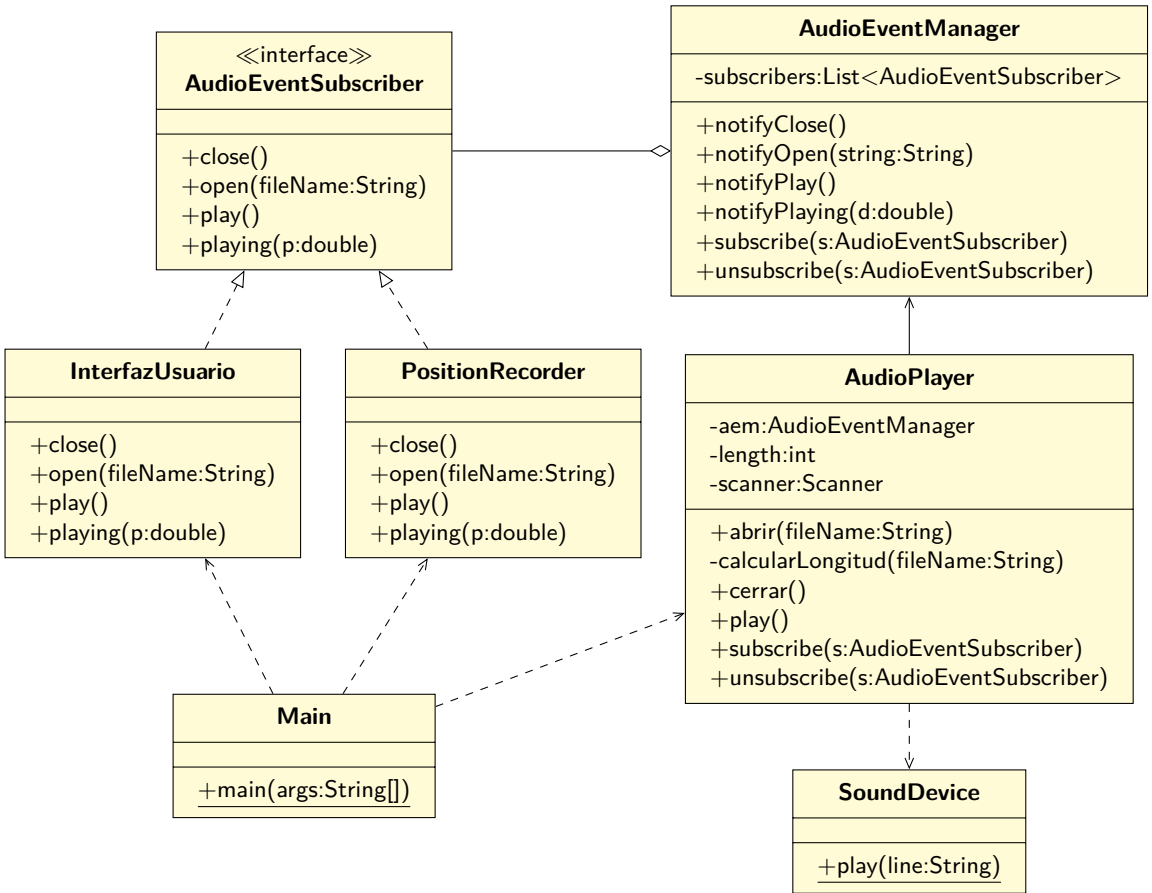
```

GUI: abriendo D:/G/eross/Clases/2022/1/Programación Avanzada 202210/HOWTO.txt
GUI: iniciando play
SoundDevice: playing!
GUI: reproduciendo 0.04
PositionRecorder: Guardando posición 0.04
SoundDevice: playing!
GUI: reproduciendo 0.08
PositionRecorder: Guardando posición 0.08
....
SoundDevice: playing!
GUI: reproduciendo 0.92
PositionRecorder: Guardando posición 0.92
SoundDevice: playing!
GUI: reproduciendo 0.96
PositionRecorder: Guardando posición 0.96
SoundDevice: playing!
GUI: reproduciendo 1.0
PositionRecorder: Guardando posición 1.0
GUI: cerrando

```

Como se puede ver, el patrón `Observer` es muy útil para separar el comportamiento de ejecutar un proceso y observarlo. Ya no es necesario que el objeto que realiza el proceso sepa por adelantado quienes son los interesados, sino que el esquema de subscripción permite desacoplar a los observadores. De hecho, éstos pueden crearse sin que la clase observada tenga conocimiento previo, y se pueden registrar y desregistrar de forma dinámica.

Un ejemplo típico son las interfaces gráficas de usuario: usando un esquema de eventos es posible crear interfaces gráficas desacopladas del proceso que están monitoreando.



10.4.5. Strategy

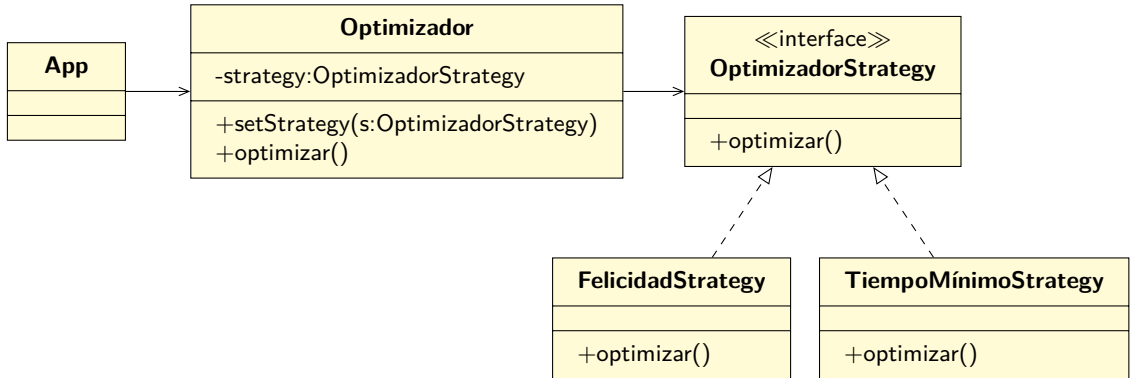
Este patrón nos permitirá tener objetos que definan algoritmos (comportamientos) encapsulados, de forma que posteriormente otra clase los pueda utilizar de forma intercambiable, sin tener que tomar en cuenta las diferencias y particularidades de cada uno.

Imaginemos que estamos creando un sistema para que los estudiantes puedan planificar qué asignaturas tomar en un semestre, de forma de maximizar su felicidad estando en su respectiva carrera de pregrado.

La forma típica de resolver este problema es implementar algún algoritmo que a través de alguna técnica de resolución de problemas, seleccione las asignaturas que maximicen el parámetro buscado. Todo lo anterior es totalmente factible e implementable sin demasiados inconvenientes.

El problema es qué va a pasar cuando se quiera encontrar el grupo de asignaturas que maximice algún otro parámetro, por ejemplo, el menor tiempo total, o permita dejar tiempo libre a las personas, o que solo se puedan tomar grupos de asignaturas que no sobrepasen cierta cantidad de créditos totales, etc. En ese caso, si se quieren incluir todos estos algoritmos dentro de la misma clase que implementaba el algoritmo original, vamos a terminar llenos de `if`, y con una clase que aumenta de tamaño con cada algoritmo nuevo.

El patron **Strategy** nos dice que debemos extraer todos esos algoritmos en clases separadas, llamadas “estrategias”. La clase original, llamada “contexto”, debe tener un atributo para referenciar a una de esas estrategias, de forma que cada vez que tenga que tomar una decisión, le delegue el trabajo en vez de hacerlo ella misma:



En nuestro ejemplo, la **App** instanciará un optimizador y le pedirá que haga su trabajo, usando diferentes estrategias:

```

1 public class App {
2     public void run()
3     {
4         Optimizador op = new Optimizador();
5         List<Asignatura> asignaturas = cargarAsignaturasPosibles();
6
7         op.setStrategy(new FelicidadStrategy());
8         List<Asignatura> seleccionSemestre1 = op.optimizar(asignaturas);
9         print("Felicidad", seleccionSemestre1);
10
11        op.setStrategy(new TiempoMínimoStrategy());
12        List<Asignatura> seleccionSemestre2 = op.optimizar(asignaturas);
13        print("MinTiempo", seleccionSemestre2);
14    }
15
16    private static void print(String txt, List<Asignatura> lista) {
17        System.out.println(txt);
18        for(Asignatura a : lista) {
19            System.out.println(a);
20        }
21        System.out.println("-----");
22    }
23
24    private List<Asignatura> cargarAsignaturasPosibles() {
25        List<Asignatura> lista = new ArrayList<>();
26        lista.add(new Asignatura("A", 6));
27        lista.add(new Asignatura("B", 4));
28        lista.add(new Asignatura("C", 6));
29        lista.add(new Asignatura("D", 6));
30        lista.add(new Asignatura("E", 7));
31        lista.add(new Asignatura("F", 5));
32        lista.add(new Asignatura("G", 5));
33        return lista;
34    }
35 }
  
```

Por su parte, el `Optimizador` no toma ninguna decisión, sino que se la delega a la estrategia de turno:

```

1 public class Optimizador
2 {
3     private OptimizadorStrategy strategy;
4
5     public void setStrategy(OptimizadorStrategy felicidadStrategy)
6     {
7         this.strategy = felicidadStrategy;
8     }
9
10    public List<Asignatura> optimizar(List<Asignatura> asignaturas)
11    {
12        List<Asignatura> seleccion = new LinkedList<>();
13
14        for(Asignatura a : asignaturas) {
15            if (strategy.incluir(a, seleccion)) {
16                seleccion.add(a);
17            }
18        }
19        return seleccion;
20    }
21 }

```

De acuerdo a nuestro diseño, la interfaz de la `Strategy` es así:

```

1 public interface OptimizadorStrategy
2 {
3     /*
4      * Decide si la asignatura indicada puede ser seleccionada, tomando en
5      * cuenta que actualmente hay otras asignaturas ya seleccionadas para el semestre.
6      */
7     boolean incluir(Asignatura posible, List<Asignatura> selecciónActual);
8 }

```

Y una de las estrategias concretas toma las decisiones de esta forma:

```

1 /**
2  * Esta estrategia solo selecciona asignaturas hasta un tope
3  * máximo de 30 créditos, para garantizar la felicidad.
4  */
5 public class FelicidadStrategy implements OptimizadorStrategy
6 {
7     @Override
8     public boolean incluir(Asignatura posible, List<Asignatura> selecciónActual)
9     {
10         int creds = 0;
11         for(Asignatura a : selecciónActual) {
12             creds += a.getCréditos();
13         }
14         int restantes = 30 - creds;
15
16         return posible.getCréditos() <= restantes;
17     }
18 }

```


Mientras que la otra estrategia usa una forma totalmente diferente de tomar decisiones:

```

1  /**
2   * Esta estrategia toma TODAS las asignaturas, para poder terminar
3   * los estudios en un tiempo mínimo.
4   */
5  public class TiempoMínimoStrategy implements OptimizadorStrategy
6  {
7      @Override
8      public boolean incluir(Asignatura posible, List<Asignatura> selecciónActual)
9      {
10         return true;
11     }
12 }

```

Una de las ventajas de usar este patrón es que se puede alterar el comportamiento del contexto en tiempo de ejecución, fácilmente y sin necesidad de llenar el código con condicionales, ni de modificarlo para agregar nuevos algoritmos.

Tal vez la única razón de no usar este patrón es cuando los algoritmos no son muchos, o cambian muy raramente.

10.4.6. Command

Supongamos que tenemos el problema de que el software que estamos creando permite que los usuarios realicen acciones, pero dichas acciones no se realizan de manera instantánea, sino que se pueden almacenar, ejecutar de forma diferida, o incluso ser rechazadas. O por ejemplo, que las acciones se puedan deshacer, y después volver a aplicarlas.

El patrón **Command** nos permite convertir este tipo de acciones en objetos, que conteniendo toda la información acerca de la acción, se pueden pasar como argumentos a métodos, o se pueden almacenar, e incluso se pueden deshacer.

Muchas veces vamos a utilizar esta técnica cuando tengamos aplicaciones interactivas, donde el usuario puede realizar y deshacer acciones, pero también se puede aplicar en otras circunstancias.

Por ejemplo, vamos a simular una aplicación que permite que los usuarios jueguen Sudoku. Cada acción que los usuarios puedan realizar se modelará como una instancia de **Command**. En particular, en esta versión del juego los usuarios solamente pueden poner un número en una celda del tablero, y borrar alguna celda. Obviamente queremos permitir que los usuarios puedan deshacer sus acciones.

Para iniciar el juego, tenemos que crear una **App**:

```

1  public class Main
2  {
3      public static void main(String[] args)
4      {
5          App app = new App();
6          app.run();
7      }
8  }

```

El objeto que mantendrá el estado del juego será muy simple:

```

1 public class Sudoku
2 {
3     private Integer[][] tablero = new Integer[9][9];
4
5     public Sudoku() { /* El tablero se inicializa lleno de nulls por defecto */ }
6
7     // Retorna el elemento que estaba previamente en la celda.
8     // Puede ser null (si es que la celda no estaba ocupada)
9     public Integer ponerNumero(int fila, int col, int num)
10    {
11        Integer prev = tablero[fila][col];
12        tablero[fila][col] = num;
13        return prev;
14    }
15
16    // Retorna el elemento que estaba previamente en la celda.
17    // Puede ser null (si es que la celda no estaba ocupada)
18    public Integer limpiarCelda(int fila, int col)
19    {
20        Integer prev = tablero[fila][col];
21        tablero[fila][col] = null;
22        return prev;
23    }
24
25    // Tal vez sería mejor sacar esto a su propia clase, pero por ahora....
26    public void print()
27    {
28        for(int f=0;f<9;f++) {
29            for(int c=0;c<9;c++) {
30                System.out.print(tablero[f][c] != null ? tablero[f][c] : "X");
31                System.out.print(" ");
32            }
33            System.out.println();
34        }
35        System.out.println("-----");
36    }
37 }

```

Y la aplicación en sí, que administrará el juego, tampoco es tan complicada:

```

1 public class App {
2     private LinkedList<Command> history = new LinkedList<>();
3
4     public void run()
5     {
6         Sudoku sudoku = new Sudoku();
7
8         Jugador jugador = new JugadorSimulado(this, sudoku);
9
10        sudoku.print();
11
12        for(int i=0;i<10;i++) {
13            Command cmd = jugador.nextComando();
14            if (cmd == null) break;
15
16            ejecutar(cmd);
17
18            sudoku.print();
19        }
20    }

```

```

21
22 public void ejecutar(Command cmd)
23 {
24     System.out.println(cmd.getClass().getName());
25     if (cmd.execute()) {
26         history.addFirst(cmd);
27     }
28 }
29
30 public void undo()
31 {
32     if (history.size() == 0) return;
33
34     Command cmd = history.removeFirst();
35     cmd.undo();
36 }
37 }

```

La clase `JugadorSimulado` la usaremos acá para obtener los comandos (las jugadas del usuario), pero a partir de una lista predefinida:

```

1 public class JugadorSimulado implements Jugador
2 {
3     private LinkedList<Command> futuro = new LinkedList<Command>();
4
5     public JugadorSimulado(App app, Sudoku sudoku)
6     {
7         futuro.add(new PonerNumeroCommand(app, sudoku, 0, 0, 1));
8         futuro.add(new PonerNumeroCommand(app, sudoku, 2, 2, 2));
9         futuro.add(new UndoCommand(app, sudoku));
10        futuro.add(new PonerNumeroCommand(app, sudoku, 1, 1, 2));
11        futuro.add(new UndoCommand(app, sudoku));
12        futuro.add(new LimpiarCeldaCommand(app, sudoku, 0, 0));
13        futuro.add(new UndoCommand(app, sudoku));
14    }
15
16    @Override
17    public Command nextComando()
18    {
19        if (futuro.size() == 0) return null;
20        return futuro.removeFirst();
21    }
22 }

```

Cuando nos corresponda conectar el juego al teclado, podemos implementar una nueva clase que implemente la interfaz `Jugador`:

```

1 public interface Jugador
2 {
3     Command nextComando();
4 }

```

De acuerdo a lo que se ve en la clase `App`, simplemente las instancias de `Command` se van ejecutando, y si la ejecución retorna `true`, entonces se almacena el comando en la lista de comandos pasados. De esta forma, si en algún momento se invoca el método `undo()` de la `App`, simplemente se tiene que buscar el último comando invocado, y se llama a su método `undo()`.

Todo esto funciona ya que los comandos contienen todo lo necesario para operar de forma independiente. La clase base de todos los comandos es así:

```

1 public abstract class Command
2 {
3     protected App app;
4     protected Sudoku sudoku;
5
6     protected Command(App app, Sudoku sudoku)
7     {
8         this.app = app;
9         this.sudoku = sudoku;
10    }
11
12    public abstract boolean execute();
13
14    public void undo() {};
15 }

```

Y un comando concreto se implementa de esta forma:

```

1 public class PonerNumeroCommand extends Command
2 {
3     private int fila;
4     private int col;
5     private int num;
6
7     private Integer valorPrevio;
8
9     public PonerNumeroCommand(App app, Sudoku sudoku, int fila, int col, int num)
10    {
11        super(app, sudoku);
12
13        this.app = app;
14        this.fila = fila;
15        this.col = col;
16        this.num = num;
17    }
18
19    public boolean execute()
20    {
21        // Retorna el elemento que previamente estaba en esa celda
22        valorPrevio = sudoku.ponerNumero(fila, col, num);
23        return true;
24    }
25
26    @Override
27    public void undo()
28    {
29        if (valorPrevio != null) {
30            sudoku.ponerNumero(fila, col, valorPrevio);
31        } else {
32            sudoku.limpiarCelda(fila, col);
33        }
34    }
35 }

```

El comando `LimpiarCeldaCommand` es básicamente igual que `PonerNumeroCommand`:

```

1 public class LimpiarCeldaCommand extends Command
2 {
3     private int fila;
4     private int col;
5
6     private Integer valorPrevio;
7
8     public LimpiarCeldaCommand(App app, Sudoku sudoku, int fila, int col)
9     {
10         super(app, sudoku);
11         this.fila = fila;
12         this.col = col;
13     }
14
15     @Override
16     public boolean execute()
17     {
18         valorPrevio = sudoku.limpiarCelda(fila, col);
19         return true;
20     }
21
22     @Override
23     public void undo()
24     {
25         if (valorPrevio != null) {
26             sudoku.ponerNumero(fila, col, valorPrevio);
27         } else {
28             sudoku.limpiarCelda(fila, col);
29         }
30     }
31 }

```

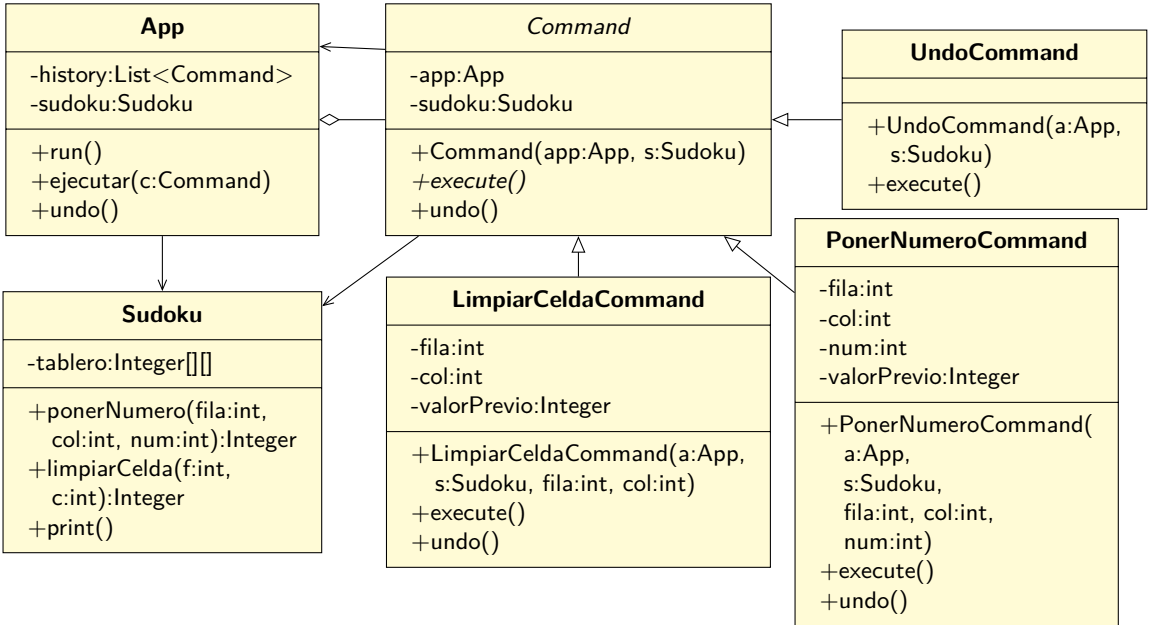
En el caso del comando para deshacer, solo llamamos a `undo()` en la instancia de `App`:

```

1 public class UndoCommand extends Command
2 {
3     protected UndoCommand(App app, Sudoku sudoku) { super(app, sudoku); }
4
5     @Override
6     public boolean execute()
7     {
8         app.undo();
9         return false;
10    }
11 }

```

Como se puede ver en el diagrama, se cumple la condición de que las instancias concretas de `Command` son elementos individuales (cada comando es representado por su propia clase), independientes (cada instancia tiene todo lo necesario para ejecutarse, independiente de los otros comandos), y que pueden almacenarse, ejecutarse y deshacerse de acuerdo a las acciones del usuario:



Hay que notar que para poder implementar la acción de “deshacer” es necesario tener la historia de las acciones realizadas. En este caso, la **App** guarda cada instancia de **Command**, y si es necesario, puede invocar la operación `undo()` para revertir la operación.

También hay que notar que en el caso mostrado, cada comando realizaba la operación, o la deshacía si era necesario. En otras palabras, era como si el usuario mismo realizara la operación. Una implementación alternativa sería guardar el estado interno del objeto (el tablero completo), y restaurarlo cuando sea necesario, pero en este caso dicho estado es privado. El patrón **Memento** puede ayudar en esta situación. Aun así, si se guarda el estado completo del objeto, podemos terminar utilizando mucha memoria, lo que afectará el rendimiento de la aplicación completa.

Un defecto de este patrón es que se agrega una capa nueva de complejidad entre los que envían los mensajes y los receptores. Todo se debe empaquetar en instancias de **Command**, lo que hace que el código sea a veces más verboso y difícil de seguir.

10.4.7. State

Hay veces en que nos conviene modelar nuestros problemas de forma de tener clases que alteren su comportamiento de acuerdo a su estado interno, llegando al punto que parecen que son clases diferentes. En otras palabras, objetos que al enviarles un mensaje responden y realizan acciones dependiendo de su estado interno actual.

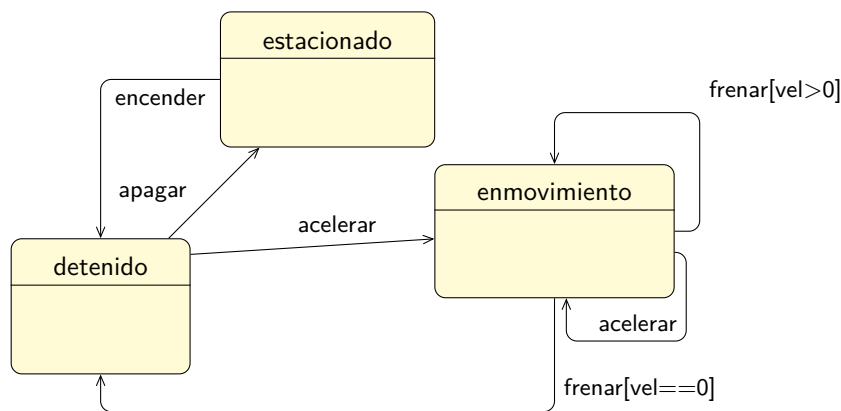
Por ejemplo, si tenemos una clase `Camión` a la que se le puede enviar un mensaje para que acelere, pero la forma de responder al mensaje depende de su estado interno, una forma inocente de realizar ésto es hacer algo así:

```
1 public class CamiónInocente
2 {
3     private String estado;
4
5     public void acelerar()
6     {
7         if (estado.equalsIgnoreCase("estacionado")) {
8             // No hago nada
9         } else if (estado.equalsIgnoreCase("detenido")) {
10            estado = "enmovimiento";
11            incrementarVelocidad();
12        } else if (estado.equalsIgnoreCase("enmovimiento")) {
13            if (getVelocidad() < 100) {
14                incrementarVelocidad();
15            }
16        } else {
17            // No hago nada
18        }
19    }
20
21    ...
22 }
```

En este caso, todo el conocimiento respecto a cómo se procesa un mensaje dependiendo del estado del objeto (al que llamaremos “contexto”) está completamente acoplado en el mismo contexto. Si nuestros estados son pocos, tal vez no sea problema, pero si comenzamos a agregar más estados, incluyendo la lógica de cómo cambia el estado dependiendo de las condiciones del entorno, lo más seguro es que obtendremos código muy difícil de mantener, con poca cohesión, y cualquier modificación podría poner en riesgo a otros estados, incluso estados que pareciera que no tienen relación con lo que se está cambiando.

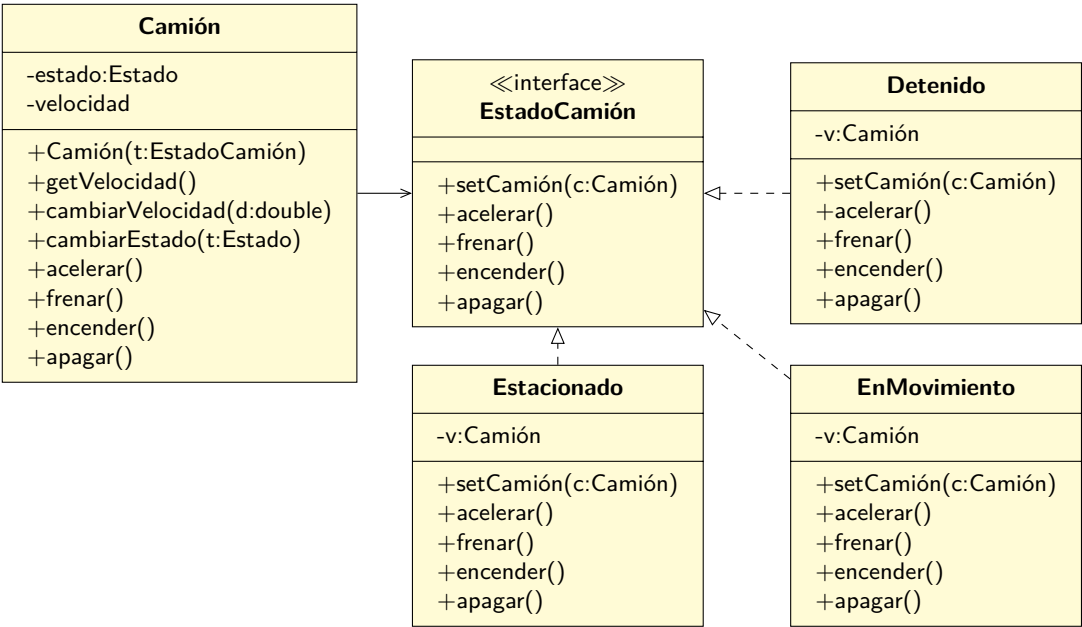
En general, lo que se está tratando de implementar es una forma de tener una máquina de estados que controle el comportamiento de un objeto, pero separando dicha lógica de forma que sea fácilmente entendible y modificable.

Una máquina de estados es una entidad que representa los estados en los que puede estar un objeto, y las condiciones que se deben cumplir para pasar de un estado a otro. Generalmente se dibujan usando UML. En el caso de nuestro `Camión`, este diagrama podría representar sus estados y las acciones que hace que cambie de un estado a otro:



El patrón **Pattern** nos indica que tenemos que crear nuevas clases, una para cada posible estado de nuestro contexto, y que extraigamos todo el comportamiento particular de esos estados y los dejemos en esas clases nuevas.

La idea es que el objeto original (el contexto) tenga una referencia a una instancia de los objetos estado, y le delegue todo el trabajo particular:



Entonces el `Camión` quedaría implementado así:

```

1 public class Camión
2 {
3     private EstadoCamión estado;
4     private double velocidad;
5
6     public Camión(EstadoCamión estado)
7     {
8         cambiarEstado(estado);
9     }
10
11     public void encender() { estado.encender(); }
12     public void apagar() { estado.apagar(); }
13     public void acelerar() { estado.acelerar(); }
14     public void frenar() { estado.frenar(); }
15
16     public void cambiarEstado(EstadoCamión sgte)
17     {
18         this.estado = sgte;
19         estado.setCamión(this);
20     }
21
22     public double getVelocidad() { return velocidad; }
23
24     public void cambiarVelocidad(double delta) { this.velocidad += delta; }
25 }

```

Por otro lado, la interfaz `EstadoCamión` es muy simple:

```

1 public interface EstadoCamión
2 {
3     void setCamión(Camión camión);
4     void encender();
5     void apagar();
6     void acelerar();
7     void frenar();
8 }

```

En el caso del estado `Estacionado`, esta clase solo contiene lo referente a dicho estado, y nada más:

```

1 public class Estacionado implements EstadoCamión {
2     private Camión camión;
3
4     @Override
5     public void setCamión(Camión camión) { this.camión = camión; }
6
7     @Override
8     public void encender()
9     {
10         System.out.println("Estacionado: encender() -> Detenido");
11         camión.cambiarEstado(new Detenido());
12     }
13     @Override
14     public void apagar() { System.out.println("Estacionado: apagar() no hace nada"); }
15     @Override
16     public void acelerar() { System.out.println("Estacionado: acelerar() no hace nada"); }
17     @Override
18     public void frenar() { System.out.println("Estacionado: frenar() no hace nada"); }
19 }

```

Lo mismo sucede con el estado **Detenido**:

```

1 public class Detenido implements EstadoCamión
2 {
3     private Camión camión;
4
5     @Override
6     public void setCamión(Camión camión) { this.camión = camión; }
7
8     @Override
9     public void encender() { System.out.println("Detenido: encender() no hace nada"); }
10
11    @Override
12    public void apagar()
13    {
14        System.out.println("Detenido: apagar() -> Estacionado");
15        camión.cambiarEstado(new Estacionado());
16    }
17
18    @Override
19    public void acelerar()
20    {
21        System.out.println("Detenido: acelerar() -> EnMovimiento");
22        camión.cambiarEstado(new EnMovimiento());
23    }
24
25    @Override
26    public void frenar() { System.out.println("Detenido: frenar() no hace nada"); }
27 }

```

El estado **EnMovimiento** es ligeramente más complicado:

```

1 public class EnMovimiento implements EstadoCamión {
2     private Camión camión;
3
4     @Override
5     public void setCamión(Camión camión) { this.camión = camión; }
6
7     @Override
8     public void encender(){System.out.println("EnMovimiento: encender() no hace nada");}
9     @Override
10    public void apagar() { System.out.println("EnMovimiento: apagar() no hace nada"); }
11
12    @Override
13    public void acelerar() {
14        System.out.println("EnMovimiento: acelerar()");
15        camión.cambiarVelocidad(1);
16    }
17
18    @Override
19    public void frenar()
20    {
21        camión.cambiarVelocidad(-1);
22
23        if (camión.getVelocidad() <= 0) {
24            System.out.println("EnMovimiento: frenar() -> Detenido");
25            camión.cambiarEstado(new Detenido());
26        } else {
27            System.out.println("EnMovimiento: frenar()");
28        }
29    }
30 }

```

Finalmente, podemos poner todo a prueba con este programa:

```

1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         EstadoCamión inicial = new Estacionado();
6         Camión c = new Camión(inicial);
7
8         Random r = new Random();
9
10        for(int i=0;i<10;i++) {
11            int n = r.nextInt(4);
12
13            switch(n) {
14                case 0:
15                    c.encender();
16                    break;
17                case 1:
18                    c.apagar();
19                    break;
20                case 2:
21                    c.acelerar();
22                    break;
23                case 3:
24                    c.frenar();
25                    break;
26            }
27        }
28    }
29 }

```

Al ejecutar lo anterior, una salida posible es la siguiente:

```

Estacionado: encender() -> Detenido
Detenido: apagar() -> Estacionado
Estacionado: apagar() no hace nada
Estacionado: encender() -> Detenido
Detenido: frenar() no hace nada
Detenido: frenar() no hace nada
Detenido: acelerar() -> EnMovimiento
EnMovimiento: acelerar()
EnMovimiento: frenar() -> Detenido
Detenido: acelerar() -> EnMovimiento

```

En general es buena idea aplicar este patrón casi en cualquier caso. La única excepción sería cuando la cantidad de estados es muy baja.

El patrón permite minimizar la posibilidad de errores, y será muy fácil agregar nuevo comportamiento en la forma de nuevos estados, ya que el código será más robusto y flexible, evitando usar lógica basada en `if` y `switch`.

10.4.8. Visitor

Imagina que estamos creando un editor de figuras, que tiene una clase principal (el editor), pero además una cantidad importante de “tipos” de figuras que se pueden editar (cuadrados, rectángulos, círculos, elipses, estrellas, espirales, etc.).

Imaginemos también que como parte del diseño, vamos a requerir guardar el contenido del editor a un archivo, por lo que hay que decidir cómo incluir ese requerimiento en la estructura del programa.

Una forma de implementarlo es simplemente agregarle un método a la clase del editor, y métodos correspondientes a cada tipo de figura, ya que cada figura se guarda de forma diferente. El problema de este enfoque es que le tendríamos que agregar a cada clase código y comportamiento que en el fondo no tiene que ver con su misión: las figuras existen para comportarse como figuras, y no necesariamente para saber cómo almacenarse en un archivo. Además, la posibilidad de tener que implementar otros métodos de almacenamiento (por ejemplo, otros formatos de archivos) nos forzaría a agregar más lógica ajena a las figuras.

Para solucionar todos los problemas anteriores podemos aplicar el patrón **Visitor**, que nos dice que podemos escribir el comportamiento nuevo en clases apartes (llamadas visitantes) en vez de escribir todo en las clases originales (las figuras en nuestro caso), de forma que el objeto sea pasado por parámetro a métodos definidos en el **Visitor**.

Por ejemplo, en nuestro caso, se podría definir un **Visitor** con la siguiente estructura:

```

1 public class TxtFileExportVisitor
2 {
3     public void visitar(Cuadrado c)
4     {
5     }
6     public void visitar(Circulo c)
7     {
8     }
9     public void visitar(Estrella c)
10    {
11    }
12    // ... otras figuras
13 }
```

Usando la clase anterior podemos implementar el comportamiento particular que requiere cada tipo de **Figura** para ser guardada en el archivo. Pero ¿cómo conectamos esa clase a las figuras?

El patrón nos dice que usemos la técnica del despacho doble⁷, para hacer que las clases ejecuten el método correcto de **TxtFileExportVisitor**. Por ejemplo:

```

1 public class Estrella
2 {
3     void accept(TxtFileExportVisitor visitor)
4     {
5         visitor.visitar(this);
6     }
7 }
```

Pero para que todo tenga más sentido, reestructuremos las clases: primero definamos una interfaz que será la base de los visitantes de **Figura**:

⁷Double Dispatch

```

1 public interface FiguraVisitor
2 {
3     void visitar(Cuadrado c);
4     void visitar(Circulo c);
5     void visitar(Estrella c);
6     // ... otras figuras
7 }

```

Y el `TxtFileExportVisitor` ahora implementa dicha interfaz:

```

1 public class TxtFileExportVisitor implements FiguraVisitor
2 {
3     @Override
4     public void visitar(Cuadrado c)
5     {
6     }
7     @Override
8     public void visitar(Circulo c)
9     {
10    }
11    @Override
12    public void visitar(Estrella c)
13    {
14    }
15    // ... otras figuras
16 }

```

Además, a la clase base de todas las `Figuras` se le agrega la capacidad de ser visitada:

```

1 public abstract class Figura
2 {
3     abstract void accept(FiguraVisitor visitor);
4 }

```

Y, por ejemplo, la clase `Estrella` cambia para respetar a su super clase `Figura`:

```

1 public class Estrella extends Figura
2 {
3     @Override
4     public void accept(FiguraVisitor visitor)
5     {
6         visitor.visitar(this);
7     }
8 }

```

Y todo converge para que cuando tengamos que guardar todas las figuras a un archivo, el `EditorFiguras` realice lo siguiente:

```

1 public class EditorFiguras
2 {
3     private List<Figura> figuras = new LinkedList<>();
4
5     public void guardar()
6     {
7         FiguraVisitor saver = new TxtFileExportVisitor();
8
9         for(Figura f : figuras) {
10             f.accept(saver);
11         }
12     }
13 }

```

La belleza del patrón es que si se requiere implementar un nuevo formato de archivo, basta crear una nueva clase concreta que implemente el nuevo comportamiento:

```

1 public class JSONFileExportVisitor implements FiguraVisitor
2 {
3     @Override
4     public void visitar(Cuadrado c)
5     {
6         // Código para guardar un cuadrado
7     }
8     @Override
9     public void visitar(Círculo c)
10    {
11        // Código para guardar un círculo
12    }
13    @Override
14    public void visitar(Estrella c)
15    {
16        // Código para guardar una estrella
17    }
18    // ... otras figuras
19 }

```

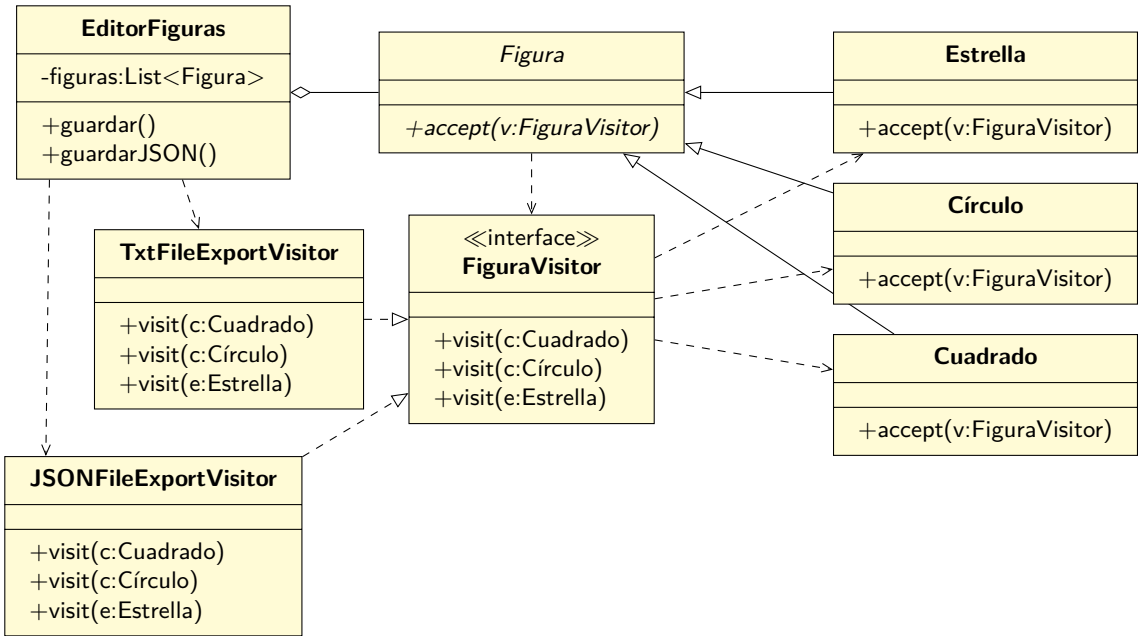
Y no se tendrá que modificar ninguna `Figura`, solo el `EditorFiguras`:

```

1 public class EditorFiguras
2 {
3     private List<Figura> figuras = new LinkedList<>();
4
5     public void guardar()
6     {
7         FiguraVisitor saver = new TxtFileExportVisitor();
8
9         for(Figura f : figuras) {
10             f.accept(saver);
11         }
12     }
13
14     public void guardarJSON()
15     {
16         FiguraVisitor saver = new JSONFileExportVisitor();
17
18         for(Figura f : figuras) {
19             f.accept(saver);
20         }
21     }
22 }

```

El diagrama de esta implementación será:



El usar este patrón nos ayudará a extender el comportamiento de ciertas clases, agregando operaciones que aunque no deben estar incluidas en las clases en sí, se deben agregar y mantener separadas. Al implementar el patrón se pueden crear muchas versiones del visitador, una para cada comportamiento que se quiera agregar a las clases originales.

10.4.9. Interpreter

Este patrón nos permite definir una representación gramatical de un lenguaje, y proveer un intérprete para manejar dicha gramática.

10.4.10. Iterator

Si alguna vez necesitamos una forma de recorrer una colección de elementos, sin exponer su representación interna, podemos usar el patrón Iterator.

Esta técnica ya se aplicó con anterioridad en la página 151.

10.4.11. Memento

Existen muchas razones por las que el estado de un objeto puede cambiar, y por lo mismo, a veces sucede que se requiere guardar dicho estado para posteriormente restaurarlo. Hay muchas formas de hacer esto, pero si queremos realizarlo de forma de no aumentar el acoplamiento entre nuestras clases, podemos usar el patrón **Memento**.

El patrón **Memento** nos dice que necesitamos dos tipos de objetos para realizar lo anterior:

Originator es el objeto cuyo estado se necesita poder guardar y después restaurar. Para realizar esta acción será necesario tener un método que retornará una instancia de una clase que llamaremos **Memento**, y que en Java será una clase interna al **Originator**.

Caretaker es una clase que nos ayudará a almacenar y restaurar el estado del **Originator** usando las instancias de **Memento**. Como las instancias de **Memento** son privadas al **Originator**, el **Caretaker** no podrá ver lo que tienen adentro.

Uno de los mejores ejemplos para demostrar este patrón es un editor (por ejemplo, de figuras geométricas), donde se pueda guardar su estado en cualquier momento y después restaurarlo.

En primer lugar, definiremos la clase base para todos los **Mementos**:

```
1 public abstract class Memento
2 {
3     public abstract void restaurar();
4 }
```

Con el **Memento** definido, podemos implementar un editor de figuras. Como se dijo anteriormente usaremos una clase interna para guardar el estado del editor:

```
1 public class EditorFiguras
2 {
3     private List<Figura> figuras = new LinkedList<Figura>();
4     private List<Figura> selecciónActual = new LinkedList<Figura>();
5     private String colorActual = null;
6
7     public void agregar(Figura f) { this.figuras.add(f); }
8     public void setColor(String color) { this.colorActual = color; }
9
10    public void seleccionarFigura(Figura f)
11    {
12        if (f == null) {
13            selecciónActual.clear();
14        } else {
15            selecciónActual.add(f);
16        }
17    }
18
19    public Memento crearSnapshot() { return new EditorFigurasMemento(this); }
20
21    private class EditorFigurasMemento extends Memento
22    {
23        private EditorFiguras editor;
24        private List<Figura> figuras = new LinkedList<Figura>();
25        private List<Figura> selecciónActual = new LinkedList<Figura>();
26        private String colorActual = null;
```



```

27
28     public EditorFigurasMemento(EditorFiguras editor)
29     {
30         // OJO que no podemos simplemente apuntar a la lista
31         // del editor, ya que el objeto referenciado va a
32         // cambiar. Tenemos que sacarle una "fotografía" a
33         // estas listas!
34         this.figuras = new LinkedList<Figura>(editor.figuras);
35         this.selecciónActual = new LinkedList<Figura>(editor.selecciónActual);
36         this.colorActual = editor.colorActual;
37         this.editor = editor;
38     }
39
40     @Override
41     public void restaurar()
42     {
43         this.editor.colorActual = this.colorActual;
44
45         this.editor.selecciónActual.clear();
46         this.editor.selecciónActual.addAll(this.selecciónActual);
47
48         this.editor.figuras.clear();
49         this.editor.figuras.addAll(this.figuras);
50     }
51 }
52
53 @Override
54 public String toString()
55 {
56     return "figuras=" + figuras + ", \nselecciónActual=" + selecciónActual +
57         ", \ncolorActual=" + colorActual + "";
58 }
59 }

```

La **Figura** es una clase muy simple para representar a una figura siendo editada:

```

1 public class Figura
2 {
3     enum Tipo { CUADRADO, CÍRCULO }
4
5     private Tipo tipo;
6     private String nombre;
7
8     public Figura(Tipo tipo, String nombre)
9     {
10         this.tipo = tipo;
11         this.nombre = nombre;
12     }
13
14     @Override
15     public String toString() { return "" + tipo + "/" + nombre + ""; }
16 }

```

El **EditorFigurasCaretaker** es la clase que administrará la historia del **EditorFiguras**:

```

1 public class EditorFigurasCaretaker
2 {
3     private EditorFiguras originador;
4     private LinkedList<Memento> history;
5
6     public EditorFigurasCaretaker(EditorFiguras originador)

```

```

7  {
8      this.originador = originador;
9      this.history = new LinkedList<Memento>();
10 }
11 public void snapshot()
12 {
13     System.out.println("* snapshot()");
14     Memento snapshot = originador.crearSnapshot();
15     history.addFirst(snapshot);
16 }
17 public void deshacer()
18 {
19     System.out.println("* deshacer()");
20     Memento memento = history.removeFirst();
21     memento.restaurar();
22 }
23 }

```

Para poner todo a prueba crearemos un `Main` ligeramente extenso, para realizar varias acciones y ver cómo se comporta el `Caretaker`:

```

1  public class Main
2  {
3      public static void main(String[] args)
4      {
5          EditorFiguras editor = new EditorFiguras();
6          EditorFigurasCaretaker ct = new EditorFigurasCaretaker(editor);
7
8          System.out.println(editor);
9          System.out.println("-----");
10         ct.snapshot();
11
12         Figura f1 = new Figura(Figura.Tipo.CUADRADO, "cuad1");
13         Figura f2 = new Figura(Figura.Tipo.CÍRCULO, "circ1");
14
15         editor.setColor("ROJO");
16         editor.agregar(f1);
17         editor.agregar(f2);
18         editor.seleccionarFigura(f2);
19
20         System.out.println(editor);
21         System.out.println("-----");
22         ct.snapshot();
23
24         editor.setColor("NEGRO");
25         System.out.println(editor);
26         System.out.println("-----");
27         ct.snapshot();
28
29         editor.seleccionarFigura(null);
30         editor.seleccionarFigura(f1);
31         editor.agregar(new Figura(Figura.Tipo.CÍRCULO, "circ2"));
32         System.out.println(editor);
33         System.out.println("-----");
34
35         ct.deshacer();
36         System.out.println(editor);
37         System.out.println("-----");
38
39         ct.deshacer();
40         System.out.println(editor);

```

```

41         System.out.println("-----");
42
43         ct.deshacer();
44         System.out.println(editor);
45         System.out.println("-----");
46     }
47 }

```

Al ejecutar lo anterior se obtiene la siguiente salida:

```

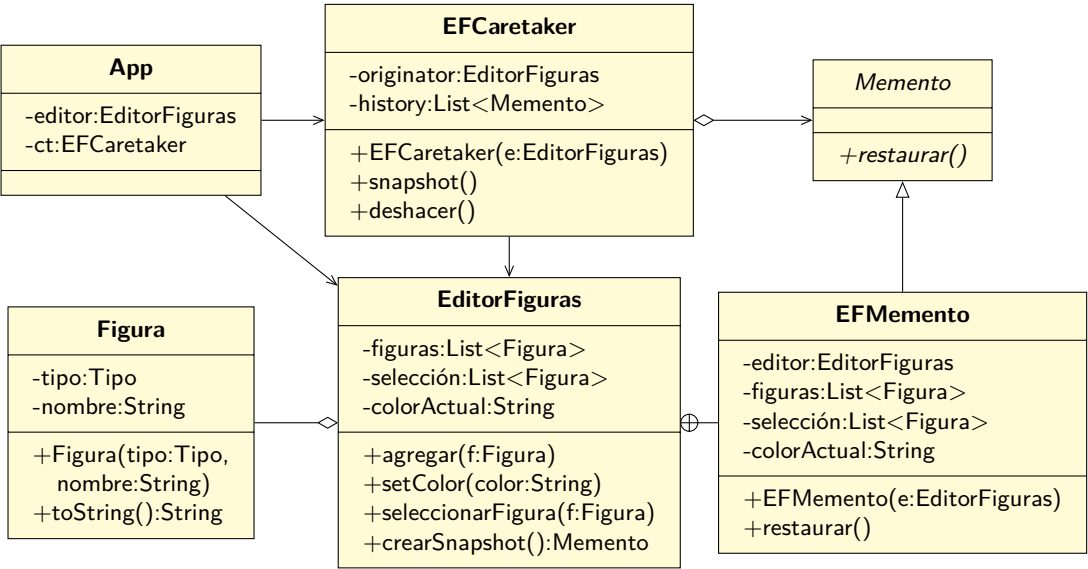
figuras=[],
selecciónActual=[],
colorActual=null
-----
* snapshot()
figuras=[CUADRADO/cuad1, CÍRCULO/circ1],
selecciónActual=[CÍRCULO/circ1],
colorActual=ROJO
-----
* snapshot()
figuras=[CUADRADO/cuad1, CÍRCULO/circ1],
selecciónActual=[CÍRCULO/circ1],
colorActual=NEGRO
-----
* snapshot()
figuras=[CUADRADO/cuad1, CÍRCULO/circ1, CÍRCULO/circ2],
selecciónActual=[CUADRADO/cuad1],
colorActual=NEGRO
-----
* deshacer()
figuras=[CUADRADO/cuad1, CÍRCULO/circ1],
selecciónActual=[CÍRCULO/circ1],
colorActual=NEGRO
-----
* deshacer()
figuras=[CUADRADO/cuad1, CÍRCULO/circ1],
selecciónActual=[CÍRCULO/circ1],
colorActual=ROJO
-----
* deshacer()
figuras=[],
selecciónActual=[],
colorActual=null
-----

```

Como se puede ver, se ha podido guardar completamente el estado interno de un objeto sin necesidad abrir sus secretos. Es el mismo objeto el encargado de crear los mementos y restaurarlos, de forma segura y privada.

Un punto a considerar al aplicar este patrón es que al estar duplicando muchas veces el estado interno de los objetos, puede que el consumo de memoria aumente de forma descontrolada si se crean muchos mementos de forma frecuente. Además, se tiene que tener cuidado de destruir los mementos que ya no se usan.

El diagrama de clases de nuestra implementación sería:



Nótese el símbolo entre `EditorFiguras` y `EFMemento`: dicho símbolo indica que `EFMemento` es una clase anidada dentro de `EditorFiguras`.

11

Otros ejemplos completos

Acá vamos a desarrollar algunos ejemplos aplicando todo lo visto hasta ahora, incluyendo abstracciones, polimorfismo, interfaces, la arquitectura propuesta, etc.

11.1. Problema 1

Existe una empresa que tiene la necesidad de ordenar sus costos respecto al recurso personas, y cómo se administran los proyectos en los que la empresa participa.

Dicha información está guardada en diversos archivos, que será necesario procesar para entregar los reportes que se necesitan.

En la empresa todas las personas que trabajan en el área de desarrollo se clasifican en una de tres especialidades: front-end, back-end, o devops.

Todas las personas tienen datos en común, como el RUT, nombre, dirección, sueldo base. Además, si es front-end se tiene información acerca del lenguaje de programación con el que tiene más experiencia (sólo uno), las horas extras trabajadas a la fecha y el nivel de conocimiento que tiene (4: experto, 3: avanzado, 2: intermedio, 1: rookie). De las personas back-end se tiene información de sus años de experiencia y de los devops se sabe qué sistemas operativos domina y las bases de datos que conoce.

La empresa desarrolla proyectos para otras empresas, y en cada uno de estos proyectos participa siempre 1 front-end, 1 back-end y 1 devops. De cada proyecto se registra el nombre, el código, la duración en meses y su costo total.

Respecto a la forma en que están almacenados los datos, se sabe que:

- Se tiene un archivo con los datos de todos los proyectos.
- Se tiene un archivo con los datos de todos los funcionarios, donde en cada registro viene la información de una persona.
- Se tiene un archivo con la asignación de las personas a los proyectos. Por cada registro viene el

código del proyecto y el rut de la persona.

El objetivo de este programa es procesar los datos, y entregar la siguiente información:

- El listado de proyectos con los costos involucrados (por meses y total).
- El listado de personas, con sus respectivos sueldos.
- Dado un proyecto, entregar el listado de las personas involucradas.
- Dado una persona, entregar el listado de proyectos en que participa.
- Para cada persona devops, su nombre y los sistemas operativos que domina.

Los sueldos se calculan de la sigte forma: para el front-end, partiendo con su sueldo base, cada hora extra realizada le abona \$5000, además de un bono extra de \$30000 por nivel y otro bono de 20 % del valor total mensual del proyecto por cada proyecto en el que participa. En el caso del back-end, su sueldo se calcula con el sueldo base más un bono de \$5000 por cada año de experiencia más el 25 % del valor total mensual del proyecto por cada proyecto en el que participa. Finalmente para un devops su sueldo es el sueldo base, más \$8000 por cada base de datos que conoce, más un 30 % del valor total mensual del proyecto por cada proyecto en el que participa.

11.1.1. Iniciando el análisis

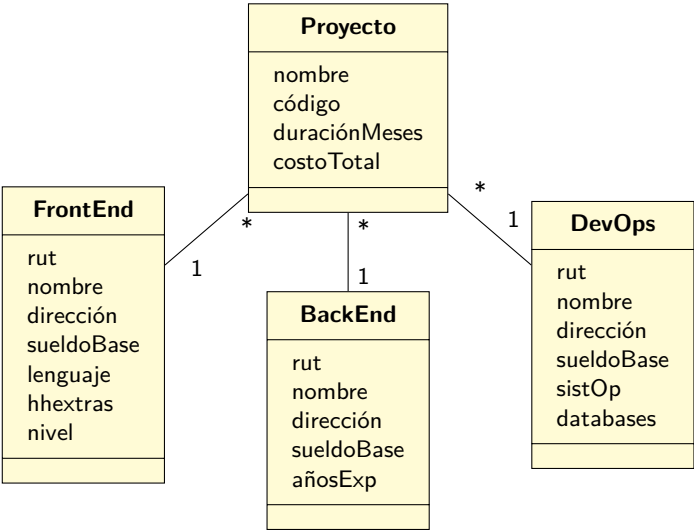
Como en todo problema, debemos iniciar el proceso analizando lo que sabemos del problema, y lo documentaremos usando un modelo del dominio (3.2).

Sabemos que en la empresa, en el área de interés, trabajan personas, y que las personas se relacionan con proyectos:



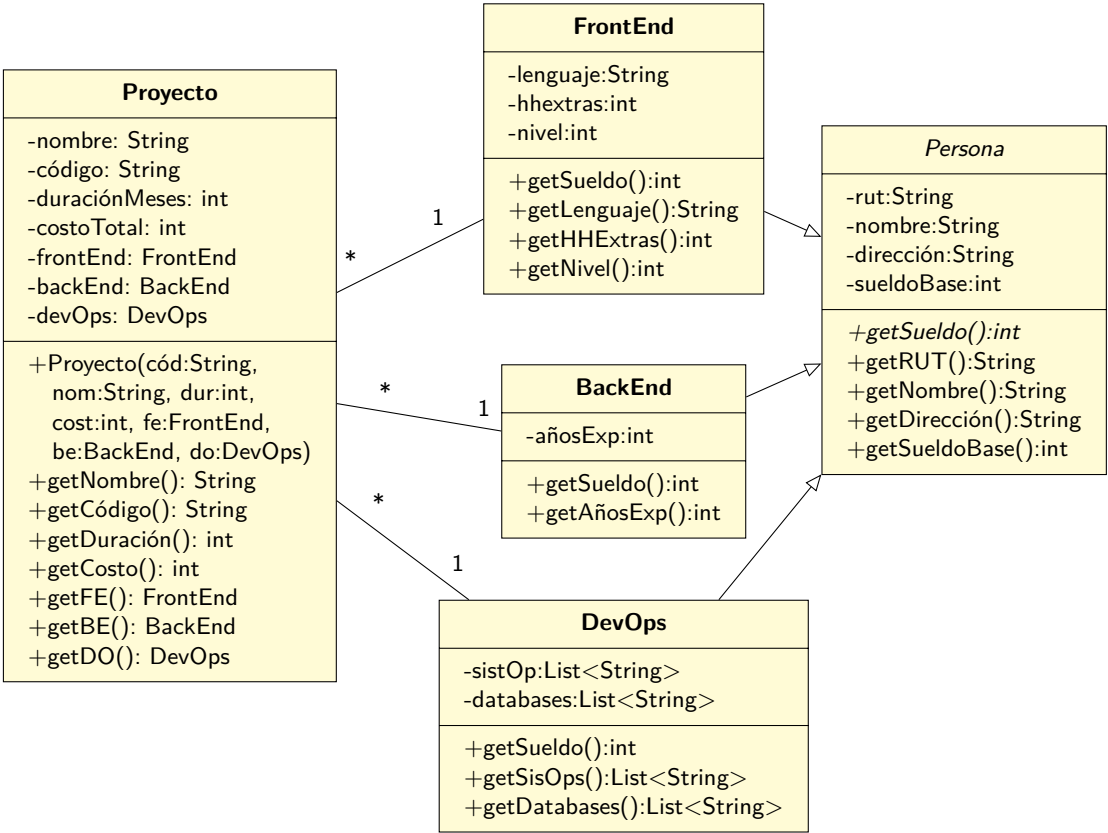
Pero además, sabemos que las personas tienen diferentes especialidades, y sobretodo, que las diferentes especialidades poseen diferentes atributos. Además, nos dicen explícitamente que un proyecto tiene una persona de cada especialidad (lo que se hace difícil de representar con el diagrama anterior).

Así que tenemos que actualizar el diagrama:



En este punto, podemos darnos cuenta de dos situaciones: hay elementos comunes entre los 3 tipos de especialistas, y además hay operaciones que son equivalentes entre los especialistas, en este caso, lo más importante es el cálculo del sueldo. Todos los especialistas tienen un sueldo, pero en cada caso se calcula de forma ligeramente diferente.

Lo anterior nos lleva a pensar que puede haber una relación de herencia entre las especialidades, así que vamos a avanzar el análisis, y empezar a modelar las clases que se han descubierto hasta ahora, incluyendo atributos y operaciones:



Antes de seguir avanzando, falta analizar un detalle que aparece nombrado en el texto: el cálculo del sueldo de cada especialidad. En la clase abstracta **Persona**¹ se ha declarado un método abstracto **getSueldo()**, que debería retornar el sueldo específico de cada especialidad. Al estar declarado así, se fuerza a que las clases derivadas tengan obligatoriamente que definir el método. Lo que sucederá es que cada clase (**FrontEnd**, **BackEnd**, **DevOps**) implementará dicho método de acuerdo a las reglas particulares de cálculo de sueldo de cada una.

11.1.2. Diseñando la solución

En este punto, nos conviene empezar a pensar cómo vamos a crear todas las instancias que participarán en el software, y cómo vamos a entregar la información que se espera. En ese sentido, se requiere de una entidad que orqueste todo el proceso, de manera que le podamos decir que se debe crear un proyecto, o una persona con cierta especialidad, y que mantenga esa información referenciada. Esto último es importante, ya que si miramos el diagrama de clases anterior, todo está “en el aire”: no hay nada que nos permita almacenar los proyectos (mejor dicho, las instancias de la clase **Proyecto** que se crean), y si hay **Personas** que no están asignadas a proyectos, éstas no tienen lugar para almacenarse.

¹La clase **Persona** se define como abstracta ya que es simplemente la base para la existencia de las otras clases. Además, dicha clase no debe ser instanciable, ya que en el problema no participan **Personas**, sino que los tipos derivados concretos.

De acuerdo a la arquitectura que vamos a usar (tal como se presentó en 9.4), crearemos primero una interfaz que contenga todas las operaciones necesarias para solucionar el problema. Esto significa que no solo definiremos el nombre de dichas operaciones, sino que su contratos completos.

De acuerdo a lo especificado, podemos pensar en diseñar la interfaz `Sistema` para que contenga las siguientes operaciones:

```

1 /**
2  * Agrega un nuevo proyecto a la lista general de proyectos
3  *
4  * POST: - Se crea un nuevo proyecto y se agrega a la lista general de proyectos
5  * del sistema
6  *
7  * @param código El código del nuevo proyecto a ingresar. No puede ser null ni
8  *             vacío. No puede estar repetido.
9  * @param nombre El nombre del nuevo proyecto a ingresar. No puede ser null ni
10 *             vacío.
11 * @param monto El monto del nuevo proyecto a ingresar. Tiene que ser un valor
12 *             mayor que cero
13 * @param meses La duración en meses del nuevo proyecto a ingresar. Tiene que
14 *             ser un valor mayor que cero
15 *
16 * @throws SistemaException Cuando no se pudo ingresar el nuevo proyecto por
17 *             problemas con los datos
18 */
19 void ingresarProyecto(String código, String nombre, int monto, int meses)
20     throws SistemaException;

```

```

1 /**
2  * Agrega una nueva persona con especialidad FrontEnd a la lista de personas del
3  * sistema.
4  *
5  * @param rut El RUT de la nueva persona front-end. No puede ser
6  *            null ni vacío. No puede estar repetido.
7  * @param nombre El nombre de la nueva persona front-end. No puede ser
8  *            null ni vacío.
9  * @param dirección La dirección de la nueva persona front-end. No puede
10 *            ser null ni vacío.
11 * @param sueldoBase El sueldo base de la nueva persona front-end. Tiene
12 *            que ser un valor mayor que cero
13 * @param añosExperiencia Los años de experiencia de la nueva persona front-end.
14 *            Tiene que ser un valor mayor que cero
15 *
16 * @throws SistemaException Cuando no se pudo ingresar el nuevo proyecto por
17 *            problemas con los datos
18 */
19 void ingresarBackEnd(String rut, String nombre, String dirección, int sueldoBase,
20     int añosExperiencia) throws SistemaException;

```

```

1 /**
2  * Agrega una nueva persona con especialidad BackEnd a la lista de personas del
3  * sistema.
4  *
5  * @param rut           El RUT de la nueva persona front-end. No puede ser
6  *                     null ni vacío.
7  * @param nombre        El nombre de la nueva persona front-end. No puede ser
8  *                     null ni vacío.
9  * @param dirección     La dirección de la nueva persona front-end. No puede
10 *                     ser null ni vacío.
11 * @param sueldoBase    El sueldo base de la nueva persona front-end. Tiene
12 *                     que ser un valor mayor que cero
13 * @param lenguajeProgramación El lenguaje de programación en el que tiene
14 *                     más experiencia. No puede ser null ni vacío.
15 *
16 * @param horasExtras   La cantidad de horas extras registradas para la persona.
17 *                     Puede ser cero o un número positivo.
18 * @param nivel          El nivel de la persona. Es un número entre 1 y 4 (inclusive)
19 *
20 * @throws SistemaException Cuando no se pudo ingresar el nuevo proyecto por
21 *                     problemas con los datos
22 */
23 void ingresarFrontEnd(String rut, String nombre, String dirección, int sueldoBase,
24                      String lenguajeProgramación, int horasExtras, int nivel)
25     throws SistemaException;

```

```

1 /**
2  * Agrega una nueva persona con especialidad DevOps a la lista de personas del
3  * sistema.
4  *
5  * @param rut           El RUT de la nueva persona front-end. No puede ser
6  *                     null ni vacío.
7  * @param nombre        El nombre de la nueva persona front-end. No puede ser
8  *                     null ni vacío.
9  * @param dirección     La dirección de la nueva persona front-end. No puede
10 *                     ser null ni vacío.
11 * @param sueldoBase    El sueldo base de la nueva persona front-end. Tiene
12 *                     que ser un valor mayor que cero
13 * @param sistemasOp    Los sistemas operativos donde la persona tiene experiencia.
14 *                     El string debe contener los valores separados por coma. No
15 *                     puede
16 *                     ser null.
17 * @param basesDatos    Las bases de datos donde la persona tiene experiencia.
18 *                     El string debe contener los valores separados por coma.
19 *                     No puede ser null.
20 *
21 * @throws SistemaException Cuando no se pudo ingresar el nuevo proyecto por
22 *                     problemas con los datos
23 */
24 void ingresarDevOps(String rut, String nombre, String dirección, int sueldoBase,
25                    String sistemasOp, String basesDatos)
26     throws SistemaException;

```

```

1 /**
2  * Asocia una persona a un proyecto
3  *
4  * PRE: - El proyecto existe - La persona existe
5  *
6  * POST: - El proyecto se ingresa a la lista de proyectos de la persona - La
7  * persona queda asociada al proyecto.
8  *
9  * @param códigoProyecto El código del proyecto a asociar. Debe corresponder a un
10 * proyecto previamente ingresado. No puede ser null.
11 * @param rutPersona      El RUT de la persona a asociar. Debe corresponder a una
12 * persona previamente ingresada. No puede ser null.
13 * @throws SistemaException Cuando el proyecto indicado, o la persona indicada
14 *                          no existen.
15 */
16 void asociarPersonaProyecto(String códigoProyecto, String rutPersona)
17     throws SistemaException;

```

```

1 /**
2  * Retorna los datos de todos los proyectos, incluyendo costo total y mensual.
3  */
4 String obtenerDatosProyectos();
5
6 /**
7  * Retorna los datos de todas las personas, incluyendo su sueldo.
8  */
9 String obtenerDatosPersonas();
10
11 /**
12 * Retorna los datos de las personas asociadas a un proyecto
13 *
14 * @param códigoProyecto El código del proyecto donde se buscarán las personas.
15 * @throws SistemaException Cuando no existe un proyecto con el código dado.
16 */
17 String obtenerDatosPersonasProyecto(String códigoProyecto)
18     throws SistemaException;
19
20 /**
21 * Retorna los datos de los proyectos asociados a una persona
22 *
23 * @param rutPersona El RUT de la persona a la que se listarán sus proyectos.
24 * @throws SistemaException Cuando no existe una persona con el RUT dado.
25 */
26 String obtenerDatosProyectosPersona(String rutPersona)
27     throws SistemaException;
28
29 /**
30 * Retorna los datos de las personas con especialidad DevOps.
31 */
32 String obtenerDatosDevOps();

```

11.1.3. Implementando la solución

Respecto a la implementación de cada operación en la clase concreta `SistemaImpl`, la mayoría de los métodos son simples:

Operación ingresarProyecto

```

1 public void ingresarProyecto(String código, String nombre, int monto, int meses)
2     throws SistemaException
3 {
4     if (buscarProyecto(código) != null)
5         throw new SistemaException("Proyecto " + código + " ya existe");
6
7     fallarSiNullVacío(código, "Código");
8     fallarSiNullVacío(nombre, "Nombre");
9
10    if (monto <= 0) throw new SistemaException("Monto debe ser mayor que CERO");
11    if (meses <= 0) throw new SistemaException("Meses debe ser mayor que CERO");
12
13    Proyecto proyecto = new Proyecto(código, nombre, monto, meses);
14    proyectos.add(proyecto);
15 }

```

Para facilitarnos la implementación, se han creado dos métodos auxiliares:

```

1 /**
2  * Retorna la instancia de proyecto que tenga el código de proyecto
3  * indicado.
4  *
5  * @param códigoProyecto El código del proyecto buscado
6  * @return Una instancia de Proyecto, o null si el código no representa
7  * o ningún proyecto.
8  */
9 private Proyecto buscarProyecto(String códigoProyecto)
10 {
11     for(Proyecto p : proyectos) {
12         if (p.getCódigo().equals(códigoProyecto)) return p;
13     }
14     return null;
15 }

```

Y un método que parece inútil, pero permitirá facilitar la verificación de los parámetros de tipo **String**:

```

1 /**
2  * Verifica que el valor del String no es null ni que es un String
3  * vacío.
4  *
5  * En caso de que así sea, se lanza una excepción, que contiene el
6  * valor de la etiqueta.
7  *
8  * @param valor El valor que se quiere verificar para asegurarnos de que
9  * no es null ni vacío.
10  * @param etiqueta En caso de que no pase la verificación, la etiqueta
11  * permite personalizar el String dentro de la excepción.
12  * @throws SistemaException En caso de que el valor sea null o vacío.
13  * El contenido de la excepción tiene el valor de etiqueta.
14  */
15 private void fallarSiNullVacío(String valor, String etiqueta) throws SistemaException
16 {
17     if (valor == null)
18         throw new SistemaException(etiqueta + " no puede ser null");
19     if (valor.length() == 0)
20         throw new SistemaException(etiqueta + " no puede ser vacío");
21 }

```

Operación `ingresarBackEnd`

De la misma forma, `ingresarBackEnd(...)` se implementa así:

```

1 public void ingresarBackEnd(String rut, String nombre, String dirección, int sueldoBase,
2   int añosExperiencia) throws SistemaException
3 {
4     if (buscarPersona(rut) != null)
5         throw new SistemaException("Persona " + rut + " ya existe");
6
7     fallarSiNullVacío(rut, "rut");
8     fallarSiNullVacío(nombre, "nombre");
9     fallarSiNullVacío(dirección, "dirección");
10
11     if (sueldoBase <= 0)
12         throw new SistemaException("sueldoBase debe ser mayor que CERO");
13     if (añosExperiencia <= 0)
14         throw new SistemaException("añosExperiencia debe ser mayor que CERO");
15
16     Persona nueva = new BackEnd(rut, nombre, dirección, sueldoBase, añosExperiencia);
17     personas.add(nueva);
18 }

```

Y el método auxiliar `buscarPersona(...)` es:

```

1 /**
2  * Retorna la persona indicada por el RUT.
3  *
4  * @param rut El RUT de la persona buscada
5  * @return Una instancia de Persona, o null si la persona con ese
6  *         RUT no existe.
7  */
8 private Persona buscarPersona(String rut)
9 {
10     for(Persona p : personas) {
11         if (p.getRut().equals(rut)) return p;
12     }
13     return null;
14 }

```

Operación `ingresarFrontEnd`

`ingresarFrontEnd(...)` se implementa así:

```

1 public void ingresarFrontEnd(String rut, String nombre, String dirección,
2   int sueldoBase, String lenguajeProgramación, int horasExtras, int nivel)
3   throws SistemaException
4 {
5     if (buscarPersona(rut) != null)
6         throw new SistemaException("Persona " + rut + " ya existe");
7
8     fallarSiNullVacío(rut, "rut");
9     fallarSiNullVacío(nombre, "nombre");
10    fallarSiNullVacío(dirección, "dirección");
11    fallarSiNullVacío(lenguajeProgramación, "lenguajeProgramación");
12
13    if (sueldoBase <= 0)
14        throw new SistemaException("sueldoBase debe ser mayor que CERO");

```

```

15     if (horasExtras < 0)
16         throw new SistemaException("horasExtras debe ser mayor que CERO");
17     if (nivel < 1 || nivel > 4)
18         throw new SistemaException("nivel debe estar entre 1 y 4");
19
20     Persona nueva = new FrontEnd(rut, nombre, dirección, sueldoBase,
21                                 lenguajeProgramación, horasExtras, nivel);
22     personas.add(nueva);
23 }

```

Operación `ingresarDevOps`

```

1 public void ingresarDevOps(String rut, String nombre, String dirección, int sueldoBase,
2                             String sistemasOp, String basesDatos)
3     throws SistemaException
4 {
5     if (buscarPersona(rut) != null)
6         throw new SistemaException("Persona " + rut + " ya existe");
7
8     fallarSiNullVacío(rut, "rut");
9     fallarSiNullVacío(nombre, "nombre");
10    fallarSiNullVacío(dirección, "dirección");
11
12    if (sueldoBase <= 0)
13        throw new SistemaException("sueldoBase debe ser mayor que CERO");
14    if (sistemasOp == null)
15        throw new SistemaException("sistemasOp no puede ser null");
16    if (basesDatos == null)
17        throw new SistemaException("basesDatos no puede ser null");
18
19    Persona nueva = new DevOps(rut, nombre, dirección, sueldoBase,
20                              sistemasOp, basesDatos);
21    personas.add(nueva);
22 }

```

Operación `asociarPersonaProyecto`

El primer problema lo empezamos a tener al implementar el siguiente método:

```

1 public void asociarPersonaProyecto(String códigoProyecto, String rutPersona)
2     throws SistemaException
3 {
4     Proyecto proyecto = buscarProyecto(códigoProyecto);
5     if (proyecto == null)
6         throw new SistemaException("No existe proyecto con código " + códigoProyecto);
7
8     Persona persona = buscarPersona(rutPersona);
9     if (persona == null)
10        throw new SistemaException("No existe persona con RUT " + rutPersona);
11
12    // .. FIXME ...
13    // Cómo asociamos la "persona" al proyecto, de acuerdo a su especialidad?
14 }

```

De acuerdo a los requerimientos del problema, cada `Proyecto` tiene asociado una persona de cada especialidad, pero en la línea 8 del código, lo único que obtenemos es una referencia a una instancia de

Persona, que efectivamente será una instancia de alguna de las clases concretas de las especialidades, pero la pregunta es ¿cómo asignamos la especialidad concreta al proyecto?

Se podría pensar en hacer algo como lo siguiente:

```

1 public void asociarPersonaProyecto(String códigoProyecto, String rutPersona)
2     throws SistemaException
3 {
4     Proyecto proyecto = buscarProyecto(códigoProyecto);
5     if (proyecto == null)
6         throw new SistemaException("No existe proyecto con código " + códigoProyecto);
7
8     Persona persona = buscarPersona(rutPersona);
9     if (persona == null)
10        throw new SistemaException("No existe persona con RUT " + rutPersona);
11
12    // .. FIXME ...
13    // Cómo asociamos la "persona" al proyecto, de acuerdo a su especialidad?
14    if (persona es de especialidad frontend) {
15        proyecto.setFE(persona);
16    } else if (persona es de especialidad backend) {
17        proyecto.setBE(persona);
18    } else if (persona es de especialidad devops) {
19        proyecto.setDO(persona);
20    }
21    persona.agregarProyecto(proyecto);
22 }
```

Pudiera parecer que lo anterior es una buena idea, pero realmente no lo es. La principal razón es que se está acoplando totalmente la operación (y la clase **Sistema**) a las tres versiones concretas de **Persona**, por lo que cualquier modificación a dichas clases, o el agregar un nuevo tipo de especialidad, causará que el código se tenga que modificar. Más aún, si se agrega una nueva especialidad, y no se modifica el método `asociarPersonaProyecto(...)` no va a pasar nada, no habrá ninguna indicación de que falta hacer algo. En ese caso lo mínimo que debería suceder es que el código no compile, para forzarnos a realizar la corrección.

Una alternativa mucho mejor es “delegar”. En este caso, el **Sistema** no tiene ninguna razón para saber el tipo concreto de **Persona** que está administrando. Pero sí le debería poder decir a la **Persona** que se asocie a un proyecto, y como la persona sabe de qué tipo concreto es, va a poder asociarse de acuerdo a su especialidad. Para que este mecanismo funcione, lo primero que hay que hacer es agregar un nuevo método abstracto a **Persona**:

```

1 public abstract class Persona
2 {
3     ...
4     protected abstract void asociarAProyecto(Proyecto proyecto);
5     ...
6 }
```

De esa forma, desde el punto de vista del **SistemaImpl** la operación para asociar una **Persona** a un **Proyecto** se convierte en:

```

1 public class SistemaImpl
2 {
3     ...
4     public void asociarPersonaProyecto(String códigoProyecto, String rutPersona)
5         throws SistemaException
6     {
```

```

7     Proyecto proyecto = buscarProyecto(códigoProyecto);
8     if (proyecto == null)
9         throw new SistemaException("No existe proyecto con código " + códigoProyecto
10    );
11
12     Persona persona = buscarPersona(rutPersona);
13     if (persona == null)
14         throw new SistemaException("No existe persona con RUT " + rutPersona);
15
16     persona.asociarAProyecto(proyecto);
17     ...
18 }

```

Por supuesto que cada tipo concreto de `Persona` debe implementar dicho método (ya que está definido como abstracto):

```

1 public class BackEnd extends Persona
2 {
3     ...
4     protected void asociarAProyecto(Proyecto proyecto)
5     {
6         proyecto.setBackEnd(this);
7         agregarProyecto(proyecto);
8     }
9 }

```

```

1 public class FrontEnd extends Persona
2 {
3     ...
4     protected void asociarAProyecto(Proyecto proyecto)
5     {
6         proyecto.setFrontEnd(this);
7         agregarProyecto(proyecto);
8     }
9 }

```

```

1 public class DevOps extends Persona
2 {
3     ...
4     protected void asociarAProyecto(Proyecto proyecto)
5     {
6         proyecto.setDevOps(this);
7         agregarProyecto(proyecto);
8     }
9 }

```

Solo falta completar el método `agregarProyecto(...)` que es lo que permite que las instancias de `Persona` sepan en cuales `Proyectos` trabajan:

```

1 public abstract class Persona
2 {
3     private List<Proyecto> proyectos;
4
5     ...
6     protected void agregarProyecto(Proyecto proyecto)
7     {
8         proyectos.add(proyecto);
9     }
10 }

```


Con todo lo anterior, finaliza la implementación de las operaciones para cargar los datos en el **Sistema**. Lo que falta implementar son las operaciones que nos permitirán obtener información a partir de los datos almacenados.

Operación `obtenerDatosProyectos`

En el caso de la operación `obtenerDatosProyectos()` se usará `StringBuilder` para que sea un poco más eficiente al componer el `String` que se retornará: ²

```

1 public String obtenerDatosProyectos()
2 {
3     StringBuilder salida = new StringBuilder("Datos de todos los proyectos:\n");
4
5     for(Proyecto proyecto : proyectos) {
6         salida
7             .append("Código: ").append(proyecto.getCódigo())
8             .append(", nombre: ").append(proyecto.getNombre())
9             .append(", meses: ").append(proyecto.getMeses())
10            .append(", monto mensual: ").append(proyecto.getMontoMensual())
11            .append(", monto total: ").append(proyecto.getMonto())
12            .append("\n");
13    }
14
15    return salida.toString();
16 }
17 
```

Operación `obtenerDatosPersonas`

Para la operación `obtenerDatosPersonas()` se usará la misma técnica:

```

1 public String obtenerDatosPersonas()
2 {
3     StringBuilder salida = new StringBuilder("Listado de personas:\n");
4
5     for(Persona p : personas) {
6         salida
7             .append("rut: ").append(p.getRut())
8             .append(", nombre: ").append(p.getNombre())
9             .append(", direccion: ").append(p.getDirección())
10            .append(", sueldo base: ").append(p.getSueldoBase())
11            .append(", sueldo: ").append(p.getSueldo())
12            .append("\n");
13    }
14
15    return salida.toString();
16 }

```

²Ver H.1 para más detalles respecto a esa clase.

Operación `obtenerDatosProyectosPersona`

Para implementar `obtenerDatosProyectosPersona(...)` tenemos que preguntarnos exactamente cómo obtendremos el listado de proyectos para una persona dada. Obviamente, primero tenemos que verificar si el contrato se cumple:

```
1 public String obtenerDatosProyectosPersona(String rutPersona)
2 {
3     Persona persona = buscarPersona(rutPersona);
4     if (persona == null)
5         throw new SistemaException("Persona " + rutPersona + " ya existe");
6
7     // Completar!
8     // Falta iterar sobre los proyectos de la persona
9
10    return null;
11 }
```

En lo que falta, tenemos que tomar la decisión de cómo, a partir de una instancia de `Persona`, obtener todos los proyectos en los que la persona participa.

Una opción es simplemente agregarle a la `Persona` un método que retorne su lista de instancias de `Proyecto`:

```
1 public abstract class Persona
2 {
3     ...
4     private List<Proyecto> proyectos;
5
6     public List<Proyecto> getProyectos() { return this.proyectos; }
7 }
```

Y de esa forma terminar de implementar la operación:

```
1 public String obtenerDatosProyectosPersona(String rutPersona)
2 {
3     Persona persona = buscarPersona(rutPersona);
4     if (persona == null)
5         throw new SistemaException("Persona " + rutPersona + " no existe");
6
7     List<Proyecto> proyectos = persona.getProyectos();
8
9     StringBuilder salida = new StringBuilder("Proyectos de la persona")
10        .append(rutPersona)
11        .append(": \n");
12
13    for(Proyecto p : proyectos) {
14        salida.append(p.getNombre()).append("\n");
15    }
16    return salida.toString();
17 }
```

El problema con esa alternativa es que cualquier cliente de `Persona` podría aprovechar de modificar la lista retornada, y agregar o remover elementos, y la `Persona` nunca se enteraría de nada.

Para solucionar el problema e implementar la operación `obtenerDatosProyectosPersona(...)` se pueden utilizar 4 alternativas diferentes:

Retornar una copia En este caso, la persona en vez de retornar la misma lista que usa para mantener los proyectos, puede crear una lista que sea una copia, pero que contenga los mismos proyectos. De esta forma, los clientes podrán iterar sobre los proyectos, e incluso podrán modificar la lista retornada, pero no tendrán ningún efecto sobre la lista “real” que mantiene la persona.

```
1 public List<Proyecto> getProyectos()
2 {
3     return new LinkedList<Proyecto>(this.proyectos);
4 }
```

Pasar una lista como parámetro Este caso es similar al anterior, pero acá el método recibe una lista (generalmente vacía), y lo que hará será iterar sobre la lista de proyectos y agregarlos a la lista pasada como parámetro (o usar `addAll` como en este código):

```
1 public void getProyectos(List<Proyecto> salida)
2 {
3     salida.addAll(this.proyectos);
4 }
```

Retornar una lista immutable Una alternativa es usar el método `unmodifiableList(...)` de la clase `Collections`. Dicho método retornará una instancia de una clase que implementa la interfaz `List`, pero que tendrá todos los métodos que permiten modificar la lista reimplementados para lanzar una excepción de tipo `UnsupportedOperationException`. Por ejemplo:

```
1 public List<Proyecto> getProyectos()
2 {
3     return Collections.unmodifiableList(this.proyectos);
4 }
```

Si alguno de los clientes de la clase recibe la lista de proyectos y trata de modificarla (por ejemplo, agregando o sacando proyectos) recibirá una excepción.

Usar un visitador Tal como se puede ver en 10.4.8, existe una forma de poder “visitar” instancias para poder realizar acciones. Esto requiere realizar algunas modificaciones, pero es un modo interesante de recopilar los datos que la operación necesita.

Primero, se declara una clase abstracta (o podría ser una interfaz) con la base del `Visitador`:

```
1 public abstract class ProyectoVisitor
2 {
3     protected abstract void visit(Proyecto p);
4 }
```

Después, a la clase `Persona` se le agrega un método para poder visitar sus `Proyectos`, pasando por parámetro una instancia de `ProyectoVisitor`:

```
1 public abstract class Persona
2 {
3     ...
4     private List<Proyecto> proyectos;
5
6     public void visitProyectos(ProyectoVisitor visitor)
7     {
8         for(Proyecto p : proyectos) {
9             p.accept(visitor);
10        }
11    }
12 }
```

A la clase `Proyecto` se le debe agregar el método `accept(...)`:

```
1 public class Proyecto
2 {
3     ...
4     public void accept(ProyectoVisitor visitor) { visitor.visit(this); }
5 }
```

Una vez que está todo listo, crearemos un `Visitador` que permita recopilar los datos que se requieren:

```
1 public class DatosProyectosPersonaVisitor extends ProyectoVisitor
2 {
3     private StringBuilder salida;
4
5     public DatosProyectosPersonaVisitor(Persona persona)
6     {
7         salida = new StringBuilder("Proyectos de la persona")
8             .append(persona.getRut())
9             .append(": \n");
10    }
11    @Override
12    protected void visit(Proyecto p)
13    {
14        salida.append(p.getNombre()).append("\n");
15    }
16    @Override
17    public String toString()
18    {
19        return salida.toString();
20    }
21 }
```

Con lo que la operación `obtenerDatosProyectosPersona(...)` se simplifica a lo siguiente:

```
1 public String obtenerDatosProyectosPersona(String rutPersona)
2     throws SistemaException
3 {
4     Persona persona = buscarPersona(rutPersona);
5     if (persona == null)
6         throw new SistemaException("Persona " + rutPersona + " ya existe");
7
8     DatosProyectosPersonaVisitor v = new DatosProyectosPersonaVisitor(persona);
9     persona.visitProyectos(v);
10    return v.toString();
11 }
```

Operación `obtenerDatosDevOps`

Continuando con la operación `obtenerDatosDevOps()`, volveremos a usar un `Visitador`, para poder diferenciar el tipo de `Persona`, ya que solamente nos interesan las instancias de `DevOps`:

```

1 public String obtenerDatosDevOps()
2 {
3     StringBuilder salida = new StringBuilder("Datos de los DevOps\n");
4
5     DevOpsPersonaVisitor v = new DevOpsPersonaVisitor(salida);
6     for(Persona p : personas) {
7         p.accept(v);
8     }
9     return salida.toString();
10 }
```

En este caso, el `Visitador` base tiene 3 métodos, uno para cada especialidad:

```

1 public interface PersonaVisitor
2 {
3     void visit(FrontEnd fe);
4     void visit(BackEnd be);
5     void visit(DevOps op);
6 }
```

Y cada subtipo de `Persona` tendrá un método `accept(...)`. Por ejemplo, para el tipo `BackEnd`:

```

1 public class BackEnd extends Persona
2 {
3     ...
4     protected void accept(PersonaVisitor visitor) { visitor.visit(this); }
5 }
```

Tenemos que crear un tipo específico de `PersonaVisitor`, para acumular los datos de las `Personas DevOps`:

```

1 public class DevOpsPersonaVisitor implements PersonaVisitor
2 {
3     private StringBuilder salida;
4
5     public DevOpsPersonaVisitor(StringBuilder salida) { this.salida = salida; }
6     @Override
7     public void visit(FrontEnd fe) { }
8     @Override
9     public void visit(BackEnd be) { }
10
11     @Override
12     public void visit(DevOps op)
13     {
14         salida
15             .append("nombre: ").append(op.getNombre())
16             .append("sistemas: ").append(op.getSistemasOp())
17             .append("bases de datos: ").append(op.getBasesDatos())
18             .append("\n");
19     }
20
21     @Override
22     public String toString() { return salida.toString(); }
23 }
```

Y finalmente usamos todo para implementar la operación:

```

1 public String obtenerDatosDevOps()
2 {
3     StringBuilder salida = new StringBuilder("Datos de los DevOps\n");
4
5     DevOpsPersonaVisitor v = new DevOpsPersonaVisitor(salida);
6     for(Persona p : personas) {
7         p.accept(v);
8     }
9     return salida.toString();
10 }

```

Operación `obtenerDatosPersonasProyecto`

La última operación es `obtenerDatosPersonasProyecto(...)`. En este caso una posible implementación podría ser:

```

1 public String obtenerDatosPersonasProyecto(String códigoProyecto)
2     throws SistemaException
3 {
4     Proyecto proyecto = buscarProyecto(códigoProyecto);
5     if (proyecto == null)
6         throw new SistemaException("No existe proyecto con código " + códigoProyecto);
7
8     StringBuilder salida = new StringBuilder();
9
10    salida.append("Personas del proyecto ").append(códigoProyecto).append("\n");
11
12    if (proyecto.getFrontEnd() != null) {
13        salida.append("FrontEnd: ").append(proyecto.getFrontEnd().getNombre())
14            .append("\n");
15    }
16    if (proyecto.getBackEnd() != null) {
17        salida.append("BackEnd: ").append(proyecto.getBackEnd().getNombre())
18            .append("\n");
19    }
20    if (proyecto.getDevOps() != null) {
21        salida.append("DevOps: ").append(proyecto.getDevOps().getNombre())
22            .append("\n");
23    }
24    return salida.toString();
25 }

```

La implementación funciona, pero tiene el mismo defecto que ya vimos en otro caso. Si se produce un cambio (por ejemplo, si a los proyectos se les agrega una cuarta persona), este código seguirá funcionando, pero en este caso, funcionará incorrectamente. La pregunta es ¿cómo podemos escribir el código (incluso re-diseñando toda la solución) de forma que sepamos si hay que cambiar algo, producto de una modificación al entorno?

Método `getSueldo()`

Antes de poder usar el `Sistema` de forma apropiada, hay que decidir cómo implementar el cálculo del sueldo de cada una de las especialidades mencionadas anteriormente.

Tal como está definido en la clase `Persona`, el método `getSueldo()` debe ser implementado en cada subclase, y de acuerdo a los requerimientos iniciales, se tiene lo siguiente:

```

1 public class Backend extends Persona
2 {
3     ...
4     public int getSueldo()
5     {
6         int sueldo = this.getSueldoBase();
7
8         sueldo += 5000 * añosExperiencia;
9
10        for(Proyecto proyecto : this.getProyectos()) {
11            int bono = (int) ((proyecto.getMonto() / proyecto.getMeses()) * 0.25);
12            sueldo += bono;
13        }
14        return sueldo;
15    }
16 }

```

```

1 public class FrontEnd extends Persona
2 {
3     ...
4     public int getSueldo()
5     {
6         int sueldo = this.getSueldoBase();
7
8         sueldo += 5000 * horasExtras;
9         sueldo += 30000 * nivel;
10
11        for(Proyecto proyecto : this.getProyectos()) {
12            int bono = (int) ((proyecto.getMonto() / proyecto.getMeses()) * 0.2);
13            sueldo += bono;
14        }
15        return sueldo;
16    }
17 }

```

```

1 public class DevOps extends Persona
2 {
3     ...
4     public int getSueldo()
5     {
6         int sueldo = this.getSueldoBase();
7
8         // Esto solo busca una lista de elementos separados por coma
9         sueldo += 8000 * this.basesDatos.split(",").length;
10
11        for(Proyecto proyecto : this.getProyectos()) {
12            int bono = (int) ((proyecto.getMonto() / proyecto.getMeses()) * 0.30);
13            sueldo += bono;
14        }
15        return sueldo;
16    }
17 }

```

Para que los métodos anteriores funcionaran, se debió agregar un nuevo método en la clase `Persona`, de forma que las subclases pudieran acceder a la lista de proyectos en cada caso:

```
1 public abstract class Persona
2 {
3     private List<Proyecto> proyectos;
4
5     protected List<Proyecto> getProyectos()
6     {
7         return proyectos;
8     }
9     ...
10 }
```

11.1.4. Usando el Sistema

Todo parte desde la clase que tiene el método estático `main`. En este caso, se instancia una `App` y se llama a uno de sus métodos para comenzar el proceso:

```
1 public class App
2 {
3     public static void main(String[] args) throws IOException
4     {
5         App app = new App();
6
7         app.iniciar();
8     }
9     ...
10 }
```

El proceso comprende leer los archivos para cargar los datos a una instancia de `Sistema`, y después obtener datos de acuerdo a los requerimientos iniciales:

```
1 private void iniciar() throws IOException
2 {
3     Sistema sist = new SistemaImpl();
4
5     leerProyectos(sist);
6     leerPersonas(sist);
7     leerAsignaciones(sist);
8
9     System.out.println(sist.obtenerDatosProyectos());
10    System.out.println(sist.obtenerDatosPersonas());
11    System.out.println(sist.obtenerDatosDevOps());
12
13    Scanner scanner = new Scanner(System.in);
14
15    mostrarUnProyecto(sist, scanner);
16    mostrarUnRUT(sist, scanner);
17
18    scanner.close();
19 }
```

En este caso, la operación `obtenerDatosProyectosPersona(...)` requiere un RUT. Nótese que la operación puede lanzar una excepción, así que se toman los resguardos necesarios:


```

1 private void mostrarUnRUT(Sistema sist, Scanner scanner)
2 {
3     System.out.println("Ingrese un RUT: ");
4     String rut = scanner.nextLine();
5
6     try {
7         sist.obtenerDatosProyectosPersona(rut);
8     } catch (SistemaException e) {
9         System.out.println("Error obteniendo datos de persona " + rut);
10    }
11 }

```

Algo similar pasa con `obtenerDatosPersonasProyecto(...)`:

```

1 private void mostrarUnProyecto(Sistema sist, Scanner scanner)
2 {
3     System.out.println("Ingrese un código de proyecto: ");
4     String códigoProyecto = scanner.nextLine();
5     try {
6         sist.obtenerDatosPersonasProyecto(códigoProyecto);
7     } catch (SistemaException e) {
8         System.out.println("Error obteniendo datos de proyecto " + códigoProyecto);
9     }
10 }

```

El proceso de lectura de los datos desde los archivos está dividido en tres:

```

1 private void leerAsignaciones(Sistema sist) throws IOException
2 {
3     Scanner sc = new Scanner(new File("asignaciones.txt"));
4
5     while (sc.hasNextLine()) {
6         String linea = sc.nextLine();
7         String[] partes = linea.split(",");
8
9         String codProyecto = partes[0];
10        String rut = partes[1];
11
12        try {
13            sist.asociarPersonaProyecto(codProyecto, rut);
14        } catch (SistemaException e) {
15            System.out.println("Error ingresando asociación " + rut + " - " +
16            codProyecto);
17        }
18    }
19    sc.close();
20 }

```

```
1 private void leerPersonas(Sistema sist) throws IOException
2 {
3     Scanner sc = new Scanner(new File("personas.txt"));
4
5     while (sc.hasNextLine()) {
6         String linea = sc.nextLine();
7         String[] partes = linea.split(",");
8
9         int tipo = Integer.valueOf(partes[0]);
10
11         switch(tipo) {
12             case 1:
13                 agregarFrontEnd(sist, partes);
14                 break;
15             case 2:
16                 agregarBackEnd(sist, partes);
17                 break;
18             case 3:
19                 agregarDevOps(sist, partes);
20                 break;
21         }
22     }
23     sc.close();
24 }
```

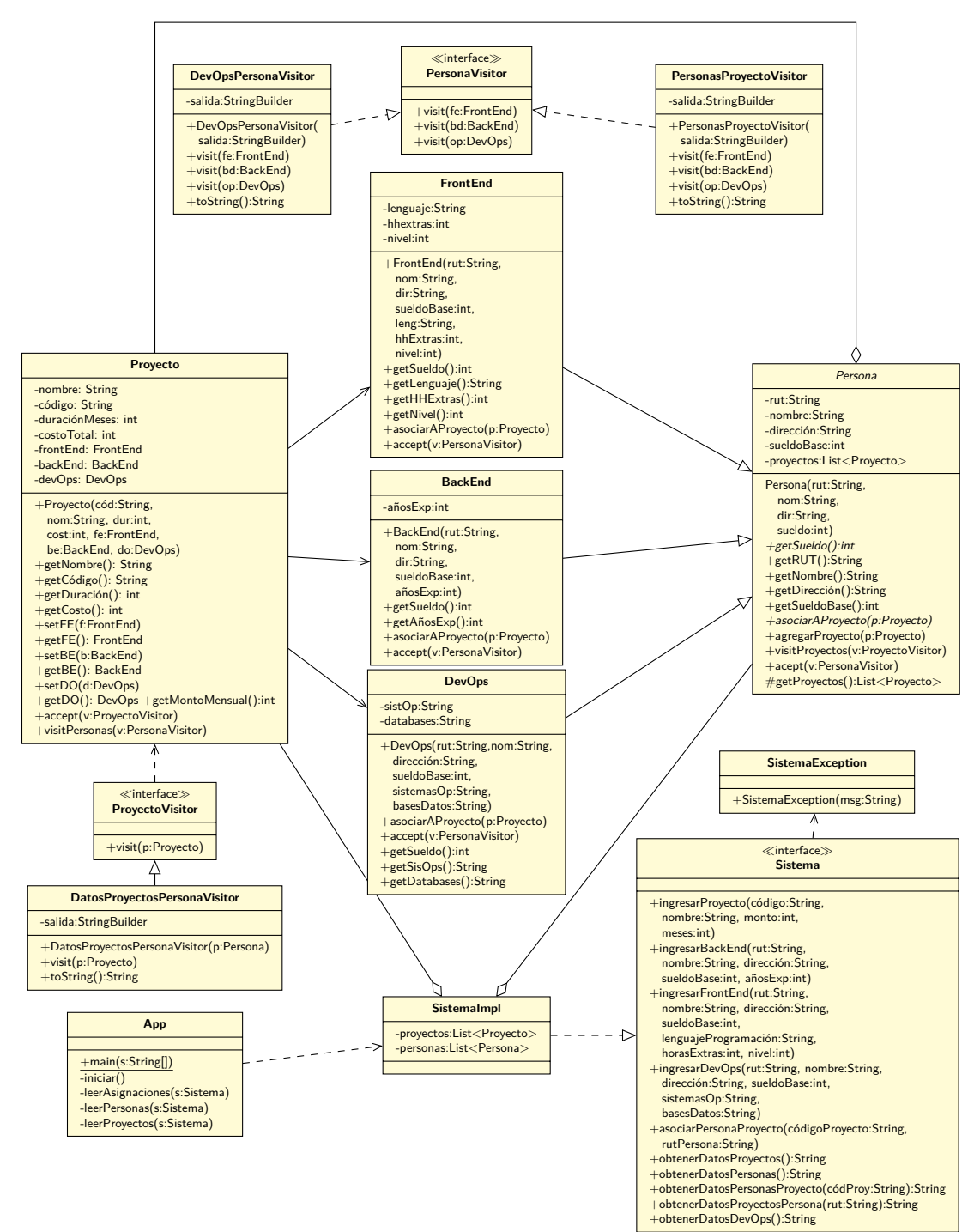
```
1 private void agregarDevOps(Sistema sist, String[] partes)
2 {
3     String rut = partes[1];
4     String nombre = partes[2];
5     String dirección = partes[3];
6     int sueldoBase = Integer.valueOf(partes[4]);
7
8     String sistemasOp = partes[5];
9     String basesDeDatos = partes[6];
10
11     try {
12         sist.ingresarDevOps(rut, nombre, dirección, sueldoBase, sistemasOp, basesDeDatos
13     );
14     } catch (SistemaException e) {
15         System.out.println("Error ingresando dev ops " + rut);
16     }
17 }
```

```
1 private void agregarBackEnd(Sistema sist, String[] partes)
2 {
3     String rut = partes[1];
4     String nombre = partes[2];
5     String dirección = partes[3];
6     int sueldoBase = Integer.valueOf(partes[4]);
7
8     int añosExperiencia = Integer.valueOf(partes[5]);
9
10    try {
11        sist.ingresarBackEnd(rut, nombre, dirección, sueldoBase, añosExperiencia);
12    } catch (SistemaException e) {
13        System.out.println("Error ingresando back end " + rut);
14    }
15 }
```

```
1 private void agregarFrontEnd(Sistema sist, String[] partes)
2 {
3     String rut = partes[1];
4     String nombre = partes[2];
5     String dirección = partes[3];
6     int sueldoBase = Integer.valueOf(partes[4]);
7     String lenguaje = partes[5];
8     int horasExtras = Integer.valueOf(partes[6]);
9     int nivel = Integer.valueOf(partes[7]);
10
11     try {
12         sist.ingresarFrontEnd(rut, nombre, dirección, sueldoBase, lenguaje, horasExtras,
13             nivel);
14     } catch (SistemaException e) {
15         System.out.println("Error ingresando front end " + rut);
16     }
17 }
```

```
1 private void leerProyectos(Sistema sist) throws IOException
2 {
3     Scanner sc = new Scanner(new File("proyectos.txt"));
4
5     while (sc.hasNextLine()) {
6         String linea = sc.nextLine();
7         String[] partes = linea.split(",");
8         String nombre = partes[0];
9         String código = partes[1];
10        int duración = Integer.valueOf(partes[2]);
11        int costo = Integer.valueOf(partes[3]);
12
13        try {
14            sist.ingresarProyecto(código, nombre, duración, costo);
15        } catch (SistemaException e) {
16            System.out.println("Error ingresando proyecto " + código);
17        }
18    }
19
20    sc.close();
21 }
```

11.1.5. Diagrama de clases actualizado



11.2. Problema 2

La empresa BCPR administra una flota de vehículos de diferentes tipos, para así poder prestar servicios a sus clientes.

Existe un archivo llamado `flota.txt` que contiene la lista de vehículos de la empresa. Cada línea define un vehículo, de los cuales existen 3 tipos: auto, camión, camioneta. Dependiendo del tipo de vehículo, cada línea contiene datos extras específicos:

auto año de fabricación, precio de compra, cantidad de pasajeros

camión año de fabricación, precio de compra, tonelaje, cantidad de neumáticos

camioneta año de fabricación, precio de compra, cantidad de pasajeros, capacidad de carga (en kilogramos)

Por ejemplo:

```
Camión,2006,46820000,120,8,C1
Camioneta,2002,14070000,1,500,M1
Camión,1993,1700000,80,8,C2
Camión,1997,64870000,150,10,C2
Auto,2020,13560000,4,A1
Auto,2008,15130000,1,A2
Camión,2014,19780000,150,12,C5
Auto,1998,3820000,4,A6
Auto,2020,14230000,4,A8
Camioneta,2019,17920000,1,250,M3
Camioneta,2014,27040000,3,750,M5
Camión,2015,62660000,50,4,C9
```

En todos los casos, el último elemento corresponde a la “patente” del vehículo.

La empresa, de forma permanente, registra mes a mes las operaciones que se realizan sobre cada vehículo de la flota, y todas estas operaciones se guardan en un mismo archivo.

Por ejemplo, el archivo `202112.txt` contiene todas las operaciones de la flota para el mes de diciembre 2021.

Una de las actividades que se guardan en el archivo son las operaciones de carga de combustible: (ignore las líneas que empiezan con #, ya que solo sirven para explicar el formato):

```
# Carga, Vehículo C1, día 4, 80 litros, diesel, kilometraje
C,C1,4,80,D,12088
# Carga, Vehículo M5, día 10, 65 litros, gasolina 95, kilometraje
C,M5,10,65,G95,67994
```

Los tipos de combustible válidos son diesel (D), gasolina 95 (G95) y gasolina 97 (G97). Nótese que en la flota solamente los camiones usan combustible diesel; el resto de los vehículos usan gasolina, en

cualquiera de sus tipos. Si en algún momento aparece un registro indicando una carga de combustible incorrecta, dicha línea se debe ignorar.

Algo particular de la operación de la empresa, es que cada vehículo comienza el mes con el estanque lleno de combustible. Asuma que la última carga del mes de cada vehículo lo deja lleno de combustible (para que pueda comenzar el siguiente mes con el estanque lleno).

Otras actividades que se guardan en el archivo son las reparaciones que se le hacen a los vehículos. Por ejemplo:

```
# Reparación, Vehículo C5, día 11, 8 horas de duración, costo 500.000, kilometraje
R,C5,11,8,500000,7890
# Reparación, Vehículo A8, día 20, 4 horas de duración, costo 120.000, kilometraje
R,A8,20,4,120000,21671
```

También se registran las veces en que se cambian los neumáticos de los vehículos:

```
# Cambio, Vehículo C2, día 7, 2 neumáticos, costo 120.000 cada uno, kilometraje
N,C2,7,2,120000,134050
# Cambio, Vehículo A8, día 22, 4 neumáticos, costo 80.000 cada uno, kilometraje
N,A8,22,4,80000,71455
```

Tanto los autos como las camionetas siempre usan 4 neumáticos. Si algún registro indica que se cambiaron más neumáticos de los posibles, entonces la línea se debe ignorar.

Dentro del archivo, también se registra el kilometraje de los vehículos al inicio del mes:

```
I,C1,12000
I,M1,5000
I,C2,13000
I,C2,13500
I,A1,4000
I,A2,5400
I,C5,67000
I,A6,57000
I,A8,9500
I,M3,5900
I,M5,13100
I,C9,19000
```

La empresa siempre busca aumentar sus utilidades, así que necesita saber el precio posible de venta de cada vehículo de su flota al final de cada periodo. Dependiendo del tipo de vehículo el precio de venta se calcula de la siguiente forma:

auto

$$preciodeventa = \text{int}(\text{preciodecompra} + \text{gastosrealizados} - \frac{\text{kilometraje}_{\text{final}}}{20000} + \text{ganancia})$$

camión

$$preciodeventa = \text{int}(\text{preciodecompra} + \frac{\text{gastosrealizados}}{2} - \frac{\text{kilometraje final}}{\text{tonelaje} \times 20000} + \text{ganancia})$$

camioneta

$$preciodeventa = \text{int}(\text{preciodecompra} + \frac{\text{gastosrealizados}}{4} - \frac{\text{kilometraje final}}{20000} + \text{ganancia})$$

El “kilometraje final” corresponde al kilometraje máximo registrado por cualquier operación realizada sobre el vehículo en el mes. “gastos realizados” corresponde a la suma de los montos para todas las operaciones en donde se incurrieron gastos. La ganancia es un valor que la empresa decide al inicio de cada periodo.

Por otra parte, a la empresa le interesa calcular la eficiencia de su flota, de acuerdo al consumo de combustible de ésta durante el mes. Es por eso, que por cada vehículo se debe calcular los *km/litro* con los datos del periodo. Esto se realiza dividiendo la cantidad de kilómetros recorridos por el vehículo por la cantidad de litros de combustible cargados durante el mes (recuerde que la última carga deja el estanque lleno, y además los vehículos inician el mes con el estanque completo). Para obtener los kilómetros recorridos la empresa solo considera las operaciones de carga de combustible.

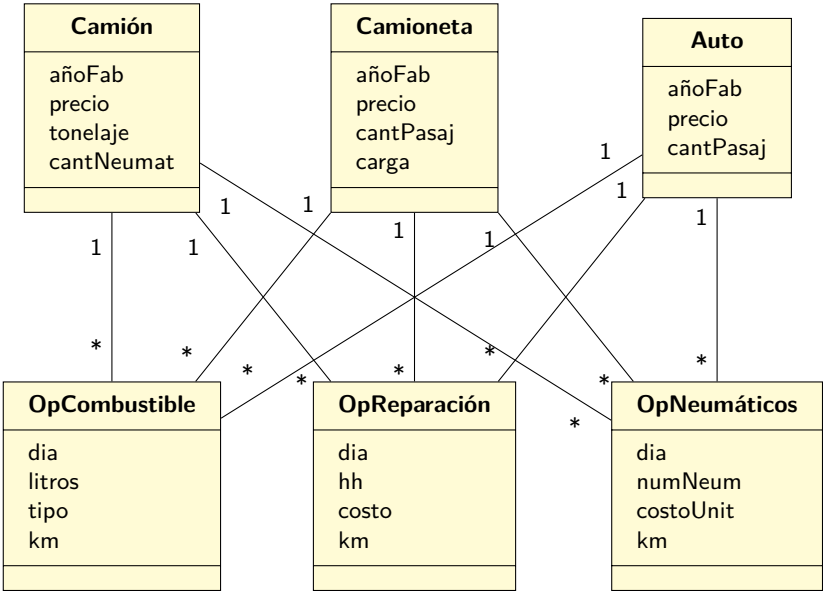
11.2.1. Trabajo a realizar

1. Al iniciar el programa, se debe preguntar por el teclado la ganancia de la empresa, leer todos los datos indicados desde los archivos y cargar su información en estructuras de datos apropiadas.
2. El programa debe mostrar el precio de venta de cada vehículo, indicando el precio inicial y la diferencia entre los dos, ordenado por la potencial ganancia de la venta, de mayor a menor.
3. Después, se deben eliminar aquellos vehículos que no tuvieron ninguna operación diferente a I.
4. El programa debe mostrar el rendimiento en *km/litro* de cada vehículo remanente.

11.2.2. Iniciando el análisis

Como se puede esperar a partir de la descripción del problema, la solución debería contener camiones, camionetas y autos, a los que se les realizan operaciones para cargar combustible, hacer reparaciones y cambios de neumáticos.

En términos de registrar las operaciones que se realizan sobre los vehículos, se puede comenzar planteando el siguiente modelo del dominio:



Este modelo es bastante simple, ya que finalmente todos los tipos de vehículos pueden acceder a las mismas operaciones, y la variación entre un tipo y otro se da en algunas acciones que se pueden realizar sobre ellos, pero para eso, hay que avanzar un poco más en el proceso.

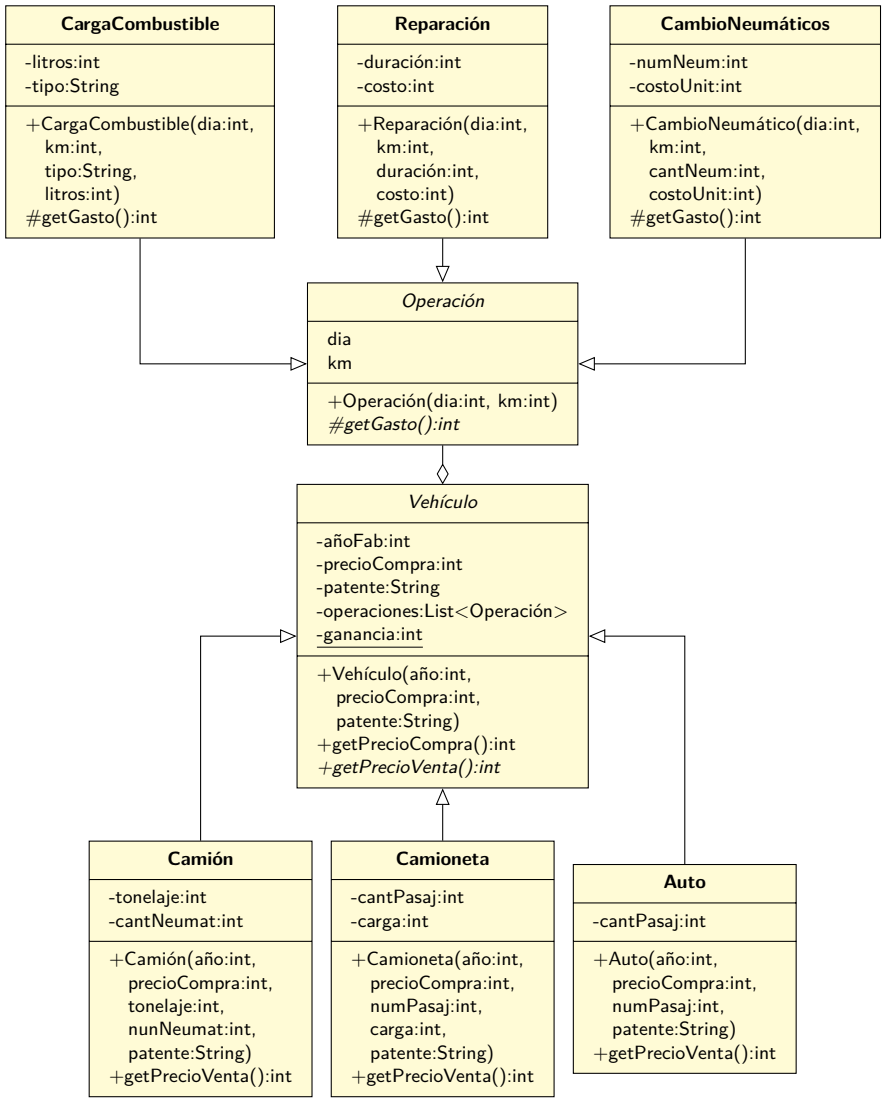
11.2.3. Diseñando la solución

Veamos el requerimiento para poder reportar el precio de venta de cada vehículo. En este caso, nos dicen explícitamente que dicho valor se calcula de forma diferente dependiendo del tipo del vehículo. Además, el cálculo depende de valores que pertenecen a cada tipo específico.

Lo anterior, además del hecho de que los diferentes tipos de vehículos comparten varios atributos, nos lleva a pensar que se puede diseñar la solución haciendo que los tres tipos de vehículos tengan una relación de herencia, donde la clase base (que será abstracta) tenga los atributos comunes entre los vehículos, y además tenga un método abstracto que cada clase concreta debe implementar para poder calcular de forma específica el precio de venta de cada uno.

Además de usarse herencia en los vehículos, los tipos de operaciones también se pueden organizar en una jerarquía, de forma que la relación entre los vehículos y las operaciones se realicen entre clases abstractas.

Algo más que debemos tener en cuenta es que de acuerdo al enunciado del problema “la ganancia es un valor que la empresa decide al inicio de cada periodo”. De acuerdo a la forma en que se calcula el precio de venta, la **ganancia** es algo global que afecta a todos los tipos de vehículos. Para simplificar el diseño, haremos que ese valor sea un atributo estático, que pueda ser configurado al iniciar la aplicación.



Al igual que en el ejemplo anterior, y de acuerdo a la arquitectura que vamos a usar (tal como se presentó en 9.4), crearemos primero una interfaz que contenga todas las operaciones necesarias para solucionar el problema. Esto significa que no solo definiremos el nombre de dichas operaciones, sino que su contratos completos.

De acuerdo a lo especificado, podemos pensar en diseñar la interfaz **Sistema** para que contenga las siguientes operaciones:

```

1 /**
2  * Crea un nuevo vehículo y lo agrega al listado de vehículos
3  * del sistema
4  *
5  * POST:
6  * - Se creó un vehículo nuevo y quedó registrado en el sistema.
7  *
8  * @param partes Un arreglo que contiene los datos necesarios para
9  *   crear cada vehículo.
10 *
11 * [0]: tipo del vehículo a crear (Camión, Camioneta, Auto)
12 *
13 * Para Camión:
14 * [1]: año [2]: precio [3]: pasajeros [4]: patente
15 *
16 * Para Camioneta:
17 * [1]: año [2]: precio [3]: pasajeros [4]: carga [5]: patente
18 *
19 * Para Auto:
20 * [1]: año [2]: precio [3]: carga [4]: neumáticos [5]: patente
21 *
22 * @throws SistemaException Cuando no es posible crear un vehículo, ya sea
23 *   porque el tipo es inválido o no se entregó la cantidad apropiada
24 *   de parámetros
25 */
26 void crearVehículo(String[] partes) throws SistemaException;

```

```

1 /**
2  * Crea una nueva operación y la agrega al listado de operaciones.
3  *
4  * POST:
5  * - Operación cargada y agregada al sistema
6  *
7  * @param partes Un arreglo que contiene los datos necesarios para
8  *   crear la operación. El contenido de cada índice
9  *   se define así:
10 *
11 * [0]: tipo (C, R, N, I)
12 *
13 * Tipo Carga:
14 * [1]: patente [2]: día [3]: litros [4]: tipo combustible [5]: km
15 *
16 * Tipo Reparación:
17 * [1]: patente [2]: día [3]: duración [4]: costo [5]: km
18 *
19 * Tipo Cambio neumáticos:
20 * [1]: patente [2]: día [3]: cantidad neumáticos [4]: costo unitario [5]: km
21 *
22 * Tipo Inicio Mes:
23 * [1]: patente [2]: km
24 *
25 * @throws SistemaException Cuando no es posible crear la operación,
26 *   ya sea por que el tipo es inválido, o se hace referencia
27 *   a vehículos que no existen, o el arreglo no contiene
28 *   la cantidad correcta de elementos.
29 */
30 void cargarOperación(String[] partes) throws SistemaException;

```

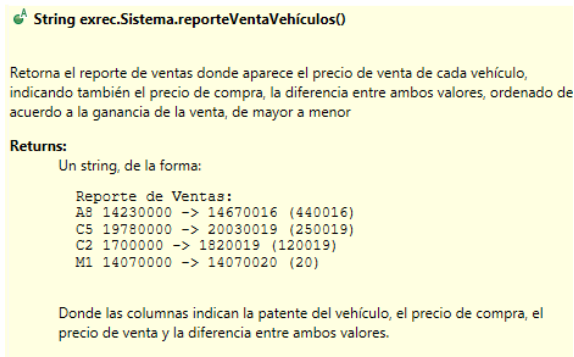
Para la siguiente operación, se especificará el formato del `String` que se debe retornar. Para eso se usará `<pre>` en el `javadoc`, que es el tag de `HTML` para indicar texto preformateado que debe ser presentado exactamente igual, sin modificación:

```

1 /**
2  * Retorna el reporte de ventas donde aparece el precio de venta
3  * de cada vehículo, indicando también el precio de compra, la
4  * diferencia entre ambos valores, ordenado de acuerdo a la ganancia
5  * de la venta, de mayor a menor
6  *
7  * @return Un string, de la forma:
8  * <pre>
9  * Reporte de Ventas:
10 * A8 14230000 -> 14670016 (440016)
11 * C5 19780000 -> 20030019 (250019)
12 * C2 17000000 -> 1820019 (120019)
13 * M1 14070000 -> 14070020 (20)
14 * </pre>
15 * Donde las columnas indican la patente del vehículo, el precio de
16 * compra, el precio de venta y la diferencia entre ambos valores.
17 */
18 String reporteVentaVehiculos();

```

Al escribir el texto de esa forma, se logrará que al revisar la documentación se muestre correctamente formateada. Por ejemplo en Eclipse:



String exrec.Sistema.reporteVentaVehiculos()

Retorna el reporte de ventas donde aparece el precio de venta de cada vehículo, indicando también el precio de compra, la diferencia entre ambos valores, ordenado de acuerdo a la ganancia de la venta, de mayor a menor

Returns:
Un string, de la forma:

```

Reporte de Ventas:
A8 14230000 -> 14670016 (440016)
C5 19780000 -> 20030019 (250019)
C2 17000000 -> 1820019 (120019)
M1 14070000 -> 14070020 (20)

```

Donde las columnas indican la patente del vehículo, el precio de compra, el precio de venta y la diferencia entre ambos valores.

```

1 /**
2  * Retorna un reporte con el rendimiento de todos los vehículos
3  * presentes actualmente.
4  *
5  * @return Un string, de la forma:
6  *
7  * <pre>
8  * Reporte de Rendimientos:
9  * A8 7.5 km/L
10 * C2 0.0 km/L
11 * A6 10.0 km/L
12 * M5 3.6363636363636362 km/L
13 * </pre>
14 *
15 * Donde las columnas indican la patente del vehículo y el rendimiento
16 * calculado.
17 */
18 String reporteRendimientos();

```

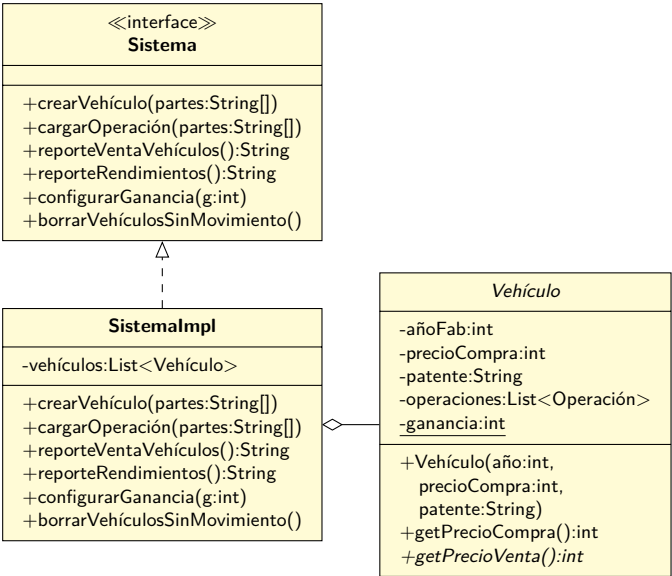
```
1 /**
2  * Configura la ganancia en la venta de vehículos, de forma que
3  * el valor sea visible para todos los vehículos.
4  *
5  * @param ganancia El valor de la ganancia a aplicar a todos
6  * los vehículos para calcular su precio de venta.
7  */
8 void configurarGanancia(int ganancia);

1 /**
2  * Elimina del sistema aquellos vehículos que no tuvieron ninguna
3  * operación diferente a I (set km de inicio).
4  *
5  * POST:
6  * - Todos los vehículos sin movimientos se eliminan del sistema
7  */
8 void borrarVehículosSinMovimiento();
```

11.2.4. Implementando la solución

Ahora corresponde crear la clase `SistemaImpl` que implementa la interfaz `Sistema`. Como podemos imaginar, para que las operaciones se puedan implementar, el sistema tiene que mantener referenciados a las instancias de `Vehículo` que se van a crear. Se podría pensar que el `Sistema` también tiene que mantener referenciadas la instancias de `Operación`, pero nos vamos a aprovechar del hecho de que las operaciones se realizan sobre los vehículos, y cada `Vehículo` guardará todas las operaciones que se le han realizado.

Por lo tanto, el `Sistema` queda así:



De acuerdo a lo anterior, la base de la case `SistemaImpl` será:

```

1 public class SistemaImpl implements Sistema
2 {
3     private List<Vehículo> vehiculos = new LinkedList<>();
4 }

```

Operación crearVehículo

Para esta operación la idea es primero detectar el tipo de vehículo que se quiere crear, y delegar la creación en sí a funciones especializadas:

```

1 public void crearVehículo(String[] partes) throws SistemaException
2 {
3     if (partes.length < 1) throw new SistemaException("No se puede crear Vehículo");
4
5     String tipo = partes[0];
6     Vehículo v;
7
8     if (tipo.equals("Camión")) {
9         v = crearCamion(partes);
10    } else if (tipo.equals("Camioneta")) {
11        v = crearCamioneta(partes);
12    } else if (tipo.equals("Auto")) {
13        v = crearAuto(partes);
14    } else {
15        throw new SistemaException("Tipo " + tipo + " inválido");
16    }
17
18    vehiculos.add(v);
19 }

```

Usando otras técnicas se podría haber implementado de forma diferente, pero por ahora, no está tan mal usar muchos `if`.³

Cada una de las funciones referenciadas son simples, ya que realizan una sola operación.

Para el caso de `Auto`:

```

1 private Vehículo crearAuto(String[] partes) throws SistemaException
2 {
3     if (partes.length != 5)
4         throw new SistemaException("No se puede crear Auto");
5
6     return new Auto(
7         Integer.valueOf(partes[1]), // año
8         Integer.valueOf(partes[2]), // precio
9         Integer.valueOf(partes[3]), // pasajeros
10        partes[4] // patente
11    );
12 }

```

³Obviamente sería mucho mejor usar una `Factory`

Para el caso de `Camioneta`:

```

1 private Vehículo crearCamioneta(String[] partes) throws SistemaException
2 {
3     if (partes.length != 6) throw new SistemaException("No se puede crear Camioneta");
4
5     return new Camioneta(
6         Integer.valueOf(partes[1]), // año
7         Integer.valueOf(partes[2]), // precio
8         Integer.valueOf(partes[3]), // pasajeros
9         Integer.valueOf(partes[4]), // carga
10        partes[5] // patente
11    );
12 }

```

Y para el caso de `Camión`:

```

1 private Vehículo crearCamion(String[] partes) throws SistemaException
2 {
3     if (partes.length != 6) throw new SistemaException("No se puede crear Camión");
4
5     return new Camión(
6         Integer.valueOf(partes[1]), // año
7         Integer.valueOf(partes[2]), // precio
8         Integer.valueOf(partes[3]), // carga
9         Integer.valueOf(partes[4]), // neumáticos
10        partes[5] // patente
11    );
12 }

```

Una verificación que faltó realizar debido a la manera en que se pasaron los parámetros de construcción de los diferentes tipos de vehículo, es asegurarse de que los valores son realmente convertibles a números: en cualquier caso que las llamadas a `Integer.valueOf(...)` falle, se lanzará una excepción que causará que el programa deje de funcionar.

Operación `cargarOperación`

Se aplicará la misma técnica en esta implementación:

```

1 public void cargarOperación(String[] partes) throws SistemaException
2 {
3     if (partes.length < 1) throw new SistemaException("No se puede cargar operación");
4
5     String tipo = partes[0];
6
7     if (tipo.equals("C")) { // Carga combustible
8         cargarCombustible(partes);
9     } else if (tipo.equals("R")) { // Reparación
10        reparar(partes);
11    } else if (tipo.equals("N")) { // Cambio neumáticos
12        cambiarNeumáticos(partes);
13    } else if (tipo.equals("I")) {
14        configurarInicioMes(partes);
15    } else {
16        throw new SistemaException("Operación " + tipo + " inválida");
17    }
18 }

```

Al implementar estas operaciones ya nos vemos en la necesidad de agregarle métodos a las clases del dominio. Por ejemplo, en este caso, después de buscar la instancia de **Vehículo** indicada, se le configura el kilómetro inicial:

```

1 private void configurarInicioMes(String[] partes) throws SistemaException
2 {
3     if (partes.length != 3) throw new SistemaException();
4
5     String patente = partes[1];
6     int km = Integer.valueOf(partes[2]);
7
8     Vehículo v = buscarVehículo(patente);
9     if (v == null) {
10         throw new SistemaException("Vehículo " + patente + " no existe");
11     }
12     v.setKilometrajeInicial(km);
13 }

```

Al cambiarle los neumáticos a un **Vehículo**, se invoca la operación correspondiente en la instancia. Cada instancia sabe si la operación se puede o no realizar, dependiendo de las condiciones particulares de cada tipo, por lo que la operación se declarará como abstracta en la clase base:

```

1 private void cambiarNeumáticos(String[] partes) throws SistemaException
2 {
3     if (partes.length != 6) throw new SistemaException();
4
5     String patente = partes[1];
6     int dia = Integer.valueOf(partes[2]);
7     int cantidadNeumáticos = Integer.valueOf(partes[3]);
8     int costoUnitario = Integer.valueOf(partes[4]);
9     int km = Integer.valueOf(partes[5]);
10
11     Vehículo v = buscarVehículo(patente);
12     if (v == null) throw new SistemaException("Vehículo " + patente + " no existe");
13
14     if (!v.cambiarNeumáticos(dia, km, cantidadNeumáticos, costoUnitario)) {
15         throw new SistemaException("No se puede cambiar neumático de vehículo " +
16             patente + "; " +
17             cantidadNeumáticos + " > " + v.getCantidadNeumáticos());
18     }
19 }

```

Para el caso de la reparación, se invoca la operación correspondiente:

```

1 private void reparar(String[] partes) throws SistemaException
2 {
3     if (partes.length != 6) throw new SistemaException();
4
5     String patente = partes[1];
6     int dia = Integer.valueOf(partes[2]);
7     int duracion = Integer.valueOf(partes[3]);
8     int costo = Integer.valueOf(partes[4]);
9     int km = Integer.valueOf(partes[5]);
10
11     Vehículo v = buscarVehículo(patente);
12     if (v == null) throw new SistemaException("Vehículo " + patente + " no existe");
13
14     v.reparar(dia, km, duracion, costo);
15 }

```

Finalmente, cada vehículo sabe si puede o no cargar el combustible indicado:

```

1 private void cargarCombustible(String[] partes) throws SistemaException
2 {
3     if (partes.length != 6) throw new SistemaException();
4
5     String patente = partes[1];
6     int dia = Integer.valueOf(partes[2]);
7     int litros = Integer.valueOf(partes[3]);
8     String tipo = partes[4];
9     int km = Integer.valueOf(partes[5]);
10
11     Vehículo v = buscarVehículo(patente);
12     if (v == null) {
13         throw new SistemaException("Vehículo " + patente + " no existe");
14     }
15
16     if (!v.cargarCombustible(dia, km, tipo, litros)) {
17         throw new SistemaException("Vehículo " + patente +
18                                     " no acepta combustible " + tipo);
19     }
20 }

```

La función auxiliar para buscar una instancia de Vehículo es muy simple:⁴

```

1 private Vehículo buscarVehículo(String patente)
2 {
3     for(Vehículo v : vehículos) {
4         if (v.getPatente().equals(patente)) return v;
5     }
6     return null;
7 }

```

⁴Y podría ser aun más simple usando algunas estructuras de datos más avanzadas, pero por ahora, una lista y una búsqueda lineal es suficiente. Si quieres complicarte la vida, puedes usar lo que aparece en H.2 en la página 462.

Operación `reporteVentaVehículos`

En esta operación, se tiene que retornar un `String` que contenga el reporte de ventas, pero ordenado de una forma especial: de acuerdo a la ganancia de la venta de cada `Vehículo`.

Hay muchas formas de lograr que el reporte se construya ordenado, pero acá vamos a aprovechar que existe un método estático en la clase `Collections` que permite ordenar una `List`. Después de ordenar la lista, basta solo recorrerla y generar el reporte:

```

1 public String reporteVentaVehículos()
2 {
3     StringBuilder txt = new StringBuilder("Reporte de Ventas:\n");
4
5     Collections.sort(vehículos);
6
7     for(Vehículo v : vehículos) {
8         int p = v.getPrecioVenta();
9         int diff = p - v.getPrecioCompra();
10
11         txt.append(v.getPatente())
12             .append(" ").append(v.getPrecioCompra())
13             .append(" -> ").append(p)
14             .append(" (").append(diff).append(")\n");
15     }
16     txt.append("\n");
17     return txt.toString();
18 }

```

Pero, para que `Collections.sort(...)` opere correctamente, se requiere una modificación a la clase `Vehículo`. Es necesario que la clase implemente la interfaz `Comparable`. Al implementar dicha interfaz nos veremos en la obligación de agregar el método `compareTo(...)` a la clase, lo que permitirá que `Collections.sort()` pueda comparar dos instancias de `Vehículo` y determine el orden relativo entre esas dos instancias. Considera el caso simple en que se tiene una lista de números enteros: en este caso, es simple comparar dos elementos cualquiera de la lista y decidir cuál es mayor y cuál es menor, ya que los elementos en sí poseen un valor que es directamente comparable. El problema con ordenar instancias más complejas (por ejemplo, `Vehículos`), es que al comparar dos de ellas, no hay una forma clara e universal de determinar cuál es “mayor” y cuál es “menor”. Por esto, al agregar el método `compareTo` se le delega a la misma clase que decida cuál es el orden relativo al comparar dos instancias. Y teniendo ese conocimiento, `Collections.sort()` va a poder ordenar la lista de forma exitosa.

Para el caso de `Vehículo`:

```

1 public abstract class Vehículo implements Comparable<Vehículo>
2 {
3     ...
4     @Override
5     public int compareTo(Vehículo otro)
6     {
7         int d1 = this.getPrecioVenta() - this.getPrecioCompra();
8         int d2 = otro.getPrecioVenta() - otro.getPrecioCompra();
9
10        if (d1 < d2) return 1;
11        if (d2 < d1) return -1;
12        return 0;
13    }
14 }

```

Nótese que el hecho de ordenar la lista de `Vehículos` no afectará el resto del funcionamiento del `Sistema`: ningún método depende de que la lista esté ordenada de alguna u otra forma. Para saber por qué a veces se retorna `-1`, `1` o a veces `0`, revisa la documentación de `Comparable`.

Operación `reporteRendimientos`

```

1 public String reporteRendimientos()
2 {
3     StringBuilder txt = new StringBuilder("Reporte de Rendimientos:\n");
4
5     Collections.sort(vehículos);
6
7     for(Vehículo v : vehículos) {
8         double r = v.getRendimiento();
9         txt.append(v.getPatente())
10            .append(" ")
11            .append(r)
12            .append(" km/L\n");
13     }
14
15     txt.append("\n");
16     return txt.toString();
17 }

```

Operación `configurarGanancia`

Para poder afectar de forma general a todos los `Vehículos` cuando calculen su precio de venta, se decidió agregarles un atributo estático y su correspondiente método para configurar el valor de la ganancia:

```

1 public void configurarGanancia(int ganancia)
2 {
3     Vehículo.setGanancia(ganancia);
4 }

```

A `Vehículo` se le agregó lo siguiente:

```

1 public abstract class Vehículo implements Comparable<Vehículo>
2 {
3     private static int ganancia;
4
5     public static void setGanancia(int ganancia)
6     {
7         Vehículo.ganancia = ganancia;
8     }
9     public static int getGanancia()
10    {
11        return ganancia;
12    }
13    ...
14 }

```

Operación `borrarVehículosSinMovimiento`

```

1 public void borrarVehículosSinMovimiento()
2 {
3     Iterator<Vehículo> it = vehículos.iterator();
4     while (it.hasNext()) {
5         Vehículo v = it.next();
6
7         if (!v.tieneOperaciones()) {
8             it.remove();
9         }
10    }
11 }

```

11.2.5. Usando el Sistema

Obviamente la aplicación debe tener un punto de partida, y en este caso simplemente usaremos el método `main` para realizar el proceso:

```

1 public class App
2 {
3     public static void main(String[] arg) throws Exception
4     {
5         Sistema sistema = new SistemaImpl();
6
7         leerGanancia(sistema);
8         leerFlota(sistema);
9         leerMes(sistema);
10
11        System.out.println(sistema.reporteVentaVehículos());
12
13        sistema.borrarVehículosSinMovimiento();
14
15        System.out.println(sistema.reporteRendimientos());
16    }
17    ...
18 }

```

Los métodos que realizan la lectura de los archivos son simples.

En el caso de `leerFlota`, se decidió que si durante la creación de los vehículos se generaba una excepción de tipo `SistemaException`, ésta no se iba a atajar, por lo que el proceso de lectura se va a detener:

```

1 private static void leerFlota(Sistema sistema)
2     throws FileNotFoundException, SistemaException
3 {
4     Scanner s = new Scanner(new FileReader("flota.txt"));
5
6     while (s.hasNextLine()) {
7         String line = s.nextLine();
8         String[] partes = line.split(",");
9         sistema.crearVehículo(partes);
10    }
11
12    s.close();
13 }

```

Por otro lado, en el caso de la lectura de los datos del mes, se decidió que si hay algún problema al cargar una operación, se continuaría con la carga del resto de los registros:

```

1 private static void leerMes(Sistema sistema) throws FileNotFoundException
2 {
3     Scanner s = new Scanner(new FileReader("mes.txt"));
4
5     while (s.hasNextLine()) {
6         String line = s.nextLine();
7         if (line.startsWith("#")) continue;
8
9         String[] partes = line.split(",");
10        try {
11            sistema.cargarOperación(partes);
12        } catch (Exception e) {
13            System.out.println(e.getMessage());
14        }
15    }
16
17    s.close();
18 }

```

Finalmente, la configuración de la ganancia del periodo se hace leyendo desde el teclado:

```

1 private static void leerGanancia(Sistema sistema)
2 {
3     Scanner s = new Scanner(System.in);
4
5     System.out.print("Ingrese la ganancia del periodo: ");
6     int ganancia = s.nextInt();
7
8     sistema.configurarGanancia(ganancia);
9
10    System.out.println();
11 }

```

11.2.6. Clases del dominio

Aun cuando la clase `Vehículo` es abstracta, implementa mucho comportamiento. Por lo mismo, los tipos concretos solo implementan las características diferentes que tienen entre ellos:

```

1 public class Auto extends Vehículo
2 {
3     private int numPasajeros;
4
5     public Auto(int año, int precioCompra, int numPasajeros, String patente)
6     {
7         super(año, precioCompra, patente);
8         this.numPasajeros = numPasajeros;
9     }
10
11    @Override
12    public String toString() { return "Auto [numPasajeros=" + numPasajeros + "]; }
13
14    @Override
15    protected boolean aceptaCombustibleDelTipo(String tipo)
16    {
17        if (tipo.equalsIgnoreCase("G95")) return true;

```

```

18         if (tipo.equalsIgnoreCase("G97")) return true;
19         return false;
20     }
21
22     @Override
23     protected int getCantidadNeumáticos() { return 4; }
24
25     @Override
26     protected int getPrecioVenta()
27     {
28         double p = 0;
29
30         p += this.getPrecioCompra();
31         p += calcularGastosRealizados();
32         p -= getKmFinal() / 20000.0;
33         p += getGanancia();
34
35         return (int) p;
36     }
37 }

```

```

1 public class Camión extends Vehículo
2 {
3     private int tonelaje;
4     private int numNeumáticos;
5
6     public Camión(int año, int precioCompra, int tonelaje,
7                 int numNeumáticos, String patente)
8     {
9         super(año, precioCompra, patente);
10        this.tonelaje = tonelaje;
11        this.numNeumáticos = numNeumáticos;
12    }
13
14    @Override
15    public String toString() { return "Camión [tonelaje=" + tonelaje +
16                                ", numNeumáticos=" + numNeumáticos + "]";
17    }
18
19    @Override
20    protected boolean aceptaCombustibleDelTipo(String tipo)
21    {
22        return tipo.equalsIgnoreCase("D");
23    }
24
25    @Override
26    protected int getCantidadNeumáticos() { return numNeumáticos; }
27
28    @Override
29    protected int getPrecioVenta()
30    {
31        double p = 0;
32
33        p += this.getPrecioCompra();
34        p += calcularGastosRealizados() / 2.0;
35        p -= getKmFinal() / (tonelaje * 20000.0);
36        p += getGanancia();
37
38        return (int) p;
39    }
40 }

```

```

1 public class Camioneta extends Vehículo
2 {
3     private int numPasajeros;
4     private int carga;
5
6     public Camioneta(int año, int precioCompra, int numPasajeros,
7                     int carga, String patente)
8     {
9         super(año, precioCompra, patente);
10
11         this.numPasajeros = numPasajeros;
12         this.carga = carga;
13     }
14
15     @Override
16     public String toString()
17     {
18         return "Camioneta [numPasajeros=" + numPasajeros + ", carga=" + carga + "];";
19     }
20
21     @Override
22     protected boolean aceptaCombustibleDelTipo(String tipo)
23     {
24         if (tipo.equalsIgnoreCase("G95")) return true;
25         if (tipo.equalsIgnoreCase("G97")) return true;
26         return false;
27     }
28
29     @Override
30     protected int getCantidadNeumáticos() { return 4; }
31
32     @Override
33     protected int getPrecioVenta()
34     {
35         double p = 0;
36
37         p += this.getPrecioCompra();
38         p += calcularGastosRealizados() / 4.0;
39         p -= getKmFinal() / 20000.0;
40         p += getGanancia();
41
42         return (int) p;
43     }
44 }

```

De esta forma la clase `Vehículo` podrá realizar muchas de las operaciones, delegando algunas acciones a la clase concreta.

La clase se inicia declarando los atributos que todos los `Vehículos` tienen y el constructor:

```

1 public abstract class Vehículo implements Comparable<Vehículo>
2 {
3     protected int año;
4     protected int precioCompra;
5     protected String patente;
6
7     private LinkedList<Operación> operaciones = new LinkedList<Operación>();
8     private int kmInicial;
9
10    private static int ganancia;
11

```

```

12 public Vehículo(int año, int precioCompra, String patente)
13 {
14     super();
15     this.año = año;
16     this.precioCompra = precioCompra;
17     this.patente = patente;
18 }

```

Después se declaran los métodos para realizar las operaciones. Nótese que el tratamiento de cada operación es genérico, y solo se hacen un par de consultas a la clase concreta que está recibiendo el mensaje, para saber si la llamada es válida o no. Por ejemplo, al cargar combustible, todo depende de si el vehículo concreto acepta o no el tipo de combustible, así que se crea un método abstracto para que cada tipo de vehículo indique si acepta o no el combustible indicado:

```

1 public boolean cargarCombustible(int dia, int km, String tipo, int litros)
2 {
3     if (!aceptaCombustibleDelTipo(tipo)) return false;
4     Operación op = new CargaCombustible(dia, km, tipo, litros);
5     operaciones.add(op);
6     return true;
7 }
8
9 protected abstract boolean aceptaCombustibleDelTipo(String tipo);

```

En el caso de las reparaciones, simplemente se crea la instancia y se almacena:

```

1 public void reparar(int dia, int km, int duracion, int costo)
2 {
3     Operación op = new Reparación(dia, km, duracion, costo);
4     operaciones.add(op);
5 }

```

El caso del cambio de neumático también requiere preguntarle a la clase concreta de vehículo si es posible realizarlo, por lo que se agrega un método abstracto para preguntar la cantidad de neumáticos de la clase actual:

```

1 public boolean cambiarNeumáticos(int dia, int km, int cantidadNeumáticos,
2                                 int costoUnitario)
3 {
4     if (cantidadNeumáticos > getCantidadNeumáticos()) return false;
5
6     Operación op = new CambioNeumáticos(dia, km, cantidadNeumáticos, costoUnitario);
7     operaciones.add(op);
8
9     return true;
10 }
11
12 protected abstract int getCantidadNeumáticos();

```

Algunos getters:

```

1 public String getPatente() { return patente; }
2
3 public int getPrecioCompra() { return precioCompra; }

```

Uno de los métodos más importantes que todo vehículo debe tener es el cálculo de su precio de venta. Como cada tipo tiene una forma diferente de realizar el cálculo, se declara como abstracto

para forzar a que cada subclase tenga que implementarlo. Además, como en la mayoría de los casos dicho cálculo depende de los gastos realizados en las operaciones, se crea el método `protected calcularGastosRealizados()` de forma que las subclases lo puedan usar para calcular su precio de venta. Lo mismo se hizo al crear `getKmFinal()`:

```

1  protected abstract int getPrecioVenta();
2
3  protected double calcularGastosRealizados()
4  {
5      double total = 0;
6      for (Operación op : operaciones) {
7          total += op.getGasto();
8      }
9      return total;
10 }
11
12 protected double getKmFinal()
13 {
14     int km = 0;
15     for (Operación op : operaciones) {
16         int k = op.getKm();
17         if (k > km)
18             km = k;
19     }
20     return km;
21 }
```

Para permitirnos comparar y ordenar las instancias de `Vehículo`, se implementó el método de la interfaz `Comparable<Vehículo>`:

```

1  @Override
2  public int compareTo(Vehículo otro)
3  {
4      int d1 = this.getPrecioVenta() - this.getPrecioCompra();
5      int d2 = otro.getPrecioVenta() - otro.getPrecioCompra();
6
7      if (d1 < d2)
8          return 1;
9      if (d2 < d1)
10         return -1;
11     return 0;
12 }
```

El cálculo del rendimiento de cada `Vehículo` es algo que también se puede realizar al nivel de la clase base, ya que no se necesita nada particular de las clases concretas:

```

1  public double getRendimiento()
2  {
3      CargaCombustible ultCarga = getÚltimaCarga();
4      if (ultCarga == null) return 0;
5      int kms = ultCarga.getKm() - kmInicial;
6      double totalCargado = getTotalCombustibleCargado();
7
8      double rendimiento = 0;
9
10     if (totalCargado != 0) rendimiento = kms / totalCargado;
11
12     return rendimiento;
13 }
```


Como se puede ver, para calcular el rendimiento se necesita un método para obtener la última carga de combustible (`getÚltimaCarga()`) y el total de combustible cargado (`getTotalCombustibleCargado()`).

Para el caso de `getÚltimaCarga()` es necesario idear una forma de recorrer la lista general de operaciones, y solamente tomar en cuenta aquellas operaciones que son del tipo concreto `CargaCombustible`. Para realizar ésto, se le agregará a la clase base `Operación` la capacidad de aceptar un `Visitor`, y se usará para poder discriminar solo las instancias que son de interés:

```
1 public abstract class Operación
2 {
3     ...
4     protected abstract void accept(OperaciónVisitor visitor);
5 }
```

Obviamente el `Visitor` permitirá operar sobre cualquiera de las subclases:

```
1 public interface OperaciónVisitor
2 {
3     void visit(CargaCombustible c);
4     void visit(Reparación r);
5     void visit(CambioNeumáticos c);
6 }
```

Por ejemplo, el tipo concreto `CambioNeumáticos` implementará el nuevo método (lo mismo que el resto de las subclases):

```
1 public class CambioNeumáticos extends Operación
2 {
3     ...
4     @Override
5     protected void accept(OperaciónVisitor visitor) { visitor.visit(this); }
6 }
```

Por lo tanto, la operación `getÚltimaCarga()` se puede implementar así (aprovechando de utilizar una expresión lambda para ahorrar código, tal como se muestra en la página 447):

```
1 private CargaCombustible getÚltimaCarga()
2 {
3     UltimaCargaVisitor v = new UltimaCargaVisitor();
4     operaciones.forEach(op -> op.accept(v));
5     return v.getÚltimaCarga();
6 }
```

Y el `Visitor` concreto `UltimaCargaVisitor` se debe implementar así:

```
1 public class UltimaCargaVisitor implements OperaciónVisitor
2 {
3     private int maxKm = -1000;
4     private CargaCombustible carga = null;
5
6     @Override
7     public void visit(CargaCombustible c)
8     {
9         if (c.getKm() > maxKm) {
10             maxKm = c.getKm();
11             carga = c;
12         }
13     }
14 }
```

```

15  @Override
16  public void visit(Reparación r) { }
17  @Override
18  public void visit(CambioNeumáticos c) { }
19
20  public CargaCombustible getÚltimaCarga() { return carga; }
21 }

```

Para calcular el rendimiento, aun nos falta completar un método, que nos retornará la totalidad de combustible cargado en el periodo. Igual que en la operación anterior, hay que revisar las operaciones y solo tomar en cuenta las que son carga de combustible:

```

1  private int getTotalCombustibleCargado()
2  {
3      SumaLitrosCombustibleVisitor v = new SumaLitrosCombustibleVisitor();
4      operaciones.forEach(op -> op.accept(v));
5      return v.getTotalLitros();
6  }

```

El **Visitor** correspondiente solo sumará los litros de las operaciones de interés:

```

1  public class SumaLitrosCombustibleVisitor implements OperaciónVisitor
2  {
3      private int total = 0;
4
5      public int getTotalLitros() { return total; }
6
7      @Override
8      public void visit(CargaCombustible c)
9      {
10         total += c.getLitros();
11     }
12     @Override
13     public void visit(Reparación r) { }
14     @Override
15     public void visit(CambioNeumáticos c) { }
16 }

```

Finalmente, **Vehículo** contendrá algunas operaciones simples que serán necesarias para implementar el resto de las funcionalidades:

```

1  public void setKilometrajeInicial(int km) { this.kmInicial = km; }
2
3  public static void setGanancia(int ganancia)
4  {
5      Vehículo.ganancia = ganancia;
6  }
7
8  public static int getGanancia() { return ganancia; }
9
10 public boolean tieneOperaciones()
11 {
12     return operaciones.size() > 0;
13 }
14 } // Fin de Vehículo

```

Para completar el sistema, las clases que implementan las operaciones son también relativamente simples. Lo único especial que tienen es que aceptan una instancia de **Visitor**, y deben implementar el

método abstracto `getGasto()` para poder calcular el precio de venta de los vehículos. Hay operaciones que implican gasto, y hay otras que no.

```

1 public abstract class Operación
2 {
3     protected int dia;
4     protected int km;
5
6     public Operación(int dia, int km)
7     {
8         this.dia = dia;
9         this.km = km;
10    }
11
12    protected abstract double getGasto();
13
14    public int getKm() { return km; }
15
16    public int getDia() { return dia; }
17
18    protected abstract void accept(OperaciónVisitor visitor);
19 }

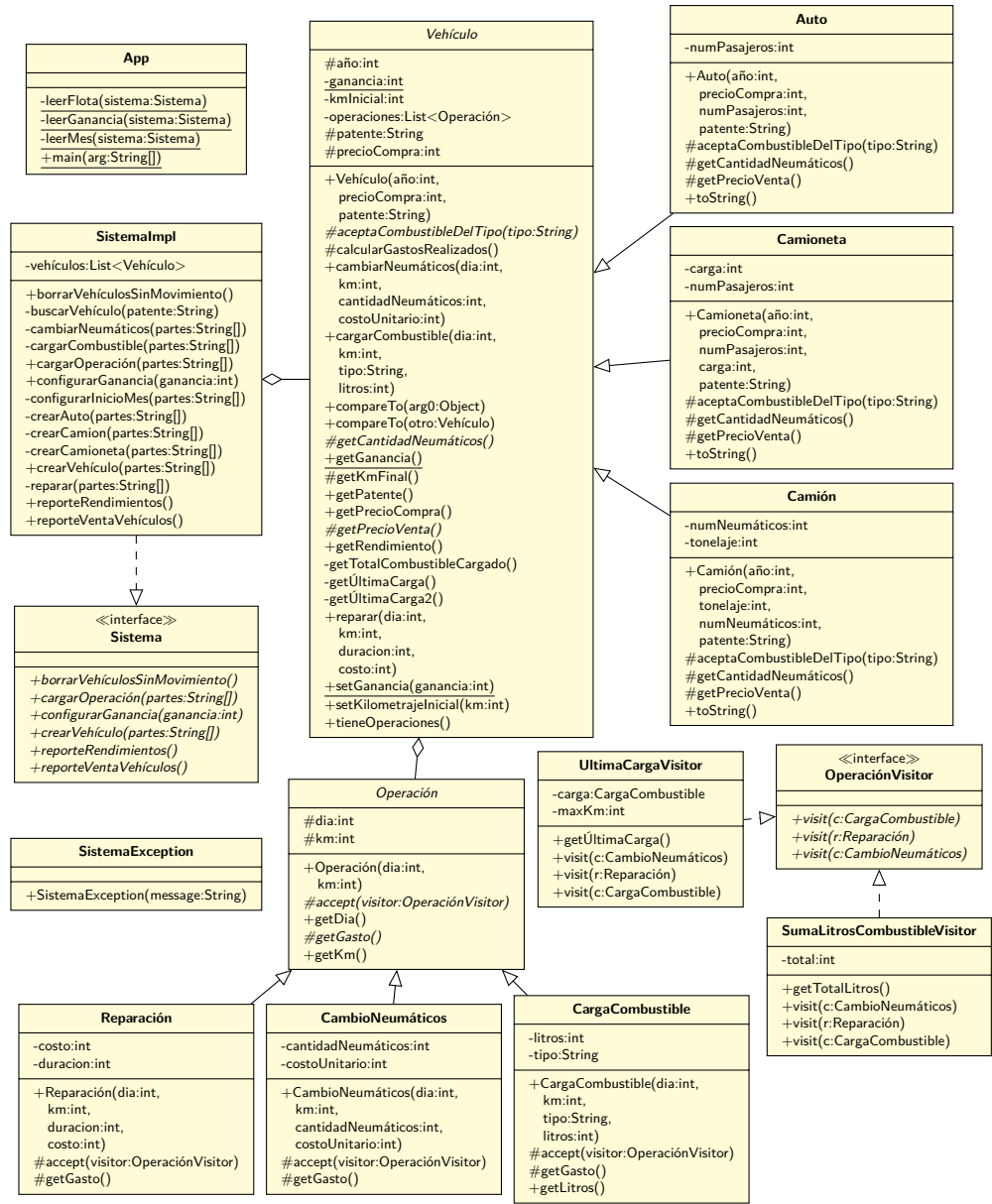
1 public class CambioNeumáticos extends Operación
2 {
3     private int cantidadNeumáticos;
4     private int costoUnitario;
5
6     public CambioNeumáticos(int dia, int km, int cantidadNeumáticos, int costoUnitario)
7     {
8         super(dia, km);
9         this.cantidadNeumáticos = cantidadNeumáticos;
10        this.costoUnitario = costoUnitario;
11    }
12
13    @Override
14    protected double getGasto() { return costoUnitario * cantidadNeumáticos; }
15
16    @Override
17    protected void accept(OperaciónVisitor visitor) { visitor.visit(this); }
18 }

1 public class CargaCombustible extends Operación
2 {
3     private String tipo;
4     private int litros;
5
6     public CargaCombustible(int dia, int km, String tipo, int litros)
7     {
8         super(dia, km);
9         this.tipo = tipo;
10        this.litros = litros;
11    }
12
13    @Override
14    protected double getGasto() { return 0; }
15
16    public int getLitros() { return litros; }
17
18    @Override
19    protected void accept(OperaciónVisitor visitor) { visitor.visit(this); }
20 }

```

```
1 public class Reparación extends Operación
2 {
3     private int duracion;
4     private int costo;
5
6     public Reparación(int dia, int km, int duracion, int costo)
7     {
8         super(dia, km);
9         this.duracion = duracion;
10        this.costo = costo;
11    }
12
13    @Override
14    protected double getGasto() { return costo; }
15
16    @Override
17    protected void accept(OperaciónVisitor visitor) { visitor.visit(this); }
18 }
```

11.2.7. Diagrama de clases actualizado



11.3. Mascotas que cambian de dueño

Se sabe que en la ciudad existen muchas clínicas veterinarias, cada una atendiendo a un sector de dicha ciudad y con muchos pacientes de diversas especies. Antes de realizar actividades con las clínicas, se necesita conocerlas para poder maximizar el impacto de dichas actividades. Para esto, se analizarán datos recopilados durante el año 2020.

En primer lugar, se tiene un archivo llamado `clinicas.txt` donde se almacena la información de todas las clínicas de la ciudad. Cada línea del archivo tiene los datos de una clínica, y los campos que contiene son RUT de la clínica, nombre de la clínica y su dirección, todos separados por coma. Por ejemplo:

```
111111,Clínica A,dirección A
222222,Clínica B,dirección B
333333,Clínica C,dirección C
444444,Clínica D,dirección D
```

Además, se tiene un archivo llamado `mascotas.txt` que contiene la lista de animales atendidos en las clínicas. Este archivo contiene en cada línea la descripción de una mascota, por ejemplo:

```
Firulais,1,Pedro,230,Ximena,330,José,400,Ximena
Rex,10,Pedro
Carlo,50,Ximena,340,Pedro
```

El primer campo es el nombre de la mascota. El resto de los campos identifican a los dueños de la mascota, ya que ésta puede cambiar de dueño en el tiempo. Cada par de número-nombre indica que a partir de ese día (del año 2020) la mascota pasó a ser propiedad de la persona indicada (considere que para efectos del programa, no hay personas con nombres repetidos, ni mascotas con nombres repetidos). En este caso, para Firulais, a partir del día 1 el dueño es Pedro, a partir del día 230 la dueña es Ximena, y José es el dueño a partir del día 330. Nótese que un dueño puede tener varias mascotas.

Finalmente, existe un archivo llamado `visitas.txt` que contiene las visitas que cada mascota hizo a las clínicas. Por ejemplo:

```
Firulais,Clínica A,250
Firulais,Clínica A,4
Firulais,Clínica C,331
Firulais,Clínica B,4
```

Cada línea indica que una mascota visitó una clínica (especificada por su nombre) en cierto día del año 2020.

11.3.1. Trabajo a realizar

1. Al iniciar el programa, se deben leer todos los archivos mencionados y cargar su información en estructuras de datos apropiadas.

- 2. Después, el programa debe borrar de sus estructuras todas aquellas clínicas que no tuvieron ninguna visita.
- 3. El programa debe mostrar el listado de clínicas, ordenado por cantidad de visitas (de menor a mayor), y mostrando para cada clínica cada visita que recibió, escribiendo el nombre de la mascota y del dueño que corresponda.
- 4. El programa debe mostrar el listado de mascotas, ordenado por la cantidad de dueños diferentes que tuvo cada una (de mayor a menor), mostrando el nombre de la mascota y la cantidad de dueños diferentes que tuvo durante el año. Para las mascotas que tuvieron la misma cantidad de dueños, el orden debe ser alfabético por su nombre.

¿Te fijaste que en el punto 3, como las mascotas cambian de dueño, una mascota puede tener varias visitas registradas, pero con diferentes dueños dependiendo de la fecha?

11.3.2. Iniciando el análisis

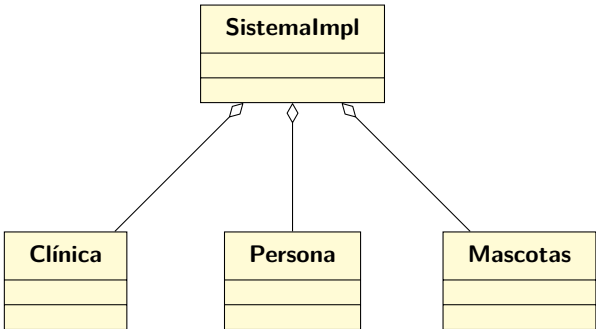
Sabemos que vamos a estructurar nuestra solución utilizando un **Sistema** con las operaciones que serán utilizadas desde la **App**.

A partir de la lectura del problema, rápidamente podemos determinar que se necesita tener un **Sistema** que permita cargar los datos que vamos a leer desde los archivos, y que nos pueda entregar los reportes que se necesitan.

Es importante considerar que se debe registrar el hecho de que las mascotas pueden cambiar de dueño, por lo que los reportes deben tomar eso en consideración.

11.3.3. Diseñando la solución

La principal decisión de diseño que se debe tomar es cómo se almacenarán las instancias. Existen varias alternativas, pero en este caso, se usará el siguiente esquema:



En otras palabras, para poder implementar las operaciones del **Sistema**, la clase **SistemaImpl** mantendrá referencias a todas las clínicas, personas y mascotas, de forma de poder crear las asociaciones entre ellas, cuando sea necesario.

11.3.4. Implementando la solución

En este ejemplo, se usará la estrategia top-down para la implementación. Esto significa que se comenzará escribiendo el código de más alto nivel primero, y al hacer eso se irán creando los objetos y métodos que se requieran para que las operaciones funcionen.

Comenzando con la clase `App`:

```

1 public class App
2 {
3     private static String folder = "/tmp/";
4
5     public static void main(String[] args) throws Exception
6     {
7         App app = new App();
8         app.iniciar();
9     }
10
11     private Sistema s = new SistemaImpl();
12
13     private void iniciar() throws Exception
14     {
15         leerClinicas(s);
16         leerMascotas(s);
17         leerVisitas(s);
18
19         s.borrarClínicasNoVisitadas();
20
21         System.out.println(s.reporteClínicas());
22         System.out.println(s.reporteMascotas());
23     }
24
25     private static void leerVisitas(Sistema s) throws Exception
26     {
27         BufferedReader r = new BufferedReader(new FileReader(folder + "visitas.txt"));
28
29         String linea;
30         while((linea = r.readLine()) != null) {
31             String[] partes = linea.split(","); // mascota,clínica,dia
32
33             int dia = Integer.parseInt(partes[2]);
34
35             if (!s.agregarVisitaMascota(partes[0], partes[1], dia)) {
36                 System.out.println("Ignorando visita de " + partes[0] +
37                                     " a " + partes[1] +
38                                     " en dia " +
39                                     dia);
40             }
41         }
42         r.close();
43     }
44
45     private static void leerMascotas(Sistema s) throws Exception
46     {
47         BufferedReader r = new BufferedReader(new FileReader(folder + "mascotas.txt"));
48
49         String linea;
50         while((linea = r.readLine()) != null) {
51             String[] partes = linea.split(",");
52

```



```

53         s.agregarMascota(partes[0]);
54
55         int p = 1;
56         while (p < partes.length) {
57             int dia = Integer.parseInt(partes[p++]);
58             String persona = partes[p++];
59
60             s.agregarPersona(persona);
61             s.agregarPertenencia(partes[0], persona, dia);
62         }
63     }
64
65     r.close();
66 }
67
68 private static void leerClinicas(Sistema s) throws Exception
69 {
70     BufferedReader r = new BufferedReader(new FileReader(folder + "clinicas.txt"));
71
72     String linea;
73     while((linea = r.readLine()) != null) {
74         String[] partes = linea.split(","); // RUT,nombre,dirección
75
76         s.agregarClínica(partes[0], partes[1], partes[2]);
77     }
78
79     r.close();
80 }
81 }

```

La clase `App` simplemente usa las operaciones de `Sistema`, pero a través de una clase concreta llamada `SistemaImpl`. Desde el punto de vista de la `App` ya todo está completado: no se requiere más, por lo que finalmente el `Sistema` que tendrá las siguientes operaciones:

```

1 /**
2  * Agrega una persona al sistema.
3  *
4  * POST:
5  * - La persona ha sido agregada al sistema.
6  *
7  * @param nombrePersona El nombre de la persona que se quiere agregar
8  * No puede ser null ni vacío.
9  * @throws SistemaException Cuando el nombre es null o vacío
10 * @return true si se agregó correctamente la persona, false si
11 * ya existía una persona con ese nombre
12 */
13 boolean agregarPersona(String nombrePersona) throws SistemaException;

```

```

1 /**
2  * Agrega una mascota al sistema
3  *
4  * POST:
5  * - La mascota ha sido agregada al sistema
6  *
7  * @param nombre El nombre de la mascota que se quiere agregar.
8  * No puede ser null ni vacío.
9  * @throws SistemaException Cuando el nombre es null o vacío
10 * @return true si se agregó correctamente la mascota, false
11 * si ya existía una mascota con ese nombre.
12 */
13 boolean agregarMascota(String nombre) throws SistemaException;

```

```

1 /**
2  * Agrega una nueva clínica al sistema
3  *
4  * POST:
5  * - La clínica ha sido agregada al sistema
6  *
7  * @param rut El RUT de la nueva clínica. No puede ser null ni vacío.
8  * @param nombre El nombre de la nueva clínica. No puede ser null ni vacío.
9  * @param dirección La dirección de la nueva clínica. No puede ser null ni vacío.
10 * @throws SistemaException Cuando rut, nombre o dirección son vacíos o null.
11 * @return true si se agregó correctamente la clínica, false si
12 * ya existía una clínica con ese rut.
13 */
14 boolean agregarClínica(String rut, String nombre, String dirección)
15     throws SistemaException;

```

```

1 /**
2  * Indica que una mascota visitó una clínica en cierto día.
3  *
4  * POST:
5  * - Se registra que la mascota visitó la clínica en cierto día
6  *
7  * @param nombreMascota El nombre de la mascota. No puede ser null ni vacío.
8  * @param nombreClínica El nombre de la clínica. No puede ser null ni vacío.
9  * @param dia El día de la visita. No puede ser negativo.
10 * @throws SistemaException Cuando no existe la mascota o la clínica,
11 * o el día es negativo o los String son vacíos o null.
12 * @return true si se agregó correctamente la visita; false si la
13 * mascota ya tenía una visita registrada para ese día (en cualquier
14 * clínica)
15 */
16 boolean agregarVisitaMascota(String nombreMascota, String nombreClínica, int dia)
17     throws SistemaException;

```

```

1 /**
2  * Registra que una mascota le pertenece a una persona a partir del día
3  * indicado.
4  *
5  * POST:
6  * - Se registra la pertenencia en el sistema.
7  *
8  * @param nombreMascota El nombre de la mascota. No puede ser null ni vacío.
9  * @param nombrePersona El nombre de la persona. No puede ser null ni vacío.
10 * @param dia El día a partir del cuál la persona es dueña de la mascota.
11 * No puede ser negativo.
12 * @throws SistemaException Cuando no existe la persona, o la mascota,
13 * o el día es negativo, o los String son vacíos o null.
14 */
15 void agregarPertenencia(String nombreMascota, String nombrePersona, int dia) throws
    SistemaException;

```

```

1 /**
2  * Borra del sistema aquellas clínicas que no recibieron ninguna
3  * visita.
4  */
5 void borrarClínicasNoVisitadas();

```

```

1 /**
2  * Retorna el reporte de clínicas.
3  *
4  * @return Un String con el reporte, con el siguiente formato:
5  *
6  * <pre>
7  * Reporte Clínicas:
8  * - Clínica C
9  * Firulais (José)
10 *
11 * - Clínica A
12 * Firulais (Ximena)
13 * Firulais (Pedro)
14 * </pre>
15 */
16 String reporteClínicas();

```

```

1 /**
2  * Retorna el reporte de mascotas.
3  *
4  * @return Un String con el reporte, con el siguiente formato:
5  *
6  * <pre>
7  * Reporte Mascotas:
8  * Firulais: 3
9  * Carlo: 2
10 * Rex: 1
11 * </pre>
12 */
13 String reporteMascotas();

```

Ahora todo está en manos de `SistemaImpl`:

```

1 public class SistemaImpl implements Sistema
2 {
3     private List<Clínica> clínicas = new LinkedList<>();
4     private List<Mascota> mascotas = new LinkedList<>();
5     private List<Persona> personas = new LinkedList<>();
6 }

```

En la clase `App` la primera operación invocada es `leerClínicas(...)` la que a la vez usa el método `agregarClínica(...)` en el `Sistema`, así que hay que implementarla:

```

1 public class SistemaImpl implements Sistema
2 {
3     ...
4     public boolean agregarClínica(String rut, String nombre, String dirección)
5         throws SistemaException
6     {
7         failIfNullOrEmpty("rut de la clínica", rut);
8         failIfNullOrEmpty("nombre de la clínica", nombre);
9         failIfNullOrEmpty("dirección de la clínica", dirección);
10
11         Clínica c = buscarClínica(nombre);
12         if (c != null) return false;
13
14         c = new Clínica(rut, nombre, dirección);
15         clínicas.add(c);
16         return true;
17     }
18 }

```

En esta implementación, y para respetar el contrato de la operación, es necesario verificar que los parámetros entregados tengan valores válidos, y para eso se creó el método `failIfNullOrEmpty(...)`:

```
1 private void failIfNullOrEmpty(String label, String str) throws SistemaException
2 {
3     if (str == null) throw new SistemaException(label + " es null");
4     if (str.length() == 0) throw new SistemaException(label + " es vacío");
5 }
```

Además se usa el método privado auxiliar `buscarClínica(...)`:

```
1 private Clínica buscarClínica(String nombreClínica)
2 {
3     for(Clínica c : clínicas) {
4         if (c.getNombre().equals(nombreClínica)) {
5             return c;
6         }
7     }
8     return null;
9 }
```

Para que todo funcione, también hay que crear la clase `Clínica` en su forma mínima:

```
1 public class Clínica
2 {
3     private String rut;
4     private String nombre;
5     private String dirección;
6
7     public Clínica(String rut, String nombre, String dirección)
8     {
9         this.rut = rut;
10        this.nombre = nombre;
11        this.dirección = dirección;
12    }
13 }
```

La siguiente acción que se realiza desde la `App` es la lectura de las mascotas, para la que se usan las operaciones `agregarPersona(...)` y `agregarPertenencia(...)` del `Sistema`:

```
1 public class SistemaImpl implements Sistema
2 {
3     ...
4     public boolean agregarPersona(String nombrePersona) throws SistemaException
5     {
6         failIfNullOrEmpty("nombre de la persona", nombrePersona);
7
8         Persona p = buscarPersona(nombrePersona);
9         if (p != null)
10             return false;
11
12         p = new Persona(nombrePersona);
13         personas.add(p);
14         return true;
15     }
16 }
```

En este caso, igual nos apoyamos en un método privado auxiliar:

```

1 private Persona buscarPersona(String nombrePersona)
2 {
3     for(Persona c : personas) {
4         if (c.getNombre().equals(nombrePersona)) {
5             return c;
6         }
7     }
8     return null;
9 }

```

El método para indicar que una *Mascota* le pertenece a una *Persona* es ligeramente más largo:

```

1 public class SistemaImpl implements Sistema
2 {
3     ...
4     public void agregarPertenencia(String nombreMascota, String nombrePersona,
5                                     int dia) throws SistemaException
6     {
7         failIfNullOrEmpty("nombre de la mascota", nombreMascota);
8         failIfNullOrEmpty("nombre de la persona", nombrePersona);
9         if (dia < 0) throw new SistemaException("Día fuera de rango");
10
11         Mascota mascota = buscarMascota(nombreMascota);
12         if (mascota == null)
13             throw new SistemaException("Mascota " + nombreMascota + " no existe");
14
15         Persona persona = buscarPersona(nombrePersona);
16         if (persona == null)
17             throw new SistemaException("Persona " + nombrePersona + " no existe");
18
19         Pertenencia p = new Pertenencia(mascota, persona, dia);
20
21         mascota.agregarPertenencia(p);
22         persona.agregarPertenencia(p);
23     }
24 }

```

En este caso, igual nos apoyamos en un nuevo método privado auxiliar:

```

1 private Mascota buscarMascota(String nombreMascota)
2 {
3     for(Mascota c : mascotas) {
4         if (c.getNombre().equals(nombreMascota)) {
5             return c;
6         }
7     }
8     return null;
9 }

```

Como se puede ver, al crear las **Pertenencias**, éstas se agregan tanto a la mascota como a la **Persona**, así que es necesario agregar dicha característica a las clases respectivas:

```

1 public class Mascota
2 {
3     private List<Pertenencia> pertenencias = new LinkedList<>();
4     private String nombre;
5
6     public Mascota(String nombre)
7     {
8         this.nombre = nombre;
9     }
10    public void agregarPertenencia(Pertenencia p) { pertenencias.add(p); }
11 }

```

```

1 public class Persona
2 {
3     private List<Pertenencia> pertenencias = new LinkedList<>();
4     private String nombre;
5
6     public Persona(String nombrePersona)
7     {
8         this.nombre = nombrePersona;
9     }
10
11    public void agregarPertenencia(Pertenencia p) { pertenencias.add(p); }
12 }

```

Desde la **App** la siguiente acción que se realiza es leer las visitas, y para completarla se usa la operación **agregarVisitaMascota(...)** del **Sistema**:

```

1 public class SistemaImpl implements Sistema
2 {
3     ...
4     public boolean agregarVisitaMascota(String nombreMascota, String nombreClínica,
5                                         int dia)
6         throws SistemaException
7     {
8         failIfNullOrEmpty("nombre de la mascota", nombreMascota);
9         failIfNullOrEmpty("nombre de la clínica", nombreClínica);
10        if (dia < 0) throw new SistemaException("Día fuera de rango");
11
12        Mascota mascota = buscarMascota(nombreMascota);
13        if (mascota == null)
14            throw new SistemaException("Mascota " + nombreMascota + " no existe");
15
16        Clínica clínica = buscarClínica(nombreClínica);
17        if (clínica == null)
18            throw new SistemaException("Clínica " + nombreClínica + " no existe");
19
20        if (mascota.yaTieneVisitaEnDia(dia)) {
21            return false;
22        }
23        Visita v = new Visita(mascota, clínica, dia);
24
25        mascota.agregarVisita(v);
26        clínica.agregarVisita(v);
27        return true;
28    }
29 }

```

En este caso, se usan varios de los métodos auxiliares ya creados, pero antes de almacenar la nueva **Visita**, se le pregunta a la **Mascota** si es que ya tenía agendada una visita ese día:

```
1 public class Mascota
2 {
3     private List<Visita> visitas = new LinkedList<>();
4
5     public boolean yaTieneVisitaEnDia(int dia)
6     {
7         for(Visita v : visitas) {
8             if (v.getDiaVisita() == dia) return true;
9         }
10        return false;
11    }
12    public void agregarVisita(Visita v) { visitas.add(v); }
13 }
```

Obviamente para que lo anterior funcione, es necesario construir la clase **Visita**:

```
1 public class Visita
2 {
3     private Mascota mascota;
4     private Clínica clínica;
5     private int diaVisita;
6
7     public Visita(Mascota mascota, Clínica clínica, int diaVisita)
8     {
9         this.mascota = mascota;
10        this.clínica = clínica;
11        this.diaVisita = diaVisita;
12    }
13
14    public Mascota getMascota() { return mascota; }
15    public Clínica getClínica() { return clínica; }
16    public int getDiaVisita() { return diaVisita; }
17 }
```

Para terminar de guardar la nueva visita, se debe modificar también la clase **Clínica**:

```
1 public class Clínica
2 {
3     ...
4     private List<Visita> visitas = new LinkedList<>();
5
6     public void agregarVisita(Visita v) { visitas.add(v); }
7 }
```

Lo siguiente que realiza la App es borrar las instancias de **Clínica** que no recibieron visitas, invocando la operación en **SistemaImpl**:

```

1 public class SistemaImpl implements Sistema
2 {
3     public void borrarClínicasNoVisitadas()
4     {
5         Iterator<Clínica> it = clínicas.iterator();
6         while (it.hasNext()) {
7             Clínica c = it.next();
8             if (c.getCantVisitas() == 0) {
9                 it.remove();
10            }
11        }
12    }
13 }

```

Por supuesto que para determinar si una **Clínica** tiene que eliminarse, tenemos que preguntarle si no ha recibido visitas, así que hay que agregar el método correspondiente:

```

1 public class Clínica
2 {
3     ...
4     public int getCantVisitas() { return visitas.size(); }
5 }

```

A continuación desde la App se invoca a **reporteClínicas()** en **SistemaImpl**:

```

1 public class SistemaImpl implements Sistema
2 {
3     ...
4     public String reporteClínicas()
5     {
6         StringBuilder reporte = new StringBuilder("Reporte Clínicas:\n");
7
8         ordenarPorCantidadVisitas(clínicas);
9
10        for(Clínica c : clínicas) {
11            reporte.append(" * ").append(c.getNombre()).append("\n")
12                .append(c.getReporteVisitas()).append("\n");
13        }
14        return reporte.toString();
15    }
16 }

```

La única complicación del reporte anterior es que tiene que entregarse ordenado, por lo que se llama al método auxiliar **ordenarPorCantidadVisitas(...)**:⁵

```

1 private void ordenarPorCantidadVisitas(List<Clínica> clínicas)
2 {
3     // Una forma de escribir un lambda que retorna algo
4     clínicas.sort((a,b) -> { return a.getCantVisitas() - b.getCantVisitas(); });
5 }

```

⁵Por si la sintaxis te es extraña, mira el punto F.2 en la página 447. Para ver cómo funciona el ordenamiento en este caso, mira H.3 en la página 465

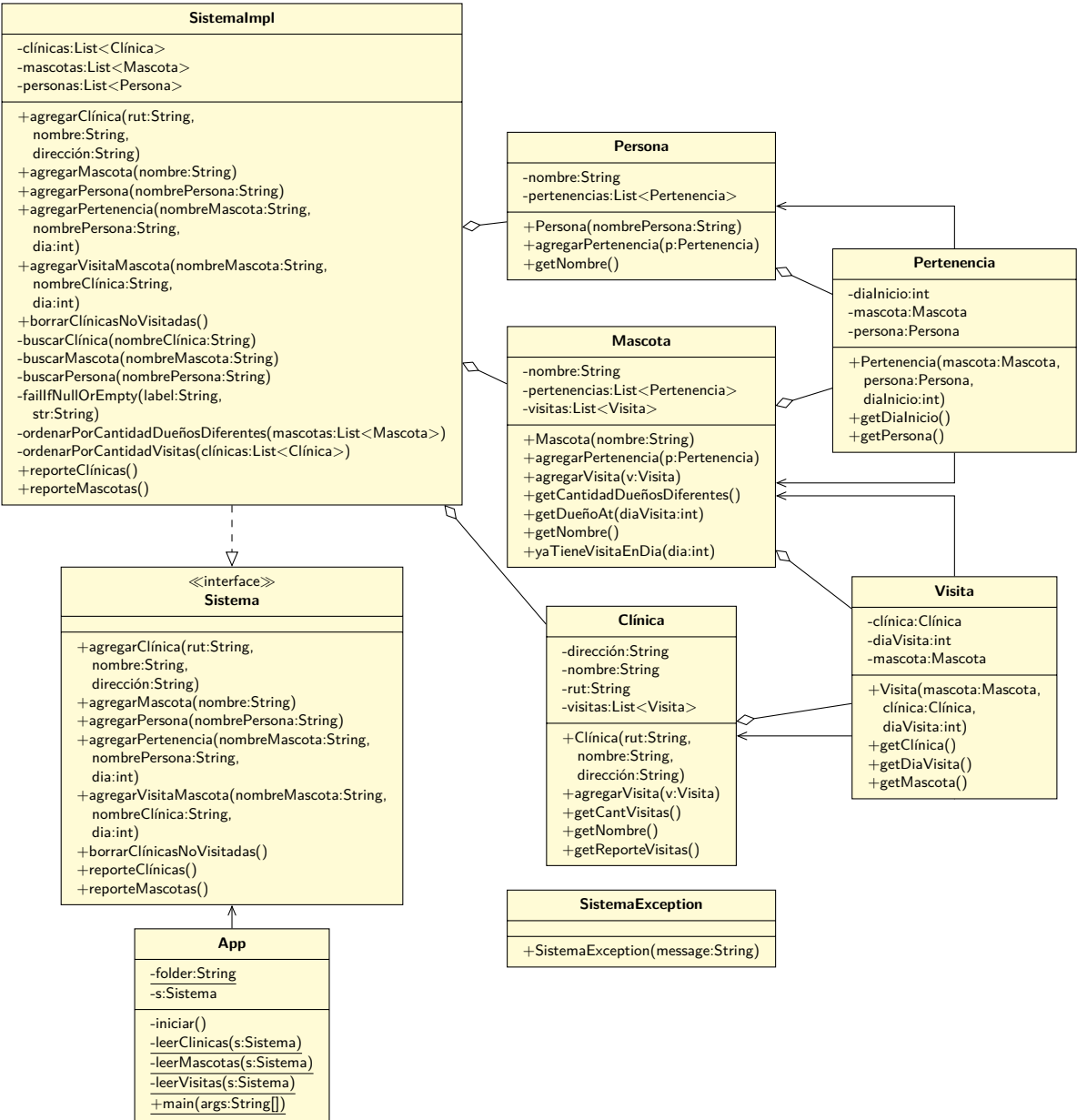
Finalmente la `App` invoca a `reporteMascotas(...)` en `SistemaImpl`:

```
1 public class SistemaImpl implements Sistema
2 {
3     ...
4     public String reporteMascotas()
5     {
6         StringBuilder reporte = new StringBuilder("Reporte Mascotas:\n");
7
8         ordenarPorCantidadDueñosDiferentes(mascotas);
9
10        for(Mascota m : mascotas) {
11            reporte.append(m.getNombre())
12                .append(": ")
13                .append(m.getCantidadDueñosDiferentes())
14                .append("\n");
15        }
16        return reporte.toString();
17    }
18 }
```

En este caso, también hay que ordenar antes de construir el reporte:

```
1 private void ordenarPorCantidadDueñosDiferentes(List<Mascota> mascotas)
2 {
3     // Otra forma de escribir un lambda que retorna algo
4     mascotas.sort((a,b) -> b.getCantidadDueñosDiferentes() - a.getCantidadDueñosDiferentes());
5 }
```

11.3.5. Diagrama de clases actualizado



11.4. Mini evaluador

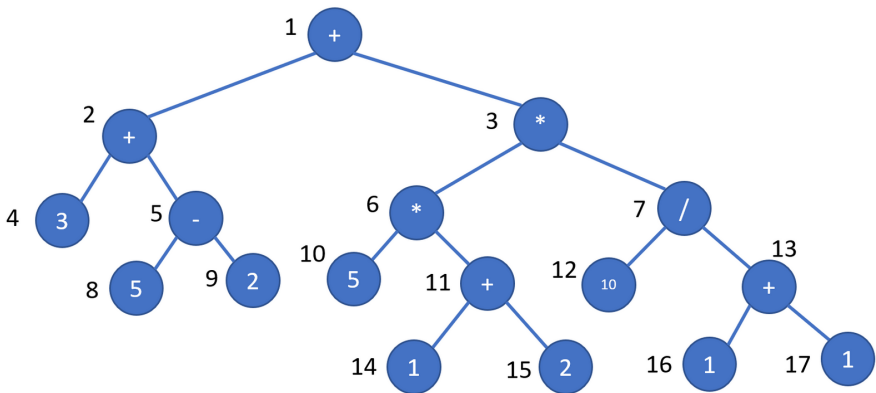
Cada vez que en alguna aplicación tenemos que ingresar números y operaciones, éstas se escriben como textos normales, y es misión de la aplicación convertir dicho texto a algo que pueda ser evaluado, para obtener el resultado que se anda buscando.

Para hacer esto hay muchas técnicas, pero muchas basan su funcionamiento en que las expresiones matemáticas se pueden expresar de la siguiente forma:

Por ejemplo, esta expresión:

$$(3 + (5 - 2)) + ((5 * (1 + 2)) * (10 / (1 + 1)))$$

se puede representar con esta estructura:



Entonces, si le pedimos al nodo 1 que entregue su resultado, éste le pedirá a su lado izquierdo que entregue su resultado, después al lado derecho, y aplicará la operación (con los valores retornados previamente) y retornará el resultado. Y así, sucesivamente, cada nodo realiza el mismo proceso.

Como la operación de convertir una expresión matemática a la estructura mostrada en el ejemplo es tema de un libro más avanzado, se entregará un archivo (llamado `input.txt`) con este formato:

```
14 N 1
15 N 2
16 N 1
17 N 1
8 N 5
9 N 2
10 N 5
11 SUMA 14 15
12 N 10
13 SUMA 16 17
4 N 3
```

```
5 RESTA 8 9
6 MULT 10 11
7 DIV 12 13
2 SUMA 4 5
3 MULT 6 7
1 SUMA 2 3
```

Cada línea representa la definición de un nodo de la expresión. Por ejemplo:

```
14 N 1
```

indica que existe el nodo número **14**, y que su tipo corresponde a un número y que su valor es **1**. Por otro lado:

```
13 SUMA 16 17
```

indica que existe el nodo número **13**, y que su tipo corresponde a una operación de suma, y que debe sumar los valores que retornan los nodos **16** y **17**. Algo similar sucede con el resto de las operaciones. Las operaciones válidas para este problema son **SUMA**, **RESTA**, **MULT** y **DIV**.

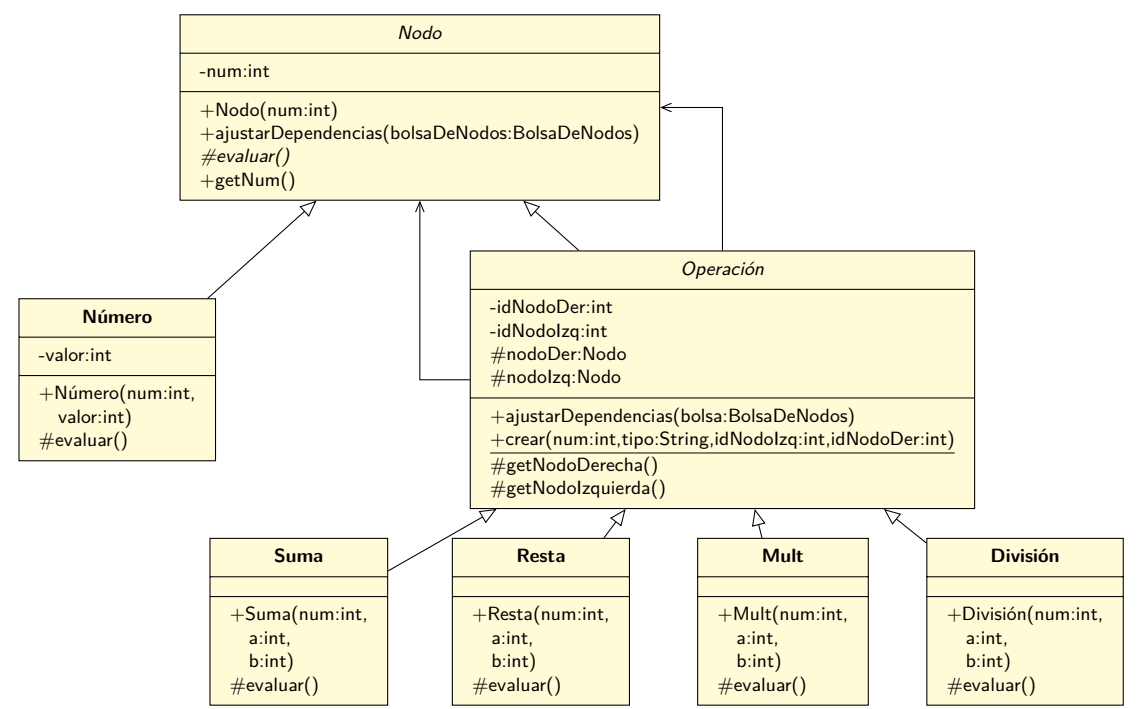
Construya un programa que lea el archivo `input.txt` y que entregue por pantalla el valor del último nodo indicado en el archivo (en el caso del ejemplo, es el nodo **1**, pero podrían darse otros casos).

En caso de que no se pueda realizar la evaluación (por ejemplo, porque un nodo no tiene sus nodos inferiores), se debe escribir por pantalla un mensaje apropiado.

11.4.1. Iniciando el análisis

Para poder resolver este problema, es necesario darse cuenta que el proceso de evaluar un nodo es igual que evaluar los nodos que están a la izquierda y a la derecha de éste. En el caso de la figura de arriba, para evaluar el valor del nodo 1, es necesario evaluar su izquierda y su derecha (nodos 2 y 3), y la evaluación de esos nodos procede igual, evaluando a izquierda y derecha. Lo único que hay que tener en consideración es que algunos nodos puede retornar su valor directamente, sin necesidad de preguntarle a sus nodos inferiores (de hecho, esos nodos no tienen nodos inferiores).

Todo lo anterior nos lleva a pensar que podemos estructurar la solución basándonos en una clase `Nodo`, que tenga varios subtipos que se comporten cada uno de acuerdo a lo que se dijo antes: algunos `Nodos` tienen un valor intrínseco, otros `Nodos` necesitan de otros `Nodos` para calcular su valor.



Todo `Nodo` tendrá un método `evaluar()` que retornará el valor de dicha instancia. La clase `Nodo` es abstracta, y tendrá una subclase que representará a todas las operaciones posibles, y las operaciones concretas serán subclases de dicha superclase. Para simplificar un poco más lo que tenemos que hacer, también crearemos una clase `Operación`, que será la superclase de todas las operaciones.

11.4.2. Diseñando la solución

Además de definir la jerarquía de `Nodos`, es necesario determinar cómo realmente realizar la asociación entre `Nodos`. Toda la información vendrá desde el archivo, el que contendrá solo números, pero en algún momento hay que convertir dicho número en una referencia a un `Nodo` en particular. Más aun,

es posible que al leer el archivo se defina un nodo con una operación, pero los nodos dependientes aun no aparecen en el archivo.

Lo anterior limita la forma en que se debe cargar la estructura: no basta asumir que si se tiene que crear un nodo `Suma`, referenciando a los `Nodos` 1 y 2, esas instancias de `Nodo` ya han sido cargadas y creadas.

Para protegernos de lo anterior, a medida que se lea el archivo, las instancias de `Nodo` (o sus subclases) se crearán, pero guardarán solo los identificadores de sus dependencias. Todos los nodos creados se guardarán en una “bolsa” de instancias de `Nodo`. Entonces, cuando llegue el momento adecuado, invocaremos a algún método que cause que los `Nodos`, a partir de los identificadores, se enlacen a las instancias correctas de `Nodo`, para después poder realizar las operaciones. En otras palabras, los nodos que tengan dependencias resolverán dichas dependencias solo al momento de evaluarlos. Implementaremos esta solución, pero nótese que hay otras formas de enfrentar el problema.

Desde el punto de vista del `Sistema`, lo que diseñaremos será muy simple: solo será necesario crear dos operaciones:

```

1 public interface Sistema
2 {
3     /**
4      * Agrega un nuevo nodo al sistema.
5      *
6      * @param partes La especificación del nodo a agregar, de acuerdo al
7      * siguiente orden:
8      *
9      * [0]: El número del nodo
10     * [1]: El tipo del nodo (N, SUMA, RESTA, MULT, DIV)
11     *
12     * Si el tipo es N:
13     * [2]: El valor del nodo
14     *
15     * Si el tipo es diferente a N:
16     * [2]: El número del nodo a la izquierda de este nodo
17     * [3]: El número del nodo a la derecha de este nodo.
18     *
19     * @throws SistemaException En caso de que no se pueda agregar
20     * un nuevo nodo al Sistema.
21     */
22     void agregarNodo(String[] partes) throws SistemaException;
23
24     /**
25      * Calcula y retorna el valor del último nodo agregado.
26      *
27      * @return El valor que se obtiene al evaluar el último nodo agregado.
28      * @throws SistemaException En caso de que no se pueda realizar
29      * la evaluación.
30     */
31     double evaluar() throws SistemaException;
32 }

```

11.4.3. Implementando la solución

Comenzando con la App:

```

1 public class Evaluador
2 {
3     private static Sistema sistema;
4
5     public static void main(String[] args)
6         throws FileNotFoundException, SistemaException
7     {
8         sistema = new SistemaImpl();
9
10        leerArchivo(sistema);
11
12        try {
13            System.out.println(sistema.evaluar());
14        } catch (SistemaException e) {
15            System.out.println("No se puede calcular");
16        }
17    }
18
19    private static void leerArchivo(Sistema sist)
20        throws FileNotFoundException, SistemaException
21    {
22        String base = "";
23        Scanner s = new Scanner(new File(base + "input.txt"));
24
25        while (s.hasNextLine()) {
26            String linea = s.nextLine();
27
28            String[] partes = linea.split(" ");
29
30            sist.agregarNodo(partes);
31        }
32        s.close();
33    }
34 }

```

La clase `Nodo` es muy simple:

```

1 public abstract class Nodo
2 {
3     private int num;
4
5     public Nodo(int num) { this.num = num; }
6
7     public int getNum() { return num; }
8
9     protected abstract double evaluar();
10 }

```

Y la clase `Número` también:

```

1 public class Número extends Nodo
2 {
3     private int valor;
4     public Número(int num, int valor)
5     {
6         super(num);
7         this.valor = valor;
8     }
9     @Override
10    protected double evaluar() { return valor; }
11 }

```

La clase `Operación` es más complicada, ya que debe proveer algunas funcionalidades a sus subclases:

```

1 public abstract class Operación extends Nodo
2 {
3     private int idNodoIzq, idNodoDer;
4     private BolsaDeNodos bolsaDeNodos;
5
6     protected Operación(BolsaDeNodos bolsaDeNodos, int num, int idNodoIzq, int idNodoDer)
7     {
8         super(num);
9
10        this.bolsaDeNodos = bolsaDeNodos;
11
12        this.idNodoIzq = idNodoIzq;
13        this.idNodoDer = idNodoDer;
14    }
15
16    protected Nodo nodoIzq, nodoDer;
17
18    protected Nodo getNodeIzquierda()
19    {
20        if (this.nodoIzq == null) nodoIzq = bolsaDeNodos.buscarNodo(this.idNodoIzq);
21        return this.nodoIzq;
22    }
23
24    protected Nodo getNodeDerecha()
25    {
26        if (this.nodoDer == null) nodoDer = bolsaDeNodos.buscarNodo(this.idNodoDer);
27        return this.nodoDer;
28    }
29 }

```

Y las 4 operaciones se definen de manera muy similar. La única diferencia real es la operación concreta que se realiza en el método `evaluar()`. Nótese que `getNodeDerecha()` y `getNodeIzquierda` están definidos en `Operación`, y que dichos métodos se encargan de asegurar que las referencias a los nodos estén resueltas.


```
1 public class Suma extends Operación
2 {
3     public Suma(BolsaDeNodos bolsaDeNodos, int num, int a, int b)
4     {
5         super(bolsaDeNodos, num, a, b);
6     }
7
8     @Override
9     protected double evaluar()
10    {
11        return getNodoIzquierda().evaluar() + getNodoDerecha().evaluar();
12    }
13 }
```

```
1 public class Resta extends Operación
2 {
3     public Resta(BolsaDeNodos bolsaDeNodos, int num, int a, int b)
4     {
5         super(bolsaDeNodos, num, a, b);
6     }
7
8     @Override
9     protected double evaluar()
10    {
11        return getNodoIzquierda().evaluar() - getNodoDerecha().evaluar();
12    }
13 }
```

```
1 public class Mult extends Operación
2 {
3     public Mult(BolsaDeNodos bolsaDeNodos, int num, int a, int b)
4     {
5         super(bolsaDeNodos, num, a, b);
6     }
7
8     @Override
9     protected double evaluar()
10    {
11        return getNodoIzquierda().evaluar() * getNodoDerecha().evaluar();
12    }
13 }
```

```
1 public class División extends Operación
2 {
3     public División(BolsaDeNodos bolsaDeNodos, int num, int a, int b)
4     {
5         super(bolsaDeNodos, num, a, b);
6     }
7
8     @Override
9     protected double evaluar()
10    {
11        return getNodoIzquierda().evaluar() / getNodoDerecha().evaluar();
12    }
13 }
```

Respecto a la implementación del **Sistema**, ya tenemos todo para poder completar las operaciones que se diseñaron:

```

1 public class SistemaImpl implements Sistema
2 {
3     private BolsaDeNodos nodos = new BolsaDeNodos();
4
5     @Override
6     public void agregarNodo(String[] partes)
7     {
8         int num = Integer.valueOf(partes[0]);
9         String tipo = partes[1];
10        Nodo nodo;
11
12        if ("N".equals(tipo)) {
13            int valor = Integer.valueOf(partes[2]);
14            nodo = new Número(num, valor);
15        } else {
16            int a = Integer.valueOf(partes[2]);
17            int b = Integer.valueOf(partes[3]);
18            nodo = // Hay que construir el tipo concreto!
19        }
20        nodos.add(nodo);
21    }
22
23    @Override
24    public double evaluar() throws SistemaException
25    {
26        if (nodos.size() == 0) throw new SistemaException("No existen nodos");
27
28        Nodo n = nodos.ultimoNodo();
29        return n.evaluar();
30    }
31 }

```

Pero nos faltaba implementar la “bolsa” de Nodos. Para ellos nos vamos a aprovechar de algunos métodos que la clase **LinkedList** ya tiene implementados:

```

1 public class BolsaDeNodos
2 {
3     private LinkedList<Nodo> nodos = new LinkedList<>();
4
5     public void add(Nodo nodo) { nodos.add(nodo); }
6
7     public Nodo buscarNodo(int num) {
8         for(Nodo n : nodos) {
9             if (n.getNum() == num) return n;
10        }
11        return null;
12    }
13
14    public Nodo ultimoNodo() { return nodos.getLast(); }
15
16    public int size() { return nodos.size(); }
17 }

```

Solamente nos falta solucionar un detalle: en `SistemaImpl` dejamos esto:

```

1 public class SistemaImpl implements Sistema
2 {
3     @Override
4     public void agregarNodo(String[] partes)
5     {
6         int num = Integer.valueOf(partes[0]);
7         String tipo = partes[1];
8         Nodo nodo;
9
10        if ("N".equals(tipo)) {
11            ...
12        } else {
13            int a = Integer.valueOf(partes[2]);
14            int b = Integer.valueOf(partes[3]);
15            nodo = // Hay que construir el tipo concreto!
16        }
17        nodos.add(nodo);
18    }
19    ...
20 }

```

Falta escribir el código que nos permitirá instanciar cada clase concreta de `Operación`. La forma más inocente de hacer eso es llenarnos de `if`, pero podemos hacerlo mejor. Deleguemos la creación del tipo concreto a la misma `Operación`:

```

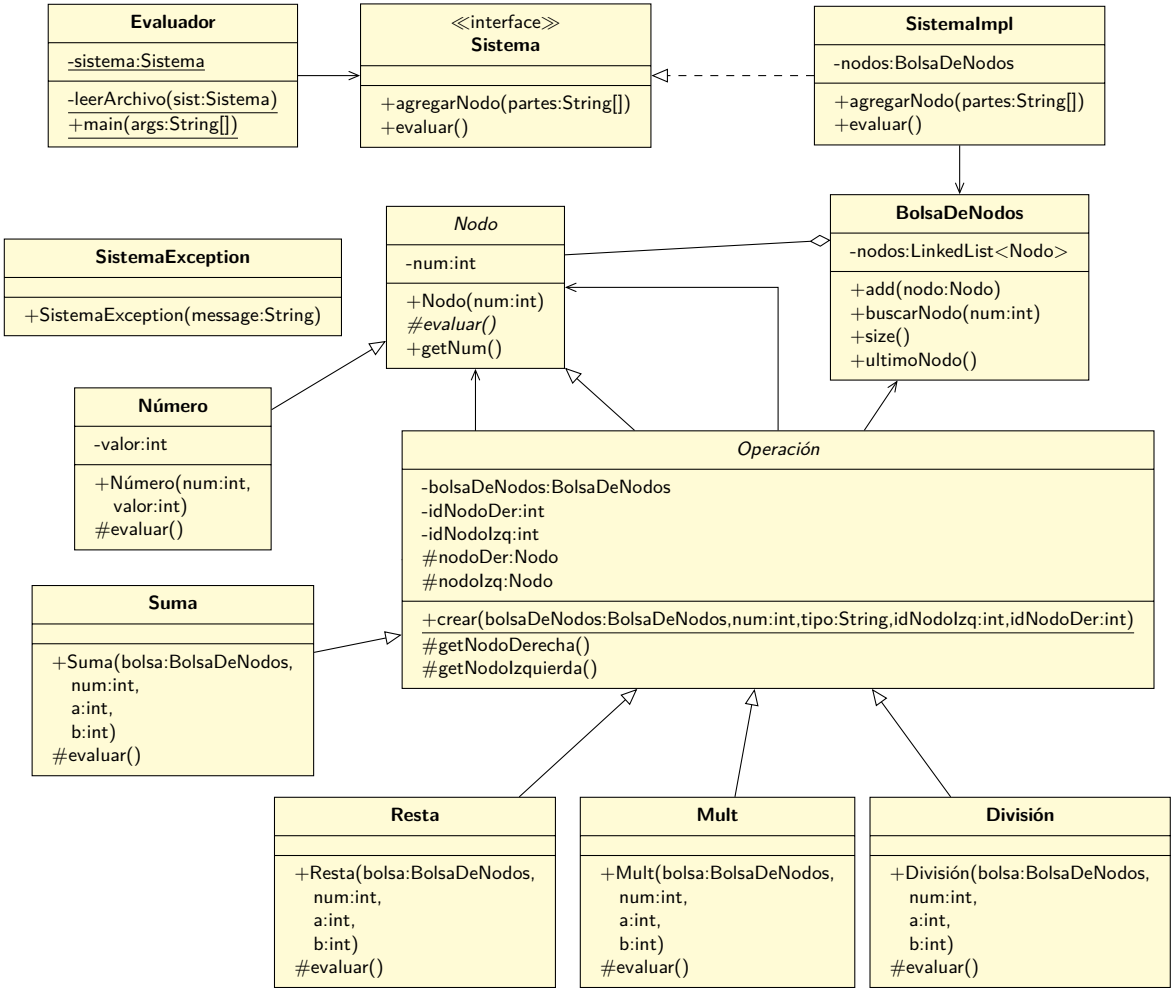
1 public abstract class Operación extends Nodo
2 {
3     public static Nodo crear(BolsaDeNodos bolsaDeNodos, int num, String tipo,
4                             int idNodoIzq, int idNodoDer)
5     {
6         if ("SUMA".equals(tipo)) {
7             return new Suma(bolsaDeNodos, num, idNodoIzq, idNodoDer);
8
9         } else if ("RESTA".equals(tipo)) {
10            return new Resta(bolsaDeNodos, num, idNodoIzq, idNodoDer);
11
12        } else if ("MULT".equals(tipo)) {
13            return new Mult(bolsaDeNodos, num, idNodoIzq, idNodoDer);
14
15        } else if ("DIV".equals(tipo)) {
16            return new División(bolsaDeNodos, num, idNodoIzq, idNodoDer);
17        }
18        return null;
19    }
20    ...
21 }

```

De forma que la clase `SistemaImpl` no quede acoplada a los tipos específicos de `Nodo`:

```
1 public class SistemaImpl implements Sistema
2 {
3     @Override
4     public void agregarNodo(String[] partes)
5     {
6         int num = Integer.valueOf(partes[0]);
7         String tipo = partes[1];
8         Nodo nodo;
9
10        if ("N".equals(tipo)) {
11            ...
12        } else {
13            int a = Integer.valueOf(partes[2]);
14            int b = Integer.valueOf(partes[3]);
15            nodo = Operación.crear(nodos, num, tipo, a, b);
16        }
17        nodos.add(nodo);
18    }
19    ...
20 }
```

11.4.4. Diagrama de clases actualizado



11.5. Vehículos de diferentes tipos

Se requiere construir un programa que maneje información de diferentes tipos de vehículos. Sabemos que todos los vehículos tienen patente, marca y año de fabricación.

Por otro lado, se sabe que hay varios tipos de vehículos, en particular, interesarán los autos y las camionetas. Los autos, además de las características que tienen todos los vehículos, poseen los kilómetros recorridos durante su vida, y la capacidad del estanque de combustible. Las camionetas tienen una capacidad de carga, además de las características propias de todos los vehículos.

El programa debe mostrar un menú, que permita realizar las siguientes acciones:

1. Ingresar vehículos. No se sabe cuántos vehículos se van a ingresar, pero se debe permitir que se ingresen tanto autos como camionetas.
2. Mostrar el listado de todos los vehículos ingresados, en el mismo orden en que fueron leídos.
3. Dada la patente de un vehículo, se deben mostrar sus datos. Además de sus datos normales, se debe mostrar su costo de reparación, el cual se calcula diferente dependiendo de si el vehículo es auto o camioneta: para las camionetas, el costo de reparación es \$100.000 por unidad de capacidad de carga; para los autos es \$20.000 por la capacidad total del estanque de combustible.
4. Escribir la cantidad total de vehículos, la cantidad total de autos y la cantidad total de camionetas.
5. Escribir por pantalla los datos de la camioneta con mayor capacidad de carga.

11.5.1. Analizando el problema

A partir del enunciado del problema, leyendo con cuidado los requerimientos, y aplicando las reglas que se han elegido para la construcción de aplicaciones, se puede determinar que se requiere que desde nuestra **App** podamos instanciar un **Sistema** que tenga las operaciones necesarias para ingresar datos, y después poder consultar el estado de todo lo que ha sido almacenado.

Aun cuando el problema se trata de almacenar dos tipos diferentes de “cosas”, en el fondo ambos tipos tienen un comportamiento común: seguramente se utilizará herencia para crear los objetos donde se almacenarán los datos, pero por ahora, procederemos construyendo la solución desde el punto de vista del **Sistema** y su cliente.

11.5.2. Diseñando la solución

De acuerdo a lo visto anteriormente, el `Sistema` estará definido de esta manera:

```

1 public interface Sistema
2 {
3     /**
4      * Ingresa un auto al sistema.
5      *
6      * POST: Auto ingresado al sistema.
7      *
8      * @param patente La patente del nuevo auto. No puede ser null ni vacío.
9      * @param marca La marca del nuevo auto. No puede ser null ni vacío.
10     * @param año El año del nuevo auto. Tiene que ser mayor que 2000.
11     * @param kilómetros Los kilómetros del nuevo auto. Tiene que ser mayor que cero.
12     * @param capacidadEstanque La capacidad del estanque del nuevo auto.
13     * Tiene que ser mayor que cero.
14     * @return True si el auto fue ingresado al sistema. False en caso contrario.
15     */
16     boolean ingresarAuto(String patente, String marca, int año, int kilómetros,
17                          int capacidadEstanque);
18
19     /**
20      * Ingresa una camioneta al sistema
21      *
22      * POST: Camioneta ingresada al sistema
23      *
24      * @param patente La patente de la nueva camioneta. No puede ser null ni vacío.
25      * @param marca La marca de la nueva camioneta. No puede ser null ni vacío.
26      * @param año El año de la nueva camioneta. Tiene que ser mayor que 2000.
27      * @param capacidadCarga La capacidad de carga de la nueva camioneta.
28      * Tiene que ser mayor que cero.
29      * @return True si la camioneta fue ingresada, False en caso contrario
30      */
31     boolean ingresarCamioneta(String patente, String marca, int año, int capacidadCarga);
32
33     /**
34      * Obtiene los datos de todos los vehículos del sistema.
35      *
36      * @return Retorna un string con los datos de todos los vehículos del sistema.
37      */
38     String obtenerDatosVehículos();
39
40     /**
41      * Obtiene los datos de un vehículo en particular.
42      *
43      * @param patente La patente del vehículo de interés.
44      * @return Retorna un String con los datos del vehículo de interés.
45      * @throws IllegalArgumentException si no existe un vehículo con la patente
46      * indicada.
47      */
48     String obtenerDatosVehículo(String patente);
49
50     /**
51      * @return Retorna la cantidad de autos actualmente en el sistema
52      */
53     int obtenerCantidadAutos();
54
55
56
57

```

```

58  /**
59   * @return Retorna la cantidad de camionetas que actualmente están en el sistema
60   */
61   int obtenerCantidadCamionetas();
62
63  /**
64   * @return Retorna la cantidad de vehículos total en el sistema
65   */
66   int obtenerCantidadVehículos();
67
68  /**
69   * Retorna los datos de la camioneta con mayor capacidad de carga.
70   *
71   * @return Los datos de la camioneta con mayor capacidad de carga. Retorna
72   * null si es que no hay ninguna camioneta.
73   */
74   String obtenerDatosCamionetaMasCarga();
75 }

```

11.5.3. Implementando la solución

Sabemos que la interfaz solo define las operaciones del `Sistema`, y que eventualmente se creará una clase que implemente dicha interfaz (y que seguramente se llamará `SistemaImpl`), pero por ahora, comenzaremos escribiendo la `App` de manera top-down:

```

1 public class App
2 {
3     private Sistema s;
4
5     public static void main(String[] args)
6     {
7         s = null;
8         menu(s);
9     }
10 }

```

El método `menu(...)` es el que ejecutará el ciclo de lectura desde el teclado y ejecutará las acciones de acuerdo a lo que ingrese el usuario. Su implementación se puede generalizar así:

```

1 private static void menu(Sistema s)
2 {
3     Scanner scanner = new Scanner(System.in);
4
5     // Muestra el menú de opciones y retorna la opción elegida
6     int opción = mostrarMenú(scanner);
7
8     // La opción CERO significa salir del programa!
9     while (opción != 0) {
10         ejecutar(opción, s, scanner);
11
12         opción = mostrarMenú(scanner);
13     }
14 }

```


Para mostrar el menú, podemos hacer lo siguiente:

```

1 private static int mostrarMenú(Scanner scanner)
2 {
3     System.out.println(" ");
4     System.out.println(" M E N U");
5     System.out.println("[1] Ingresar vehiculo ");
6     System.out.println("[2] Listado de vehiculos ");
7     System.out.println("[3] Buscar patente y desplegar costo de reparacion");
8     System.out.println("[4] Cantidad de vehiculos (autos y camionetas) ");
9     System.out.println("[5] Datos de la camioneta con la mayor capacidad de carga ");
10    System.out.println("[0] Salir ");
11
12    System.out.println("Ingrese opción: ");
13    int opción = Integer.parseInt(scanner.nextLine());
14    return opción;
15 }

```

Nótese que se requiere pasar el `Scanner` como parámetro para poder leer la opción que se ingresa desde el teclado.

Para poder “ejecutar” la opción ingresada, se requiere discriminar cuál fue la opción, y actuar de acuerdo al número ingresado:

```

1 private static void ejecutar(int opción, Sistema sistema, Scanner scanner)
2 {
3     switch (opción) {
4     case 1:
5         leerVehículo(sistema, scanner);
6         break;
7     case 2:
8         mostrarVehículos(sistema);
9         break;
10    case 3:
11        buscarUnaPatente(sistema, scanner);
12        break;
13    case 4:
14        mostrarTotales(sistema);
15        break;
16    case 5:
17        mostrarCamionetaEspecial(sistema);
18        break;
19    }
20 }

```

Para el caso de ingresar un vehículo, se requiere además discriminar el tipo de vehículo que se quiere ingresar:

```

1 private static void leerVehículo(Sistema sistema, Scanner scanner)
2 {
3     System.out.println("Tipo de Vehículo: [A]uto [C]amioneta: ");
4     String tipo = scanner.nextLine();
5
6     if (tipo.equals("A")) {
7         leerAuto(sistema, scanner);
8     } else {
9         leerCamioneta(sistema, scanner);
10    }
11 }

```

Ahora el código puede concentrarse en ingresar el tipo específico de vehículo. Para el caso de querer ingresar un “auto” podríamos escribir lo siguiente:

```

1 private static void leerAuto(Sistema sistema, Scanner scanner)
2 {
3     String patente = leerString("Patente: ", scanner);
4     String marca = leerString("Marca: ", scanner);
5     int año = leerInt("Año: ", scanner);
6     int km = leerInt("Km: ", scanner);
7     int capacidad = leerInt("Capacidad estanque: ", scanner);
8
9     boolean x = sistema.ingresarAuto(patente, marca, año, km, capacidad);
10    if (!x) {
11        System.out.println("No se pudo realizar el ingreso");
12    }
13 }

```

Nos apoyaremos en un par de métodos para poder fácilmente pedir datos desde el teclado:

```

1 private static int leerInt(String título, Scanner scanner)
2 {
3     System.out.println(título);
4     int num = Integer.parseInt(scanner.nextLine());
5     return num;
6 }
7
8 private static String leerString(String título, Scanner scanner)
9 {
10    System.out.println(título);
11    String txt = scanner.nextLine();
12    return txt;
13 }

```

Para el otro tipo de vehículo:

```

1 private static void leerCamioneta(Sistema sistema, Scanner scanner)
2 {
3     String patente = leerString("Patente: ", scanner);
4     String marca = leerString("Marca: ", scanner);
5     int año = leerInt("Año: ", scanner);
6     int capacidad = leerInt("Capacidad carga: ", scanner);
7
8     boolean x = sistema.ingresarCamioneta(patente, marca, año, capacidad);
9     if (!x) {
10        System.out.println("No se pudo realizar el ingreso");
11    }
12 }

```

Para el caso de la opción 2 del menú, se puede implementar simplemente pidiéndolo al **Sistema** que nos entregue los datos de todos los vehículos:

```

1 private static void mostrarVehiculos(Sistema sistema)
2 {
3     System.out.println(sistema.obtenerDatosVehiculos());
4 }

```

Para la opción 3, se requiere solicitar primero que se ingrese una patente en particular, y después se le pide al **Sistema** que nos indique los datos de aquella:

```

1  private static void buscarUnaPatente(Sistema sistema, Scanner scanner)
2  {
3      String patente = leerString("Patente: ", scanner);
4
5      try {
6          System.out.println(sistema.obtenerDatosVehículo(patente));
7      } catch (IllegalArgumentException e) {
8          System.out.println(e.getMessage());
9      }
10 }

```

De acuerdo al contrato de la operación `obtenerDatosVehículo(..)`, si la patente no existe se lanza una `IllegalArgumentException`, así que es necesario protegernos utilizando un `try/catch`.

Para la opción 4, solo le consultamos al `Sistema`:

```

1  private static void mostrarTotales(Sistema sistema)
2  {
3      System.out.println("Cantidad de vehículos: " + sistema.obtenerCantidadVehículos());
4      System.out.println("Cantidad de autos: " + sistema.obtenerCantidadAutos());
5      System.out.println("Cantidad de camionetas: " + sistema.obtenerCantidadCamionetas());
6      ;
7  }

```

Y para la opción 5, nuevamente solo le pedimos el dato al `Sistema`, tomando en consideración que el contrato dice que si no hay camionetas, se retorna `null`:

```

1  private static void mostrarCamionetaEspecial(Sistema sistema)
2  {
3      String datos = sistema.obtenerDatosCamionetaMasCarga();
4      if (datos != null) {
5          System.out.println(datos);
6      }
7  }

```

Nótese que hasta ahora se ha implementado la `App` en su totalidad, y técnicamente el proyecto debería compilar, pero no se ejecutará de forma exitosa. Nótese también que la `App` se ha construido completamente tomando en cuenta el contrato del `Sistema`, pero aun no se ha implementado de forma concreta ninguna clase que implemente dicha interfaz. A este fenómeno se le llama “programar contra la interfaz”. En realidad, no importa el tipo concreto de la clase, solo importa que implemente la interfaz indicada (en este caso, la interfaz `Sistema`).

De hecho, si nos damos cuenta, hasta ahora ni siquiera se han implementado las clases para almacenar los vehículos (autos y camionetas). Ésto pone de manifiesto que dichas clases no son relevantes para la `App`, sino que son algo que el `Sistema` utiliza, pero que para los clientes del `Sistema` no deberían ser visibles (de acuerdo a las restricciones auto-impuestas en la arquitectura que se está siguiendo).⁶

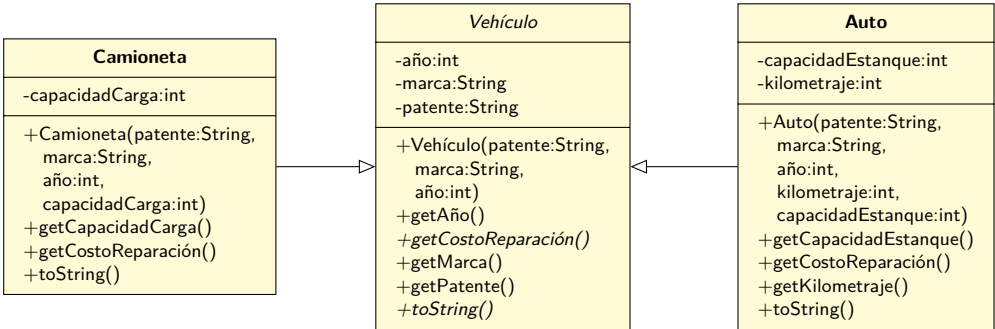
⁶De hecho, sería un buen ejercicio construir *otra* versión de `Sistema`, que implemente las operaciones, pero de otra forma, incluso sin usar herencia.

11.5.4. Implementando el Sistema

La implementación del Sistema requiere crear una clase que implemente la interfaz:

```
1 public class SistemaImpl implements Sistema
2 {
3 }
```

Como el Sistema debe almacenar una serie de vehículos, y aunque hemos pospuesto la decisión hasta ahora, será necesario tener un esquema de herencia, donde crearemos una clase Vehículo y dos subclases, Auto y Camioneta:



Nótese cómo para cumplir con uno de los requerimientos, se definió el método abstracto getCostoReparación() en Vehículo, para así forzar a que todas sus subclases lo implementen, para informar el costo de reparación específico de cada tipo:

```
1 public abstract class Vehículo
2 {
3     private String patente;
4     private String marca;
5     private int año;
6
7     public Vehículo(String patente, String marca, int año)
8     {
9         this.patente = patente;
10        this.marca = marca;
11        this.año = año;
12    }
13
14    public String getPatente() { return patente; }
15    public String getMarca() { return marca; }
16    public int getAño() { return año; }
17
18    public abstract int getCostoReparación();
19
20    public abstract String toString();
21 }
```

```

1 public class Auto extends Vehículo
2 {
3     private int kilometraje;
4     private int capacidadEstanque;
5
6     public Auto(String patente, String marca, int año, int kilometraje,
7                 int capacidadEstanque)
8     {
9         super(patente, marca, año);
10
11         this.kilometraje = kilometraje;
12         this.capacidadEstanque = capacidadEstanque;
13     }
14
15     public int getKilometraje() { return kilometraje; }
16
17     public int getCapacidadEstanque() { return capacidadEstanque; }
18
19     @Override
20     public int getCostoReparación() { return this.capacidadEstanque * 20000; }
21
22     @Override
23     public String toString()
24     {
25         return "Auto [kilometraje=" + kilometraje
26             + ", capacidadEstanque=" + capacidadEstanque
27             + ", getCostoReparación()=" + getCostoReparación()
28             + ", getPatente()=" + getPatente()
29             + ", getMarca()=" + getMarca()
30             + ", getAño()=" + getAño() + "];"
31     }
32 }

```

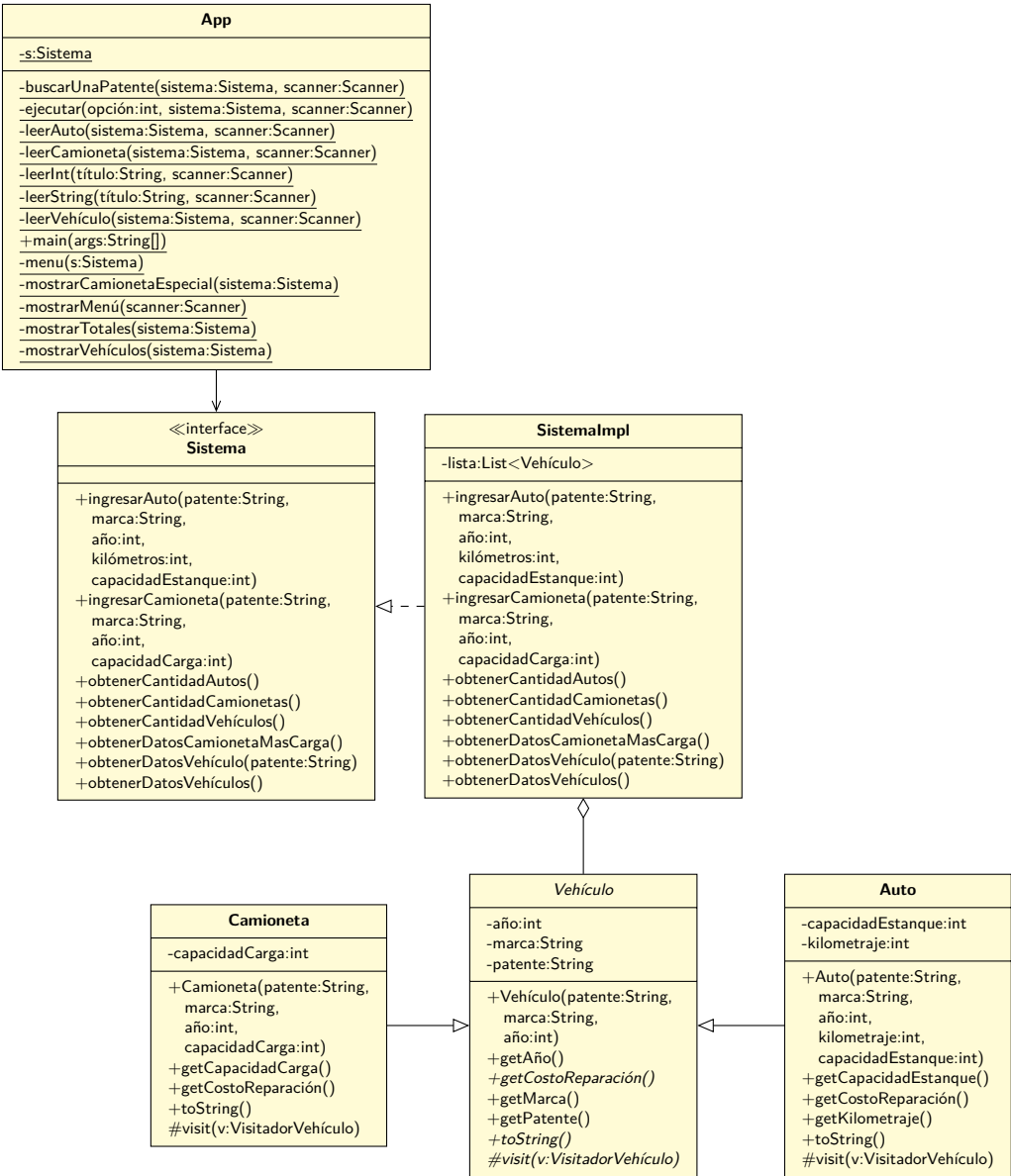
```

1 public class Camioneta extends Vehículo
2 {
3     private int capacidadCarga;
4
5     public Camioneta(String patente, String marca, int año, int capacidadCarga)
6     {
7         super(patente, marca, año);
8         this.capacidadCarga = capacidadCarga;
9     }
10
11     public int getCapacidadCarga() { return capacidadCarga; }
12
13     @Override
14     public int getCostoReparación() { return capacidadCarga * 100000; }
15
16     @Override
17     public String toString()
18     {
19         return "Camioneta [capacidadCarga=" + capacidadCarga
20             + ", getCostoReparación()=" + getCostoReparación()
21             + ", getPatente()=" + getPatente()
22             + ", getMarca()=" + getMarca()
23             + ", getAño()=" + getAño() + "];"
24     }
25 }

```

Además, la clase `SistemaImpl` necesitará una forma de almacenar las instancias, por lo que es necesario

que tenga una colección de **Vehículo**:



Ahora veamos cómo el sistema implementa cada una de sus operaciones. Comenzando por su constructor:

```
1 public SistemaImpl()
2 {
3     lista = new ArrayList<>();
4 }
```

La operación para ingresar un auto es simple:

```
1 public boolean ingresarAuto(String patente, String marca, int año, int kilómetros,
2                             int capacidadEstanque)
3 {
4     return lista.add(new Auto(patente, marca, año, kilómetros, capacidadEstanque));
5 }
```

Lo mismo que el ingreso de una camioneta:

```
1 public boolean ingresarCamioneta(String patente, String marca, int año,
2                                 int capacidadCarga)
3 {
4     return lista.add(new Camioneta(patente, marca, año, capacidadCarga));
5 }
```

Nótese que estamos usando una sola lista para almacenar ambos tipos, ya que al hacer un esquema de herencia, tanto los autos como las camioneta son *Vehículo*.

Para la operación que permite obtener los datos de todos los vehículos ingresados, solo le delegamos la operación a la lista:

```
1 public String obtenerDatosVehículos()
2 {
3     StringBuilder sb = new StringBuilder();
4
5     for(Vehículo v : lista) {
6         sb.append(v.toString());
7     }
8
9     return sb.toString();
10 }
```

Para implementar la operación que busca los datos de un vehículo en particular, tenemos que respetar el contrato definido previamente: desde el punto de vista del *Sistema*, se debe lanzar una *IllegalArgumentException* cuando la patente no existe, pero la lista nos informa del hecho al retornarnos *null* al buscarla:

```
1 public String obtenerDatosVehículo(String patente)
2 {
3     Vehículo vehículo = buscar(lista, patente);
4     if (vehículo == null) {
5         throw new IllegalArgumentException("No existe vehículo con patente " + patente);
6     }
7
8     return vehículo.toString();
9 }
10 private Vehículo buscar(List<Vehículo> lista, String patente)
11 {
12     for(Vehículo v : lista) {
13         if (patente.equals(v.getPatente())) return v;
14     }
15     return null;
16 }
```

Para retornar la cantidad de vehículos, solamente se lo preguntamos a la lista:

```
1 public int obtenerCantidadVehículos()
2 {
3     return lista.size();
4 }
```

Para las otras operaciones surge el problema de cómo discriminar cuando en la lista se tiene una instancia de una de las clases concretas. En este caso, para implementar la operación que cuenta la cantidad de vehículos presentes en la lista, el problema es cómo saber si una instancia de **Vehículo** de la lista realmente representa a un **Auto** y no a otro tipo.

En la práctica hay varias formas de resolver esto. Por ejemplo, se podría agregar un método abstracto en **Vehículo** que retorne el tipo específico como un **int**, y así cada tipo concreto retornaría su propio valor, y posteriormente un **if** podría usarse para discriminar el tipo.

El problema con esa técnica (y otras similares) es que acopla totalmente la implementación de la operación a los diferentes tipos de clases de la herencia. Si después se agrega un nuevo tipo de vehículo será necesario recordar modificar las operaciones para incorporar un nuevo caso al **if**, ya que el código mismo seguirá compilando correctamente incluso si el **if** no toma en cuenta todos los tipos necesarios.

Una mejor alternativa es aprovechar que ya existe una solución a este problema, en la forma del patrón Visitor, tal como se explica en la sección 10.4.8 en la página 304. Usando la técnica de este patrón se puede lograr una implementación mucho más desacoplada, flexible y que en caso de que se incorpore un nuevo tipo de **Vehículo** nos obligue a realizar las modificaciones que correspondan.

Para implementar el patrón, es necesario realizar un par de cambios pequeños a las clases ya existentes. En primer lugar se debe definir una interfaz nueva que contenga las operaciones de todos los visitantes de los vehículos:

```
1 public interface VisitadorVehículo
2 {
3     void aceptar(Auto auto);
4
5     void aceptar(Camioneta camioneta);
6 }
```

Después, se debe modificar la clase **Vehículo** para agregarle una operación abstracta que permita que una instancia de un visitante visite la instancia:

```
1 public abstract class Vehículo
2 {
3     ...
4     protected abstract void visit(VisitadorVehículo v);
5 }
```

Finalmente, en ambas clases concretas (**Auto** y **Camioneta**) se implementa el método abstract **visit(...)**:

```
1 public class Camioneta extends Vehículo
2 {
3     ...
4     protected void visit(VisitadorVehículo v)
5     {
6         v.aceptar(this);
7     }
8 }
```



```

1 public class Auto extends Vehículo
2 {
3     ...
4     protected void visit(VisitadorVehículo v)
5     {
6         v.aceptar(this);
7     }
8 }

```

Volviendo a la clase `SistemaImpl`, la implementación de las siguientes operaciones dependerá de implementar un visitador apropiado. Por ejemplo, para contar la cantidad de instancias de `Vehículo`, se puede crear el siguiente visitador:

```

1 public class VisitadorVehículoContadorAuto implements VisitadorVehículo
2 {
3     private int contador = 0;
4
5     @Override
6     public void aceptar(Auto auto) { contador++; }
7
8     @Override
9     public void aceptar(Camioneta camioneta) { /* Nada! */ }
10
11     public int getContador() { return contador; }
12 }

```

Y después usar este visitador para implementar la operación en `SistemaImpl`:

```

1 public int obtenerCantidadAutos()
2 {
3     VisitadorVehículoContadorAuto v = new VisitadorVehículoContadorAuto();
4     visitarTodos(lista, v);
5     return v.getContador();
6 }
7 private void visitarTodos(List<Vehículo> lista, VisitadorVehículo vi)
8 {
9     for(Vehículo v : lista) {
10         v.visit(vi);
11     }
12 }

```

Para contar la cantidad de camionetas podemos usar la misma técnica:

```

1 public class VisitadorVehículoContadorCamionetas implements VisitadorVehículo
2 {
3     private int contador = 0;
4
5     @Override
6     public void aceptar(Auto auto) { }
7
8     @Override
9     public void aceptar(Camioneta camioneta) { contador++; }
10
11     public int getContador() { return contador; }
12 }

```

Por lo que la operación en `SistemaImpl` sería:

```

1 public int obtenerCantidadCamionetas()
2 {
3     VisitadorVehículoContadorCamionetas v = new VisitadorVehículoContadorCamionetas();
4     visitarTodos(lista, v);
5     return v.getContador();
6 }

```

Finalmente, para obtener la camioneta que soporta más carga:

```

1 public class VisitadorVehículoCamionetaMasCarga implements VisitadorVehículo
2 {
3     private Camioneta selección;
4
5     @Override
6     public void aceptar(Auto auto) { }
7
8     @Override
9     public void aceptar(Camioneta camioneta)
10    {
11        Camioneta actual = getSelección();
12
13        if (actual != null &&
14            actual.getCapacidadCarga() > camioneta.getCapacidadCarga()) return;
15
16        selección = camioneta;
17    }
18
19    public Camioneta getSelección() { return selección; }
20 }

```

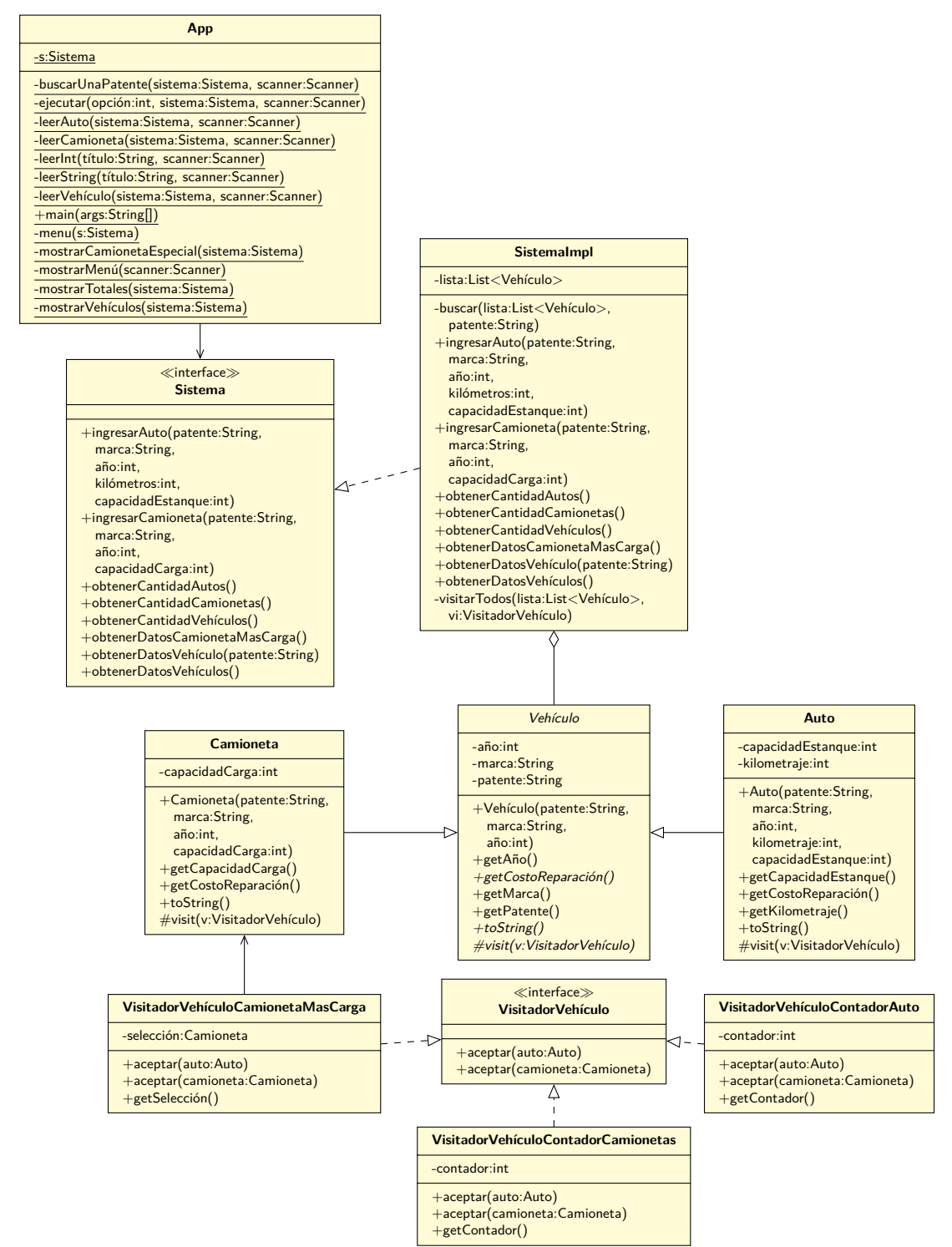
```

1 public String obtenerDatosCamionetaMasCarga()
2 {
3     VisitadorVehículoCamionetaMasCarga v = new VisitadorVehículoCamionetaMasCarga();
4     visitarTodos(lista, v);
5     Camioneta c = v.getSelección();
6     if (c == null) return null;
7     return c.toString();
8 }

```

Nótese cómo, al usar esta técnica, si posteriormente se crea un nuevo tipo de `Vehículo` (por ejemplo, un `Camión`), se estará obligado a implementar su operación `visit(...)`, lo que causará que se tenga que modificar la interfaz `VisitadorVehículo`, lo que a su vez causará que se tengan que actualizar todas las clases que implementaron dicha interfaz. En resumen, será imposible olvidar que el código se debe actualizar, por lo que hay menos posibilidades de cometer algún error, o dejar el código en un estado inconsistente.

11.5.5. Diagrama de clases final



12

El Final

En este libro hemos aprendido los conceptos básicos de la programación orientada a objetos (POO). Hemos visto cómo los objetos se pueden utilizar para modelar el mundo real y cómo la herencia y el polimorfismo nos permiten crear código más modular y reutilizable.

Al inicio del libro aprendimos sobre los conceptos fundamentales de la POO, como las clases, los objetos, los atributos y los métodos. También vimos cómo utilizar la herencia para crear clases que hereden el comportamiento y las propiedades de otras clases, y lo especializan.

También aprendimos acerca de polimorfismo, que nos permite utilizar objetos de diferentes clases de forma intercambiable. También vimos cómo utilizar interfaces para definir los comportamientos que deben implementar las clases.

Vimos los principios SOLID, y algunos patrones de diseño que es totalmente necesario que conozcas.

Y todo lo aplicamos en el contexto de Java y ejemplos, tanto pequeños como grandes ¡revisamos mucho código!

12.1. Lo aprendido

A lo largo de este libro, hemos aprendido lo siguiente:

- Los objetos son unidades que pueden representar entidades del mundo real.
- Las clases son plantillas que se utilizan para crear objetos.
- Los atributos son propiedades de los objetos.
- Los métodos son acciones que pueden realizar los objetos.
- Los objetos se envían mensajes entre sí, para poder solucionar los problemas.
- ¡Las clases también se envían mensajes!

- La herencia es un mecanismo que permite a las clases heredar el comportamiento y las propiedades de otras clases.
- El polimorfismo es un mecanismo que nos permite utilizar objetos de diferentes clases de forma intercambiable.
- Las interfaces definen los comportamientos que deben implementar las clases.
- La programación orientada a objetos en Java se basa en los conceptos fundamentales de la POO.
- ¡SOLID!
- Patrones de diseño

12.2. Invitación a seguir aprendiendo

Este libro ha proporcionado una introducción a los conceptos básicos de la POO. Sin embargo, hay muchos otros tópicos que pueden explorarse en el futuro.

Algunos tópicos que pueden ser de tu interés más adelante:

Programación orientada a objetos en otros lenguajes Java es un lenguaje popular para la programación orientada a objetos, pero existen muchos otros lenguajes que también apoyan este paradigma: Python, C++, etc

Programación orientada a objetos avanzada Existen muchos conceptos avanzados de la POO que pueden ser explorados por los estudiantes más avanzados, como la *reflexión*, la *programación concurrente y paralela*, y por supuesto seguir aprendiendo más *patrones de diseño*.

Programación orientada a objetos para juegos Los juegos son una aplicación común de la POO. Los estudiantes pueden aprender a crear juegos simples utilizando los conceptos básicos de la POO.

Programación orientada a objetos para sistemas empresariales Los sistemas empresariales son otra aplicación común de la POO. Los estudiantes pueden aprender a crear sistemas empresariales utilizando conceptos avanzados de la POO.

Antipatrones Un antipatrón en ingeniería de software es un concepto que describe un enfoque, diseño o práctica de programación que parece ser una solución válida para un problema, pero que en realidad conduce a resultados negativos o problemas significativos en el desarrollo de software.

12.3. Finalmente...

La POO es un paradigma de programación poderoso que puede utilizarse para crear software más modular, reutilizable y extensible. Este libro ha proporcionado una introducción a los conceptos básicos de la POO, pero hay muchos otros tópicos que pueden explorarse en el futuro.

La programación orientada a objetos es un pilar fundamental en la era digital. Ha resistido la prueba del tiempo y sigue siendo una de las metodologías más efectivas para desarrollar software de calidad

y escalable. A medida que la tecnología evoluciona y los desafíos se vuelven más complejos, la POO seguirá siendo una herramienta esencial para los desarrolladores.

Tu viaje en la programación orientada a objetos está lejos de terminar. Sigue experimentando, construyendo proyectos y colaborando con otras personas. La comunidad es un recurso invaluable, y el aprendizaje continuo es la clave para el éxito en este campo en constante cambio.

En nombre de todos los que han contribuido a este libro, te agradecemos por tu dedicación y entusiasmo. La POO ofrece un mundo de posibilidades y desafíos.

¡Que tu camino sea fructífero y lleno de logros!

Vive mucho y prospera.

Parte II

Ejercicios Propuestos

13

Ejercicios Propuestos

13.1. Mantenciones UCN

La UCN tiene un problema administrando los trabajos de mantención que se realizan al interior del campus Coquimbo, por lo que se requiere crear un pequeño sistema que ayude a las personas a registrar las “órdenes de trabajo” que se generan durante el año.

Una orden de trabajo se crea y queda asociada a una persona responsable, y además a varias personas que “trabajan” la orden de trabajo. Estas personas son las que realmente ejecutan el trabajo, y el responsable se encarga de gestionar cualquier problema que se presente.

Por otro lado, las órdenes de trabajo (OT) pueden ser de dos tipos diferentes: virtuales y manuales. Las OT manuales requieren materiales para ser ejecutadas, por ejemplo, 2 unidades de cemento, 3 unidades de fierro. Por otro lado, las OT virtuales no requieren ningún material y se asume que corresponden a trabajos que se realizan mediante videoconferencias.

El sistema debe ser capaz de registrar a las personas de la UCN, además de crear OTs (tanto manuales como virtuales), y asociar personas y los materiales (y cantidades) que se requieren. Es importante considerar que las personas reciben un sueldo, y que los materiales tienen un costo, por lo que a toda OT se le puede calcular un “costo final”, que corresponde a la suma de los sueldos de todas las personas involucradas, mas el costo de todos los materiales usados en la OT (si es que corresponde).

Construya un programa que realice las siguientes acciones:

1. Al ejecutarse debe leer el archivo `personas.txt` que contiene a las personas de la UCN. Su formato es:

```
rut,nombre,sueldo
```

2. Después debe leer el archivo `materiales.txt` que contiene los materiales disponibles. Su formato es:

```
código,nombre, costo
```

3. Después debe presentar un menú que permita realizar las siguientes acciones:

- 0. Salida
- 1. Agregar persona
- 2. Agregar OT
- 3. Estadísticas

4. "Salida" termina el programa

5. "Agregar persona" debe permitir agregar una nueva persona al sistema. Debe controlar que la persona que se está tratando de ingresar no esté ya presente.

6. "Agregar OT" debe permitir agregar una nueva OT al sistema. Recuerde que hay dos tipos de OT, con requerimientos diferentes. En el caso de las OT manuales debe controlar que una persona no esté ingresada múltiples veces a la OT, ni que el encargado sea también trabajador.

7. "Estadísticas" debe escribir por pantalla la siguiente información:

Costo promedio OT Virtuales
Costo promedio OT Manuales

Ejemplo de ejecución:

MENÚ

```
0. Salir
1. Agregar persona
2. Agregar OT
3. Estadísticas
> 2
Tipo de OT ([M]anual [V]irtual): V
Seleccione un encargado:
1: juan
2: ximena
0: Cancelar
Ingrese el RUT: 2
Indique cantidad de trabajadores: 1
Seleccione trabajador 1:
1: juan
2: ximena
0: Cancelar
Ingrese el RUT: 1
```

MENÚ

```
0. Salir
1. Agregar persona
2. Agregar OT
3. Estadísticas
> 3
Estadísticas:
Costo promedio OT Virtuales: 800.0
Costo promedio OT Manuales: 0.0
```

MENÚ

```
0. Salir
1. Agregar persona
2. Agregar OT
3. Estadísticas
> 2
Tipo de OT ([M]anual [V]irtual): M
Seleccione un encargado:
1: juan
2: ximena
0: Cancelar
Ingrese el RUT: 1
Indique cantidad de trabajadores: 0
Indica número de materiales: 2
Seleccione material 1:
A: pilas
B: cemento
Ingrese el código: A
Indique la cantidad de material: 3
Seleccione material 2:
A: pilas
B: cemento
Ingrese el código: B
Indique la cantidad de material: 5
```

MENÚ

```
0. Salir
1. Agregar persona
2. Agregar OT
3. Estadísticas
> 3
Estadísticas:
Costo promedio OT Virtuales: 800.0
Costo promedio OT Manuales: 3600.0
```

MENÚ

```
0. Salir
1. Agregar persona
2. Agregar OT
3. Estadísticas
> 1
Ingrese RUT: 2
Ingrese Nombre: Josefa
Ingrese Sueldo: 100000
Error ingresando 2 Josefa: Persona 2 ya existe
```

MENÚ

```
0. Salir
1. Agregar persona
2. Agregar OT
3. Estadísticas
> 3
Estadísticas:
Costo promedio OT Virtuales: 800.0
Costo promedio OT Manuales: 3600.0
```

MENÚ

```
0. Salir
1. Agregar persona
2. Agregar OT
3. Estadísticas
> 0
```

13.2. La repisa

Durante el último temblor grado 7, al doctor Chapatín le sucedió una calamidad: se le cayeron todos los libros de su repisa. Como toda crisis es una oportunidad, él quiere aprovechar de reordenar su colección de libros, y ordenarlos de forma más armónica, y también con menos riesgo para temblores futuros.

A partir de los libros que quedaron en el suelo, el doctor generó un archivo con la lista de todos éstos, y aprovechó de documentar algunas de sus características en un archivo llamado `datos.txt`. El archivo tiene esta forma:

```
1,REVISTA,52,ANILLADA
2,CARPETA,317,COMPUESTA
3,LIBRO,402,TAPABLANDA
4,REVISTA,98,EMPASTADA
5,LIBRO,539,TAPADURA
6,CARPETA,507,COMPUESTA
7,LIBRO,762,TAPADURA
8,LIBRO,367,TAPADURA
9,LIBRO,1184,TAPADURA
10,LIBRO,591,TAPADURA
11,REVISTA,15,EMPASTADA
12,LIBRO,960,TAPABLANDA
13,CARPETA,384,COMPUESTA
14,CARPETA,161,COMPUESTA
15,REVISTA,49,ANILLADA
16,REVISTA,91,ANILLADA
17,CARPETA,45,SIMPLE
18,LIBRO,79,TAPABLANDA
19,LIBRO,949,TAPADURA
20,CARPETA,459,COMPUESTA
```

Nótese que el doctor Chapatín tenía en su repisas otras cosas aparte de libros: revistas y carpetas. Para cada cosa en la repisa, el doctor registró un identificador único por elemento (la primera columna del archivo), la cantidad de páginas de cada elemento (la tercera columna en cada línea del archivo), además de una característica extra dependiendo del tipo de elemento:

Revista Anillada o Empastada

Carpeta Compuesta o Simple

Libro Tapa Blanda o Tapa Dura

Esas características son importantes, ya que el doctor quiere crear un software para poblar la repisa, pero respetando ciertas reglas. La repisa tendrá un tamaño (un ancho) en centímetros que la aplicación debe preguntar al iniciarse. Además, la aplicación también debe preguntar por el peso máximo que soportará la repisa en kilogramos

Entonces, el doctor quiere que la aplicación lea el archivo con los datos, y en el orden en que se leen las líneas, vaya probando si dicho elemento se puede posicionar en la repisa. Para que eso suceda debe quedar espacio en la repisa (considerando el “ancho” de cada elemento), y también la repisa debe poder soportar la masa del nuevo elemento. Si ambas cosas suceden, entonces el elemento (ya sea libro, revista o carpeta) se agrega a la repisa. Si no es así, entonces el elemento no se agrega.

El doctor Chapatín ha tabulado las reglas que se deben usar para calcular tanto la masa como el ancho de cada elemento a partir de su tipo y la cantidad de páginas:

Elemento	Ancho (mm)	Masa (gramos)
Libro	0.02mm por página. Si es de tapa dura, se agregan 5mm	Si es de tapa blanda, se agregan 40gr. Si es de tapa dura, se agregan 120gr. Además, 0.9gr por página
Revista	0.07mm por página. Si es una revista anillada, se agrega 1cm	0.2gr por página. Si es anillada, se agregan 190gr. Si no es anillada, se agregan 20gr.
Carpeta	0.1mm por página. Siempre se agrega 1cm	Si es una carpeta simple, se agregan 250gr. Si no es una carpeta simple, se agregan 350gr. Además, 1.1gr por página

Como el doctor Chapatín siempre anda buscando cómo complementar su sueldo, ha visto que a muchos de sus colegas les ha sucedido lo mismo que a él, por lo que quiere que usted le construya (gratis) la aplicación, para que el resto de las personas escriban el contenido de sus repisas y se les calcule automáticamente los libros que deben quedar y el orden de éstos, dependiendo de los parámetros de ancho y masa máxima soportada por la repisa.

Nótese que al doctor se le acaba de ocurrir que es posible que el resultado (los elementos que queden en la repisa) será diferente si se aplican diferentes estrategias para seleccionar los elementos desde el archivo. Así que el doctor necesita que la aplicación le diga cuál de las siguientes estrategias es la mejor:

- 1. Tomar los elementos en el mismo orden que vienen en el archivo
- 2. Tomar los elementos en el orden inverso a como vienen en el archivo
- 3. Tomar los elementos en orden aleatorio

Como al doctor Chapatín le va muy bien jugando a la ruleta en el casino, piensa que esa última estrategia puede tener potencial, pero también sabe que seguramente usted no tiene experiencia con números aleatorios, por lo que le entrega la siguiente función:

```
1  /**
2   * Retorna un número aleatorio. El valor retornado estará entre
3   * mínimo y máximo (incluyendo los extremos).
4   *
5   * @param mínimo El mínimo valor para el número aleatorio
6   * @param máximo El máximo valor para el número aleatorio
7   * @return Un número aleatorio entre mínimo y máximo
8   */
9  public int obtenerAleatorioEntre(int mínimo, int máximo)
10 {
11     java.util.Random random = new java.util.Random();
12     int randomNumber = random.nextInt(máximo - mínimo + 1) + mínimo;
13     return randomNumber;
14 }
```

Usando dicha función, el doctor confía en que va a poder ir seleccionando los elementos leídos desde el archivo (de manera aleatoria).

Al ejecutar la aplicación, ésta debe mostrar los elementos tal como fueron leídos desde el archivo, y el orden usando cada estrategia, y finalmente debe indicar cuál es mejor. La mejor estrategia será la que use mejor la masa máxima indicada.

Te damos la bienvenida a la app

Masa máxima (kg):

30

Ancho (cm) :

30

Simulando 30000 gramos y 300 mm

Elementos -----

```
1 REVISTA 13mm 200gr 13mm 200gr
2 CARPETA 41mm 698gr 41mm 698gr
3 LIBRO 8mm 401gr 8mm 401gr
4 REVISTA 6mm 39gr 6mm 39gr
5 LIBRO 15mm 605gr 15mm 605gr
6 CARPETA 60mm 907gr 60mm 907gr
7 LIBRO 20mm 805gr 20mm 805gr
8 LIBRO 12mm 450gr 12mm 450gr
9 LIBRO 28mm 1185gr 28mm 1185gr
10 LIBRO 16mm 651gr 16mm 651gr
11 REVISTA 1mm 23gr 1mm 23gr
12 LIBRO 19mm 904gr 19mm 904gr
13 CARPETA 48mm 772gr 48mm 772gr
14 CARPETA 26mm 527gr 26mm 527gr
15 REVISTA 13mm 199gr 13mm 199gr
16 REVISTA 16mm 208gr 16mm 208gr
17 CARPETA 14mm 299gr 14mm 299gr
18 LIBRO 1mm 111gr 1mm 111gr
19 LIBRO 23mm 974gr 23mm 974gr
20 CARPETA 55mm 854gr 55mm 854gr
```

Resultado en orden normal -----

```
1 REVISTA 13mm 200gr 13mm 200gr
2 CARPETA 41mm 698gr 41mm 698gr
3 LIBRO 8mm 401gr 8mm 401gr
4 REVISTA 6mm 39gr 6mm 39gr
5 LIBRO 15mm 605gr 15mm 605gr
6 CARPETA 60mm 907gr 60mm 907gr
7 LIBRO 20mm 805gr 20mm 805gr
8 LIBRO 12mm 450gr 12mm 450gr
9 LIBRO 28mm 1185gr 28mm 1185gr
10 LIBRO 16mm 651gr 16mm 651gr
11 REVISTA 1mm 23gr 1mm 23gr
12 LIBRO 19mm 904gr 19mm 904gr
13 CARPETA 48mm 772gr 48mm 772gr
15 REVISTA 13mm 199gr 13mm 199gr
```

Remanente: 22161gr

Usado: 7839gr

Número de elementos: 14

Resultado en orden reverso -----

```
20 CARPETA 55mm 854gr 55mm 854gr
19 LIBRO 23mm 974gr 23mm 974gr
18 LIBRO 1mm 111gr 1mm 111gr
17 CARPETA 14mm 299gr 14mm 299gr
16 REVISTA 16mm 208gr 16mm 208gr
15 REVISTA 13mm 199gr 13mm 199gr
14 CARPETA 26mm 527gr 26mm 527gr
13 CARPETA 48mm 772gr 48mm 772gr
12 LIBRO 19mm 904gr 19mm 904gr
11 REVISTA 1mm 23gr 1mm 23gr
10 LIBRO 16mm 651gr 16mm 651gr
9 LIBRO 28mm 1185gr 28mm 1185gr
8 LIBRO 12mm 450gr 12mm 450gr
7 LIBRO 20mm 805gr 20mm 805gr
4 REVISTA 6mm 39gr 6mm 39gr
```

Remanente: 21999gr

Usado: 8001gr

Número de elementos: 15

Resultado en orden aleatorio -----

```
11 REVISTA 1mm 23gr 1mm 23gr
15 REVISTA 13mm 199gr 13mm 199gr
13 CARPETA 48mm 772gr 48mm 772gr
3 LIBRO 8mm 401gr 8mm 401gr
10 LIBRO 16mm 651gr 16mm 651gr
6 CARPETA 60mm 907gr 60mm 907gr
17 CARPETA 14mm 299gr 14mm 299gr
9 LIBRO 28mm 1185gr 28mm 1185gr
2 CARPETA 41mm 698gr 41mm 698gr
5 LIBRO 15mm 605gr 15mm 605gr
8 LIBRO 12mm 450gr 12mm 450gr
14 CARPETA 26mm 527gr 26mm 527gr
1 REVISTA 13mm 200gr 13mm 200gr
18 LIBRO 1mm 111gr 1mm 111gr
```

Remanente: 22972gr

Usado: 7028gr

Número de elementos: 14

Resultado final -----

El mejor es el orden reverso

13.3. Doña Filomena

En la cocinería “Doña Filomena” se preocupan de la salud de sus comensales, por lo que desde el año pasado han iniciado acciones para informar de mejor forma a sus clientes acerca de la calidad de las comidas que ofrecen.

Para tal efecto, doña Filomena ha empezado a planear diariamente los platos que ofrece, y en un archivo (`oferta.txt`) escribe el detalle de toda la oferta de un día, de la siguiente forma:

```
ALMUERZO1,entrada,ensalada normal,200
ALMUERZO1,fondo,arroz,150,guatitas,100
ALMUERZO1,postre,helado de vainilla,150
CENA1,fondo,arroz,200,asado,150
ALMUERZO2,entrada,ensalada chilena,300
ALMUERZO2,fondo,pure,200,asado,200
```

En este archivo primero aparece el nombre del servicio, después el plato (cada servicio se compone de varios “platos”), y finalmente la lista de componentes de dicho plato. Para cada componente, aparece su nombre y la cantidad de gramos que se usa.

Además, doña Filomena tiene un archivo (`componentes.txt`) donde están definidos los ingredientes que usa cada componente:

```
ensalada normal,lechuga,0.7,aceitunas,0.15,tomate,0.15
arroz,arroz,1
guatitas,guatitas,0.3,tomate,0.2,agua,0.3,arvejas,0.2
helado de vainilla,helado de vainilla,1
arroz,arroz,1
asado,carne,1
ensalada chilena,tomate,0.75,cebolla,0.25
pure,papa,0.75,leche,0.15,mantequilla,0.10
```

Cada línea de este archivo indica el componente, y los ingredientes necesarios para preparar el componente, especificando la proporción de cada ingrediente.

Acompañando a este archivo, doña Filomena también llena un archivo (`ingredientes.txt`) con los ingredientes que tiene disponibles:

```
pure,300
helado de vainilla,800
carne,250
cebolla,100
arroz,90
guatitas,170
lechuga,10
aceitunas,30
tomate,50
agua,0
arvejas,70
leche,80
mantequilla,230
```

Cada ingrediente indica además la cantidad de calorías por cada 100 gramos de alimento.

Trabajo a realizar

1. Al iniciar el programa, se deben leer todos los archivos mencionados y cargar su información en estructuras de datos apropiadas.
2. Después de cargada la información, el programa debe preguntarle al usuario qué servicio desea. Una vez que el usuario ingresa el servicio elegido, el programa debe escribir por pantalla la lista de todos los platos que componen el servicio, y para cada plato, todos sus componentes, y para cada componente, todos sus ingredientes, indicando los gramos y calorías (esto se debe calcular por cada plato, ya que un mismo ingrediente puede utilizarse en distintas cantidades en diferentes platos)
3. El programa debe continuar preguntando servicios hasta que se ingrese el servicio "FIN". En ese momento, el programa debe indicar el ingrediente más utilizado, especificando la cantidad de gramos totales.

Observaciones

1. El contenido de los archivos `txt` no está ordenado de ninguna forma.
2. Los servicios, platos, componentes e ingredientes mostrados son solamente ejemplos. Doña Filomena constantemente cambia lo que ofrece, agregando y sacando servicios, platos, componentes e ingredientes.
3. Considere que lo que doña Filomena escribe en los archivos no necesariamente es consistente. Si es que el programa detecta alguna situación anómala (por ejemplo, un componente que hace referencia a un ingrediente que no existe) entonces el programa debe indicar la situación por pantalla y terminar su ejecución.

13.4. Aerolíneas PR

Aerolíneas PR requiere mejorar su proceso de actualización de su flota de aviones, ya que en los últimos años ha perdido mucho dinero al vender sus aviones antiguos y comprar aviones nuevos.

Para ayudarlos en este proceso, es necesario construir un simulador que permita a los ejecutivos de la aerolínea analizar lo que pasaría en diferentes casos de compra, uso y venta de aviones. Para esto, los datos se encontrarán en un archivo llamado `movimientos.txt`, el cual tiene muchas líneas, y cada línea corresponde a una acción a realizar en la simulación. Cada acción se va “ejecutando” de forma secuencial, y cada línea afecta a las líneas que vienen después en el archivo. Por ejemplo:

```
CA boeing 737 CQ556 2 150 3.7
CA boeing 737 CQ901 2 0 4.3
CA airbus 320 CQ139 2 15 8.9
CA tupolev 154 CQ778 3 200 3.3
CA airbus 340 CQ318 4 0 12.1
```

```
CM Rolls-Royce R813 56 1
CM Rolls-Royce R813 57 0
CM Rolls-Royce R813 58 10
CM Rolls-Royce R813 59 0
```

```
CM CFM C9931 58 0
CM CFM C9931 59 12
CM CFM C9931 60 0
CM CFM C9931 61 10
```

```
IM 2 CFM 60 CQ556
IM 1 CFM 59 CQ556
IM 2 CFM 58 CQ556
```

```
IM 4 Rolls-Royce 59 CQ318
IM 2 Rolls-Royce 58 CQ318
IM 3 Rolls-Royce 56 CQ318
IM 1 Rolls-Royce 57 CQ318
```

```
DM 1 CQ556
DM 3 CQ318
```

```
IM 1 CFM 58 CQ556
IM 2 CFM 58 CQ139
```

```
V CQ556 6
V CQ318 8
V CQ901 8
```

```
DM 4 CQ318
```

```
IM 4 CFM 58 CQ318
IM 3 CFM 61 CQ318
```

```
V CQ318 12
```

```
IM 1 CFM 58 CQ556
IM 2 CFM 61 CQ556
```

```
V CQ556 20
V CQ778 12
```

```
DM 2 CQ556
```

```
VA CQ139
VA CQ901
VA CQ778
```

Las acciones disponibles para la simulación son:

- CA** “Comprar Avión”. Se especifica la marca del avión, el modelo, su matrícula¹, la cantidad de motores del avión, la cantidad de horas de vuelo actuales² y su precio (en millones de USD). La aerolínea solo utiliza aviones de 2, 3 o 4 motores. Al momento de comprar un avión, éste viene sin motores.
- CM** “Comprar Motor”. Se especifica la marca del motor, el modelo, su número de serie y la cantidad de horas de vuelo actuales. La aerolínea solamente compra dos marcas: “Rolls-Royce” y “CFM”. El número de serie de un motor son únicos, pero para un fabricante en particular: por ejemplo, puede existir más de un motor con número de serie 34, pero solo un motor CFM con número de serie 34. Para todos efectos prácticos, considere que los motores son gratis, o sea, no tienen precio de compra.
- IM** “Instalar Motor”. Se especifica la posición del motor (en el avión), la marca del motor, su número de serie y la matrícula del avión donde se instalará el motor. Nótese que no se puede instalar un motor en una posición inválida: las posiciones válidas van del 1 a la cantidad de motores del avión. Si el motor que se va a instalar ya estaba instalado en otro avión, entonces automáticamente se desinstala antes de instalarlo en su ubicación final. Además, si en la posición indicada ya estaba instalado un motor, éste se desinstala también.
- DM** “Desinstalar Motor”. Se especifica la posición del motor y la matrícula del avión donde se desinstalará el motor.
- V** “Viajar”. Esto representa el hecho que el avión, tal como se encuentra en este momento, realiza un viaje. Se especifica la matrícula del avión y la cantidad de horas que duró el viaje. Nótese que un avión solo puede realizar el viaje si su configuración es correcta: los aviones de 4 motores solo pueden volar si tienen 4 o 3 motores; los aviones de 3 motores solo pueden volar si tienen 3 o 2 motores; los aviones de 2 motores solo pueden volar si tienen sus 2 motores instalados. Al realizarse el viaje, las horas de vuelo se suman a las horas de vuelo del avión, y también cada motor que realizó el viaje acumula horas de vuelo.
- VA** “Vender Avión”. Se especifica la matrícula del avión que se venderá. Los aviones se venden sin motores.

Una vez que se termina de procesar el archivo, el simulador debe entregar unas estadísticas por pantalla:

¹El equivalente a la “patente”. No hay dos aviones que tengan la misma matrícula

²Las horas de vuelo de un avión indican la cantidad de horas que el avión ha pasado volando. Al momento de comprar un avión, es un dato importante de saber para conocer qué tan “usado” está el avión

No se puede realizar viaje en línea 33: avión no configurado
No se puede realizar viaje en línea 35: avión no configurado
No se puede realizar viaje en línea 48: avión no configurado

Flota actual:

airbus 340 CQ318 4 0/20
 1: Rolls-Royce 57 0/20
 2: Rolls-Royce 58 10/30
 3:
 4:
boeing 737 CQ556 2 150/170
 1: CFM 58 0/32
 2:

Motores en bodega:

CFM C9931 60 0/0
Rolls-Royce R813 56 1/1
Rolls-Royce R813 59 0/8
CFM C9931 59 12/12
CFM C9931 61 10/42

Estado financiero:

Gastos (Combustible): 136.2284
Ingresos: 15.9804
Inventario: 15.8

Toma en cuenta que

- En el caso de que la simulación indique un viaje, pero éste no se pueda realizar, se debe indicar el hecho, especificando el número de línea dentro del archivo donde está especificado el viaje.
- La flota actual corresponde a los aviones que la aerolínea actualmente posee (los que compró, pero no ha vendido).
- La flota se debe mostrar tal como en el ejemplo, especificando su marca, modelo, matrícula, cantidad de motores, horas y los motores que tiene instalado cada avión. La lista debe escribirse ordenada por horas de vuelo total, de menor a mayor.
- Tanto los aviones como los motores se deben mostrar con la cantidad de horas de vuelo iniciales (al ser “comprados”) y las actuales. Por ejemplo, 0/20 del primer avión.
- Los motores en bodega son aquellos que la aerolínea posee, pero no están instalados actualmente en ningún avión.
- Los motores se deben mostrar con su marca, modelo, número de serie y tanto la cantidad de horas de vuelo iniciales (al ser “comprados”) como actuales. La lista debe escribirse ordenada por horas de vuelo total, de menor a mayor.
- El estado financiero corresponde a los movimientos de dinero en los que ha incurrido la aerolínea:
 - Ingresos: Millones de USD recibidos por la venta de aviones. El precio de venta de un avión se calcula de la siguiente forma:

$$\text{precioVenta} = \text{precioOriginal} * (1 - \text{impuesto}) - \text{horasDeVueloTotales} * \text{factor}$$

Donde

- o *precioOriginal*: es el precio al que se compró el avión
- o *impuesto*: un valor que depende de la cantidad de motores del avión

Tipo	<i>impuesto</i>
2 motores	0.012
3 motores	0.014
4 motores	0.019

- o *horasDeVueloTotales*: es la cantidad total de horas de vuelo del avión
- o *factor*: un valor que depende de la cantidad de motores del avión

Tipo	<i>factor</i>
2 motores	0.0010
3 motores	0.0015
4 motores	0.0021

- **Gastos(combustible)**: Cantidad de dinero total gastado en combustible por los viajes realizados. El gasto de cada viaje se calcula a partir de las horas que dura dicho viaje:

$$\text{monto} = (1 + \text{factor}) * \text{sumatoria}$$

Donde

- o *factor*: tal como está definido más arriba
- o *sumatoria*: la suma del monto gastado por cada motor del avión. Este monto se calcula diferente dependiendo de la marca del motor:

Marca	<i>monto</i>
Rolls-Royce	$\text{montoMotor} = \text{horasViaje} * 1.1$
CFM	$\text{montoMotor} = \text{horasViaje} * 1.3$

- **Inventario**: Millones de USD de los aviones en la flota. Para esto, se considera la sumatoria de los precio de compra de cada avión que fue comprado pero no ha sido vendido.

Trabajo a realizar

1. Debe construir un programa que lea el archivo indicado, y vaya procesando cada línea con órdenes de la simulación.
2. Al terminar de procesar el archivo, se debe escribir el resumen financiero.
3. El archivo puede contener líneas en blanco, las cuales deben ser ignoradas.

Parte III

Apéndices

Apéndice A

Instalación del JDK

A.1. Cómo instalar el JDK de Java en Windows

El Java Development Kit (JDK) es un conjunto de herramientas de desarrollo de software que incluye el compilador Java, la máquina virtual Java (JVM) y las bibliotecas API de Java. Es necesario para desarrollar, depurar y ejecutar aplicaciones Java.

En este tutorial, aprenderemos a instalar el JDK de Java en Windows 10. El proceso de instalación es similar en otros sistemas operativos, pero puede haber algunas diferencias menores.

A.1.1. Requisitos previos

Para instalar el JDK, necesitarás lo siguiente:

- Un computador con Windows 10 instalado.
- Privilegios de administrador en el computador.
- Una conexión a Internet para descargar el instalador del JDK.

A.1.2. Descargando el instalador del JDK

Para descargar el instalador del JDK, ve al sitio web de Oracle y haz clic en la página *Descargas de Java SE*. En la sección *Plataforma Java, edición estándar*, haz clic en el botón *Descargar JDK*.

En la siguiente página, selecciona la versión del JDK que deseas instalar. La versión más reciente siempre es la recomendada, pero es posible que necesites instalar una versión anterior si estás desarrollando para una plataforma o entorno específico. En este libro hemos apuntado a usar la antigua versión 8!

Una vez que hayas seleccionado la versión del JDK, haz clic en el botón *Aceptar acuerdo de licencia* y luego haz clic en el enlace *Descargar*.

El instalador del JDK se descargará en tu computador. Una vez que la descarga se complete, abre el archivo de instalación para comenzar el proceso de instalación.

A.1.3. Instalando el JDK

Para instalar el JDK, sigue las instrucciones que aparecen en la pantalla del instalador. Por lo general, deberás seleccionar el directorio de instalación y luego hacer clic en el botón *Siguiente*.

El instalador copiará los archivos del JDK a tu computador. Una vez que la instalación se complete, el instalador te pedirá que reinicies tu computador.

A.1.4. Configuración de la variable de entorno `JAVA_HOME`

Una vez que el JDK esté instalado, deberás configurar la variable de entorno `JAVA_HOME`. Esta variable le dice a tu sistema operativo dónde está instalado el JDK.

Para configurar la variable de entorno `JAVA_HOME`:

1. Haz clic derecho en el icono *Mi PC* y selecciona *Propiedades*.
2. Haz clic en el enlace *Configuración avanzada del sistema*.
3. En la ventana *Propiedades del sistema*, haz clic en el botón *Variables de entorno*.
4. En la sección *Variables del sistema*, encuentra la variable `JAVA_HOME`. Si no existe, haz clic en el botón *Nuevo* y crea una nueva variable llamada `JAVA_HOME`.
5. Establece el valor de la variable `JAVA_HOME` en el directorio donde está instalado el JDK. Por ejemplo, si el JDK está instalado en el directorio `C:\Java\jdk-19`, entonces el valor de la variable `JAVA_HOME` sería `C:\Java\jdk-19`.
6. Haz clic en el botón *Aceptar* para guardar tus cambios.

A.1.5. Verificando la instalación de Java

Para verificar que el JDK esté instalado correctamente, abre una ventana de símbolo del sistema y escribe el siguiente comando:

```
java -version
```

Si el JDK está instalado correctamente, deberías ver la siguiente salida (o similar):

```
java version "19.0.1" 2023-09-19 LTS (build 19.0.1+10-21)
OpenJDK Runtime Environment 19.0.1+10-21 (build 19.0.1+10-21)
OpenJDK 64-Bit Server VM 19.0.1+10-21 (build 19.0.1+10-21, mixed mode)
```

Si en cambio recibes un mensaje de error diciendo que el comando `java` o `javac` es desconocido, entonces te falta agregar el directorio donde viven los ejecutables a la variable de entorno `PATH`.

A.1.6. Configurando la variable de entorno PATH

Para agregar el directorio donde está instalado el JDK (Java Development Kit) a la variable **PATH** en Windows 10, sigue estos pasos:

1. Abre el “Explorador de archivo” y navega hasta el directorio donde tienes instalado el JDK. Por lo general, la ubicación predeterminada es `C:\Archivos de programa\Java` o similar. Dentro de este directorio, deberías ver una carpeta que corresponde a la versión del JDK que tienes instalada, por ejemplo, `jdk1.8.0_291` o `jdk-16.0.2`.
2. Copia la ruta completa de la carpeta del JDK. Puedes hacerlo haciendo clic en la barra de direcciones del Explorador de archivos y copiando la ruta que se muestra allí.
3. Ahora, abre el “Panel de control” de Windows. Puedes hacerlo buscándolo en el menú de inicio o utilizando la combinación de teclas “Windows + X” y seleccionando “Panel de control” desde el menú.
4. En el “Panel de control,” busca “Sistema” y haz clic en él para abrir la ventana de configuración del sistema.
5. En la ventana de configuración del sistema, haz clic en “Configuración avanzada del sistema” en el panel izquierdo. Esto abrirá la ventana “Propiedades del sistema”.
6. En la ventana “Propiedades del sistema”, ve a la pestaña “Opciones avanzadas” y haz clic en el botón “Variables de entorno” en la sección “Variables de sistema”.
7. En la sección “Variables de sistema”, busca la variable llamada “Path” en la lista de variables y selecciónala. Luego, haz clic en el botón “Editar...”
8. En la ventana “Editar variable de sistema”, verás una lista de rutas separadas por punto y coma. No elimines ninguna de las rutas existentes. En su lugar, al final de la lista, pega la ruta que copiaste en el paso 2. Asegúrate de que esté separada por punto y coma de las rutas existentes.
9. Haz clic en “Aceptar” para cerrar todas las ventanas de configuración.
10. Ahora, la variable **PATH** se ha actualizado para incluir la ruta al directorio del JDK. Abre una nueva ventana de símbolo del sistema o reinicia cualquier ventana de símbolo del sistema abierta para que los cambios surtan efecto.

Una vez que hayas seguido estos pasos, podrás ejecutar comandos de Java desde cualquier ubicación en el símbolo del sistema sin tener que especificar la ruta completa del JDK. Esto facilitará el desarrollo y la ejecución de programas Java en tu sistema.

A.2. Cómo instalar el JDK de Java en Ubuntu Linux

Instalar el JDK de Java en el sistema operativo Linux Ubuntu es un proceso esencial para desarrolladores y usuarios que desean ejecutar aplicaciones y programas Java en su entorno.

A.2.1. Actualizar el sistema

Antes de comenzar la instalación del JDK, es importante asegurarse de que el sistema esté actualizado. Abre una terminal y ejecuta los siguientes comandos:

```
sudo apt update  
sudo apt upgrade
```

Estos comandos actualizarán la lista de paquetes disponibles y actualizarán todos los paquetes instalados en tu sistema.

A.2.2. Instalar el JDK de Java

Ubuntu generalmente utiliza OpenJDK, una implementación de código abierto de la plataforma Java. Puedes instalarlo fácilmente con el siguiente comando:

```
sudo apt install default-jdk
```

Este comando instalará la última versión de OpenJDK disponible en los repositorios de Ubuntu.

A.2.3. Verificar la instalación

Una vez que la instalación se complete, puedes verificar si el JDK se ha instalado correctamente utilizando los siguientes comandos:

Para verificar la versión de Java:

```
java -version
```

Para verificar la versión del compilador Java (`javac`):

```
javac -version
```

Apéndice B

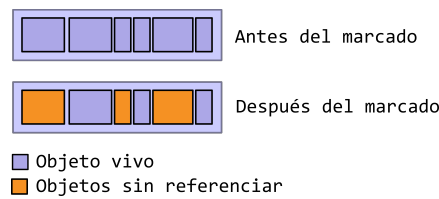
El recolector

La recolección automática de basura es un proceso que mira la memoria utilizada por el programa e identifica los objetos que están siendo usados y los que no, y borra los objetos sin uso.

Un objeto que está en uso, o que está referenciado, equivale a decir que alguna parte del programa mantiene una referencia hacia él. Un objeto sin uso, o no referenciado, es aquel que no es referenciado por ninguna parte del programa. En este último caso, la memoria utilizada por este objeto puede ser recolectada para ser utilizada en otros objetos en el futuro.

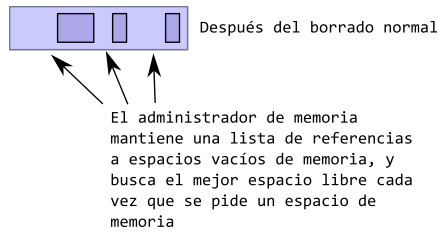
Generalmente, el recolector de basura funciona en dos pasos:

Marcado El primer paso es el marcado de la memoria, en el cual el recolector de basura identifica los espacios de memoria que están en uso, y los que no.

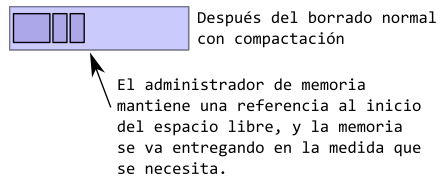


Este es un proceso que ocuparía demasiado tiempo si es que todos los objetos del sistema necesitan ser revisados.

Borrado normal Se remueven los objetos, devolviendo el espacio a la lista de memoria libre. En este caso, la memoria quedará fragmentada, lo que puede causar problemas con pedidos de memoria futura y que no se puedan satisfacer ya que no existen espacios lo suficientemente grandes para satisfacer los requerimientos.



Borrado con compactación Para solucionar el problema anterior, además de reclamar la memoria liberada, se procede a compactar la memoria liberada, de manera de dejar espacios contiguos de memoria disponible, lo que facilita su gestión futura.



El recolector de basura (o “garbage collector” en inglés) es una característica fundamental en Java. Esta funcionalidad automatizada tiene varias ventajas significativas que han contribuido en gran medida a la popularidad y eficacia de Java como lenguaje de programación:

Gestión automática de la memoria Una de las principales ventajas del recolector de basura en Java es que simplifica la gestión de la memoria. En lenguajes de programación sin recolector de basura, los programadores deben asignar y liberar manualmente la memoria, lo que puede llevar a errores de programación, como fugas de memoria y corrupción de datos. El recolector de basura se encarga de liberar automáticamente la memoria de los objetos que ya no se utilizan, reduciendo drásticamente la posibilidad de errores relacionados con la memoria.

Mejora la productividad del programador Al eliminar la necesidad de gestionar manualmente la memoria, los programadores pueden centrarse en la lógica de su código y en la resolución de problemas, en lugar de preocuparse por los detalles de la gestión de la memoria. Esto aumenta la productividad y permite que los desarrolladores sean más eficientes.

Reducción de fugas de memoria Con el recolector de basura, las fugas de memoria se vuelven mucho menos comunes. Una fuga de memoria ocurre cuando un programa reserva memoria pero nunca la libera, lo que puede llevar a un agotamiento gradual de los recursos del sistema. El recolector de basura se encarga de identificar y eliminar automáticamente los objetos no utilizados, evitando fugas de memoria.

Prevención de corrupción de datos En lenguajes donde la gestión de la memoria es manual, liberar memoria de manera incorrecta o prematura puede resultar en la corrupción de datos. El recolector de basura asegura que la memoria se libere solo cuando ya no se necesita, evitando así problemas de corrupción de datos.

Optimización del rendimiento Los recolectores de basura modernos en Java están altamente optimizados para minimizar el impacto en el rendimiento del programa. Utilizan algoritmos sofisticados para determinar cuándo y cómo liberar la memoria, lo que reduce al mínimo la interrupción en la ejecución del programa.

Facilita la depuración La gestión manual de la memoria puede complicar la depuración de errores, ya que los problemas relacionados con la memoria pueden ser difíciles de detectar. Con el recolector de basura, los programadores pueden centrarse en la lógica de la aplicación y utilizar herramientas de depuración más efectivamente.

Reducción de riesgos de seguridad La gestión manual de la memoria a menudo lleva a errores de programación que pueden explotarse para ataques de seguridad, como desbordamientos de búfer. El recolector de basura reduce la probabilidad de estos errores, lo que mejora la seguridad de las aplicaciones Java.

Apéndice C

Protecciones

En Java el acceso a atributos y métodos está definido de la siguiente manera:

Protección	Clase	Package	Subclase (mismo package)	Subclase (package diferente)	Resto del mundo
<code>public</code>	✓	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✓	
<i>sin modificador</i>	✓	✓	✓		
<code>private</code>	✓				

Para más información se puede visitar

<https://docs.oracle.com/javase/tutorial/java/java00/accesscontrol.html>

Apéndice D

javadoc

En Java es posible incluir comentarios dentro del mismo código que tienen un significado especial. Estos comentarios se distinguen de los comentarios normales en que están delimitados por `/**` y `*/`. La herramienta `javadoc` procesa esos comentarios para generar documentación.

D.1. Formato General

Los comentarios deben preceder la declaración de clases, atributos, constructores o métodos.

Se compone de dos partes: una descripción seguida de bloques de tags. Por ejemplo:

```
1 /**
2  * Returns the string representation of the {@code boolean} argument.
3  *
4  * @param   b    a {@code boolean}.
5  * @return  if the argument is {@code true}, a string equal to
6  *          {@code "true"} is returned; otherwise, a string equal to
7  *          {@code "false"} is returned.
8  */
9 public static String valueOf(boolean b)
10 {
11     return b ? "true" : "false";
12 }
```

En este caso, los tags destacados son `@param` y `@return`.

- Cada línea está indentada respecto al código que está comentando.
- La primera línea contiene el delimitador de inicio (`/**`).
- Nótese el uso de `{@code ...}` para indicar texto que tiene un significado especial. En este caso se usa para señalar palabras que corresponden a código.
- Si el texto contiene varios párrafos, se deben separar usando un `<p>`.

- Se debe insertar un espacio en blanco entre el fin de la descripción y el inicio de los tags.
- La última línea contiene el `*/` que cierra el comentario.

Normalmente las IDE usan estos comentarios para generar documentación directamente:

```
String java.lang.String.valueOf(boolean b)

Returns the string representation of the boolean argument.

Parameters:
    b a boolean.

Returns:
    if the argument is true, a string equal to "true" is returned;
    otherwise, a string equal to "false" is returned.
```

Aunque también la herramienta javadoc se puede utilizar para generar documentación fuera de la IDE, en este caso del mismo método:

java.awt.print

java.beans

java.beans.beancontext

java.io

java.lang

java.lang.annotation

java.lang.instrument

java.lang.invoke

java.lang.management

java.lang.ref

java.lang.reflect

java.math

java.net

Stream.Builder

Streamable

StreamableValue

StreamCorruptedExceptio

StreamFilter

StreamHandler

StreamPrintService

StreamPrintServiceFactor

StreamReaderDelegate

StreamResult

StreamSource

StreamSupport

StreamTokenizer

StrictMath

String

StringBuffer

StringBufferInputStream

valueOf

public static String valueOf(boolean b)

Returns the string representation of the boolean argument.

Parameters:

b - a boolean.

Returns:

if the argument is true, a string equal to "true" is returned; otherwise, a string equal to "false" is returned.

valueOf

public static String valueOf(char c)

Returns the string representation of the char argument.

Parameters:

c - a char.

Returns:

a string of length 1 containing as its single character the argument c.

Y acá de la clase `String` en sí:

Java™ Platform
Standard Ed. 8

All Classes All Profile

Packages

java.applet
java.awt
java.awt.color
java.awt.datatransfer
java.awt.dnd
java.awt.event
~~java.awt.font~~
Stream.Builder
Streamable
StreamableValue
StreamCorruptedExceptio
StreamFilter
StreamHandler
StreamPrintService
StreamPrintServiceFactor
StreamReaderDelegate
StreamResult
StreamSource
StreamSupport
StreamTokenizer
StrictMath
String
StringBuffer
StringBufferInputStream
StringBuilder
StringCharacterIterator
StringContent
StringHolder
StringIndexOutOfBoundsException
StringJoiner
StringMonitor
StringMonitorMBean

OVERVIEW PACKAGE **CLASS** USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3
java.lang

Class String

java.lang.Object
 java.lang.String

All Implemented Interfaces:
Serializable, CharSequence, Comparable<String>

public final class String
 extends Object
 implements Serializable, Comparable<String>, CharSequence

The String class represents character strings. All string literals in Java programs, such as "abc", are implemented as instances of this class.

Strings are constant; their values cannot be changed after they are created. String buffers support mutable strings. Because String objects are immutable they can be shared. For example:

String str = "abc";

is equivalent to:

char data[] = {'a', 'b', 'c'};
String str = new String(data);

D.2. Recomendaciones

- La primera línea del comentario debería ser un resumen del elemento siendo documentado.
- Las descripciones de los métodos deben comenzar con un verbo.
- Las descripciones de clases, interfaces, atributos pueden omitir el objeto y solo indicar el estado.

```
1 /**  
2  * Una etiqueta de botón  
3  */
```

en vez de

```
1 /**  
2  * Este campo es una etiqueta de botón  
3  */
```

- No se deben incluir descripciones que no agregan nada más de lo que se puede saber leyendo el nombre del método:

```
1 /**  
2  * Setea el nombre de este usuario  
3  *  
4  * @param nombre El nombre del usuario  
5  */  
6 public void setNombre(String nombre) {
```

D.3. Tags

Los tags se deben incluir en el siguiente orden:

- `@author` (solo para clases e interfaces, requerido)
- `@version` (solo para clases e interfaces)
- `@param` (solo para métodos y constructores)
- `@return` (solo para métodos)
- `@exception` (`@throws` es un sinónimo)
- `@see`
- `@since`

D.4. Algunos ejemplos

```

1 /**
2  * A CharSequence is a readable sequence of char values. This
3  * interface provides uniform, read-only access to many different kinds of
4  * char sequences.
5  * A char value represents a character in the Basic
6  * Multilingual Plane (BMP) or a surrogate. Refer to Character.html#unicode for details.
8  *
9  * 

This interface does not refine the general contracts of the {@link
10 * java.lang.Object#equals(java.lang.Object) equals and {@link
11 * java.lang.Object#hashCode() hashCode methods. The result of comparing two
12 * objects that implement CharSequence is therefore, in general,
13 * undefined. Each object may be implemented by a different class, and there
14 * is no guarantee that each class will be capable of testing its instances
15 * for equality with those of the other. It is therefore inappropriate to use
16 * arbitrary CharSequence instances as elements in a set or as keys in
17 * a map.


18 *
19 * @author Mike McCloskey
20 * @since 1.4
21 * @spec JSR-51
22 */
23
24 public interface CharSequence { ...

```

```

1 /**
2  * Returns the length of this character sequence. The length is the number
3  * of 16-bit chars in the sequence.
4  *
5  * @return the number of chars in this sequence
6  */
7 int length();

```

```

1 /**
2  * Returns the char value at the specified index. An index ranges from
3  * zero
4  * to length() - 1. The first char value of the sequence is at
5  * index zero, the next at index one, and so on, as for array
6  * indexing.
7  *
8  * <p>If the char value specified by the index is a
9  * <a href="{@docRoot}/java/lang/Character.html#unicode">surrogate</a>, the surrogate
10  * value is returned.
11  *
12  * @param index the index of the char value to be returned
13  *
14  * @return the specified char value
15  *
16  * @throws IndexOutOfBoundsException
17  *         if the index argument is negative or not less than
18  *         length()
19  */
20 char charAt(int index);

```



```

1 /** Cache the hash code for the string */
2 private int hash; // Default to 0

```

Para más detalles, se puede ver la documentación oficial de Java. Por ejemplo:

<https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>

Apéndice E

Uso de interfaces en Swing

Como se dijo anteriormente, las interfaces permiten definir el contrato que una clase debe cumplir.

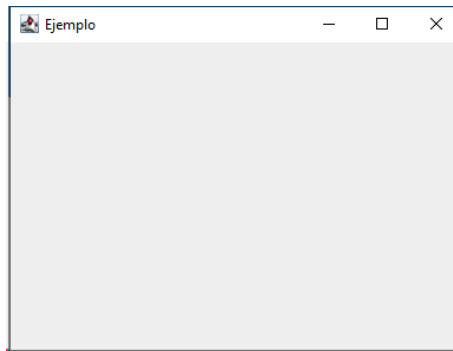
Esta es una cualidad que puede ser usada de diferentes maneras, y en particular lo ejemplificaremos usando algo que el mismo Java trae.

En Java se pueden crear interfaces gráficas de usuario (GUI en inglés). En otras palabras, se pueden crear ventanas, botones y todo lo que se necesita para crear aplicaciones visuales.

Por ejemplo, mira este código:

```
1 import javax.swing.JFrame;
2
3 public class Test
4 {
5     public static void main(String[] args)
6     {
7         JFrame frame = new JFrame("Ejemplo");
8
9         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
10
11        frame.setSize(400, 300);
12
13        frame.setVisible(true);
14    }
15 }
```

Al ejecutarlo, genera la siguiente ventana:



Ahora, ¿qué tiene que ver esto con las interfaces? La respuesta se relaciona con un concepto que se llama *Eventos*.

¿Qué es un evento? Cuando creamos interfaces gráficas, siempre necesitaremos responder a las acciones que el usuario hace en la pantalla. Por ejemplo, presionar un botón. Si el usuario presiona un botón, significa que se va a generar el evento “presionar el botón” y de alguna forma tenemos que escribir código que responda a dicho evento y haga lo que tenga que hacer.

En el caso de Java, al usar las clases de **Swing** (son las clases que contienen la funcionalidad para crear componentes gráficos), el sistema de manejo de eventos se diseñó basándose en interfaces. Hay otros lenguajes en que esto se implementa de otras maneras, pero en el caso de Java, sus diseñadores decidieron que se implementaría así.

En términos prácticos, en **Swing**, cuando se desea responder a cierto tipo de eventos, se le debe indicar al componente (por ejemplo, un botón) que queremos realizar acciones cuando el evento suceda. Eso se realiza invocando un método en el componente y pasándole como parámetro una instancia de una clase que “recibirá” los eventos y nos permitirá reaccionar a ellos. A esto muchas veces se le denomina “registrar un escuchador”.

Por ejemplo, vamos a reaccionar al evento “click” de la ventana misma, de forma de escribir algo en la consola cada vez que el usuario hace click con su mouse. Para esto, le indicamos al componente (en este caso, la ventana) que estamos interesados en los eventos del mouse, y le pasamos una instancia de una clase:

```
1  JFrame frame = new JFrame("Ejemplo");
2
3  frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
4
5  frame.getContentPane().addMouseListener(.....);
6
7  frame.setSize(400, 300);
8
9  frame.setVisible(true);
```

En este caso, en la tercera línea le estamos diciendo a la ventana que estamos interesados en responder a los eventos producidos por el mouse (o sea, registrar un escuchador). En este caso, no se ha especificado nada y se requiere reemplazar “.....” por una instancia que recibirá los eventos.

En este caso concreto, `Swing` define el método `addMouseListener` de esta forma:

```
1 public void addMouseListener(MouseListener l)
2 {
3     // Acá va el código (no nos interesa actualmente)
4 }
```

Como se puede ver, `Swing` requiere que la instancia que reciba los eventos **DEBE** implementar la interfaz especificada. Los diseñadores especificaron que esta interfaz tenga las siguientes operaciones:

```
1 public interface MouseListener
2 {
3     /**
4      * Invoked when the mouse button has been clicked (pressed
5      * and released) on a component.
6      * @param e the event to be processed
7      */
8     public void mouseClicked(MouseEvent e);
9
10    /**
11     * Invoked when a mouse button has been pressed on a component.
12     * @param e the event to be processed
13     */
14    public void mousePressed(MouseEvent e);
15
16    /**
17     * Invoked when a mouse button has been released on a component.
18     * @param e the event to be processed
19     */
20    public void mouseReleased(MouseEvent e);
21
22    /**
23     * Invoked when the mouse enters a component.
24     * @param e the event to be processed
25     */
26    public void mouseEntered(MouseEvent e);
27
28    /**
29     * Invoked when the mouse exits a component.
30     * @param e the event to be processed
31     */
32    public void mouseExited(MouseEvent e);
33 }
```

De esta forma, los componentes sabrán qué métodos invocar cuando se produzcan los diferentes eventos del mouse.

Desde nuestro punto de vista, fácilmente podemos implementar las clases que recibirán los eventos. Concretamente, en nuestro ejemplo, solo tenemos que crear una clase que implemente la interfaz anterior:

```

1 class EventosDelMouse implements MouseListener
2 {
3     @Override
4     public void mouseClicked(MouseEvent e)
5     {
6         System.out.println("Alguien hizo click!");
7     }
8
9     @Override
10    public void mousePressed(MouseEvent e)
11    {
12    }
13
14    @Override
15    public void mouseReleased(MouseEvent e)
16    {
17    }
18
19    @Override
20    public void mouseEntered(MouseEvent e)
21    {
22    }
23
24    @Override
25    public void mouseExited(MouseEvent e)
26    {
27    }
28 }
29
30 public class Test
31 {
32     public static void main(String[] args)
33     {
34         JFrame frame = new JFrame("Ejemplo");
35
36         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
37
38         EventosDelMouse escuchador = new EventosDelMouse(); // <-- Lo instanciamos
39         frame.getContentPane().addMouseListener(escuchador); // <-- Lo registramos
40         frame.setSize(400, 300);
41
42         frame.setVisible(true);
43     }
44 }

```

Como se puede ver, se ha definido una clase que implementa la interfaz. Esto significa que **TODAS** las operaciones deben estar en la clase concreta¹. Después, se usa `addMouseListener` para indicar que la instancia “escuchador” será la que responderá a los eventos del mouse de la ventana.

Cuando el usuario haga “click” en la ventana, `Swing` llamará automáticamente el método `mouseClicked` de la instancia, ejecutándose todo el código ahí presente (en este caso, solamente un `print`). Además pasará como parámetro una instancia de `MouseEvent`, que contendrá información relevante del evento sucedido.

En este ejemplo en particular, uno se puede preguntar qué pasa con el resto de los eventos. En teoría, se puede reaccionar a otros eventos diferentes del “click” (botón presionado, botón liberado, etc). De

¹Hay otras formas de hacer esto mismo, sin tener que implementar **TODAS** las operaciones. Si quieres saber cómo, mira <https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/java/awt/event/MouseAdapter.html>

acuerdo a cómo está el código, `Swing` llamará a los métodos correspondientes cuando los eventos suceden. Pero no hay código en esos métodos, así que no sucederá nada.

Nuevamente, nótese que aun cuando no necesitemos reaccionar a otros eventos, la instancia que pasemos a `addMouseListener` debe implementar todas las operaciones de la interfaz `MouseListener`. Si no es así, entonces el código no compilará ya que técnicamente la clase no implementa correctamente la interfaz:

```
1 class EventosDelMouse implements MouseListener
2 {
3     @Override
4     public void mouseClicked(MouseEvent e)
5     {
6         System.out.println("Alguien hizo click!");
7     }
8 }
```

En este caso, esta clase no implementa correctamente la interfaz.

En `Swing` se pueden definir otros eventos, por ejemplo, podemos detectar cuando el mouse se mueve sobre un componente. Estos eventos se recogen usando otro tipo de interfaz:

```
1 JFrame frame = new JFrame("Ejemplo");
2
3 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
4
5 EventosDelMouse escuchador = new EventosDelMouse();
6 frame.getContentPane().addMouseListener(escuchador);
7
8 frame.getContentPane().addMouseMotionListener(escuchadorMotion); // <--
9
10 frame.setSize(400, 300);
11
12 frame.setVisible(true);
```

En este caso, llamando a `addMouseMotionListener` especificamos una instancia que recibirá los eventos de movimiento. La interfaz que se debe implementar es:

```
1 public interface MouseMotionListener {
2     /**
3      * Invoked when a mouse button is pressed on a component and then
4      * dragged.  {@code MOUSE_DRAGGED} events will continue to be
5      * delivered to the component where the drag originated until the
6      * mouse button is released (regardless of whether the mouse position
7      * is within the bounds of the component).
8      * <p>
9      * Due to platform-dependent Drag&Drop implementations,
10     * {@code MOUSE_DRAGGED} events may not be delivered during a native
11     * Drag&Drop operation.
12     * @param e the event to be processed
13     */
14     public void mouseDragged(MouseEvent e);
15
16     /**
17     * Invoked when the mouse cursor has been moved onto a component
18     * but no buttons have been pushed.
19     * @param e the event to be processed
20     */
21     public void mouseMoved(MouseEvent e);
22 }
```

Para completar el ejemplo, se requiere instanciar un objeto que implemente dicha interfaz:

```

1 class EventosMotion implements MouseMotionListener
2 {
3     @Override
4     public void mouseDragged(MouseEvent e)
5     {
6
7     }
8     @Override
9     public void mouseMoved(MouseEvent e)
10    {
11        System.out.println("Mouse movido!");
12    }
13 }
14 class EventosDelMouse implements MouseListener { ... }
15
16 public class Test
17 {
18     public static void main(String[] args)
19     {
20         JFrame frame = new JFrame("Ejemplo");
21
22         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23
24         EventosDelMouse escuchador = new EventosDelMouse();
25         frame.getContentPane().addMouseListener(escuchador);
26
27         EventosMotion escuchadorMotion = new EventosMotion();
28         frame.getContentPane().addMouseMotionListener(escuchadorMotion);
29
30         frame.setSize(400, 300);
31
32         frame.setVisible(true);
33     }
34 }

```

En este caso, se han declarado dos clases diferentes, y de cada una se creó una instancia que recibirá los mensajes desde Swing. Esto es totalmente válido y funciona.

Muchas veces, al crear clases que implementan interfaces, conviene recordar que una clase puede realmente implementar varias interfaces en forma simultánea. Podemos refactorizar nuestro ejemplo para demostrar esto:

```

1 class EventosDelMouse implements MouseListener, MouseMotionListener // DOS interfaces!
2 {
3     @Override
4     public void mouseClicked(MouseEvent e) { System.out.println("Click!"); }
5     @Override
6     public void mouseMoved(MouseEvent e) { System.out.println("Mouse movido!"); }
7     @Override
8     public void mousePressed(MouseEvent e) {}
9     @Override
10    public void mouseReleased(MouseEvent e) {}
11    @Override
12    public void mouseEntered(MouseEvent e) {}
13    @Override
14    public void mouseExited(MouseEvent e) {}
15    @Override
16    public void mouseDragged(MouseEvent e) {}
17 }

```

```
1 public class Test
2 {
3     public static void main(String[] args)
4     {
5         JFrame frame = new JFrame("Ejemplo");
6
7         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
8
9         EventosDelMouse escuchador = new EventosDelMouse(); // <-- Una instancia
10        frame.getContentPane().addMouseListener(escuchador); // <-- Registrada ...
11        frame.getContentPane().addMouseMotionListener(escuchador); // <-- ... dos veces
12
13        frame.setSize(400, 300);
14
15        frame.setVisible(true);
16    }
17 }
```

Desde el punto de vista de `Swing`, solo le interesa que la instancia que recibe implemente la interfaz indicada. En este caso, como la clase (y por lo tanto la instancia) implementa ambas interfaces, no hay problema en usarla como receptora de ambos tipos de eventos (`Swing` no se confundirá al llamar a los métodos correspondientes cuando sucedan los eventos).

Esta misma técnica descrita para eventos del “mouse” se utiliza para otros tipos de eventos, y es la base para poder construir aplicaciones gráficas en Java.

Apéndice F

Utilidades

Java incluye muchas cosas que pueden ser muy útiles para realizar tareas frecuentes o repetitivas, y que muchas veces son más fáciles de usar y más eficientes que ejecutarlas de la forma “normal”.

F.1. ¿Qué es Integer?

Seguramente ya has visto (y seguirás viendo) casos donde en vez de usar el tipo de dato primitivo `int` se usa `Integer`. Y tal vez te has preguntado cuál es la diferencia.

La clase `Integer` y el tipo primitivo `int` son dos tipos de datos que representan números enteros. Sin embargo, existen algunas diferencias clave entre ambos:

- La clase `Integer` es un tipo de datos de referencia, mientras que `int` es un tipo de datos primitivo. Esto significa que una variable de tipo `Integer` apunta a un objeto de la clase `Integer`, mientras que una variable de tipo `int` simplemente almacena el valor del número entero.
- La clase `Integer` proporciona una serie de métodos y propiedades que no están disponibles en el tipo primitivo `int`. Por ejemplo, la clase `Integer` tiene métodos para convertir números enteros a `String`, para realizar operaciones aritméticas con números enteros, y para comprobar si un número entero es par o impar.

La clase `Integer` es una clase de wrapper que encapsula un valor de tipo primitivo `int`. Lo anterior permite que pueda ser usada en todos aquellos lugares donde se requiera una instancia, por ejemplo, al guardar elementos en una `List`, o como llave o valor en una `Hashtable`.¹

Algo interesante de la clase `Integer` es que para ahorrar memoria se usa un cache de instancias, que pueden ser reutilizadas de forma automática.

Por ejemplo, mira este código:

¹Por si no te habías dado cuenta, no se puede crear una `ArrayList<int>`

```
1 public class EjemploInteger
2 {
3     public static void main(String[] args)
4     {
5         Integer a = Integer.valueOf(10);
6         Integer b = 10;
7
8         System.out.println(a == b);
9
10        Integer c = new Integer(10);
11
12        System.out.println(a == c);
13
14        Integer d = Integer.valueOf("10");
15
16        System.out.println(a == d);
17    }
18 }
```

Y su resultado:

```
$ java EjemploInteger
true
false
true
```

En otras palabras:

```
1 public class EjemploInteger
2 {
3     public static void main(String[] args)
4     {
5         Integer a = Integer.valueOf(10);
6         Integer b = 10;
7
8         // Aun cuando "b" se creó de forma diferente que "a", ambas variables
9         // referencian a la misma instancia de Integer
10        System.out.println(a == b);
11
12        Integer c = new Integer(10); // Forzamos la creación de una nueva instancia
13
14        // Por lo que verificamos que son instancias diferentes
15        System.out.println(a == c);
16
17        Integer d = Integer.valueOf("10");
18
19        // Incluso si viene de un String, se recibe la misma instancia
20        System.out.println(a == d);
21    }
22 }
```

F.2. Expresiones Lambda

Este tipo de expresiones son pequeños trozos de código, que toman parámetros como entrada y producen un resultado. Son similares a los métodos, pero no necesitan tener nombre y se implementan directamente donde los necesitemos.

Estas expresiones se escriben así:

```
1 (parámetro1, parámetro2) -> expresión
```

Si solo tenemos un solo parámetro, se puede escribir más simple aun:

```
1 parámetro -> expresión
```

Cuando la expresión es simple, ésta debe retornar un valor de forma inmediata. Tampoco puede contener variables.

Si se requiere incluir código más complejo, se puede especificar un bloque de código usando `{ }`. Si la expresión debe retornar un valor, se debe incluir el `return` correspondiente.

```
1 (parámetro1, parámetro2) -> { bloque de código }
```

F.3. `forEach()` en la interfaz `Iterable`

Un uso muy útil de las expresiones `lambda` es al iterar sobre colecciones. Por ejemplo, si queremos mostrar por pantalla cada elemento de una lista, la forma común de hacerlo es escribiendo lo siguiente:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         List<Integer> lista = new ArrayList<>();
6
7         lista.add(4);
8         lista.add(7);
9         lista.add(1);
10        lista.add(3);
11        lista.add(8);
12
13        for(int i=0;i<lista.size();i++) {
14            System.out.println(lista.get(i));
15        }
16    }
17 }
```

O también se puede escribir así usando iteradores:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         List<Integer> lista = new ArrayList<>();
6
7         lista.add(4);
8         lista.add(7);
9         lista.add(1);
10        lista.add(3);
11        lista.add(8);
12
13        for (Iterator<Integer> iterator = lista.iterator(); iterator.hasNext();) {
14            Integer t = iterator.next();
15            System.out.println(t);
16        }
17    }
18 }
```

Y una forma más moderna de hacer lo mismo es:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         List<Integer> lista = new ArrayList<>();
6
7         lista.add(4);
8         lista.add(7);
9         lista.add(1);
10        lista.add(3);
11        lista.add(8);
12
13        for (Integer t : lista) {
14            System.out.println(t);
15        }
16    }
17 }
```

Pero la mejor forma es usando una expresión `lambda`:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         List<Integer> lista = new ArrayList<>();
6
7         lista.add(4);
8         lista.add(7);
9         lista.add(1);
10        lista.add(3);
11        lista.add(8);
12
13        lista.forEach( x -> System.out.println(x) );
14    }
15 }
```

La forma de analizar la expresión, es que se iterará sobre la lista, y en cada elemento se llamará a la expresión, haciendo que `x` adquiriera el valor del elemento. No es el valor del índice, sino que el valor del elemento en sí. Al ejecutar este código se obtiene lo esperado:

```
$ java Main
4
7
1
3
8
```

En este caso, como la lista es de enteros, corresponde que la expresión `lambda` tenga un solo parámetro, y el cuerpo de la expresión es solo de una instrucción, pero igual se puede escribir como si fuera una expresión más compleja:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         List<Integer> lista = new ArrayList<>();
6
7         lista.add(4);
8         lista.add(7);
9         lista.add(1);
10        lista.add(3);
11        lista.add(8);
12
13        lista.forEach( (x) -> { System.out.println(x); } );
14    }
15 }
```

Para más información y detalles acerca de las expresiones lambda, se puede revisar la documentación oficial en

<https://docs.oracle.com/javase/tutorial/java/java00/lambdaexpressions.html>

F.4. Interfaces funcionales

Una interfaz que tenga exactamente un método se convierte en algo llamado una “interfaz funcional”. Ya que solo tienen un método, se pueden usar expresiones `lambda` al utilizarlas, en vez de tener que declarar clases concretas que la implementen.

Por ejemplo, podemos declarar una interface para sumar enteros:

```
1 public interface Sumador
2 {
3     int sumar(int a, int b);
4 }
```

Y crear un sumador concreto que realice la acción:

```
1 public class SumadorConcreto implements Sumador
2 {
3     @Override
4     public int sumar(int a, int b)
5     {
6         return a+b;
7     }
8 }
```

Y probar que todo funciona instanciándolo:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         Sumador s = new SumadorConcreto();
6         System.out.println(s.sumar(1, 2));
7         System.out.println(s.sumar(3, 7));
8     }
9 }
```

Como se puede esperar, el resultado de la ejecución es:

```
$ java Main
3
10
```

Pero, lo que también se podría hacer es ahorrarnos la creación de la clase. Debido a que la interfaz tiene solo un método, podemos eliminar la clase `SumadorConcreto` y cambiar el `main` a esto:

```
1 public class Main
2 {
3     public static void main(String[] args)
4     {
5         Sumador s = (x,y) -> x + y;
6         System.out.println(s.sumar(1, 2));
7         System.out.println(s.sumar(3, 7));
8     }
9 }
```

Y el resultado seguirá siendo el mismo que antes. Se creó una clase anónima (o sea, una clase que es puro código, pero sin nombre), que tiene la misma funcionalidad de la clase inicial. Java sabe que el

tipo de la “clase” creada debe ser compatible con `Sumador`, ya que estamos asignando el resultado a la variable `s` (que es de tipo `sumador`), y como dicha interfaz solo tiene una operación, queda totalmente especificada.

El uso de interfaces funcionales, aun cuando no es obligatorio, ofrece varios beneficios:

Expresividad Las interfaces funcionales permiten definir comportamientos específicos de manera más concisa y expresiva. En lugar de crear clases completas para implementar una única función, puedes utilizar expresiones lambda para representar el comportamiento de manera más directa y legible.

Reducción de código El uso de interfaces funcionales puede reducir la cantidad de código que necesitas escribir. Las expresiones lambda y las referencias a métodos son mucho más breves que las implementaciones de clases completas, lo que facilita el mantenimiento y la lectura del código.

Composición de funciones Con las interfaces funcionales puedes componer funciones de manera más sencilla. Puedes encadenar múltiples funciones pequeñas para crear una lógica más compleja, lo que promueve la modularidad y la reutilización del código.

Parallelismo y concurrencia La programación funcional y el uso de interfaces funcionales facilitan la escritura de código paralelo y concurrente. Las funciones inmutables y sin estado son más seguras para el paralelismo, ya que no tienen efectos secundarios, lo que puede hacer que tu código sea más robusto y escalable.

API de Java Streams Java Streams, introducido en Java 8, es un ejemplo poderoso del uso de interfaces funcionales. Permite operaciones de procesamiento de datos en secuencias de elementos de manera eficiente y declarativa. Al usar interfaces funcionales puedes realizar transformaciones y filtrados en colecciones de manera más legible y eficiente.

Interoperabilidad Las interfaces funcionales son compatibles con las bibliotecas y API existentes en Java.

F.5. Tipos de excepciones en Java

En Java las excepciones pueden ser de dos categorías: “checked exceptions” y “unchecked exceptions”. En ambos casos se trata de excepciones, pero difieren en su significado y la forma en que escribiremos el código para tomar en cuenta su existencia.

F.5.1. Checked Exceptions

Esta categoría de excepciones son del tipo que se verifican en tiempo de compilación, y representan errores que están fuera del control del programa. La forma en que se definen asegura que se está tomando acción sobre una excepción lanzada.

Por ejemplo, la definición del constructor de la clase `Scanner` que recibe una instancia de `File` como parámetro es así:

```
1 public Scanner(File source) throws FileNotFoundException
2 {
3     ...
4 }
```

La parte del `throws FileNotFoundException` indica que cualquiera que quiera llamar a ese constructor (o a un método, si es que fuera un método el que tuviera la especificación de `throws`), debe obligatoriamente tomar en cuenta que la invocación puede lanzar una excepción. Ésto se puede realizar de dos maneras:

O se ataja directamente la posible excepción:

```
1 private void g()
2 {
3     try {
4         Scanner scan = new Scanner(new File("archivo.txt"));
5     } catch (FileNotFoundException e) {
6     }
7 }
8 }
```

O se indica que el método donde se está haciendo la invocación también podrá lanzar una excepción:

```
1 private void f() throws FileNotFoundException
2 {
3     Scanner scan = new Scanner(new File("archivo.txt"));
4 }
```

En este caso, cualquier método que a la vez invoque al método `f()` también deberá tomar en cuenta que la invocación podría lanzar una excepción.

La clase `Exception` es la super clase de todas las excepciones de este tipo, excepto las que derivan de `RuntimeException`, así que es fácil crear un nuevo tipo:

```
1 public class WrongDayException extends Exception
2 {
3     public WrongDayException(String mensaje)
4     {
5         super(mensaje);
6     }
7 }
```

F.5.2. Unchecked Exceptions

El otro tipo de excepciones se usan generalmente para señalar errores lógicos dentro del programa. Por ejemplo:

```
1 private static void dbz()
2 {
3     int a = 1;
4     int b = 0;
5     int c = a / b;
6 }
```

Este método no declara que pueda lanzar ninguna excepción, pero al ser ejecutado lanzará de todos modos una `ArithmeticException`.

El proceso de compilación no detecta este tipo de errores, pero muchas de las excepciones más comunes son de este tipo:

- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `IllegalArgumentException`
- etc.

La clase `RuntimeException` es la superclase de todos tipos de excepciones, por lo que es fácil crear excepciones nuevas:

```
1 public class UglyDay extends RuntimeException
2 {
3     public UglyDay(String mensaje)
4     {
5         super(mensaje);
6     }
7 }
```

F.5.3. Cuándo usar una o la otra

De acuerdo a la documentación oficial:

Si es razonable esperar que un cliente se pueda recuperar de una excepción, que sea de tipo checked. Si un cliente nada puede hacer para recuperarse de una excepción, entonces que sea de tipo unchecked.

Para más detalles, se puede revisar la documentación oficial:

<https://docs.oracle.com/javase/tutorial/essential/exceptions/runtime.html>

La documentación de toda la rama de errores contenidos en la JVM se puede encontrar² en

<https://docs.oracle.com/javase/8/docs/api/java/lang/Throwable.html>

A continuación se puede ver un extracto resumido de los tipos de excepciones más comunes:

Throwable La clase **Throwable** es la superclase de todos los errores y excepciones en Java. Solo los objetos que son instancias de esta clase (o una de sus subclases) son lanzados por la máquina virtual, o pueden ser lanzados usando la sentencia **throw**. De forma similar, solo esta clase o una de sus subclases se pueden usar como tipo en la sentencia **catch**. La clase **Throwable** y cualquiera de sus subclases, que no sea también subclase de **RuntimeException** o **Error** se consideran excepciones checked.

Error Un **Error** es una subclase que indica un problema serio que cualquiera aplicación razonable no debe tratar de atrapar. La mayoría de estos errores son condiciones anormales.

IOException Se lanza cuando ha ocurrido un error serio de I/O.³

VirtualMachineError Se lanza para indicar que la JVM no funciona, o se le han acabado los recursos necesarios para continuar operando.

etc Hay muchas otras clases en esta categoría.

Exception La clase **Exception** y sus subclases son una forma de **Throwable** que indican condiciones que una aplicación razonable puede querer atajar.

RuntimeException Esta es la super clase de aquellas excepciones que se pueden lanzar durante la operación normal de la JVM. **RuntimeException** y sus subclase son unchecked exceptions, por lo que no necesitan ser declaradas en la cláusula throws en los métodos o constructores.

ArithmeticException Lanzada cuando ocurre una condición aritmética excepcional.

IndexOutOfBoundsException Lanzada para indicar que algún tipo de índice (en un arreglo, string, vector) está fuera de rango.

IllegalStateException Señaliza que un método ha sido invocado en un momento ilegal o inapropiado.

²Para su versión 8.

³Input/Output: Entrada/Salida, todo lo que tiene que ver con leer desde archivos, leer recursos desde redes, etc.

`IllegalArgumentException` Se lanza para indicar que a un método se le ha pasado un argumento ilegal o inapropiado.

`NullPointerException` Se lanza cuando una aplicación intenta usar un `null` cuando en realidad se requiere un objeto. Estos casos incluyen:

- Llamar un método de instancia de un objeto `null`.
- Accesar o modificar un atributo de un objeto `null`.
- Obtener la longitud de `null` como si fuera un arreglo.
- Accesar o modificar las celdas de `null` como si fuera un arreglo.
- Lanzar `null` como si fuera una instancia de `Throwable`.

etc Hay muchas otras clases en esta categoría.

`IOException` Señaliza que una excepción de I/O ha ocurrido. Esta clase es la base para las excepciones producidas en el caso de operaciones de I/O falladas o interrumpidas.

etc Hay muchas otras clases en esta categoría.

Apéndice G

Refactorización

El refactoring es una práctica esencial en el desarrollo de software que implica la reestructuración del código existente para mejorar su legibilidad, mantenibilidad y eficiencia sin cambiar su funcionalidad. Esta técnica es fundamental para mantener un código limpio y fácil de mantener a medida que evoluciona un proyecto. Exploraremos qué es el refactoring, por qué es importante y proporcionaremos ejemplos concretos para ilustrar cómo se aplica en la programación.

G.1. ¿Qué es el refactoring?

El refactoring es el proceso de modificar el código fuente de una aplicación sin cambiar su comportamiento externo. Se centra en mejorar la estructura interna del código para hacerlo más claro, eficiente y fácil de mantener. El objetivo principal del refactoring es reducir la deuda técnica, que es el costo a largo plazo de mantener un código de baja calidad.

G.2. ¿Por qué es importante el refactoring?

El refactoring es esencial por varias razones:

- Mejora la legibilidad: Un código bien refactorizado es más claro y fácil de entender, lo que facilita la colaboración entre desarrolladores y reduce la probabilidad de errores.
- Facilita el mantenimiento: Un código refactorizado es más fácil de mantener y actualizar, lo que ahorra tiempo y recursos a lo largo del ciclo de vida del software.
- Aumenta la eficiencia: Un código más limpio y eficiente puede mejorar el rendimiento de una aplicación.
- Facilita la detección temprana de errores: El código refactorizado suele ser más robusto y menos propenso a errores, lo que facilita la detección temprana de problemas durante el desarrollo.

G.3. Ejemplos de refactoring en Java

A continuación, se presentan algunos ejemplos concretos de técnicas de refactoring en Java:

Extracción de métodos Supongamos que tenemos un método largo que realiza varias tareas. Podemos refactorizarlo dividiéndolo en varios métodos más pequeños, cada uno encargado de una tarea específica. Esto hace que el código sea más modular y fácil de entender.

```
1 // Código original
2 public void procesarDatos() {
3     // Lógica para obtener datos
4     // Lógica para validar datos
5     // Lógica para guardar datos
6 }
7
8 // Código refactorizado
9 public void procesarDatos() {
10     obtenerDatos();
11     validarDatos();
12     guardarDatos();
13 }
14
15 private void obtenerDatos() {
16     // Lógica para obtener datos
17 }
18
19 private void validarDatos() {
20     // Lógica para validar datos
21 }
22
23 private void guardarDatos() {
24     // Lógica para guardar datos
25 }
```

Renombramiento de variables A veces, los nombres de variables pueden ser poco descriptivos. Refactorizarlos con nombres más significativos mejora la comprensión del código.

```
1 // Código original
2 int a = 10;
3 int b = 20;
4
5 // Código refactorizado
6 int numero1 = 10;
7 int numero2 = 20;
```

Extracción de constantes Cuando se utilizan valores constantes en varias partes del código, es una buena práctica crear constantes para ellos. Esto hace que el código sea más claro y facilita la actualización de valores.

```
1 // Código original
2 double precio = 100.0;
3 double impuesto = 0.15;
4
5 double total = precio + (precio * impuesto);
6
7 // Código refactorizado
8 final double PRECIO_BASE = 100.0;
9 final double IMPUESTO = 0.15;
10
11 double total = PRECIO_BASE + (PRECIO_BASE * IMPUESTO);
```

Eliminación de código no utilizado Es importante eliminar el código que ya no se utiliza, ya que puede causar confusión y aumentar la complejidad.

```
1 // Código original
2 public void metodoAntiguo() {
3     // Código obsoleto
4 }
5
6 // Código refactorizado
7 // El método obsoleto se elimina por completo
```

Uso de ciclos mejorados Cuando se trabaja con colecciones, es preferible utilizar ciclos mejorados (for-each) en lugar de ciclos tradicionales para mejorar la legibilidad del código.

```
1 // Código original
2 List<String> nombres = new ArrayList<>();
3 for (int i = 0; i < nombres.size(); i++) {
4     String nombre = nombres.get(i);
5     System.out.println(nombre);
6 }
7
8 // Código refactorizado
9 List<String> nombres = new ArrayList<>();
10 for (String nombre : nombres) {
11     System.out.println(nombre);
12 }
```

Reemplazo de código duplicado Eliminar duplicados es una parte fundamental del refactoring. Si encuentra el mismo código en varios lugares, debe refactorizarlo para reutilizarlo en una sola ubicación.

```
1 // Código original
2 public double calcularDescuento(double precio) {
3     if (precio > 100) {
4         return precio * 0.1;
5     } else {
6         return precio * 0.05;
7     }
8 }
9
10 // Código refactorizado
11 public double calcularDescuento(double precio) {
12     double porcentaje = (precio > 100) ? 0.1 : 0.05;
13     return precio * porcentaje;
14 }
```

El refactoring en Java es una práctica esencial para mantener un código limpio y de alta calidad. Los ejemplos proporcionados ilustran algunas de las técnicas comunes utilizadas en el refactoring. Al aplicar estas técnicas de manera consistente, los desarrolladores pueden mejorar la legibilidad, la mantenibilidad y la eficiencia de su código Java, lo que a su vez conduce a un desarrollo de software más exitoso y menos propenso a errores.

Apéndice H

Clases Avanzadas

H.1. StringBuilder

Tal como dice la documentación oficial, las instancias de esta clase representan una secuencia mutable de caracteres. Sus métodos permiten agregar contenido a un buffer, y posteriormente recibir el `String` resultante de todas las operaciones.

Con el siguiente ejemplo se puede ver la diferencia en velocidad al realizar la misma operación usando una concatenación normal con `String` versus usando un `StringBuilder`:

```
1 public class Main {
2     public static void main(String[] args) {
3         testConcatString();
4         testStringBuilder();
5     }
6
7     private static void testStringBuilder() {
8         StringBuilder sb = new StringBuilder();
9         long start = System.currentTimeMillis(); // Guardamos la hora actual
10        for (int i = 0; i < 10000; i++) {
11            sb.append("numero ").append(i).append("\n");
12        }
13        String texto = sb.toString();
14        long elapsed = System.currentTimeMillis() - start; // Cuánto tiempo pasó
15        System.out.println(texto.length() + " en " + elapsed + " mseg");
16    }
17
18    private static void testConcatString() {
19        long start = System.currentTimeMillis();
20        String texto = "";
21        for (int i = 0; i < 10000; i++) {
22            texto += "numero " + i + "\n";
23        }
24        long elapsed = System.currentTimeMillis() - start;
25        System.out.println(texto.length() + " en " + elapsed + " mseg");
26    }
27 }
```

Al ejecutar lo anterior se obtiene:

```
$ java Main
118890 en 405 mseg
118890 en 1 mseg
```

La diferencia en desempeño se explica ya que en Java las instancias de `String` son inmutables: cada vez que se concatena algo a un `String`, se crea una nueva instancia. En cambio el `StringBuilder` recibe todas las concatenaciones en un mismo buffer, y solo debe crear un solo `String` al final, cuando invocamos `toString()`.

H.2. Hashtable

La clase `Hashtable` es una estructura de datos que asocia llaves o claves con valores. Es una implementación de la interfaz `Map`, que proporciona métodos para agregar, eliminar, buscar y obtener valores de una tabla hash.

Una tabla hash es una estructura de datos que asocia llaves o claves con valores. Las llaves son únicas y se utilizan para acceder a los valores. Los valores pueden ser de cualquier tipo de datos.

Las tablas hash funcionan mediante un proceso llamado hashing. El hashing es básicamente una función que transforma una llave (por ejemplo, el valor de un `String`) en un índice de la tabla hash. El índice se utiliza para finalmente saber en qué posición de la tabla se encuentra el valor que andamos buscando.

La clase `Hashtable` proporciona muchos métodos, entre los que se encuentran:

- `put(key, value)`: Agrega una nueva entrada a la tabla hash.
- `get(key)`: Devuelve el valor asociado a la llave.
- `remove(key)`: Elimina una entrada de la tabla hash.
- `containsKey(key)`: Devuelve true si la tabla hash contiene la llave.
- `containsValue(value)`: Devuelve true si la tabla hash contiene el valor.
- `size()`: Devuelve el número de entradas en la tabla hash.
- `isEmpty()`: Devuelve true si la tabla hash está vacía.

Nótese que la clase es genérica, por lo que se requiere especificar sus tipos (tanto de llave como valor) al momento de instanciarla.

H.2.1. Ejemplos

1. Agregar una nueva entrada

```
1 Hashtable<String, Integer> hashtable = new Hashtable<>();
2
3 hashtable.put("nombre", 1);
4 hashtable.put("edad", 25);
```

2. Obtener el valor asociado a una llave

```
1 Hashtable<String, Integer> hashtable = new Hashtable<>();
2
3 hashtable.put("nombre", 1);
4 hashtable.put("edad", 25);
5
6 int valor = hashtable.get("nombre");
7
8 System.out.println(valor); // 1
```

3. Eliminar una entrada

```
1 Hashtable<String, Integer> hashtable = new Hashtable<>();
2
3 hashtable.put("nombre", 1);
4 hashtable.put("edad", 25);
5
6 hashtable.remove("nombre");
7
8 int tamaño = hashtable.size();
9
10 System.out.println(tamaño); // 1
```

4. Comprobar si la tabla hash contiene una llave

```
1 Hashtable<String, Integer> hashtable = new Hashtable<>();
2
3 hashtable.put("nombre", 1);
4 hashtable.put("edad", 25);
5
6 boolean contiene = hashtable.containsKey("nombre");
7
8 System.out.println(contiene); // true
```

5. Comprobar si la tabla hash contiene un valor

```
1 Hashtable<String, Integer> hashtable = new Hashtable<>();
2
3 hashtable.put("nombre", 1);
4 hashtable.put("edad", 25);
5
6 boolean contiene = hashtable.containsValue(1);
7
8 System.out.println(contiene); // true
```

H.2.2. ¿Por qué usar una Hashtable?

La principal ventaja de usar este objeto (en vez de buscar los elementos en una lista), es su **velocidad**.

```

1 class Persona {
2     private String rut;
3     private String nombre;
4
5     public Persona(String rut, String nombre)
6     {
7         this.rut = rut;
8         this.nombre = nombre;
9     }
10    public String getRut() { return rut; }
11    public String getNombre() { return nombre; }
12 }
13
14 public class Hashspeed {
15     private static final int NUM = 20000;
16
17     public static void main(String[] args)
18     {
19         testBuscarList();
20         testHashtable();
21     }
22
23     private static void testHashtable()
24     {
25         Hashtable<String, Persona> lista = new Hashtable<>();
26
27         for (int i = 0; i < NUM; i++) {
28             Persona p = new Persona(String.valueOf(i), "Nombre"+i);
29             lista.put(p.getRut(), p);
30         }
31         long start = System.currentTimeMillis(); // Guardamos la hora actual
32         for (int i = 0; i < NUM; i++) {
33             lista.get(String.valueOf(i));
34         }
35         long elapsed = System.currentTimeMillis() - start; // Cuánto tiempo pasó
36         System.out.println("testHashtable en " + elapsed + " mseg");
37     }
38
39     private static void testBuscarList()
40     {
41         List<Persona> lista = new LinkedList<>();
42
43         for (int i = 0; i < NUM; i++) {
44             lista.add(new Persona(String.valueOf(i), "Nombre"+i));
45         }
46         long start = System.currentTimeMillis(); // Guardamos la hora actual
47         for (int i = 0; i < NUM; i++) {
48             for (Persona p : lista) {
49                 if (p.getRut().equals(String.valueOf(i))) {
50                     break; // encontrado!
51                 }
52             }
53         }
54         long elapsed = System.currentTimeMillis() - start; // Cuánto tiempo pasó
55         System.out.println("testBuscarList en " + elapsed + " mseg");
56     }
57 }

```

Al ejecutar lo anterior se obtiene:

```
$ java Hashspeed
testBuscarList en 3807 mseg
testHashtable en 2 mseg
```

En resumen, buscar secuencialmente un elemento es muchas órdenes de magnitud más lento que usar una Hashtable, ya que la función de hash permite saber casi instantáneamente la ubicación del elemento que se está buscando.

H.3. Ordenando

Tal como se mencionó en la página 349, existen varias formas de ordenar una colección de elementos, y seleccionar la forma apropiada depende del problema particular que estemos resolviendo.

H.3.1. Interfaz Comparable

Esta forma de ordenar ya se explicó en la página 349.

H.3.2. Usando un Comparator y Collections

Una forma más flexible de ordenar una colección es usando una instancia de alguna clase que implemente la interfaz `Comparator`, y aprovechar el método estático `sort(...)` de la clase `Collections`. A dicho método se le pasará la instancia de la colección a ordenar y una instancia de una clase que sabe cómo ordenar las instancias presentes dentro de la colección.

Por ejemplo:

```
1 public class Orden
2 {
3     public static void main(String[] args)
4     {
5         ArrayList<Integer> enteros = new ArrayList<>();
6
7         enteros.add(40);
8         enteros.add(10);
9         enteros.add(3);
10        enteros.add(50);
11        enteros.add(20);
12
13        System.out.println(enteros.toString());
14        Collections.sort(enteros, new OrdenadorEnteros());
15        System.out.println(enteros.toString());
16    }
17 }
```

Para que todo funcione, se necesita implementar la clase `OrdenadorEnteros`:

```
1 public class OrdenadorEnteros implements Comparator<Integer>
2 {
3     @Override
4     public int compare(Integer o1, Integer o2)
5     {
6         return 0;
7     }
8 }
```

Al ejecutar el programa se obtiene:

```
$ java Orden
[40, 10, 3, 50, 20]
[40, 10, 3, 50, 20]
```

La verdad, no ordena nada, pero es por que aun no terminamos de implementar el ejemplo. Lo que falta es que se necesita completar el método `compare(...)`. De acuerdo a la documentación de la interfaz `Comparator`, el método debe cumplir el siguiente contrato:

Compara los dos argumentos para determinar su orden relativo. Retorna un entero negativo, cero o positivo si el primero argumento es menor, igual o mayor que el segundo.

Con lo anterior, entonces podemos completar el código:

```
1 public class OrdenadorEnteros implements Comparator<Integer>
2 {
3     @Override
4     public int compare(Integer a, Integer b)
5     {
6         if (a < b) return -1;
7         if (a > b) return 1;
8         return 0;
9     }
10 }
```

Ahora, al ejecutar el nuevo programa, se obtiene:

```
$ java Orden
[40, 10, 3, 50, 20]
[3, 10, 20, 40, 50]
```

Esto significa que si queremos invertir el orden de la lista, basta invertir los criterios en la clase comparadora:

```
1 public class OrdenadorEnteros implements Comparator<Integer>
2 {
3     @Override
4     public int compare(Integer a, Integer b)
5     {
6         if (a < b) return 1;
7         if (a > b) return -1;
8         return 0;
9     }
10 }
```

Ahora, al ejecutar el nuevo programa, se obtiene:

```
$ java Orden
[3, 10, 20, 40, 50]
[50, 40, 20, 10, 3]
```

Una forma de ahorrar espacio (y archivos), es aprovechar el hecho de que la interfaz `Comparator` es una interfaz funcional (revisa la página 451 para más detalles), y reescribir el ejemplo de esta manera:

```
1 public class Orden
2 {
3     public static void main(String[] args)
4     {
5         List<Integer> enteros = new ArrayList<>();
6
7         enteros.add(40);
8         enteros.add(10);
9         enteros.add(3);
10        enteros.add(50);
11        enteros.add(20);
12
13        System.out.println(enteros.toString());
14        Collections.sort(enteros, (a,b) -> {
15            if (a < b) return -1;
16            if (a > b) return 1;
17            return 0;
18        });
19        System.out.println(enteros.toString());
20    }
21 }
```

Ahora, al ejecutar el nuevo programa, se obtiene:

```
$ java Orden
[40, 10, 3, 50, 20]
[3, 10, 20, 40, 50]
```

Y una última versión se puede escribir aprovechando la relación matemática de los parámetros del sort y el resultado esperado (y nótese que el método anónimo ya no tiene `{}`):

```
1 public class Orden
2 {
3     public static void main(String[] args)
4     {
5         List<Integer> enteros = new ArrayList<>();
6
7         enteros.add(40);
8         enteros.add(10);
9         enteros.add(3);
10        enteros.add(50);
11        enteros.add(20);
12
13        System.out.println(enteros.toString());
14        Collections.sort(enteros, (a,b) -> a - b);
15        System.out.println(enteros.toString());
16    }
17 }
```

Finalmente, al ejecutar el nuevo programa, se sigue obteniendo el resultado correcto:

```
$ java Orden  
[40, 10, 3, 50, 20]  
[3, 10, 20, 40, 50]
```

H.3.3. Usando un Comparator y .sort()

Una última forma de ordenar una lista es aprovechar que la interfaz `List` tiene el método `sort()`, cuyo propósito es ordenar la misma lista, usando un comparador para decidir el orden de los elementos:

```
1 public class Orden  
2 {  
3     public static void main(String[] args)  
4     {  
5         List<Integer> enteros = new ArrayList<>();  
6  
7         enteros.add(40);  
8         enteros.add(10);  
9         enteros.add(3);  
10        enteros.add(50);  
11        enteros.add(20);  
12  
13        System.out.println(enteros.toString());  
14        enteros.sort((a,b) -> a - b);  
15        System.out.println(enteros.toString());  
16    }  
17 }
```

Ahora, al ejecutar el nuevo programa, se sigue obteniendo el resultado correcto:

```
$ java Orden  
[40, 10, 3, 50, 20]  
[3, 10, 20, 40, 50]
```


H.3.4. Ordenando otros tipos de objetos

Si los objetos a ordenar son de tipo `String`, podemos aprovechar el método `compareTo(...)` que la misma clase nos provee:

```
1 public class Orden2
2 {
3     public static void main(String[] args)
4     {
5         List<String> lista = new ArrayList<>();
6
7         lista.add("hola");
8         lista.add("malo");
9         lista.add("barco");
10        lista.add("zulu");
11        lista.add("bye");
12
13        System.out.println(lista.toString());
14        lista.sort((a,b) -> a.compareTo(b));
15        System.out.println(lista.toString());
16    }
17 }
```

Al ejecutar el programa, se obtiene el resultado correcto:

```
$ java Orden2
[hola, malo, barco, zulu, bye]
[barco, bye, hola, malo, zulu]
```

Si los objetos son más complejos (por ejemplo, tipos específicos del problema que se está resolviendo) se debe determinar cómo se calculará el orden de dichos elementos, tal como se discutió en la página 349. En este caso, en vez de implementar dicha lógica en el método `compareTo(...)` de la clase misma, se escribiría en el comparador. Por ejemplo, si tenemos la clase `Mascota`:

```
1 class Mascota
2 {
3     private String nombre;
4     private int edad;
5
6     public Mascota(String nombre, int edad)
7     {
8         this.nombre = nombre;
9         this.edad = edad;
10    }
11
12    public String getNombre() { return nombre; }
13    public int getEdad() { return edad; }
14    @Override
15    public String toString() { return nombre + "/" + edad; }
16 }
```

Para ordenar una lista de mascotas de acuerdo a su nombre podríamos escribir:

```
1 public class Orden3
2 {
3     public static void main(String[] args)
4     {
5         List<Mascota> lista = new ArrayList<>();
6
7         lista.add(new Mascota("Rufus", 5));
8         lista.add(new Mascota("Bona", 2));
9         lista.add(new Mascota("Zeila", 7));
10        lista.add(new Mascota("Carlota", 6));
11
12        System.out.println(lista.toString());
13        lista.sort((a,b) -> a.getNombre().compareTo(b.getNombre()));
14        System.out.println(lista.toString());
15    }
16 }
```

Al ejecutar el programa, se obtiene el resultado correcto:

```
$ java Orden3
[Rufus/5, Bona/2, Zeila/7, Carlota/6]
[Bona/2, Carlota/6, Rufus/5, Zeila/7]
```

Si queremos ordenar la lista por edad:

```
1 public class Orden3
2 {
3     public static void main(String[] args)
4     {
5         List<Mascota> lista = new ArrayList<>();
6
7         lista.add(new Mascota("Rufus", 5));
8         lista.add(new Mascota("Bona", 2));
9         lista.add(new Mascota("Zeila", 7));
10        lista.add(new Mascota("Carlota", 6));
11
12        System.out.println(lista.toString());
13        lista.sort((a,b) -> a.getEdad() - b.getEdad());
14        System.out.println(lista.toString());
15    }
16 }
```

Al ejecutar el programa, se obtiene el resultado correcto:

```
$ java Orden3
[Rufus/5, Bona/2, Zeila/7, Carlota/6]
[Bona/2, Carlota/6, Rufus/5, Zeila/7]
```

Nótese cómo en el comparador, tanto `a` como `b` adquieren el tipo correcto de variable (`Mascota` en este caso), aun cuando no está directamente especificado. El compilador infiere el tipo correcto a través del contexto en el cuál se está realizando la llamada.

Bibliografía

- [1] Michael Feathers. *Working Effectively With Legacy Code: Work Effect Leg Code __p1*. Prentice Hall Professional, 2004.
- [2] Erich Gamma, Richard Helm, Ralph Johnson, and John M. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1 edition, 1994.
- [3] C. Larman and P. Kruchten. *Applying UML and Patterns: An Introduction to Object-oriented Analysis and Design and the Unified Process*. Safari electronic books. Prentice Hall PTR, 2002.
- [4] Robert C Martin. Design principles and design patterns. *Object Mentor*, 1(34):597, 2000.

Índice alfabético

.class, 10
.length, 37

abstracción, 84
abstract factory, 249
adapter, 260
agregación, 90
alcance de bloque, 118
alcance de ciclo, 118
alcance de clase, 117
alcance de las variables, 117
alcance de método, 118
alta cohesión, 84
archivos, 56
ArithmeticException, 76, 77
arreglo sobredimensionado, 37
arreglo, tamaño, 37
arreglos, 32
arreglos bidimensionales, 34
arreglos tridimensionales, 35
arreglos unidimensionales, 32
arreglos, eliminación de un elemento, 41
arreglos, errores comunes, 38
arreglos, inserción ordenada, 43
arreglos, ordenamiento, 39
aseguramiento de la calidad, 71
asociación, 89
atributo de clase, 98
atributo estático, 98
atributos, 86
atributos estáticos, 100

bajo acoplamiento, 84
bloque de instrucciones, 18
boolean, 14
bridge, 276

buenas prácticas, 99
builder, 253
byte, 13

calidad, 71
camelcase, 19
casting, 16
catch, 75
chain of responsibility, 280
char, 14
checked exception, 453
ciclo do, 31
ciclo for, 27
ciclo while, 29
ciclos, 27
clase, 85
clase Integer, 445
clases, 85
coerción, 16
coerción de tipos, 16
colecciones, 141
comentarios, 11
comentarios de una línea, 12
comentarios multilínea, 12
command, 293
compilación, 10
composición, 91
composite, 263
constructor, 105, 107
constructor por defecto, 107
contratos, 234
convenciones de escritura, 19
convenciones java, 19
cubos, 35

debugging, 72

- declaración de variables, 13, 18
- decorator, 277
- dependency inversion, 230
- depuración, 72
- deuda técnica, 457
- diagrama de clases, 86, 96
- do, 31
- documentación de los programas, 11
- double, 14

- ejecutando programas, 103
- ejecutando programas en Java, 10
- eliminación de un elemento en arreglo, 41
- encapsulación, 83
- equals, 135
- errores al manipular arreglos, 38
- estado interno, 85
- estructura de los programas, 17, 47
- excepciones, 74
- expresiones lambda, 447

- facade, 274
- factory, 247
- final, 216
- finally, 77
- float, 14
- flyweight, 271
- for, 27
- foreach, 448
- formato de las funciones en Java, 53
- funciones en Java, 44

- generalización, 92
- generics, 148
- genéricos, 148

- Hashtable, 462
- herencia, 201

- IllegalArgumentException, 78, 79
- IllegalArgumentException, uso, 79
- incrementando variables, 25
- incrementar y después usar, 25
- IndexOutOfBoundsException, 78
- ingeniería de software, 67
- inicio del programa, 18
- inserción ordenada en un arreglo, 43
- instalación del jdk, 421
- instancia, 85
- int, 13

- Integer, 445
- interface segregation, 226
- interfaces, 202
- interfaces funcionales, 451
- interpreter, 307
- iteraciones, 69
- iteradores, 151
- iterator, 307

- javadoc, 54, 431
- JRE, 10
- JVM, 10

- lambda, 447
- lectura de archivos, 56
- lectura desde el teclado, 21
- Liskov substitution, 224
- long, 13

- main, 18, 103
- matrices, 34
- mediator, 280
- memento, 308
- mensajes, 97
- modelo cascada, 69
- modelo del dominio, 86
- modelo incremental, 69
- mostrar el contenido de un arreglo, 35
- multiplicidad, 87
- método, 99
- método de clase, 98
- método estático, 98
- métodos estáticos, 100
- métodos virtuales, 189

- new, 103
- null, 112
- NullPointerException, 78, 111, 113
- NumberFormatException, 78

- objetos, 83
- observer, 285
- OOP, 83
- open/closed, 221
- operador de post-incremento, 25
- operador de pre-incremento, 25
- operadores en Java, 20
- operadores lógicos, 20
- operadores relacionales, 20
- Ordenamiento, 465

- ordenamiento de arreglos, 39
- ordenamiento multicriterio, 40
- overflow, 15
- parámetro formal, 45
- parámetro real, 45
- paso de parámetros con referencias, 51
- patrón abstract factory, 249
- patrón adapter, 260
- patrón bridge, 276
- patrón builder, 253
- patrón chain of responsibility, 280
- patrón command, 293
- patrón composite, 263
- patrón decorator, 277
- patrón facade, 274
- patrón factory, 247
- patrón flyweight, 271
- patrón interpreter, 307
- patrón iterator, 307
- patrón mediator, 280
- patrón memento, 308
- patrón observer, 285
- patrón prototype, 258
- patrón proxy, 267
- patrón singleton, 246
- patrón state, 299
- patrón strategy, 290
- patrón template, 280
- patrón visitor, 304
- polimorfismo, 191
- POO, 83
- Principio de sustitución de Liskov, 224
- proceso de desarrollo, 11
- programación defensiva, 234, 239
- prototype, 258
- proxy, 267
- punto de entrada, 18
- pythonismos, 20
- rango de tipos, 13
- recolector de basura, 425
- refactoring, 457
- refactorización, 457
- relacionamientos, 89
- Scanner, 21
- scanner.nextLine(), 21
- Segregación por interfaces, 226
- shadowing, 119
- short, 13
- single responsibility, 220
- singleton, 246
- sobre-escritura de métodos, 178
- sobredimensionamiento, 37
- SOLID, 219
- soporte, 70
- Sorting, 465
- stack trace, 77
- state, 299
- static, 100
- strategy, 290
- StringBuilder, 461
- switch, 24
- tamaño de un arreglo, 37
- template, 280
- this, 110, 111
- tipos de datos, 13
- tipos de datos decimales, 14
- tipos de datos enteros, 13
- tipos de datos primitivos, 13
- tipos de excepciones, 453
- tipos de soporte, 70
- tipos primitivos, 13
- toda clase es un tipo, 103
- toString, 213
- try, 75
- try-catch anidados, 79
- type mismatch, 15
- UML, 86
- unchecked exception, 454
- underflow, 15
- usar y después incrementar, 25
- validación de datos, 73
- valor de una expresión, 25
- valor por defecto, 106
- vectores, 32
- virtual, 189
- visibilidad, 96
- visitor, 304
- while, 29
- índice, 32

